AI 應用雲端部署

Enos Chou

© 2026 Enos Chou

1. AI 應用雲端部署 ft. 虛擬機
2. Docker 基礎
3. AI 應用雲端部署 ft. 容器
4. AI 模型部署
5. LLM 部署
6. AI 靜態網頁部署
7. 雲端 API Gateway
8. 相關技術

© 2026 Enos Chou

AI 應用雲端部署 ft. 虛擬機

© 2026 Enos Chou

© 2026 Enos Chou

1. Key
2. VM

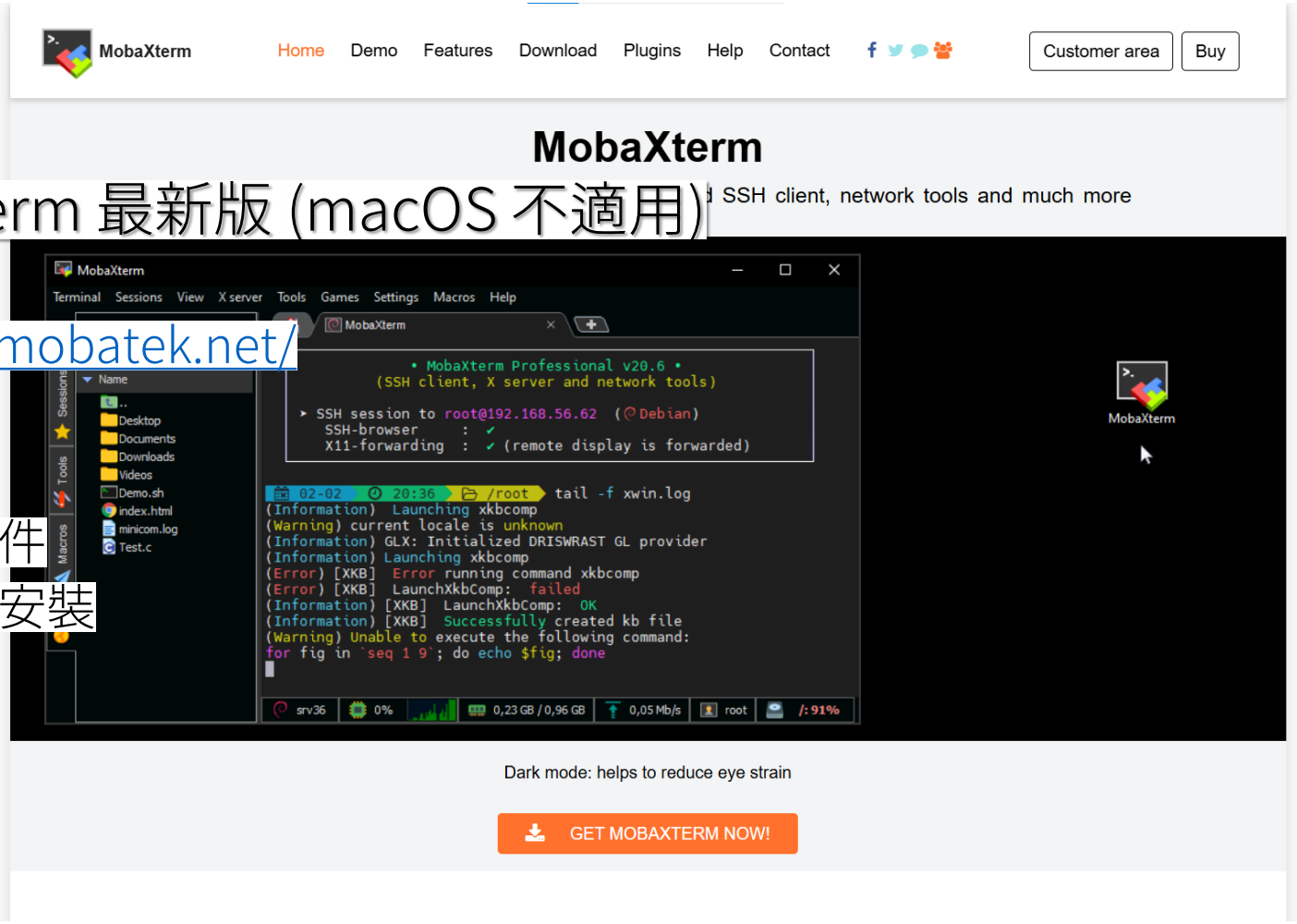
© 2026 Enos Chou

架設 VM

1. 建立登入金鑰

a. 下載並安裝 MobaXterm 最新版 (macOS 不適用)

① https://mobaxterm.mobatek.net/
② Download
③ Download now
④ 下載最新版本安裝套件
⑤ 解壓縮安裝套件完成安裝

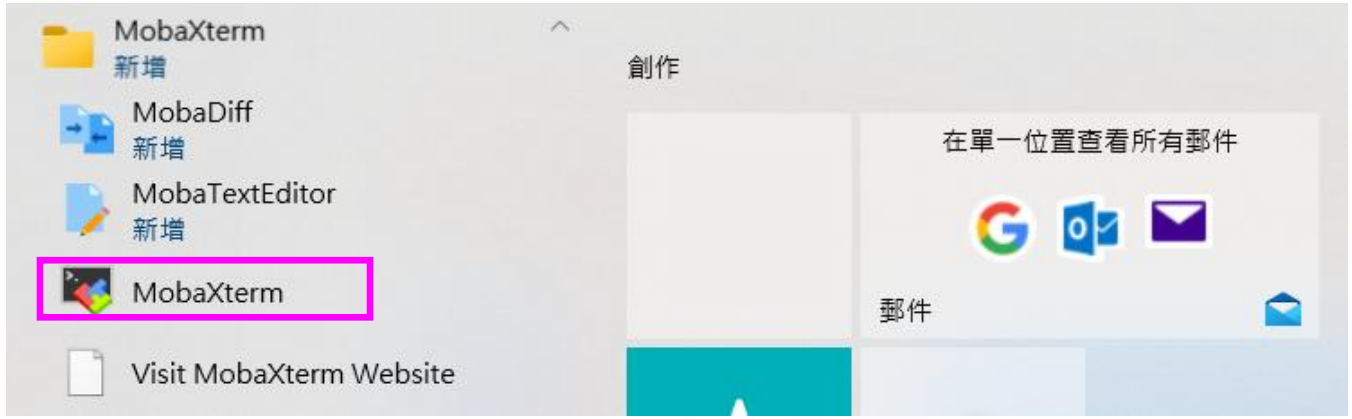


架設 VM

1. 建立登入金鑰

b. 啟動 MobaXterm

啟動 MobaXterm

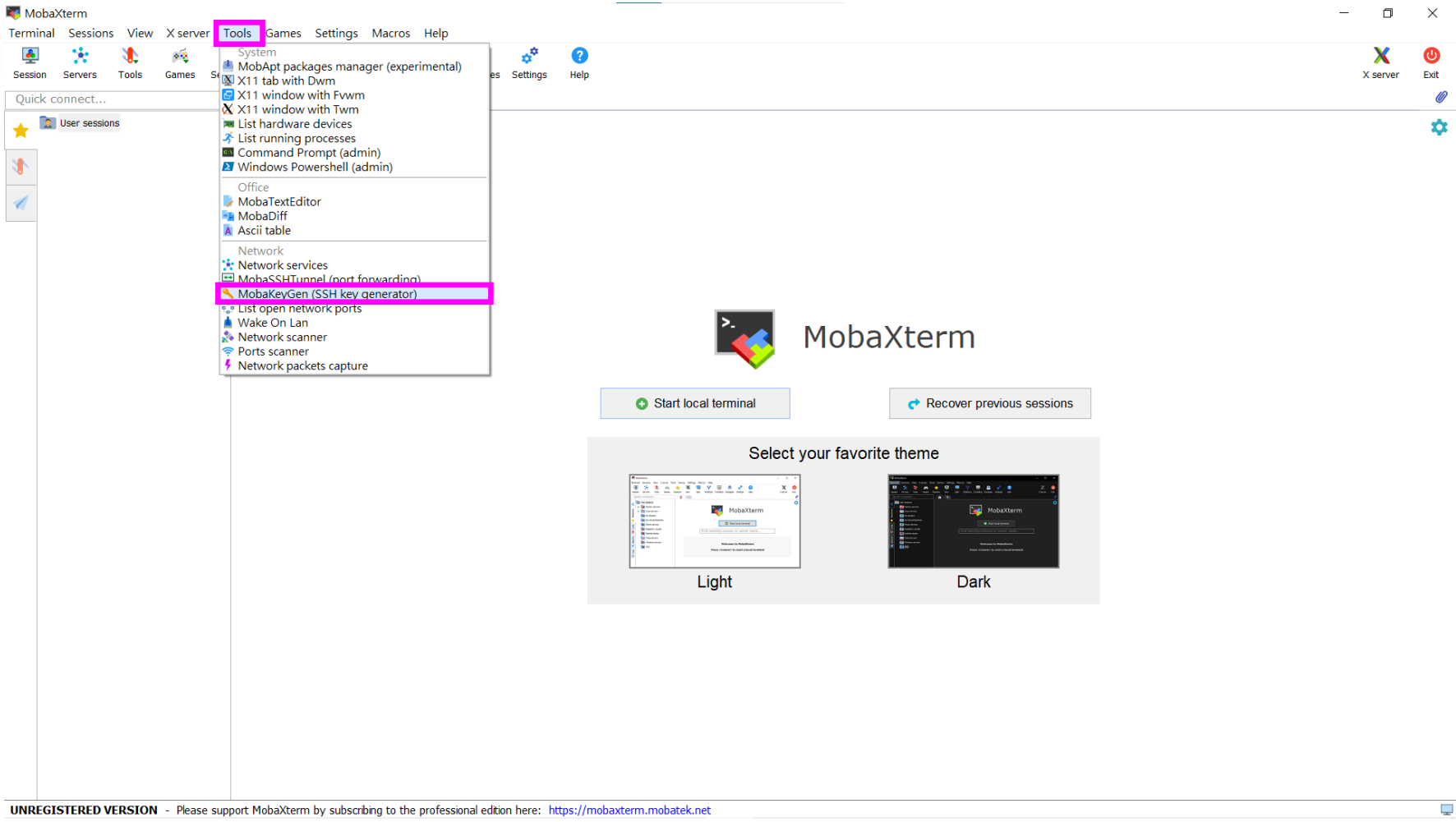


© 2026 Enos Chou

架設 VM

1. 建立登入金鑰

   c. 建立金鑰

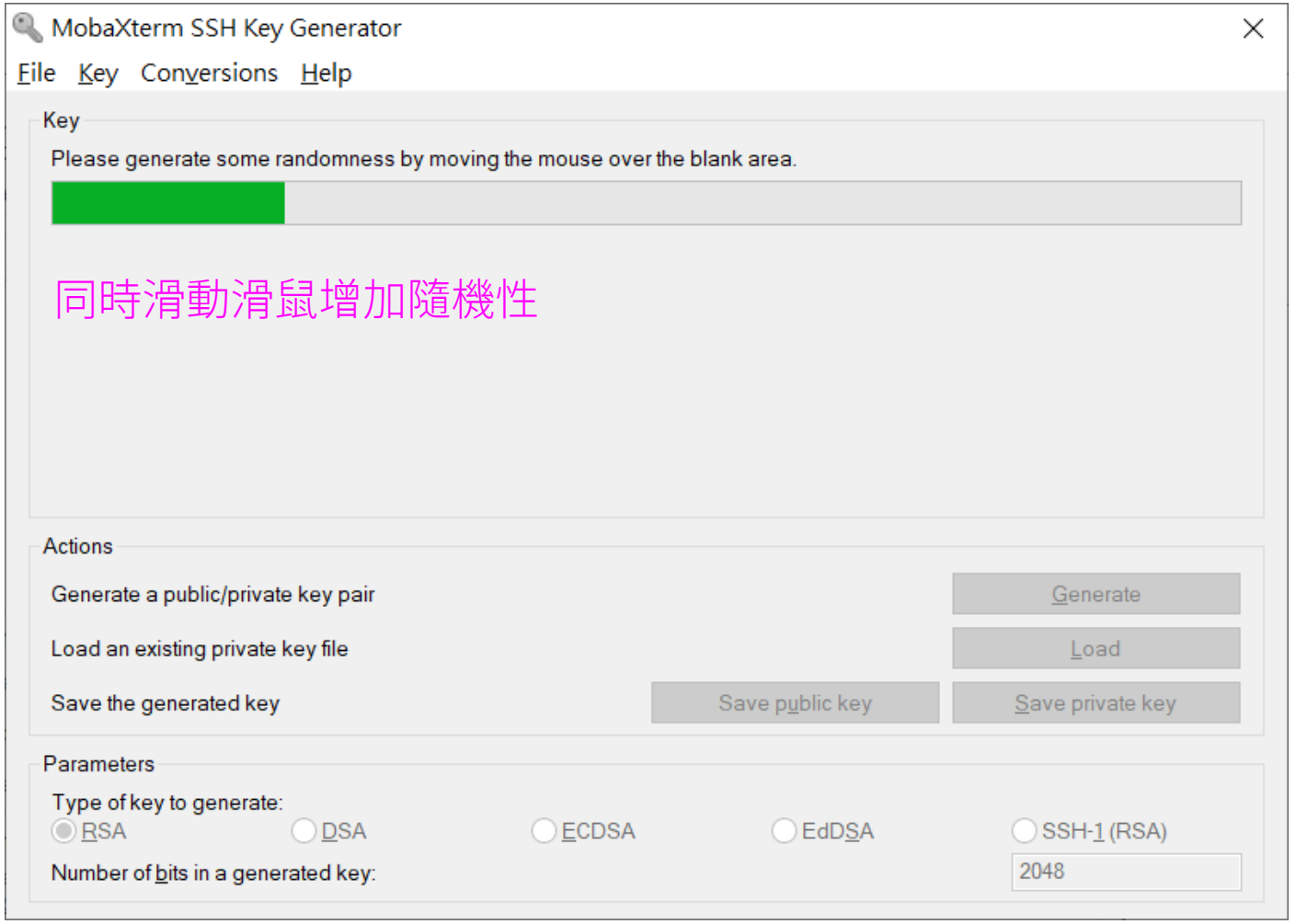


© 2026 Enos Chou

架設 VM

1. 建立登入金鑰

c. 建立金鑰



© 2026 Enos Chou

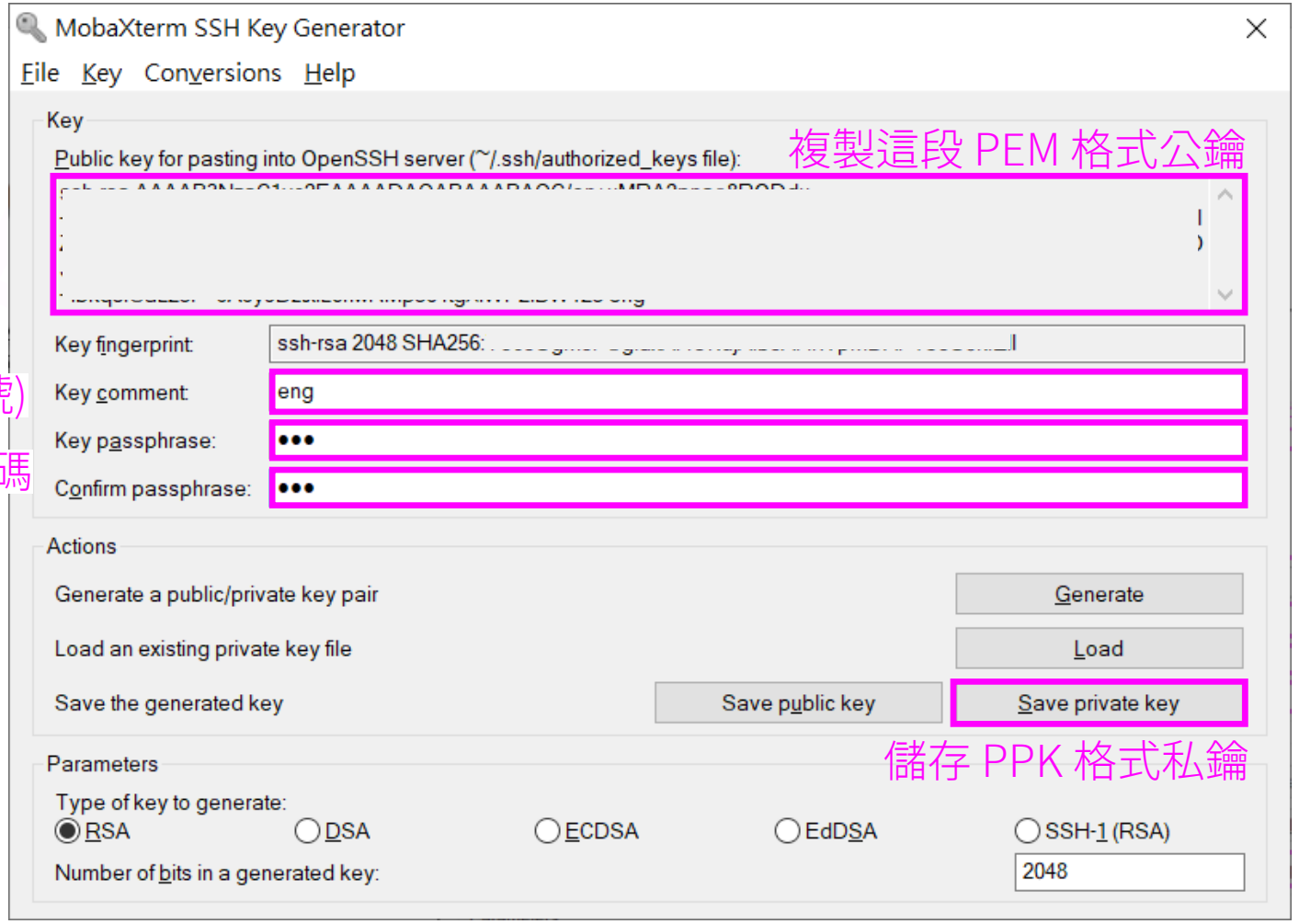
架設 VM

1. 建立登入金鑰

c. 建立金鑰

帳號名稱 (勿使用 root、test 等特殊帳號)

多設一組密碼



金鑰

- 公鑰 (public key) vs 私鑰 (private key)
- PuTTY vs OpenSSH
- PPK (PuTTY Private Key) vs PEM (Private Enhanced Mail)

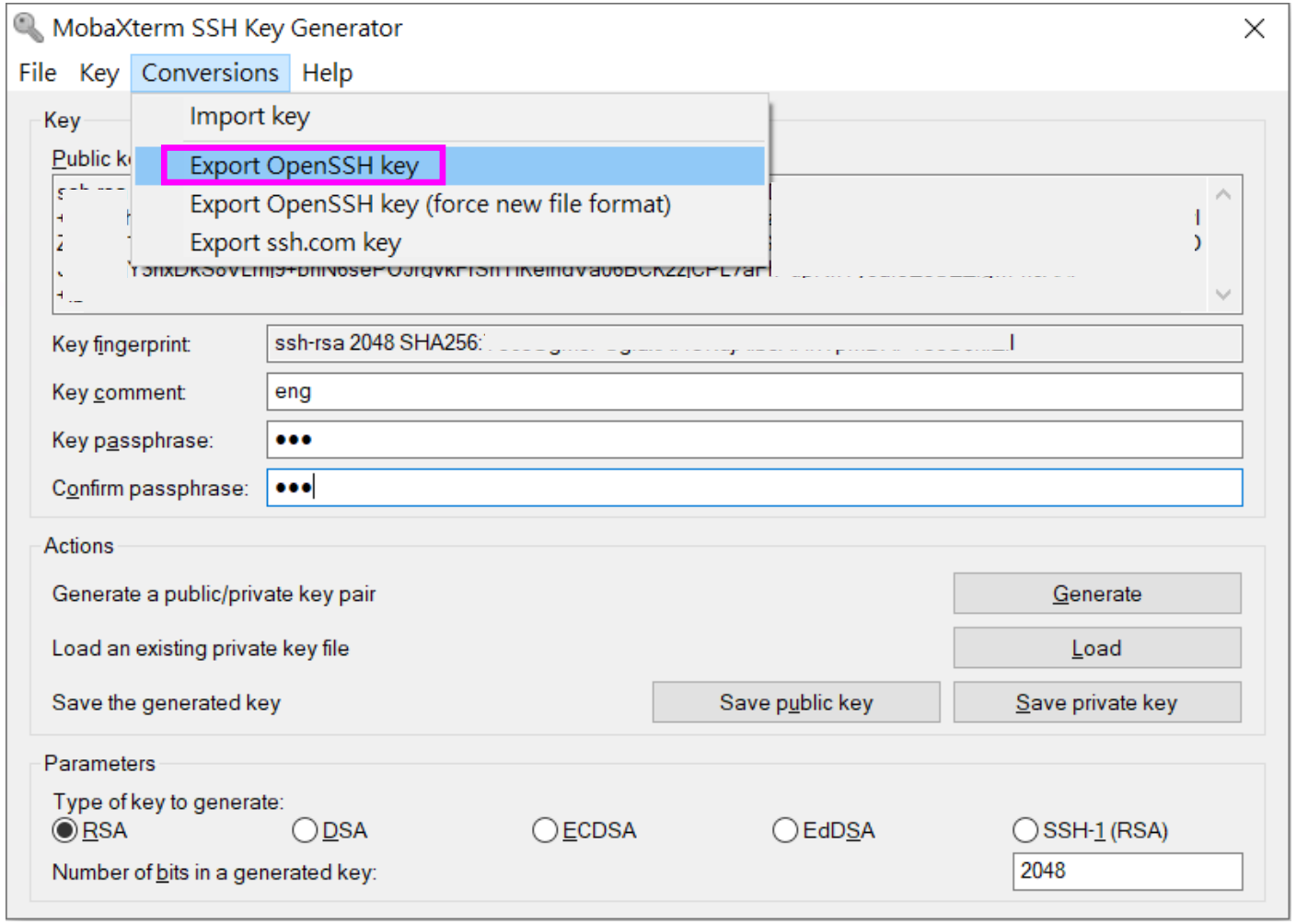
© 2026 Enos Chou

架設 VM

1. 建立登入金鑰

c. 建立金鑰

(匯出 OpenSSH 格式的 Private Key 稍後於 Cloud Shell 使用；建議儲存為 .pem)



架設 VM

1. 建立登入金鑰

c. 建立金鑰 on macOS/ Linux/ Cloud Shell

① 開啟終端機

② `ssh-keygen -f your_keyfile -C your_account`
`ssh-keygen -f eng -C eng  # 可選擇追加密碼或留空`

③ 產出 *your_keyfile* (私鑰) 以及 *your_keyfile*.pub (公鑰)

④ `cat your_keyfile.pub`
`cat eng.pub`

⑤ 複製螢幕輸出之公鑰內容

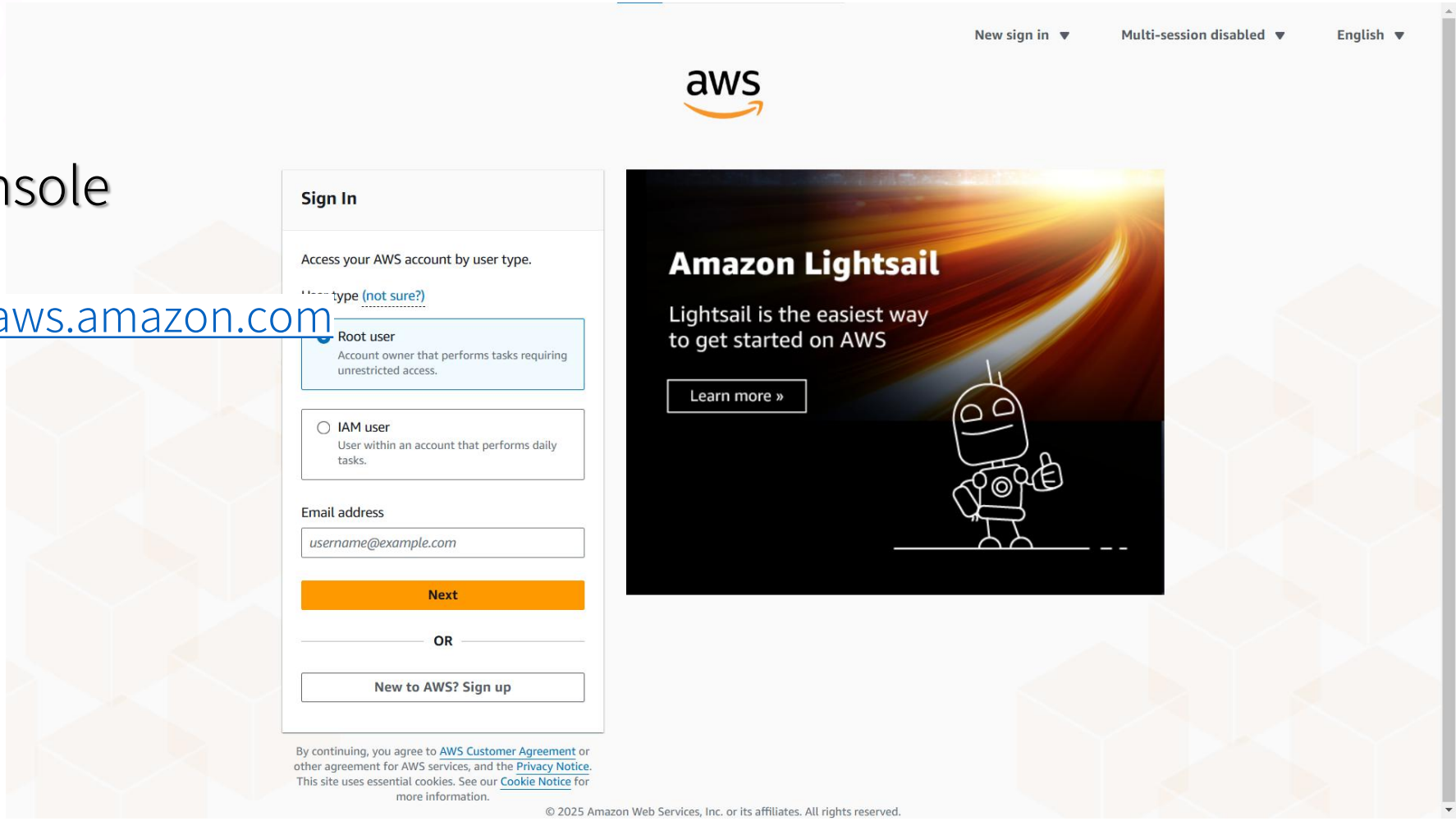
© 2026 Enos Chou

架設 VM - AWS EC2

2. 建立 VM

a. 登入 AWS Console

https://console.aws.amazon.com

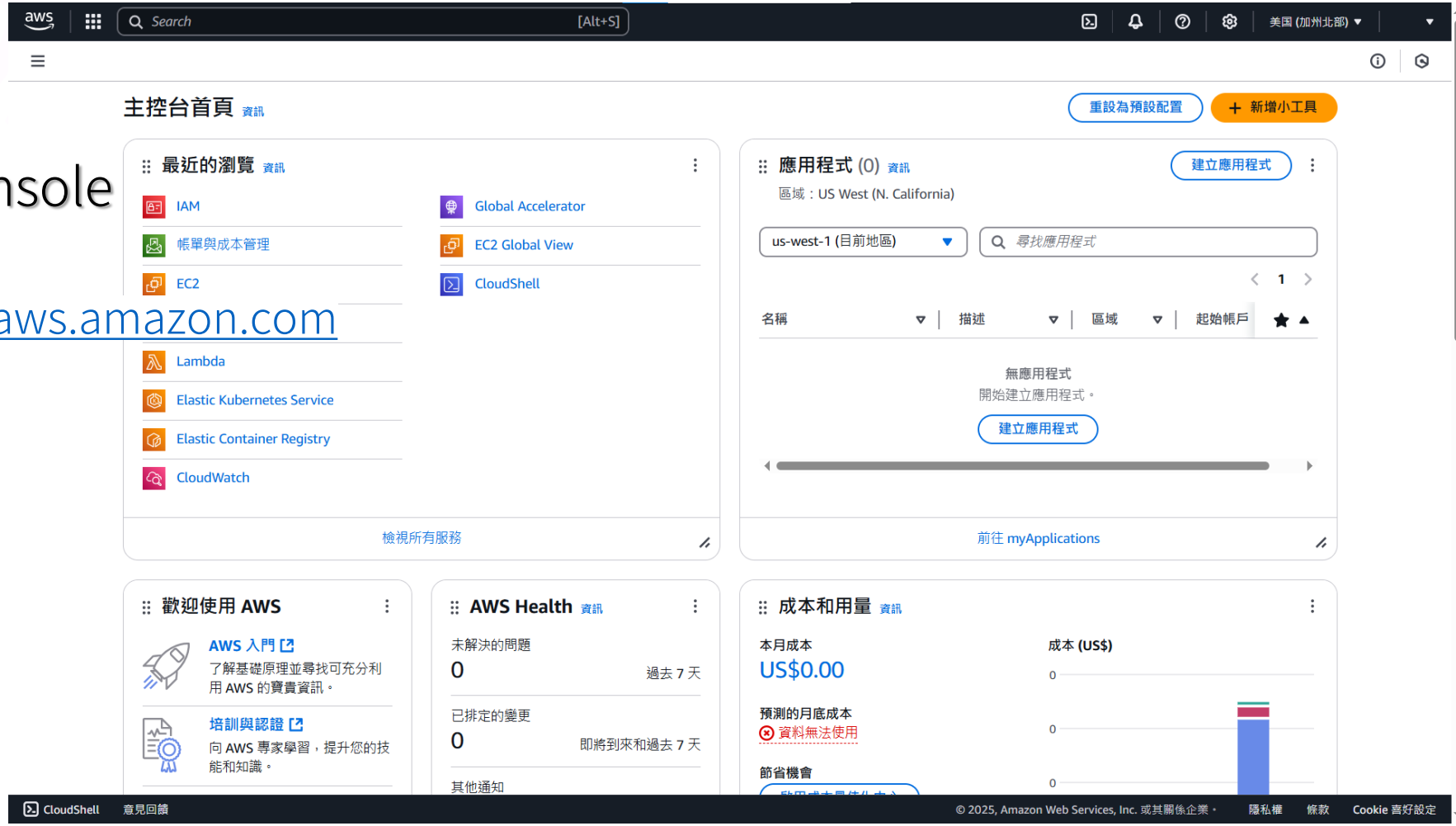


架設 VM - AWS EC2

2. 建立 VM

a. 登入 AWS Console

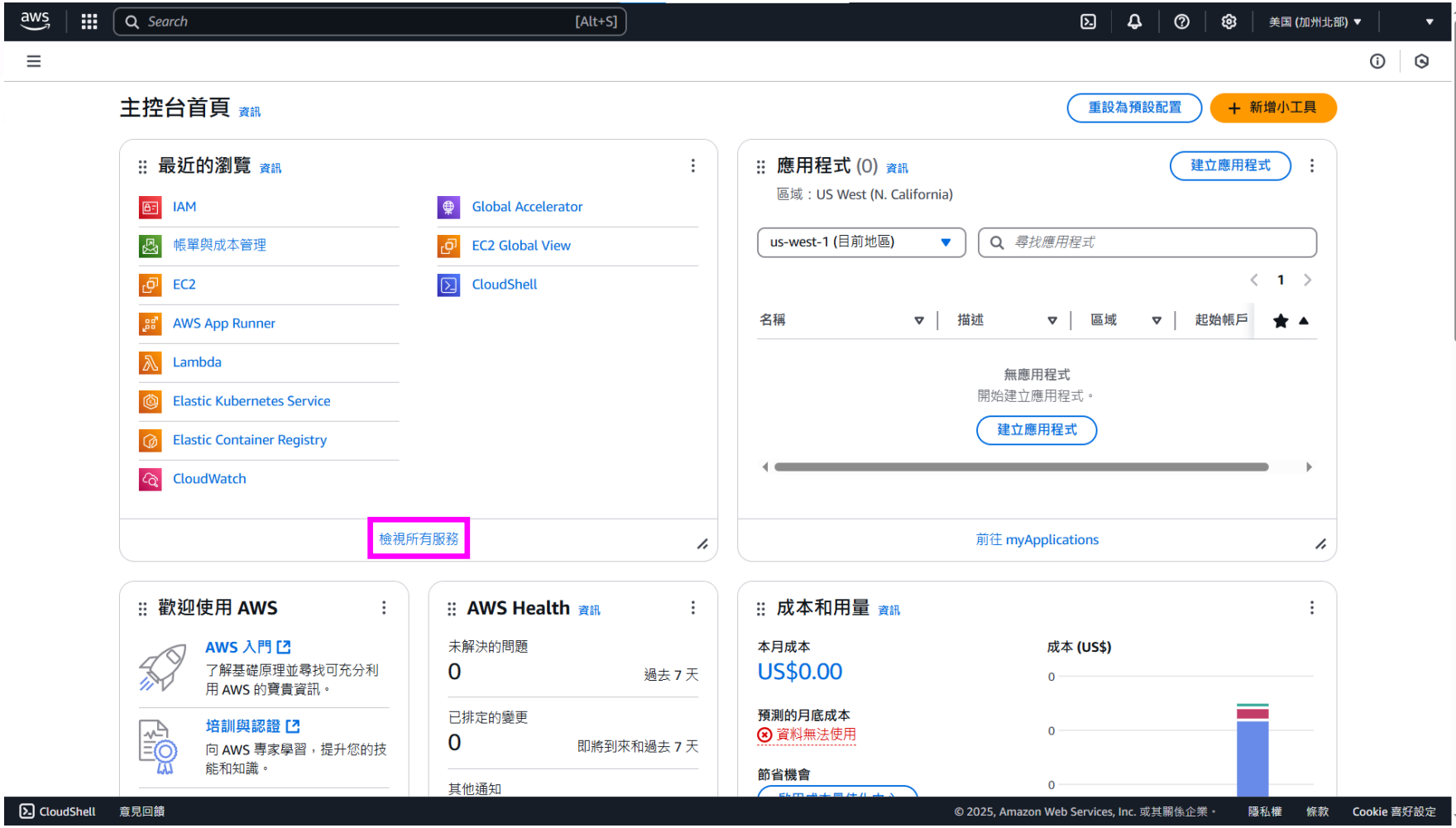
https://console.aws.amazon.com



架設 VM - AWS EC2

2. 建立 VM

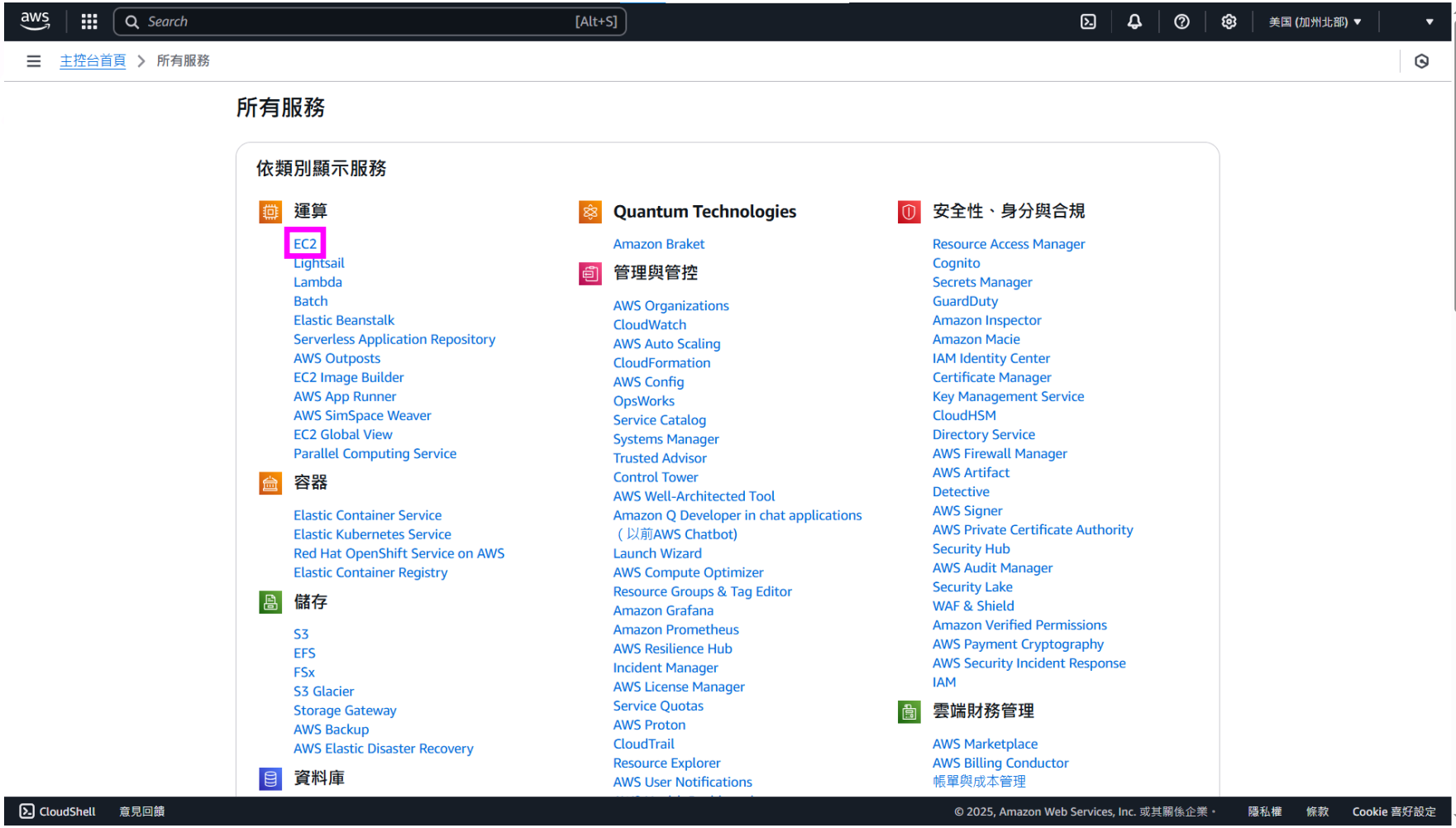
b. 建立虛擬機器



架設 VM - AWS EC2

2. 建立 VM

b. 建立虛擬機器



架設 VM - AWS EC2

2. 建立 VM

b. 建立虛擬機器

架設 VM - AWS EC2

2. 建立 VM

b. 建立虛擬機器

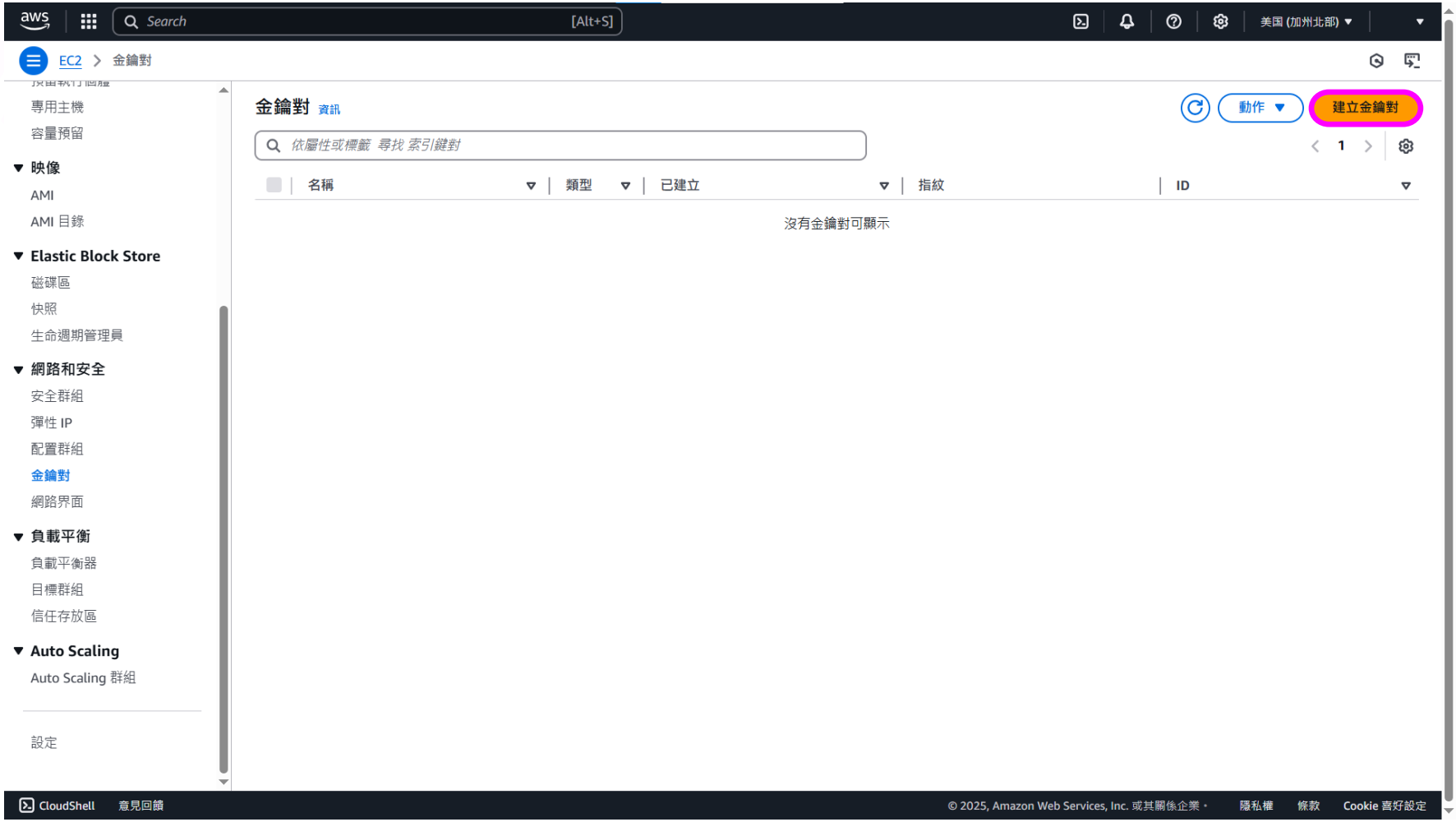
① 匯入金鑰

架設 VM - AWS EC2

2. 建立 VM

b. 建立虛擬機器

① 匯入金鑰

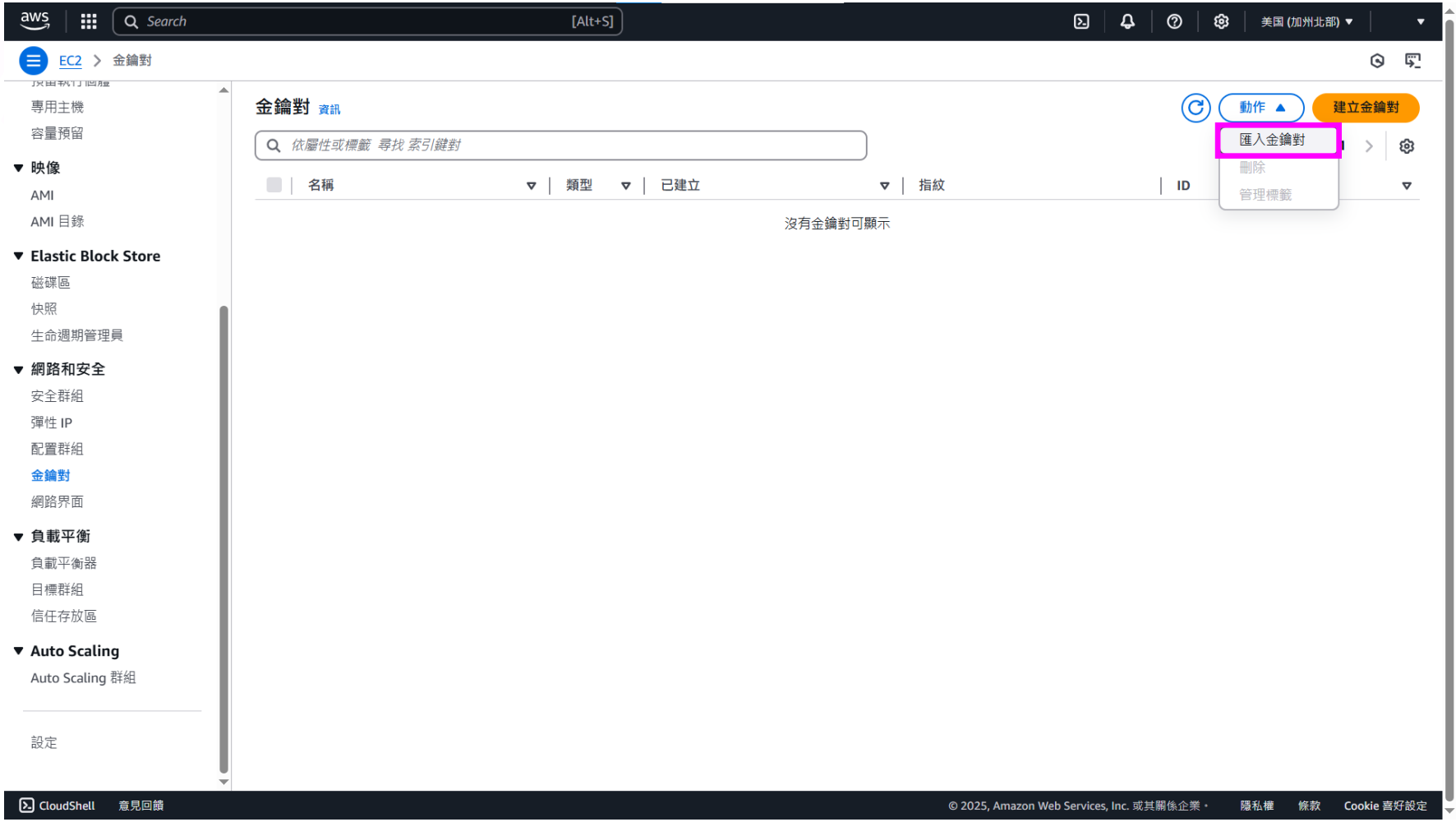


架設 VM - AWS EC2

2. 建立 VM

b. 建立虛擬機器

① 匯入金鑰



架設 VM - AWS EC2

2. 建立 VM

b. 建立虛擬機器

① 匯入金鑰

自行命名

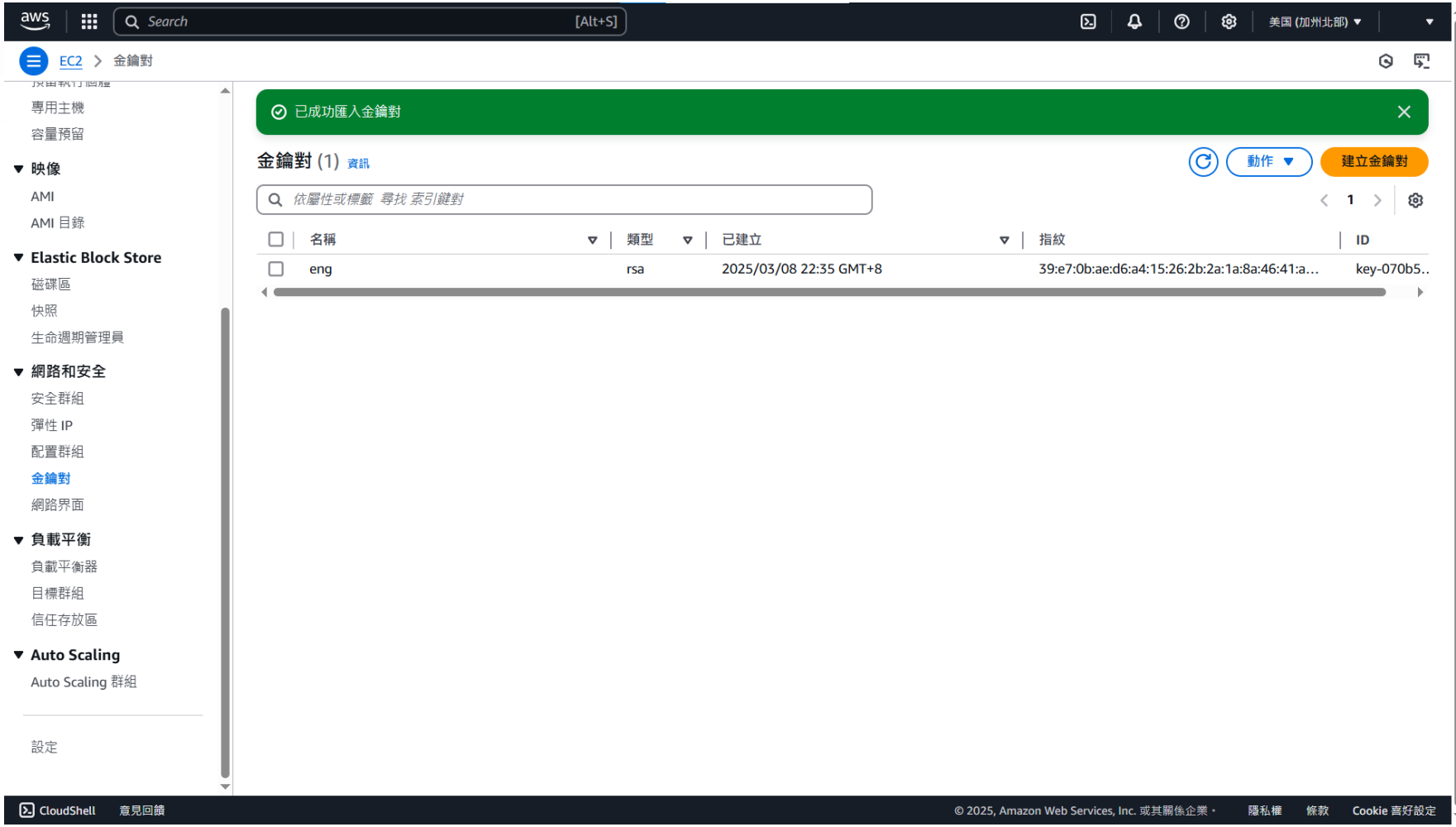
貼上公鑰

架設 VM - AWS EC2

2. 建立 VM

b. 建立虛擬機器

① 匯入金鑰

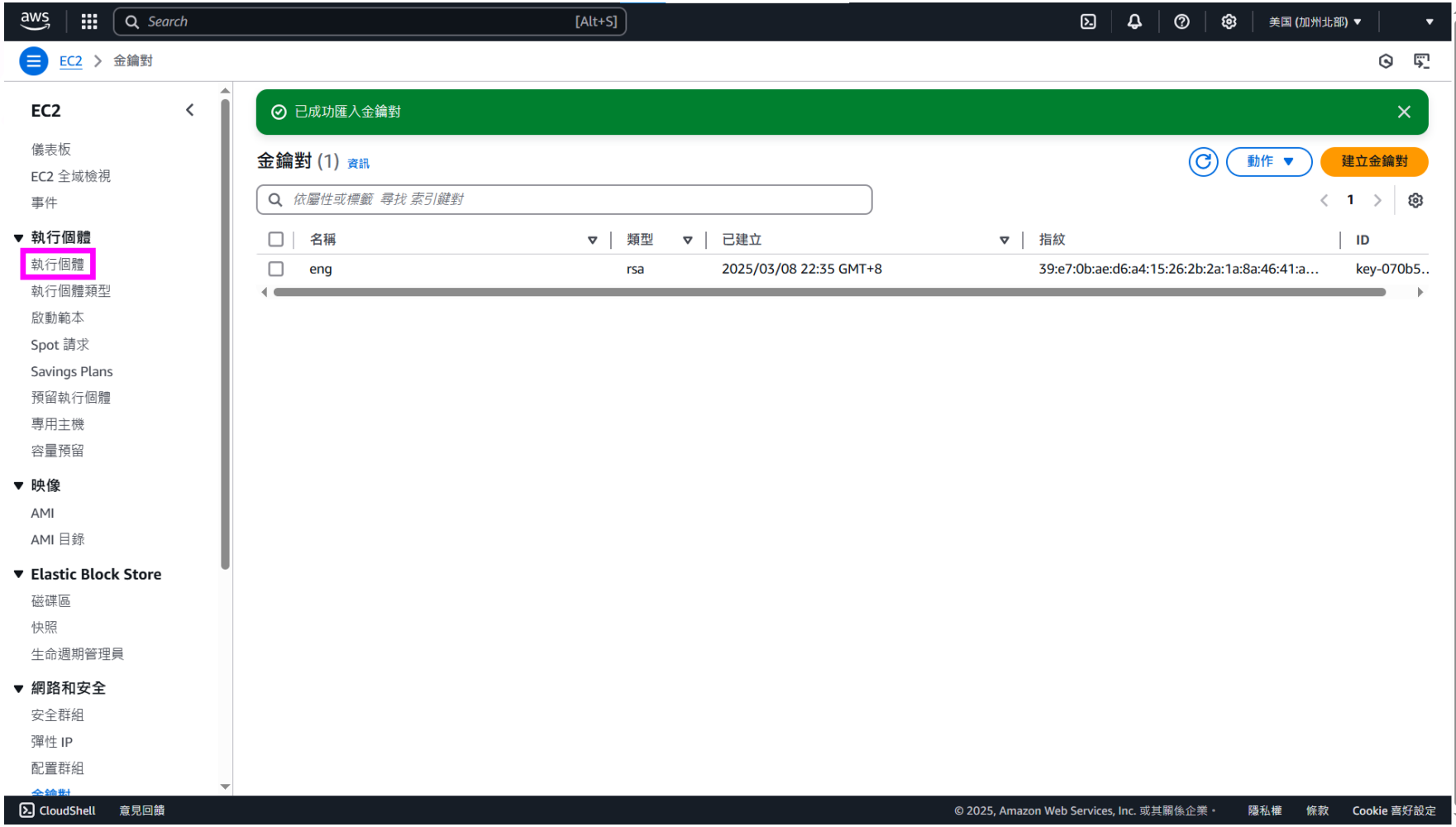


架設 VM - AWS EC2

2. 建立 VM

b. 建立虛擬機器

② 建立 VM

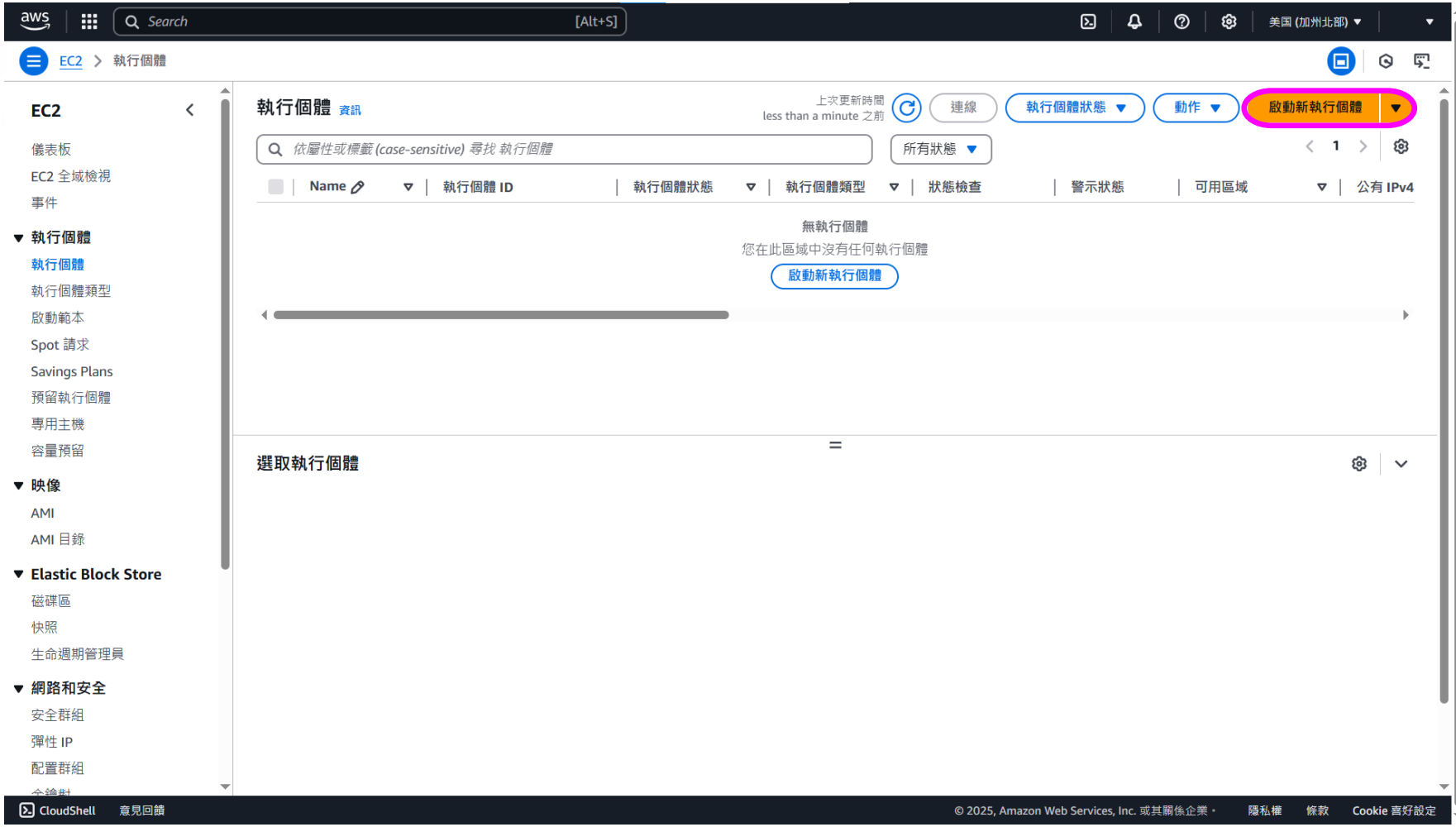


架設 VM - AWS EC2

2. 建立 VM

b. 建立虛擬機器

② 建立 VM

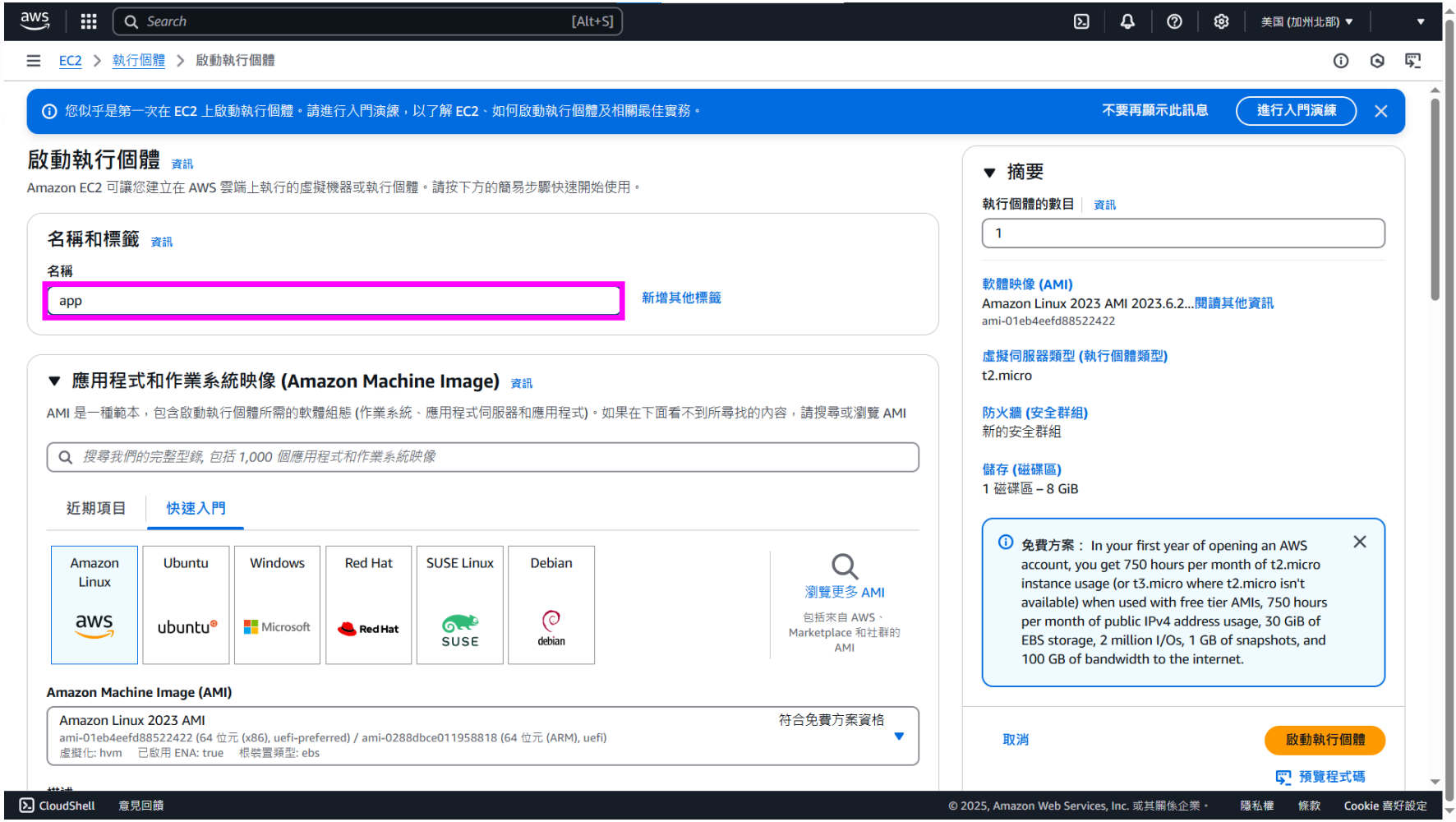


架設 VM - AWS EC2

2. 建立 VM

b. 建立虛擬機器

② 建立 VM



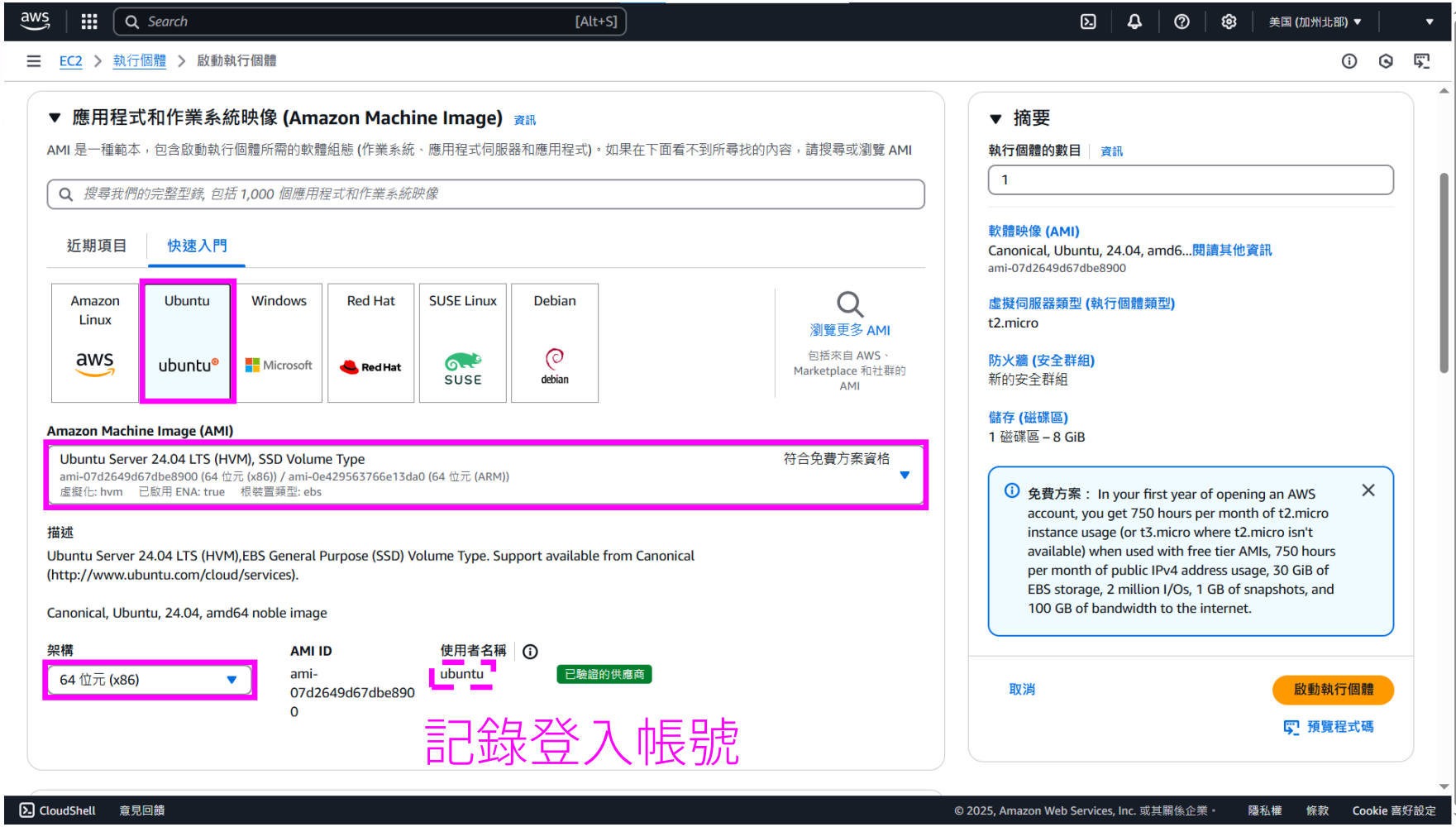
架設 VM - AWS EC2

2. 建立 VM

b. 建立虛擬機器

② 建立 VM

作業系統選擇
Ubuntu 24.04 LTS

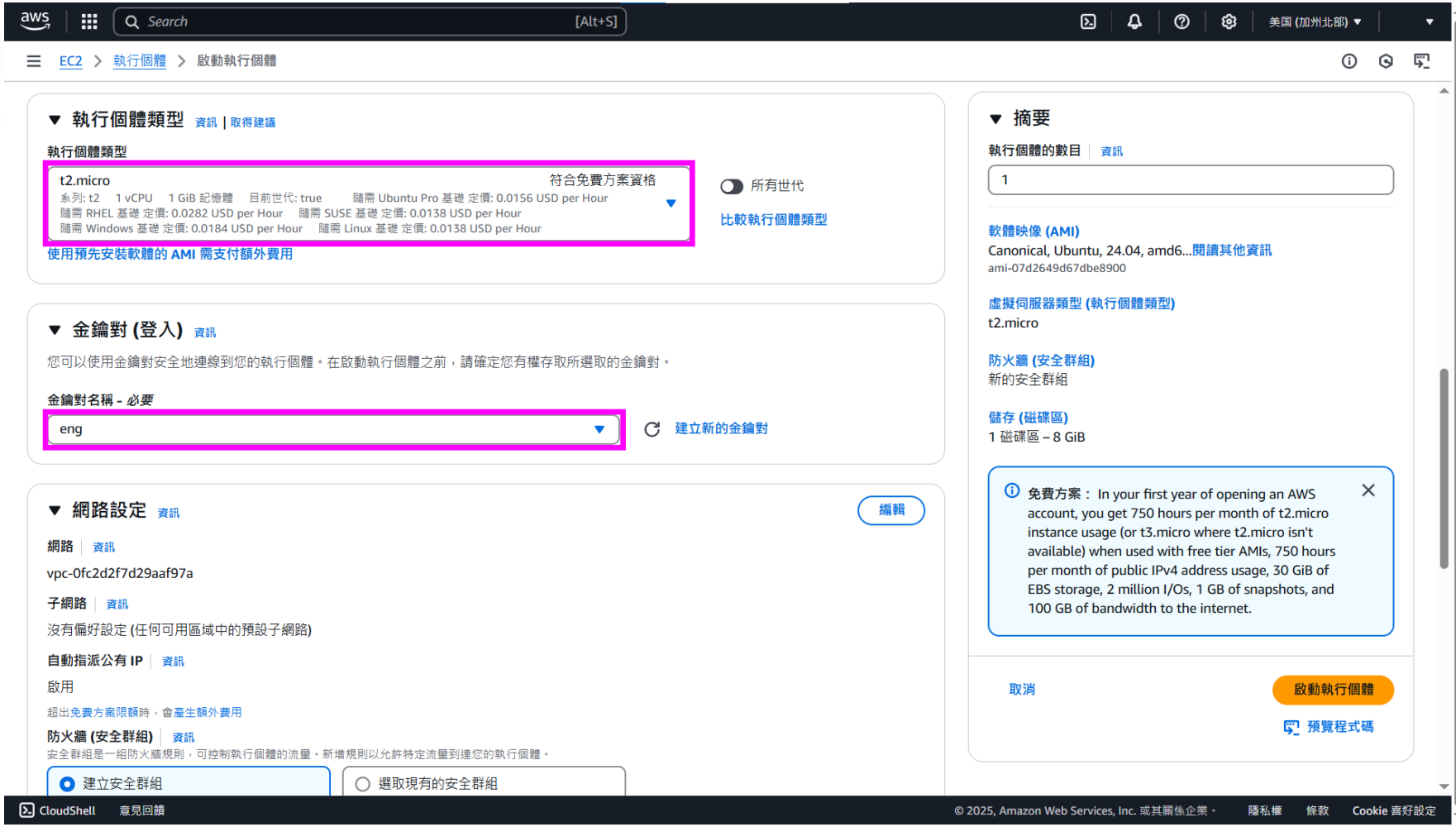


架設 VM - AWS EC2

2. 建立 VM

b. 建立虛擬機器

② 建立 VM

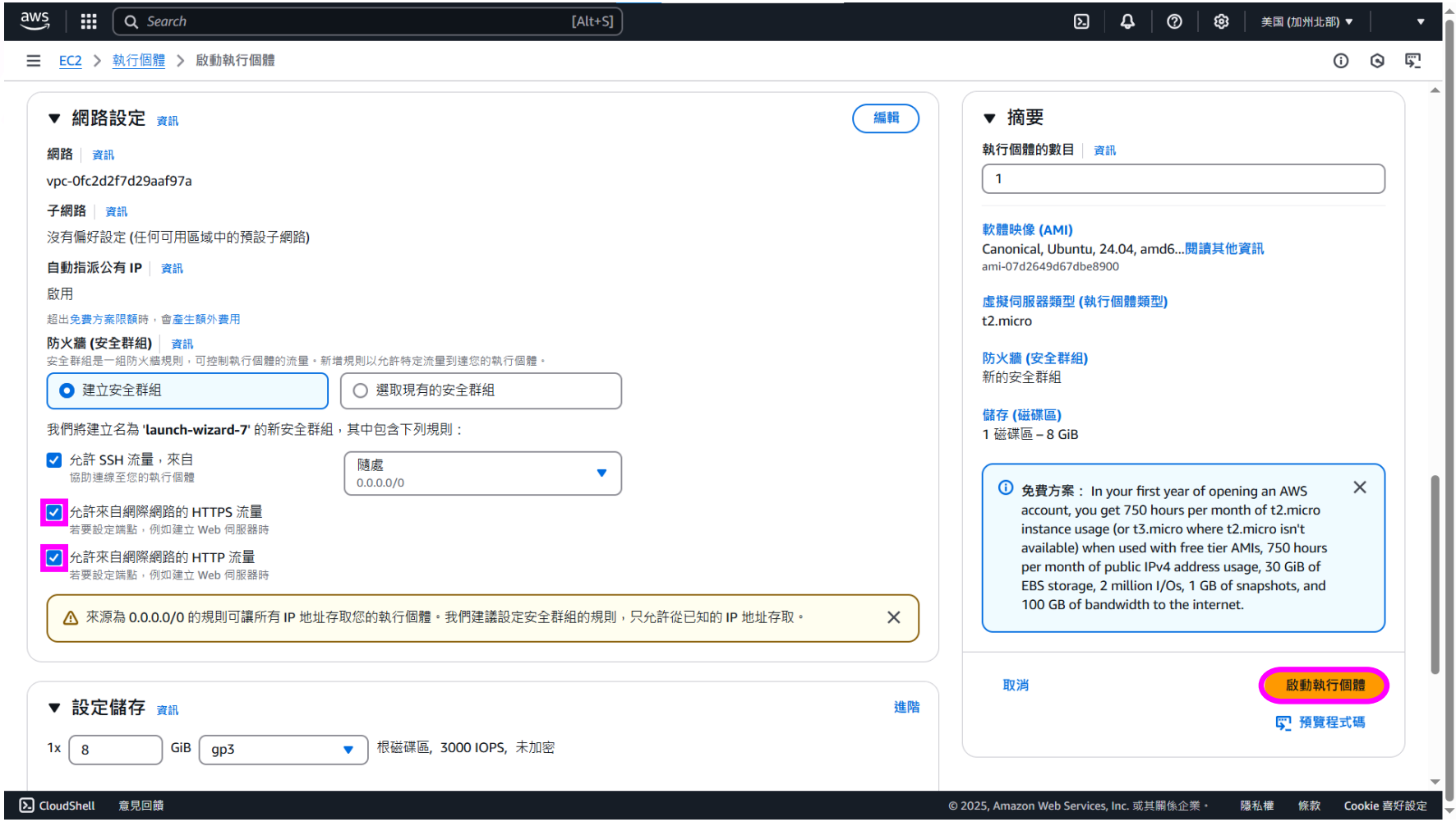


架設 VM - AWS EC2

2. 建立 VM

b. 建立虛擬機器

② 建立 VM

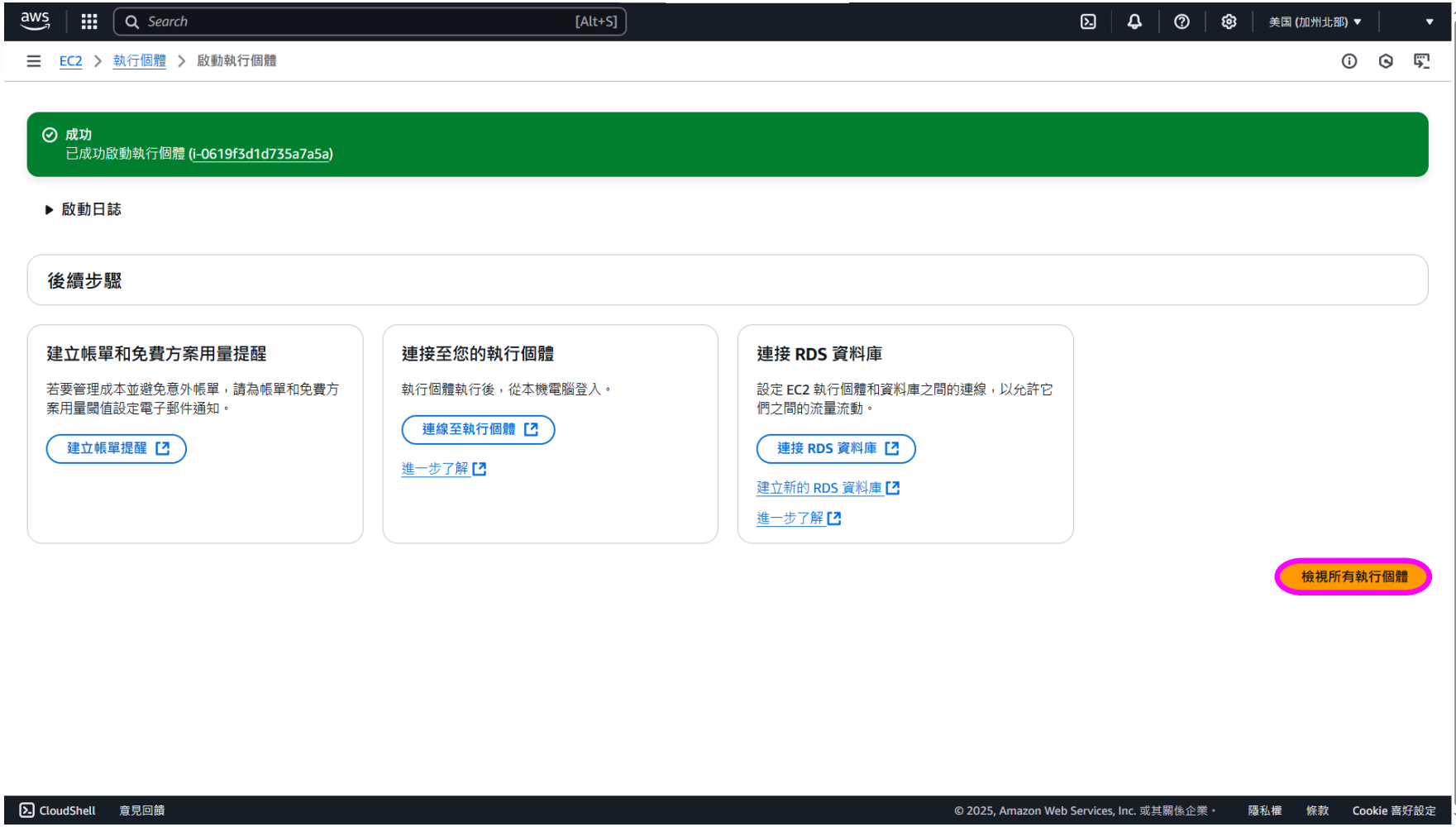


架設 VM - AWS EC2

2. 建立 VM

b. 建立虛擬機器

② 建立 VM

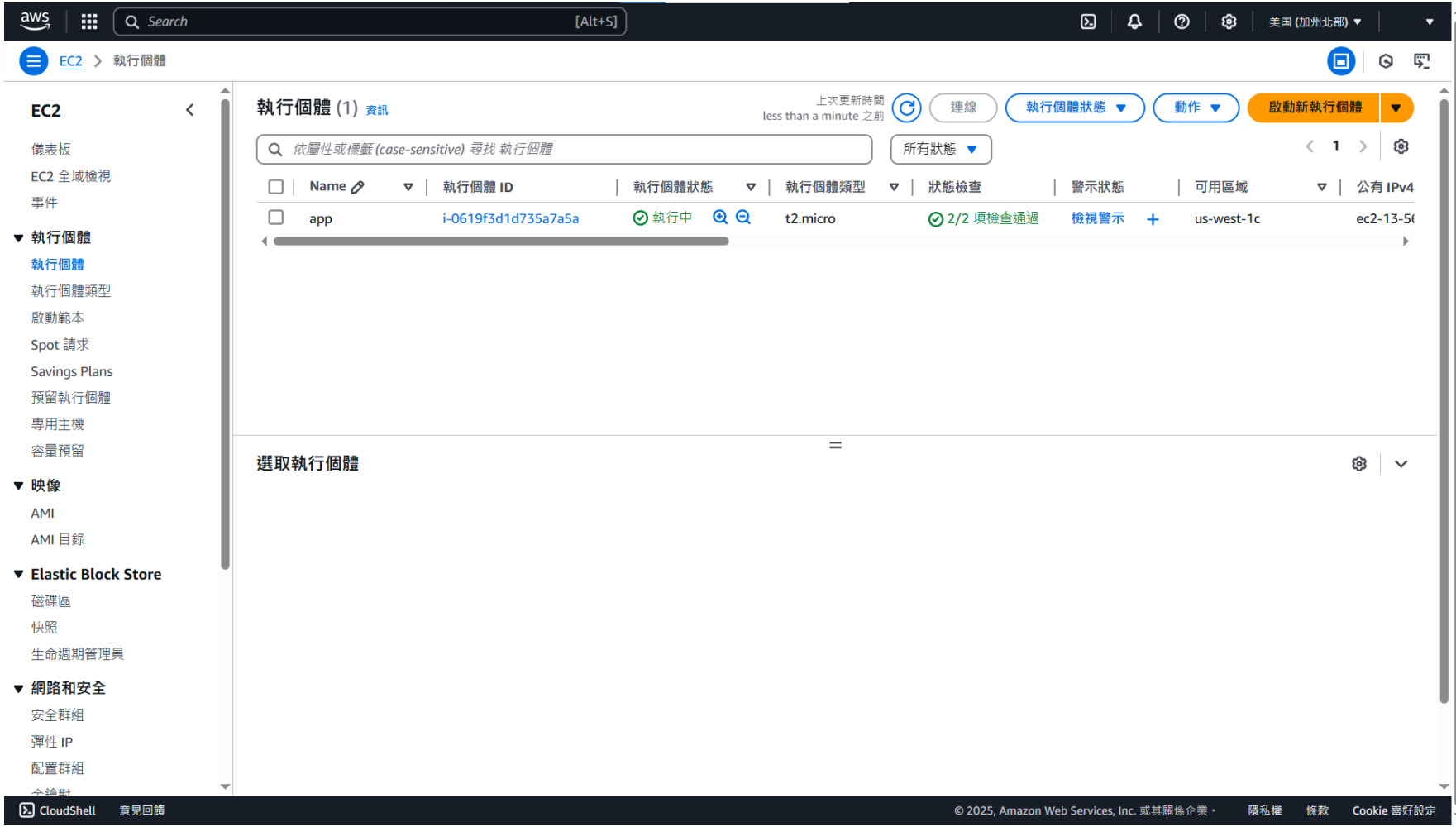


架設 VM - AWS EC2

2. 建立 VM

b. 建立虛擬機器

② 建立 VM

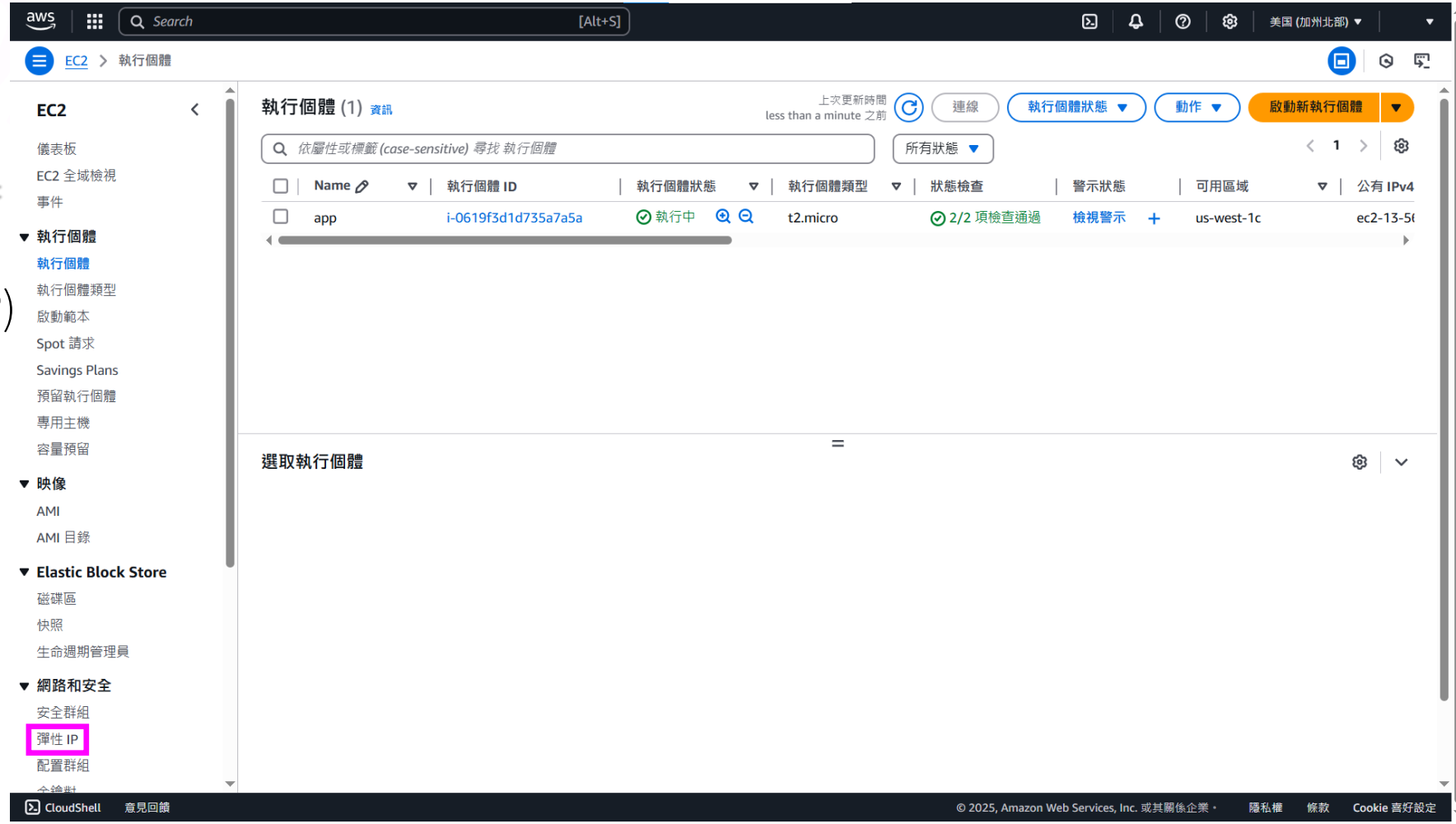


架設 VM - AWS EC2

2. 建立 VM

b. 建立虛擬機器

③ (設定固定 IP)

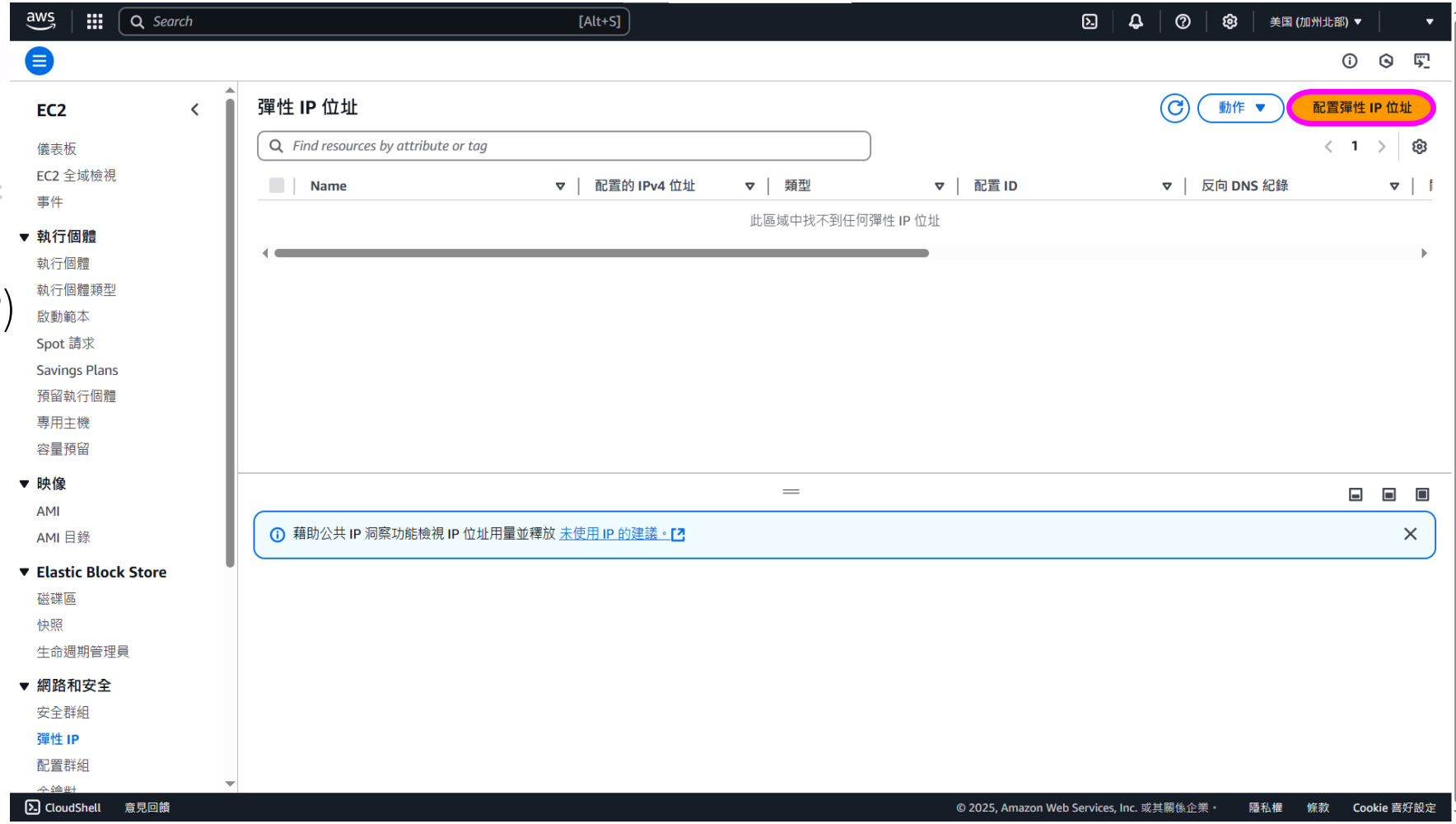


架設 VM - AWS EC2

2. 建立 VM

b. 建立虛擬機器

③ (設定固定 IP)



架設 VM - AWS EC2

2. 建立 VM

b. 建立虛擬機器

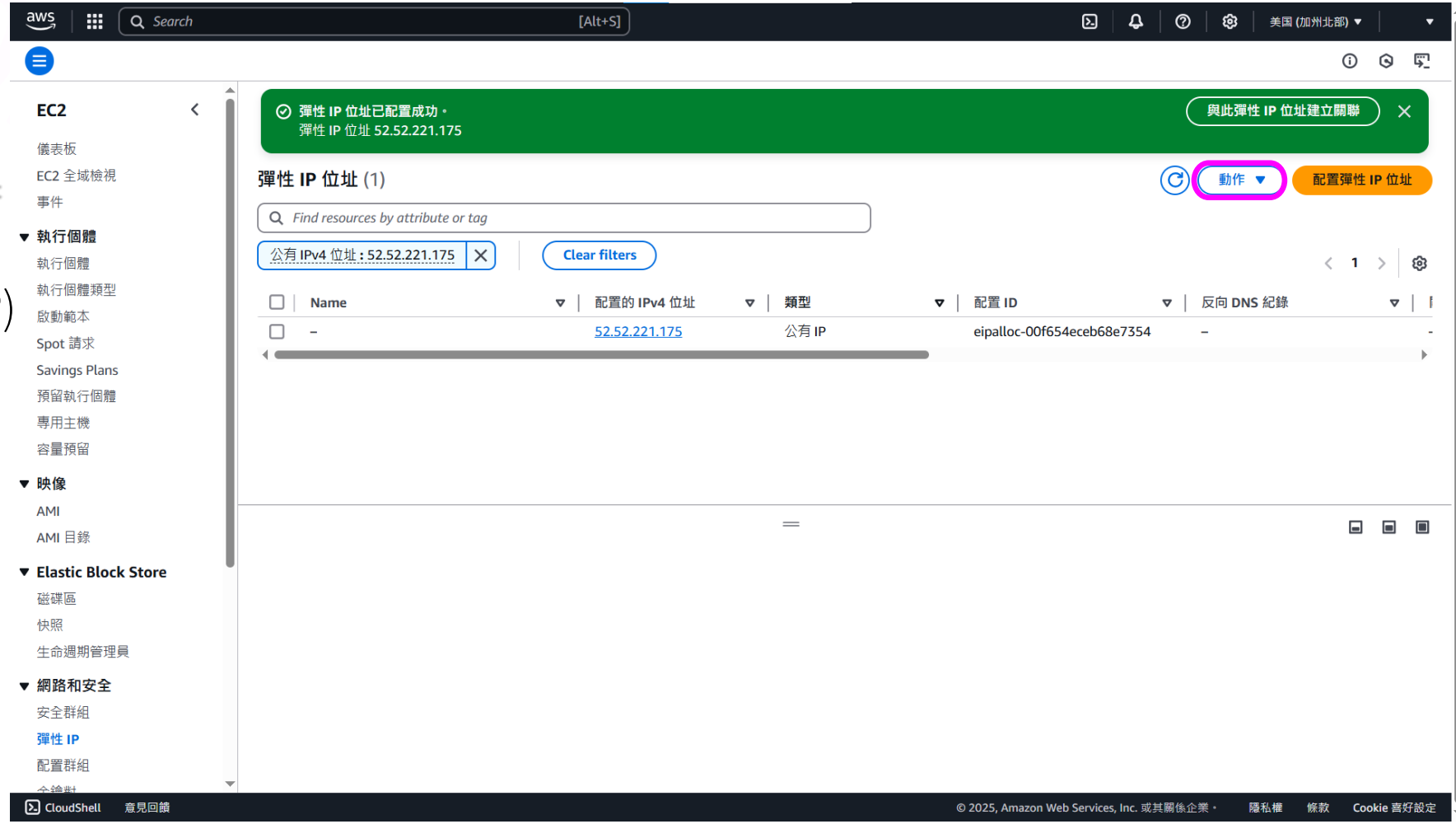
③ (設定固定 IP)

架設 VM - AWS EC2

2. 建立 VM

b. 建立虛擬機器

③ (設定固定 IP)

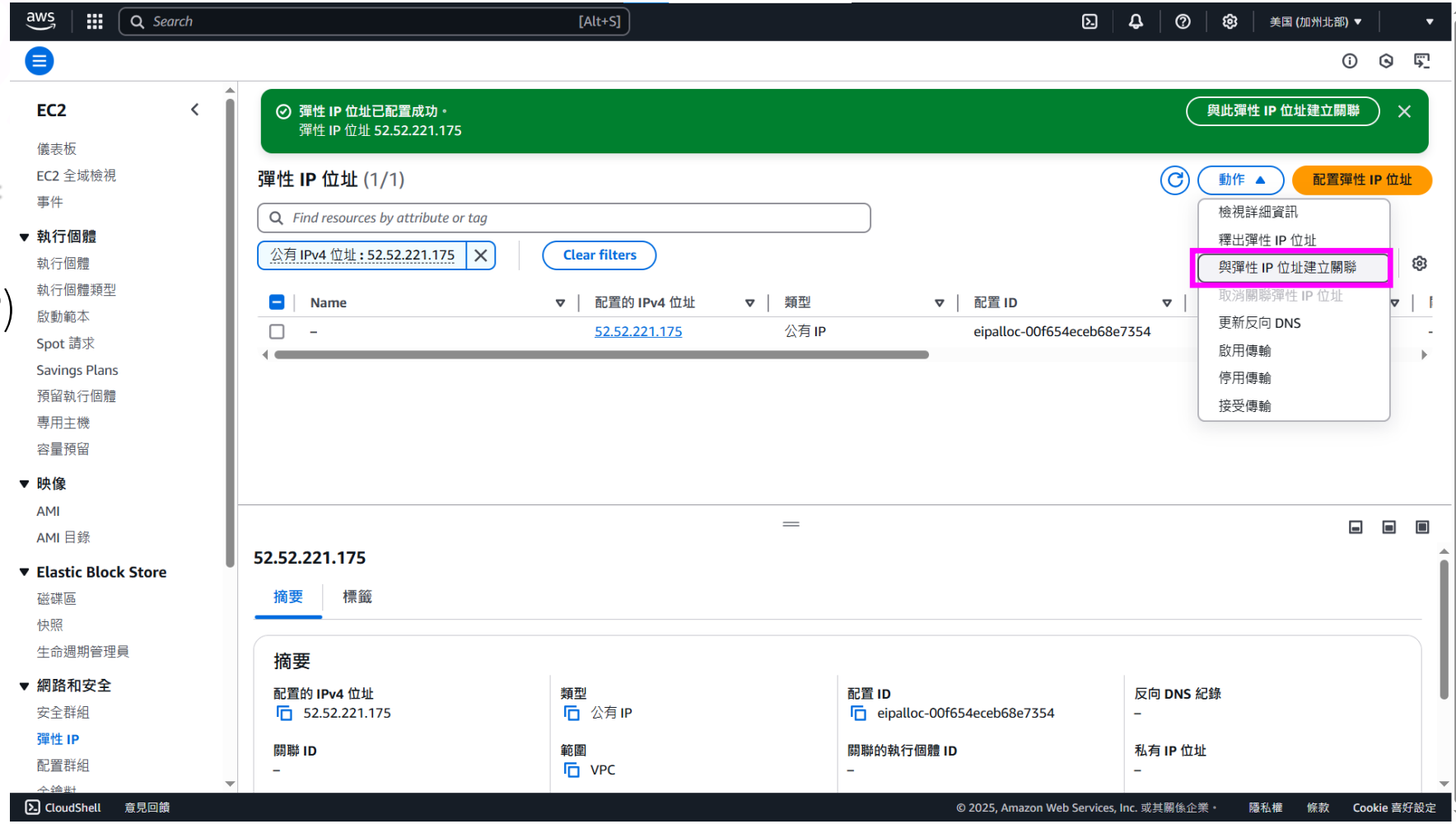


架設 VM - AWS EC2

2. 建立 VM

b. 建立虛擬機器

③ (設定固定 IP)

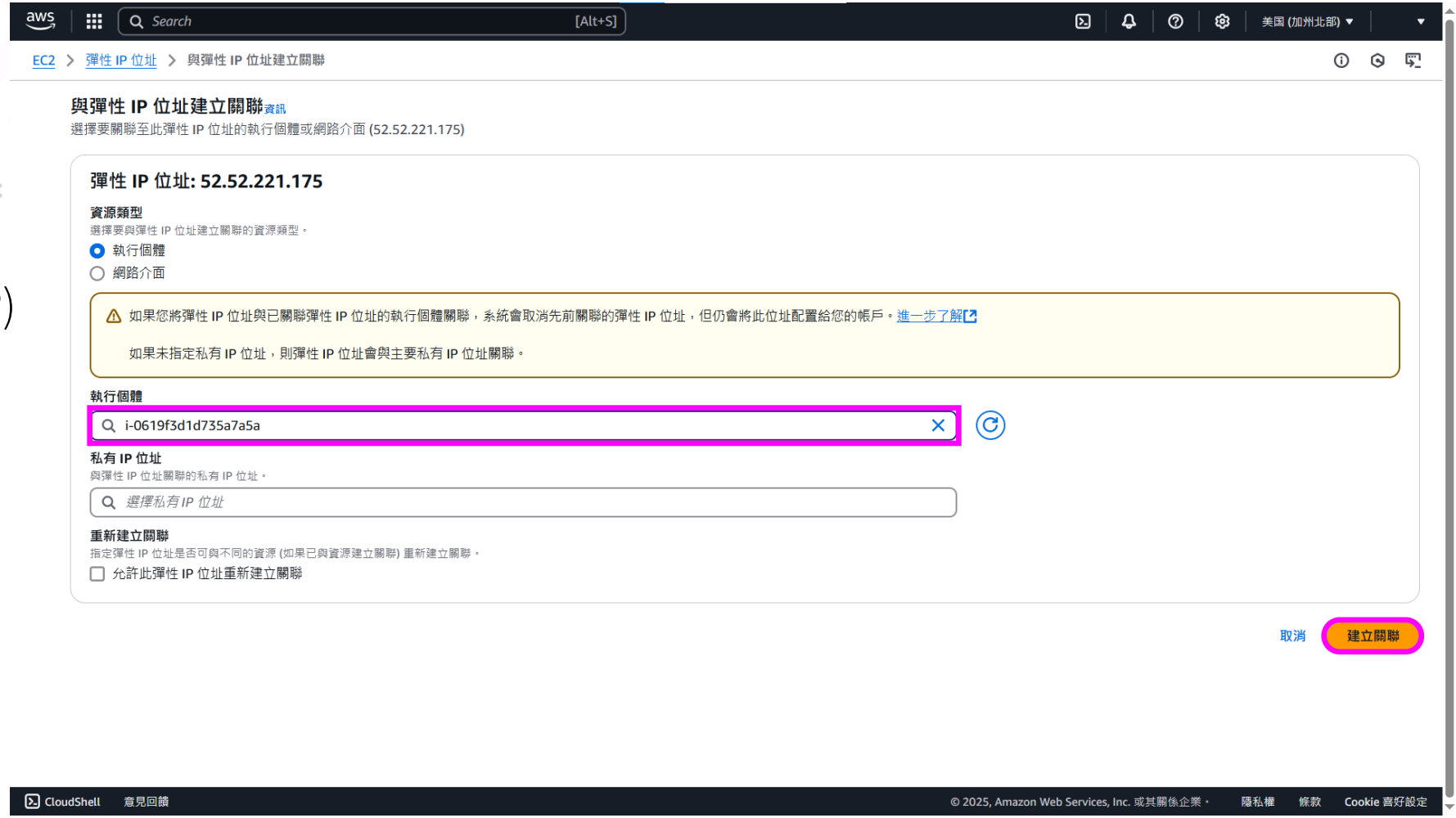


架設 VM - AWS EC2

2. 建立 VM

b. 建立虛擬機器

③ (設定固定 IP)

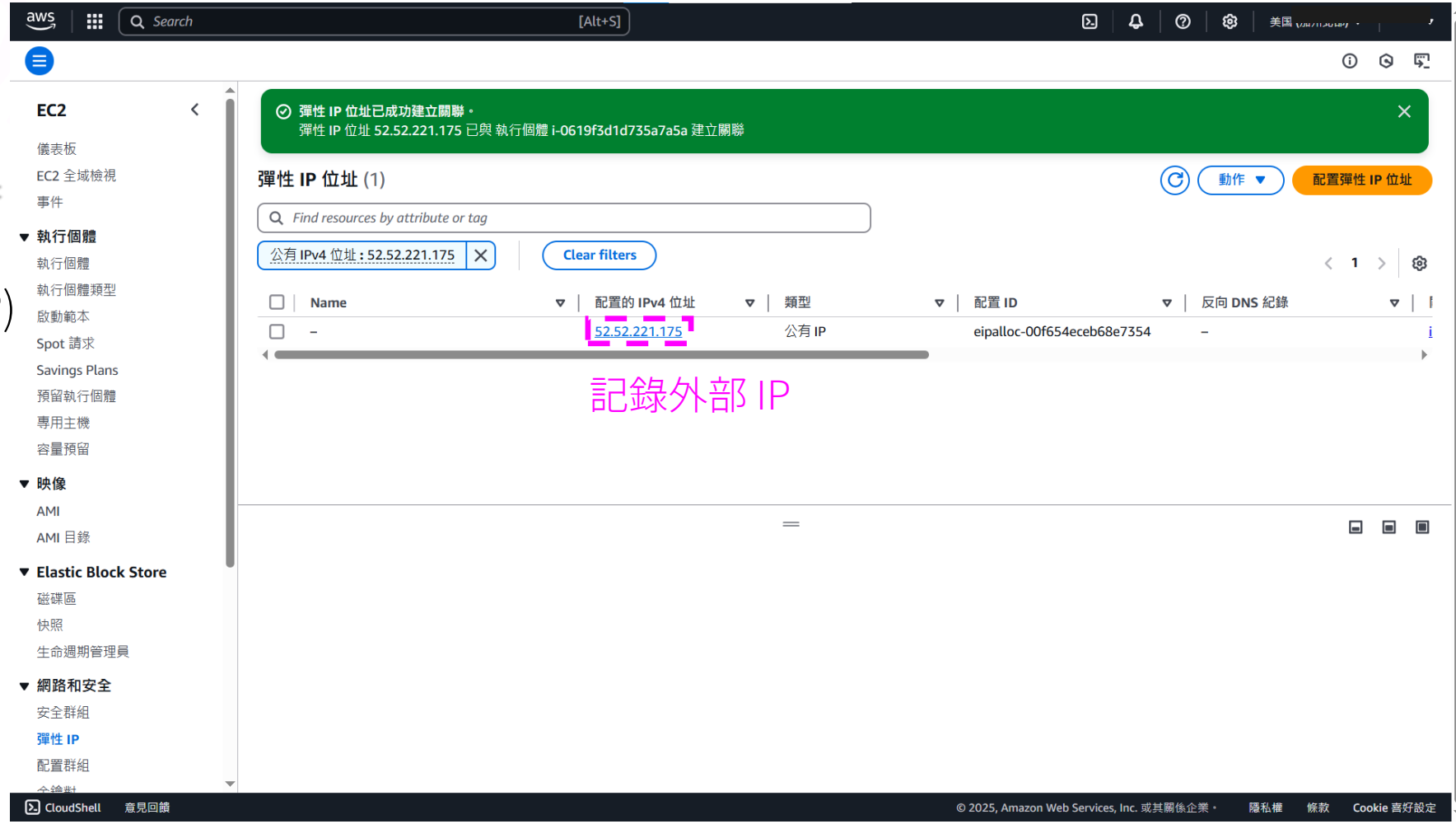


架設 VM - AWS EC2

2. 建立 VM

b. 建立虛擬機器

③ (設定固定 IP)



架設 VM - Azure Virtual Machines

2. 建立 VM

a. 登入 Azure Portal

https://portal.azure.com

架設 VM - Azure Virtual Machines

2. 建立 VM

b. 建立虛擬機器

架設 VM - Azure Virtual Machines

2. 建立 VM

b. 建立虛擬機器

設定資源群組

設定主機名稱
選擇區域

選擇作業系統 (選擇 Ubuntu 24.04 LTS)

架設 VM - Azure Virtual Machines

2. 建立 VM

b. 建立虛擬機器

自訂帳號
使用現有的公開金鑰
貼上 PEM 格式公鑰

架設 VM - Azure Virtual Machines

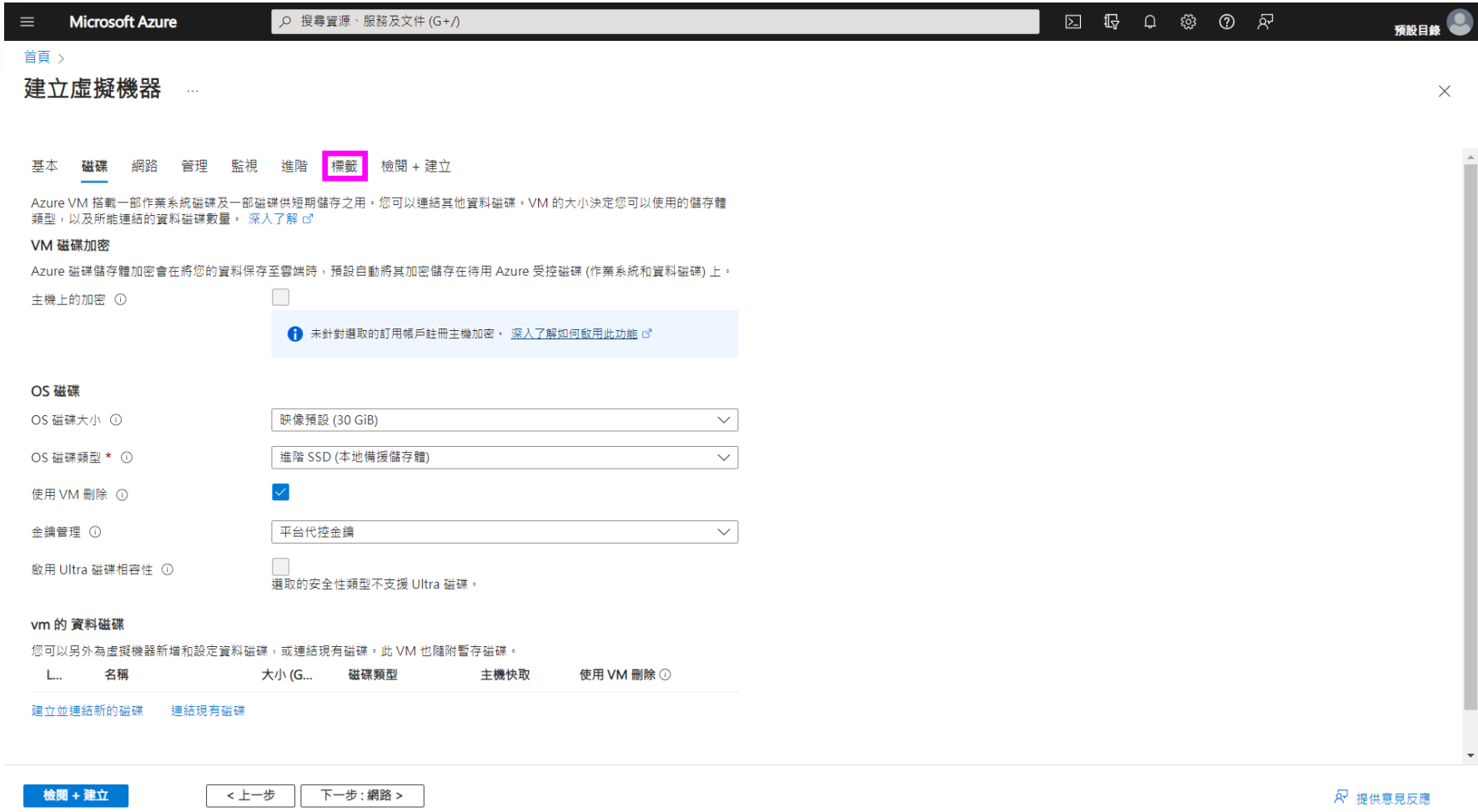
2. 建立 VM

b. 建立虛擬機器

架設 VM - Azure Virtual Machines

2. 建立 VM

b. 建立虛擬機器



架設 VM - Azure Virtual Machines

2. 建立 VM

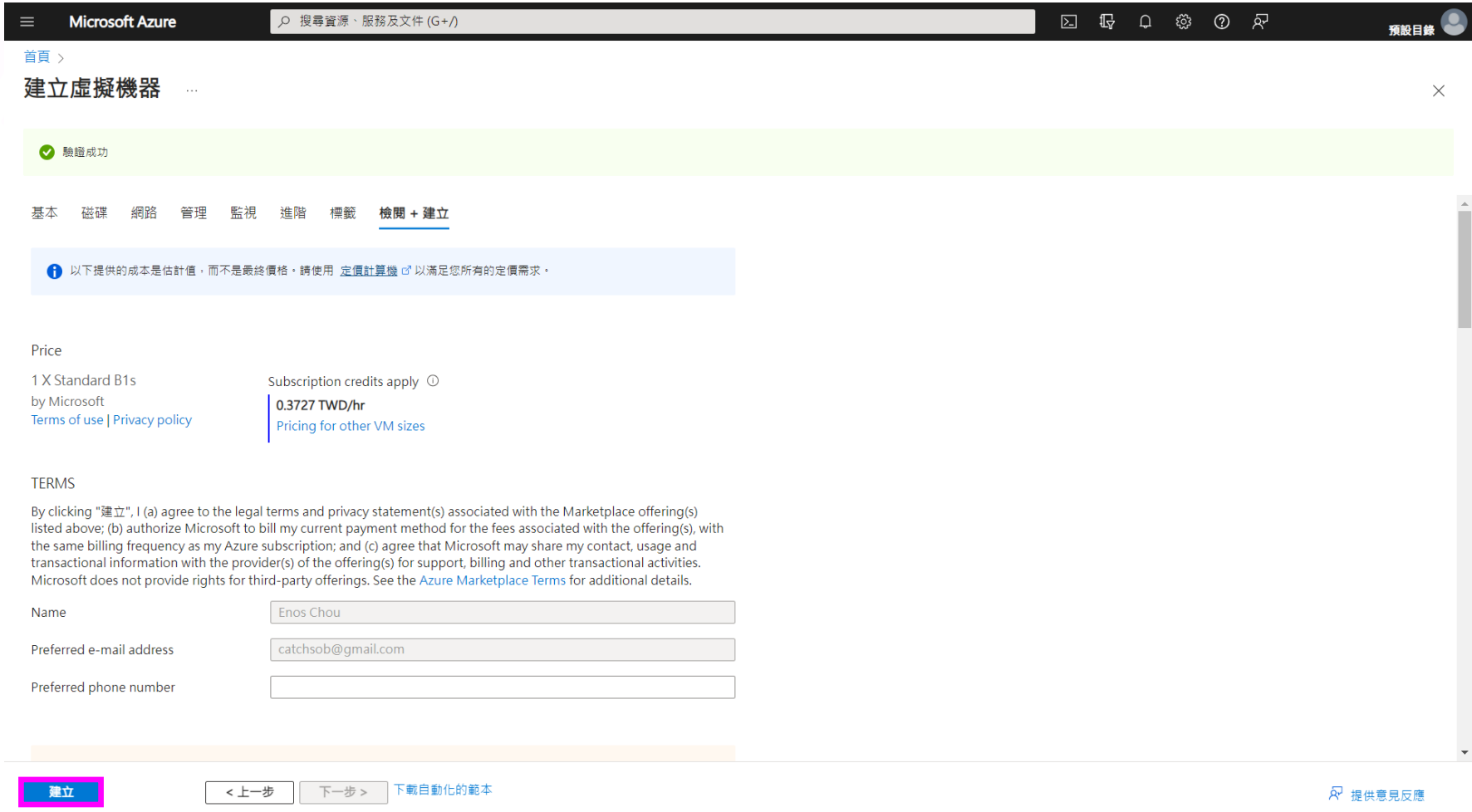
b. 建立虛擬機器

視需要加入識別用標籤

架設 VM - Azure Virtual Machines

2. 建立 VM

b. 建立虛擬機器



© 2026 Enos Chou

架設 VM - Azure Virtual Machines

2. 建立 VM

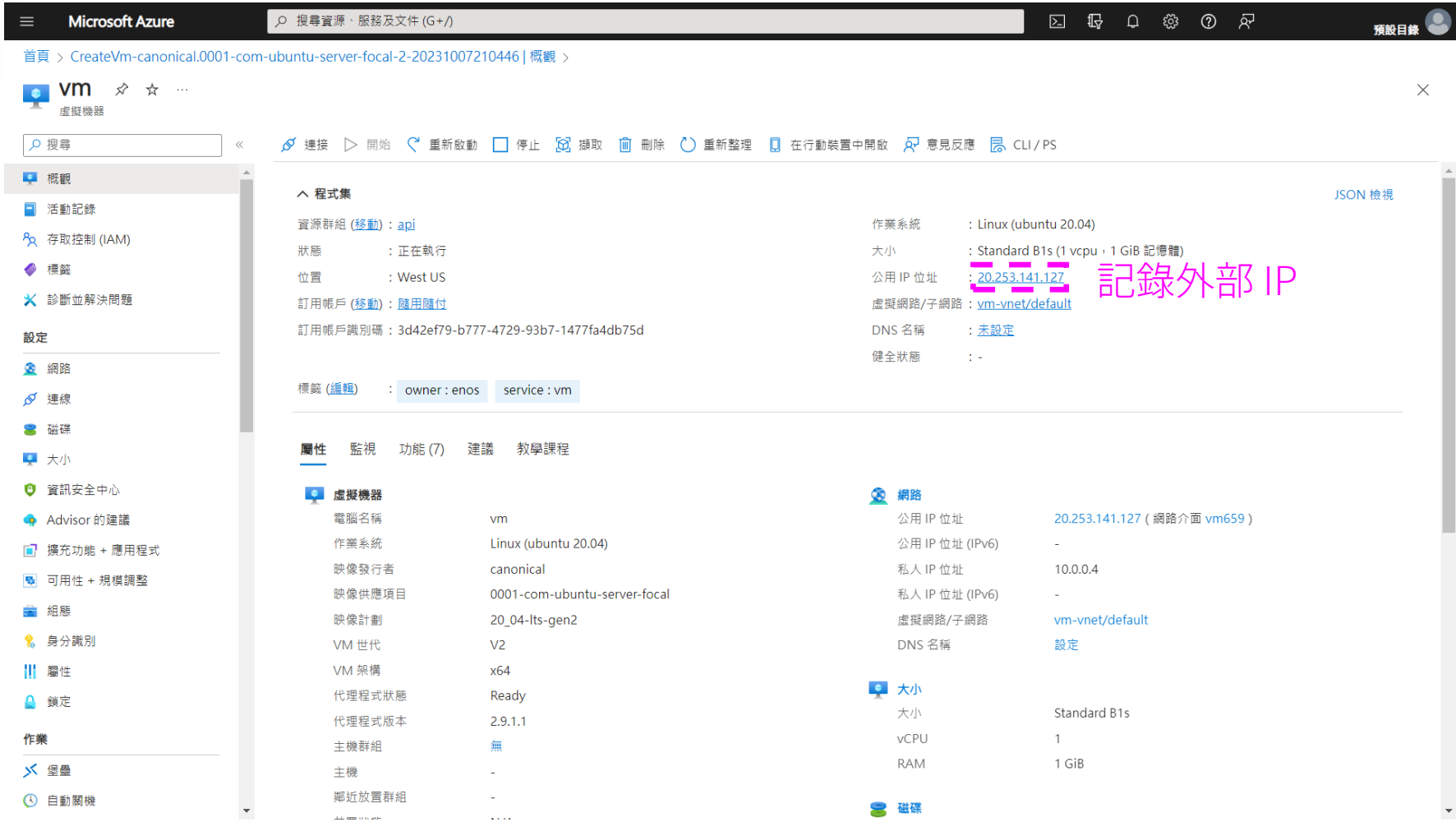
b. 建立虛擬機器

© 2026 Enos Chou

架設 VM - Azure Virtual Machines

2. 建立 VM

b. 建立虛擬機器



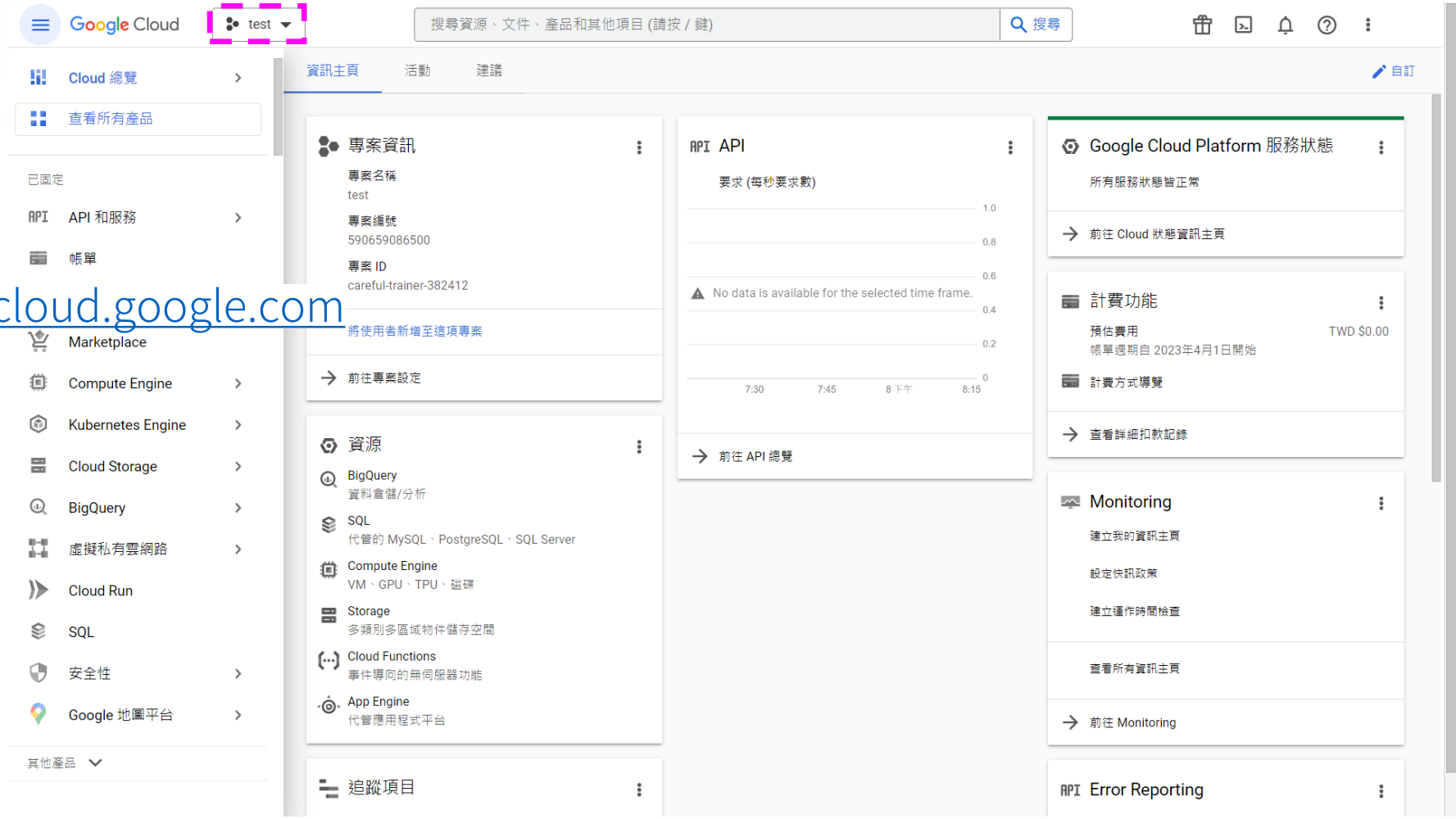
© 2026 Enos Chou

架設 VM - Google Compute Engine

2. 建立 VM

a. 登入

https://console.cloud.google.com

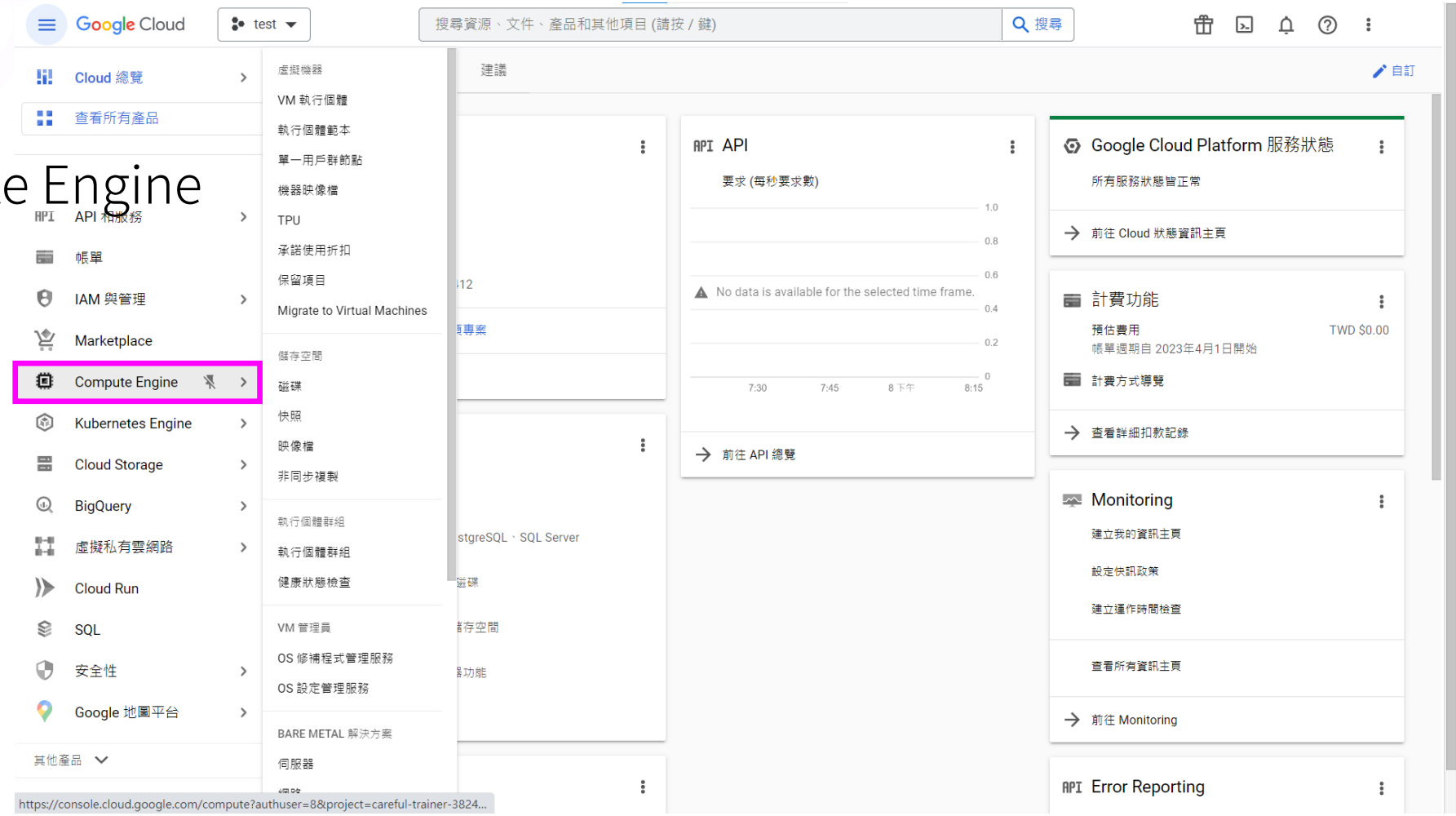


© 2026 Enos Chou

架設 VM - Google Compute Engine

2. 建立 VM

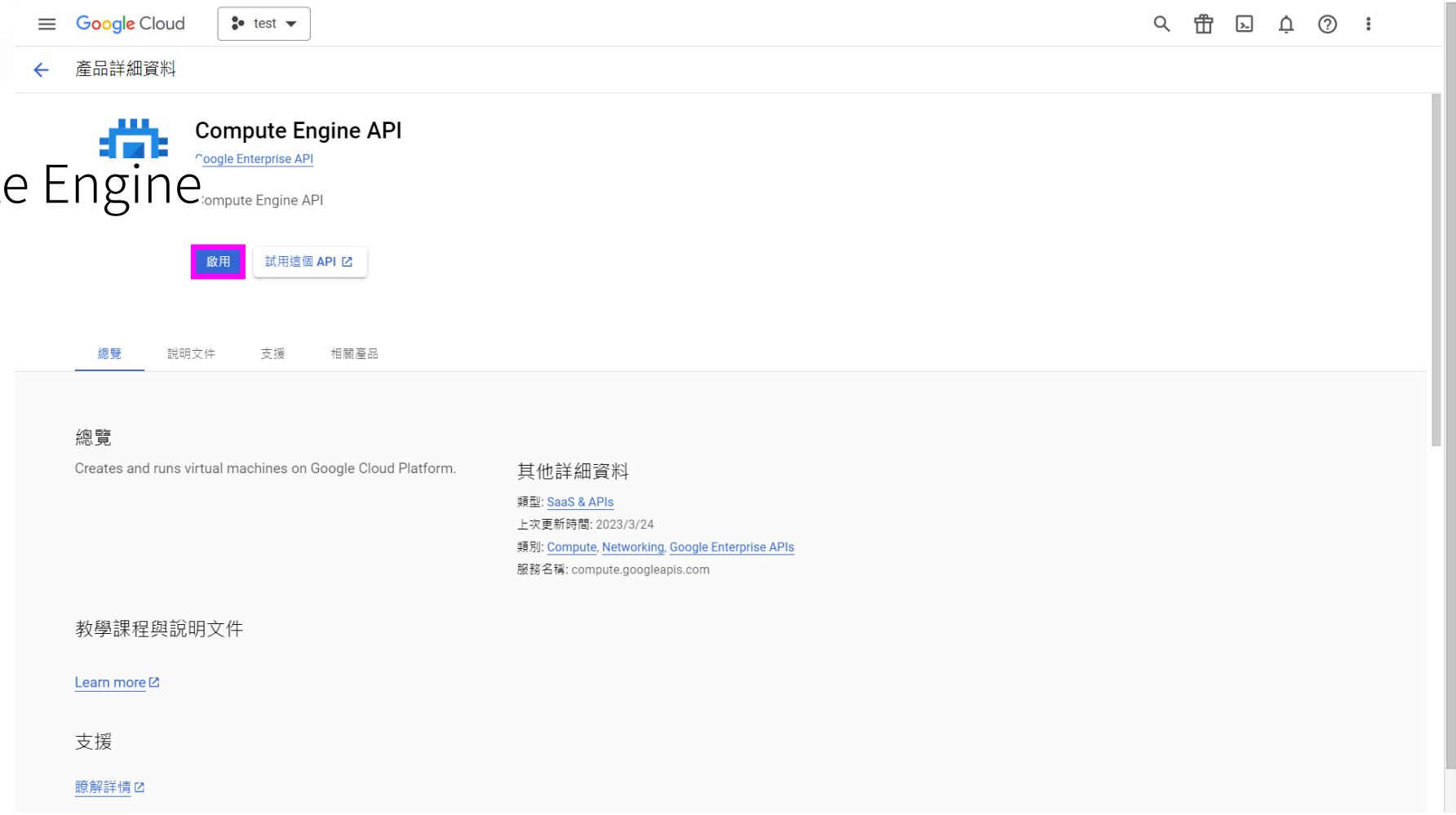
b. 啟用 Compute Engine



架設 VM - Google Compute Engine

2. 建立 VM

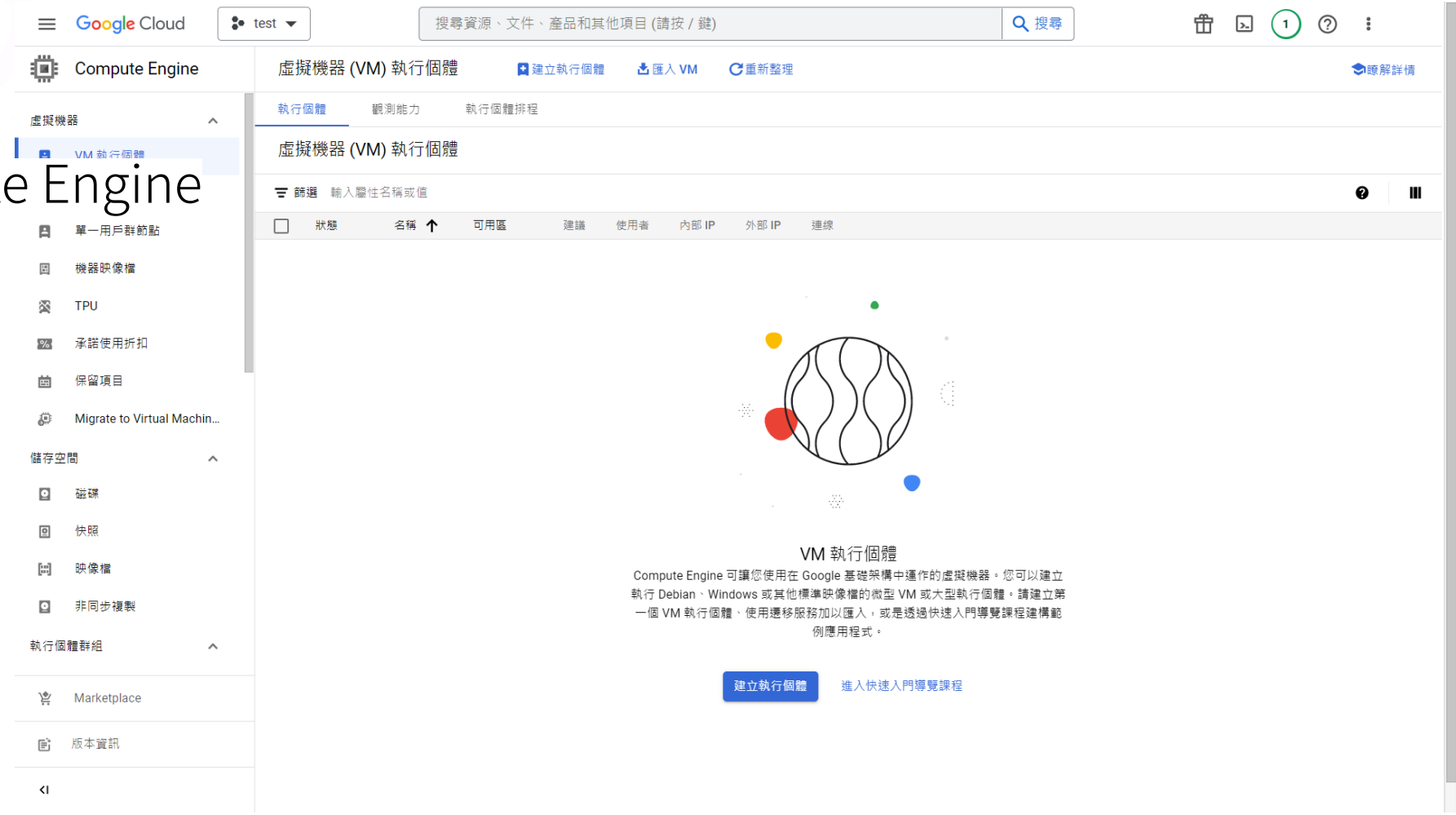
b. 啟用 Compute Engine



架設 VM - Google Compute Engine

2. 建立 VM

b. 啟用 Compute Engine

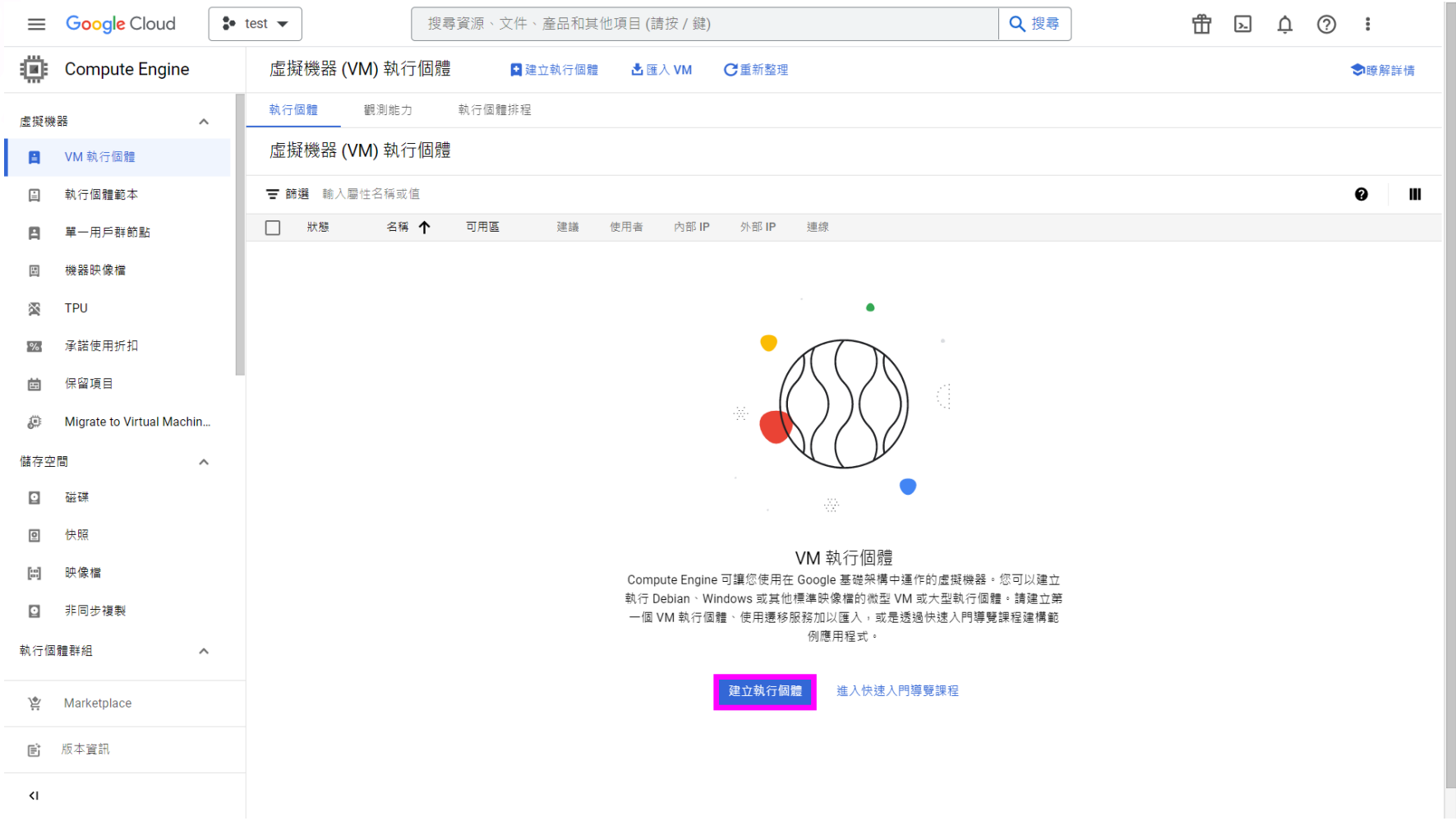


© 2026 Enos Chou

架設 VM - Google Compute Engine

2. 建立 VM

c. 建立 VM



© 2026 Enos Chou

架設 VM - Google Compute Engine

2. 建立 VM

c. 建立 VM

GCE 一律免費方案 (Free Tier)

https://cloud.google.com/ > 取得價值 $300 美元的免費抵免額，並免費使用超過 20 項產品 > 每月用量限制 > 往下捲

Compute Engine

每月 1 個 e2-micro 執行個體

- 2 個 vCPU
- 1GB 記憶體
- 地區限制：奧勒岡州： us-west1、愛荷華州：us-central1、南卡羅來納州：us-east1

每月 30GB 標準永久磁碟

每月 1GB 網路輸出流量

注意：預設為**已平衡的永久磁碟**。移除 VM 後需個別移除磁碟

注意：VM 停止後，VM 不收費但磁碟與外部 IP 仍會收費

© 2026 Enos Chou

架設 VM - Google Compute Engine

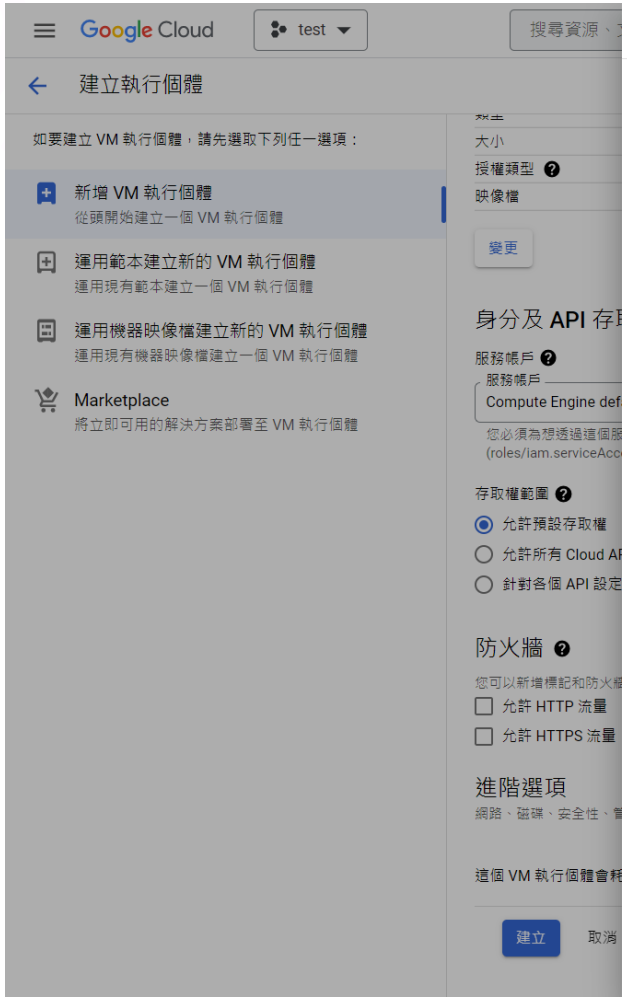
2. 建立 VM

c. 建立 VM

架設 VM - Google Compute Engine

2. 建立 VM

c. 建立 VM



作業系統選擇
Ubuntu 24.04 LTS

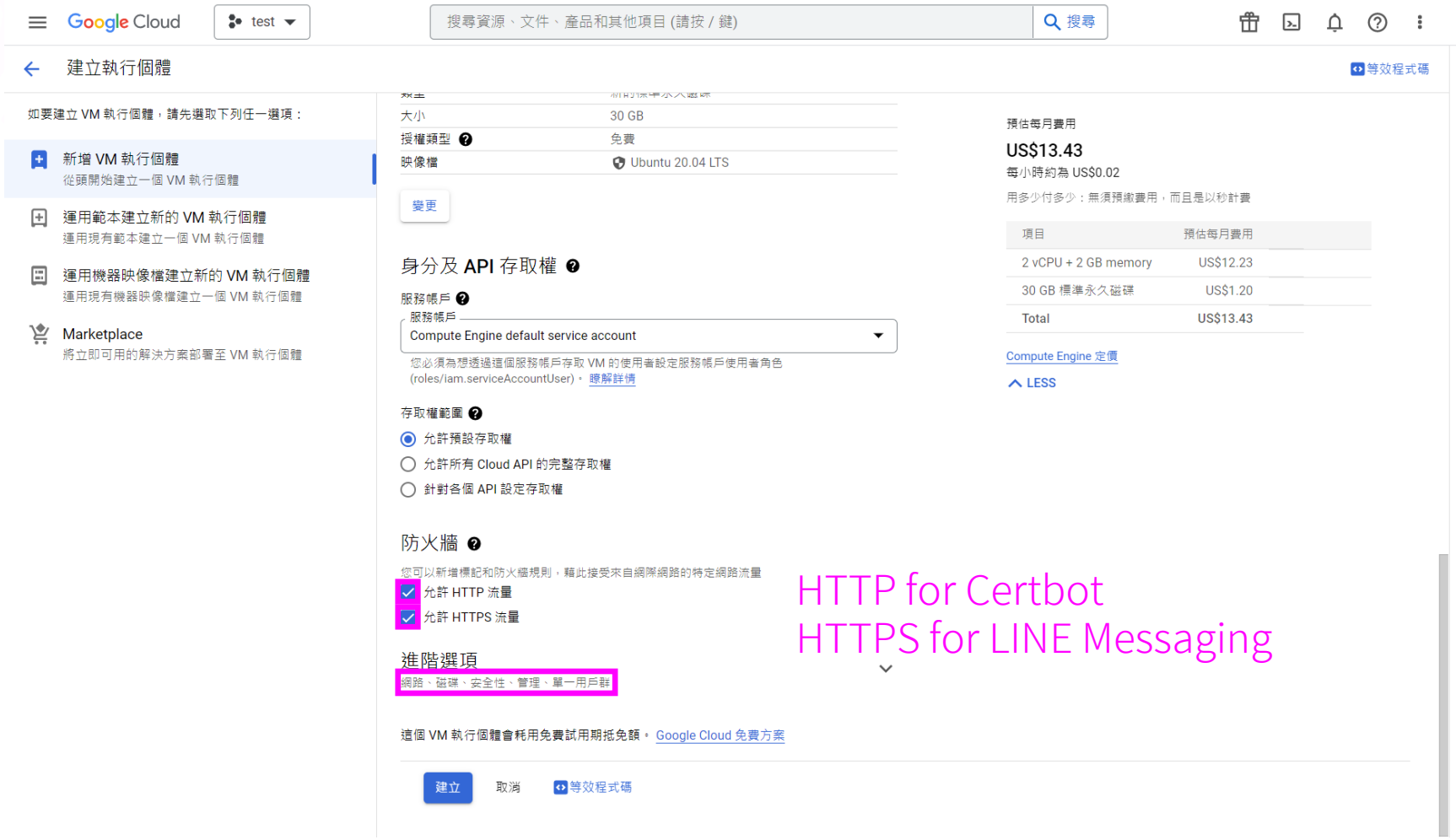
1. 硬碟空間視需求設定，一般應用可設為 30G
2. 預設的已平衡的永久硬碟非永久免費，為防未來若轉為正式帳號導致誤出帳可改為標準永久磁碟

© 2026 Enos Chou

架設 VM - Google Compute Engine

2. 建立 VM

c. 建立 VM



架設 VM - Google Compute Engine

2. 建立 VM

c. 建立 VM

架設 VM - Google Compute Engine

2. 建立 VM

c. 建立 VM

架設 VM - Google Compute Engine

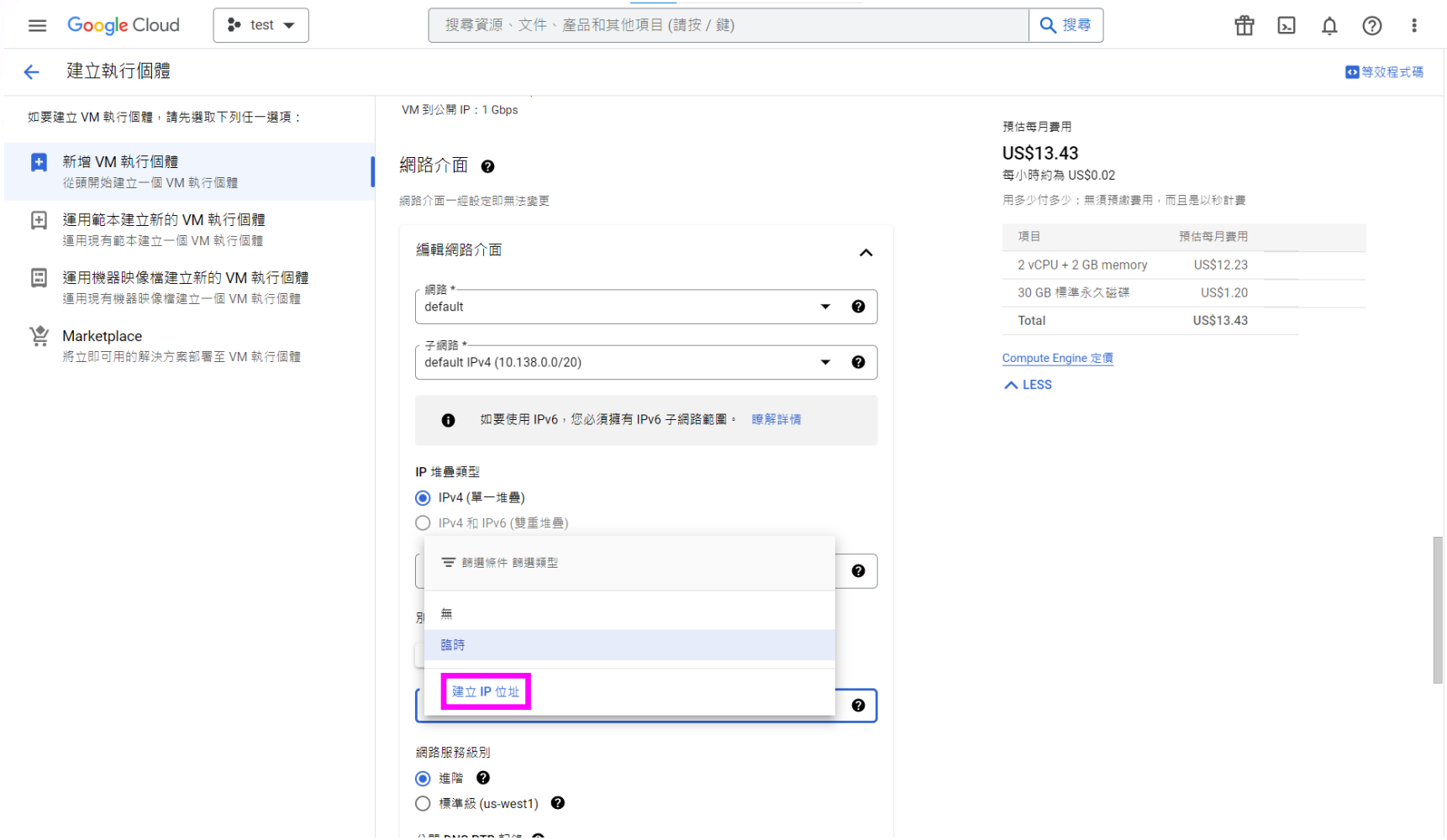
2. 建立 VM

c. 建立 VM

架設 VM - Google Compute Engine

2. 建立 VM

c. 建立 VM



架設 VM - Google Compute Engine

2. 建立 VM

c. 建立 VM

架設 VM - Google Compute Engine

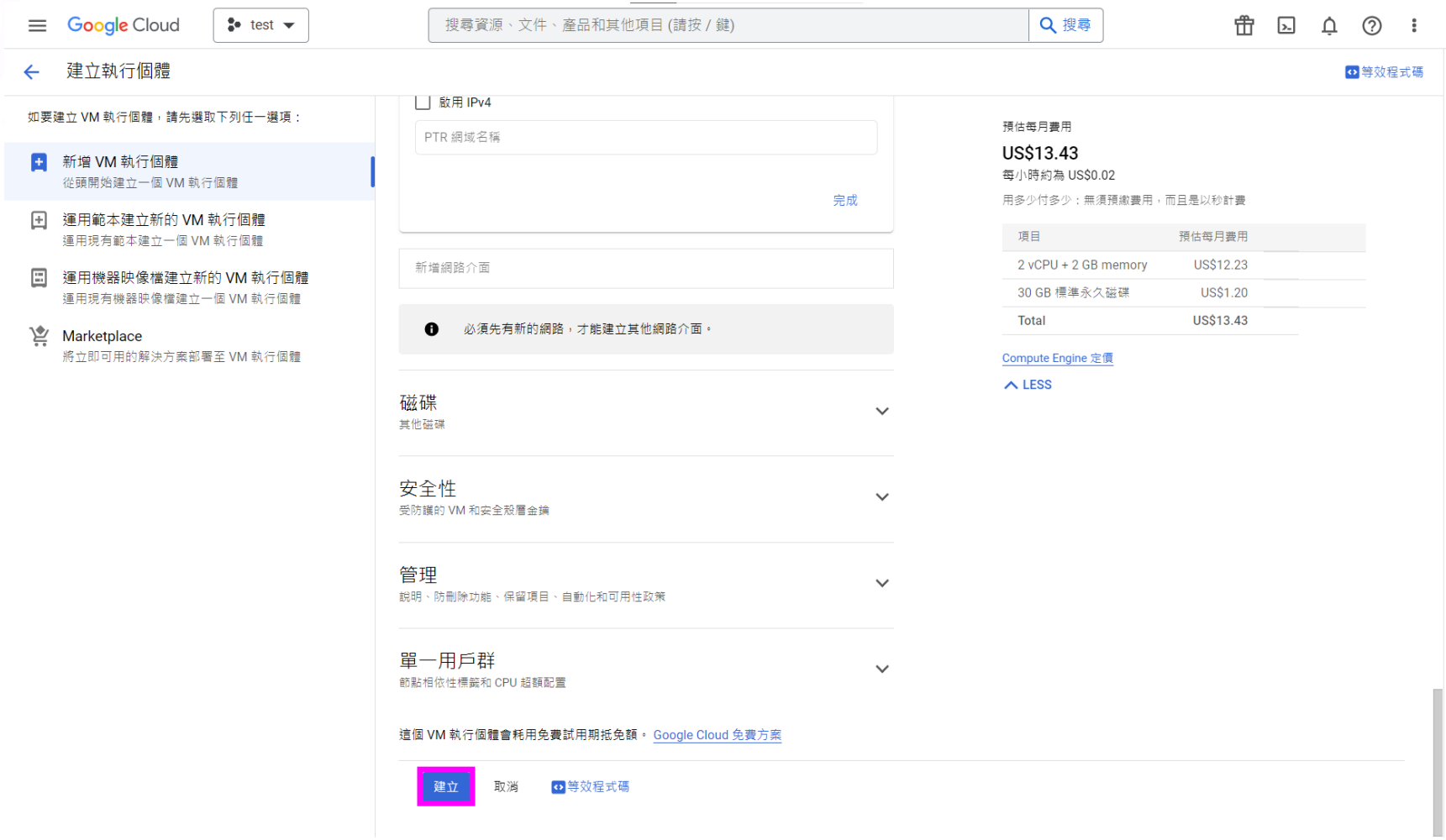
2. 建立 VM

c. 建立 VM

架設 VM - Google Compute Engine

2. 建立 VM

c. 建立 VM



架設 VM - Google Compute Engine

2. 建立 VM

c. 建立 VM

© 2026 Enos Chou

架設 VM - Google Compute Engine

2. 建立 VM

d. 匯入公鑰

架設 VM - Google Compute Engine

2. 建立 VM

d. 匯入公鑰

架設 VM - Google Compute Engine

2. 建立 VM

d. 匯入公鑰

架設 VM - Google Compute Engine

2. 建立 VM

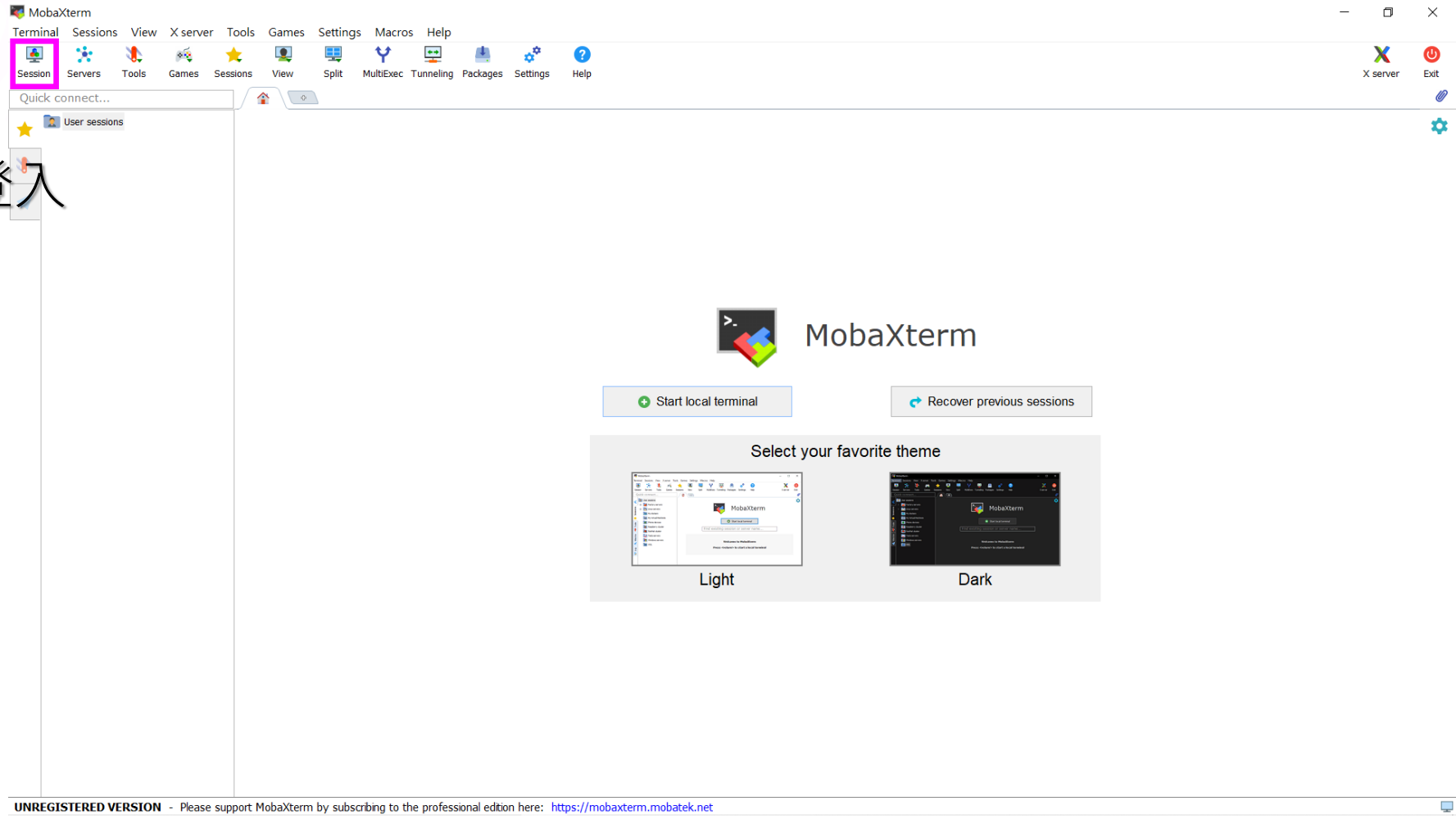
d. 匯入公鑰

© 2026 Enos Chou

架設 VM

3. 登入 VM

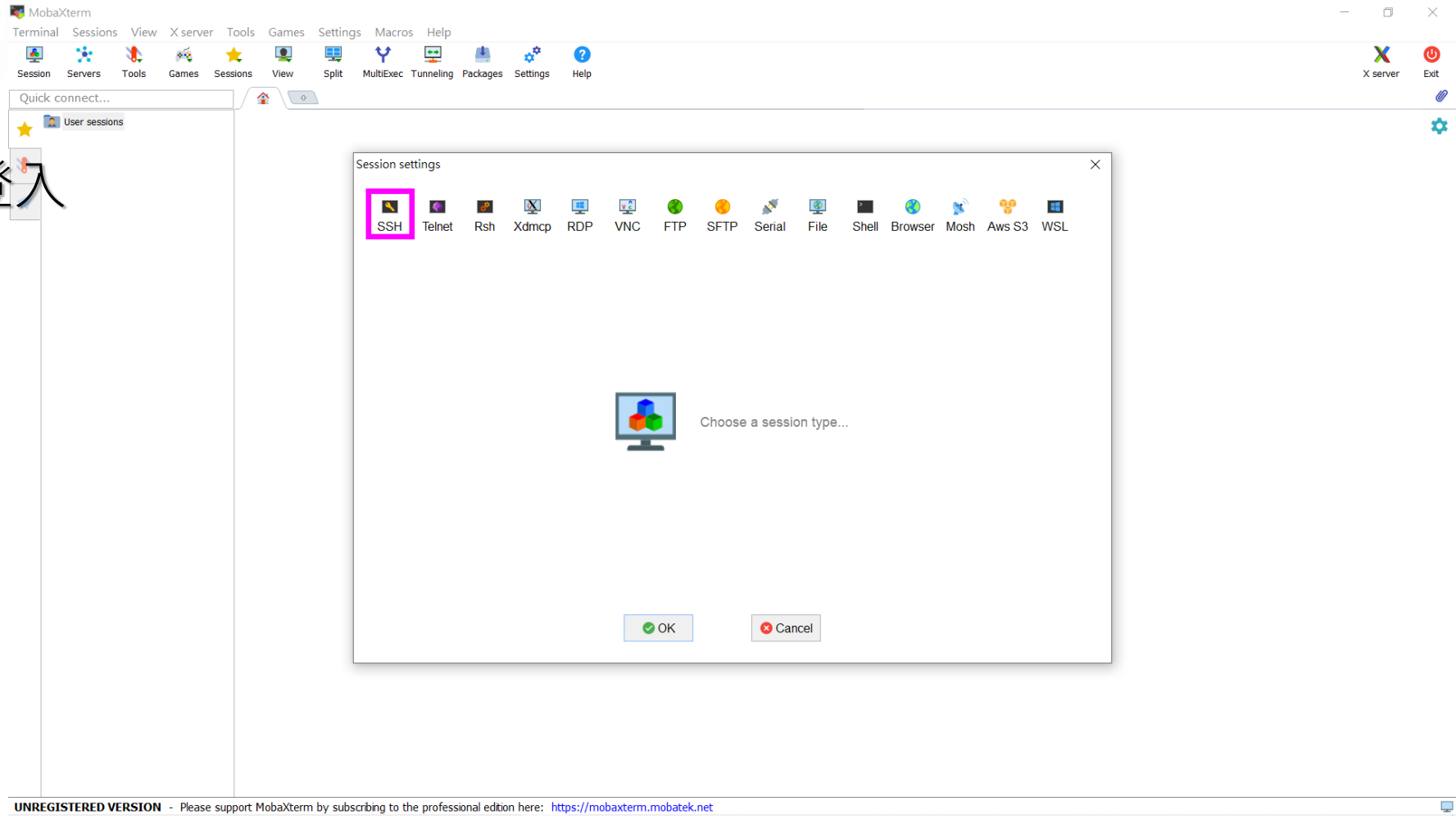
a. MobaXterm 登入



架設 VM

3. 登入 VM

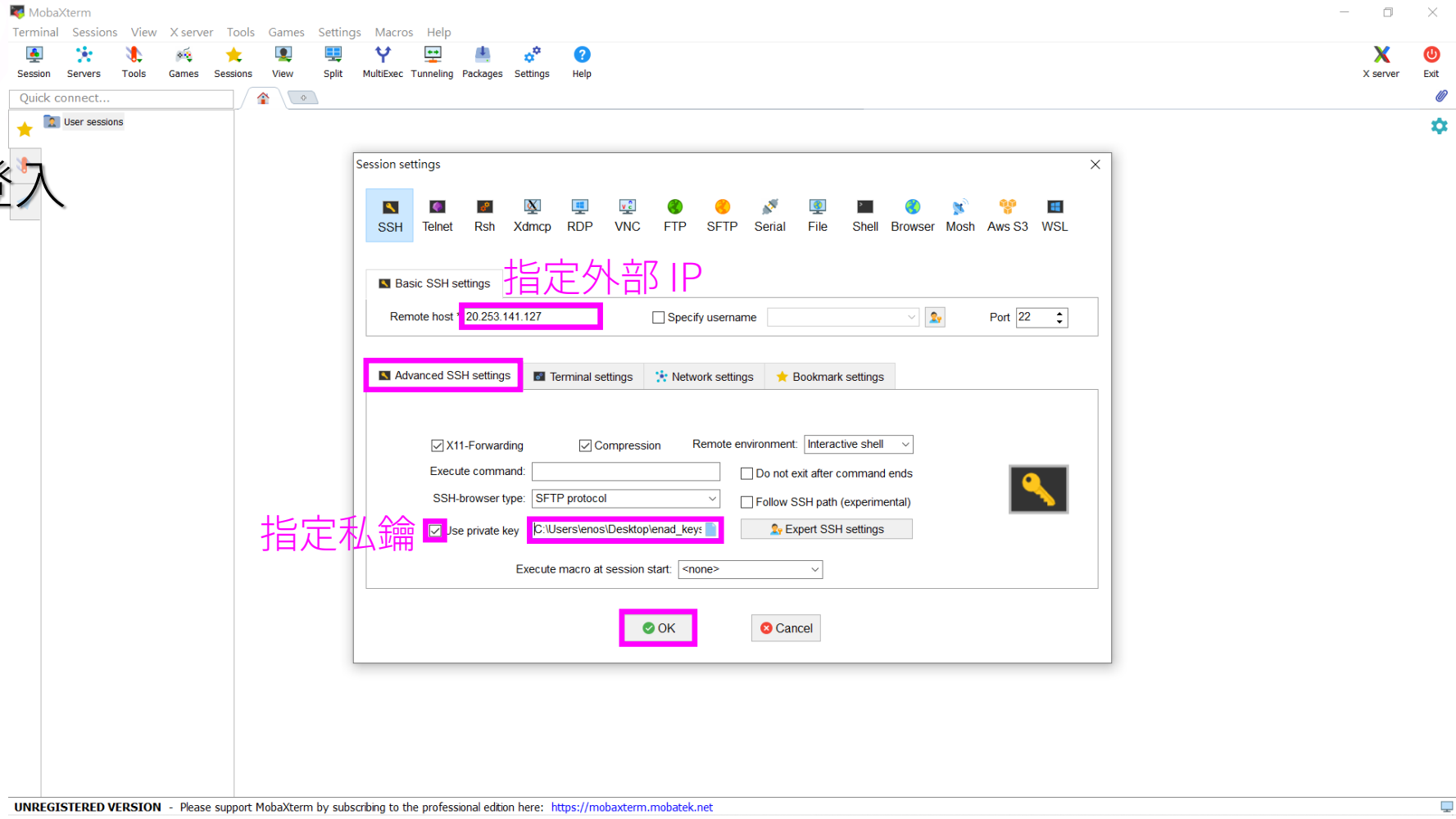
a. MobaXterm 登入



架設 VM

3. 登入 VM

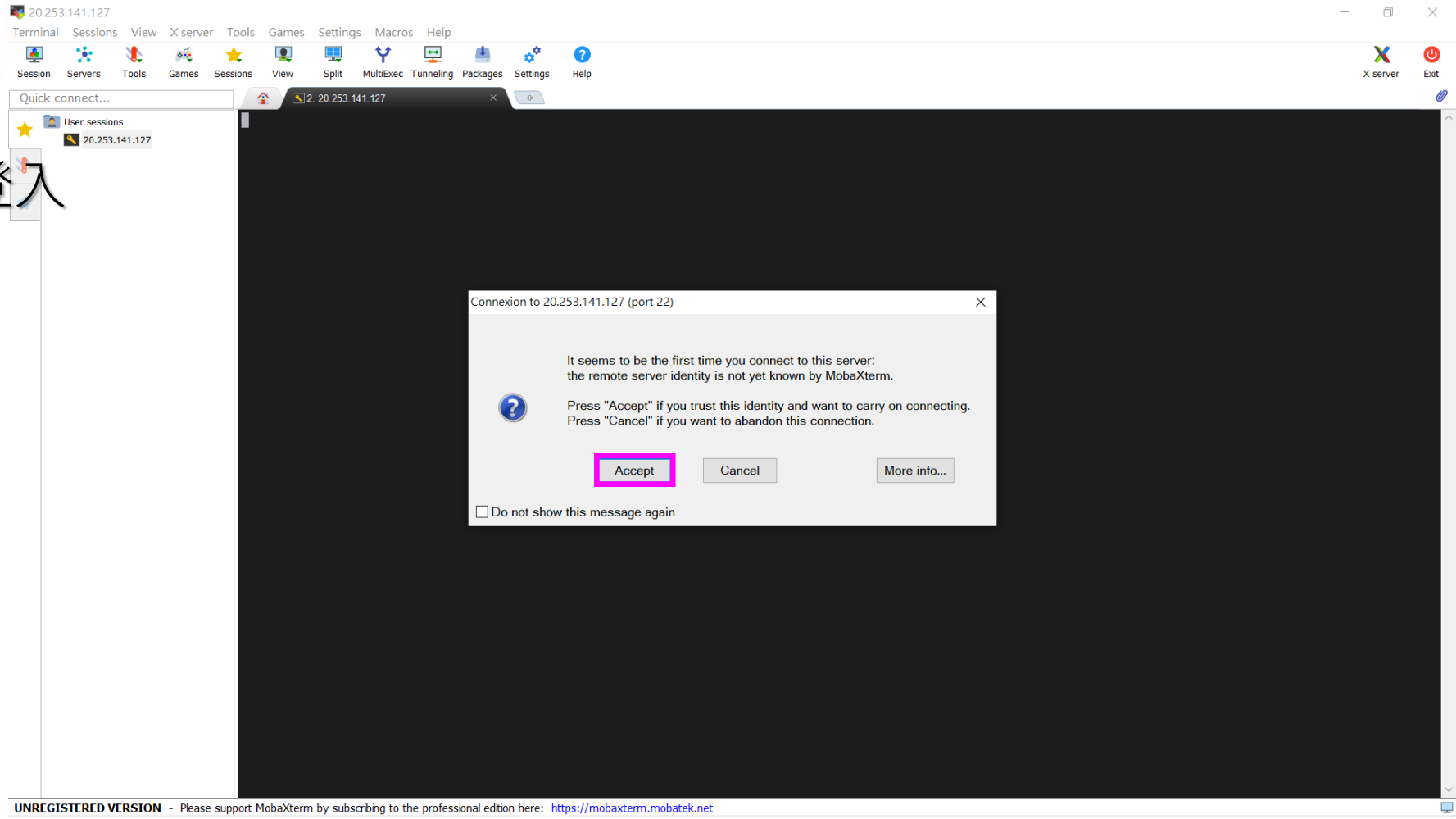
a. MobaXterm 登入



架設 VM

3. 登入 VM

a. MobaXterm 登入

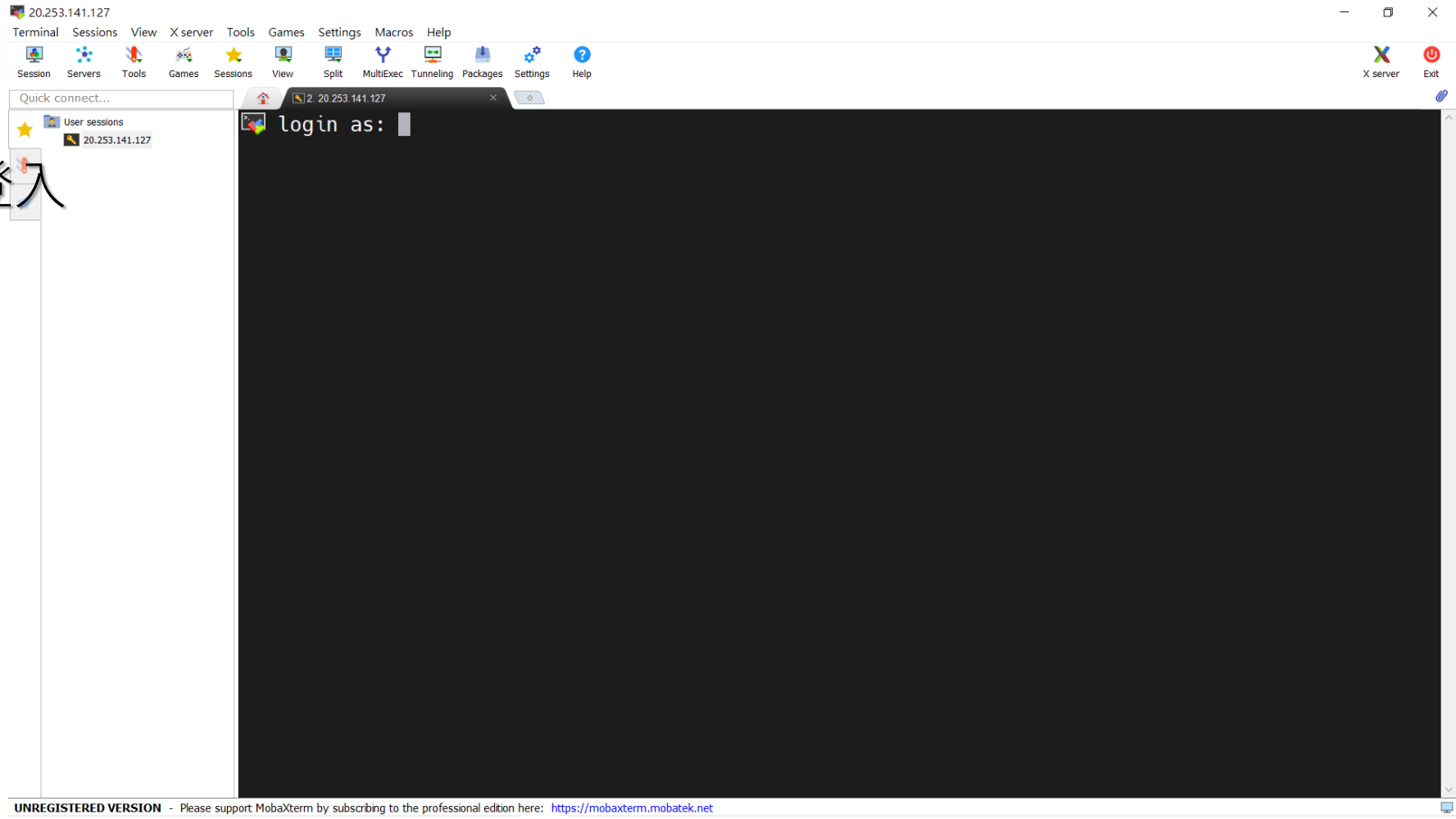


© 2026 Enos Chou

架設 VM

3. 登入 VM

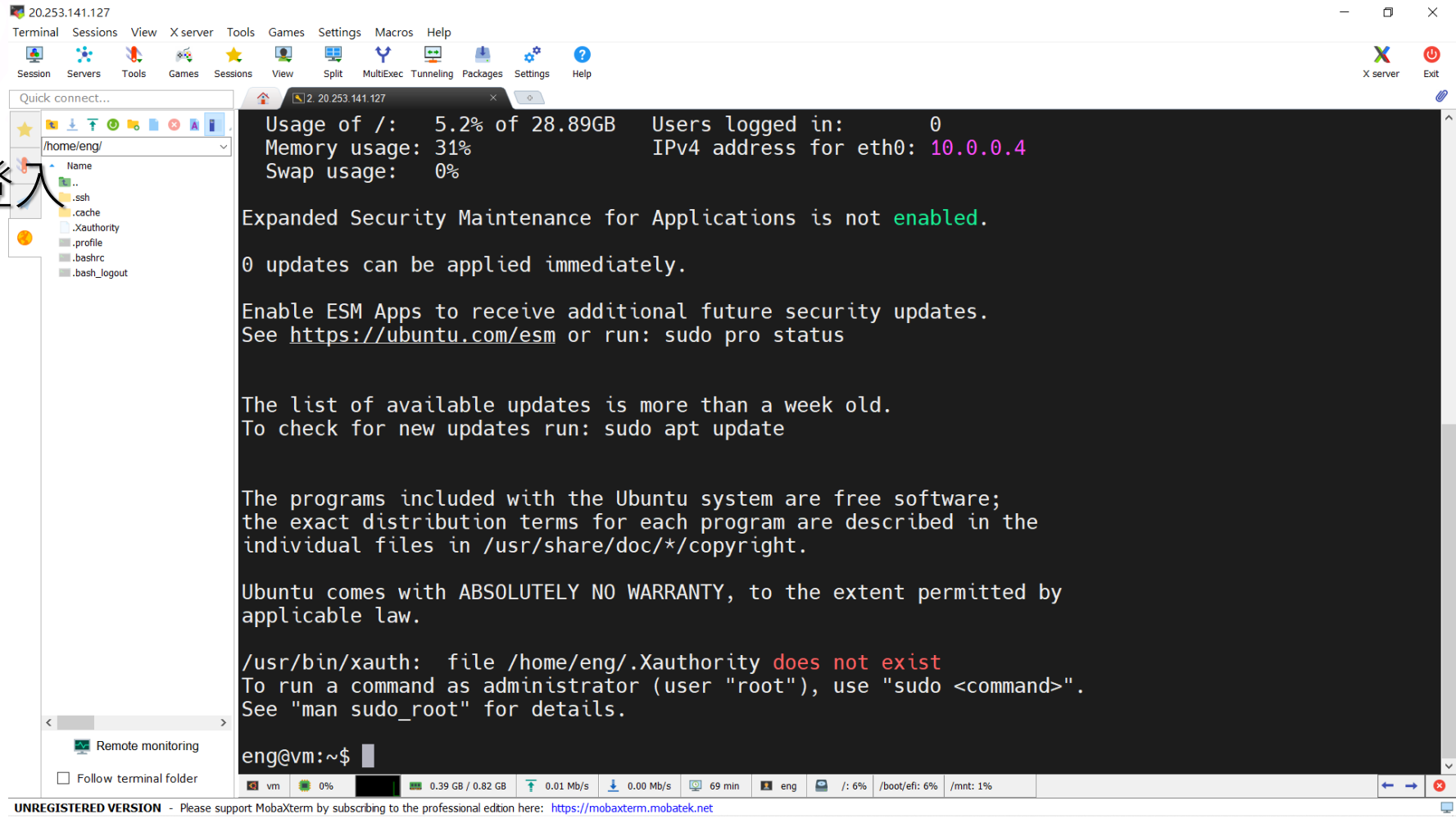
a. MobaXterm 登入



架設 VM

3. 登入 VM

a. MobaXterm 登入



© 2026 Enos Chou

架設 VM

3. 登入 VM

a. 終端機登入 on macOS/ Linux/ Cloud Shell

① (chmod 400 *your_keyfile*)
(chmod 400 eng.pem)

② ssh -i *your_keyfile your_account@your_vmip*
ssh -i eng eng@20.59.117.195 # 有詢問請輸入 yes
(ssh -i eng.pem eng@20.59.117.195)

Note

若將 *your_keyfile*.pem 上傳至 Cloud Shell 使用，執行 ssh 前須先執行
chmod 400 *your_keyfile*.pem

架設 VM - AWS EC2

4. (更換帳號)

a. 建立新帳號

```
sudo adduser --disabled-password --gecos "" your_account
sudo mkdir /home/your_account/.ssh
sudo cp /home/ubuntu/.ssh/authorized_keys /home/your_account/.ssh/
sudo chown -R your_account:your_account /home/your_account/.ssh
sudo chmod 700 /home/your_account/.ssh
sudo chmod 600 /home/your_account/.ssh/authorized_keys
```

© 2026 Enos Chou

架設 VM - AWS EC2

4. (更換帳號)

b. 建立 sudo 權限

```
sudo usermod -aG sudo your_account
echo "your_account ALL=(ALL) NOPASSWD:ALL" | sudo tee /etc/sudoers.d/your_account
sudo chmod 440 /etc/sudoers.d/your_account
```

© 2026 Enos Chou

架設 VM - AWS EC2

4. (更換帳號)

c. 新帳號重新登入

```
logout
SHIFT+r  # in MobaXterm
```

© 2026 Enos Chou

1. Code
2. Package
3. WSGI

© 2026 Enos Chou

安裝 APP

1. 準備程式碼

a. 上傳程式

① 製做 APP 相關程式

your_module.py

...

requirements.txt

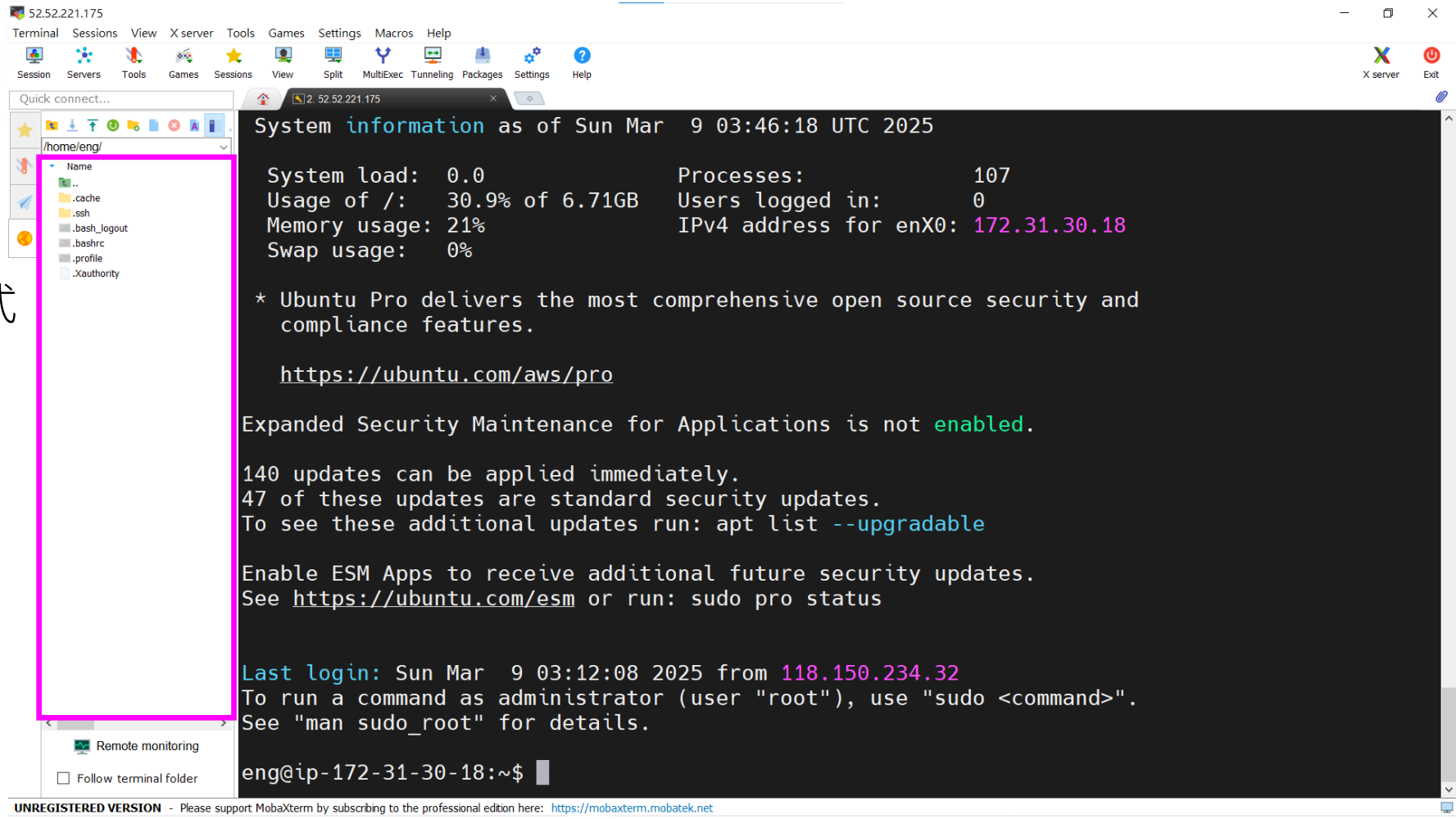
© 2026 Enos Chou

安裝 APP

1. 準備程式碼

a. 上傳程式

② 上傳前述程式

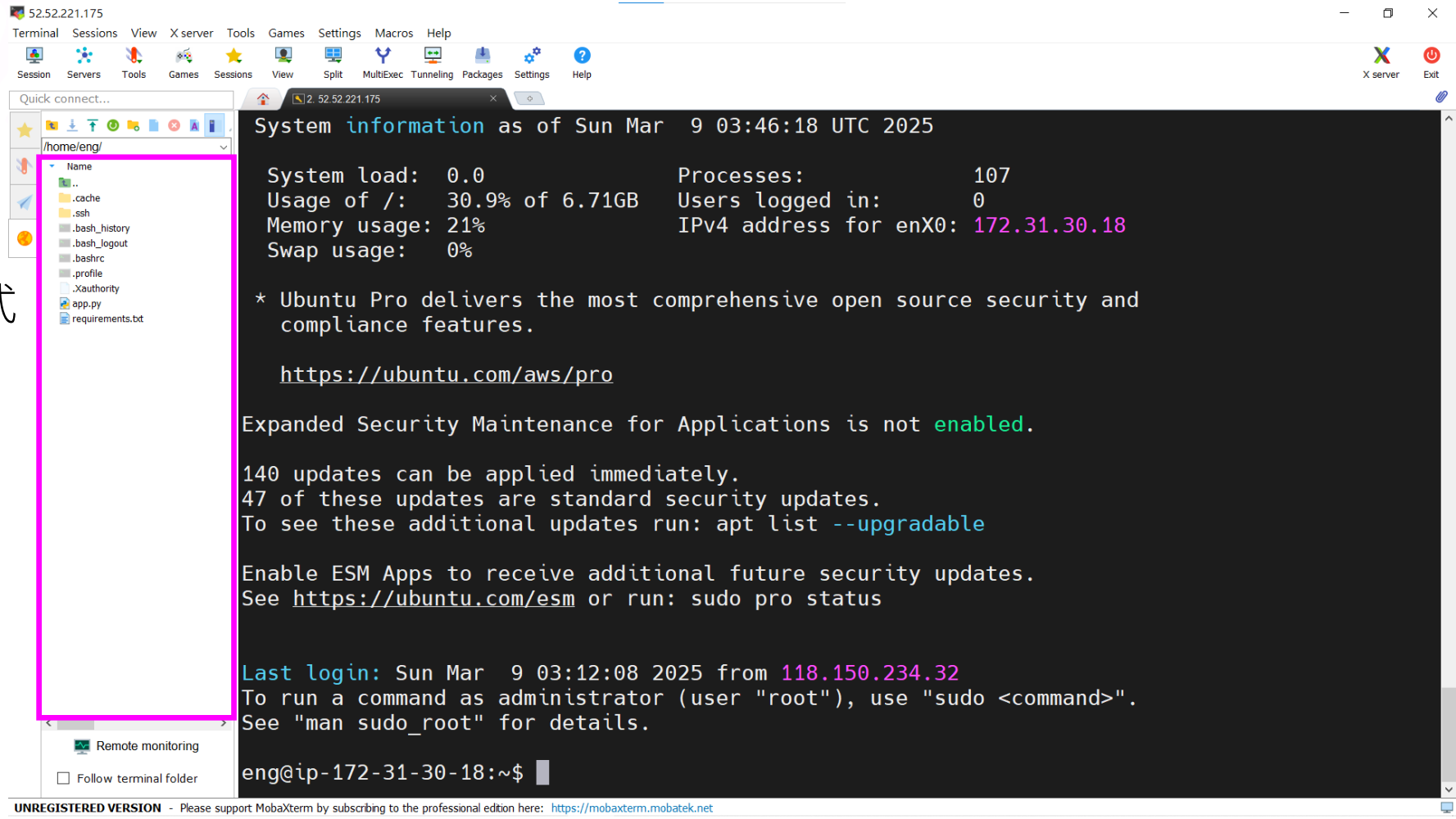


安裝 APP

1. 準備程式碼

a. 上傳程式

② 上傳前述程式



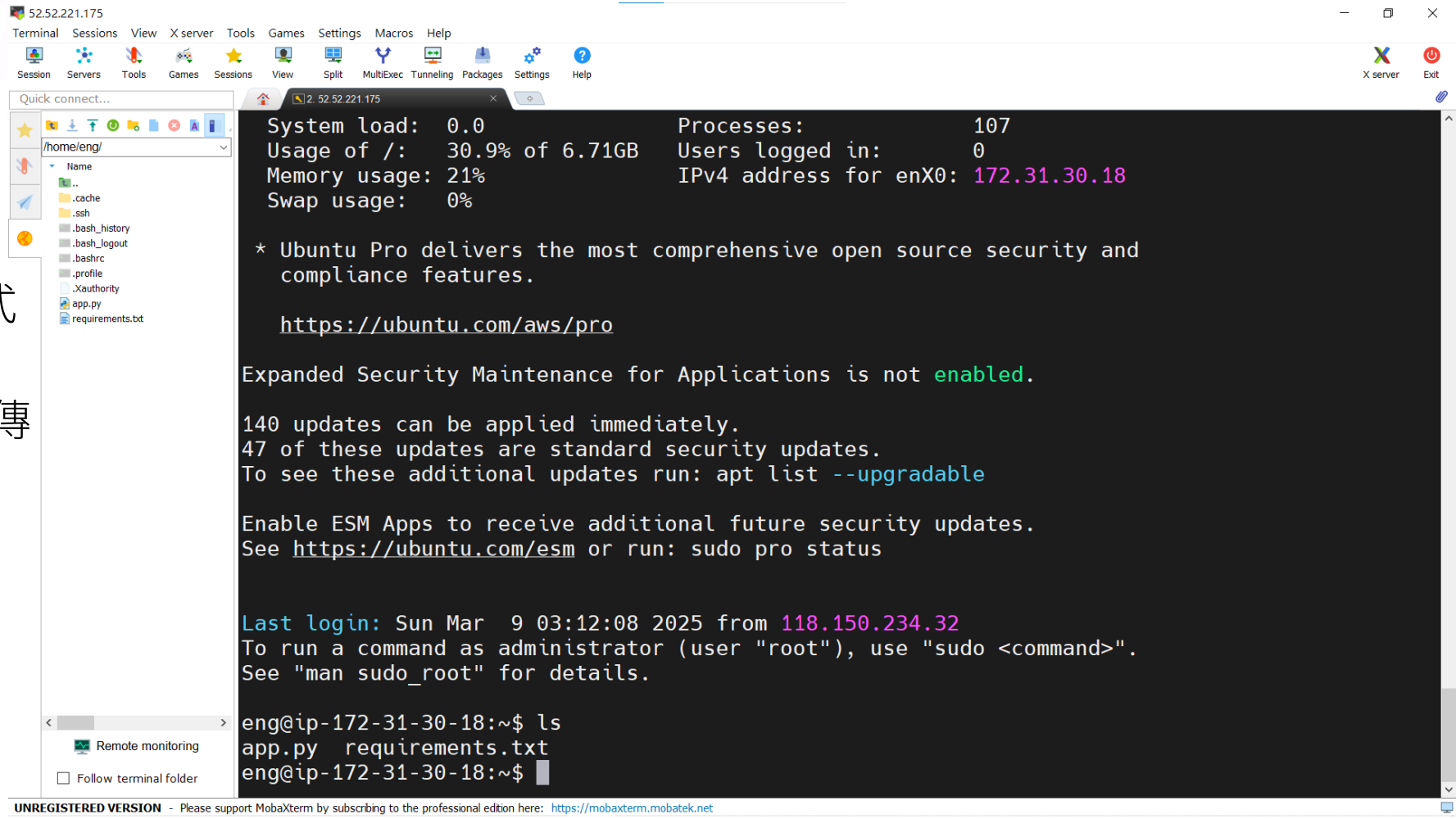
安裝 APP

1. 準備程式碼

a. 上傳程式

② 上傳前述程式

```
# 確認檔案已上傳
ls
```



© 2026 Enos Chou

安裝 APP

1. 準備程式碼

a. 上傳程式 on macOS/ Linux/ Cloud Shell

② 上傳前述程式

```
scp -i your_keyfile -r your_source_path your_account@your_vmip:
scp -i eng -r weapi eng@172.31.30.18:

# 確認已上傳
ssh -i your_keyfile your_account@your_vmip ls your_source_path
ssh -i eng eng@172.31.30.18 ls weapi
```

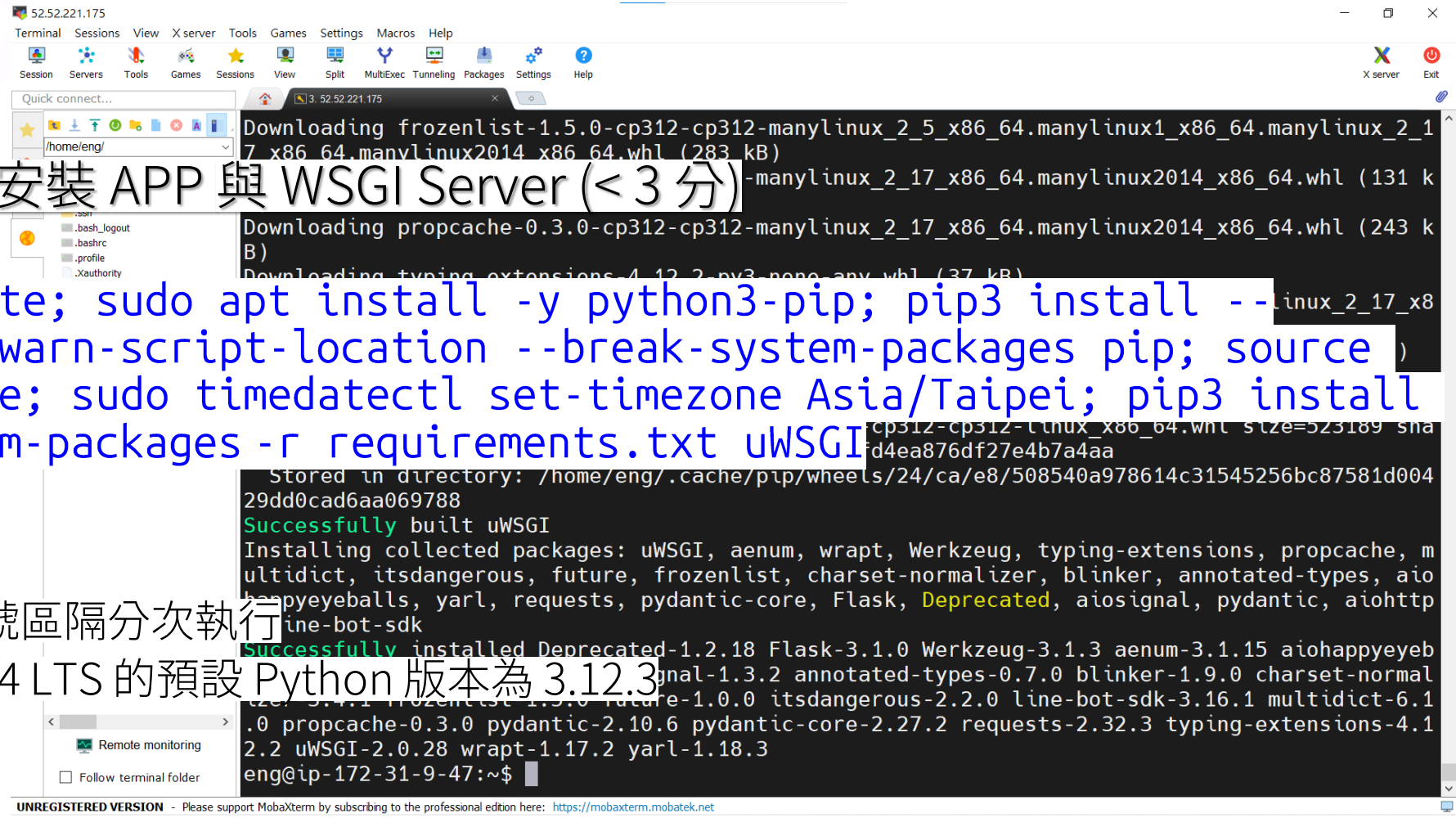
© 2026 Enos Chou

安裝 APP

2. 安裝 APP

a. 更新系統同時安裝 APP 與 WSGI Server (< 3 分)

```
sudo apt update; sudo apt install -y python3-pip; pip3 install --
upgrade --no-warn-script-location --break-system-packages pip; source
$HOME/.profile; sudo timedatectl set-timezone Asia/Taipei; pip3 install
--break-system-packages -r requirements.txt uWSGI
```



Note

1. 指令可依分號區隔分次執行
2. Ubuntu 24.04 LTS 的預設 Python 版本為 3.12.3

Why WSGI?

Warning from Flask

```
python3 your_module.py  # ft. app.run()
```

```
 * Serving Flask app 'your_module'
 * Debug mode: off
WARNING: This is a development server. Do not use it in a production deployment.
Use a production WSGI server instead.
 * Running on http://127.0.0.1:5000
Press CTRL+C to quit
```

© 2026 Enos Chou

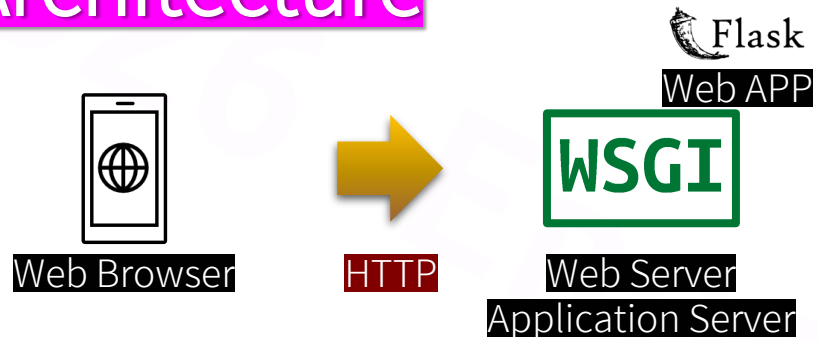
WSGI

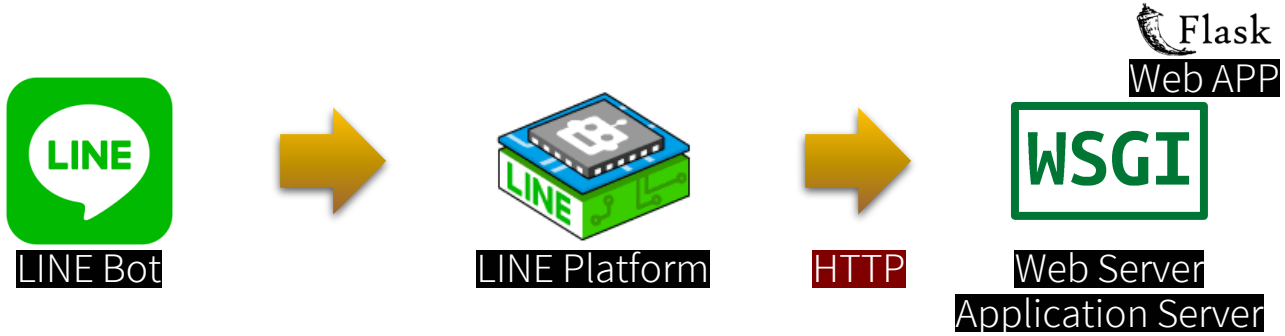
Warning from Flask

© 2026 Enos Chou

WSGI

Reasonable Architecture





© 2026 Enos Chou

安裝 APP

2. 安裝 APP

WSGI Server 的選擇

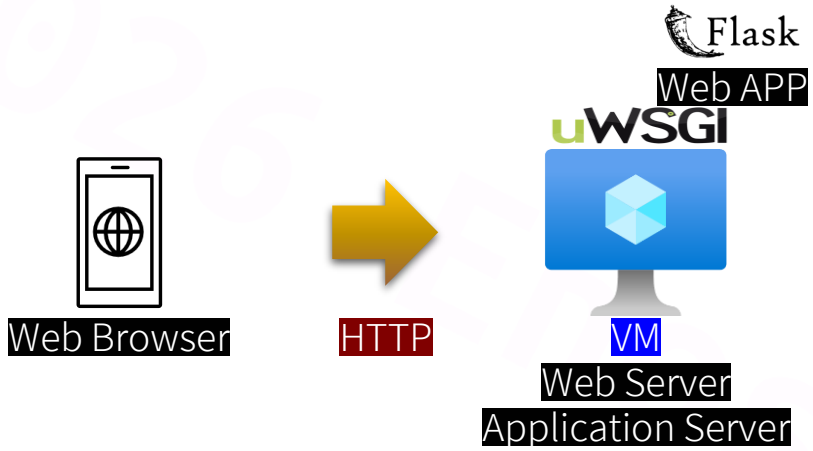
uWSGI: 經典/ 支援 HTTP/ 搭配 NGINX 支援 uwsgi protocol

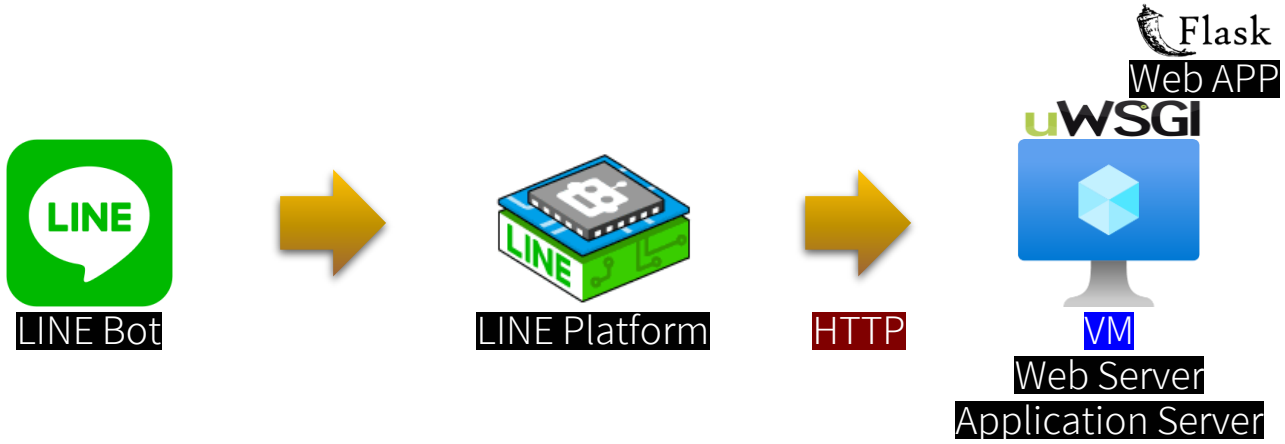
gunicorn: 新/ 支援 HTTP

© 2026 Enos Chou

安裝 APP

2. 安裝 APP





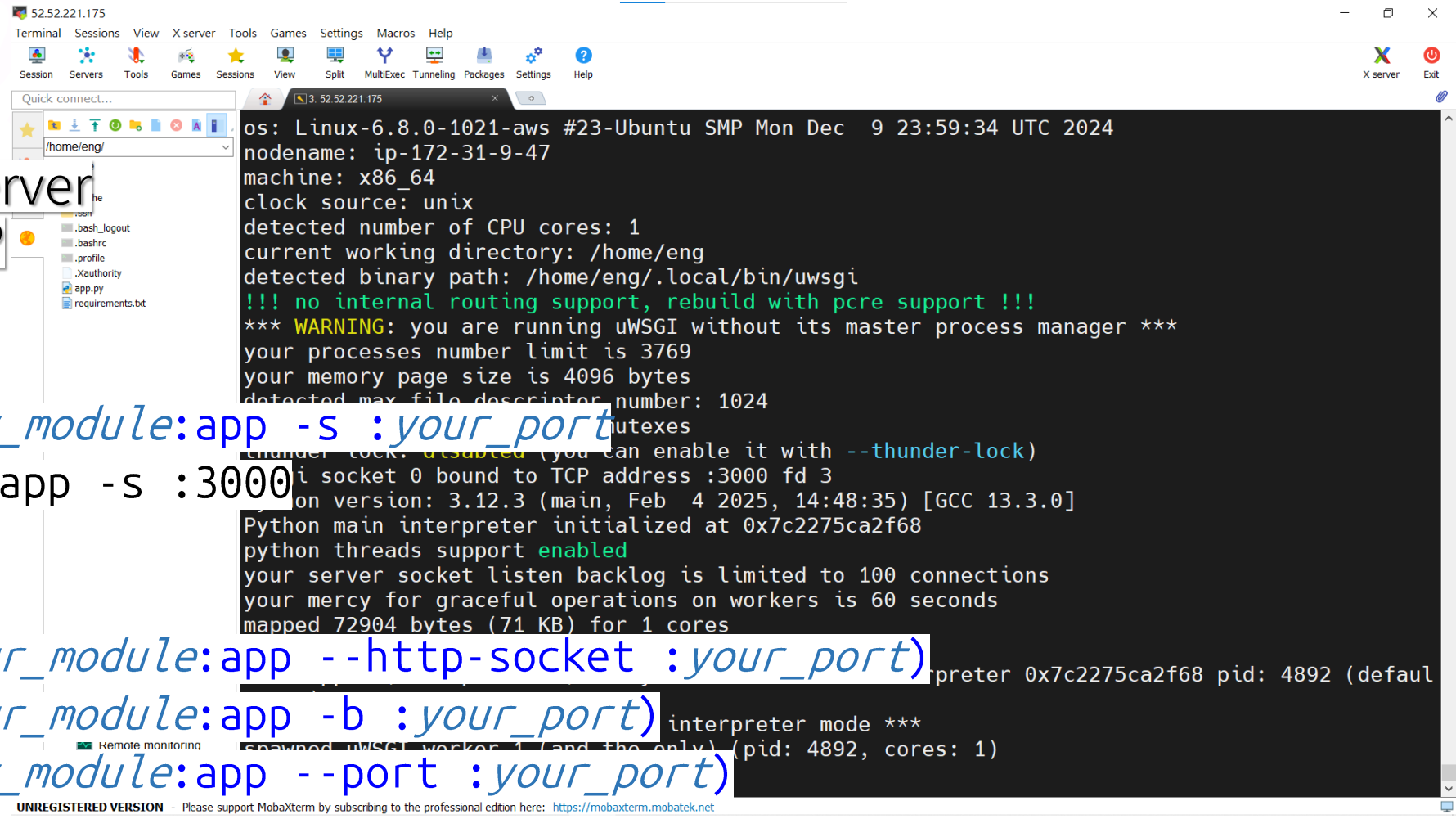
© 2026 Enos Chou

安裝 APP

3. 啟動 APP

a. 驗證 WSGI Server 正確帶起 APP

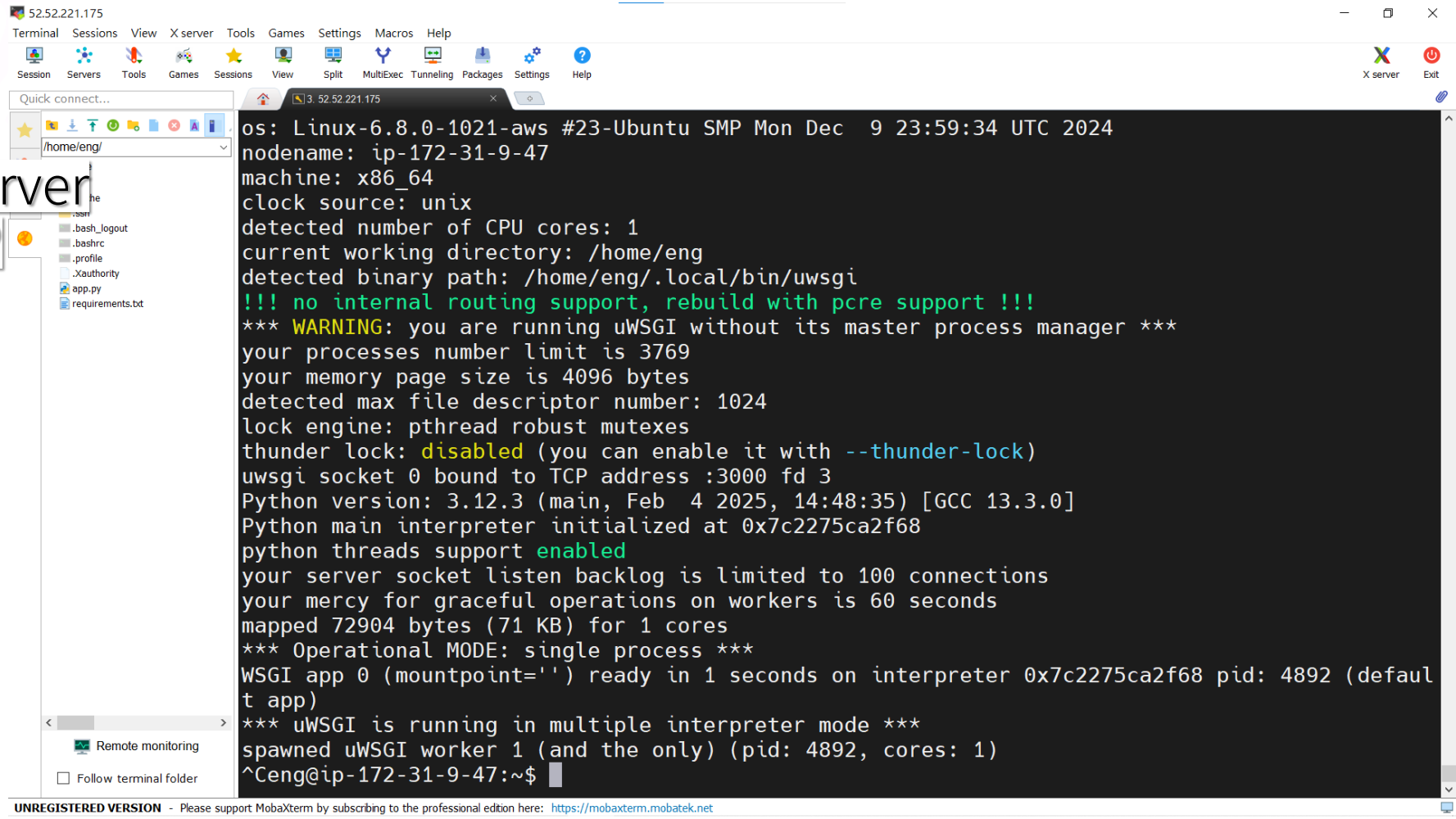
```
# uwsgi
uwsgi -w your_module:app -s :your_port
uwsgi -w app:app -s :3000

# HTTP
(uwsgi -w your_module:app --http-socket :your_port)
(gunicorn your_module:app -b :your_port)
(uvicorn your_module:app --port :your_port)
```



安裝 APP

3. 啟動 APP

a. 驗證 WSGI Server 正確帶起 APP



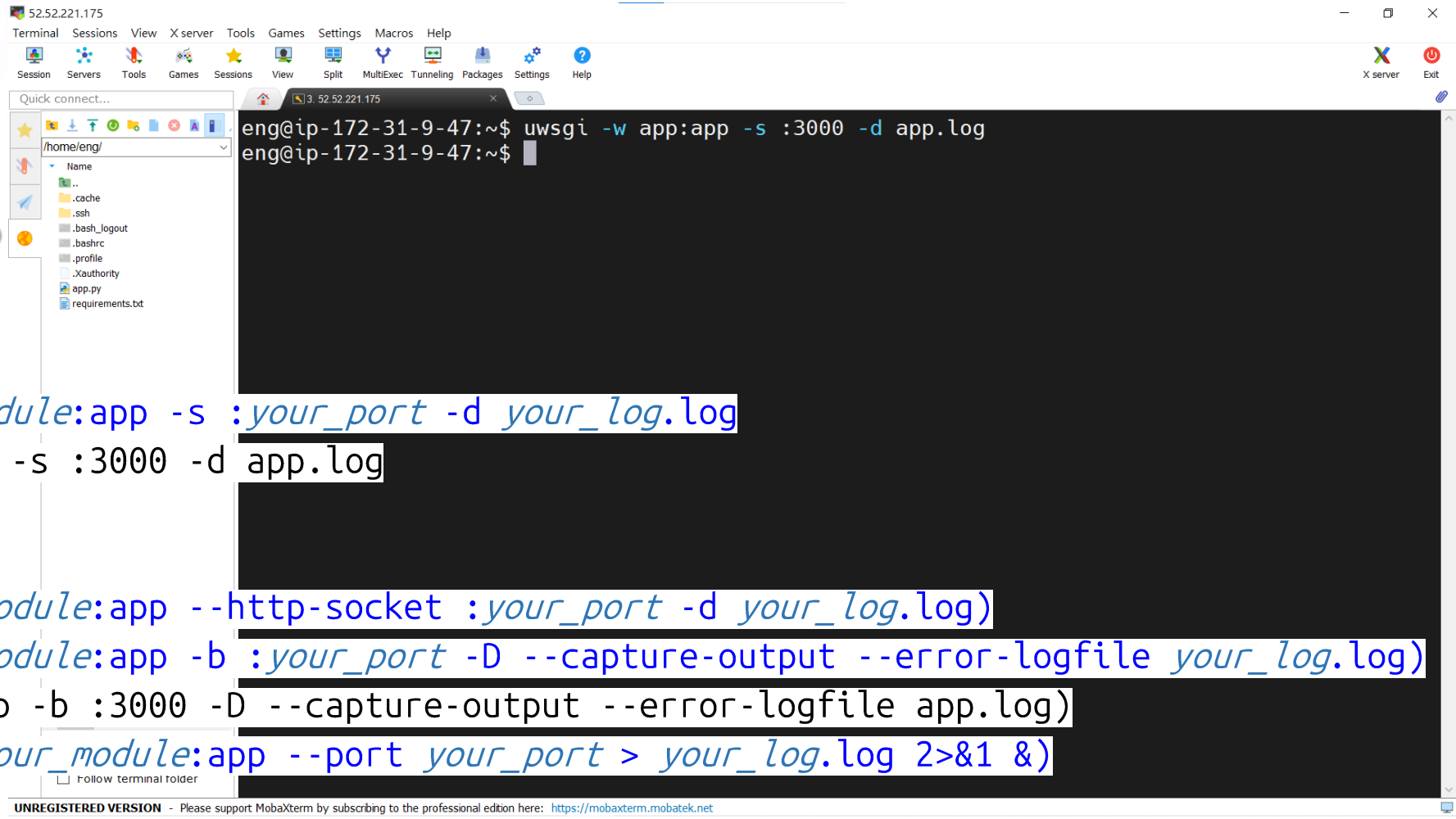
退出

CTRL+c

安裝 APP

3. 啟動 APP

b. WSGI Server 背景帶起 APP




```
# uwsgi
uwsgi -w your_module:app -s :your_port -d your_log.log
uwsgi -w app:app -s :3000 -d app.log

# HTTP
(uwsgi -w your_module:app --http-socket :your_port -d your_log.log)
(gunicorn your_module:app -b :your_port -D --capture-output --error-logfile your_log.log)
(gunicorn app:app -b :3000 -D --capture-output --error-logfile app.log)
(nohup uvicorn your_module:app --port your_port > your_log.log 2>&1 &)
```

© 2026 Enos Chou

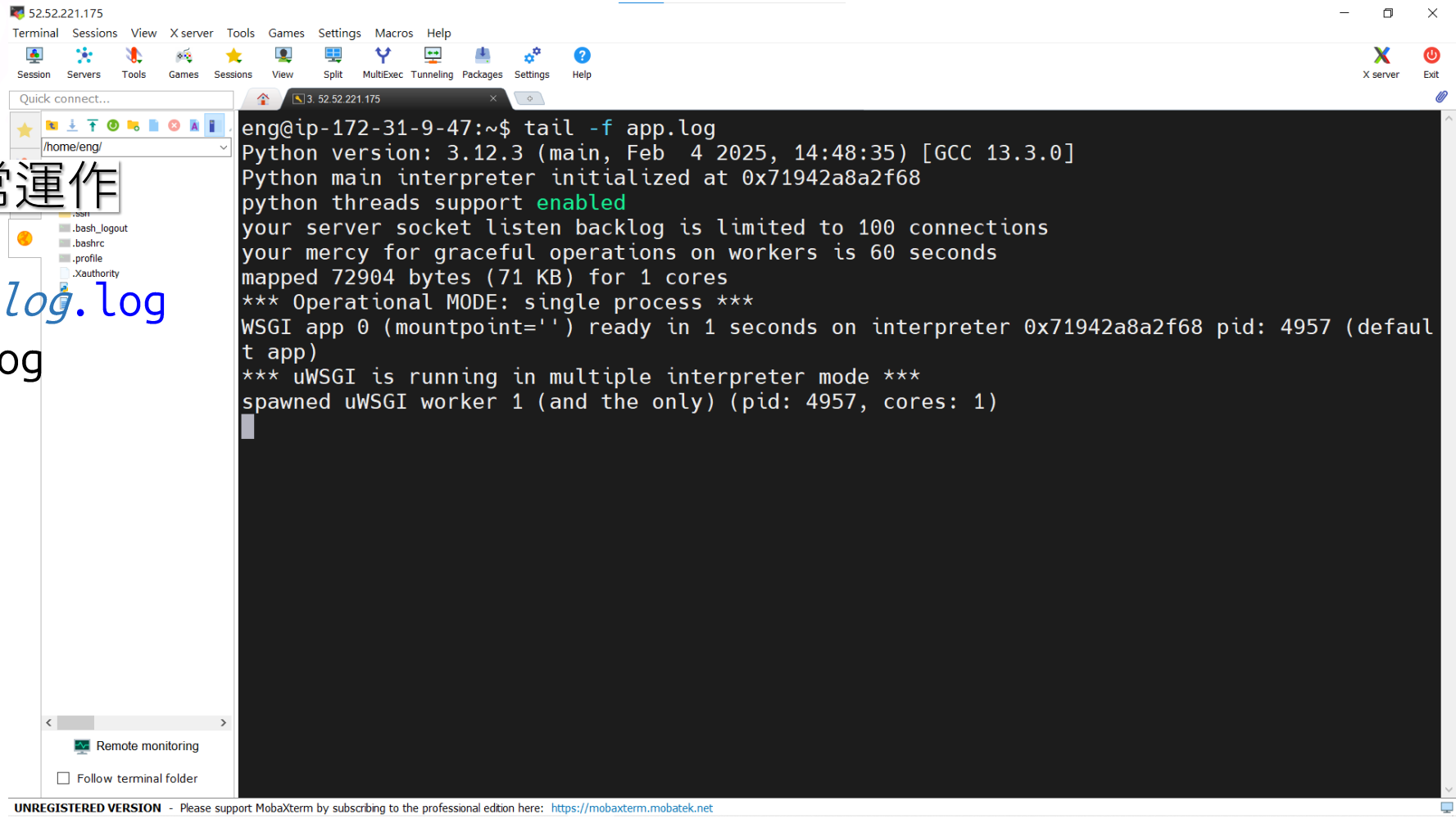
安裝 APP

3. 啟動 APP

c. 確認 APP 正常運作

```
tail -f your_log.log
tail -f app.log
```

正常範例



© 2026 Enos Chou

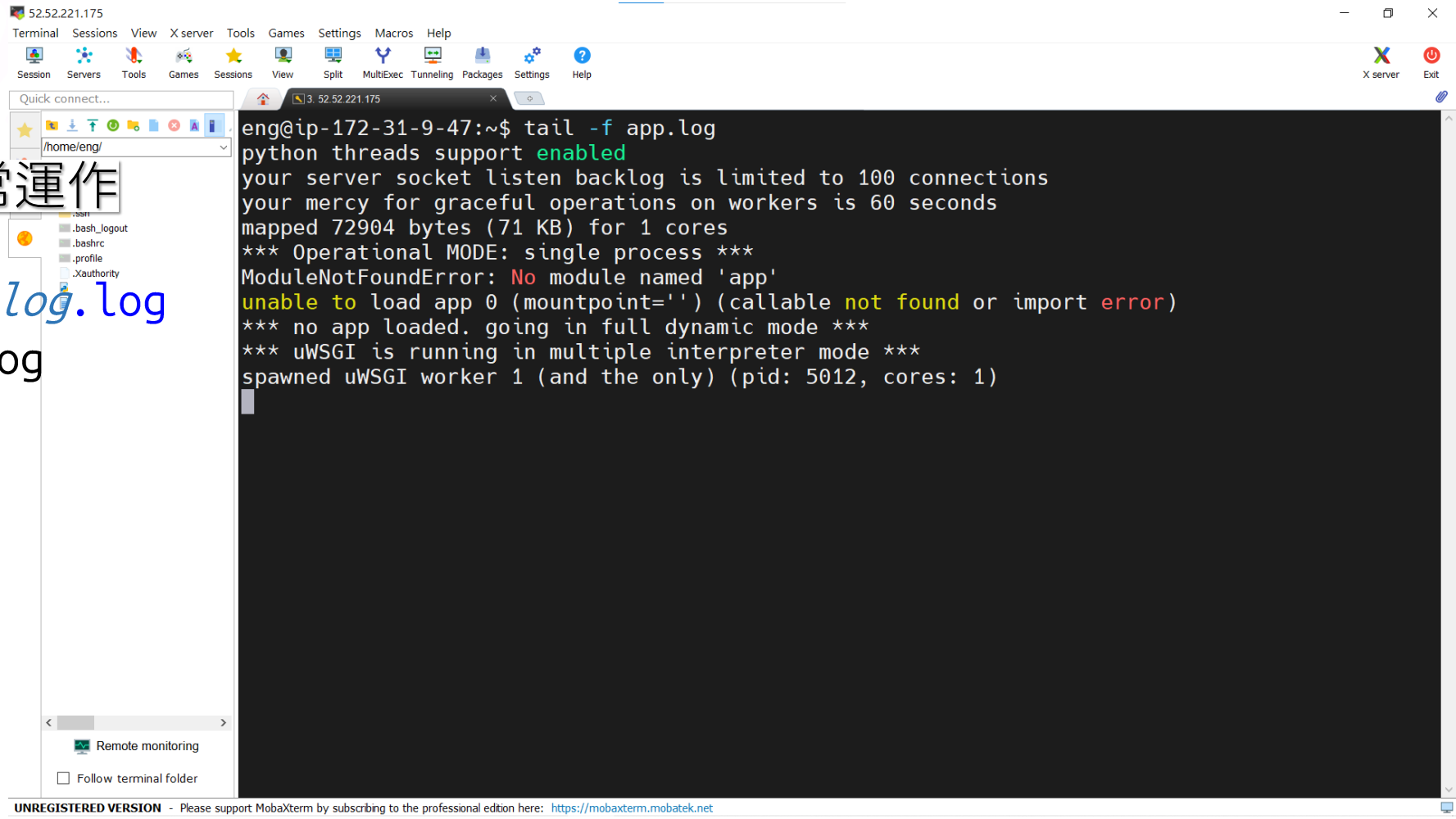
安裝 APP

3. 啟動 APP

c. 確認 APP 正常運作

```
tail -f your_log.log
tail -f app.log
```

異常範例



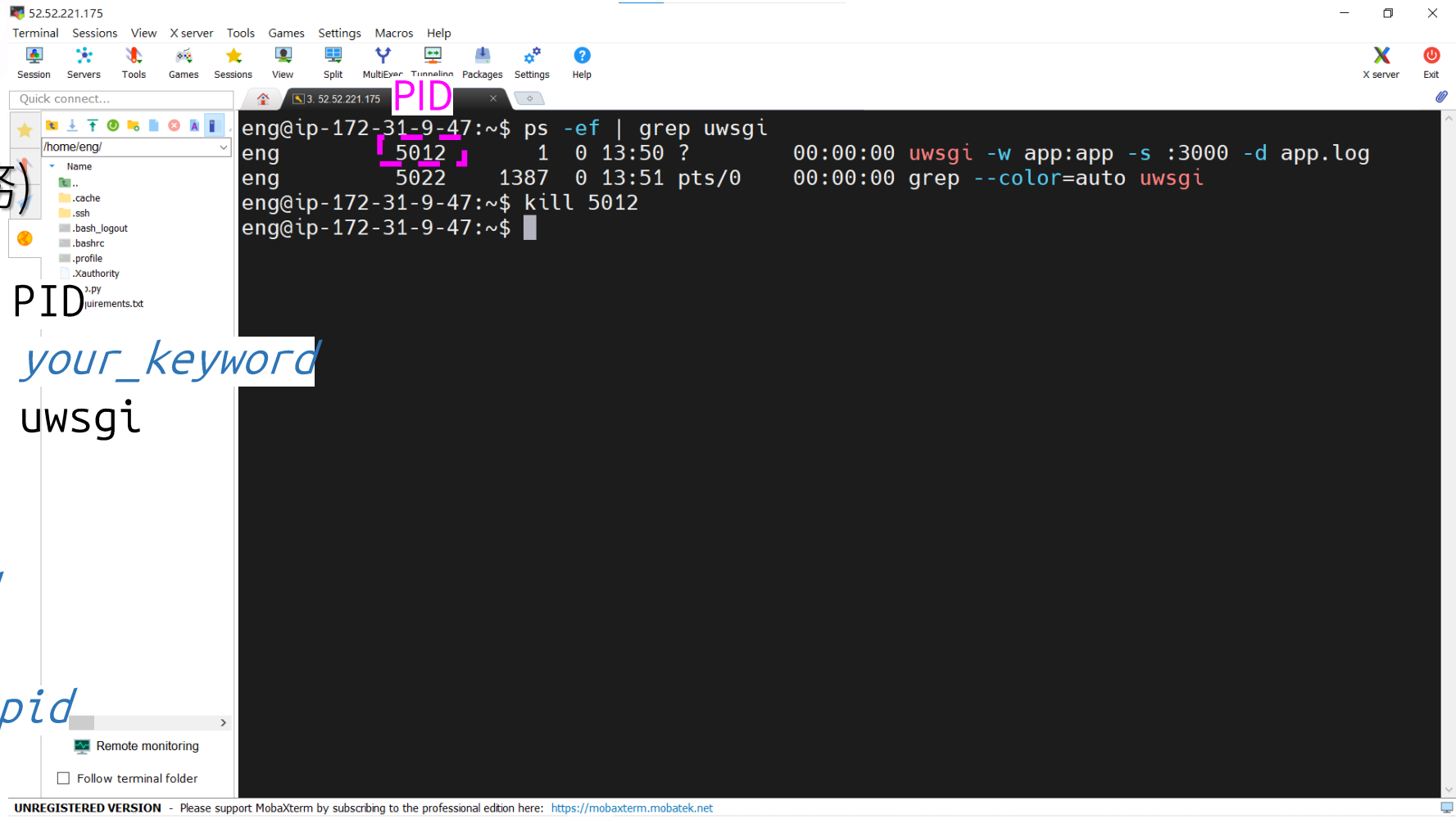
© 2026 Enos Chou

安裝 APP

3. 啟動 APP

d. (日常上下服務)

```
# 取得背景程序 PID
ps -ef | grep your_keyword
ps -ef | grep uwsgi

# 移除背景程序
kill your_pid

kill -9 your_pid
```



安裝 APP

4. (開機自動啟動 APP)

a. 製作服務

① 於 /lib/systemd/system 建立 *your_service*.service

```
[Unit]
After=network.target

[Service]
ExecStart=your_script.sh
RemainAfterExit=yes

[Install]
WantedBy=multi-user.target
```

© 2026 Enos Chou

安裝 APP

4. (開機自動啟動 APP)

a. 製作服務

② 將 *your_service*.service 建立 symbolic link 至 /etc/systemd/system

```
sudo ln -s /lib/systemd/system/your_service.service
/etc/systemd/system/your_service.service
sudo ln -s /lib/systemd/system/app.service
/etc/systemd/system/app.service
```

© 2026 Enos Chou

安裝 APP

4. (開機自動啟動 APP)

b. 製作啟動程式

① 於專案目錄建立啟動程式 *your_script*.sh

```
#!/bin/bash
export PATH=/home/your_account/.local/bin:$PATH
cd /home/your_account

uwsgi -w your_module:app -s :your_port --uid your_account -d your_log.log --logfile-chown

exit 0
```

© 2026 Enos Chou

安裝 APP

4. (開機自動啟動 APP)

b. 製作啟動程式

② 調整啟動程式權限為可執行

```
chmod 755 your_script.sh
chmod 755 app.service.sh
```

© 2026 Enos Chou

安裝 APP

4. (開機自動啟動 APP)

c. 設定自動啟動服務

① 建立自動啟動服務

```
sudo systemctl enable your_service
sudo systemctl enable app
```

② 啟動服務

```
sudo systemctl start your_service
sudo systemctl start app
```

© 2026 Enos Chou

安裝 APP

4. (開機自動啟動 APP)

c. 設定自動啟動服務

③ 查詢服務狀態

```
sudo systemctl status your_service
sudo systemctl status app
```

④ 列出所有服務

```
systemctl
```

⑤ 重新載入服務設定

```
sudo systemctl daemon-reload
```

© 2026 Enos Chou

安裝 APP

4. (開機自動啟動 APP)

d. 取消自動啟動服務

① 停止服務

```
sudo systemctl stop your_service
sudo systemctl stop app
```

② 移除自動啟動服務

```
sudo systemctl disable your_service
sudo systemctl disable app
```

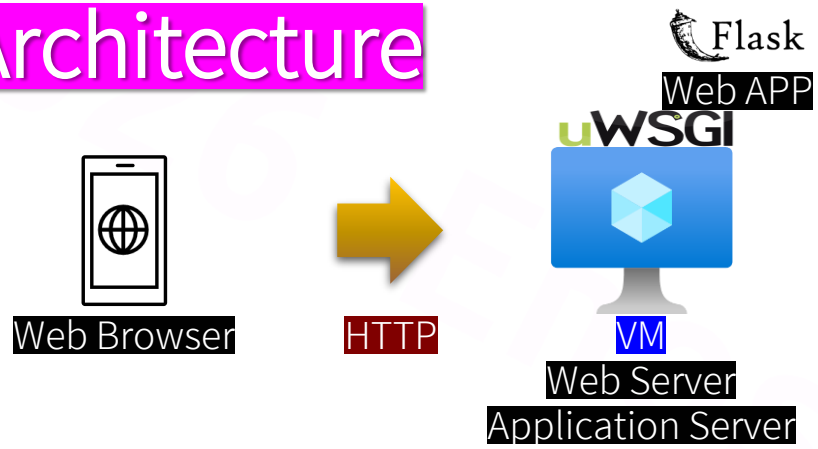
© 2026 Enos Chou

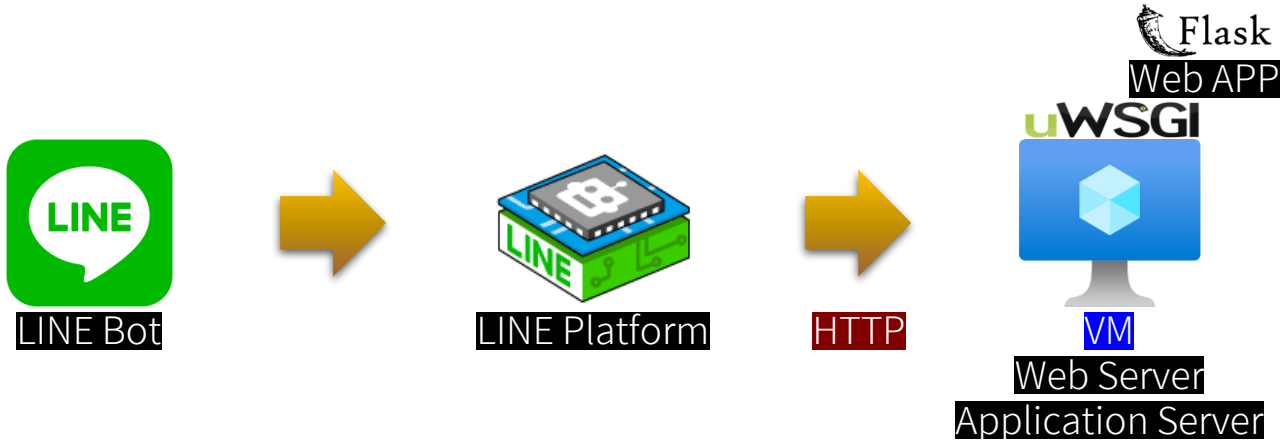
1. Domain Name
2. Web Server
3. Certificate

© 2026 Enos Chou

WSGI

Reasonable Architecture

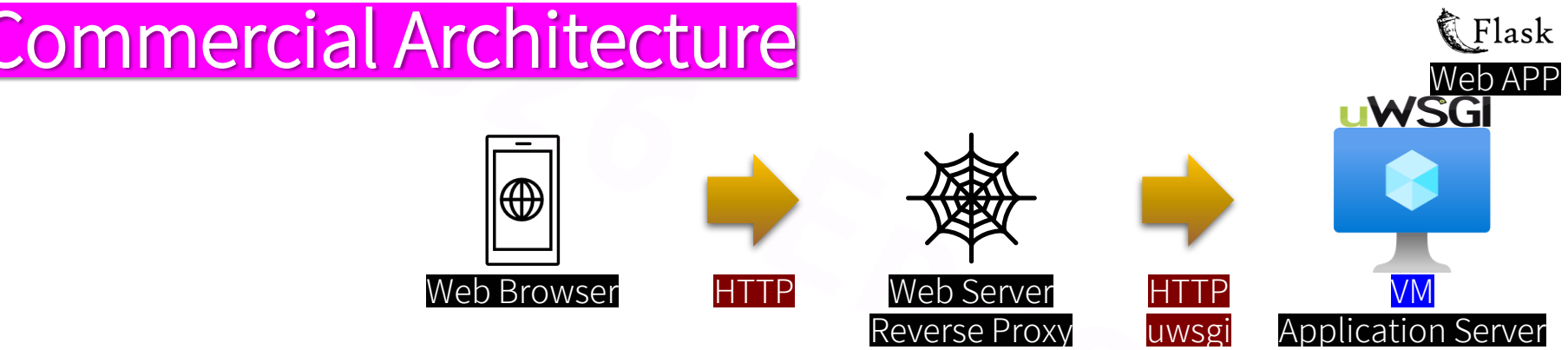




© 2026 Enos Chou

WSGI

Commercial Architecture



網域名稱

何謂網域？

a. 網域為可讀且符合格式的字串，對應特定 IP Address
b. 主網域擁用有者可設定多組子網域

中央氣象局

	IP Address	俗稱
cwa.gov.tw		主網域
www.cwa.gov.tw	168.95.246.2	子網域
opendata.cwa.gov.tw	61.56.14.16	子網域
opendata.cwa.gov.tw/dataset		子目錄

© 2026 Enos Chou

安裝 Web Server

1. 設定網域名稱

   a. (購買主網域)

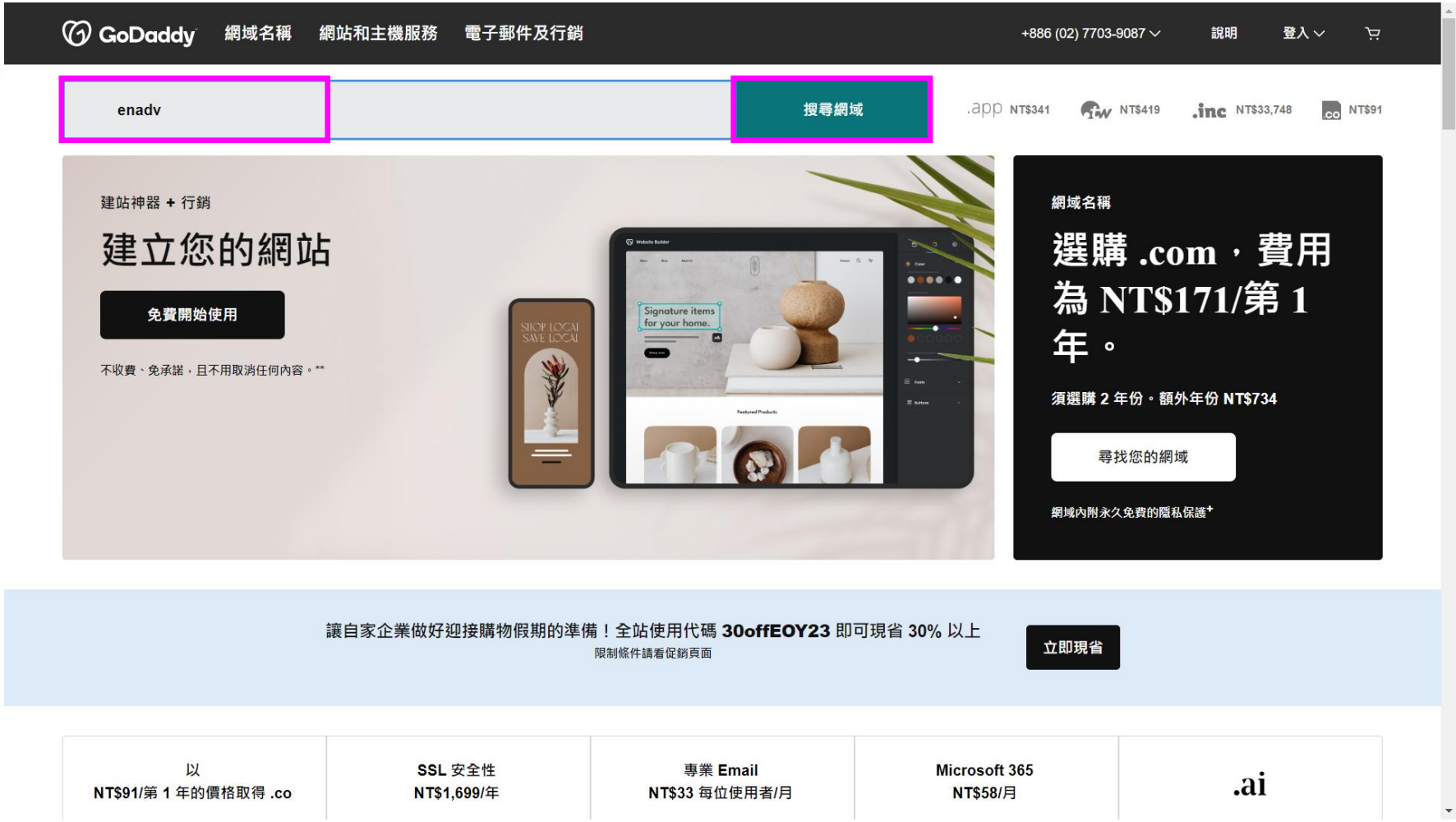
      ① https://tw.godaddy.com/

安裝 Web Server

1. 設定網域名稱

a. (購買主網域)

② 瀏覽主網域



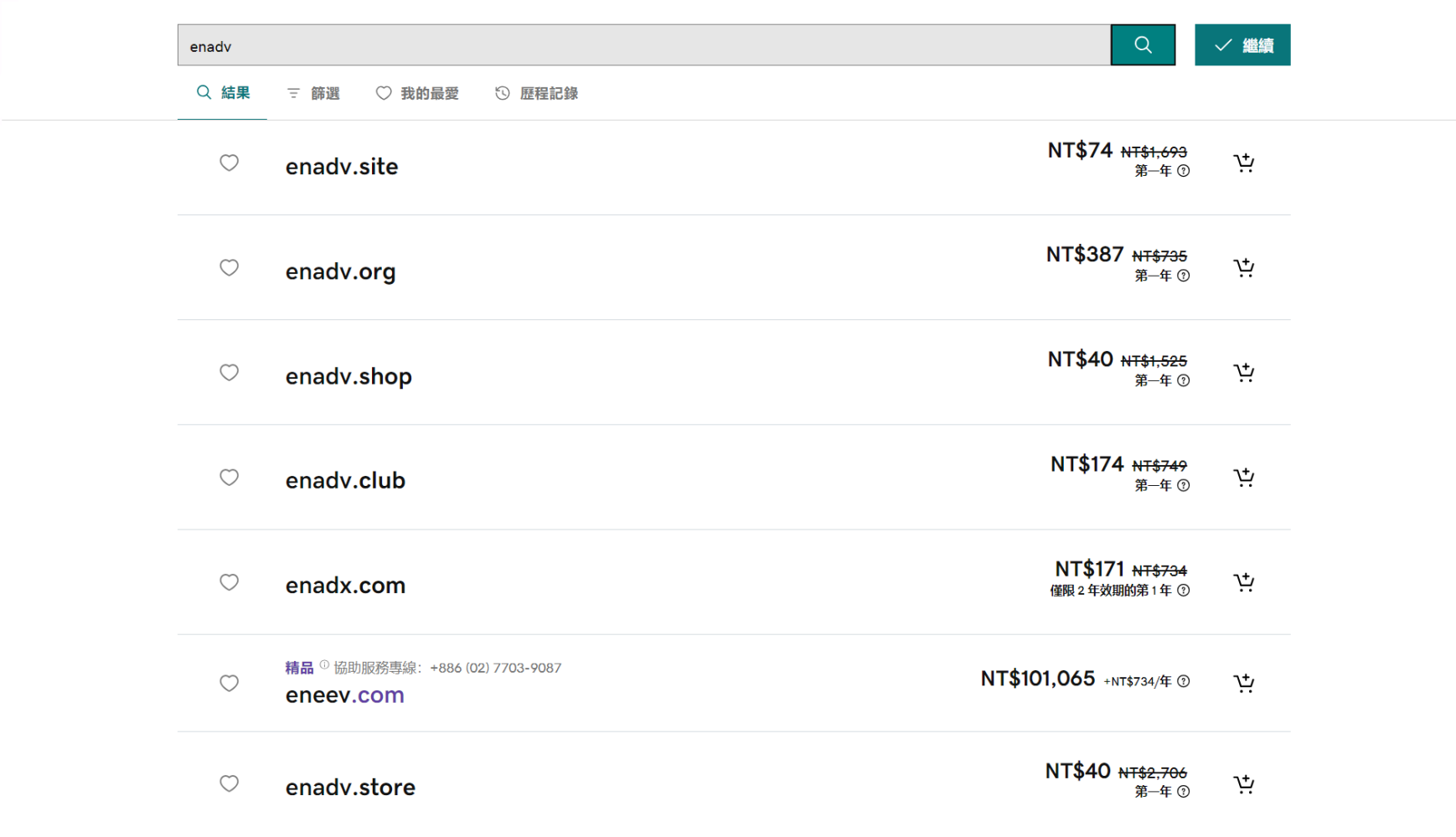
© 2026 Enos Chou

安裝 Web Server

1. 設定網域名稱

a. (購買主網域)

② 瀏覽主網域



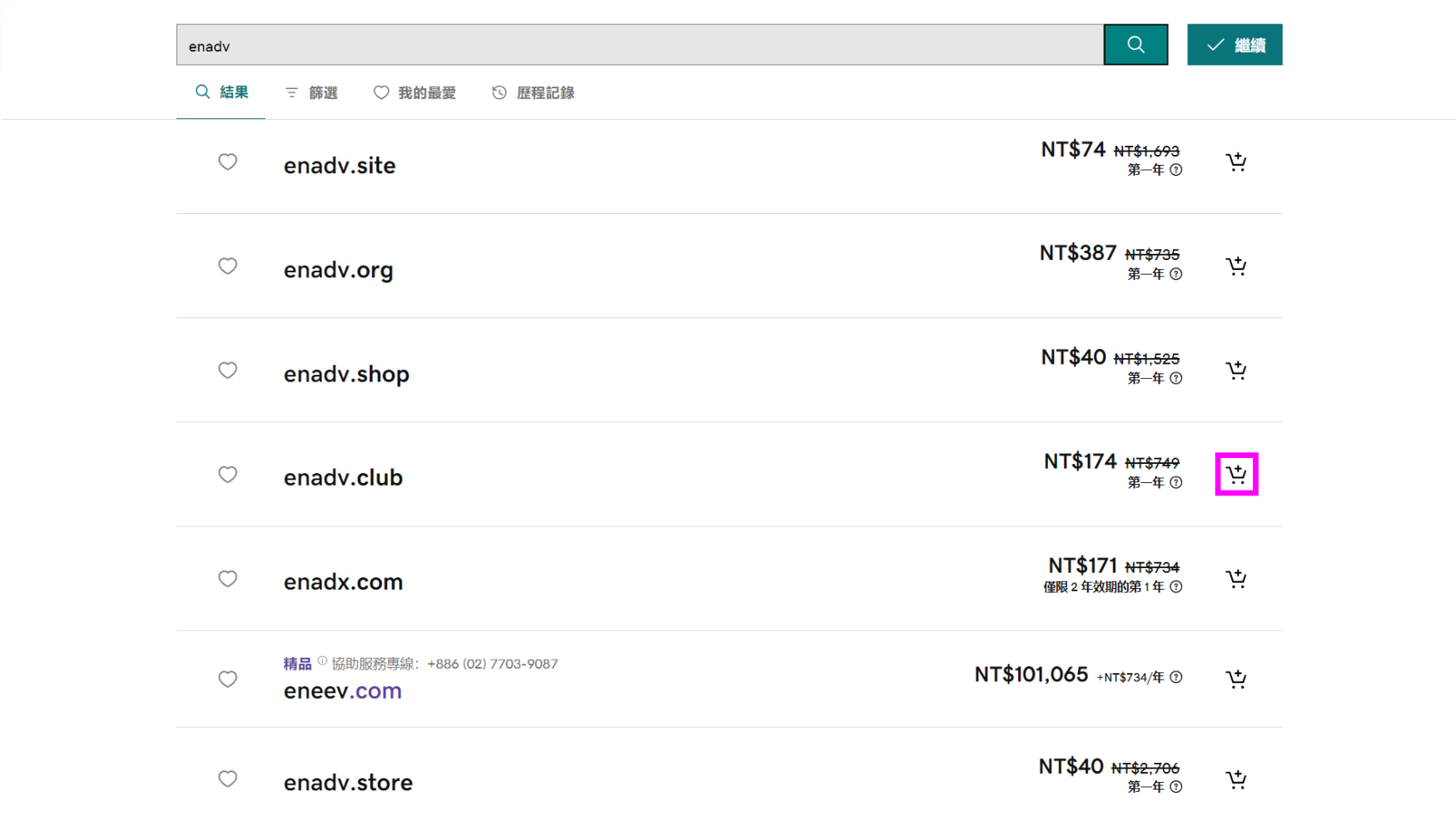
© 2026 Enos Chou

安裝 Web Server

1. 設定網域名稱

   a. (購買主網域)

      ③ 購買主網域



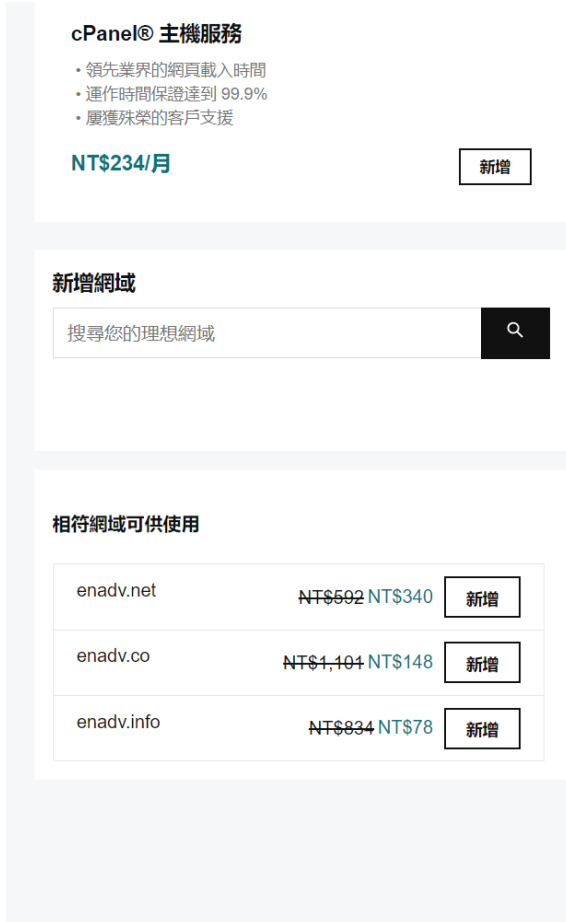
© 2026 Enos Chou

安裝 Web Server

1. 設定網域名稱

a. (購買主網域)

③ 購買主網域



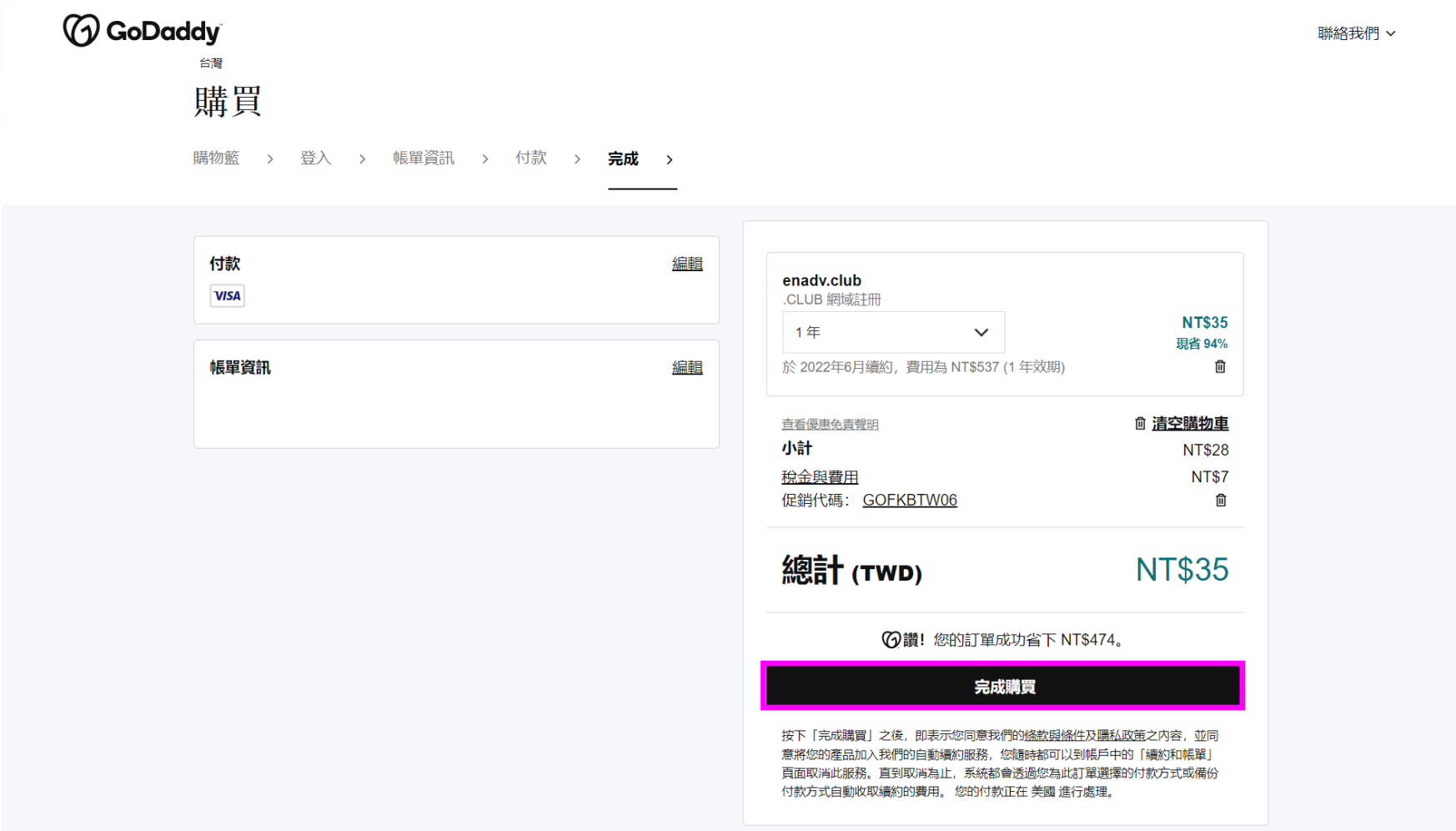
Copyright © 1999 - 2021 GoDaddy Operating Company, LLC.保留所有權利。 隱私權政策

安裝 Web Server

1. 設定網域名稱

a. (購買主網域)

③ 購買主網域

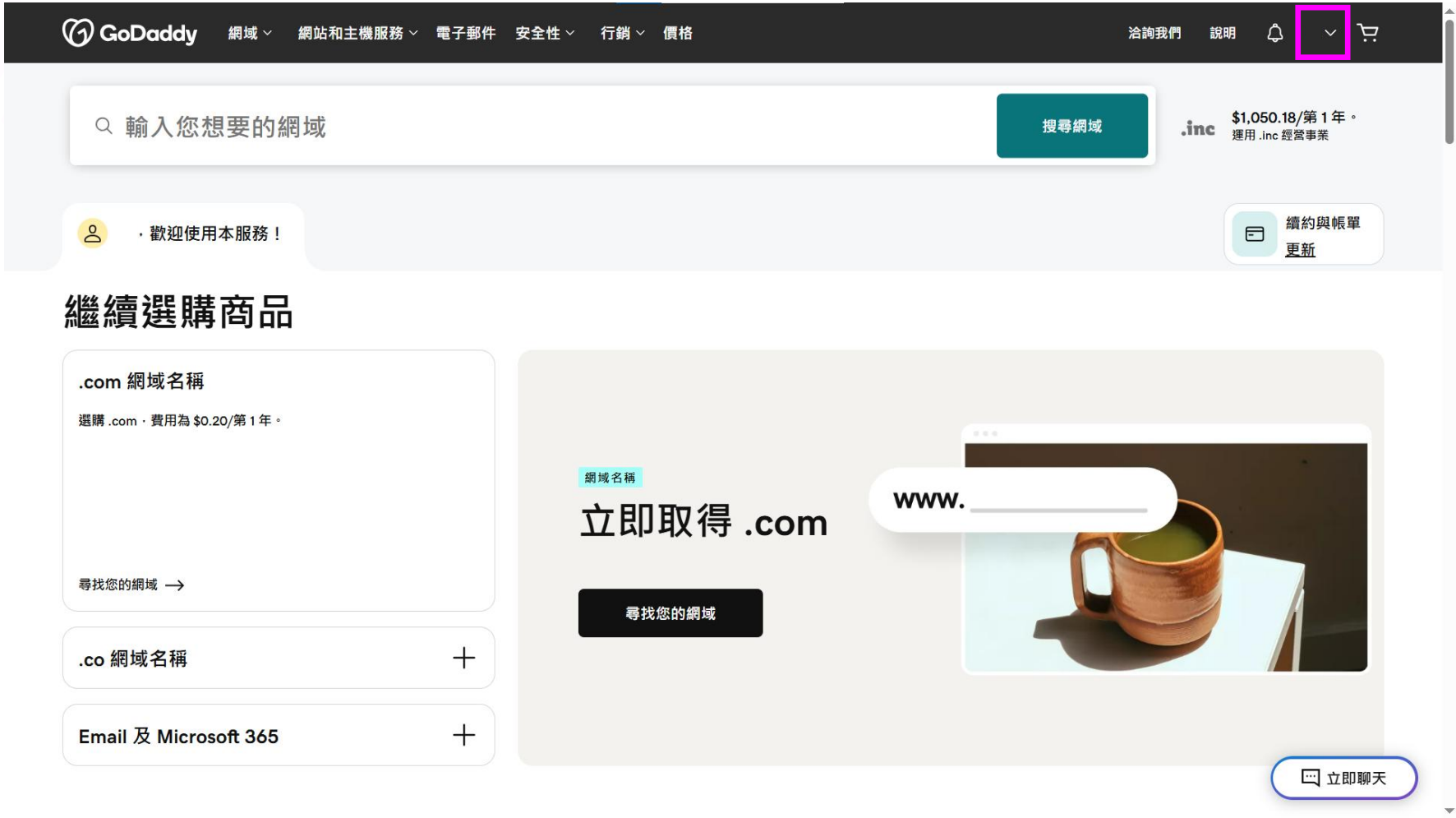


安裝 Web Server

1. 設定網域名稱

b. 設定子網域

① 進入 DNS

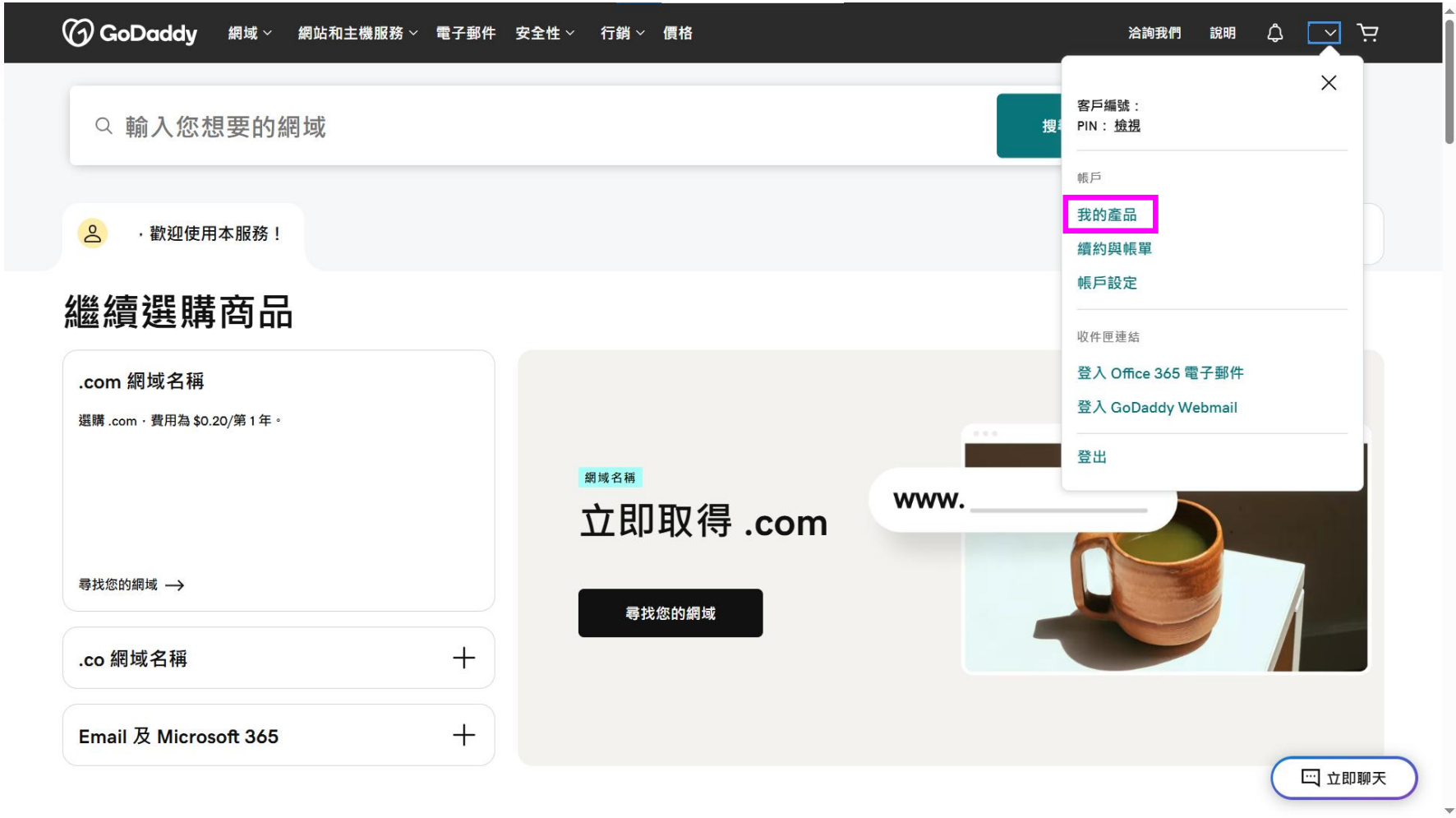


安裝 Web Server

1. 設定網域名稱

   b. 設定子網域

      ① 進入 DNS



安裝 Web Server

1. 設定網域名稱

b. 設定子網域

① 進入 DNS

安裝 Web Server

1. 設定網域名稱

b. 設定子網域

① 進入 DNS

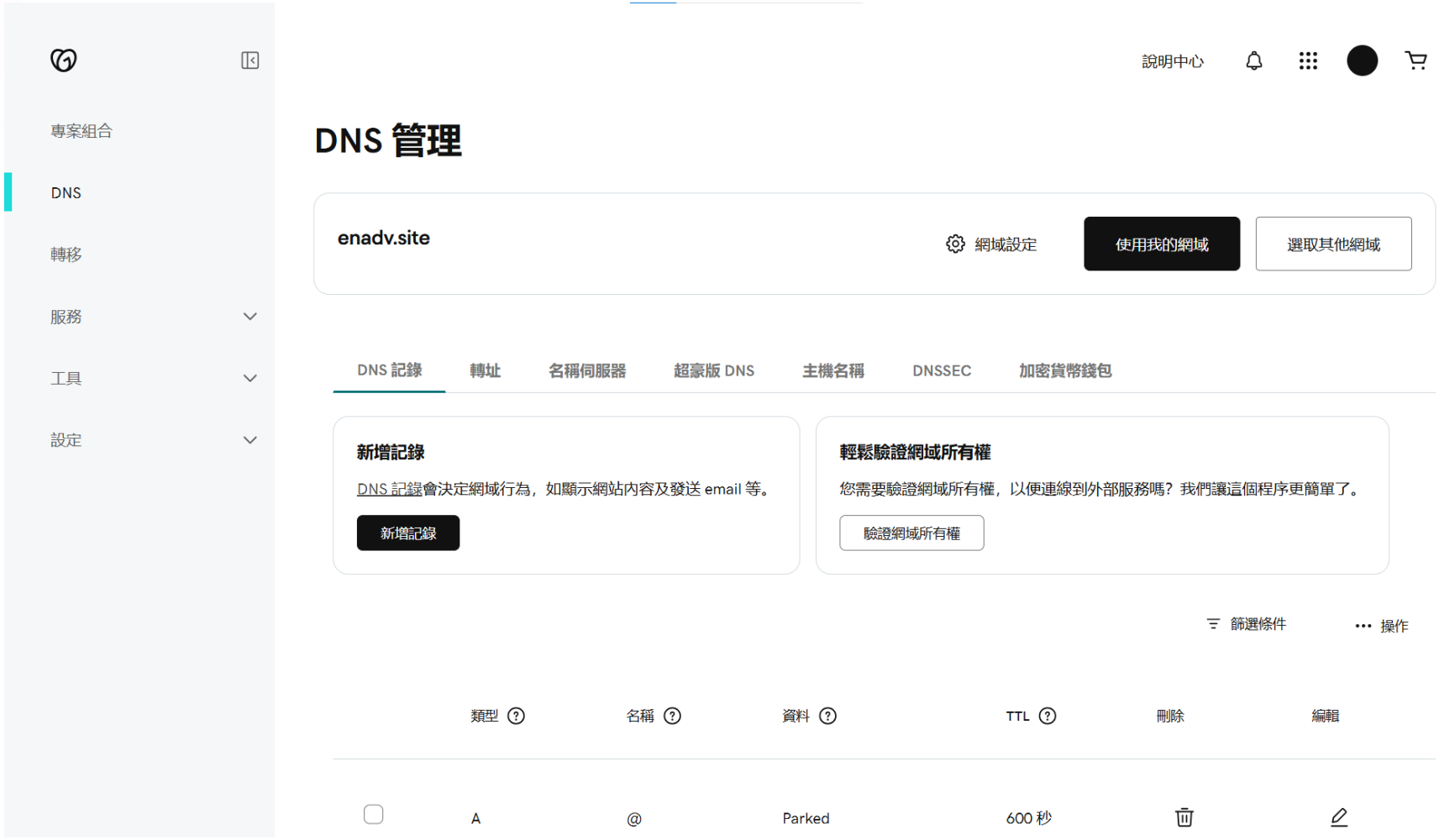
Copyright © 1999 - 2025 GoDaddy Operating Company, LLC.保留所有權利。 隱私權政策
立即聊天

安裝 Web Server

1. 設定網域名稱

b. 設定子網域

① 進入 DNS

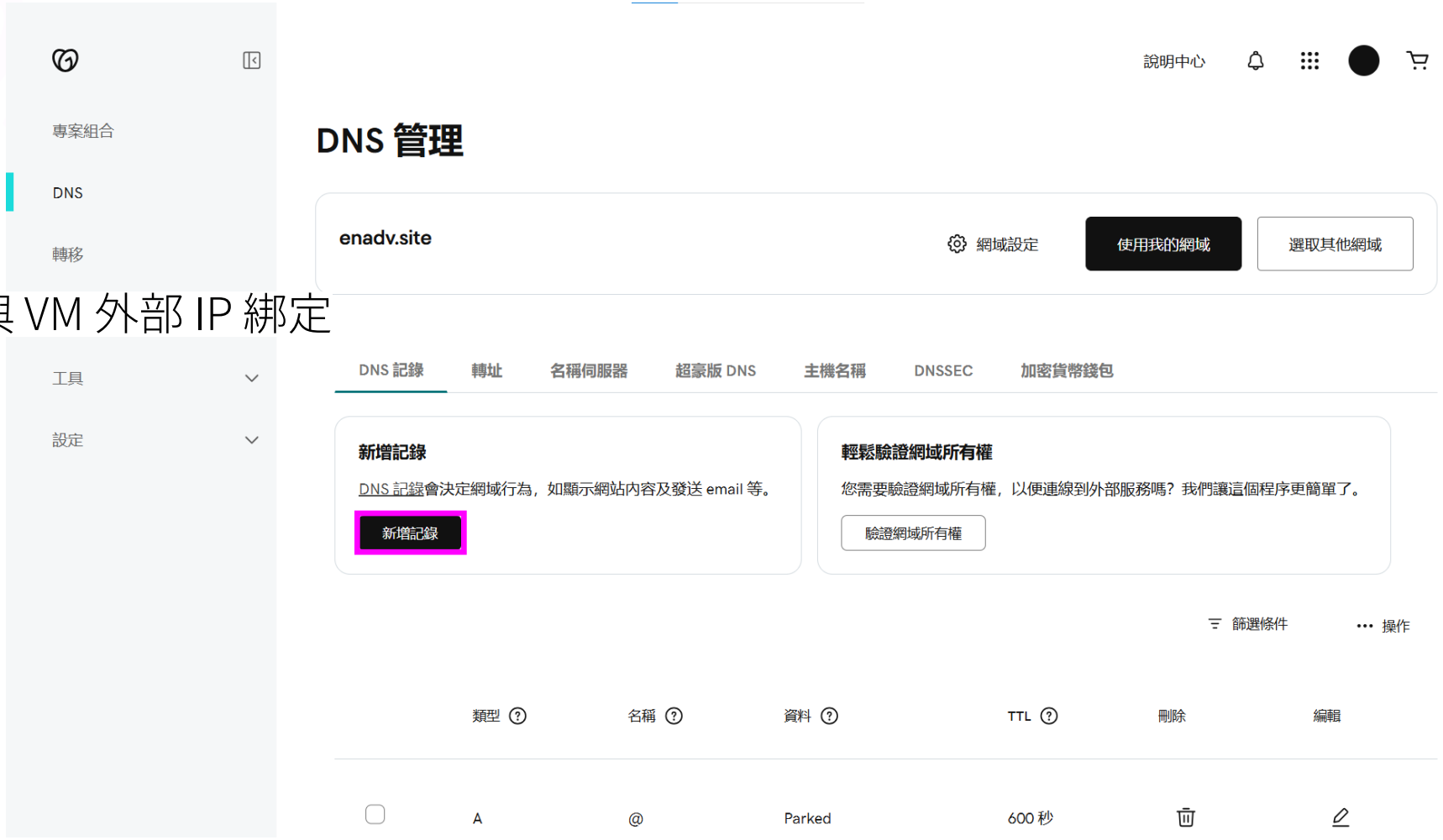


安裝 Web Server

1. 設定網域名稱

b. 設定子網域

② 設定子網域與 VM 外部 IP 綁定

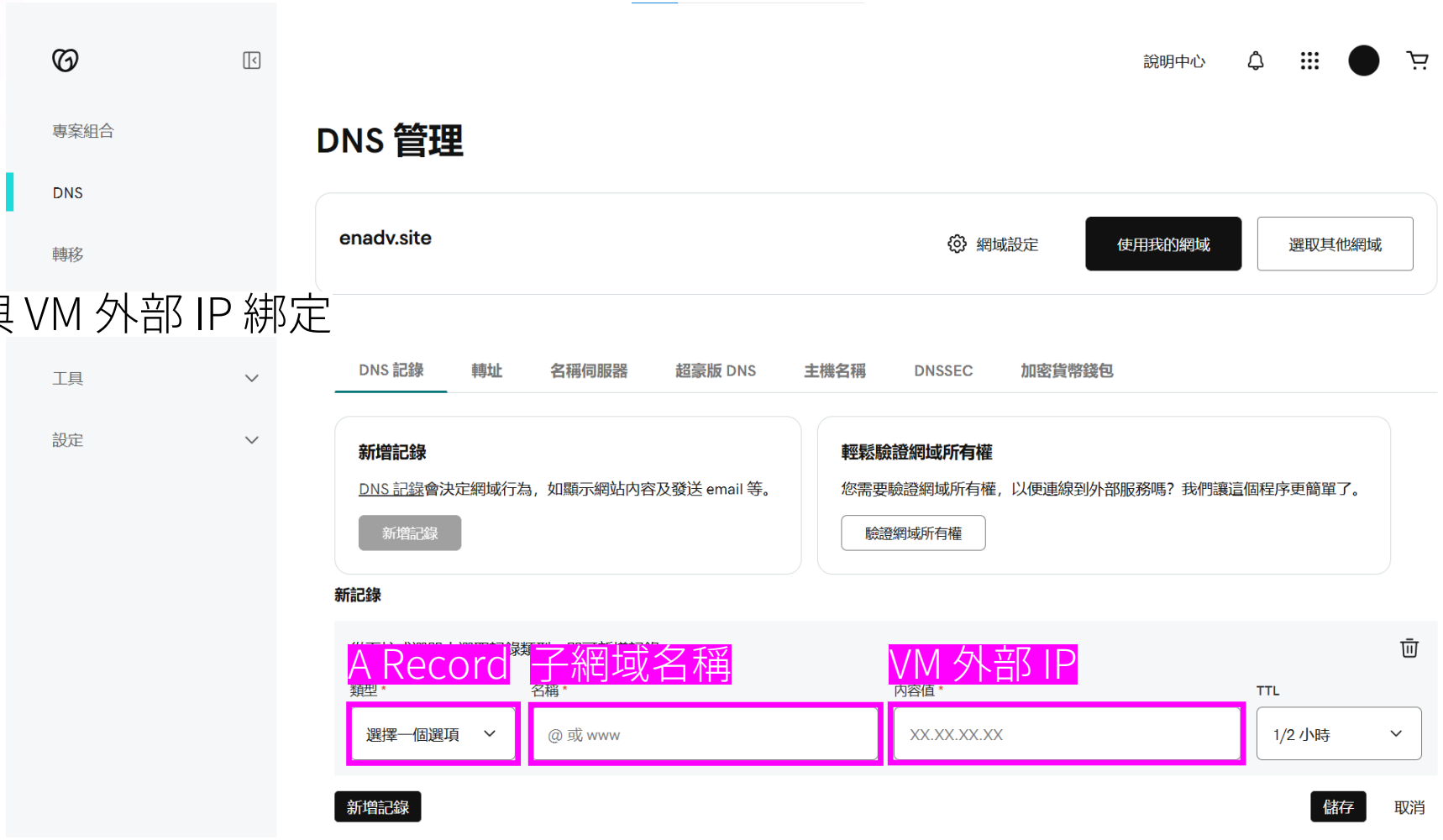


安裝 Web Server

1. 設定網域名稱

b. 設定子網域

② 設定子網域與 VM 外部 IP 綁定



安裝 Web Server

1. 設定網域名稱

b. 設定子網域

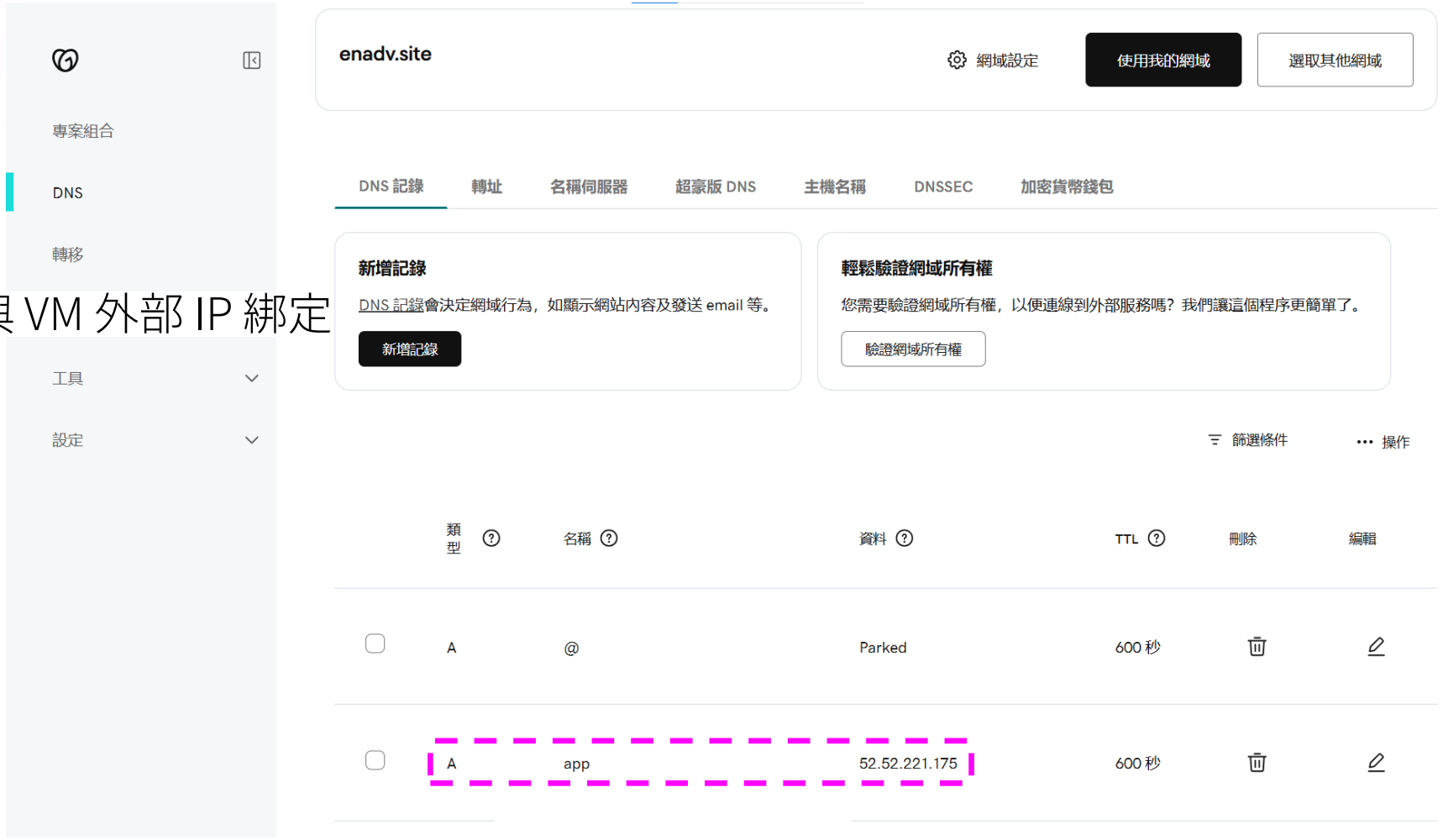
② 設定子網域與 VM 外部 IP 綁定

安裝 Web Server

1. 設定網域名稱

b. 設定子網域

② 設定子網域與 VM 外部 IP 綁定



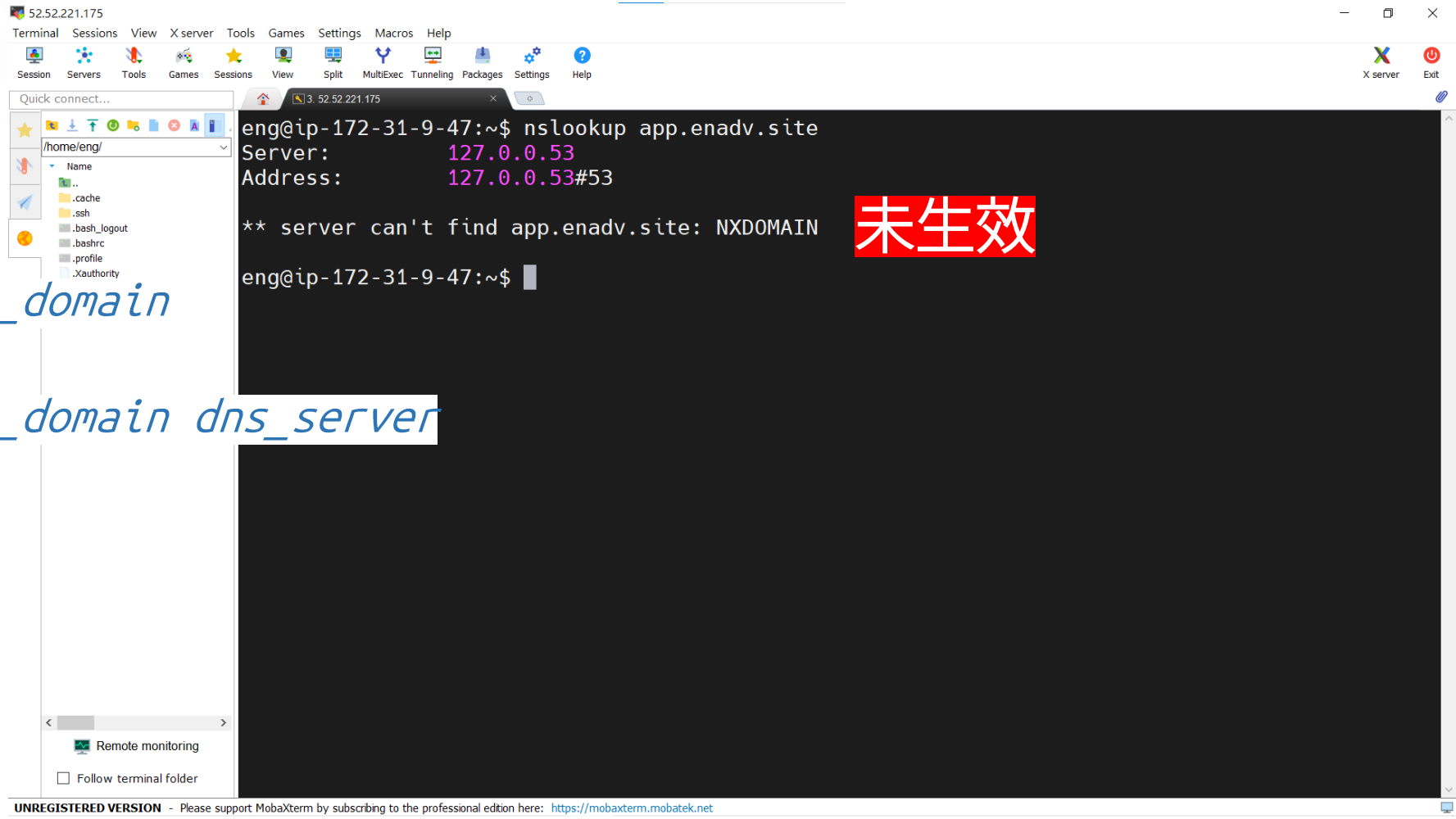
安裝 Web Server

1. 設定網域名稱

c. 確認子網域

```
nslookup your_domain
```

```
nslookup your_domain dns_server
```



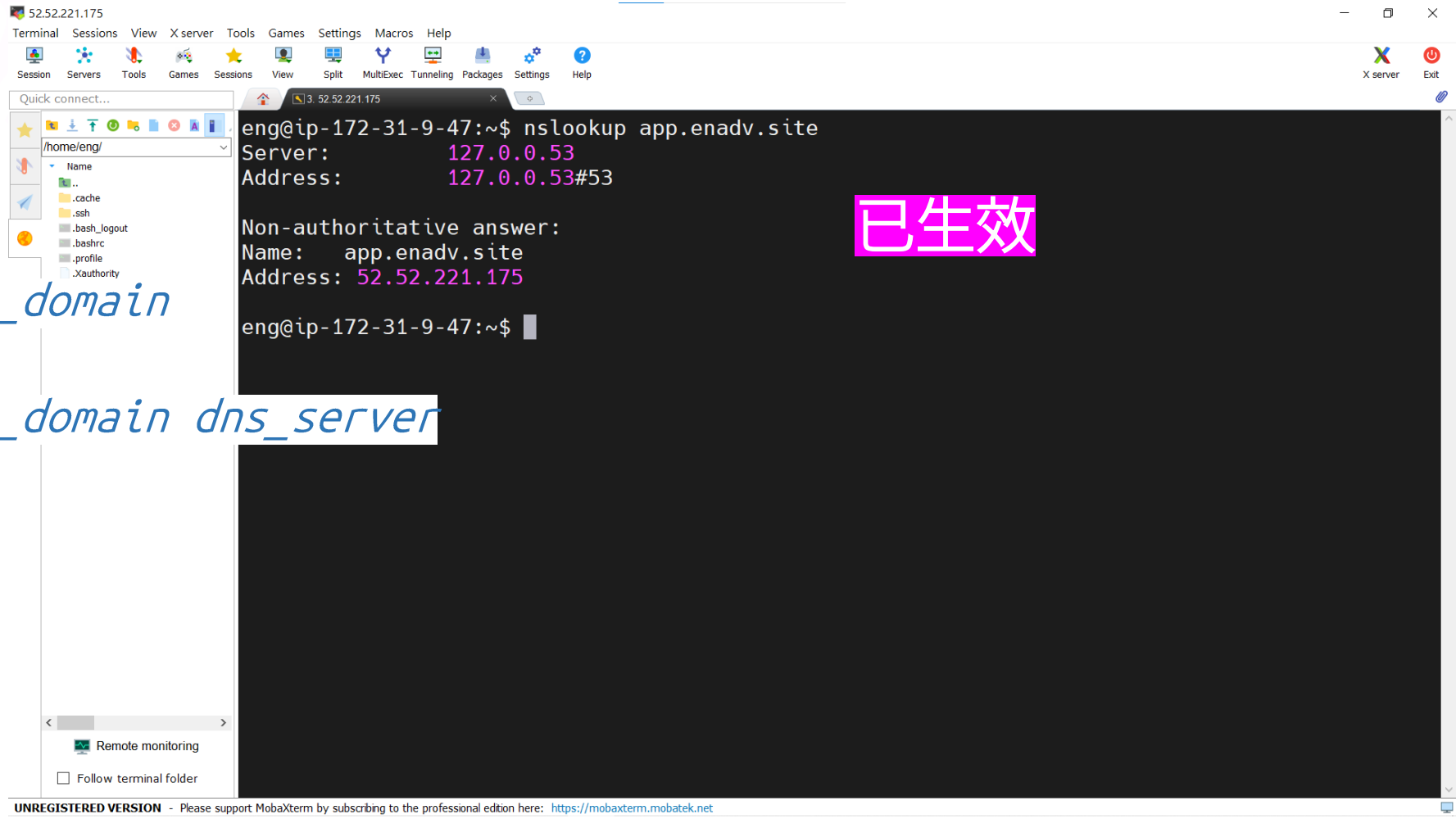
安裝 Web Server

1. 設定網域名稱

   c. 確認子網域

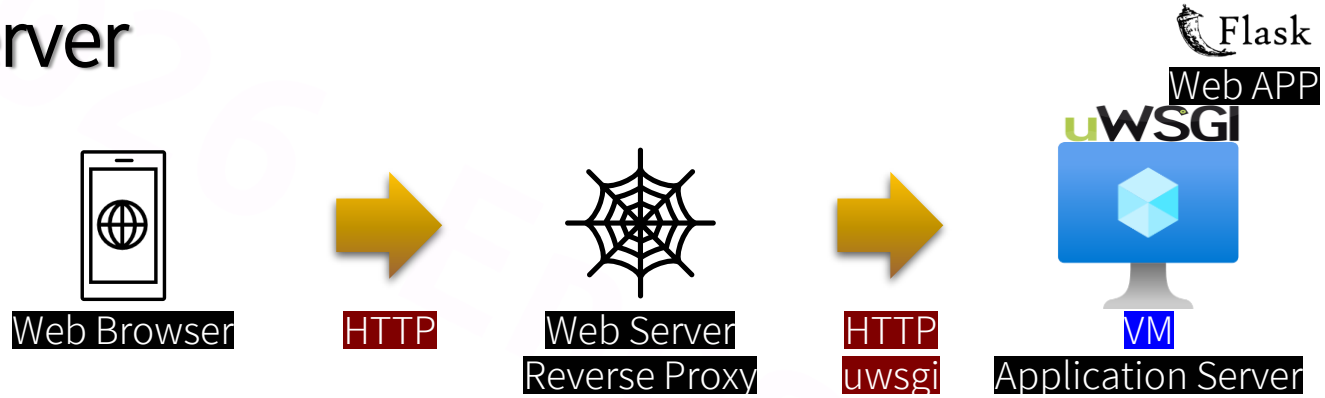
```
nslookup your_domain
```

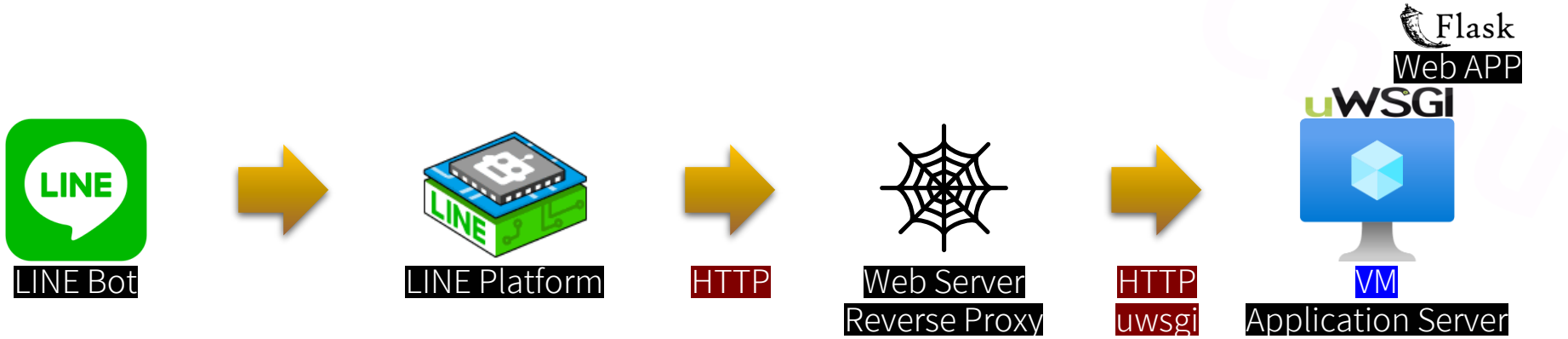
```
nslookup your_domain dns_server
```



安裝 Web Server

2. 安裝 Web Server





© 2026 Enos Chou

安裝 Web Server

2. 安裝 Web Server

Web Server 的選擇

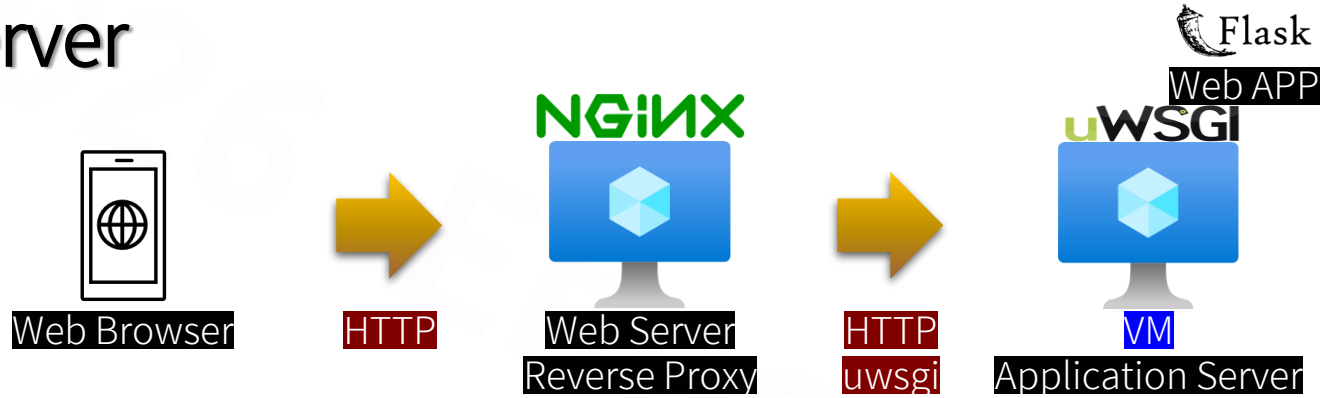
Apache: 經典/ 須另行啟用 reverse proxy/ reverse proxy 支援 HTTP

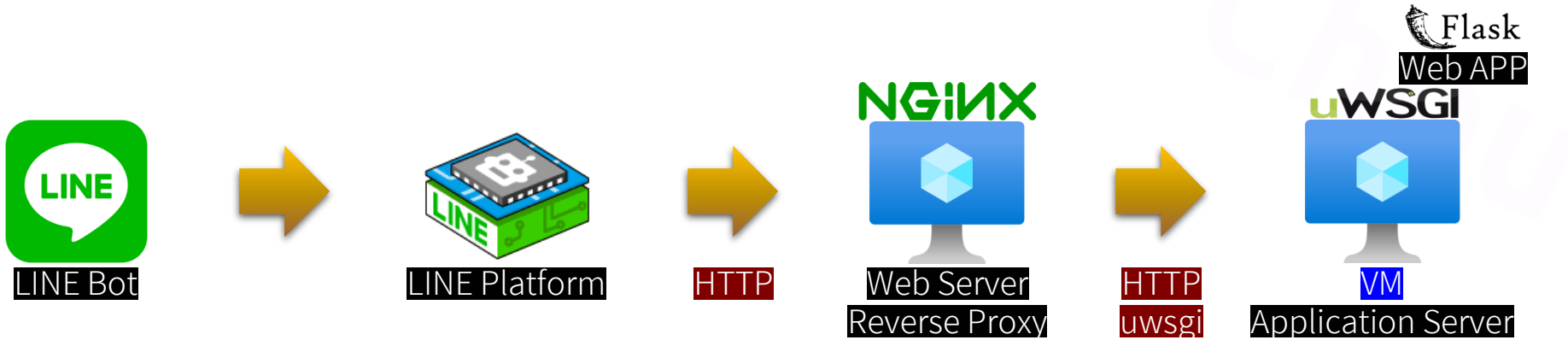
NGINX: 較新/ 內建 reverse proxy/ reverse proxy 支援 HTTP & uwsgi protocol

© 2026 Enos Chou

安裝 Web Server

2. 安裝 Web Server





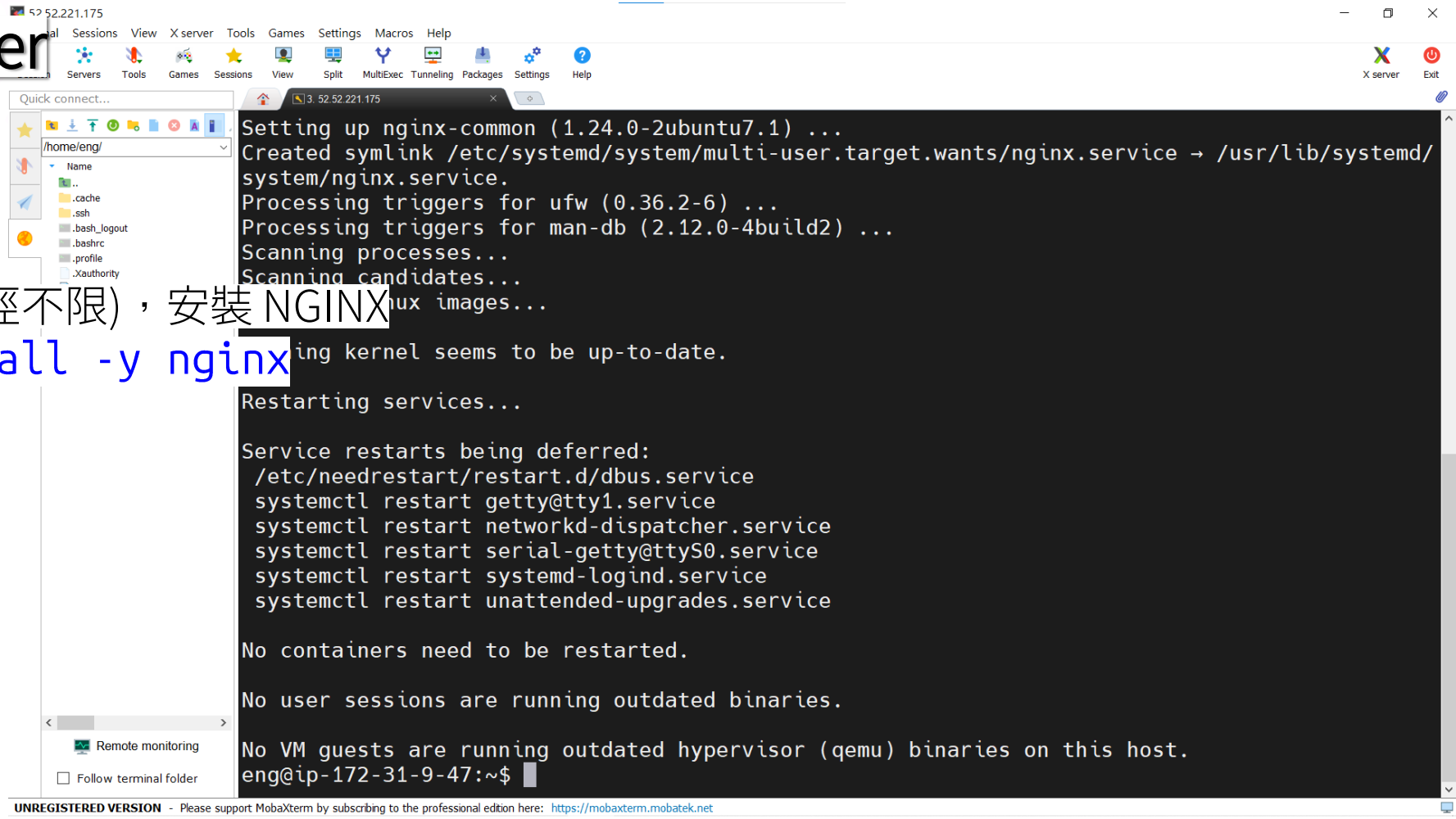
安裝 Web Server

2. 安裝 Web Server

a. 安裝 NGINX

① 登入 VM (路徑不限)，安裝 NGINX

```
sudo apt install -y nginx
```



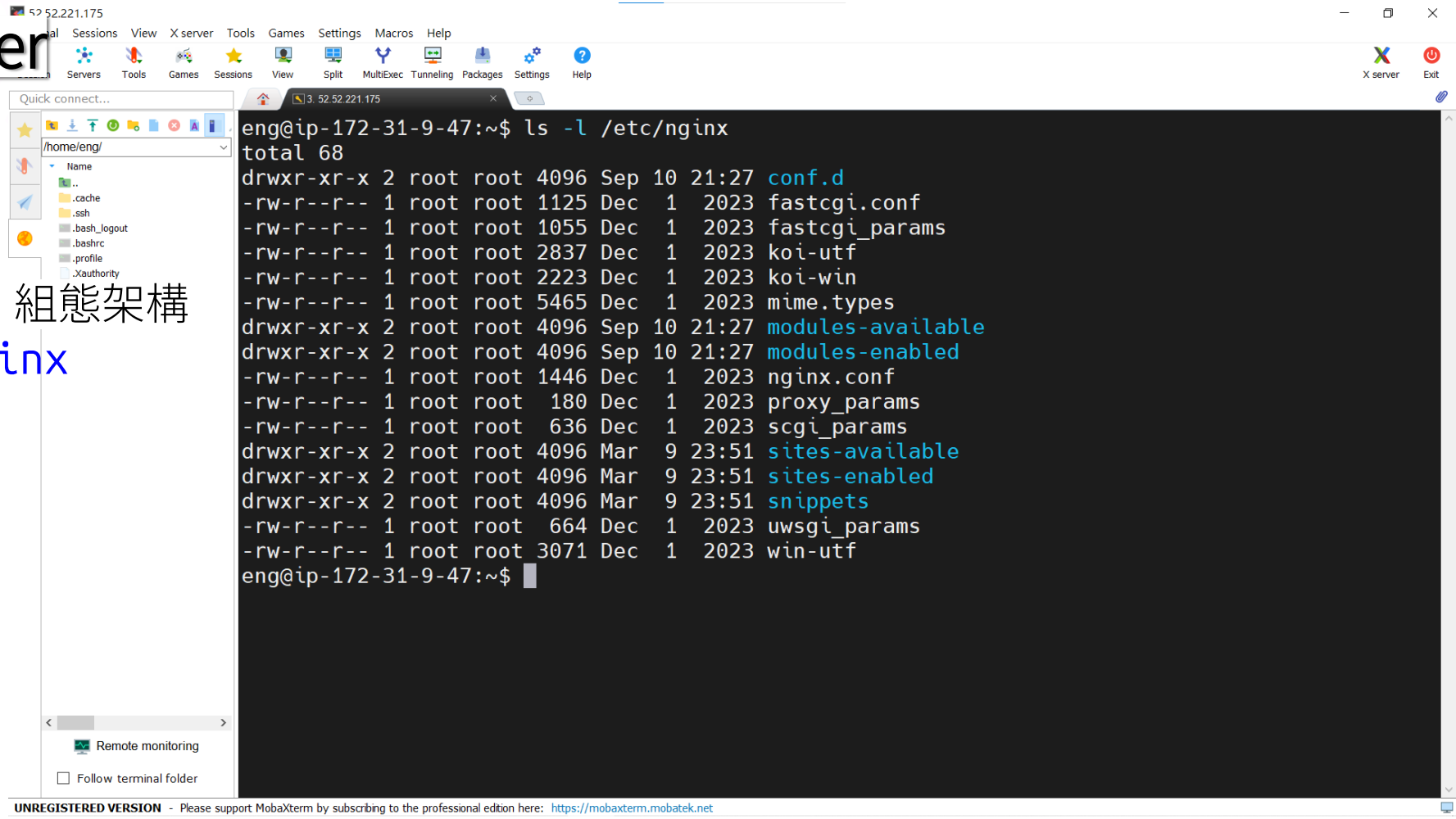
安裝 Web Server

2. 安裝 Web Server

a. 安裝 NGINX

② 觀察 NGINX 組態架構

```
ls -l /etc/nginx
```



安裝 Web Server

2. 安裝 Web Server

a. 安裝 NGINX

② 觀察 NGINX 組態架構

```
ls -l /etc/nginx
```

nginx.conf

NGINX 啟動時預設讀取的組態檔，內容包含掃描 sites-enabled 中所有組態檔

sites-available 目錄

儲存各種啟動組態檔，但未必會被啟動。欲啟動的組態檔應以 symbolic link 方式置於 sites-enabled 目錄

sites-enabled 目錄

實際欲啟動的組態檔

© 2026 Enos Chou

安裝 Web Server

2. 安裝 Web Server

b. 建立組態檔

① 編輯組態檔

```
sudo nano /etc/nginx/sites-available/your_conf.conf
sudo nano /etc/nginx/sites-available/app.conf
```

```
server {
    server_name your_domain;
    location / {
        include uwsgi_params;
        uwsgi_pass your_ip:your_port;
    }
}
```

Note

調整 *your_domain*, *your_ip*, *your_port*

© 2026 Enos Chou

安裝 Web Server

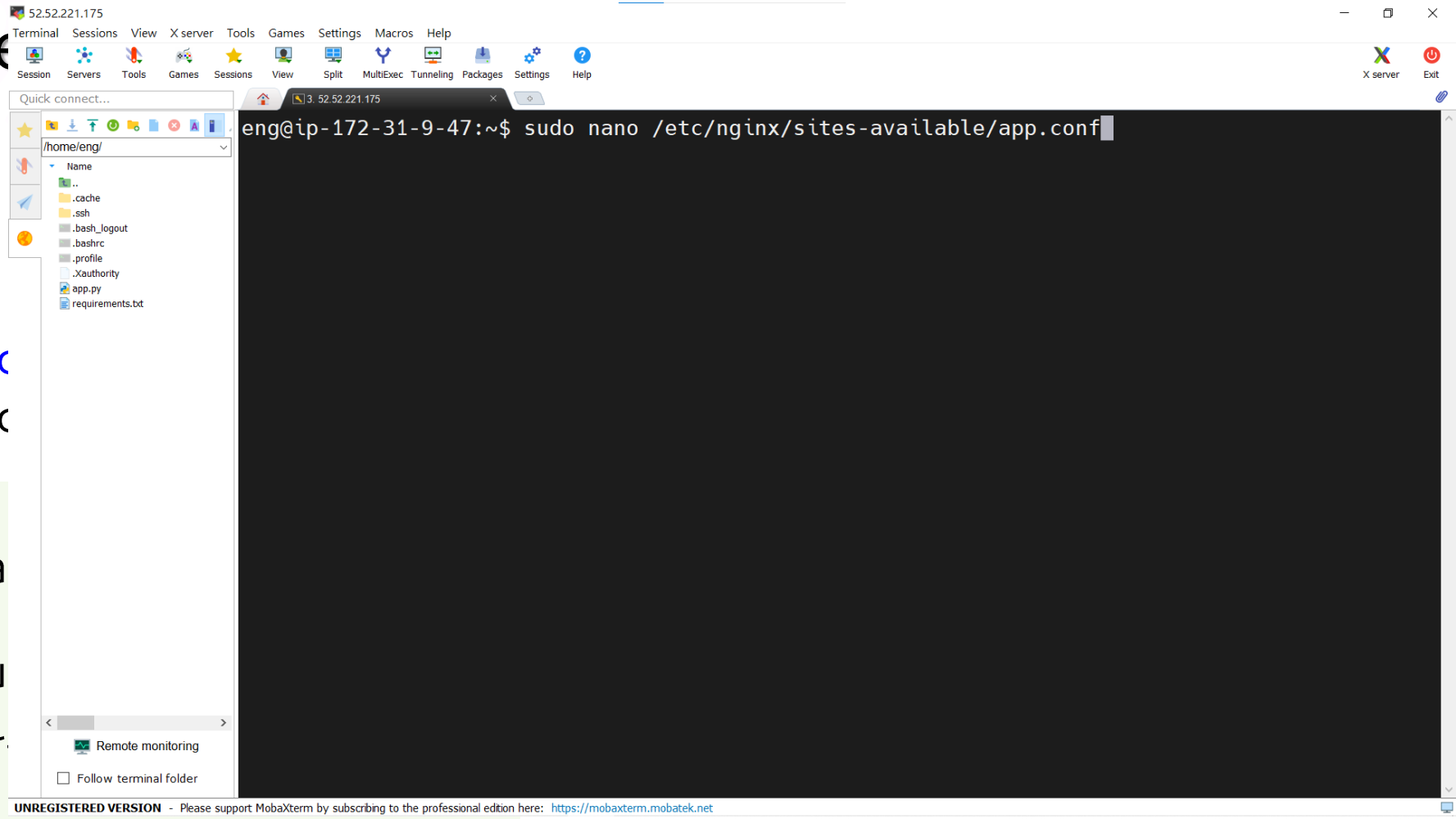
2. 安裝 Web Serve

b. 建立組態檔

① 編輯組態檔

```
sudo nano /etc
sudo nano /etc
```

```
server {
    server_na
    location
        inclu
        uwsgi
    }
}
```



© 2026 Enos Chou

安裝 Web Server

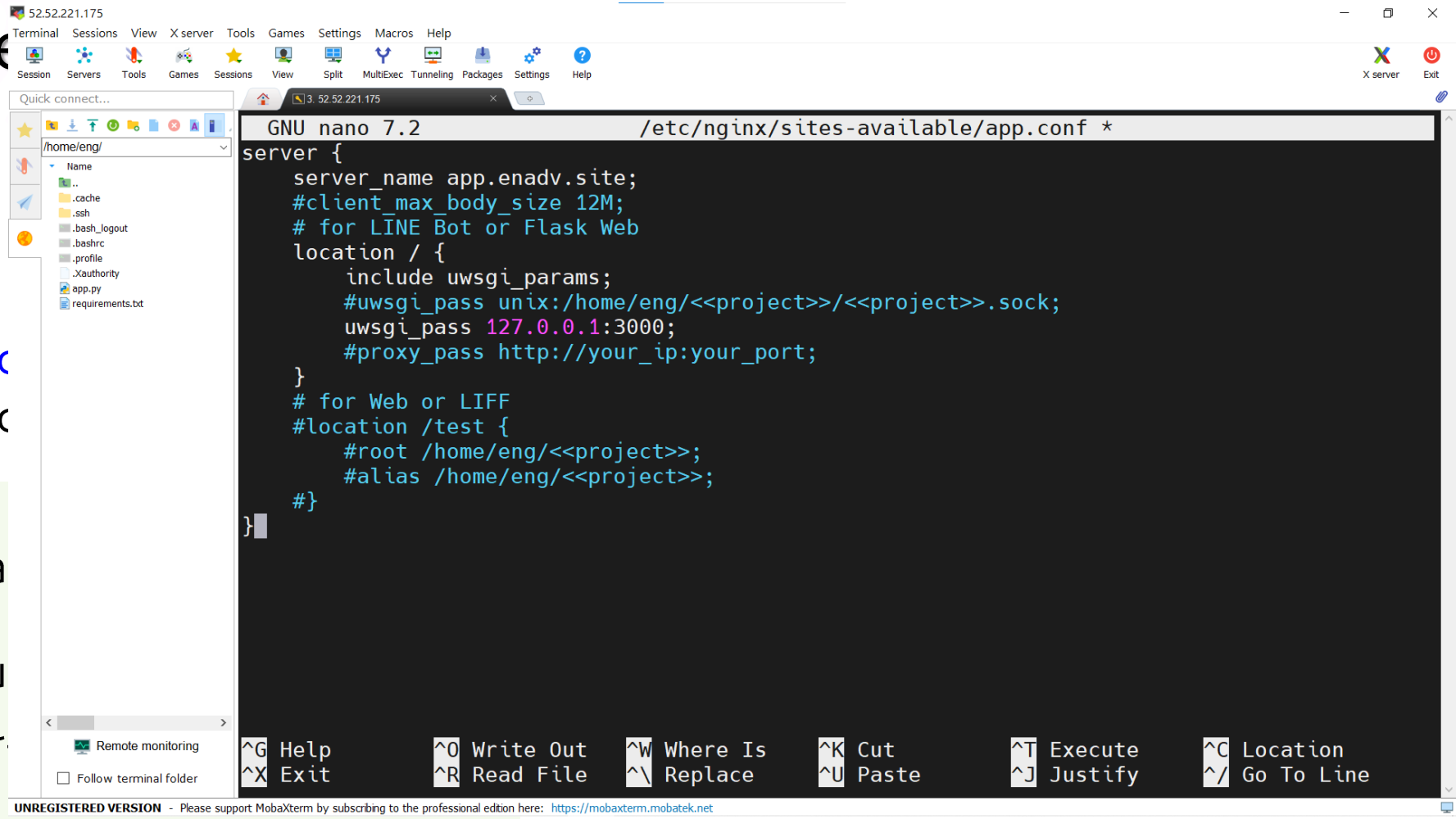
2. 安裝 Web Serve

b. 建立組態檔

① 編輯組態檔

```
sudo nano /etc
sudo nano /etc
```

```
server {
    server_na
    location
        inclu
        uwsgi
    }
}
```



© 2026 Enos Chou

安裝 Web Server

2. 安裝 Web Server

b. 建立組態檔

② 儲存組態檔

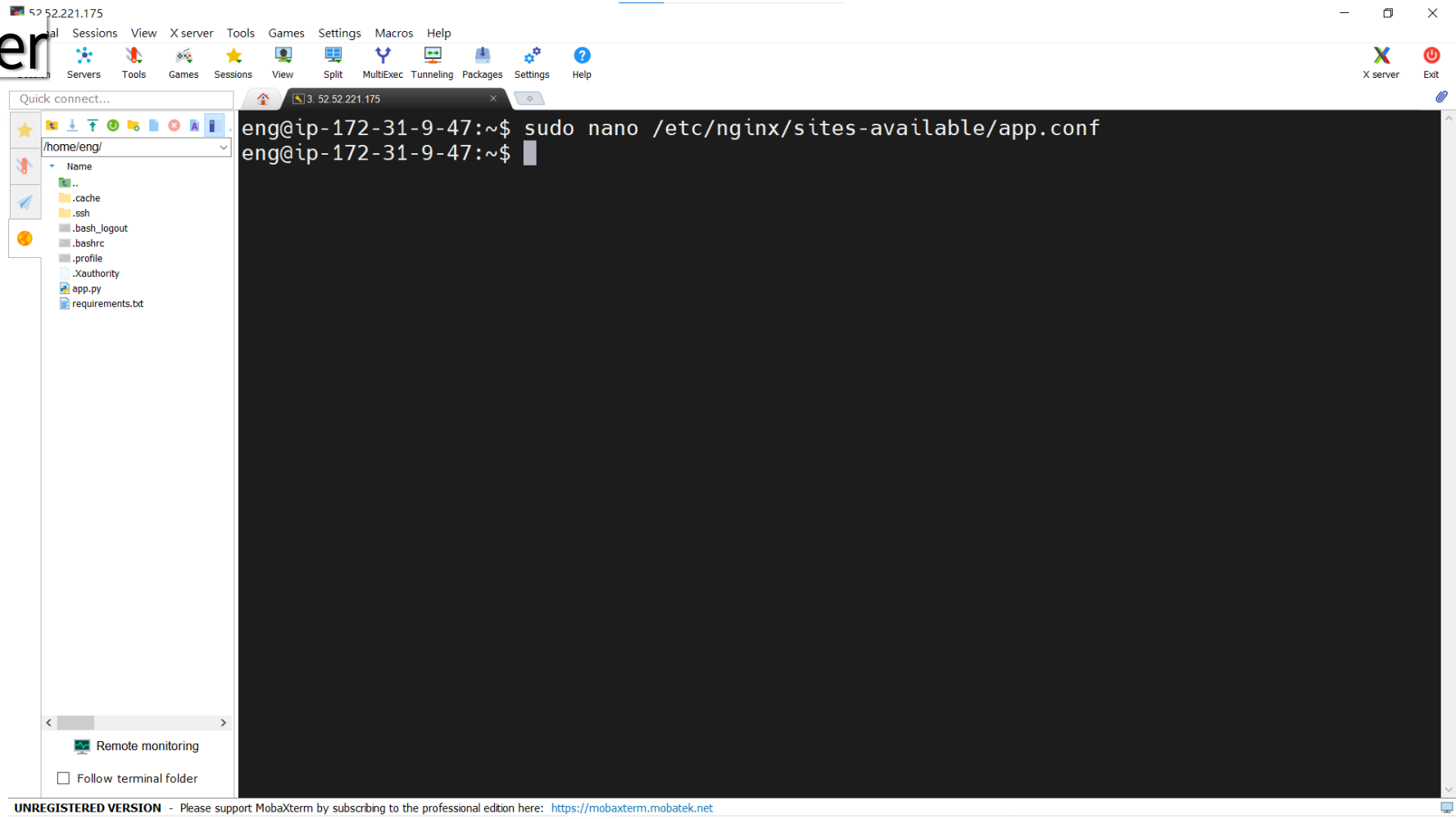
儲存

CTRL+o

ENTER

退出

CTRL+x



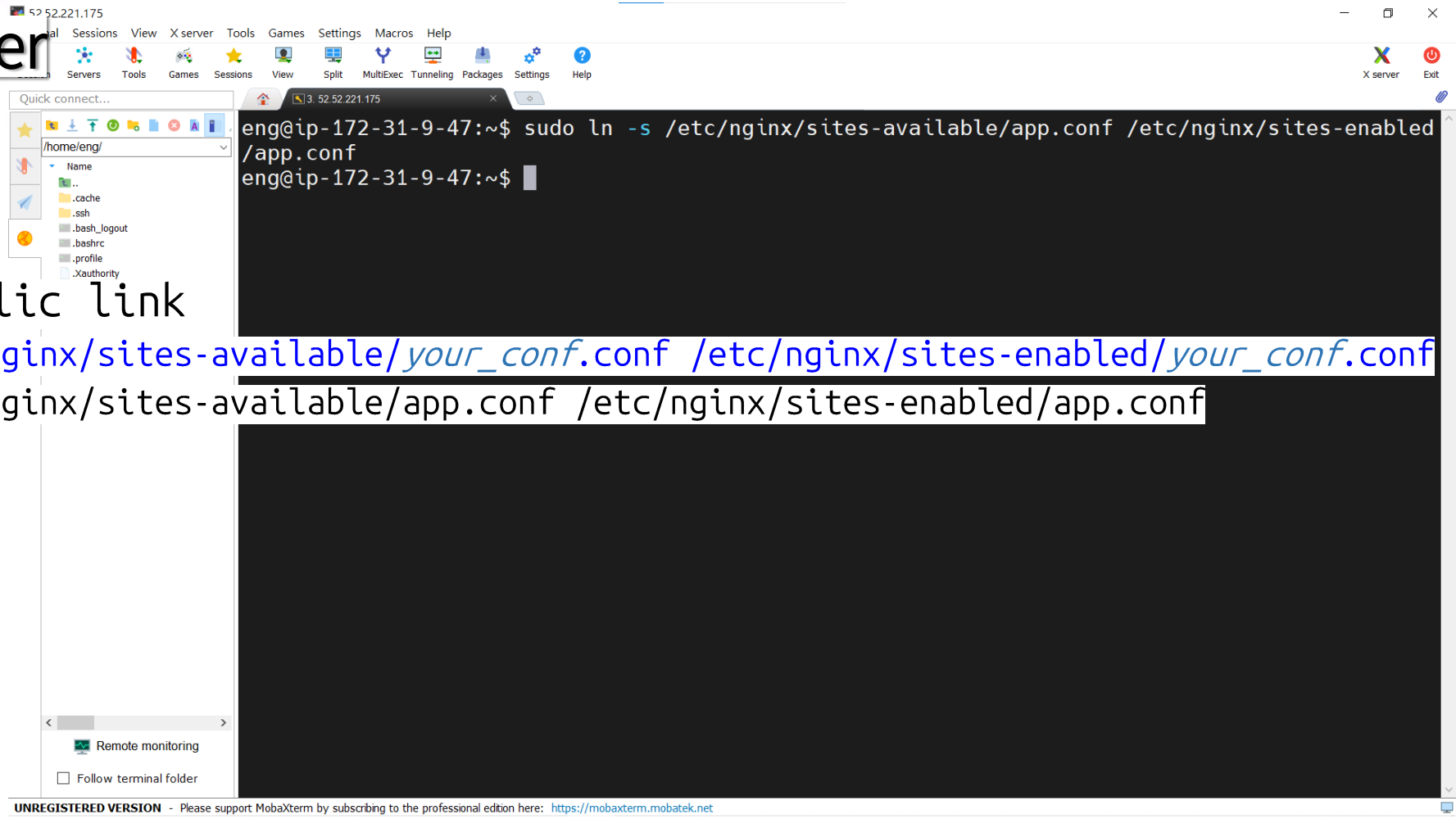
安裝 Web Server

2. 安裝 Web Server

b. 建立組態檔

③ 建立 symbolic link

```
sudo ln -s /etc/nginx/sites-available/your_conf.conf /etc/nginx/sites-enabled/your_conf.conf
sudo ln -s /etc/nginx/sites-available/app.conf /etc/nginx/sites-enabled/app.conf
```



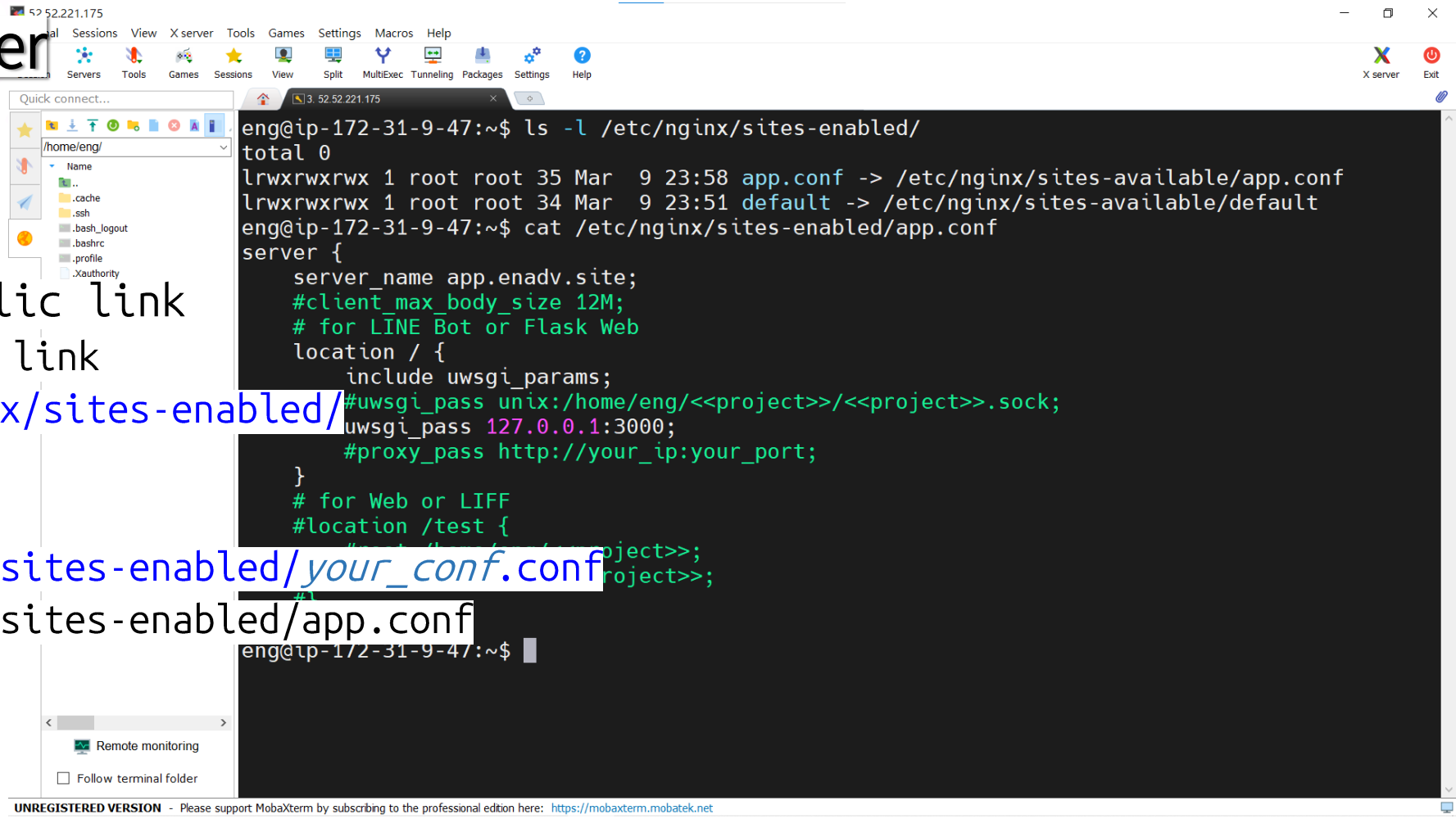
安裝 Web Server

2. 安裝 Web Server

b. 建立組態檔

④ 確認 symbolic link

```
# 確認 symbolic link
ls -l /etc/nginx/sites-enabled/

# 確認組態檔
cat /etc/nginx/sites-enabled/your_conf.conf
cat /etc/nginx/sites-enabled/app.conf
```

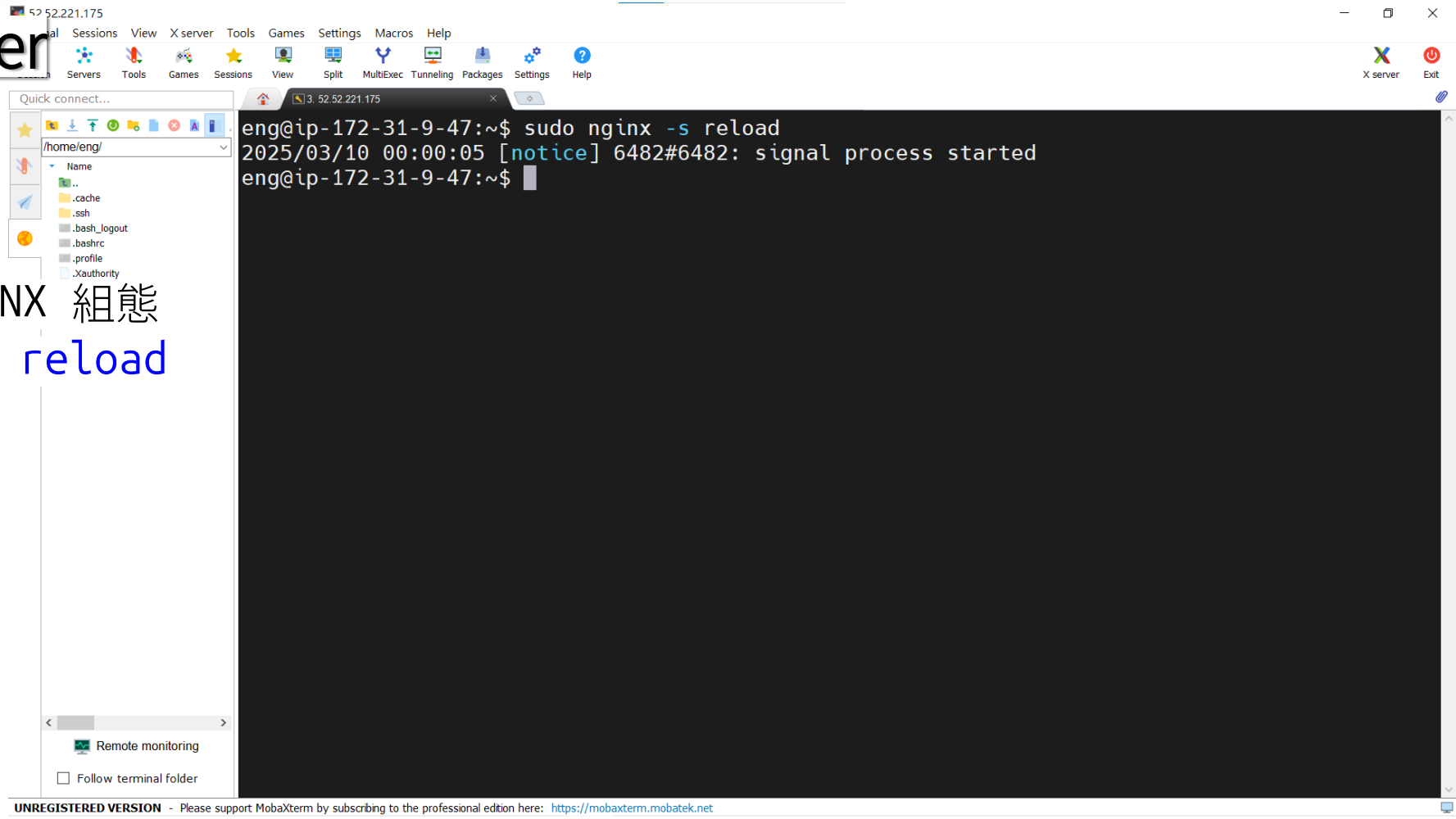


安裝 Web Server

2. 安裝 Web Server

c. 載入組態檔

```
# 重新載入 NGINX 組態
sudo nginx -s reload
```

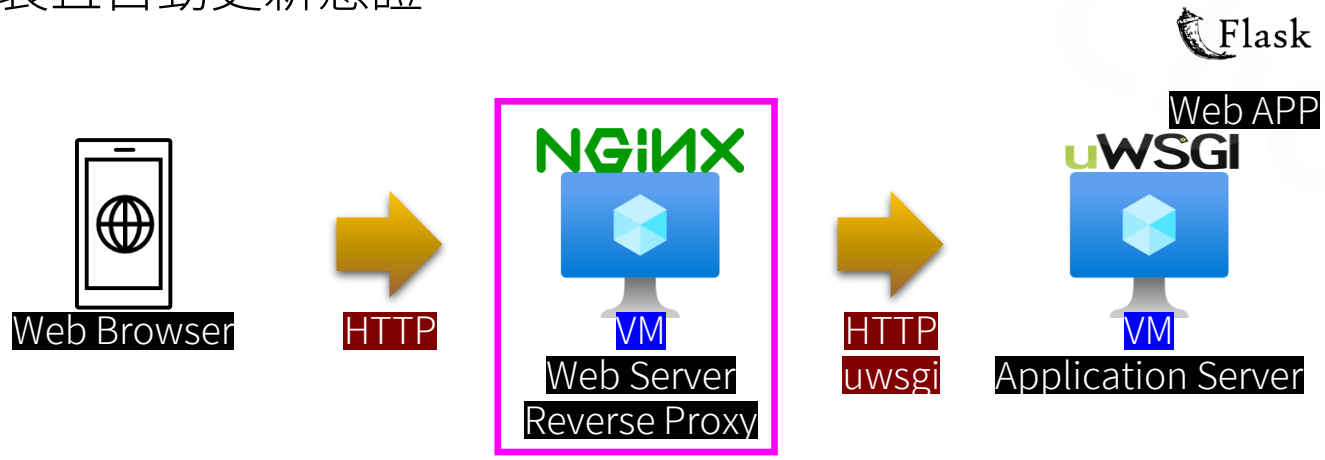


安裝 Web Server

3. 安裝憑證

免費憑證機器人 Certbot 特色

- https://certbot.eff.org/
- 採用 Let's Encrypt 免費憑證
- 自動安裝且自動更新憑證

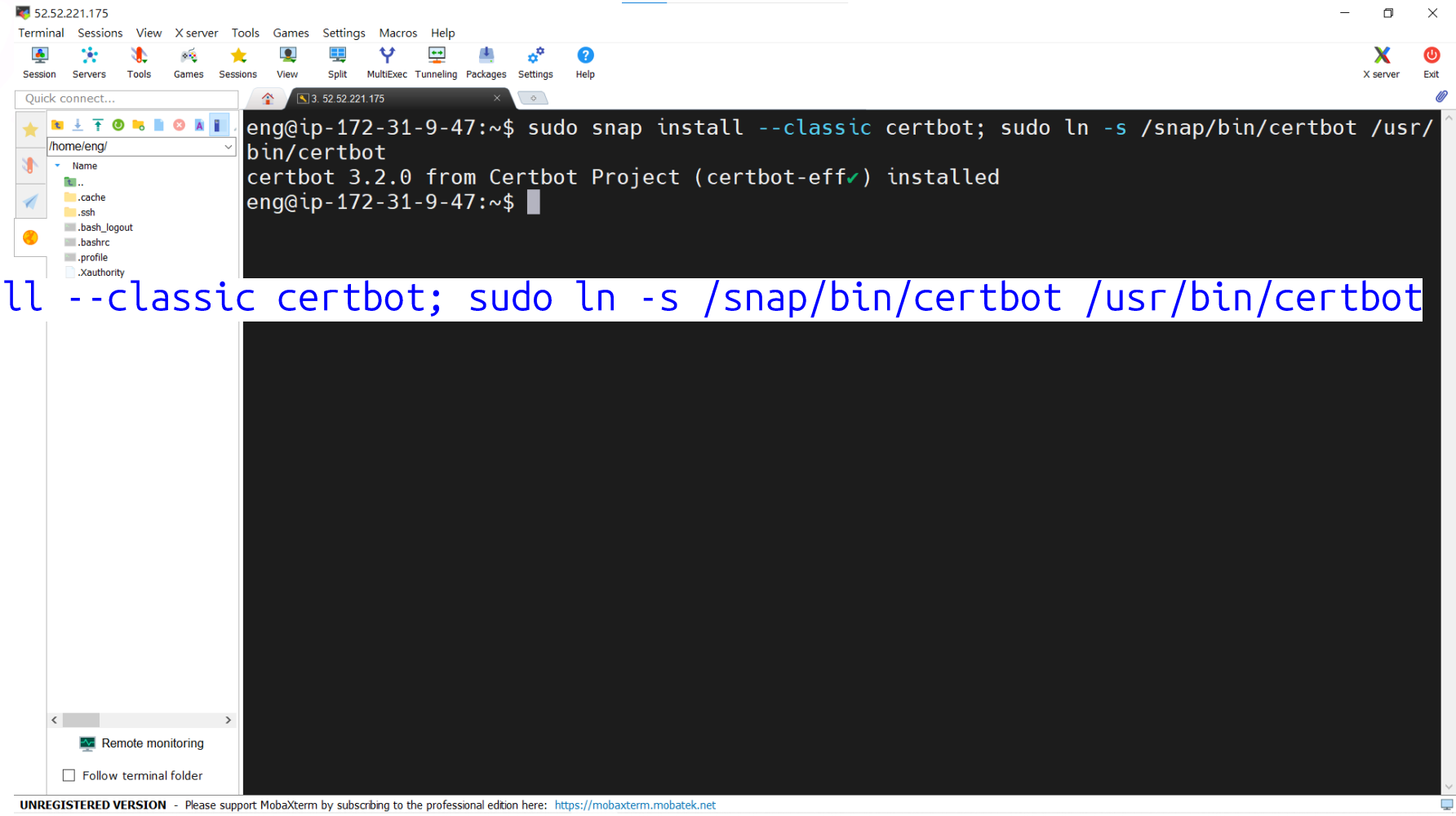


安裝 Web Server

3. 安裝憑證

a. 安裝 Certbot

```
sudo snap install --classic certbot; sudo ln -s /snap/bin/certbot /usr/bin/certbot
```

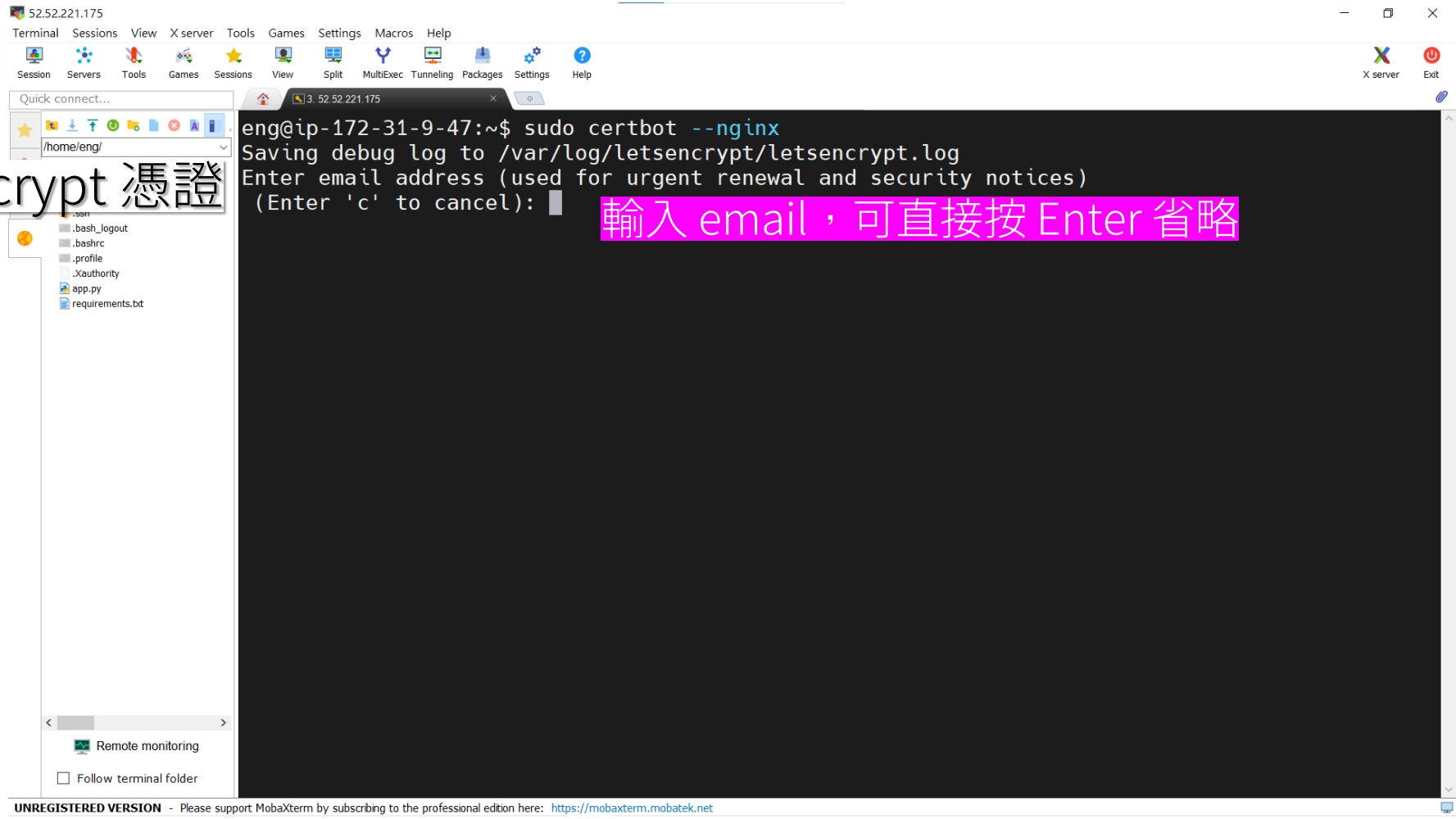


© 2026 Enos Chou

安裝 Web Server

3. 安裝憑證

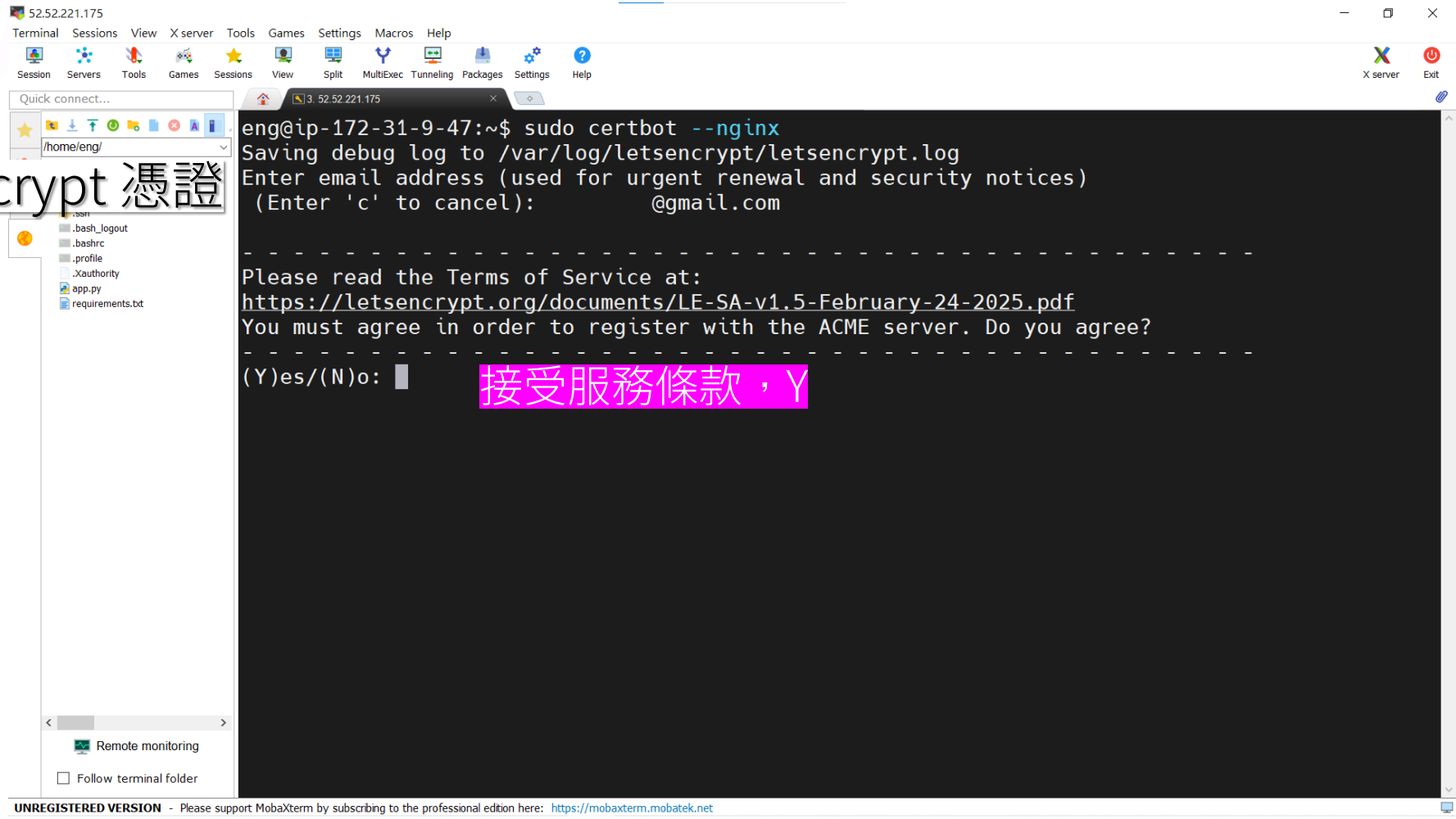
b. 申請 Let's Encrypt 憑證



安裝 Web Server

3. 安裝憑證

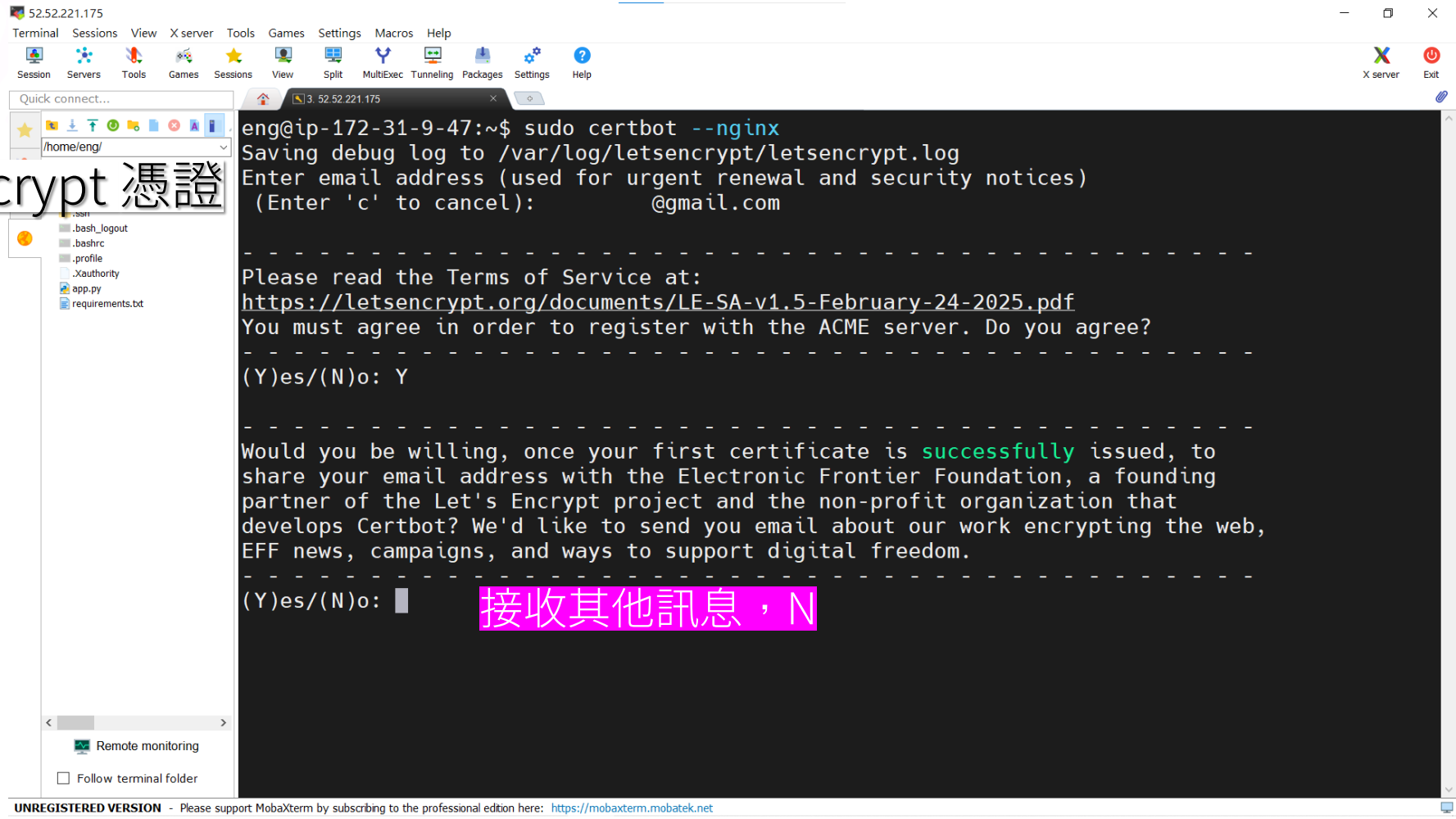
b. 申請 Let's Encrypt 憑證



安裝 Web Server

3. 安裝憑證

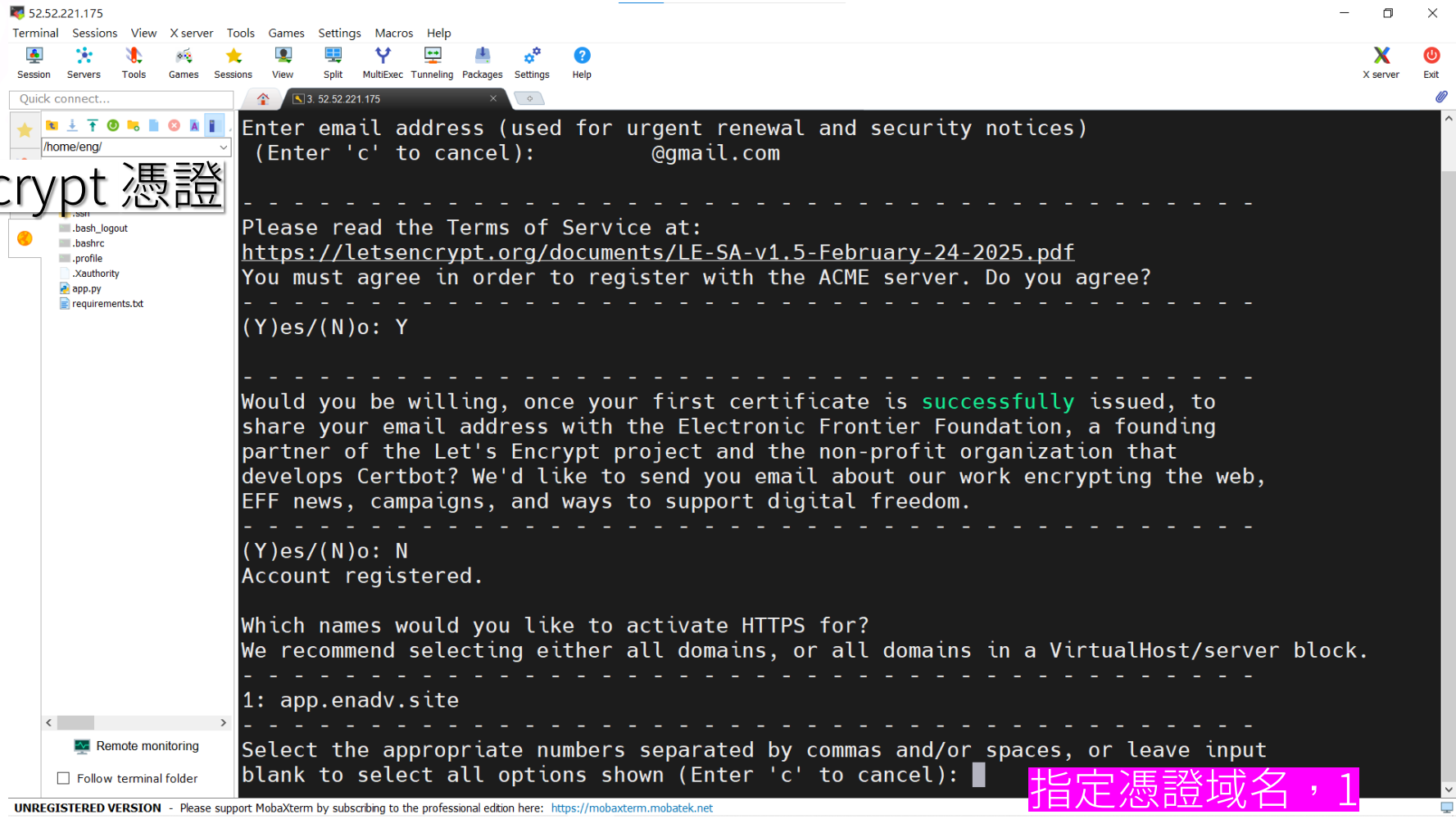
b. 申請 Let's Encrypt 憑證



安裝 Web Server

3. 安裝憑證

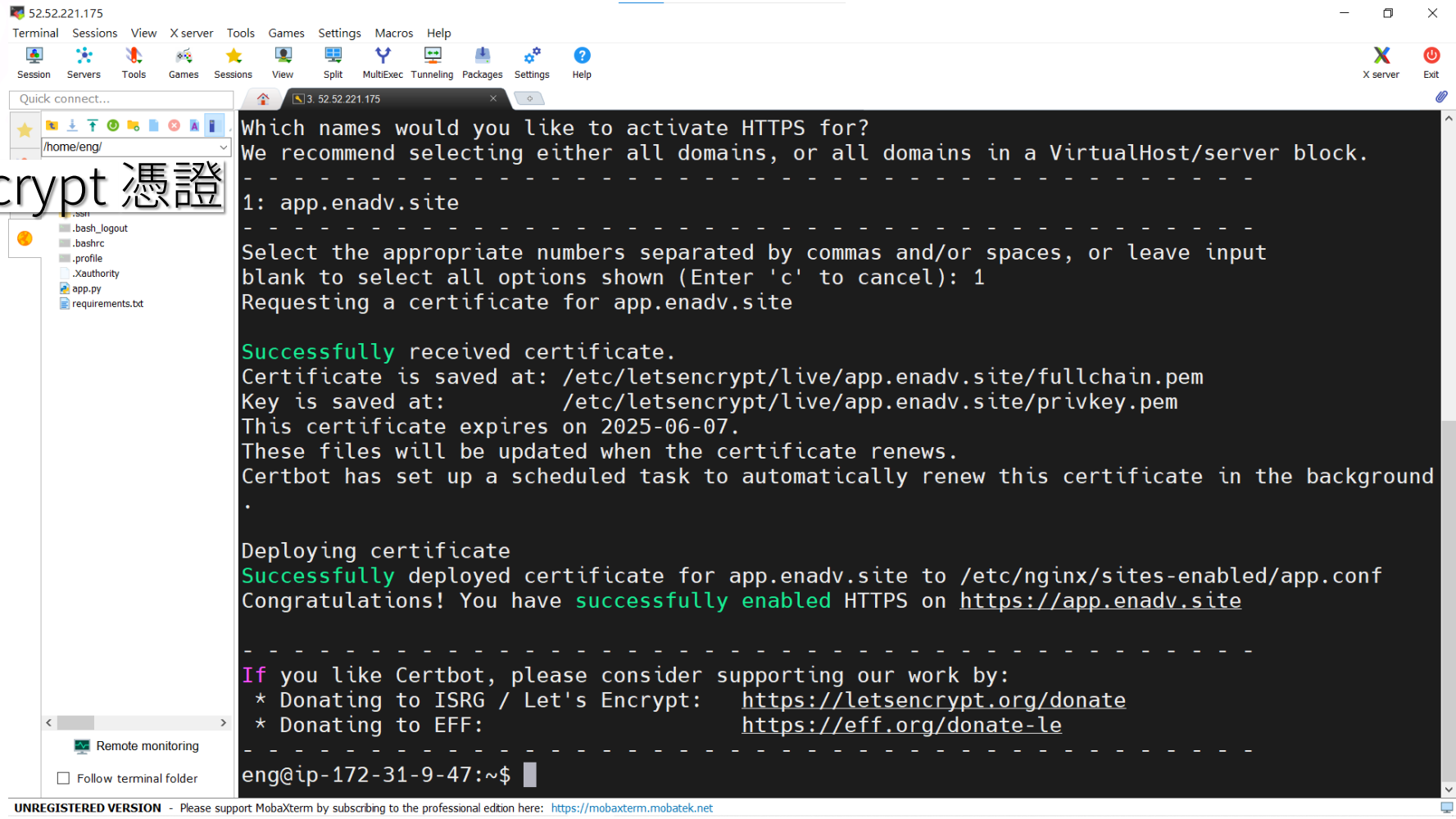
b. 申請 Let's Encrypt 憑證



安裝 Web Server

3. 安裝憑證

b. 申請 Let's Encrypt 憑證

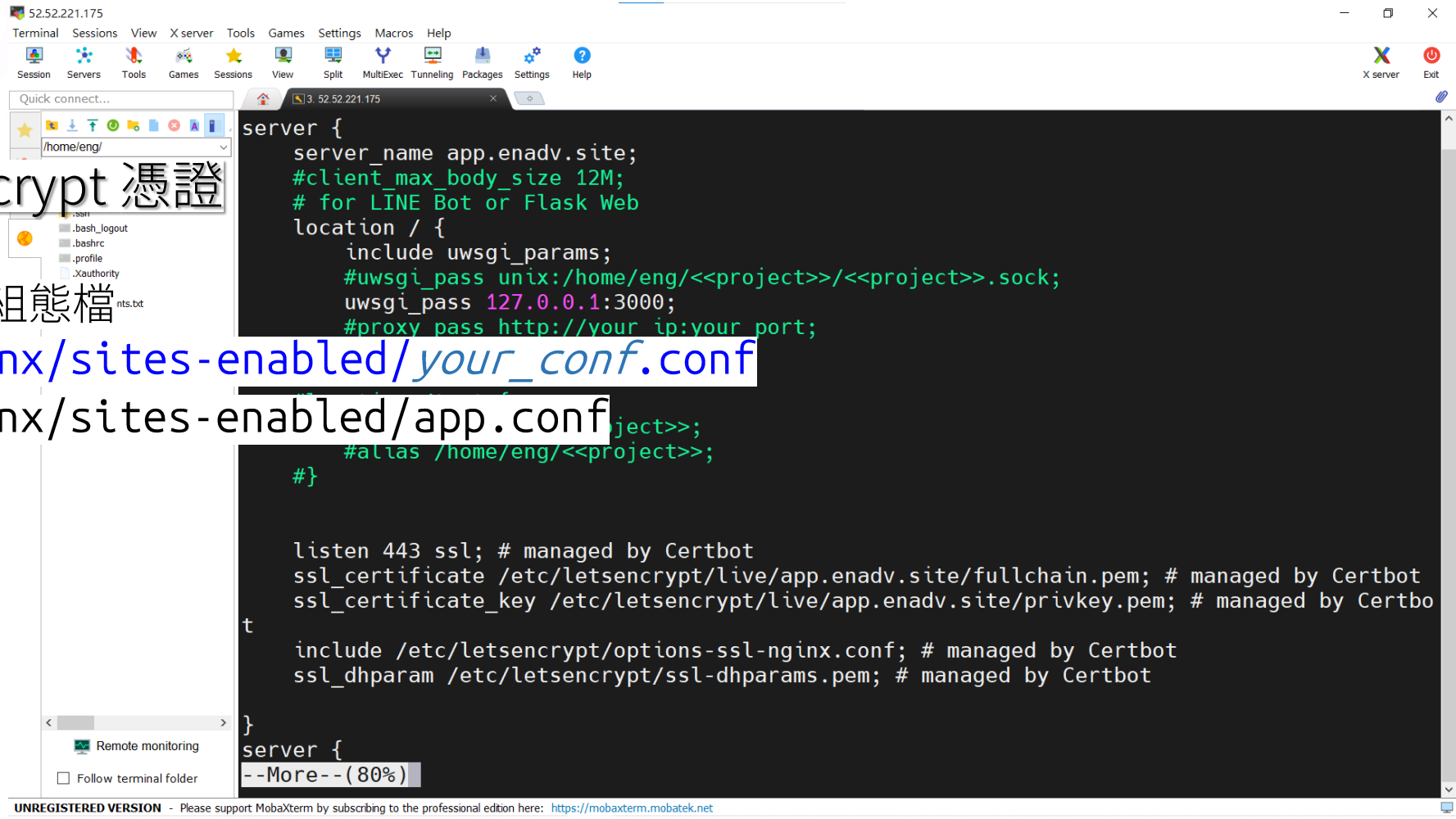


安裝 Web Server

3. 安裝憑證

c. 觀察 Let's Encrypt 憑證

```
# 觀察 NGINX 組態檔
more /etc/nginx/sites-enabled/your_conf.conf
more /etc/nginx/sites-enabled/app.conf
```



安裝 Web Server

4. (移除憑證)

a. 移除 Let's Encrypt 憑證

① `sudo certbot delete --cert-name your_domain`

② 手動移除 `/etc/nginx/sites-enabled/your_conf.conf` 中 Certbot 增加的設定 (`# managed by Certbot`)

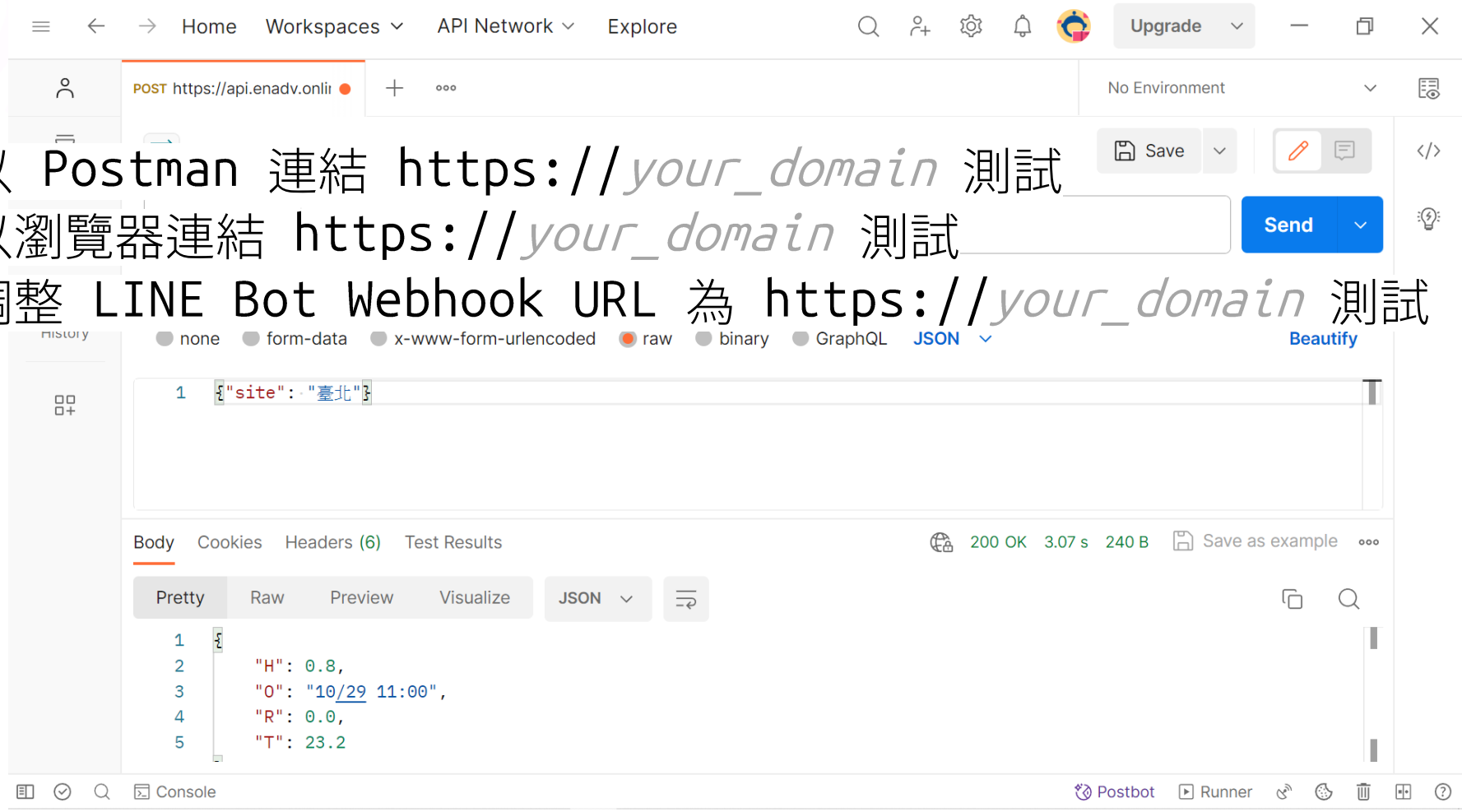
© 2026 Enos Chou

© 2026 Enos Chou

測試與除錯

1. 測試

 # API - 以 Postman 連結 https://*your_domain* 測試
 # Web - 以瀏覽器連結 https://*your_domain* 測試
 # LINE Bot - 調整 LINE Bot Webhook URL 為 https://*your_domain* 測試

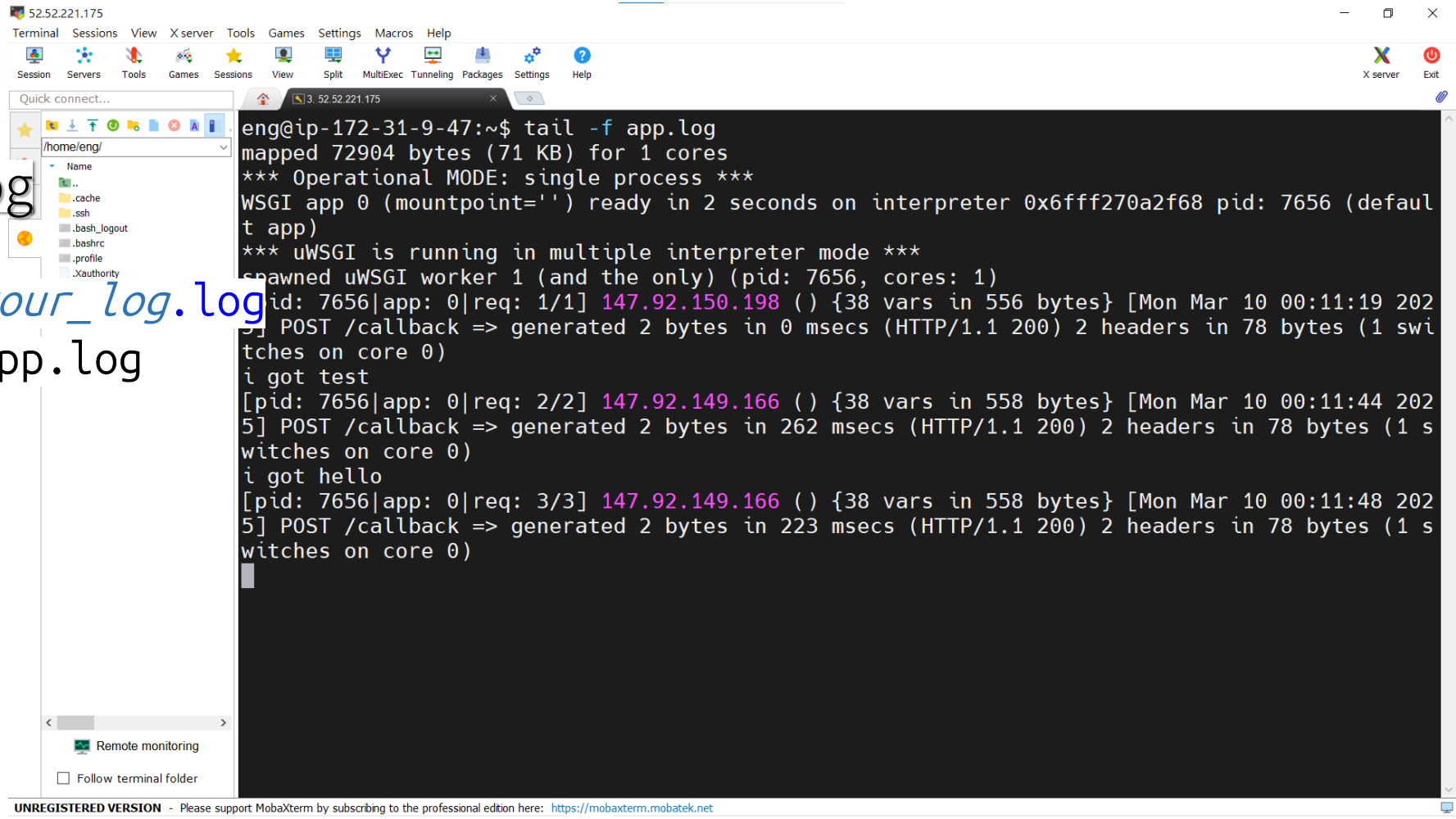


測試與除錯

2. 除錯

a. 檢視 uWSGI log

```
cd; tail -f your_log.log
cd; tail -f app.log
```

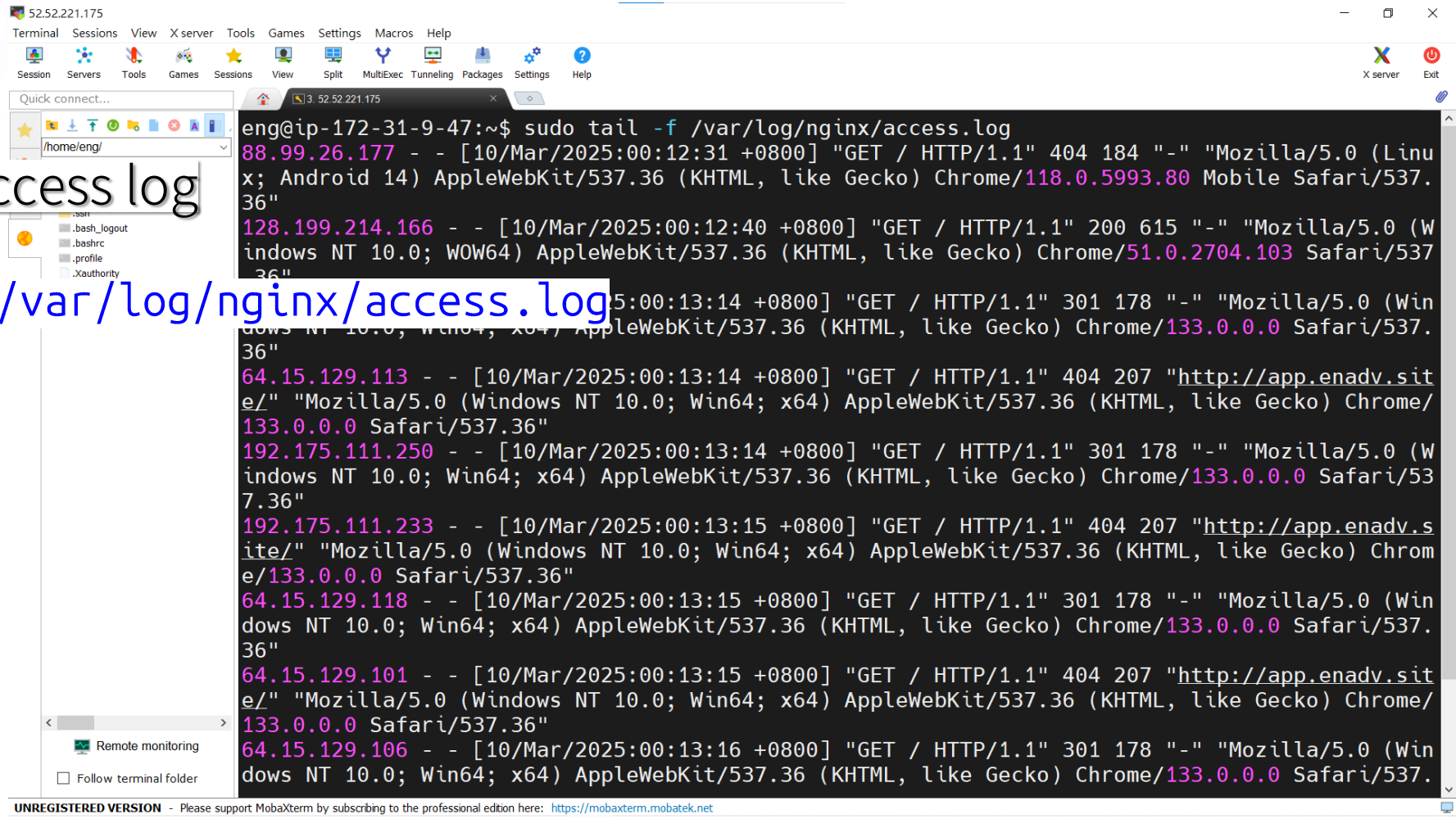


測試與除錯

2. 除錯

b. 檢視 NGINX access log

```
sudo tail -f /var/log/nginx/access.log
```

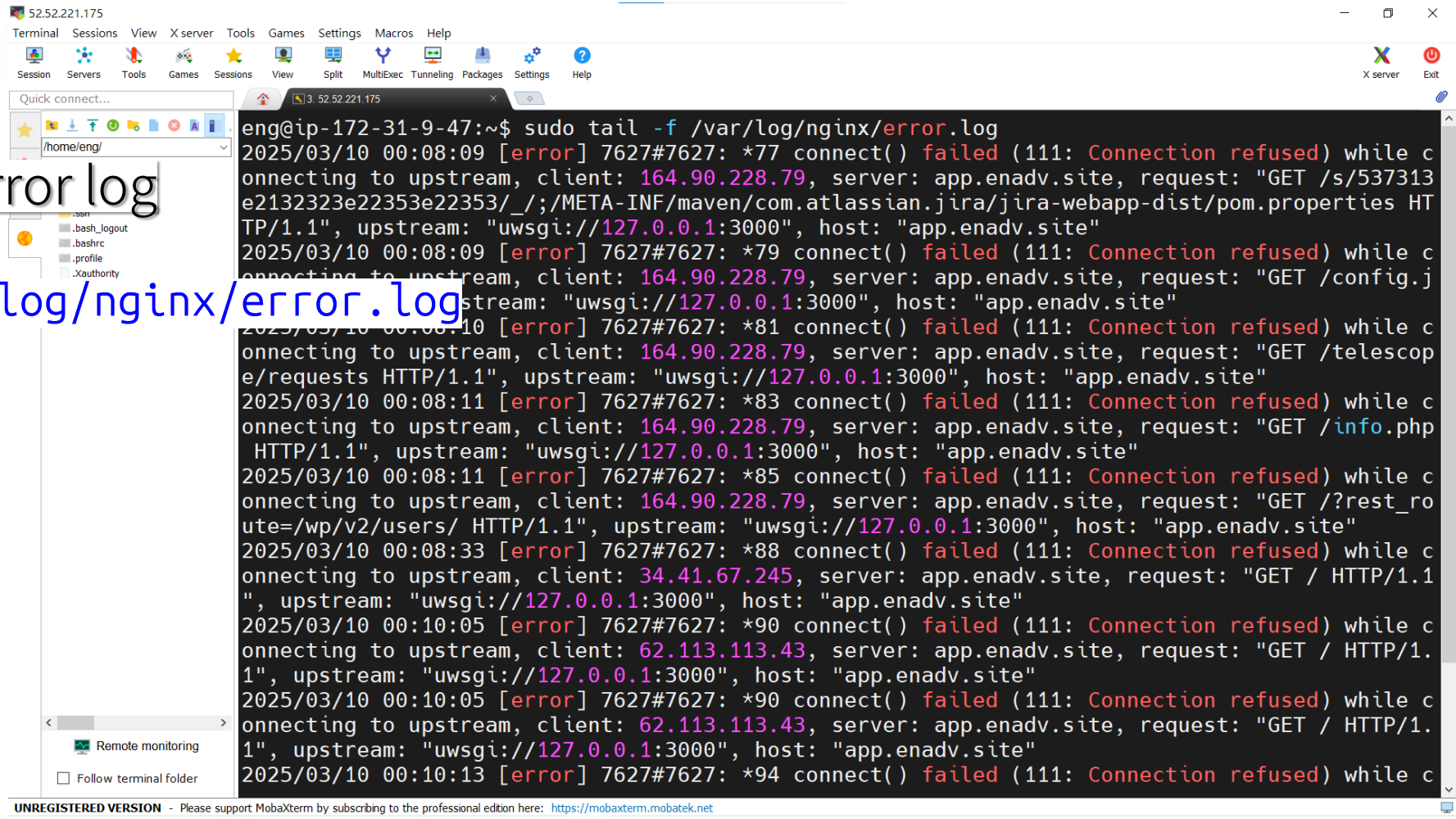


測試與除錯

2. 除錯

c. 檢視 NGINX error log

```
tail -f /var/log/nginx/error.log
```



© 2026 Enos Chou

Docker 基礎

Docker Concept

Container Security

Docker Environment

Environment ft. Pure Docker

a. Windows

WSL2

① 完成最新的 Windows Update 更新
② 以系統管理員開啟 Windows PowerShell
③ `wsl --install`
④ 若未自動安裝 Ubuntu，則於重開機後再執行 `wsl --install Ubuntu`

Docker Desktop

© 2026 Enos Chou

Docker Environment

Environment ft. Pure Docker

b. macOS

Docker Desktop

© 2026 Enos Chou

Docker Environment

Environment ft. Pure Docker

c. VirtualBox

Linux Image

© 2026 Enos Chou

Docker Environment

Environment ft. Pure Docker

d. VM ft. Ubuntu 20

```
sudo apt update; sudo apt install ca-certificates curl gnupg; sudo
install -m 0755 -d /etc/apt/keyrings; curl -fsSL
https://download.docker.com/linux/ubuntu/gpg | sudo gpg --dearmor -o
/etc/apt/keyrings/docker.gpg; sudo chmod a+r
/etc/apt/keyrings/docker.gpg; echo "deb [arch="$(dpkg --print-
architecture)" signed-by=/etc/apt/keyrings/docker.gpg]
https://download.docker.com/linux/ubuntu "$(. /etc/os-release && echo
"$VERSION_CODENAME")" stable" | sudo tee
/etc/apt/sources.list.d/docker.list > /dev/null; sudo apt update; sudo
apt install -y docker-ce docker-ce-cli containerd.io docker-buildx-
plugin docker-compose-plugin; sudo usermod -aG docker $USER; newgrp
docker
```

© 2026 Enos Chou

Docker Environment

Environment ft. Cloud Terminal

a. AWS
   AWS CloudShell

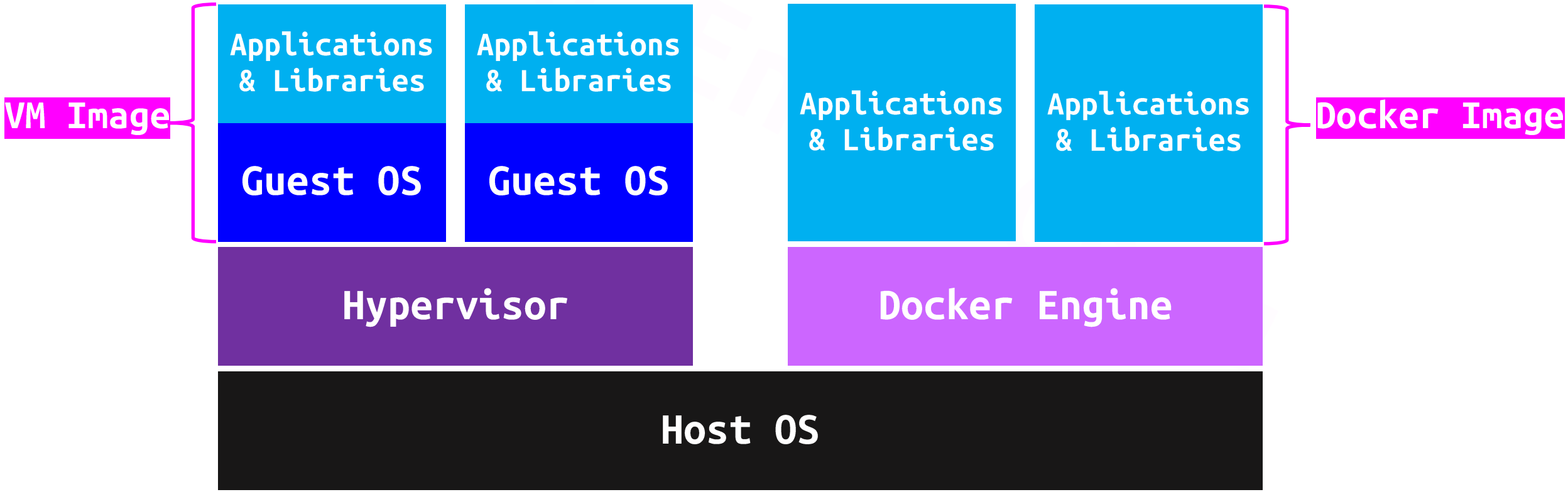
b. Azure
   N/A

c. Google Cloud
   Google Cloud Shell

© 2026 Enos Chou

Docker Concept

VM vs Docker



Docker Concept

VM vs Docker

VM Image OS-based vs App-based Docker Image

© 2026 Enos Chou

Docker Concept

Terminology

Docker
Docker Image
Docker Container
Dockerfile
Repository
Docker Hub
Registry
Docker Compose

© 2026 Enos Chou

Docker Concept

Basic Docker Commands

docker images
docker build
docker commit
docker tag
docker push
docker pull
docker rmi

docker run
docker exec
docker ps
docker stop
docker kill
docker system prune
docker inspect

© 2026 Enos Chou

Docker Commands

映像檔管理

docker images：查詢已下載映像檔

```
docker images
```

docker pull：下載映像檔

```
docker pull your_repo:your_tag
ex: docker pull ubuntu:24.04
```

docker push：上傳映像檔

```
docker push your_repo:your_tag
ex: docker push test:v1
```

© 2026 Enos Chou

Docker Commands

映像檔管理

docker tag：增加標籤

```
docker tag your_repo:your_tag your_newrepo:your_newtag
ex: docker tag test:v1 testing:0.0.0
```

docker rmi：刪除映像檔

```
docker rmi your_repo:your_tag
ex: docker rmi test:v1
```

© 2026 Enos Chou

Docker Commands

映像檔管理

docker system prune：清理未使用資源

```
docker system prune
```

© 2026 Enos Chou

Docker Commands

容器運行

docker run：執行映像檔

```
docker run --name your_name -d --rm -it -p
your_port_host:your_port_container -e your_env=your_value -v
your_path_host:your_path_container your_repo:your_tag your_shell
ex: docker run --name test -d --rm -p 8080:80 test:v1
```

© 2026 Enos Chou

Docker Commands

容器運行

docker attach：接回原容器環境

```
docker attach your_containerid
ex: docker attach test
```

docker exec：以新環境進入容器

```
docker exec -it your_containerid your_shell
ex: docker exec -it test bash
```

© 2026 Enos Chou

Docker Commands

容器運行

docker ps：查詢容器執行狀況

```
docker ps -a
ex: docker ps
ex: docker ps -a
```

docker stop：停止容器執行

```
docker stop your_containerid
ex: docker stop test
```

docker rm：移除容器

```
docker rm your_containerid
ex: docker rm test
```

© 2026 Enos Chou

Docker Commands

映像檔備份

docker save：儲存映像檔

```
docker save -o your_path.tar your_repo:your_tag
ex: docker save -o d:\test.tar test:v1
```

docker load：載入映像檔

```
docker load -i your_path.tar
ex: docker load -i d:\test.tar
```

© 2026 Enos Chou

Docker Commands

映像檔製作

docker commit：儲存異動後的容器內容至新的映像檔

```
docker commit your_containerid your_repo:your_tag
ex: docker commit a1e9432e6fb7 test:v2
```

```
docker commit --change your_change your_containerid your_repo:your_tag
ex: docker commit --change 'WORKDIR /app' a1e9432e6fb7 test:v2
```

© 2026 Enos Chou

Docker Commands

映像檔製作

docker build：依據 Dockerfile 建立映像檔

```
docker build --no-cache -t your_repo:your_tag -f your_dockerfile .
```

ex: docker build --no-cache -t test:v1 .

Enos Chou

Docker Image by Dockerfile

Dockerfile

目的

- 描述 Docker Image 建構程序與 Docker Container 啟動指令

© 2026 Enos Chou

Docker Image by Dockerfile

Dockerfile Example

```
FROM ubuntu:24.04 指定基底映像檔，後續指令都以此映像檔為基礎疊加
WORKDIR /app 於映像檔中建立目錄，後續指令將在此目錄下進行
COPY app.py requirements.txt ./ 複製檔案到映像檔中指定目錄
ENV TZ=Asia/Taipei \  設定映像檔之環境變數
    LANG=C.UTF-8 \
    PYTHONUNBUFFERED=1
RUN ln -snf /usr/share/zoneinfo/$TZ /etc/localtime && \  以 root 於映像檔中執行指令
    echo $TZ > /etc/timezone && \
    apt update && \
    apt install -y python3-pip tzdata && \
    dpkg-reconfigure -f noninteractive tzdata && \
    pip3 install --break-system-packages --no-cache-dir -r requirements.txt uWSGI
    apt clean && rm -rf /var/lib/apt/lists/*
CMD exec uwsgi -w app:app --http-socket :${PORT:-8080}   容器啟動時的執行指令
```

© 2026 Enos Chou

Docker Image by Dockerfile

Dockerfile Notes

a. 映像檔層層疊加，每層皆佔據些許空間。上線版本映像檔應力求最小層數
b. Dockerfile 異動時會從被更動的最內層開始往外層更動映像檔，在開發階段，愈常被更動的內容宜放在愈外層以提高建立效率
c. 容器執行時帳號為 root
d. 容器之啟動程式須在前景執行；若在背景執行則容器將立刻結束
e. 公有雲容器型應用會自動將所有 port 1 to 1 expose，因此不需額外設定 -p
f. 公有雲容器型應用多期待由雲端指定 PORT，因此公有雲容器型應用的映像檔於啟動程式啟動使用的 port 宜設為環境變數 $PORT
g. docker build 執行時預設搜尋 Dockerfile，若為其他檔名需以 -f 指定

© 2026 Enos Chou

Practice of the Container Life Cycle

1. From Build to Run

2. Verify and Stop

© 2026 Enos Chou

Practice of the Container Life Cycle

1. From Build to Run

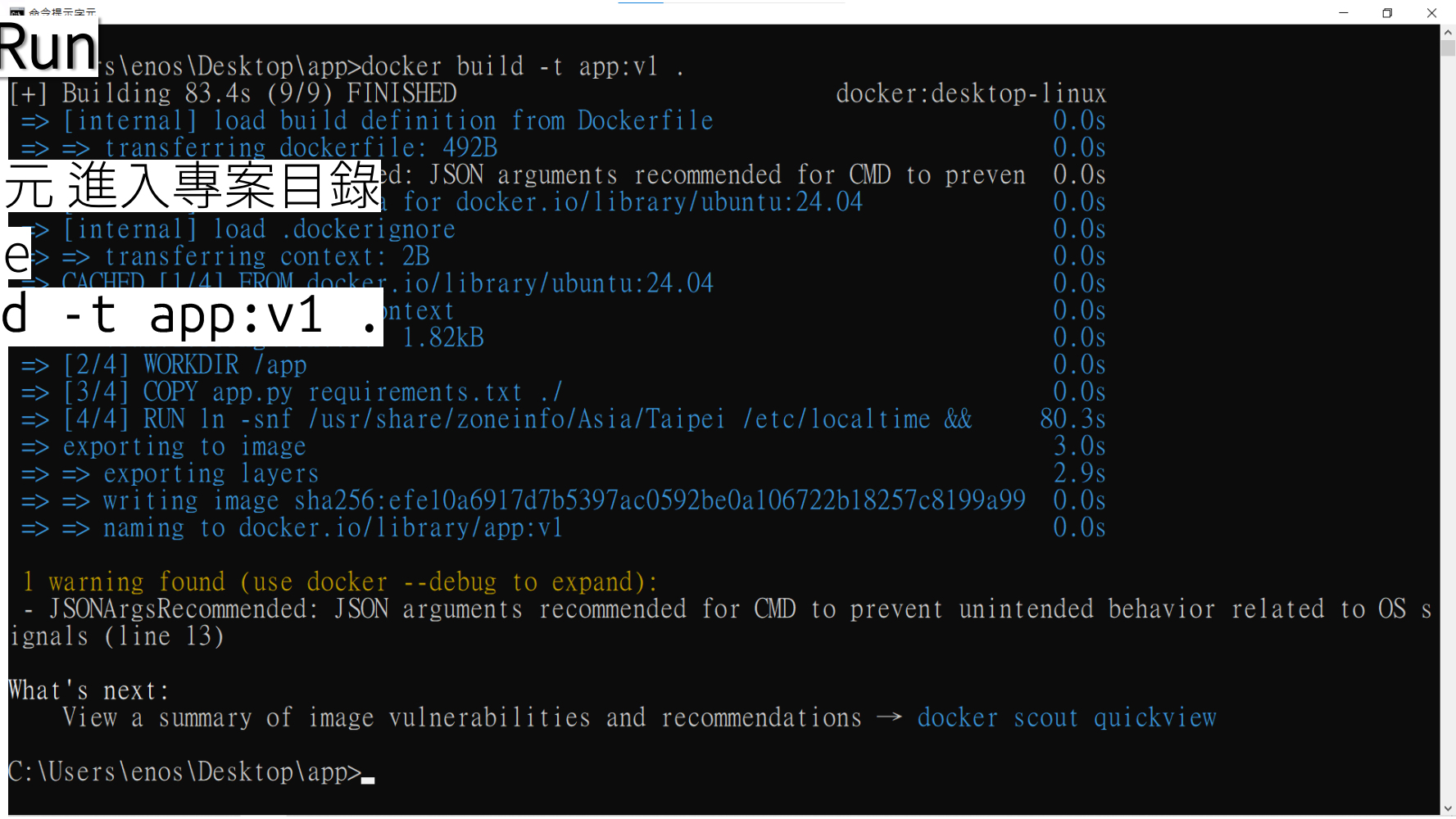
a. 於 命令提示字元 進入專案目錄
b. 建立 Dockerfile

```
FROM ubuntu:24.04
WORKDIR /app
COPY app.py requirements.txt ./
ENV TZ=Asia/Taipei \
    LANG=C.UTF-8 \
    PYTHONUNBUFFERED=1
RUN ln -snf /usr/share/zoneinfo/$TZ /etc/localtime && \
    echo $TZ > /etc/timezone && \
    apt update && \
    apt install -y python3-pip tzdata && \
    dpkg-reconfigure -f noninteractive tzdata && \
    pip3 install --break-system-packages --no-cache-dir -r requirements.txt uWSGI
    apt clean && rm -rf /var/lib/apt/lists/*
CMD exec uwsgi -w app:app --http-socket :${PORT:-8080}
```

Practice of the Container Life Cycle

1. From Build to Run

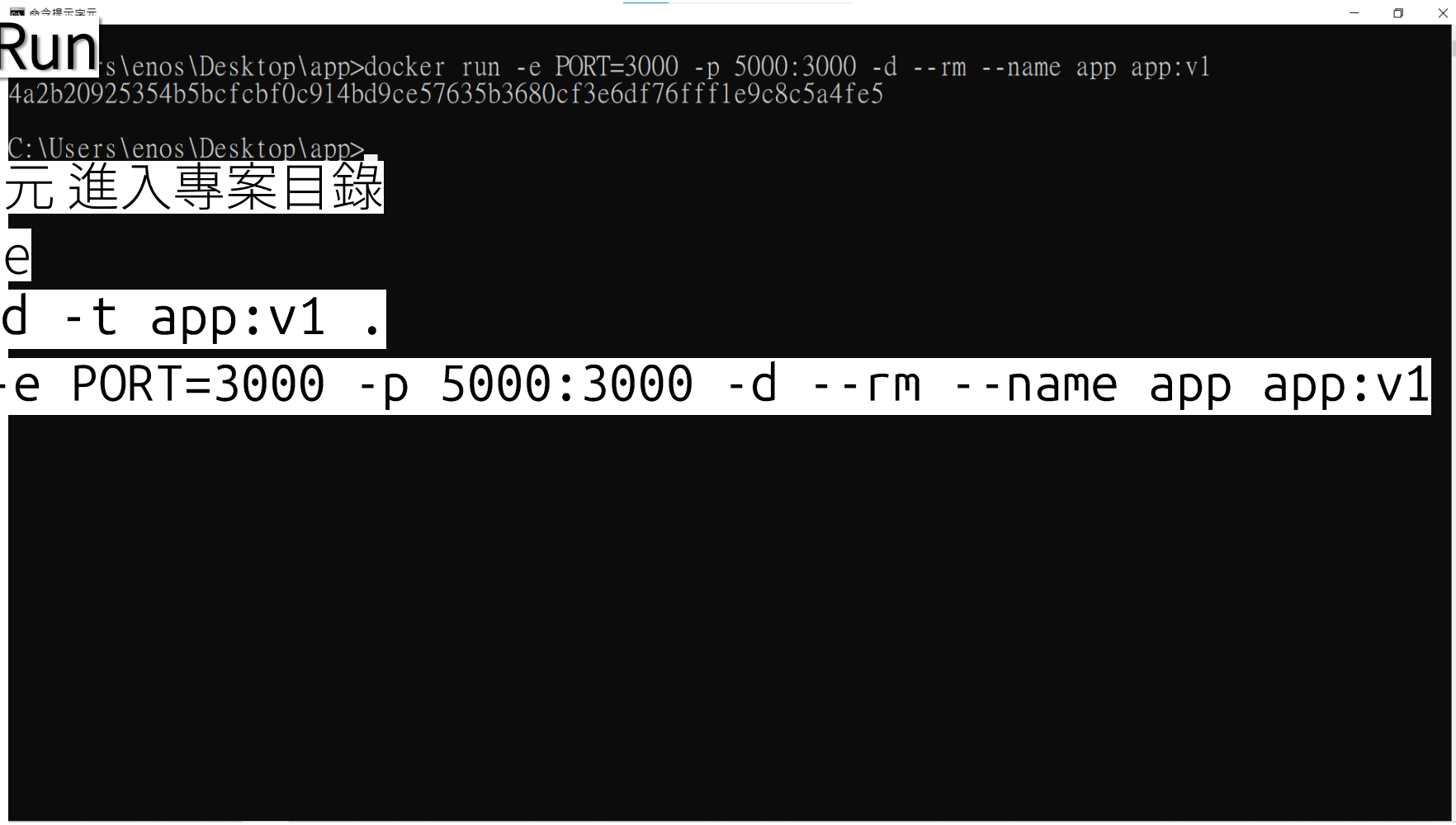
   a. 於 命令提示字元 進入專案目錄
   b. 建立 Dockerfile
   c. `docker build -t app:v1 .`



Practice of the Container Life Cycle

1. From Build to Run

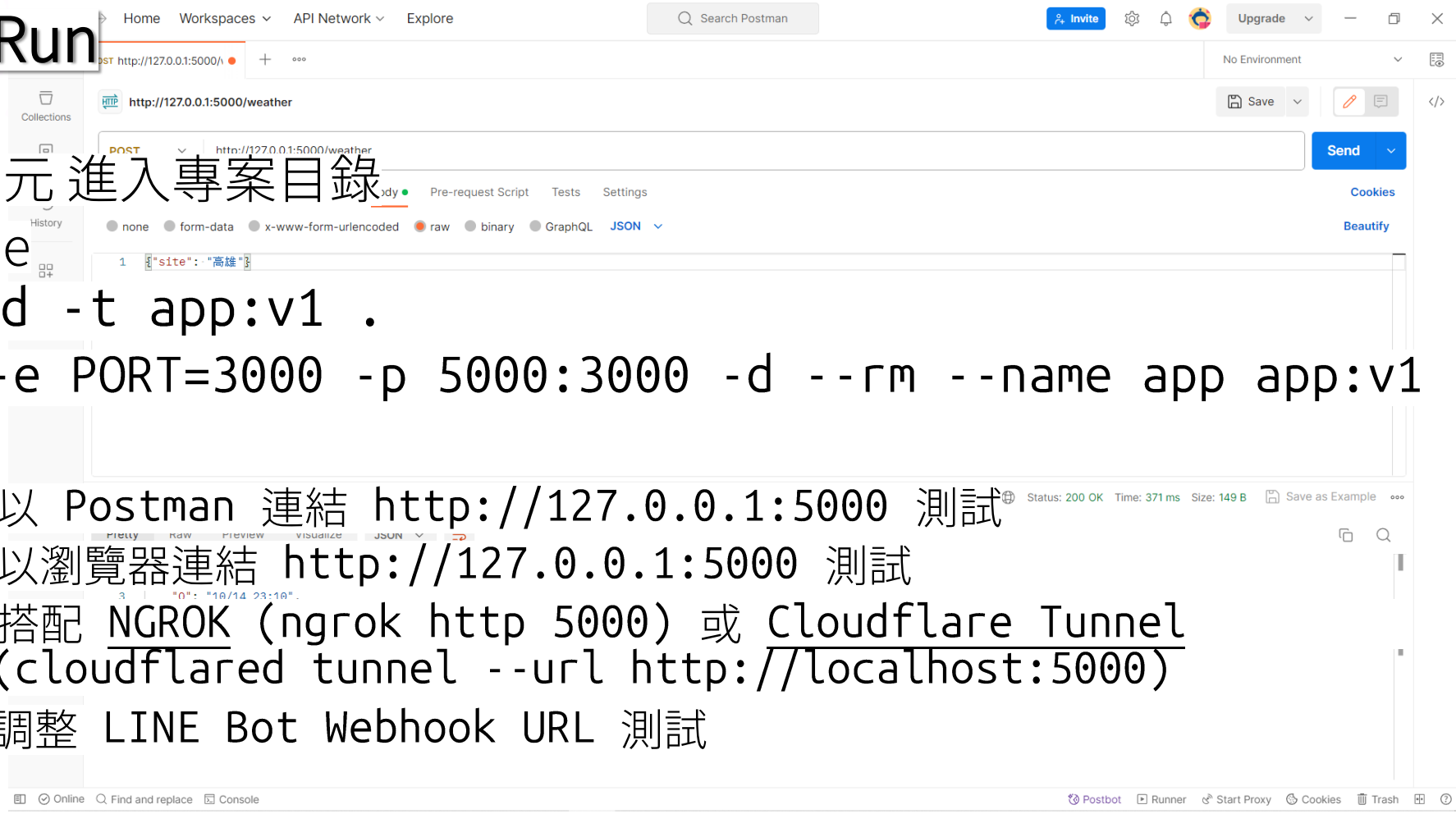
   a. 於 命令提示字元 進入專案目錄
   b. 建立 Dockerfile
   c. `docker build -t app:v1 .`
   d. `docker run -e PORT=3000 -p 5000:3000 -d --rm --name app app:v1`



Practice of the Container Life Cycle

1. From Build to Run

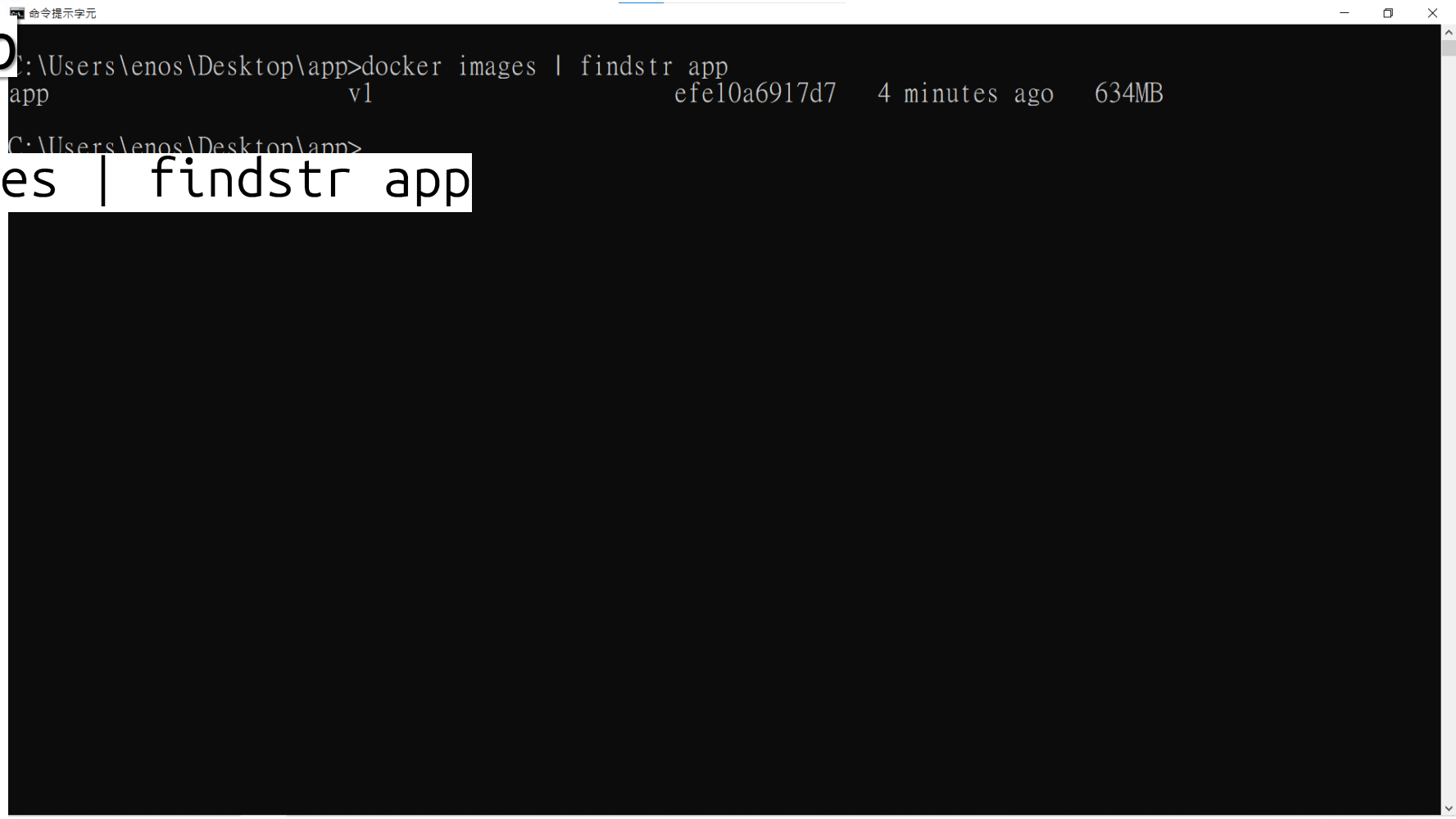
a. 於 命令提示字元 進入專案目錄
b. 建立 Dockerfile
c. `docker build -t app:v1 .`
d. `docker run -e PORT=3000 -p 5000:3000 -d --rm --name app app:v1`
e. 測試
   - **# API** - 以 Postman 連結 http://127.0.0.1:5000 測試
   - **# Web** - 以瀏覽器連結 http://127.0.0.1:5000 測試
   - **# LINE Bot** - 搭配 NGROK (ngrok http 5000) 或 Cloudflare Tunnel (cloudflared tunnel --url http://localhost:5000)
     調整 LINE Bot Webhook URL 測試



Practice of the Container Life Cycle

2. Verify and Stop

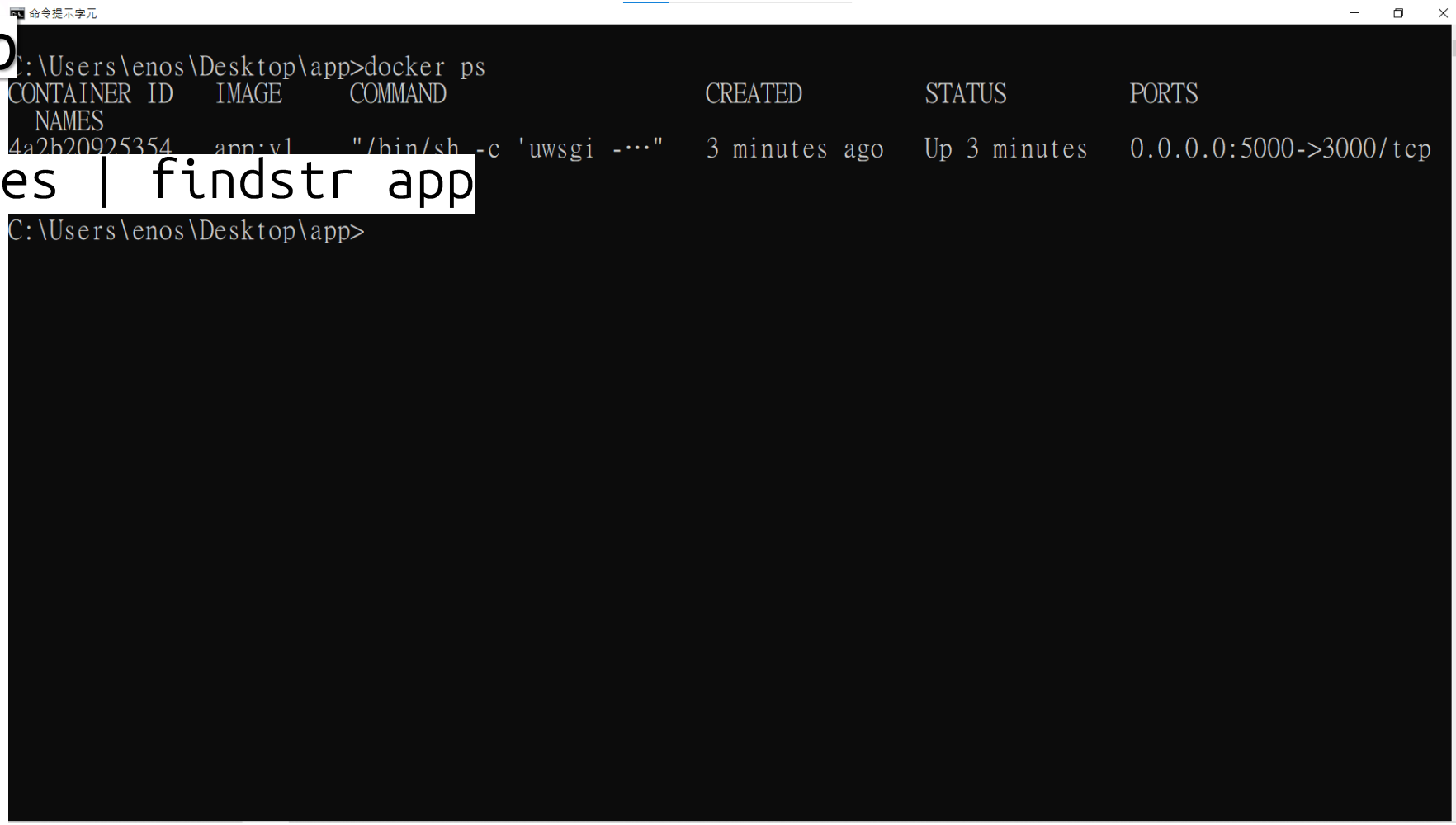
   a. `docker images | findstr app`



Practice of the Container Life Cycle

2. Verify and Stop

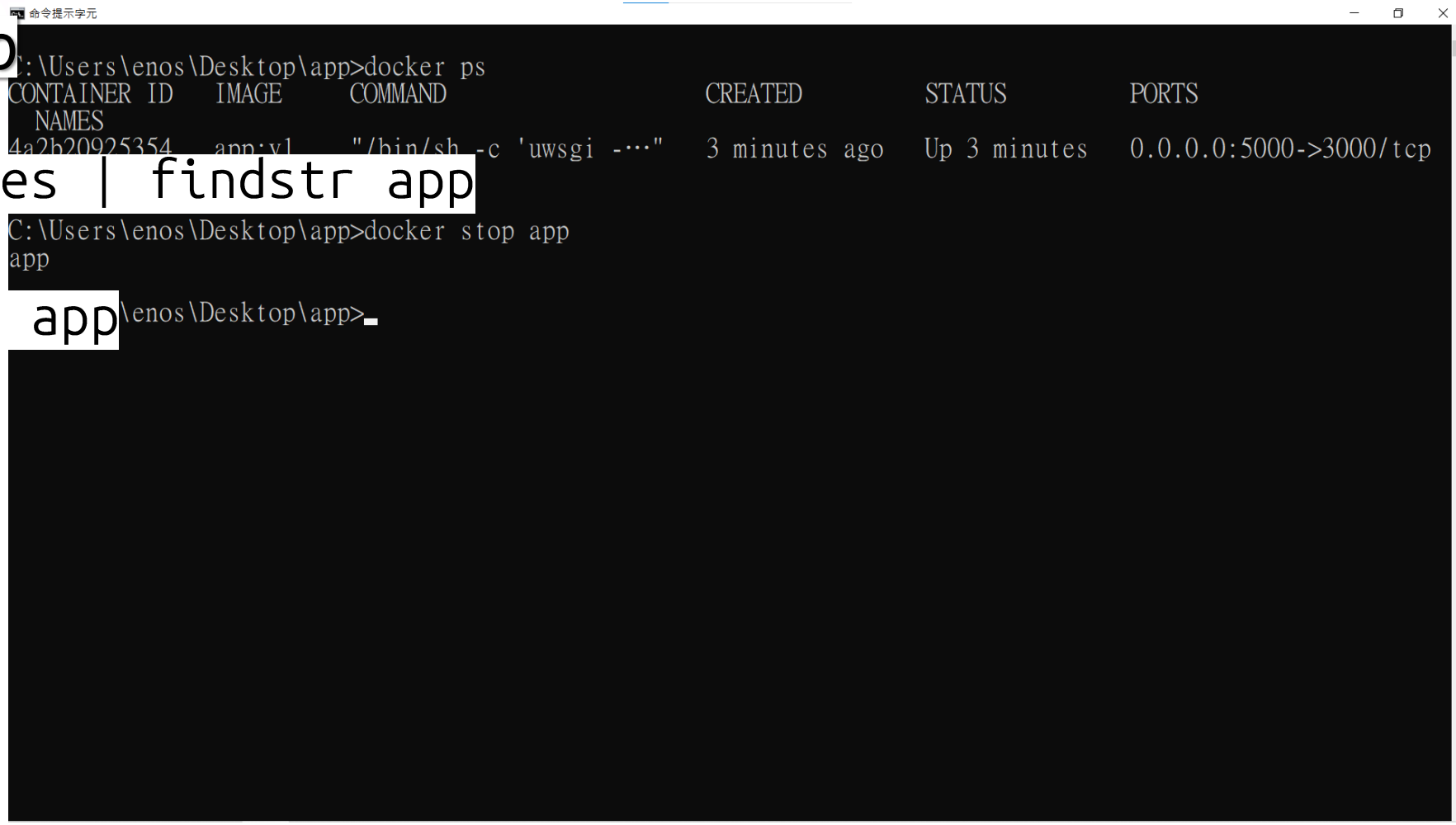
   a. `docker images | findstr app`
   b. `docker ps`



Practice of the Container Life Cycle

2. Verify and Stop

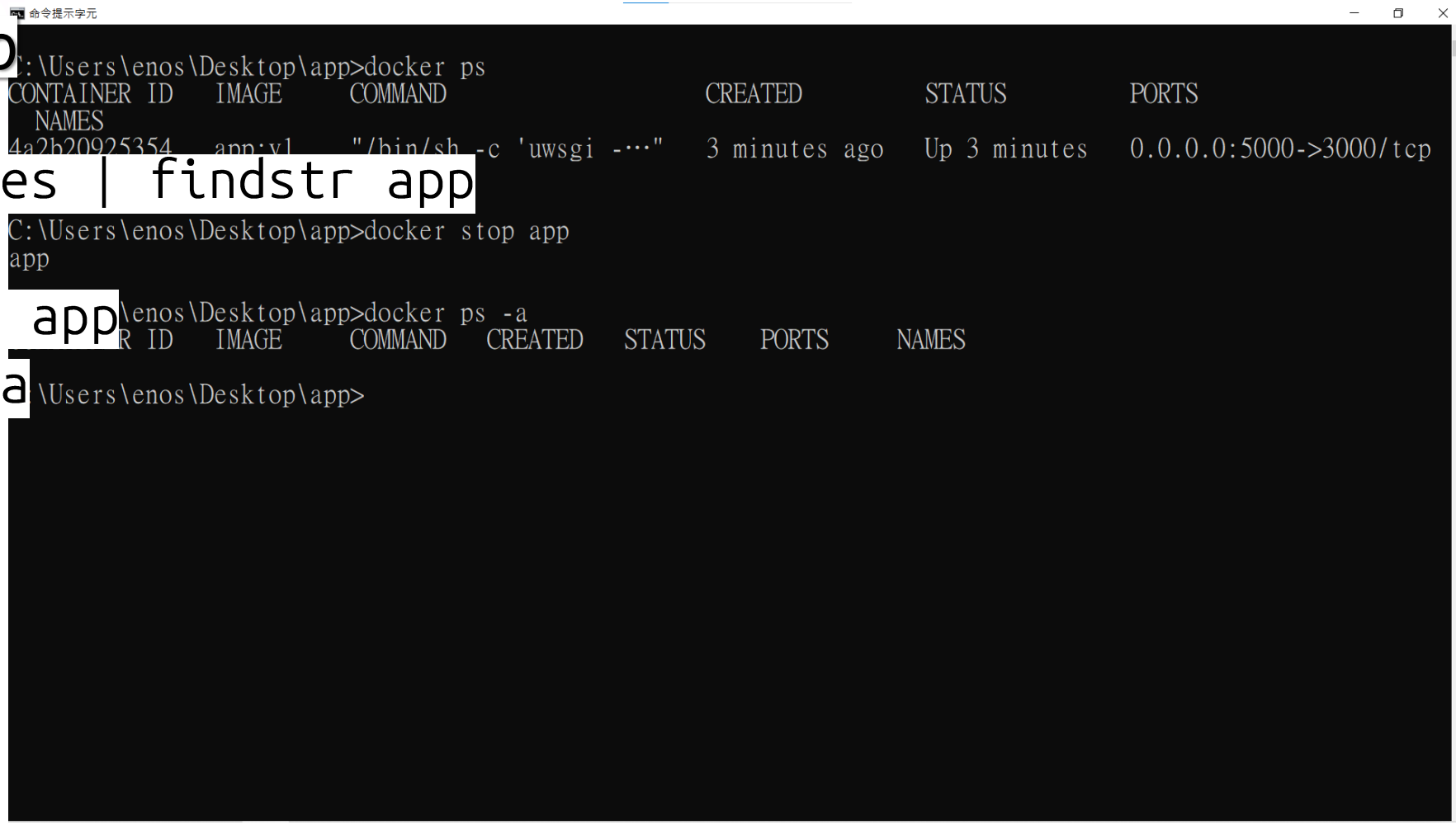
   a. `docker images | findstr app`
   b. `docker ps`
   c. `docker stop app`



Practice of the Container Life Cycle

2. Verify and Stop

   a. `docker images | findstr app`
   b. `docker ps`
   c. `docker stop app`
   d. `docker ps -a`



Storage of Docker Image

Repository

a. Local

b. Docker Hub

a. Repository in Cloud

© 2026 Enos Chou

Your Turn - Dockerfile from Python

以 Python 官方 Docker Image 為基礎建立 Dockerfile，製作天氣查詢 LINE Bot 的 Docker Image，並完成運作與測試 (30 mins)

Hints

- Debian bullseye ~ Ubuntu 20.04
- Debian bookworm ~ Ubuntu 22.04
- Debian trixie ~ Ubuntu 24.04

© 2026 Enos Chou

Container Security

1. 弱點掃描

Trivy on Windows

安裝

① 啟動 命令提示字元或 PowerShell

② `winget install AquaSecurity.Trivy`

© 2026 Enos Chou

Container Security

1. 弱點掃描

Trivy on Windows

掃描

① 啟動 命令提示字元或 PowerShell

② `trivy image your_repo:your_tag`

© 2026 Enos Chou

Container Security

1. 弱點掃描

Trivy on macOS

安裝

① 啟動 終端機

② `brew install trivy`

© 2026 Enos Chou

Container Security

1. 弱點掃描

Trivy on macOS

掃描

① 啟動 終端機

② `trivy image your_repo:your_tag`

© 2026 Enos Chou

Container Security

2. 強化方式

a. 移除非必要套件與程式 (範例)

```
rm -rf \
requirements.txt \
/usr/local/bin/pip{,3,3.12} \
/usr/local/lib/python3.12/site-packages/pip{,-*.dist-info}
```

© 2026 Enos Chou

Container Security

2. 強化方式

b. 更新套件與程式 (範例)

```
pip install --upgrade your_package
```

© 2026 Enos Chou

Container Security

2. 強化方式

c. 更新作業系統 (範例)

```
apt-get upgrade -y
```

© 2026 Enos Chou

Container Security

2. 強化方式

d. Distroless 容器設計 (範例)

Temp Image

```
FROM python:3.13.18-slim-trixie AS builder

WORKDIR /your_module

COPY requirements.txt ./
RUN pip install --no-cache-dir --target=/your_module/pkgs -r requirements.txt gunicorn

# Introduce exec.py to replace CMD environment variable resolution in Distroless
RUN cat << 'EOF' > exec.py
import os
port = os.environ.get("PORT", "8080")
args = ["python", "-m", "gunicorn", "your_module:app", "-b", f":{port}"]
os.execvp("python", args)
EOF
```

Container Security

2. 強化方式

d. Distroless 容器設計 (範例)

Actual Image

```
FROM gcr.io/distroless/python3-debian13:nonroot

WORKDIR /your_module

COPY --from=builder /your_module/pkgs /your_module/pkgs
COPY --from=builder /your_module/exec.py ./
COPY --chown=nonroot:nonroot . ./

ENV PYTHONPATH=/your_module/pkgs \
    TZ=Asia/Taipei \
    LANG=C.UTF-8 \
    PYTHONUNBUFFERED=1

CMD ["exec.py"]
```

© 2026 Enos Chou

AI 應用雲端部署 ft. 容器

容器化部署 ft. AWS

容器化部署 ft. Azure

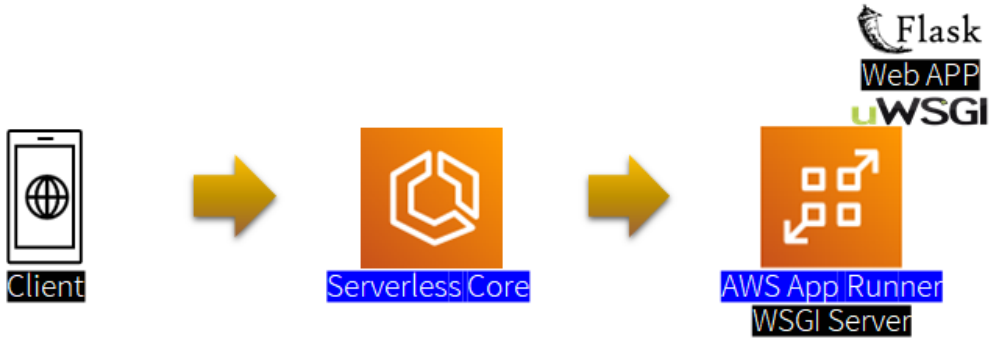
容器化部署 ft. Google Cloud

© 2026 Enos Chou

容器化部署 ft. AWS

AWS App Runner

Containerized/ Serverless



© 2026 Enos Chou

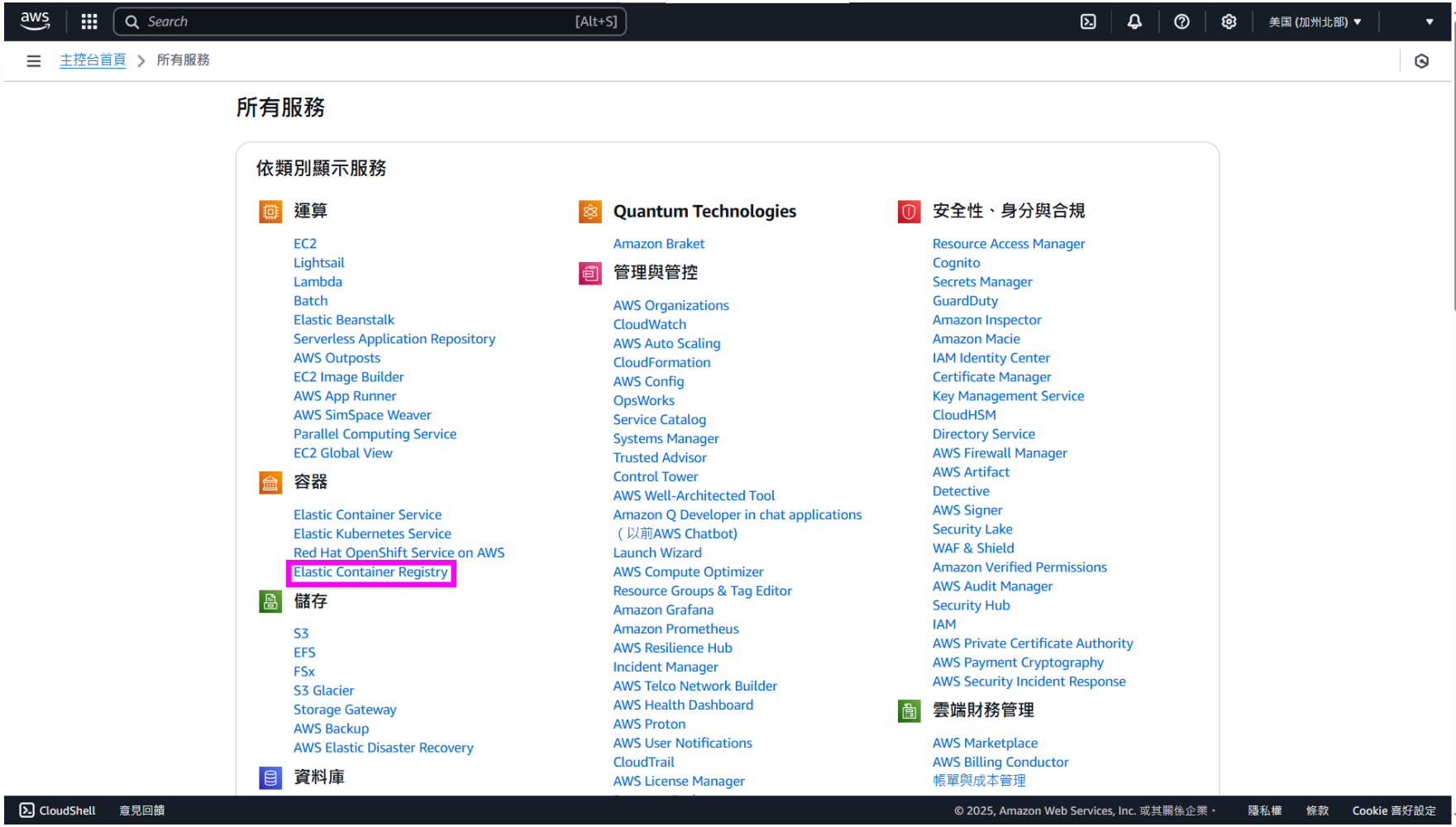
Docker Image into Amazon ECR

1. 建立 Amazon Elastic Container Registry (ECR)
2. 建立並推送 Docker Image 至 Amazon ECR

© 2026 Enos Chou

Docker Image into Amazon ECR

1. (建立 ECR)
   a. 建立 ECR



Docker Image into Amazon ECR

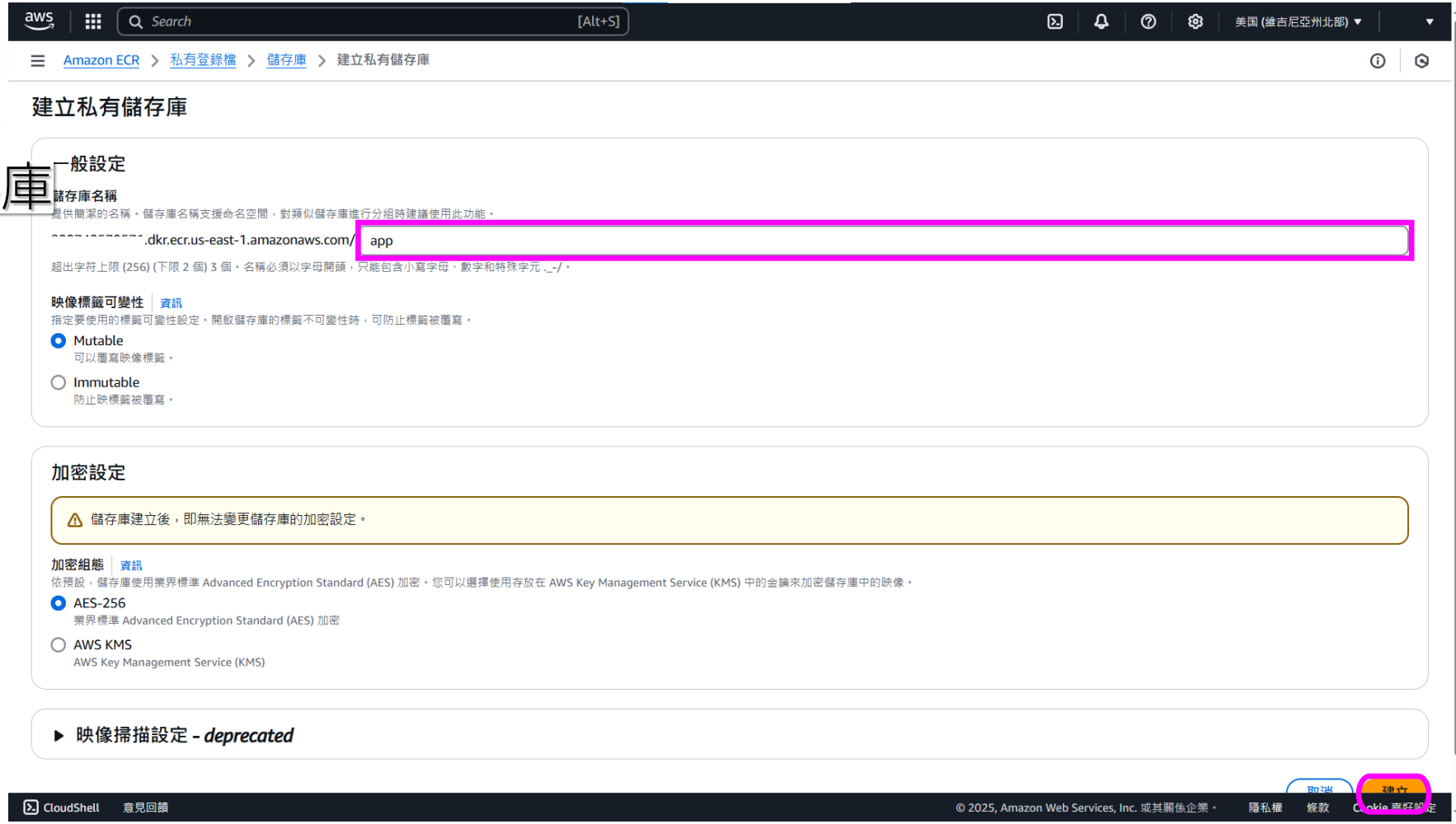
1. (建立 ECR)
   a. 建立 ECR

Docker Image into Amazon ECR

1. (建立 ECR)
   a. 建立 ECR

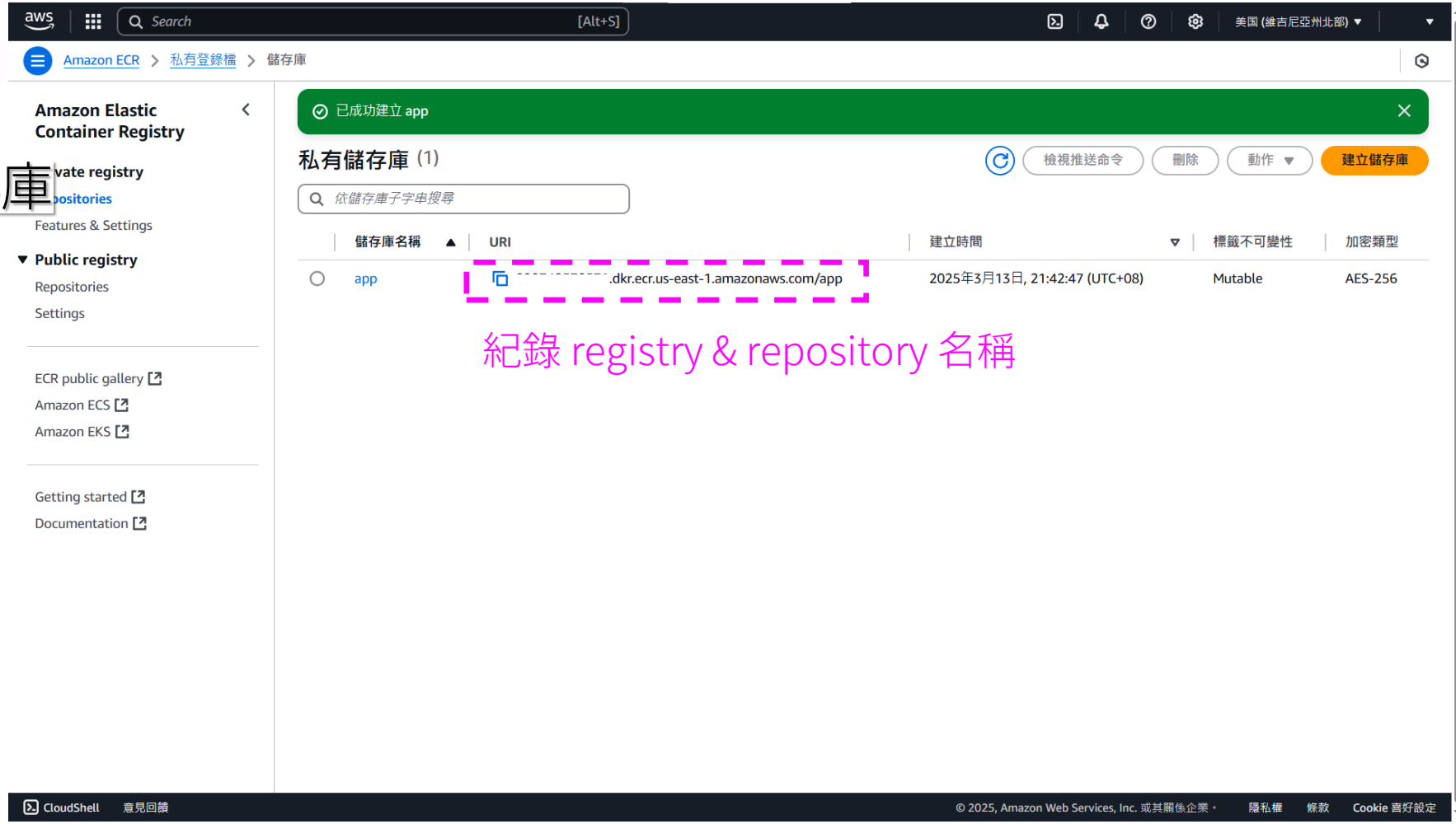
Docker Image into Amazon ECR

1. (建立 ECR)
   a. 建立私有儲存庫



Docker Image into Amazon ECR

1. (建立 ECR)
   a. 建立私有儲存庫

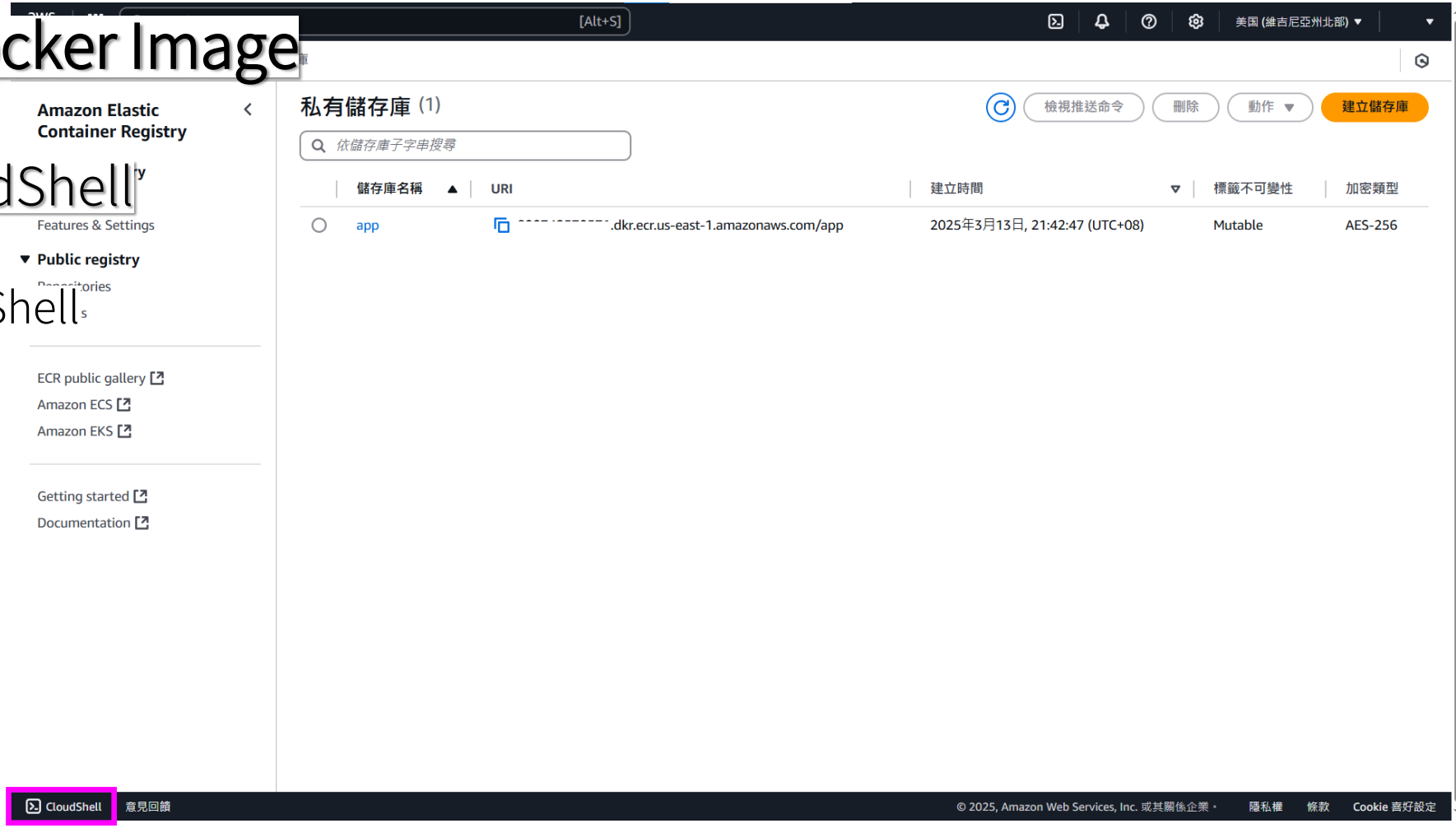


Docker Image into Amazon ECR

2. 建立並推送 Docker Image

a. by AWS CloudShell

① 開啟 CloudShell

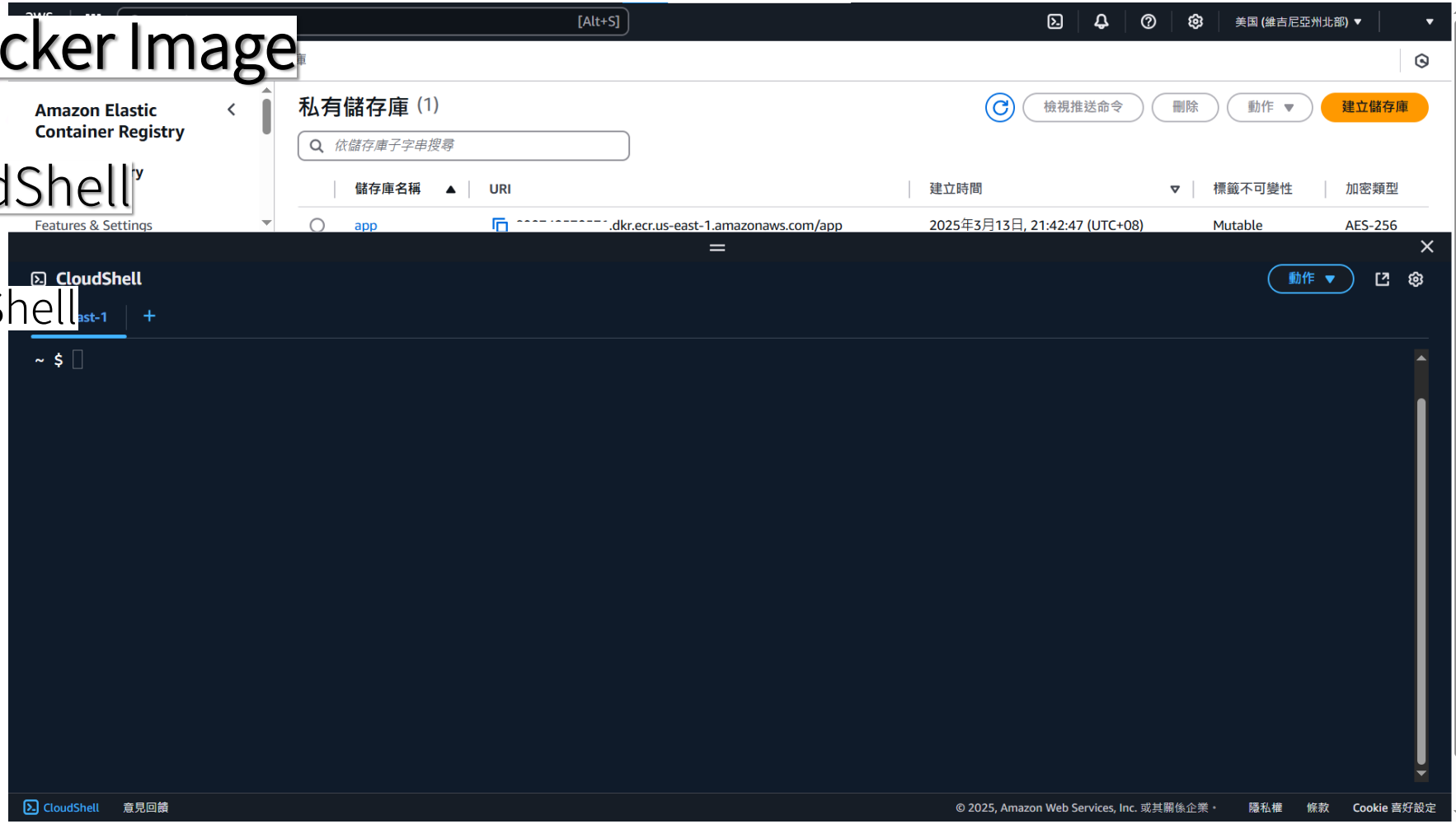


Docker Image into Amazon ECR

2. 建立並推送 Docker Image

a. by AWS CloudShell

① 開啟 CloudShell

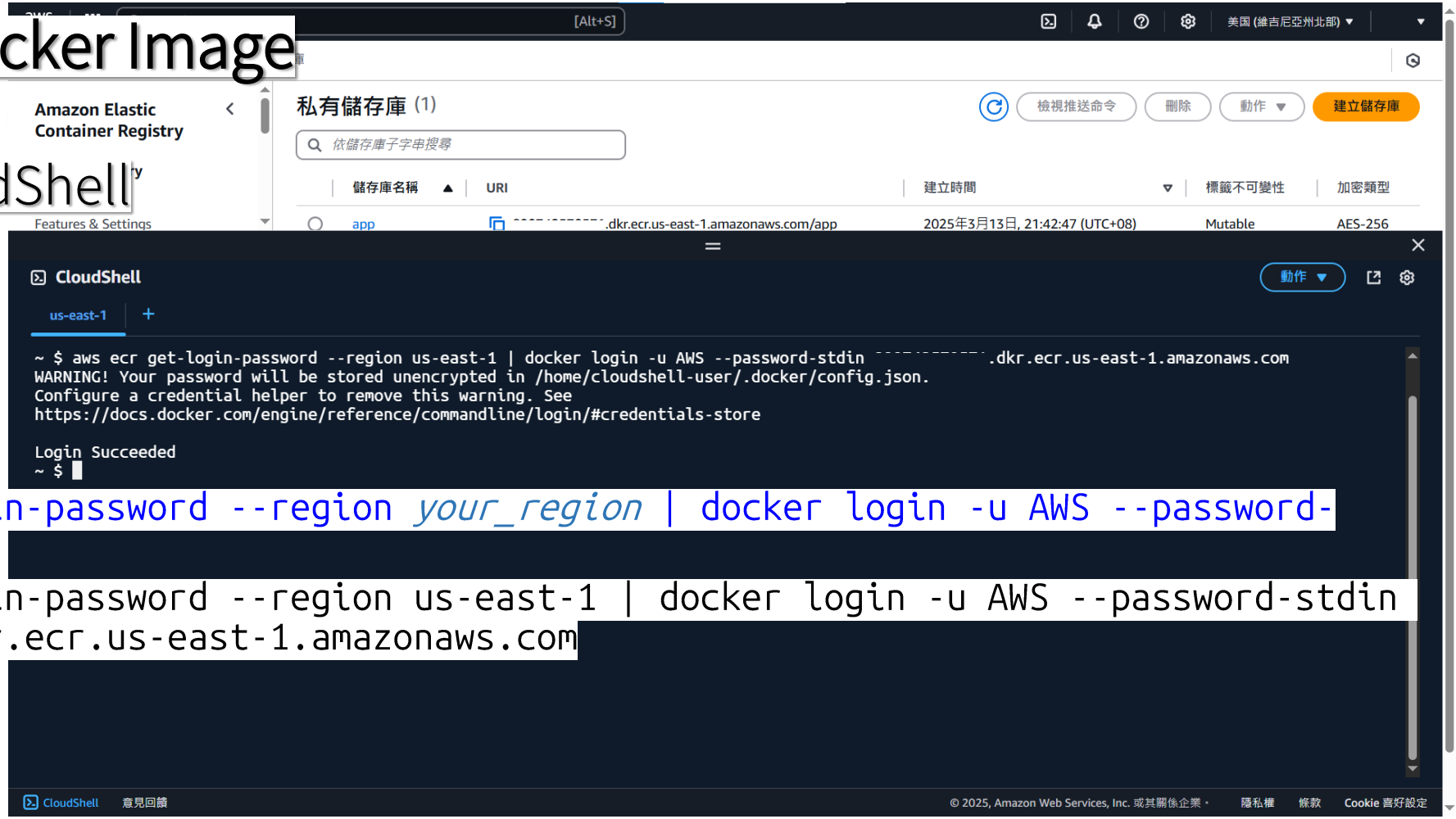


Docker Image into Amazon ECR

2. 建立並推送 Docker Image

a. by AWS CloudShell

② 登入 ECR




```
aws ecr get-login-password --region your_region | docker login -u AWS --password-stdin your_reg
```

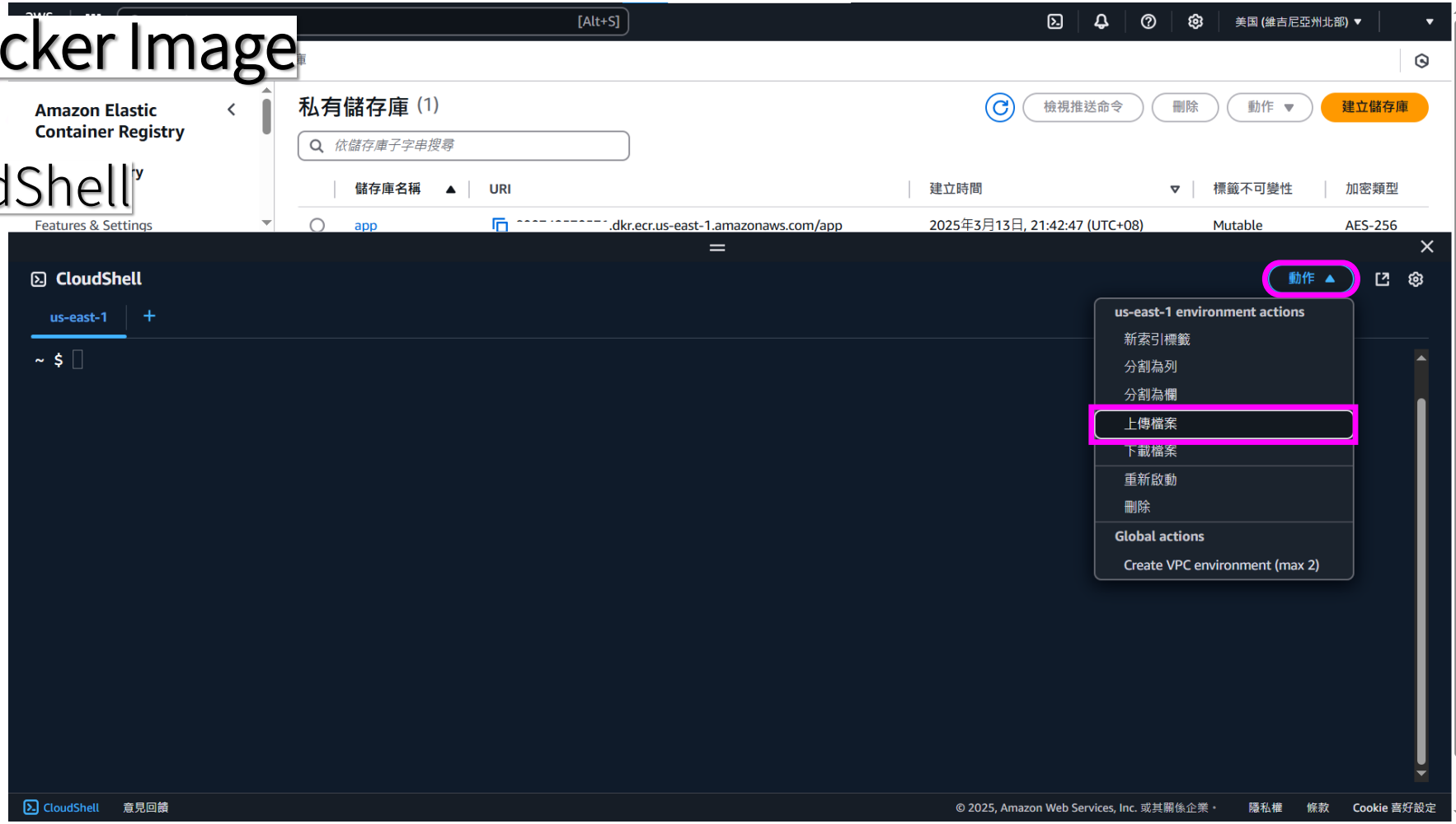
```
aws ecr get-login-password --region us-east-1 | docker login -u AWS --password-stdin 890742570571.dkr.ecr.us-east-1.amazonaws.com
```

Docker Image into Amazon ECR

2. 建立並推送 Docker Image

a. by AWS CloudShell

③ 上傳 source



Docker Image into Amazon ECR

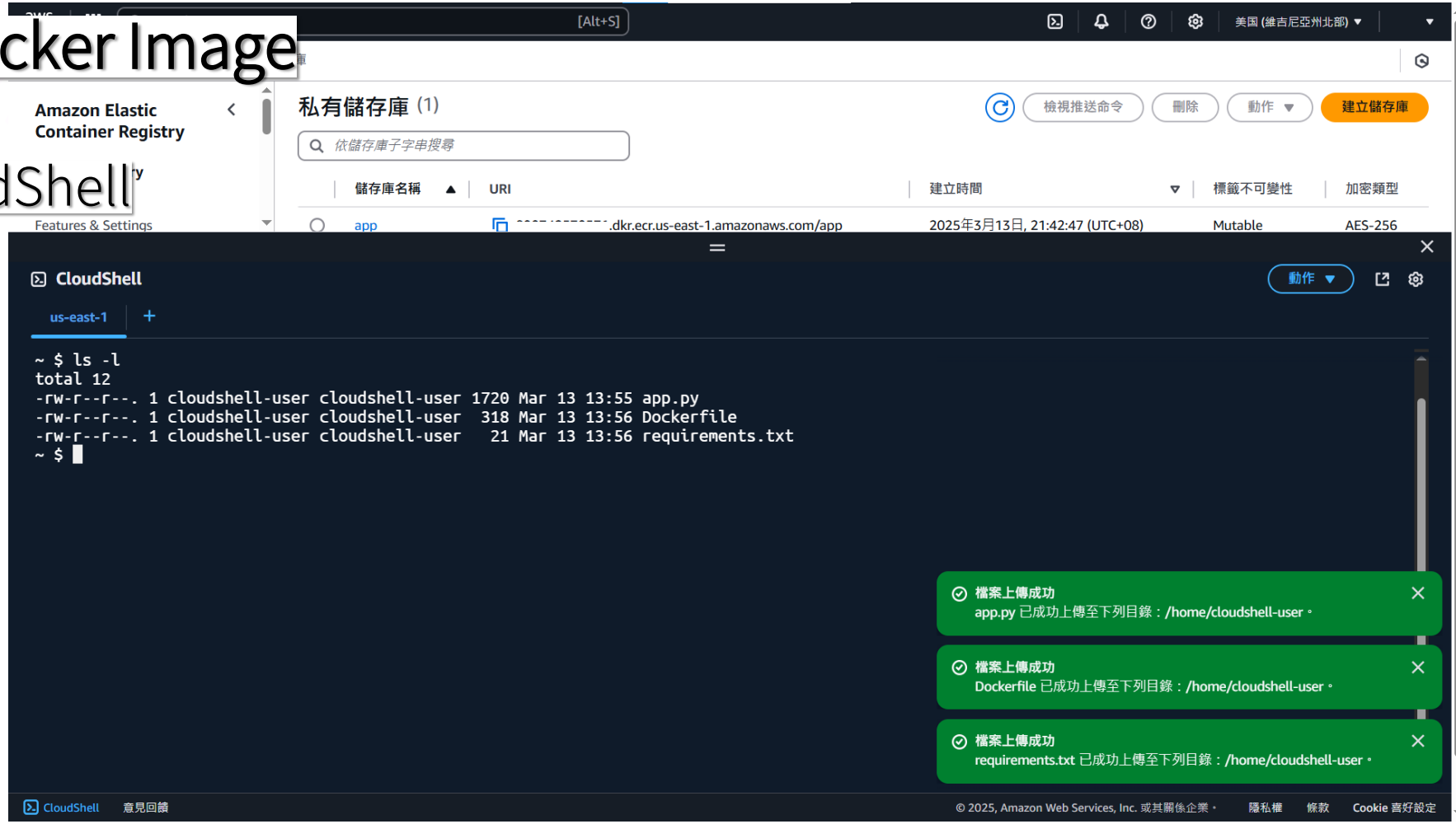
2. 建立並推送 Docker Image

a. by AWS CloudShell

③ 上傳 source

確認成功上傳

```
ls -l
```



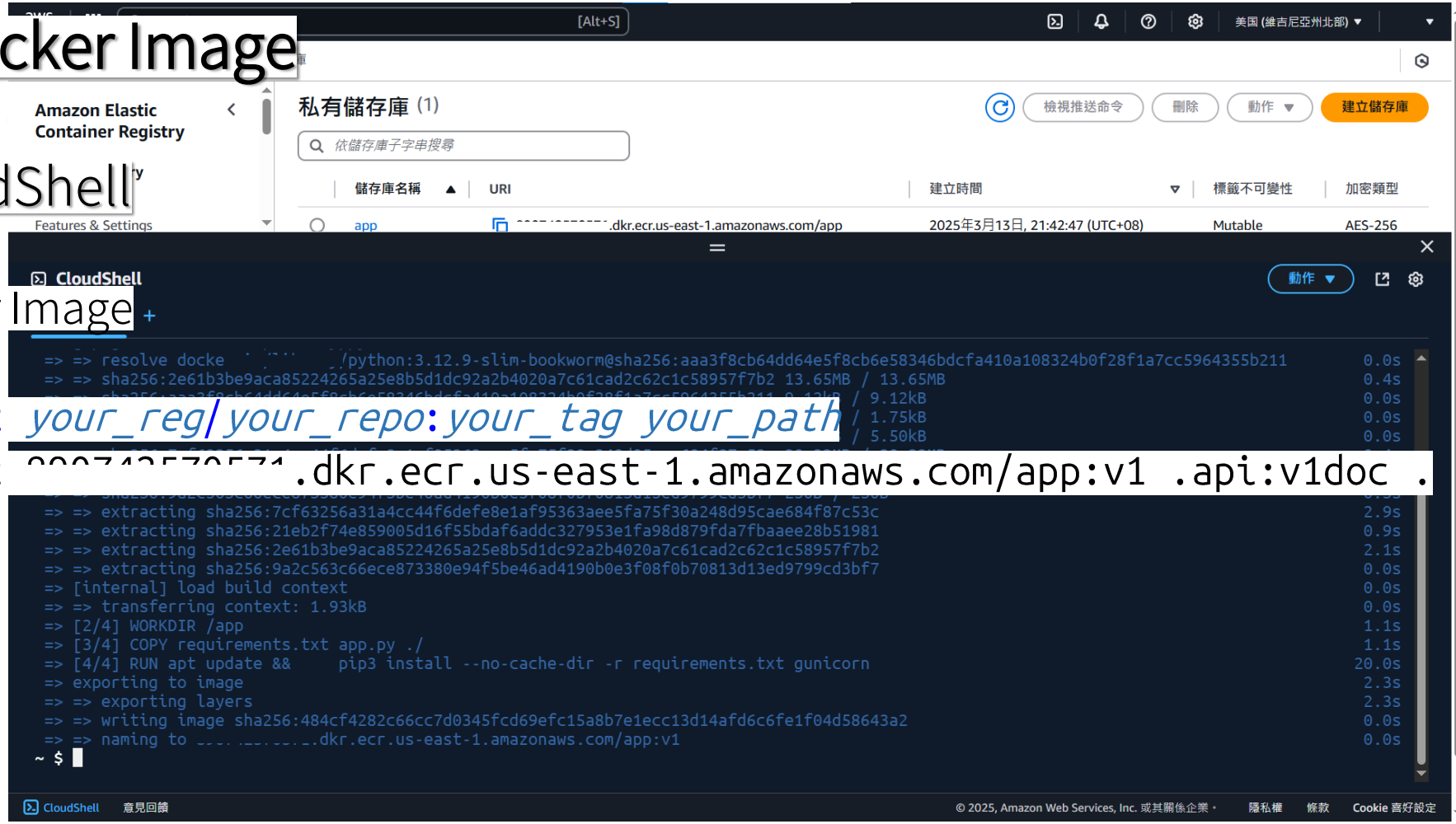
Docker Image into Amazon ECR

2. 建立並推送 Docker Image

a. by AWS CloudShell

④ 建立 Docker Image

```
docker build -t your_reg/your_repo:your_tag your_path
docker build -t 999742557571.dkr.ecr.us-east-1.amazonaws.com/app:v1 .api:v1doc .
```



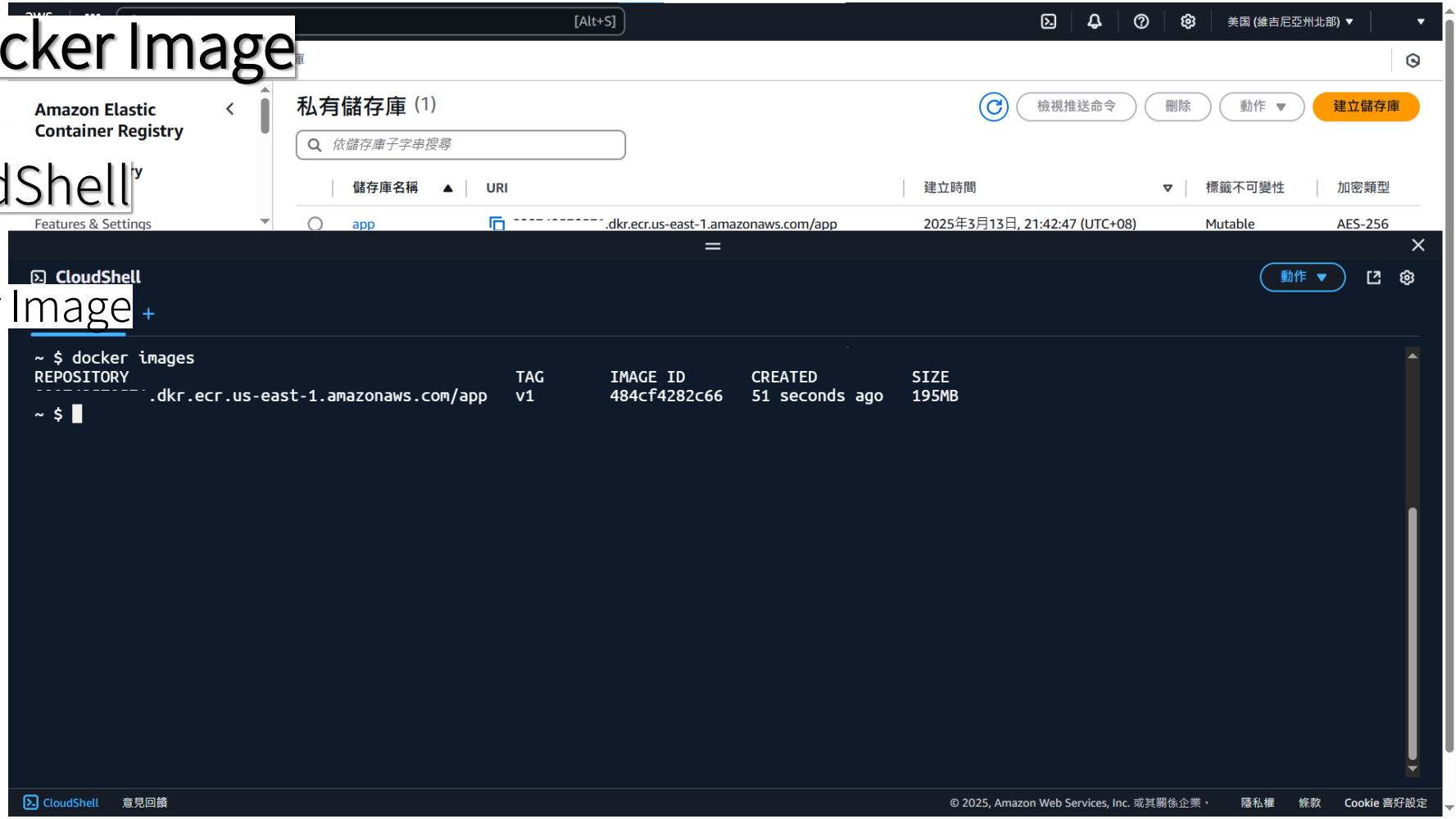
Docker Image into Amazon ECR

2. 建立並推送 Docker Image

a. by AWS CloudShell

④ 建立 Docker Image

確認成功建立
```
docker images
```

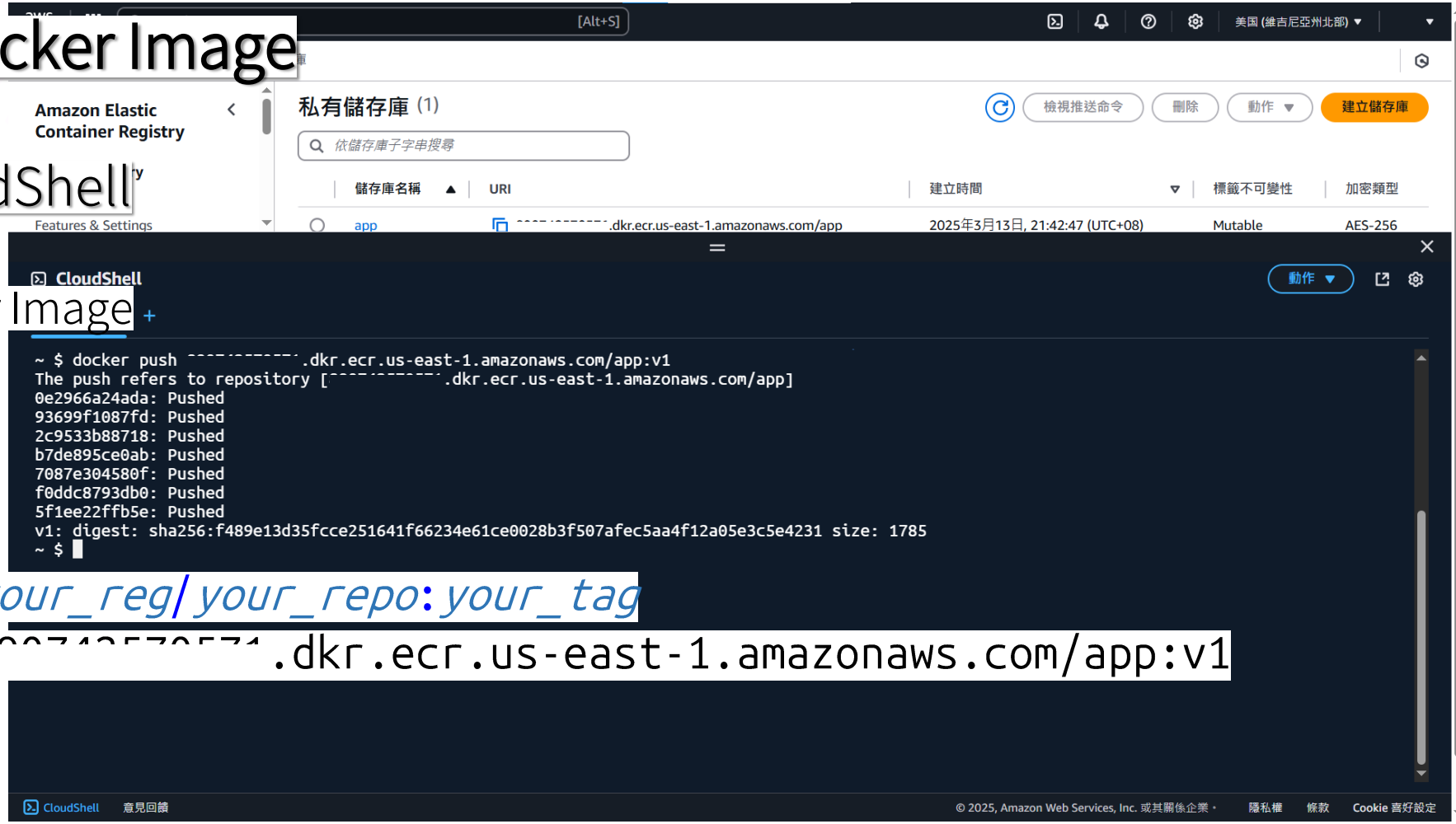


Docker Image into Amazon ECR

2. 建立並推送 Docker Image

a. by AWS CloudShell

⑤ 上傳 Docker Image



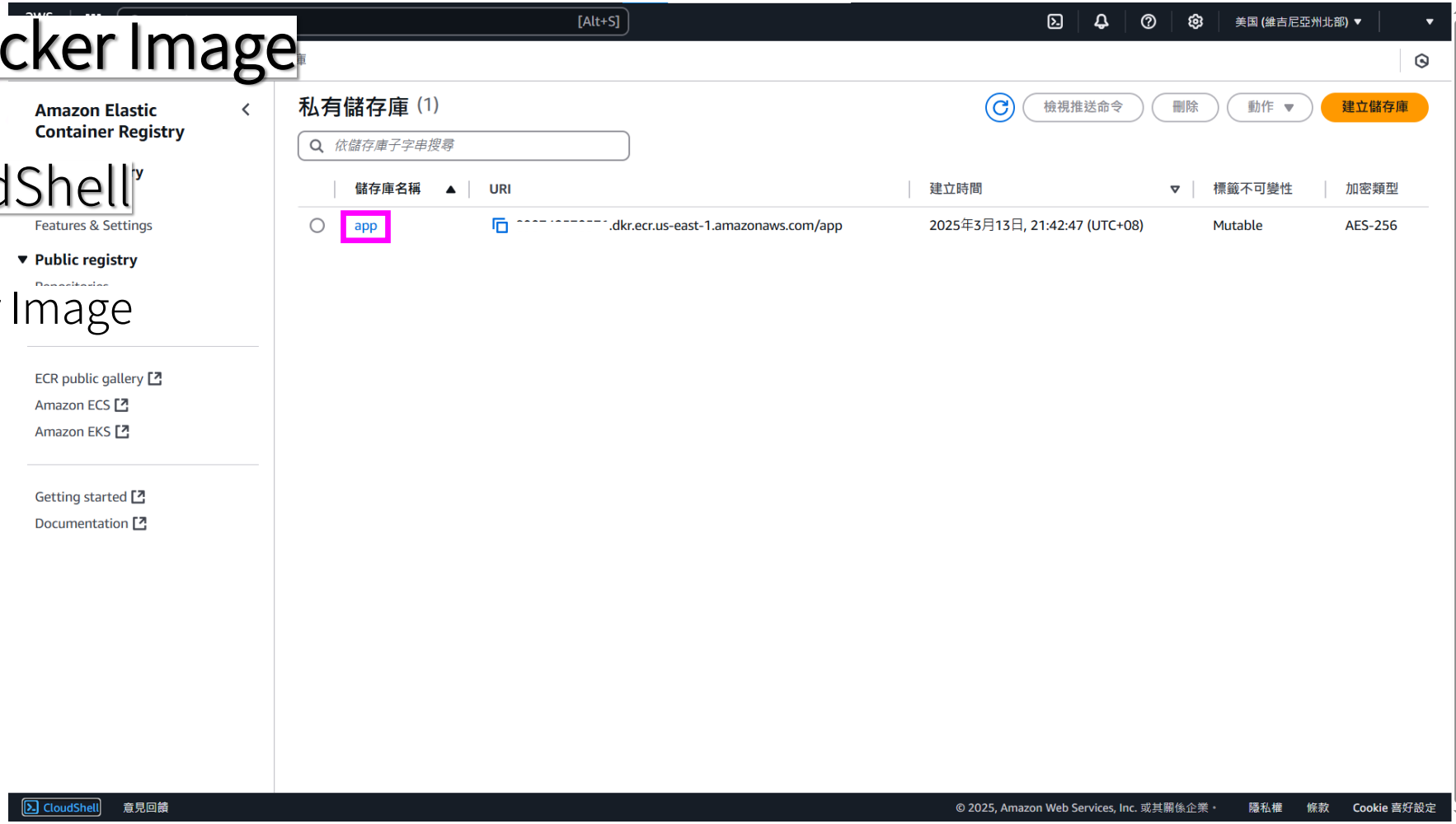

```
docker push your_reg/your_repo:your_tag
docker push 200742570571.dkr.ecr.us-east-1.amazonaws.com/app:v1
```

Docker Image into Amazon ECR

2. 建立並推送 Docker Image

a. by AWS CloudShell

⑤ 上傳 Docker Image

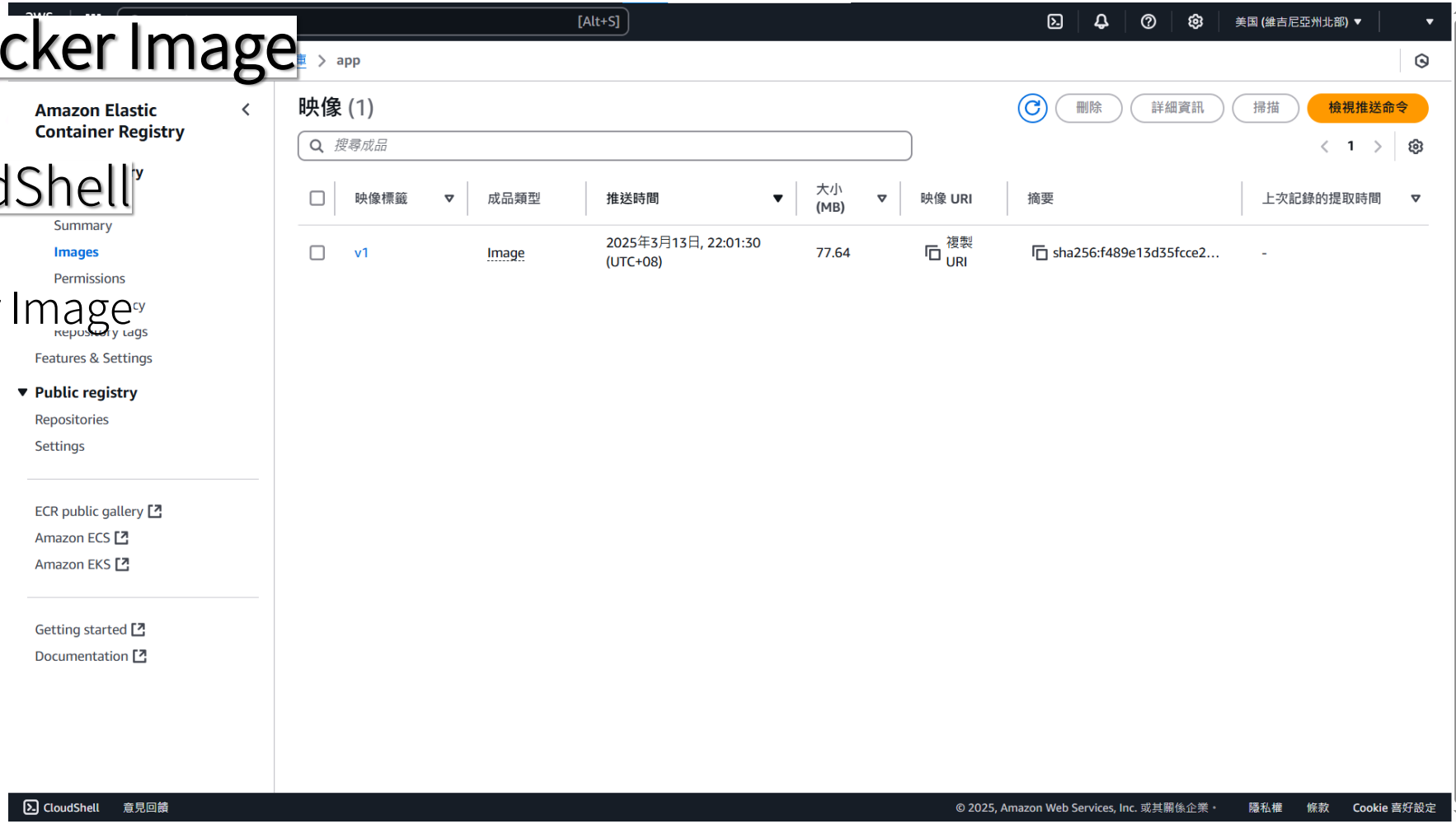


Docker Image into Amazon ECR

2. 建立並推送 Docker Image

a. by AWS CloudShell

⑤ 上傳 Docker Image



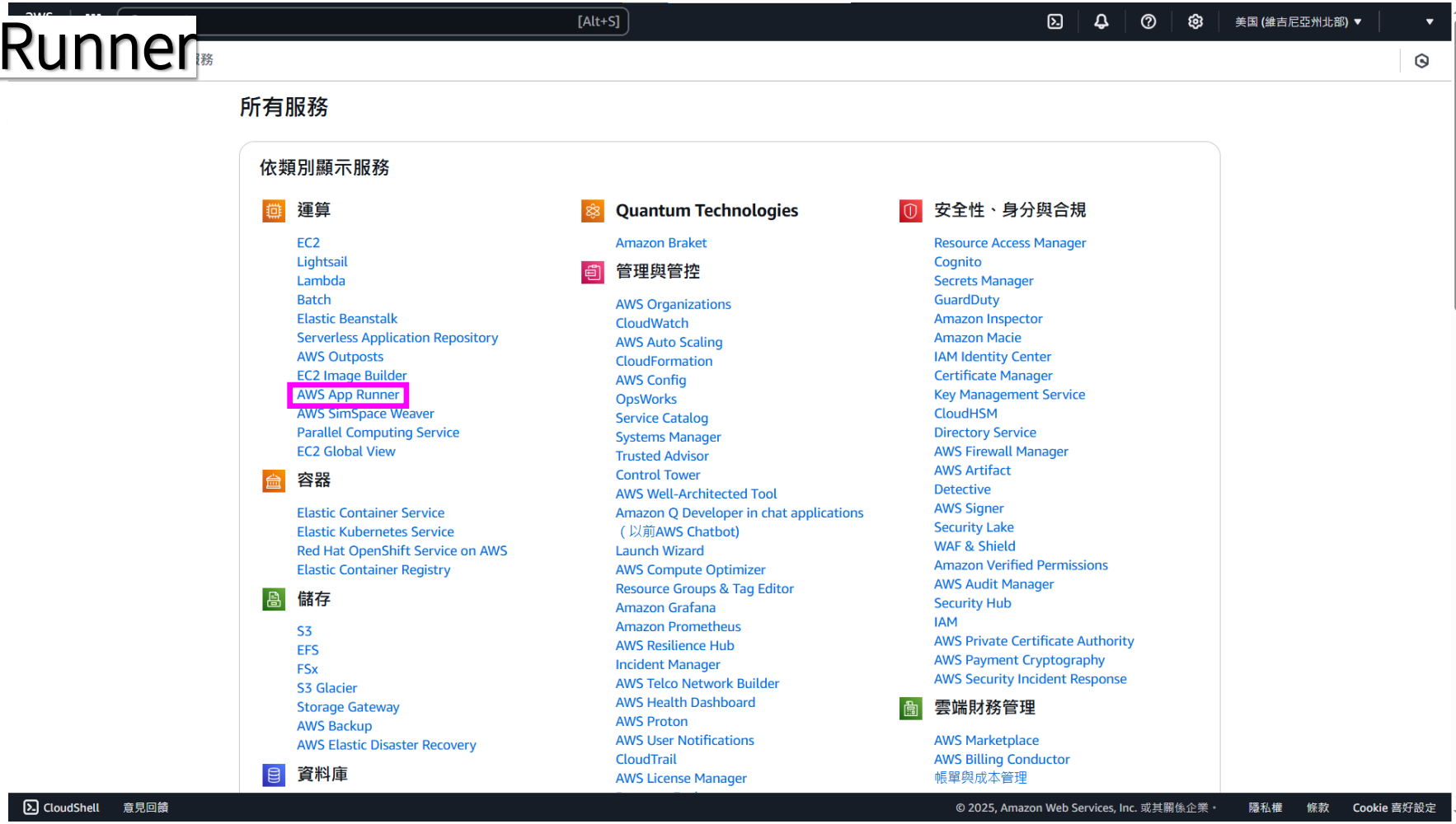
APP 部署 ft. AWS App Runner

1. 建立 AWS App Runner
2. 測試與除錯

© 2026 Enos Chou

APP 部署 ft. AWS App Runner

1. 建立 AWS App Runner



APP 部署 ft. AWS App Runner

1. 建立 AWS App Runner

APP 部署 ft. AWS App Runner

1. 建立 AWS App Runner

APP 部署 ft. AWS App Runner

1. 建立 AWS App Runner

APP 部署 ft. AWS App Runner

1. 建立 AWS App Runner

APP 部署 ft. AWS App Runner

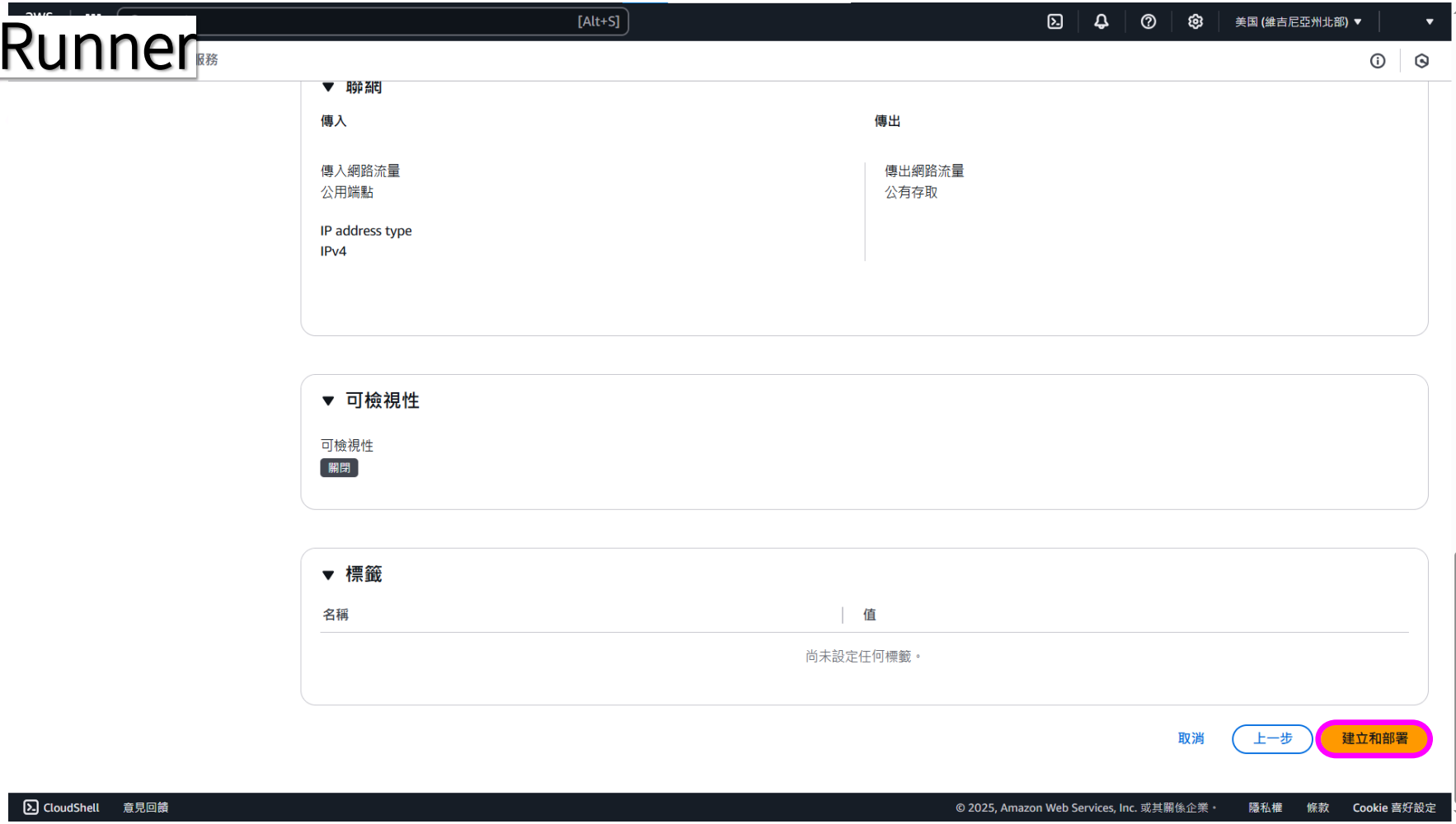
1. 建立 AWS App Runner

APP 部署 ft. AWS App Runner

1. 建立 AWS App Runner

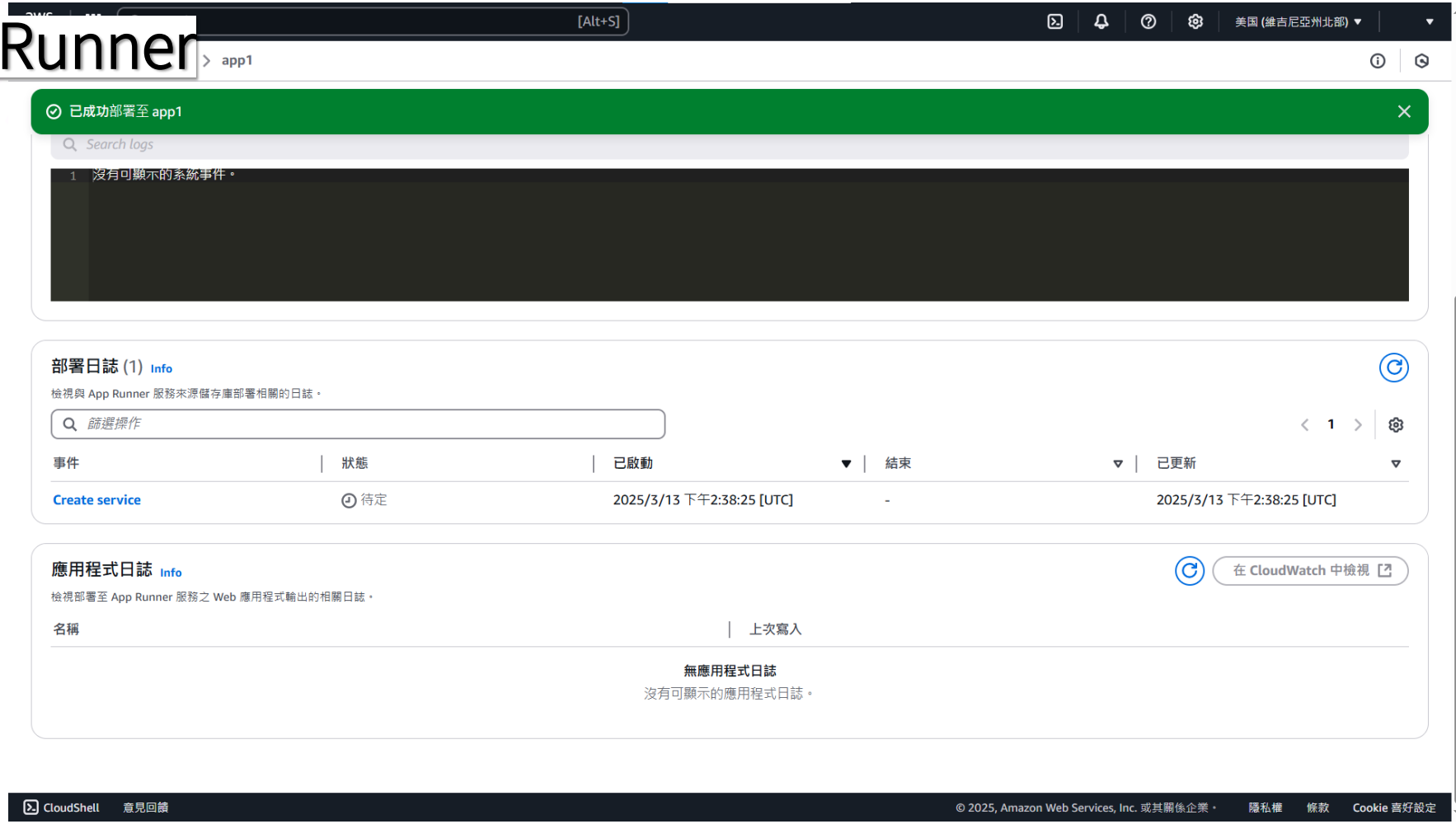
APP 部署 ft. AWS App Runner

1. 建立 AWS App Runner



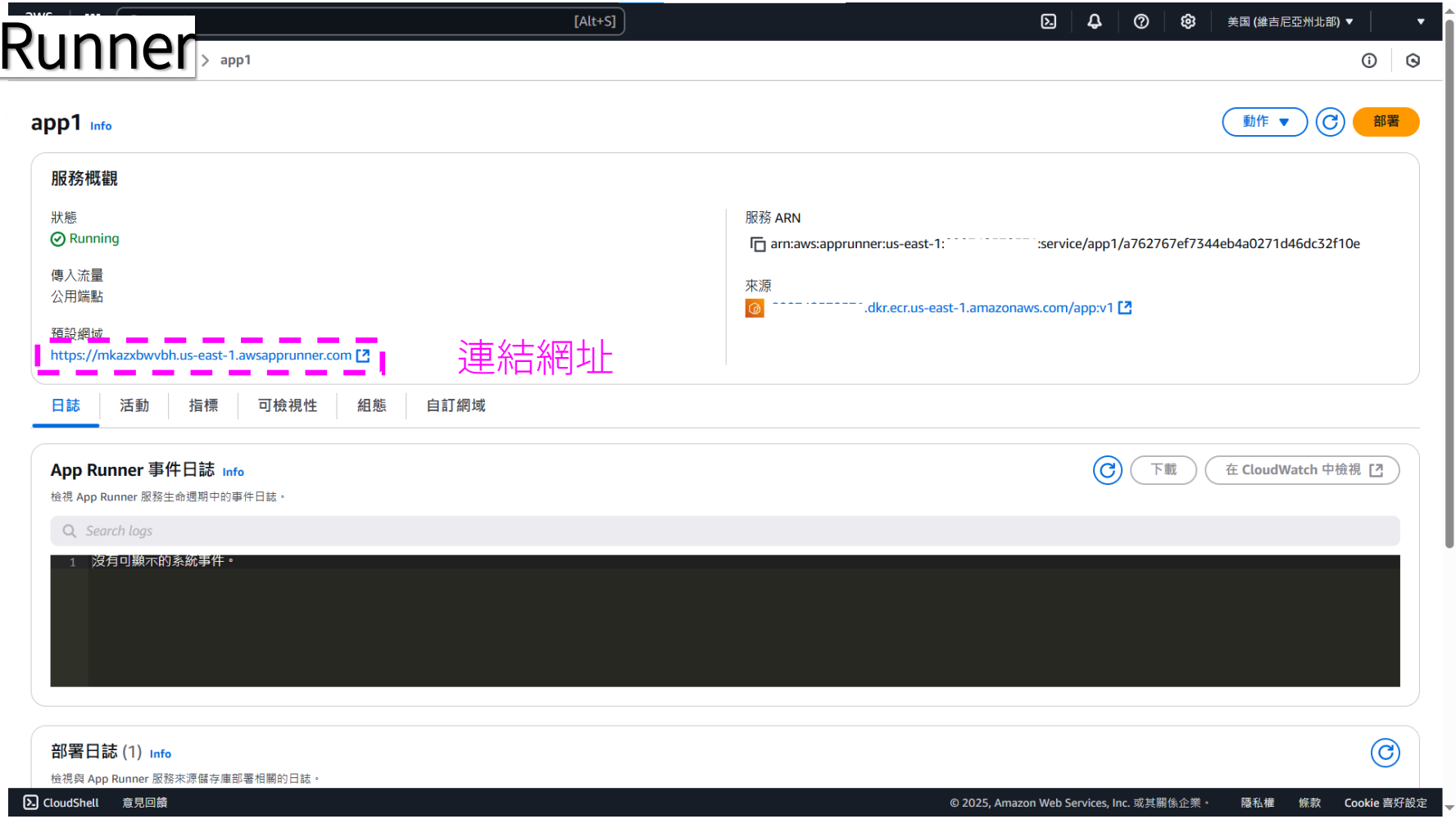
APP 部署 ft. AWS App Runner

1. 建立 AWS App Runner



APP 部署 ft. AWS App Runner

1. 建立 AWS App Runner



APP 部署 ft. AWS App Runner

2. 測試與除錯

a. 測試

```
# API       - 以 Postman 連結 https://your_domain 測試
# Web       - 以瀏覽器連結 https://your_domain 測試
# LINE Bot  - 調整 LINE Bot Webhook URL 為 https://your_domain 測試
```

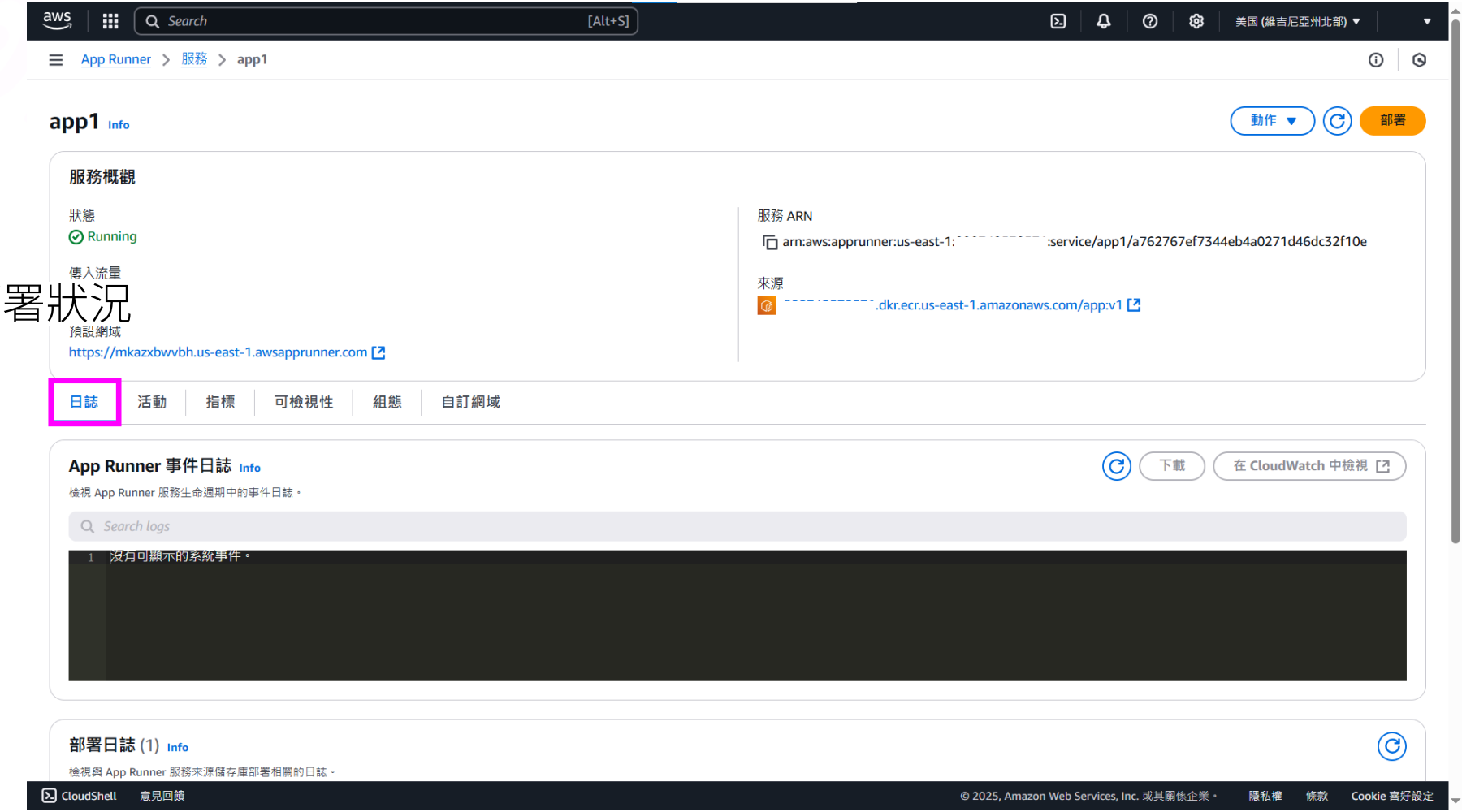
© 2026 Enos Chou

APP 部署 ft. AWS App Runner

2. 測試與除錯

b. 除錯

① 觀察容器部署狀況

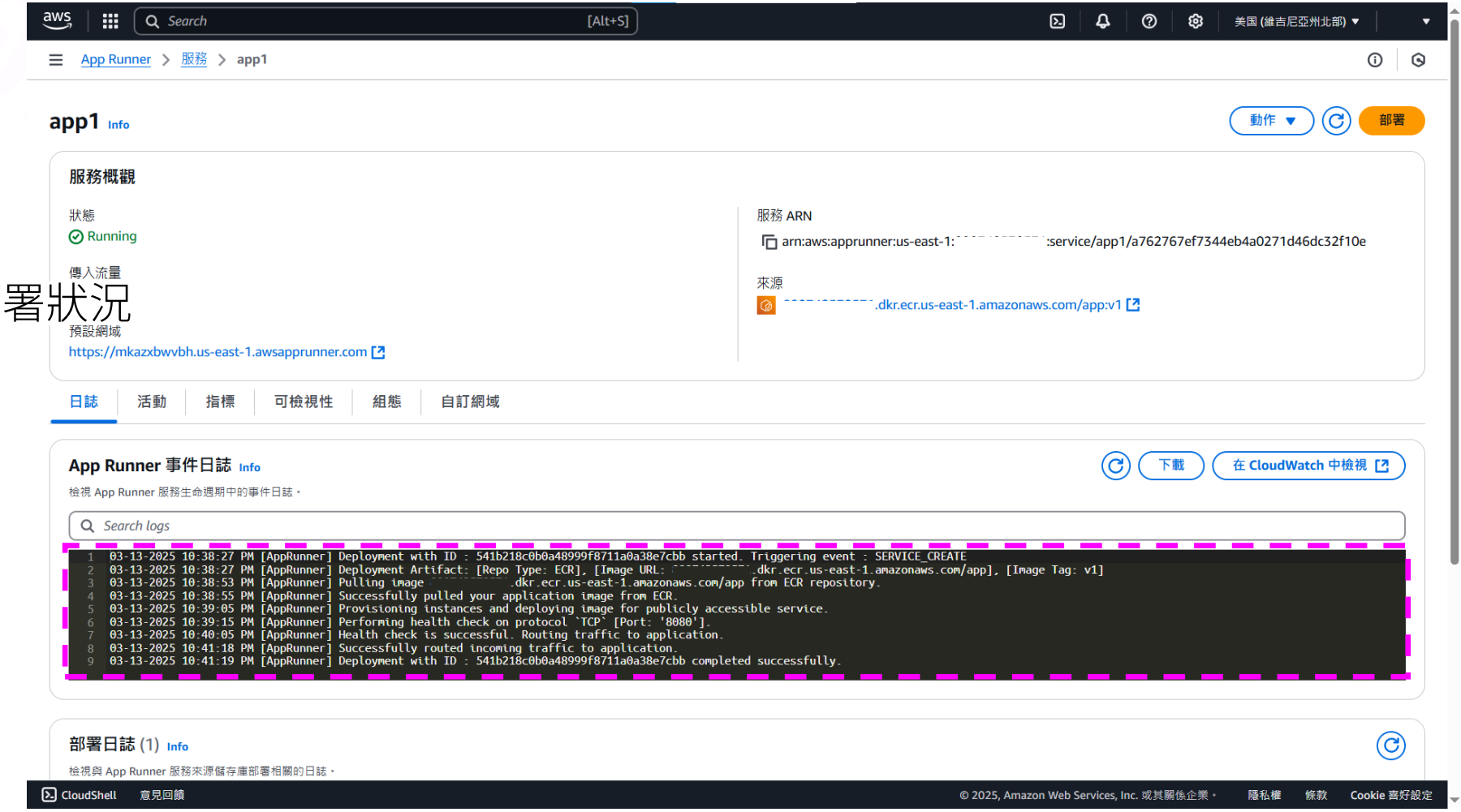


APP 部署 ft. AWS App Runner

2. 測試與除錯

b. 除錯

① 觀察容器部署狀況

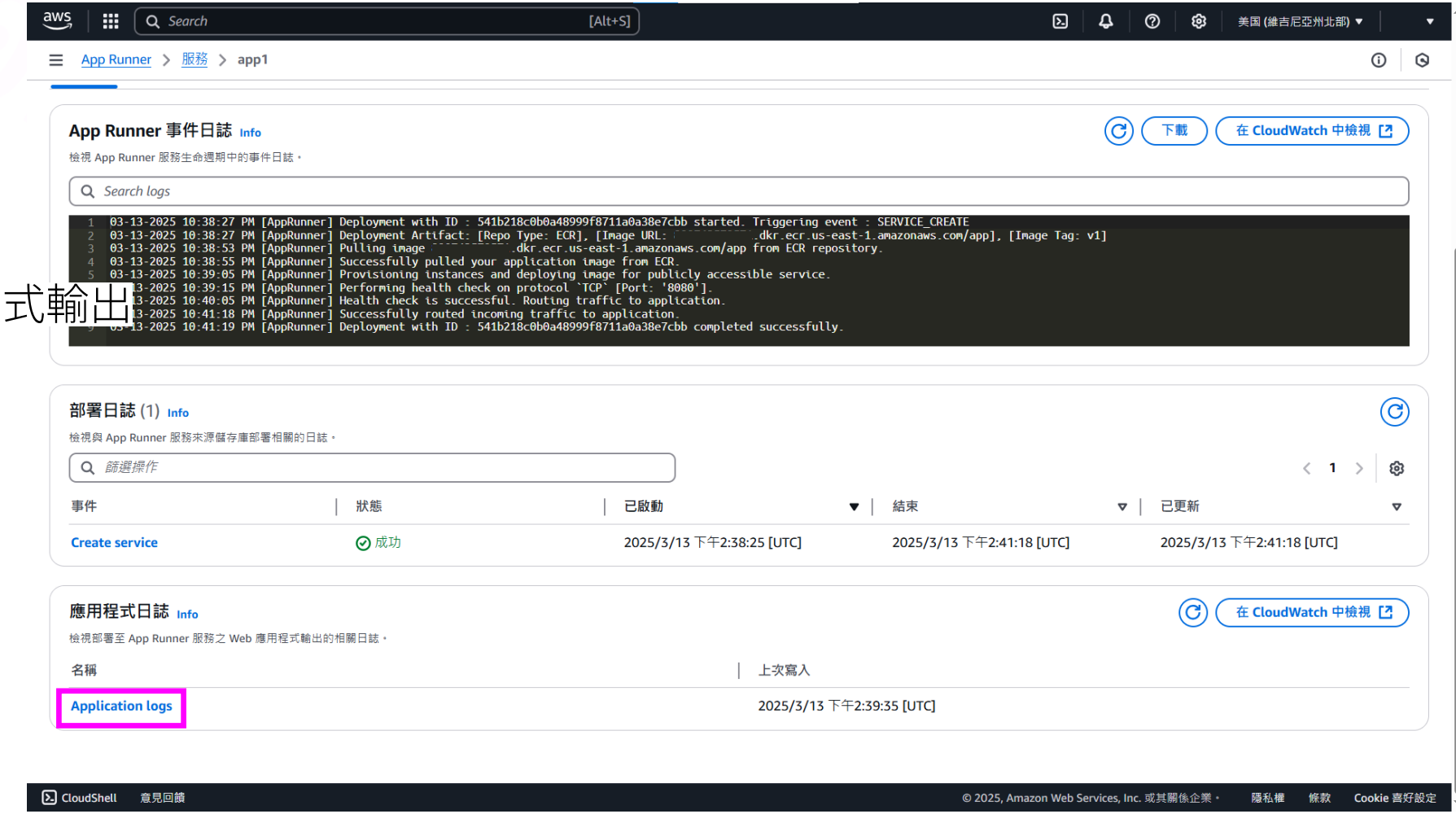


APP 部署 ft. AWS App Runner

2. 測試與除錯

b. 除錯

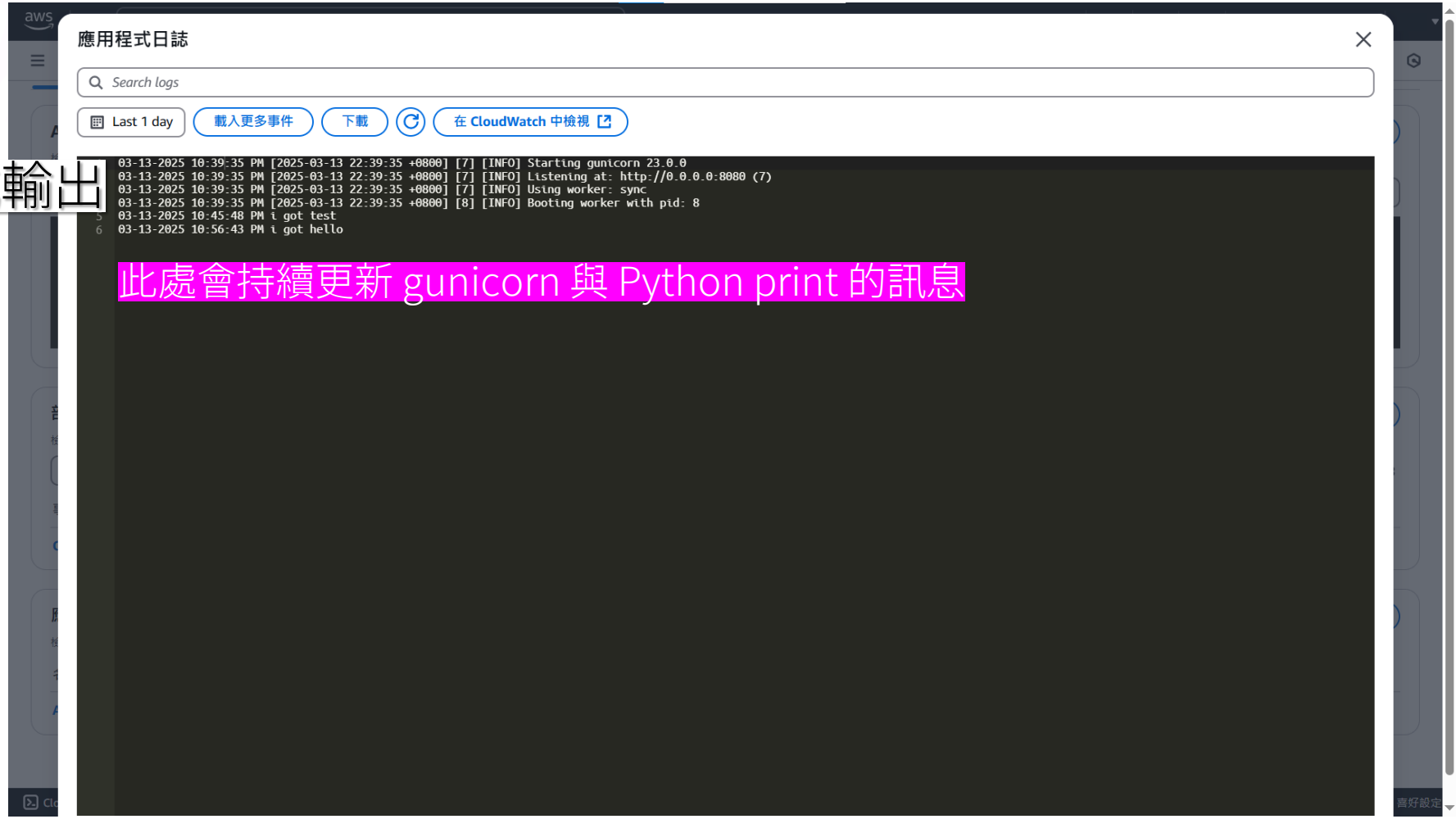
② 觀察應用程式輸出



APP 部署 ft. AWS App Runner

2. 測試與除錯

b. 觀察應用程式輸出

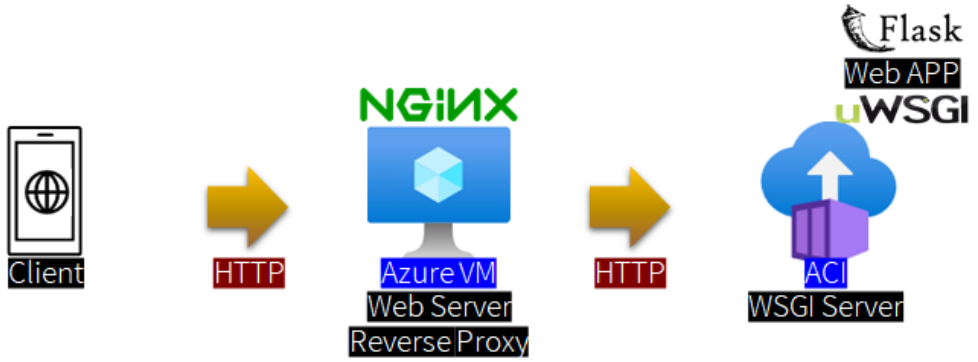


© 2026 Enos Chou

容器化部署 ft. Azure

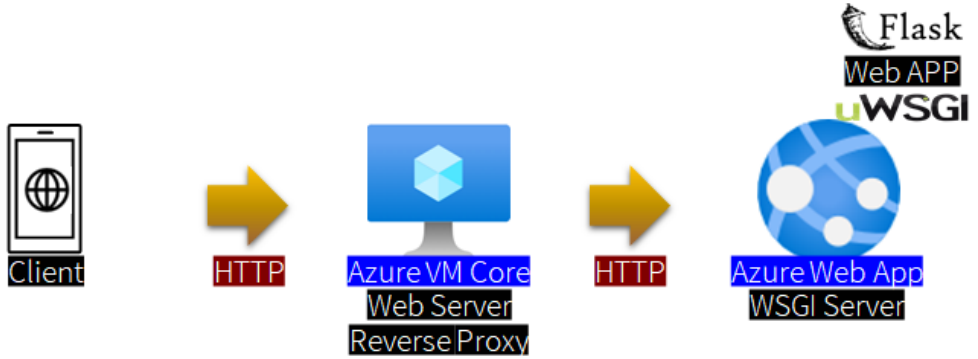
Azure Container Instance (ACI)

Containerized VM



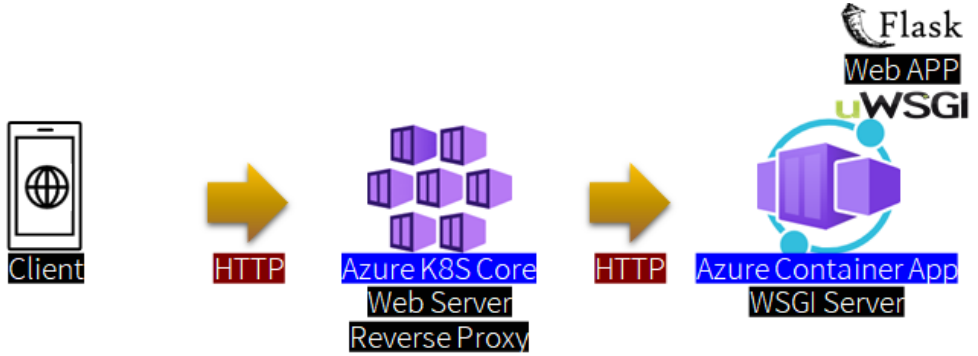
Azure Web App (App Service)

Containerized/ VM-based



Azure Container App

Containerized/ Serverless



© 2026 Enos Chou

Docker Image into ACR

1. 建立容器登錄 Azure Container Registry (ACR)
2. 建立並推送 Docker Image 至 ACR

© 2026 Enos Chou

Docker Image into ACR

1. (建立 ACR)
   a. 建立容器登錄

Docker Image into ACR

1. (建立 ACR)
   a. 建立容器登錄

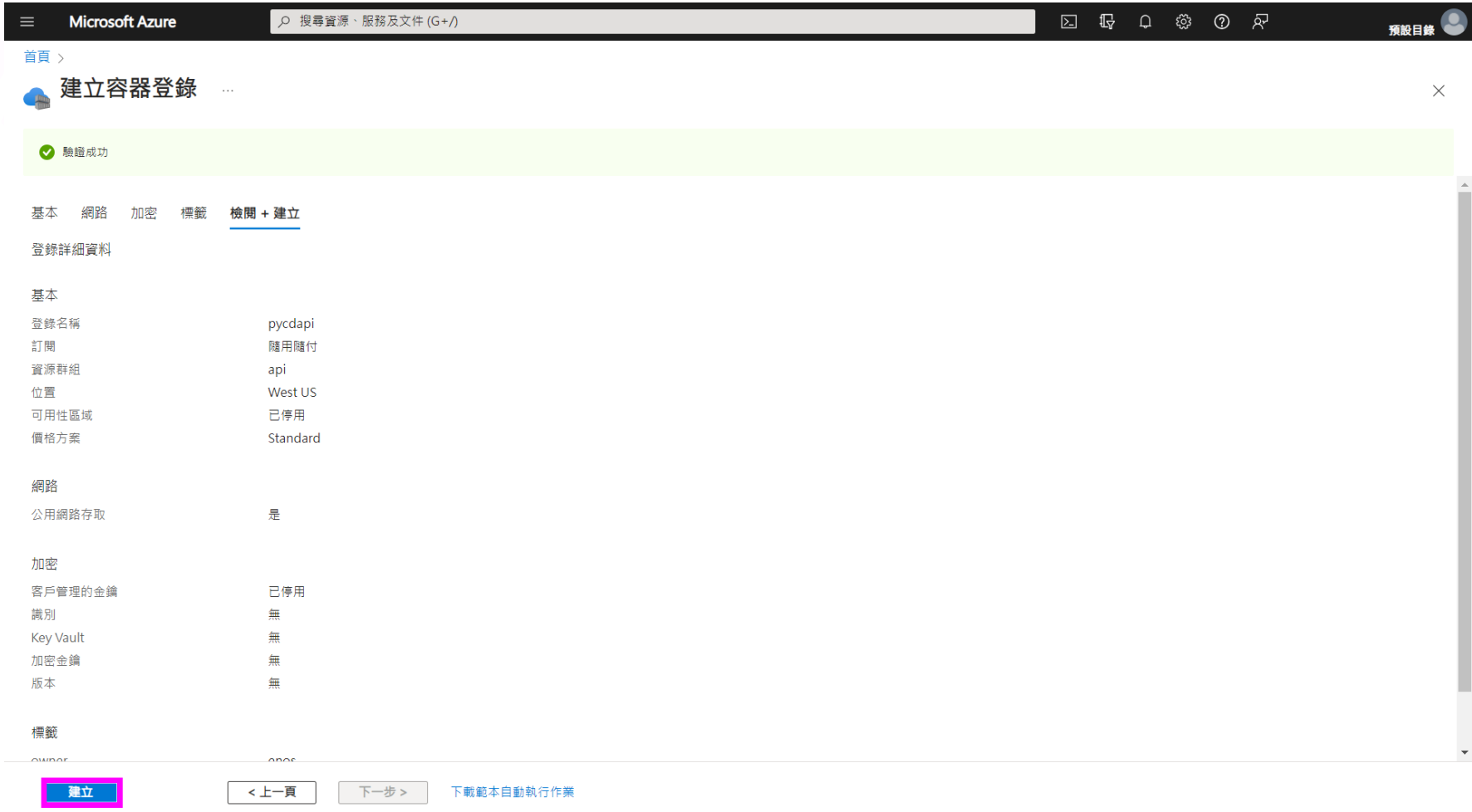
Docker Image into ACR

1. (建立 ACR)
   a. 建立容器登錄

視需要加入識別用標籤

Docker Image into ACR

1. (建立 ACR)
   a. 建立容器登錄



Docker Image into ACR

1. (建立 ACR)
   a. 建立容器登錄

Docker Image into ACR

1. (建立 ACR)

   b. 允許存取

      ① 進入已建立的容器登錄

Docker Image into ACR

1. (建立 ACR)

   b. 允許存取

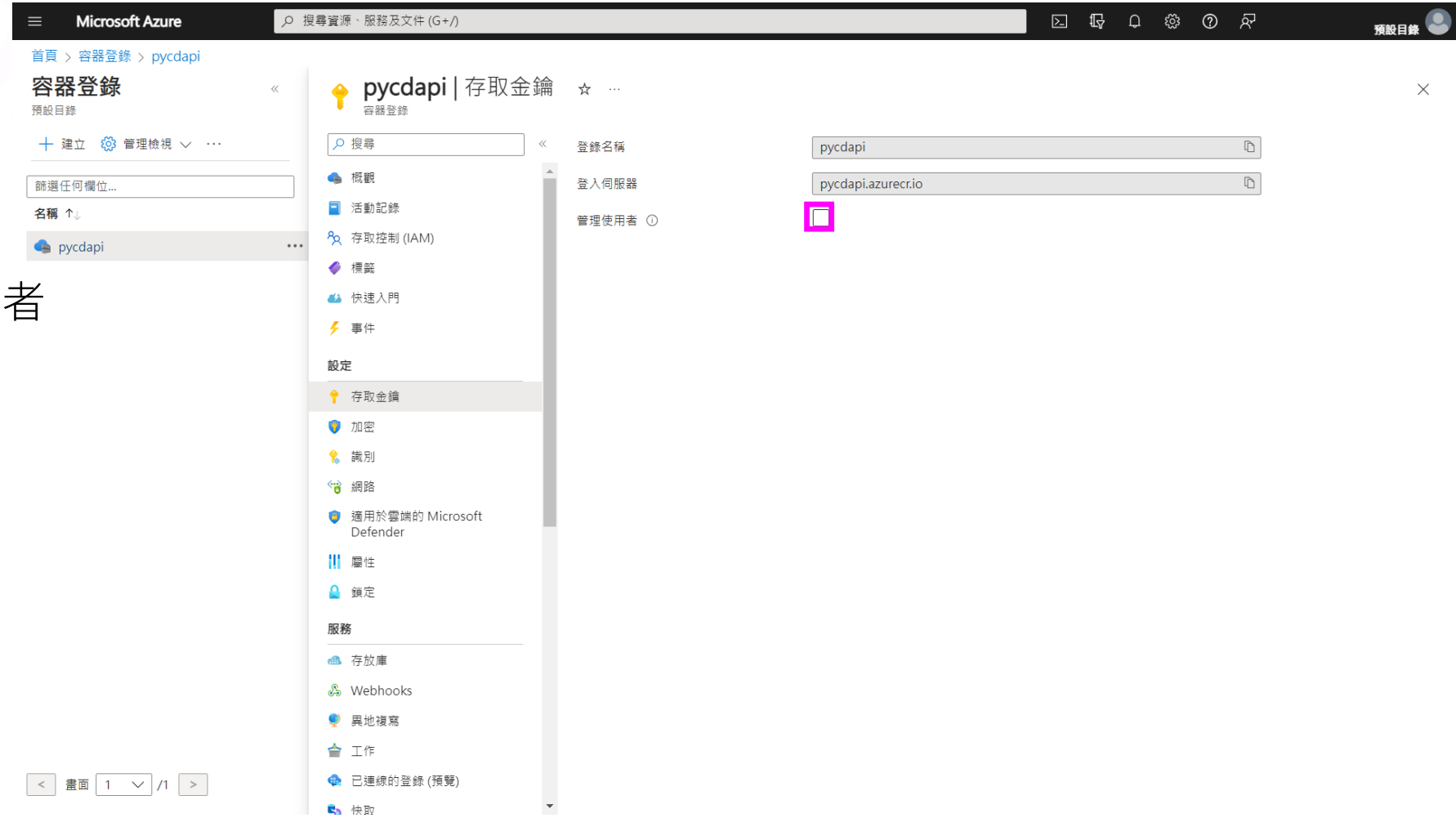
      ② 存取金鑰

Docker Image into ACR

1. (建立 ACR)

   b. 允許存取

      ③ ☑ 管理使用者

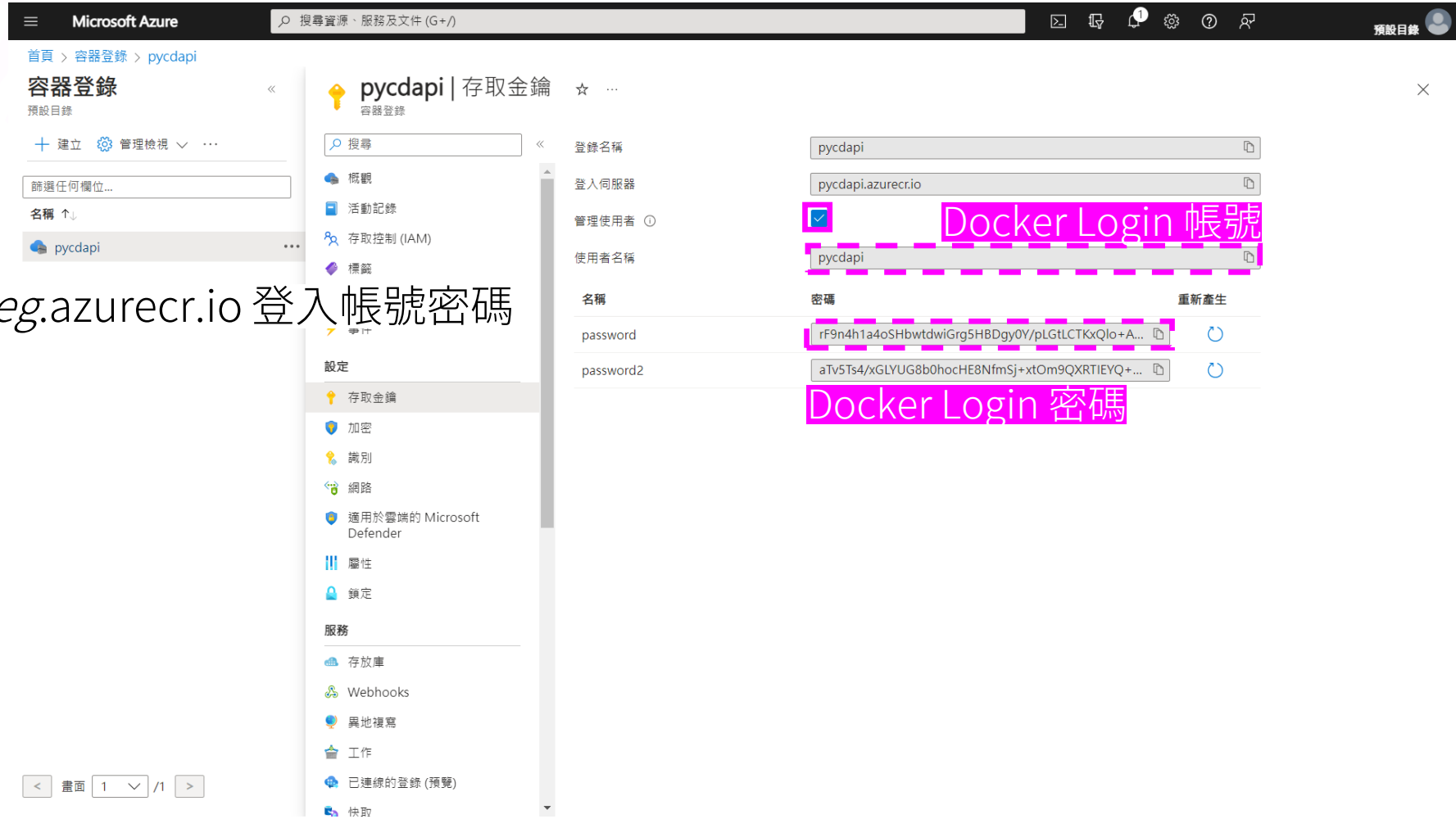


Docker Image into ACR

1. (建立 ACR)

   b. 允許存取

      ④ 取得 *your_reg*.azurecr.io 登入帳號密碼



Docker Image into ACR

2. 建立並推送 Docker Image

a. by Docker

① 建立 Docker Image

```
docker build -t your_reg.azurecr.io/your_repo:your_tag your_path
docker build -t pycdapi.azurecr.io/api:v1doc .
```

© 2026 Enos Chou

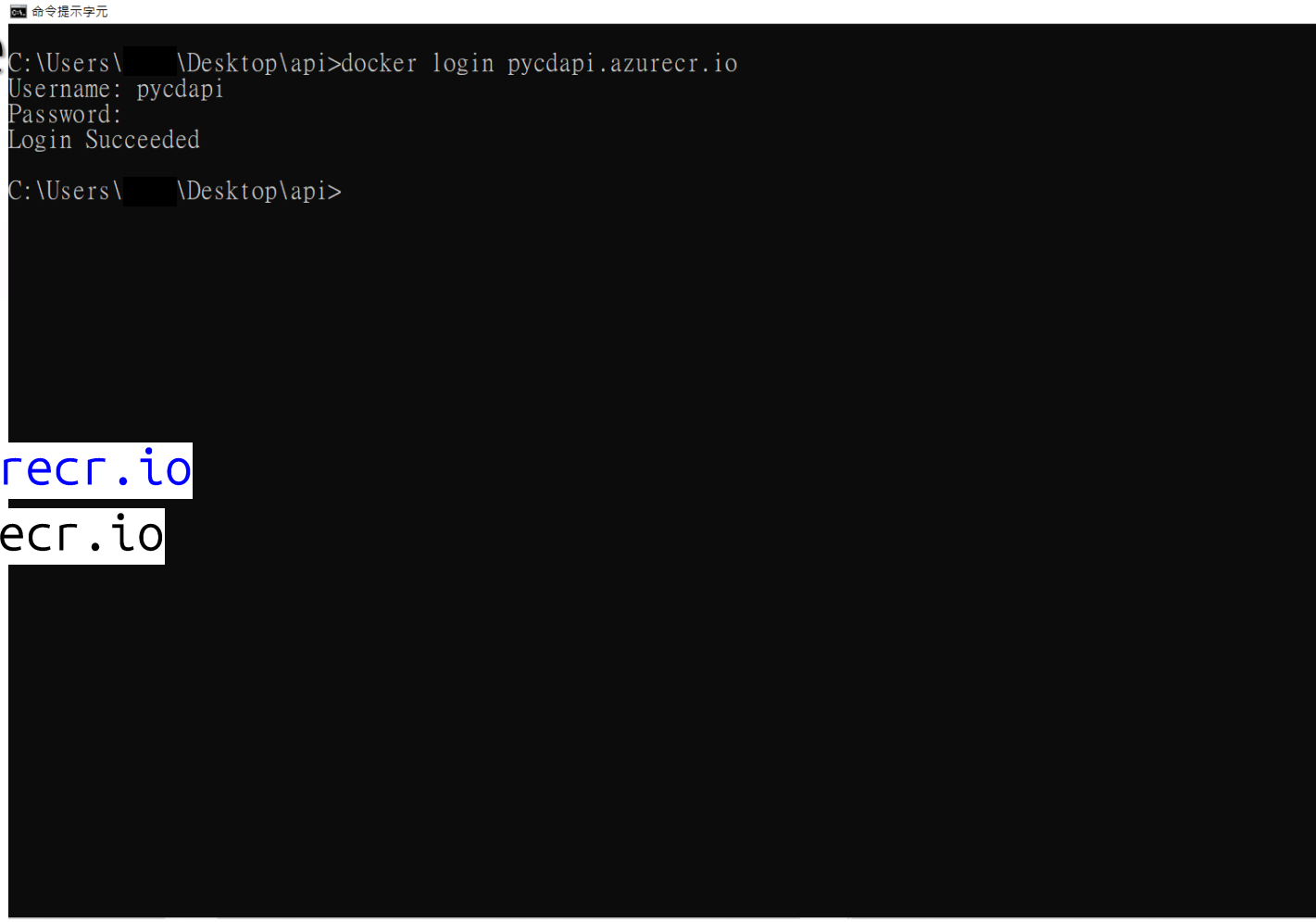
Docker Image into ACR

2. 建立並推送 Docker Image

a. by Docker

② 登入 *your_reg*.azurecr.io

```
docker login your_reg.azurecr.io
docker login pycdapi.azurecr.io
```



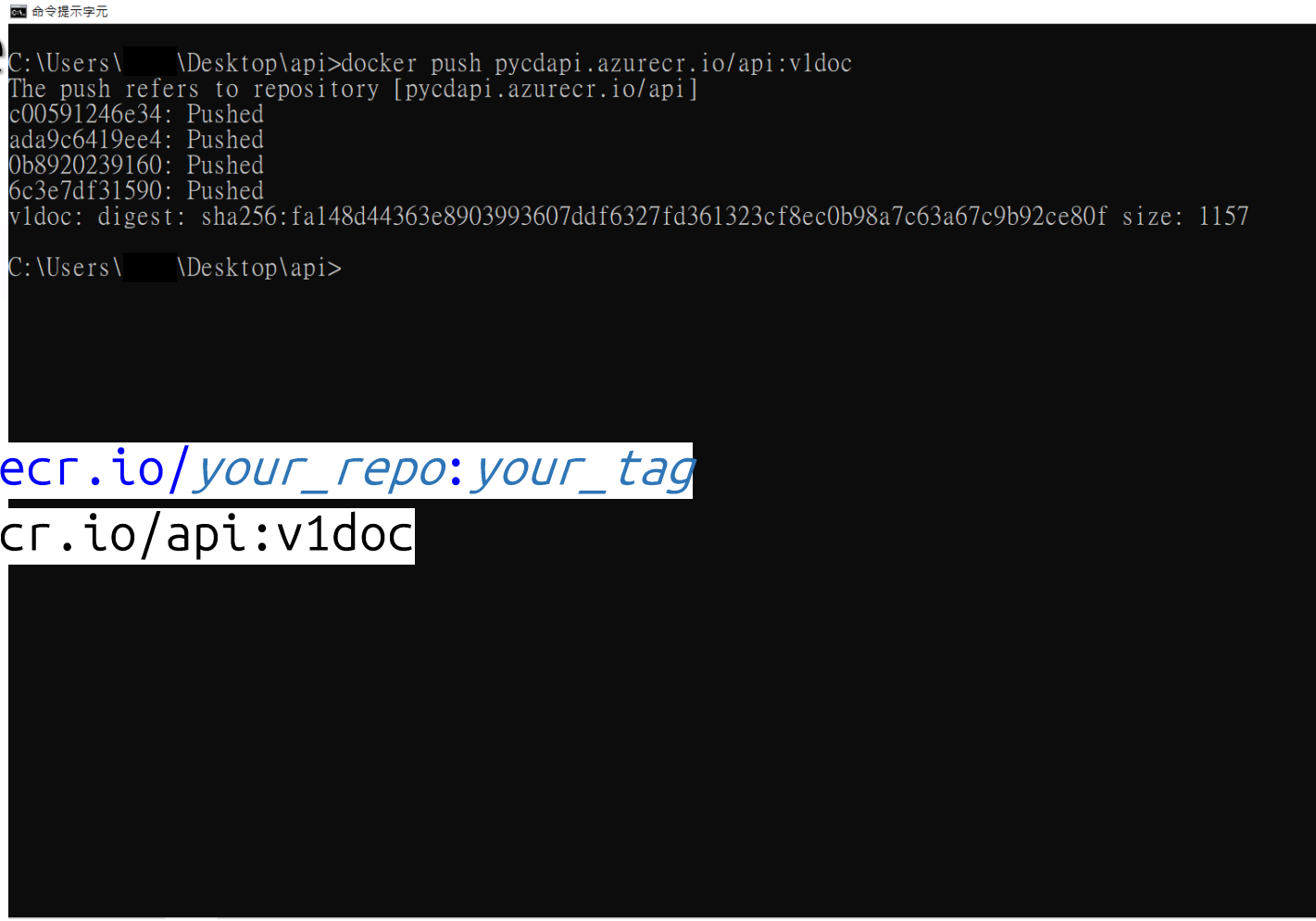
Docker Image into ACR

2. 建立並推送 Docker Image

a. by Docker

③ 上傳 Docker Image

```
docker push your_reg.azurecr.io/your_repo:your_tag
docker push pycdapi.azurecr.io/api:v1doc
```



Docker Image into ACR

2. 建立並推送 Docker Image

b. by Azure CLI

① 於桌機安裝下列軟體

Azure CLI
WSL2 (Windows Only)
Docker Desktop

© 2026 Enos Chou

Docker Image into ACR

2. 建立並推送 Docker Image

b. by Azure CLI

② 以 Azure CLI 完成帳號登入

```
az login
```

© 2026 Enos Chou

Docker Image into ACR

2. 建立並推送 Docker Image

b. by Azure CLI

③ 以 Azure CLI 完成容器登入

```
az acr login -n your_reg
az acr login -n pycdapi
```

© 2026 Enos Chou

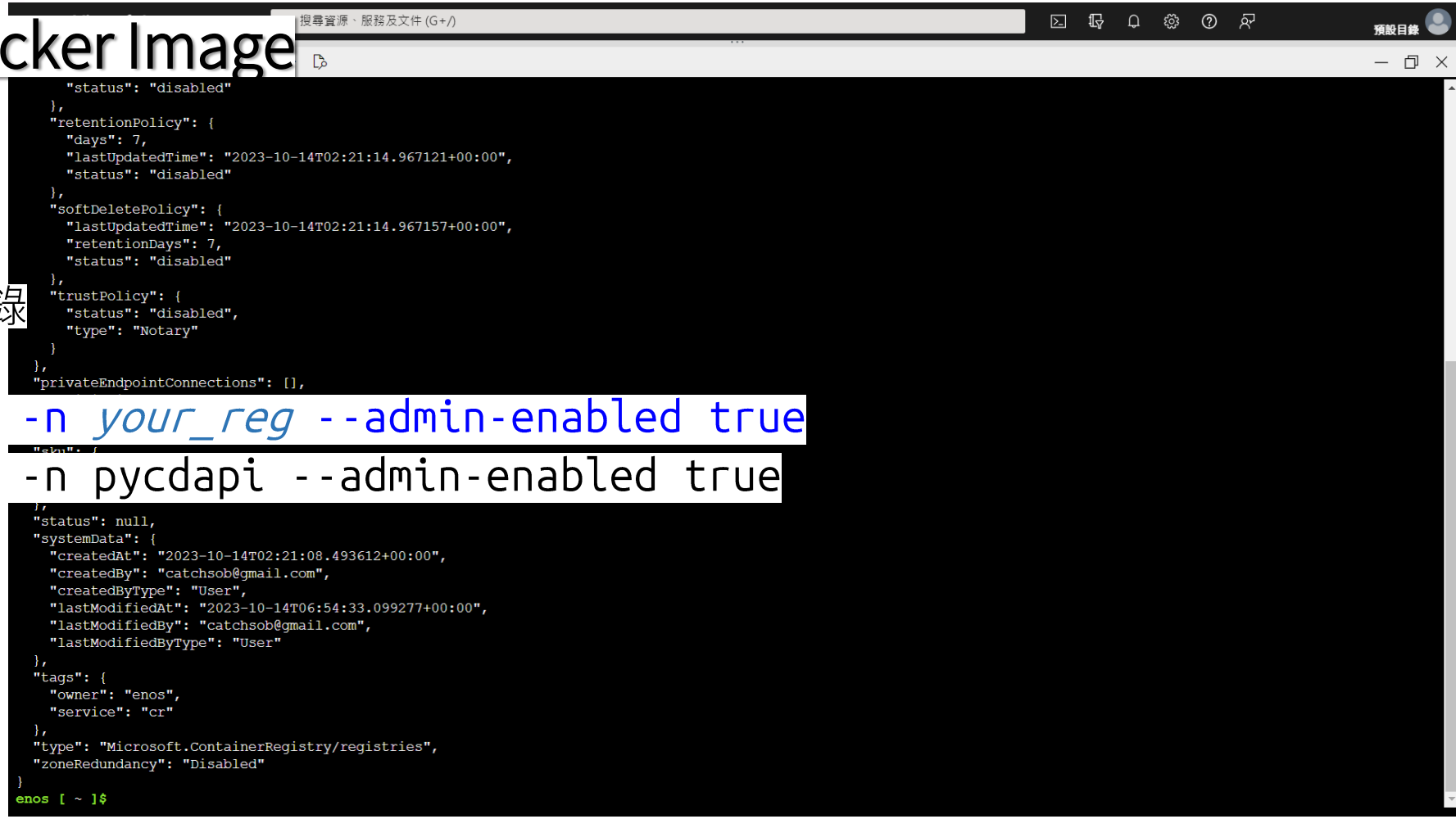
Docker Image into ACR

2. 建立並推送 Docker Image

b. by Azure CLI

④ 授權容器登錄

```
az acr update -n your_reg --admin-enabled true
az acr update -n pycdapi --admin-enabled true
```



© 2026 Enos Chou

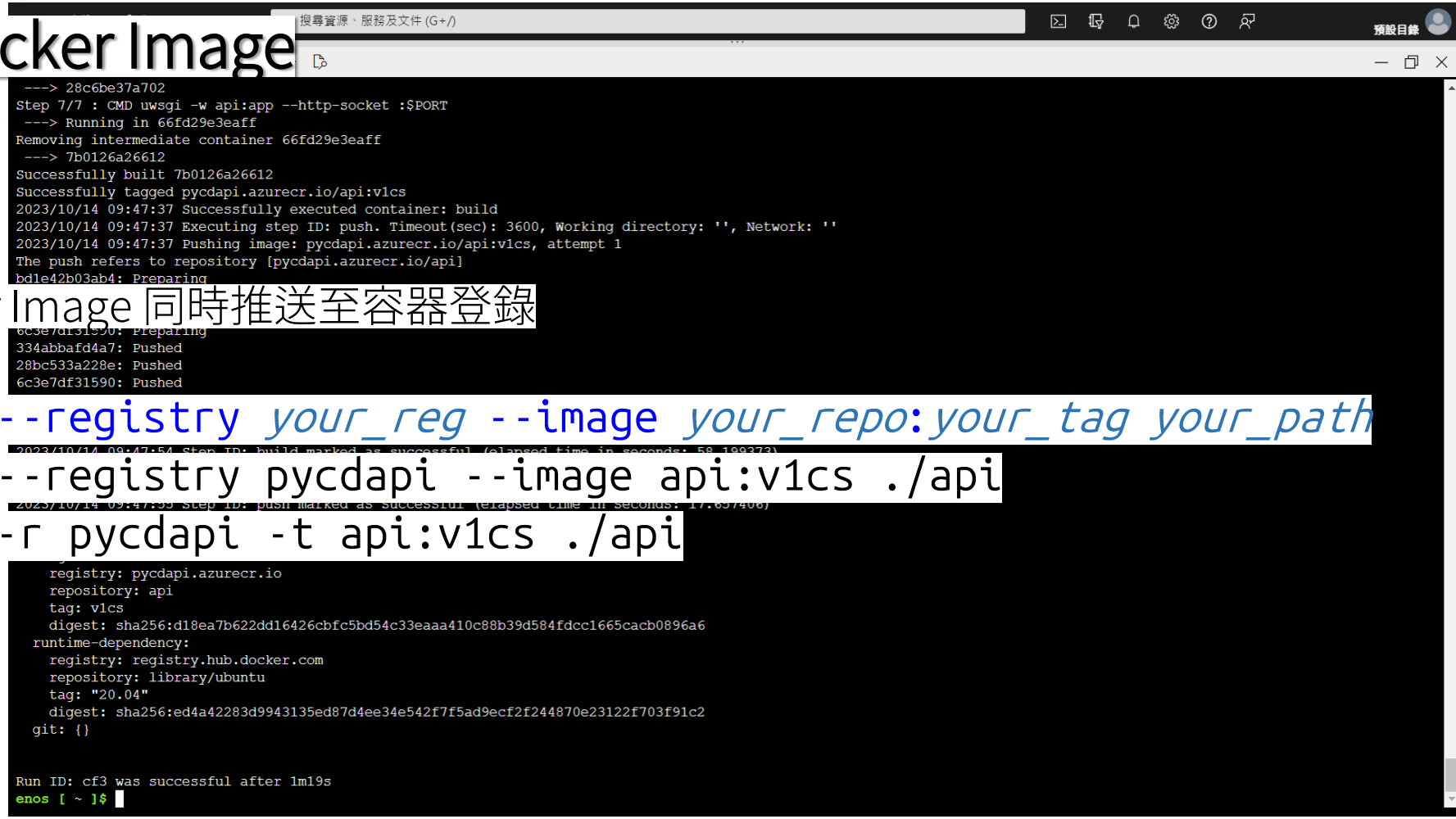
Docker Image into ACR

2. 建立並推送 Docker Image

b. by Azure CLI

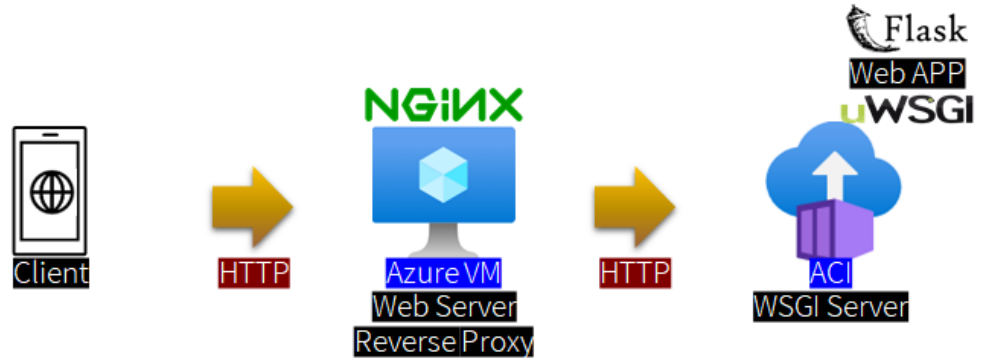
⑤ 建立 Docker Image 同時推送至容器登錄

```
az acr build --registry your_reg --image your_repo:your_tag your_path
az acr build --registry pycdapi --image api:v1cs ./api
az acr build -r pycdapi -t api:v1cs ./api
```



Azure Container Instance (ACI)

Containerized VM



Azure Web App (App Service)

Azure Container App

© 2026 Enos Chou

APP 部署 ft. ACI

1. 建立容器執行個體 Azure Container Instance (ACI)
2. 調整 VM 之 NGINX 反向代理設定
3. 測試

© 2026 Enos Chou

APP 部署 ft. ACI

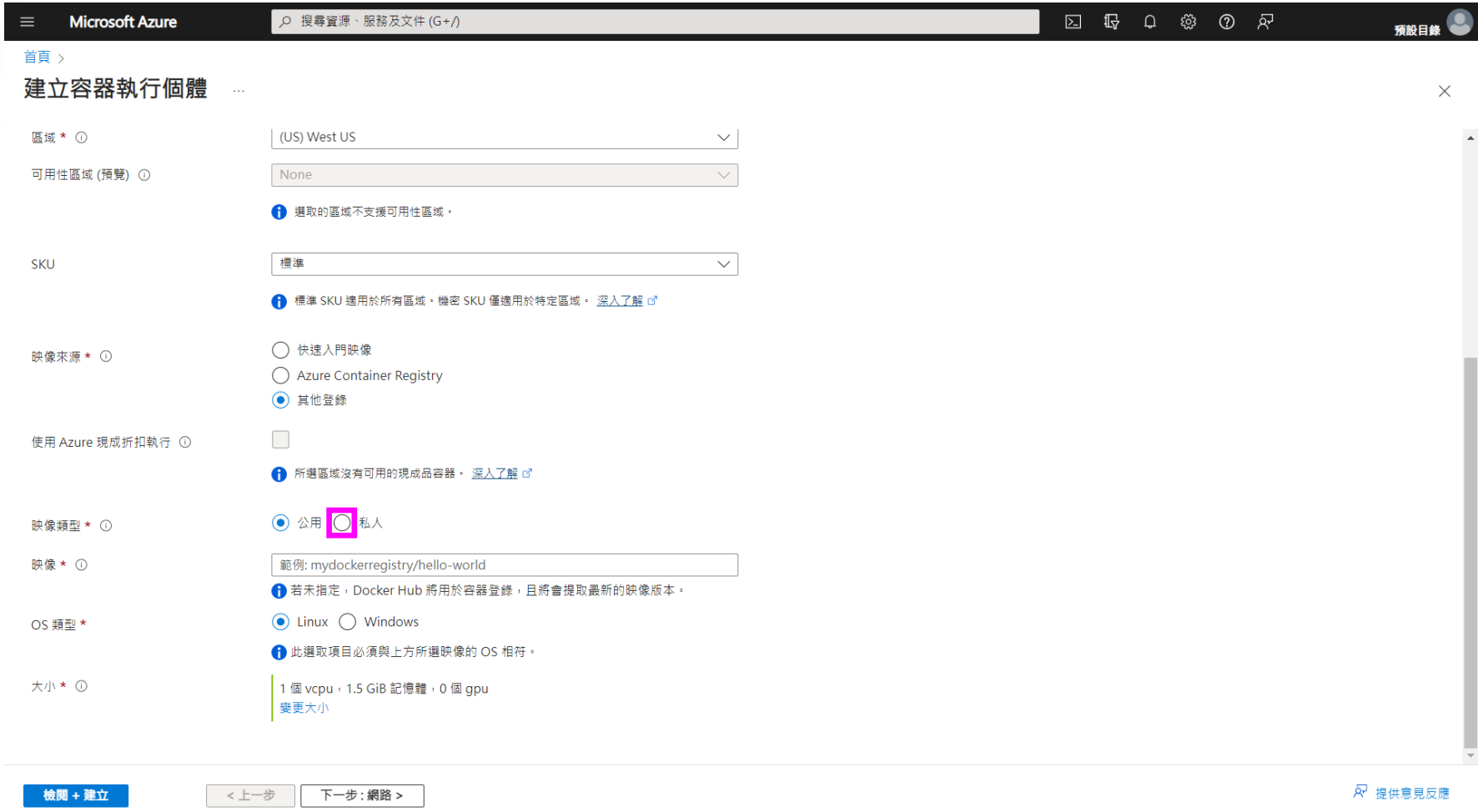
1. 建立 ACI

APP 部署 ft. ACI

1. 建立 ACI

APP 部署 ft. ACI

1. 建立 ACI

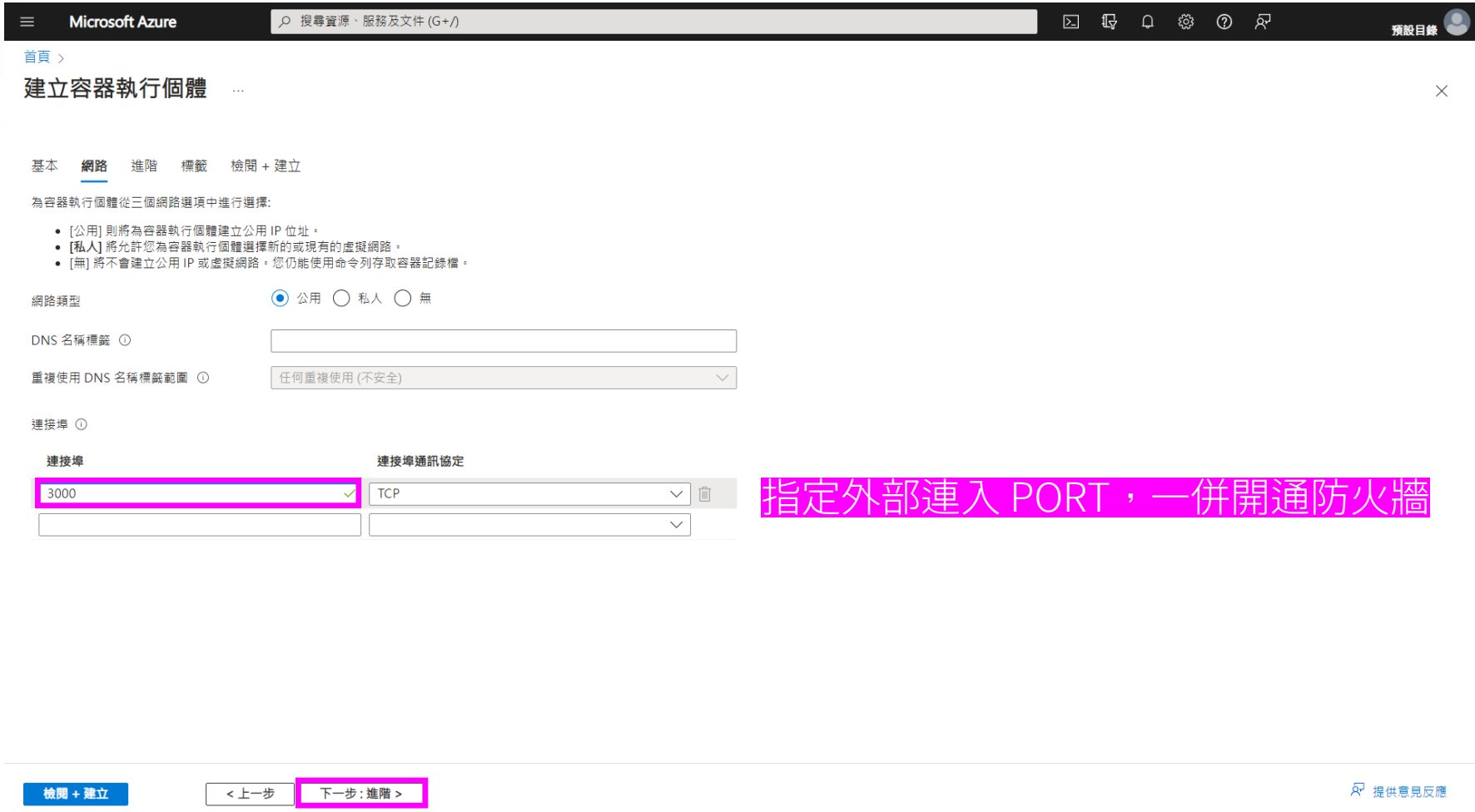


APP 部署 ft. ACI

1. 建立 ACI

APP 部署 ft. ACI

1. 建立 ACI



指定外部連入 PORT，一併開通防火牆

APP 部署 ft. ACI

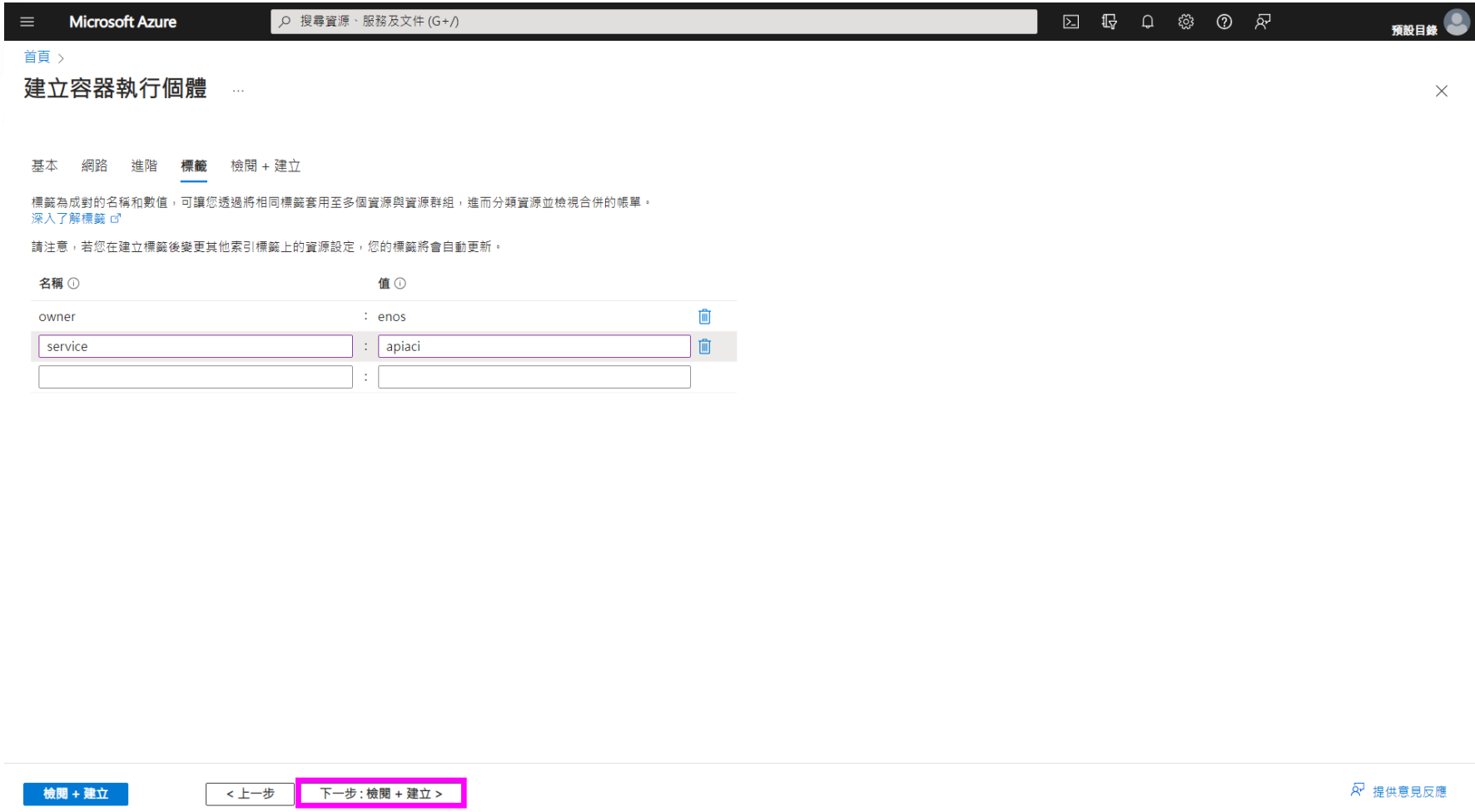
1. 建立 ACI

以環境變數指定 Container 使用 PORT

Note
若監聽的 PORT 寫死於 Container
(未設定為環境變數) 則省略此步驟

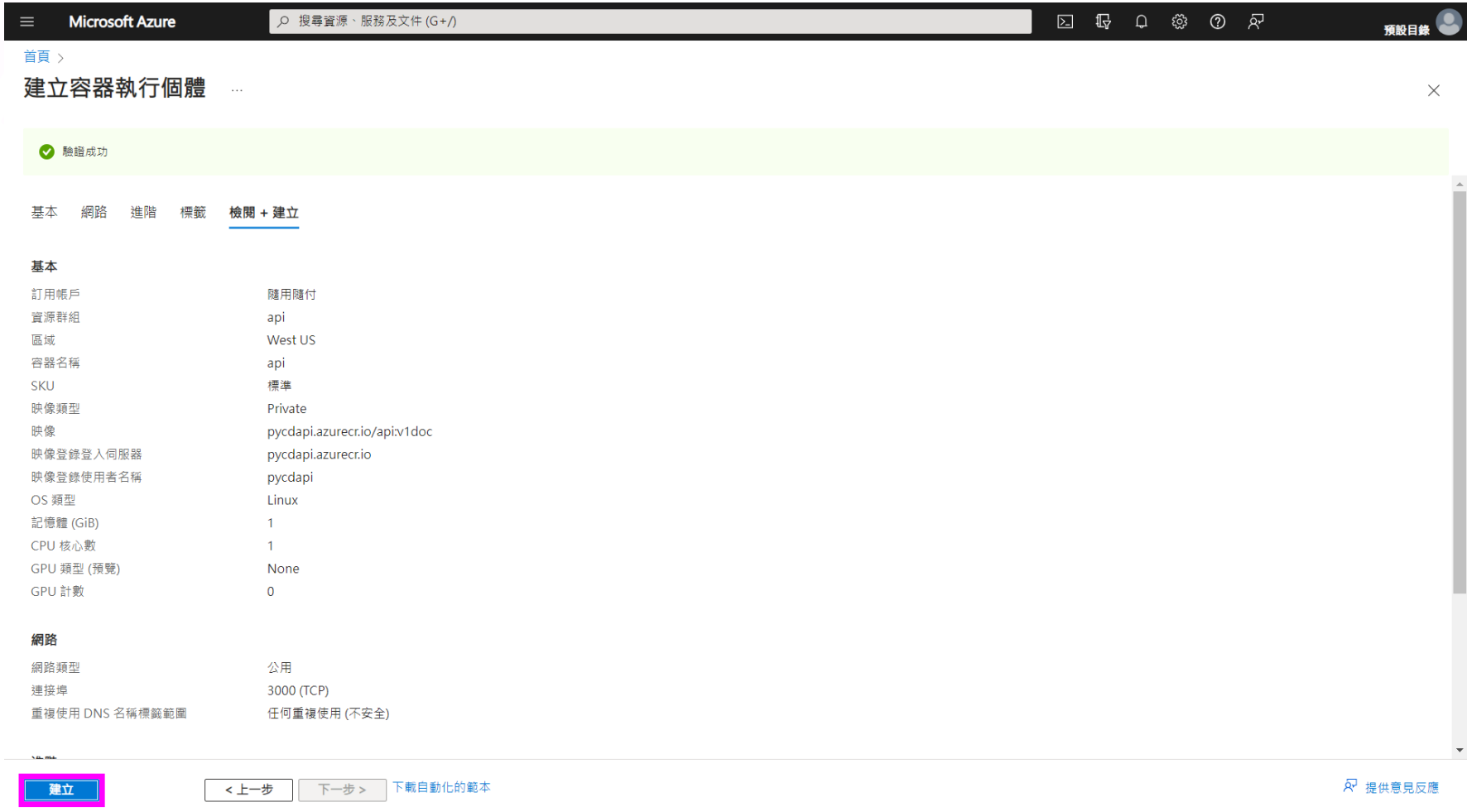
APP 部署 ft. ACI

1. 建立 ACI



APP 部署 ft. ACI

1. 建立 ACI

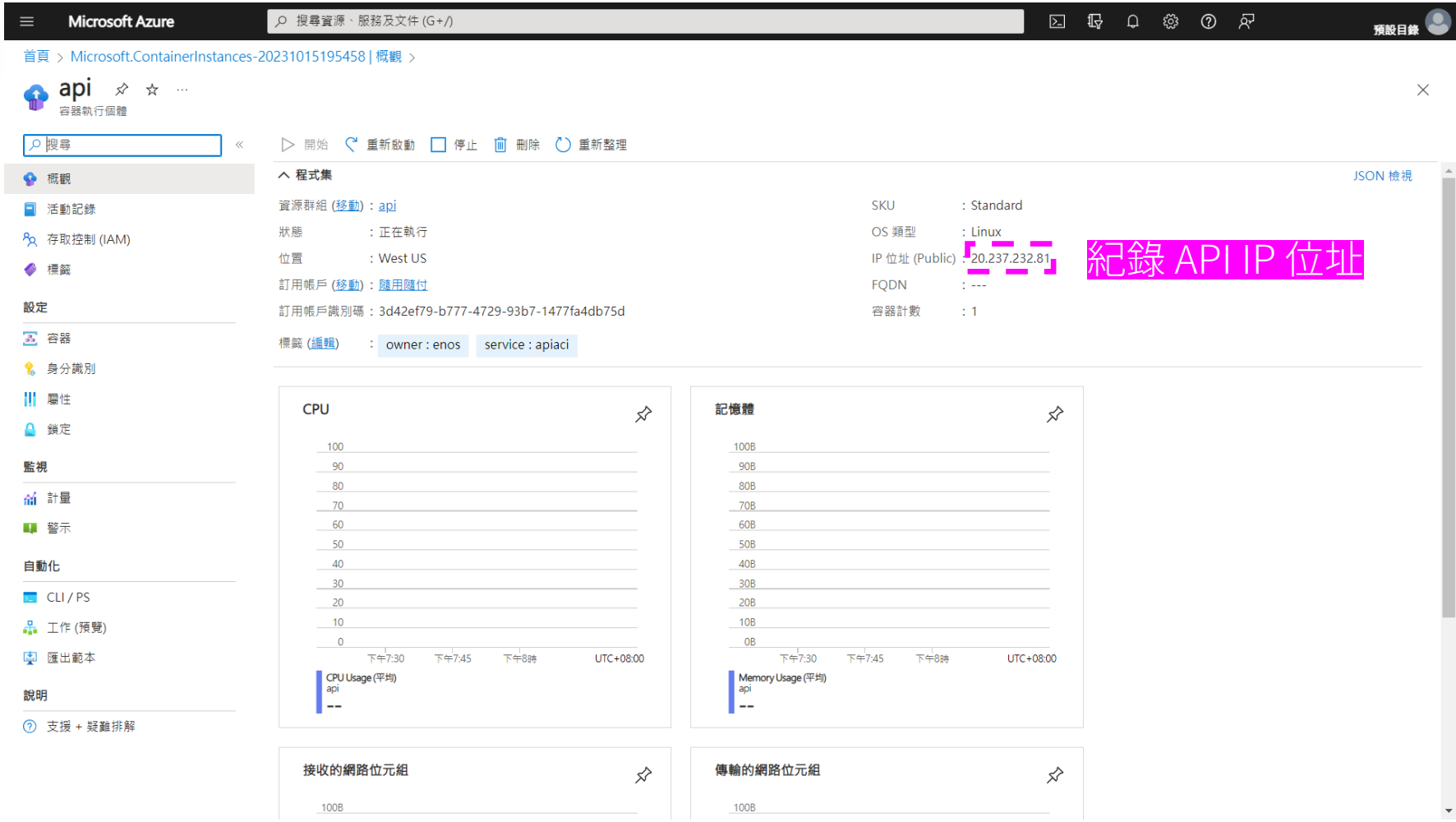


APP 部署 ft. ACI

1. 建立 ACI

APP 部署 ft. ACI

1. 建立 ACI

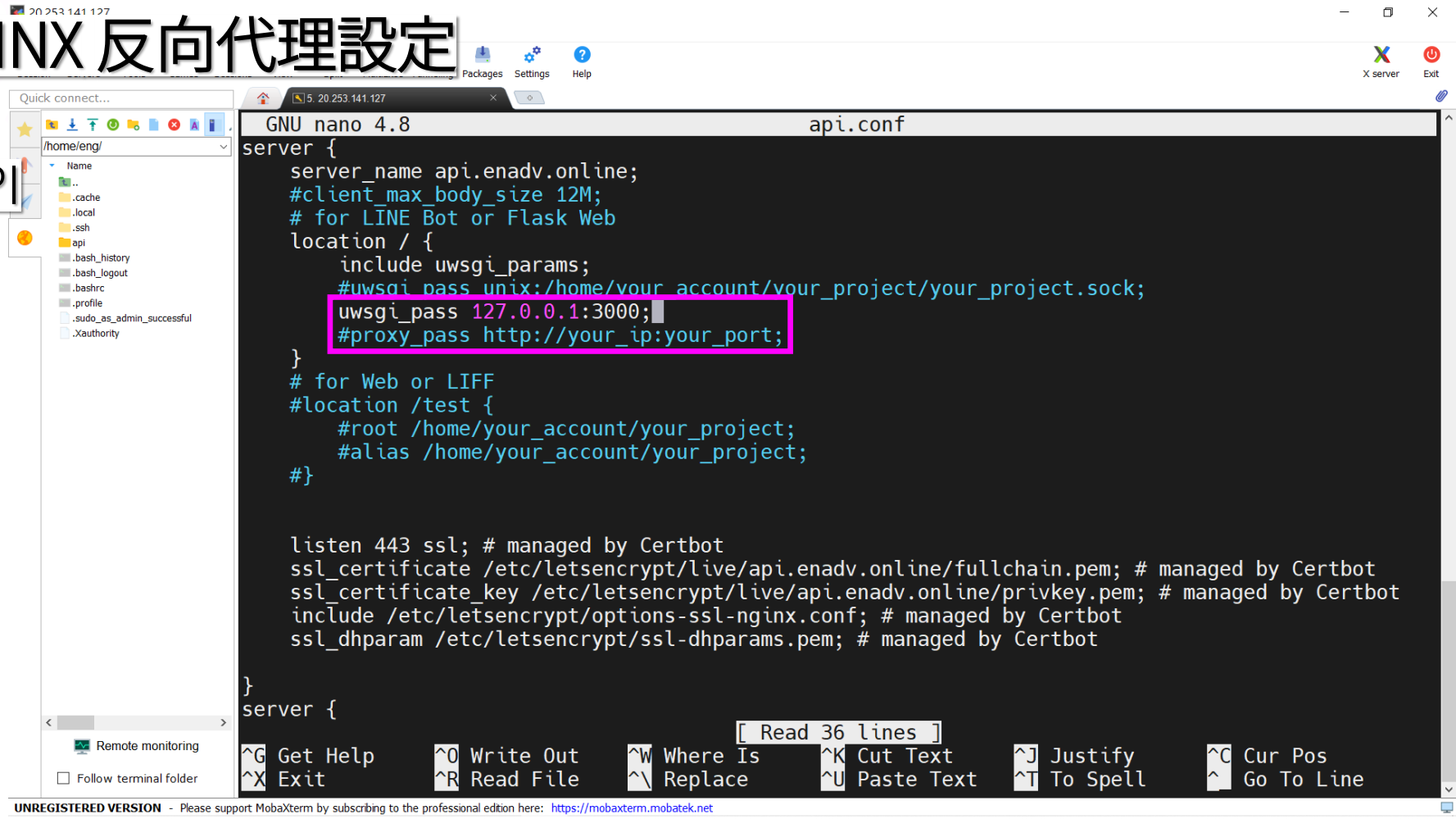


APP 部署 ft. ACI

2. 調整 VM 之 NGINX 反向代理設定

a. 轉導至 ACI API

① 調整組態

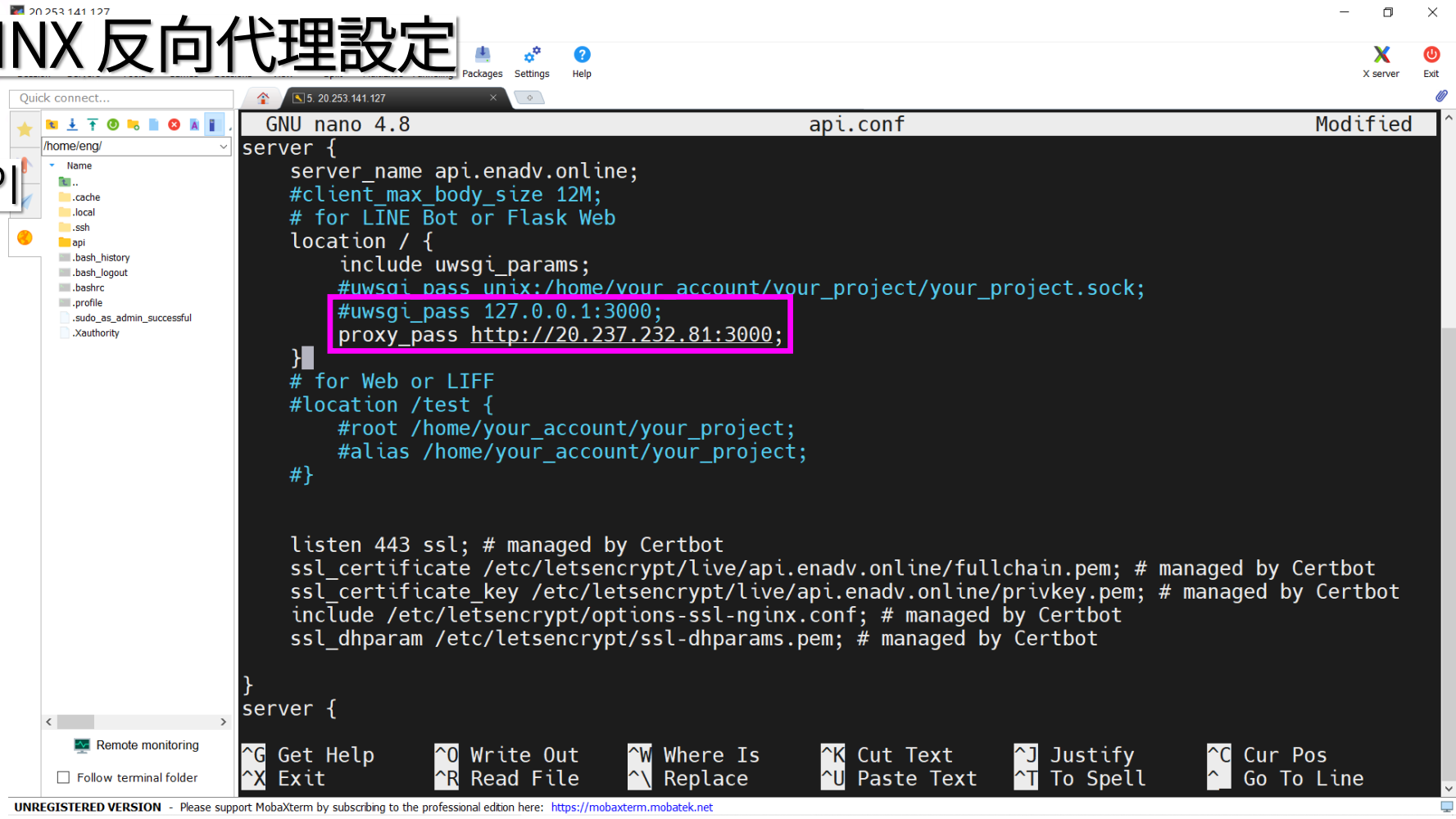


APP 部署 ft. ACI

2. 調整 VM 之 NGINX 反向代理設定

a. 轉導至 ACI API

① 調整組態



APP 部署 ft. ACI

2. 調整 VM 之 NGINX 反向代理設定

a. 轉導至 ACI API

② 組態生效

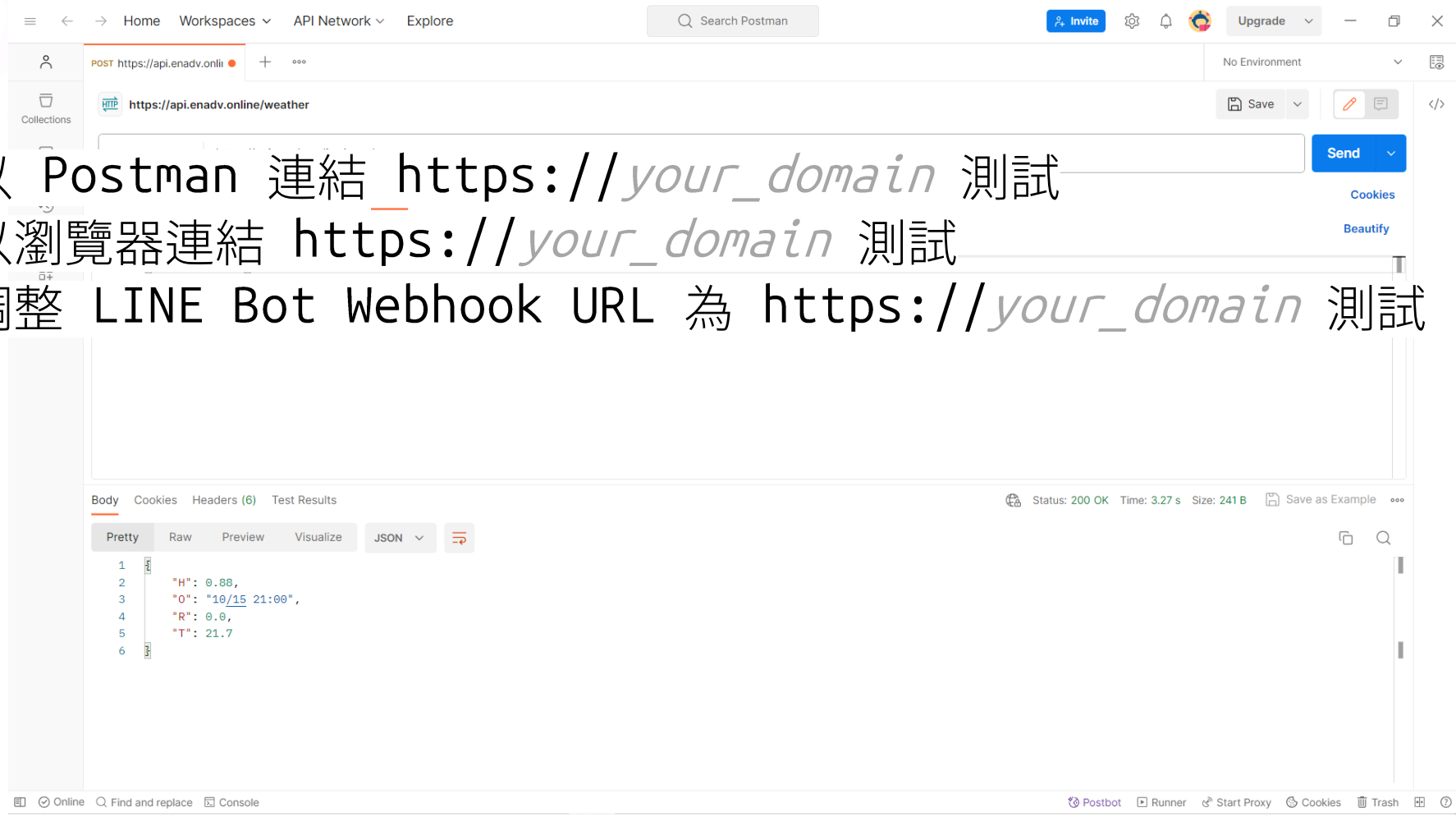
```
sudo nginx -s reload
```

© 2026 Enos Chou

APP 部署 ft. ACI

3. 測試

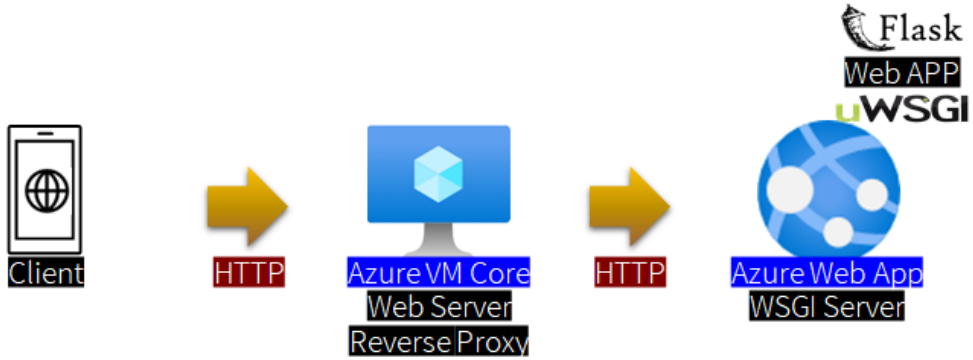
 # API - 以 Postman 連結 https://*your_domain* 測試
 # Web - 以瀏覽器連結 https://*your_domain* 測試
 # LINE Bot - 調整 LINE Bot Webhook URL 為 https://*your_domain* 測試



Azure Container Instance (ACI)

Azure Web App (App Service)

Containerized/ VM-based



Azure Container App

APP 部署 ft. Web App

1. 建立 Web 應用程式 Web App
2. 測試

© 2026 Enos Chou

APP 部署 ft. Web App

1. 建立 Web App

© 2026 Enos Chou

APP 部署 ft. Web App

1. 建立 Web App

APP 部署 ft. Web App

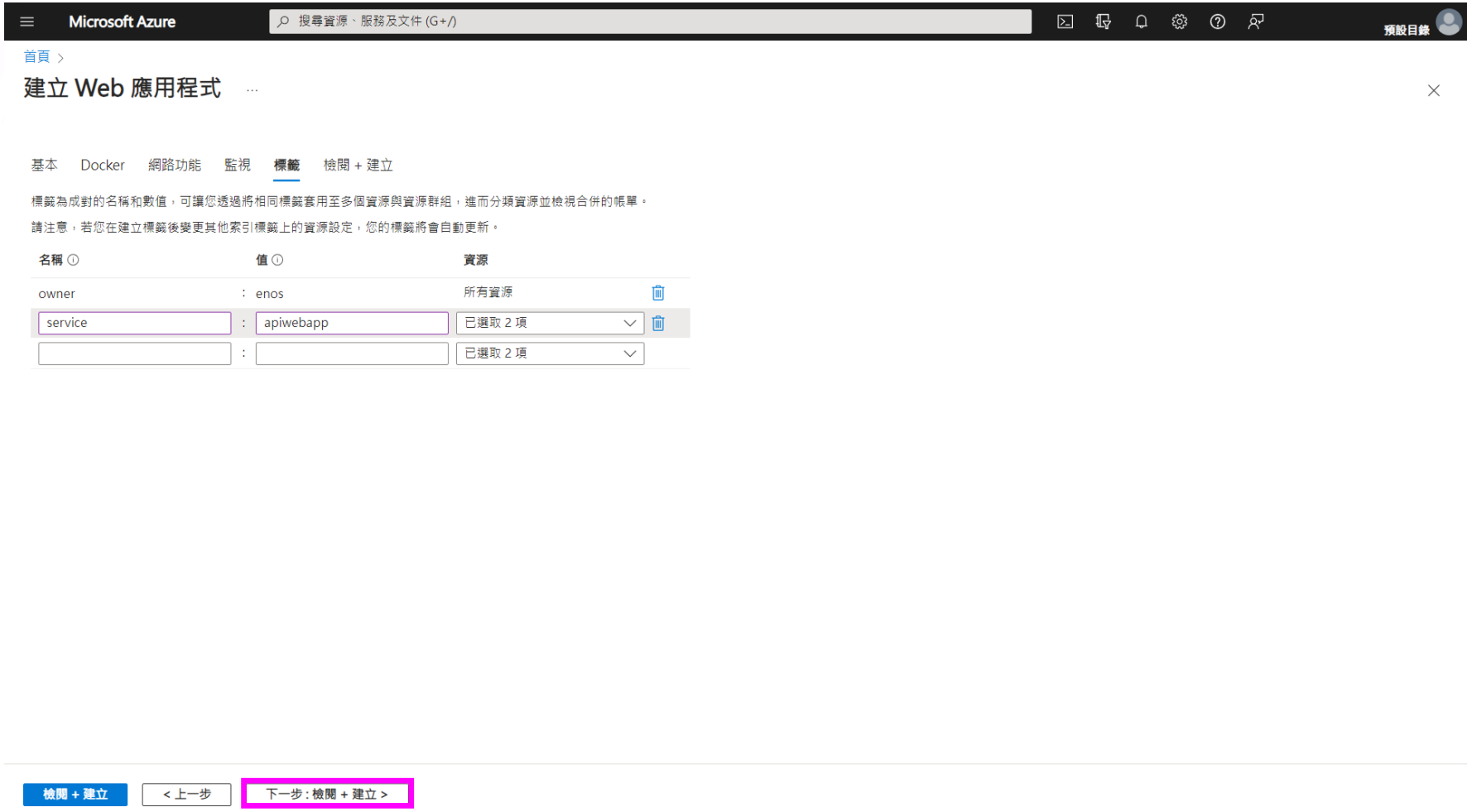
1. 建立 Web App

APP 部署 ft. Web App

1. 建立 Web App

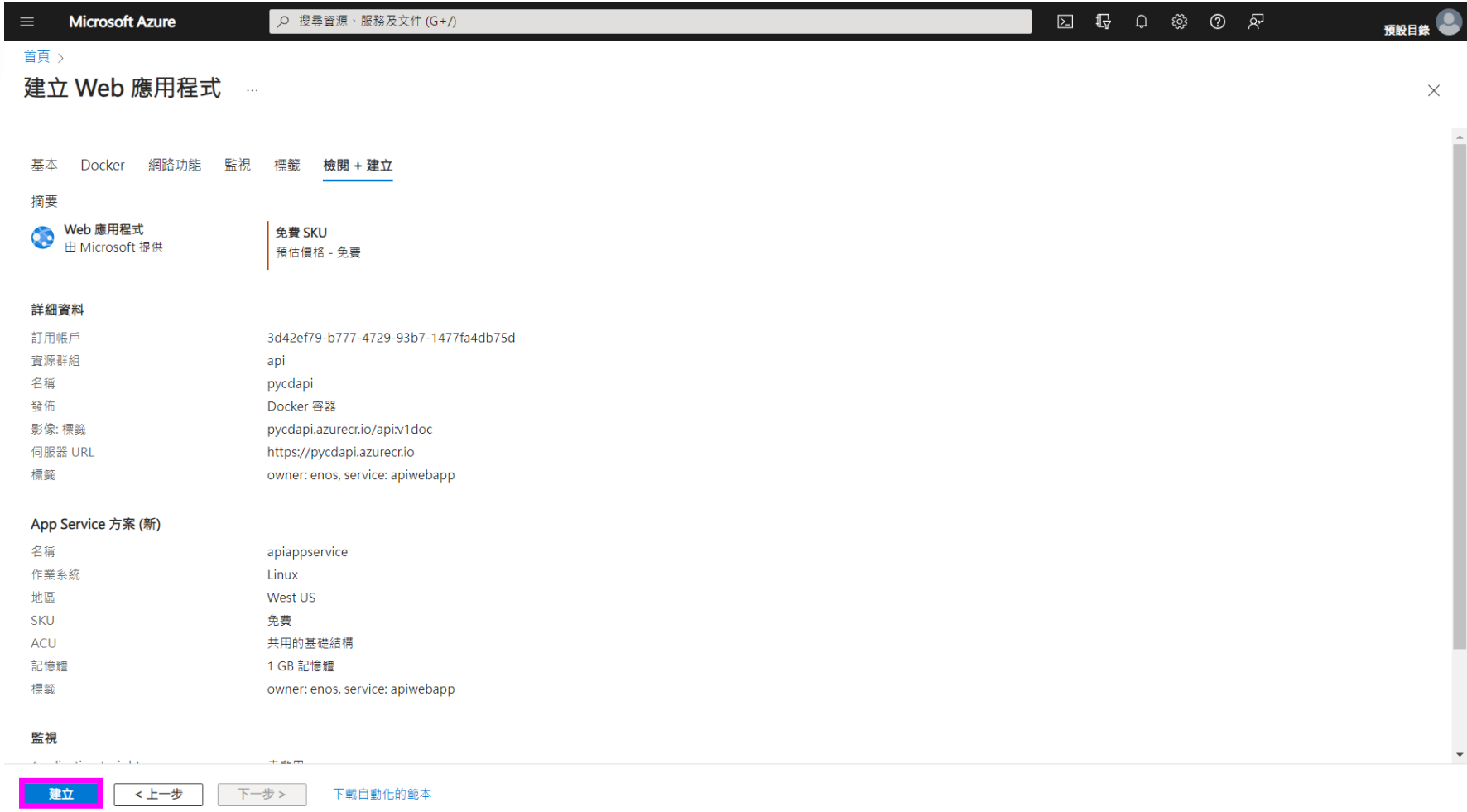
APP 部署 ft. Web App

1. 建立 Web App



APP 部署 ft. Web App

1. 建立 Web App



© 2026 Enos Chou

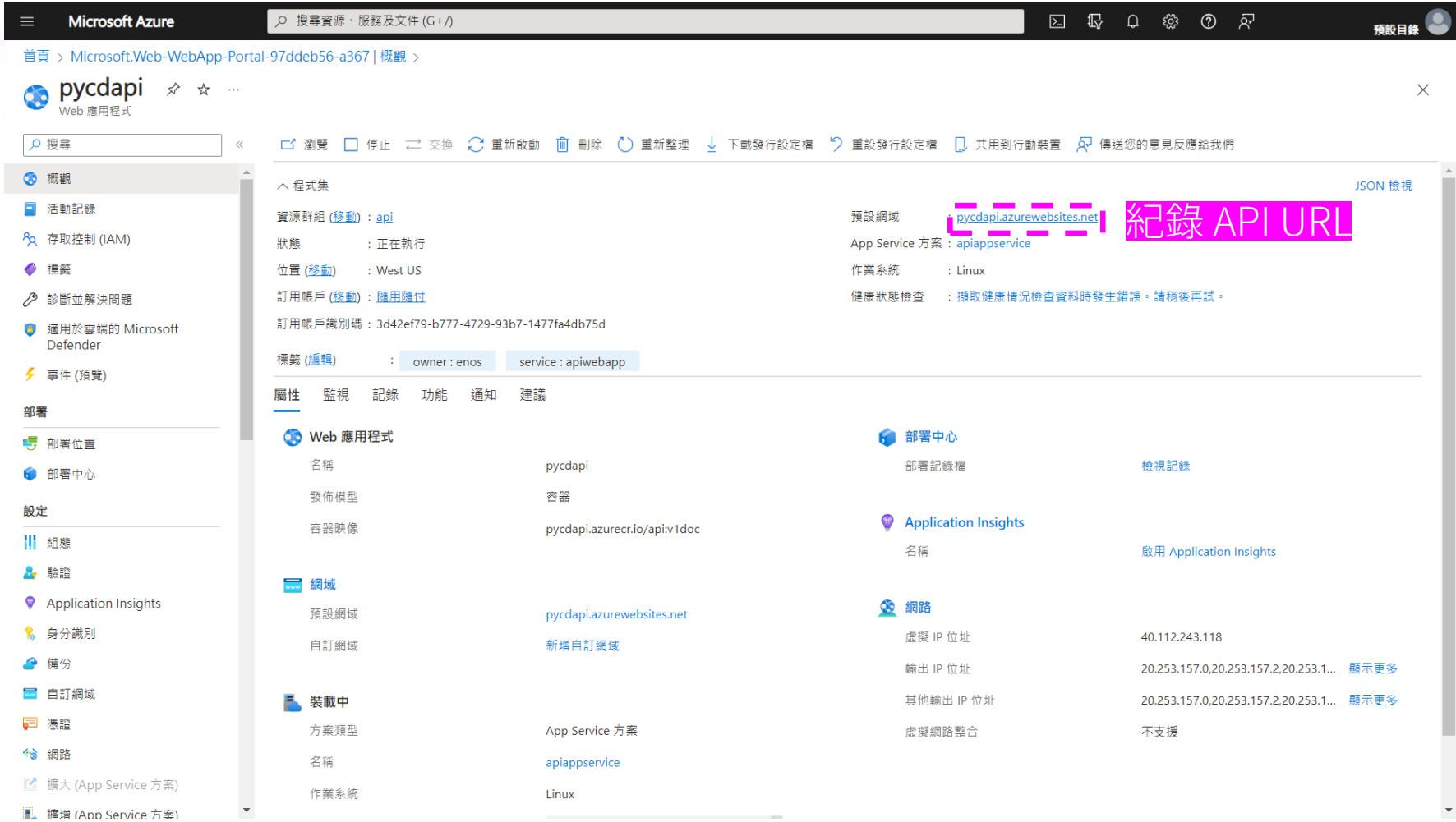
APP 部署 ft. Web App

1. 建立 Web App

© 2026 Enos Chou

APP 部署 ft. Web App

1. 建立 Web App

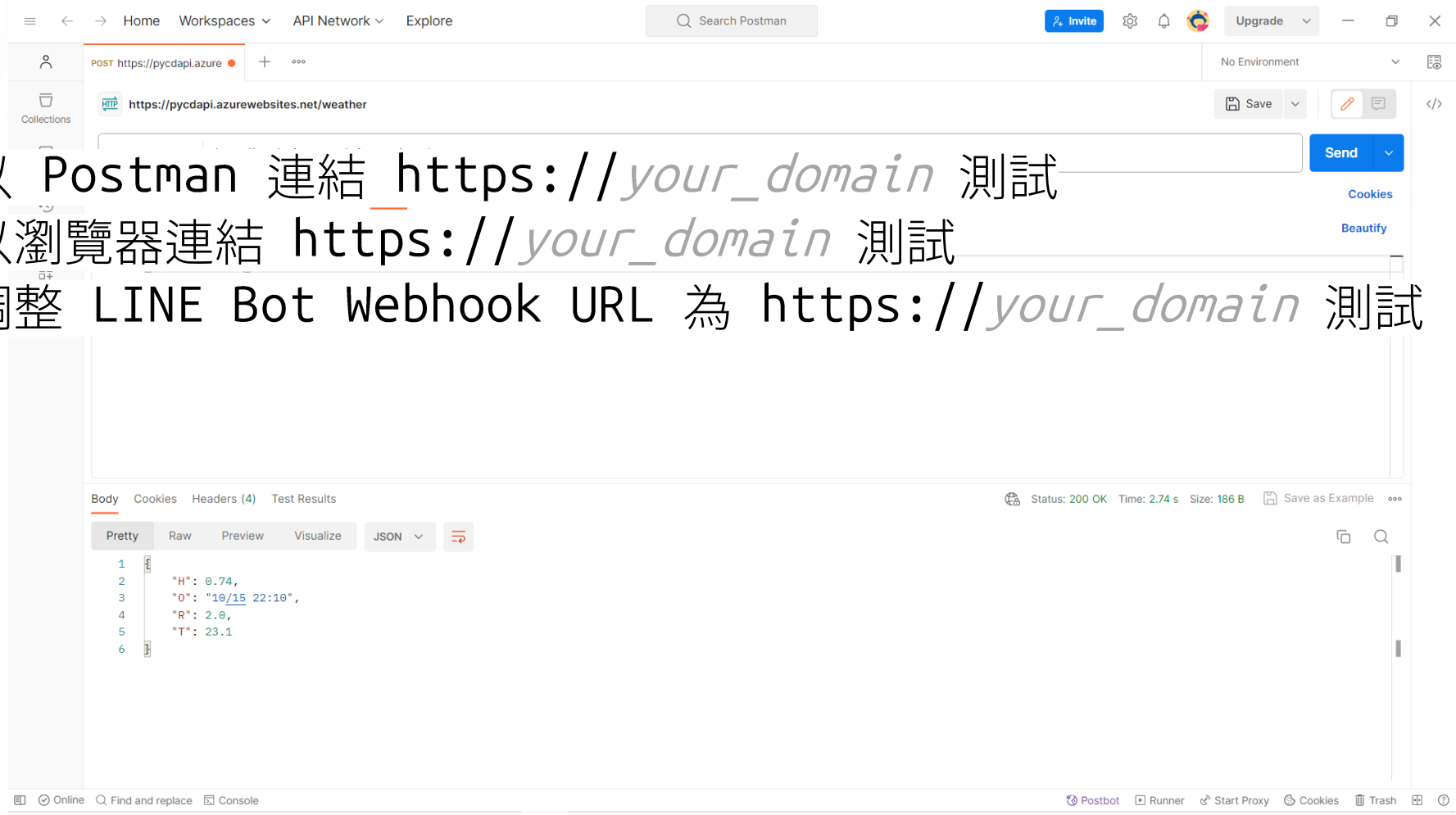


© 2026 Enos Chou

APP 部署 ft. Web App

2. 測試

```
# API       - 以 Postman 連結 https://your_domain 測試
# Web       - 以瀏覽器連結 https://your_domain 測試
# LINE Bot  - 調整 LINE Bot Webhook URL 為 https://your_domain 測試
```

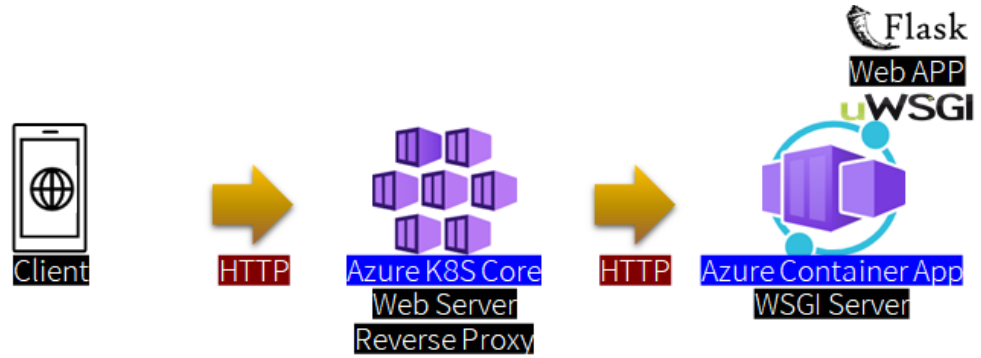


Azure Container Instance (ACI)

Azure Web App (App Service)

Azure Container App

Containerized/ Serverless



© 2026 Enos Chou

APP 部署 ft. Container App

1. 建立容器應用程式 Container App
2. 測試
3. 連結內網 VM
4. 轉換網域
5. 取得 Log

© 2026 Enos Chou

APP 部署 ft. Container App

1. 建立 Container App

APP 部署 ft. Container App

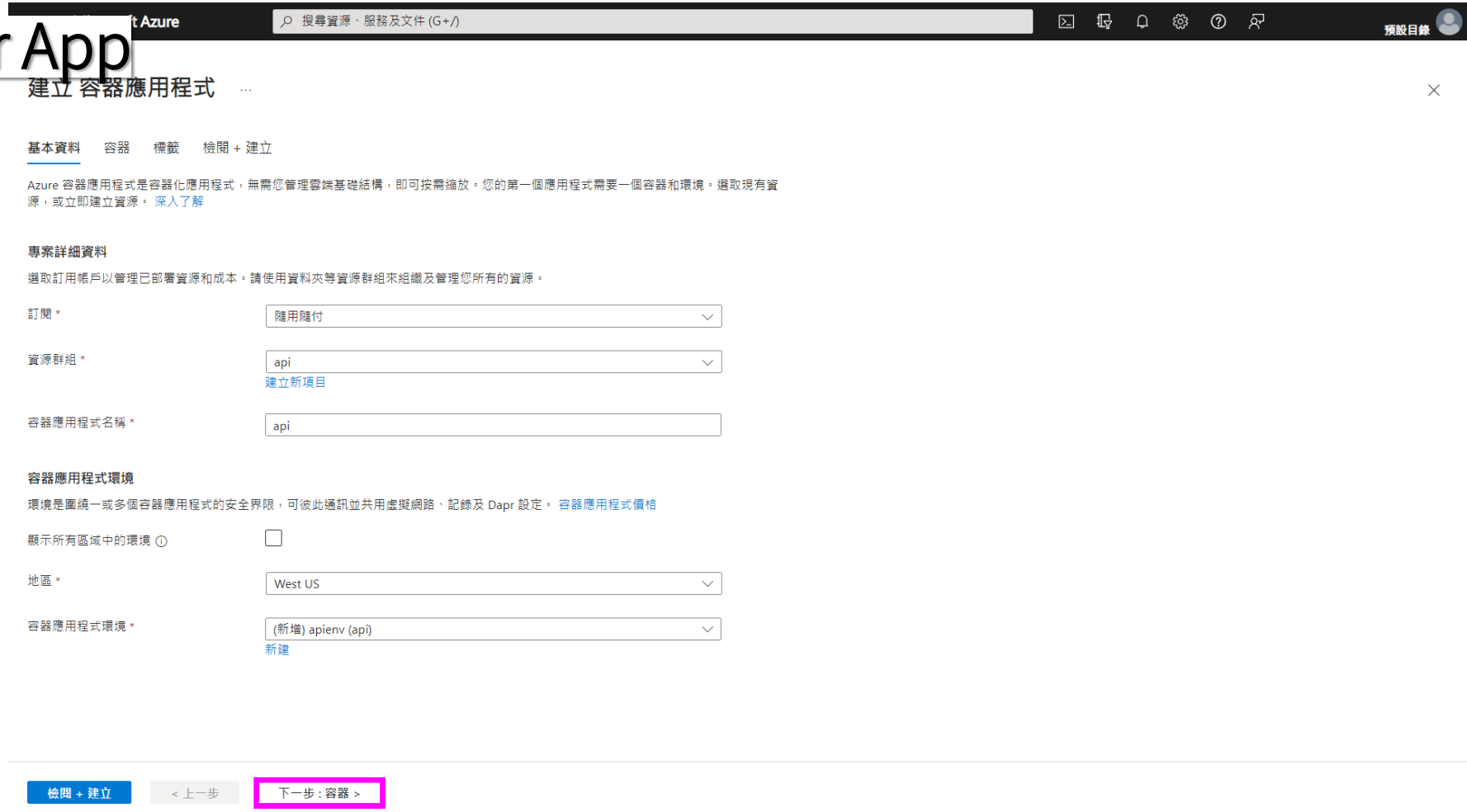
1. 建立 Container App

APP 部署 ft. Container App

1. 建立 Container App

APP 部署 ft. Container App

1. 建立 Container App



APP 部署 ft. Container App

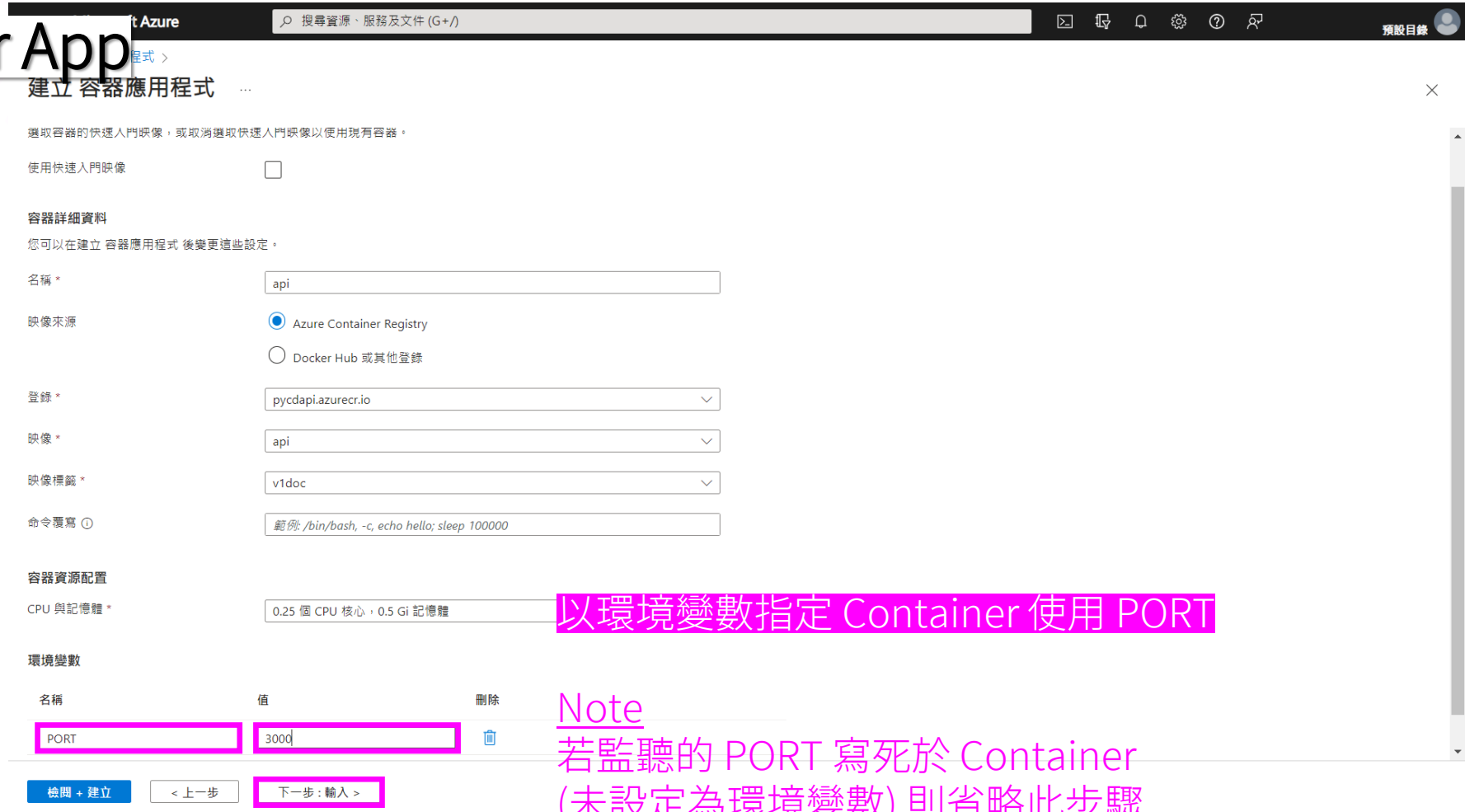
1. 建立 Container App

APP 部署 ft. Container App

1. 建立 Container App

APP 部署 ft. Container App

1. 建立 Container App



以環境變數指定 Container 使用 PORT

Note
若監聽的 PORT 寫死於 Container
(未設定為環境變數) 則省略此步驟

APP 部署 ft. Container App

1. 建立 Container App

APP 部署 ft. Container App

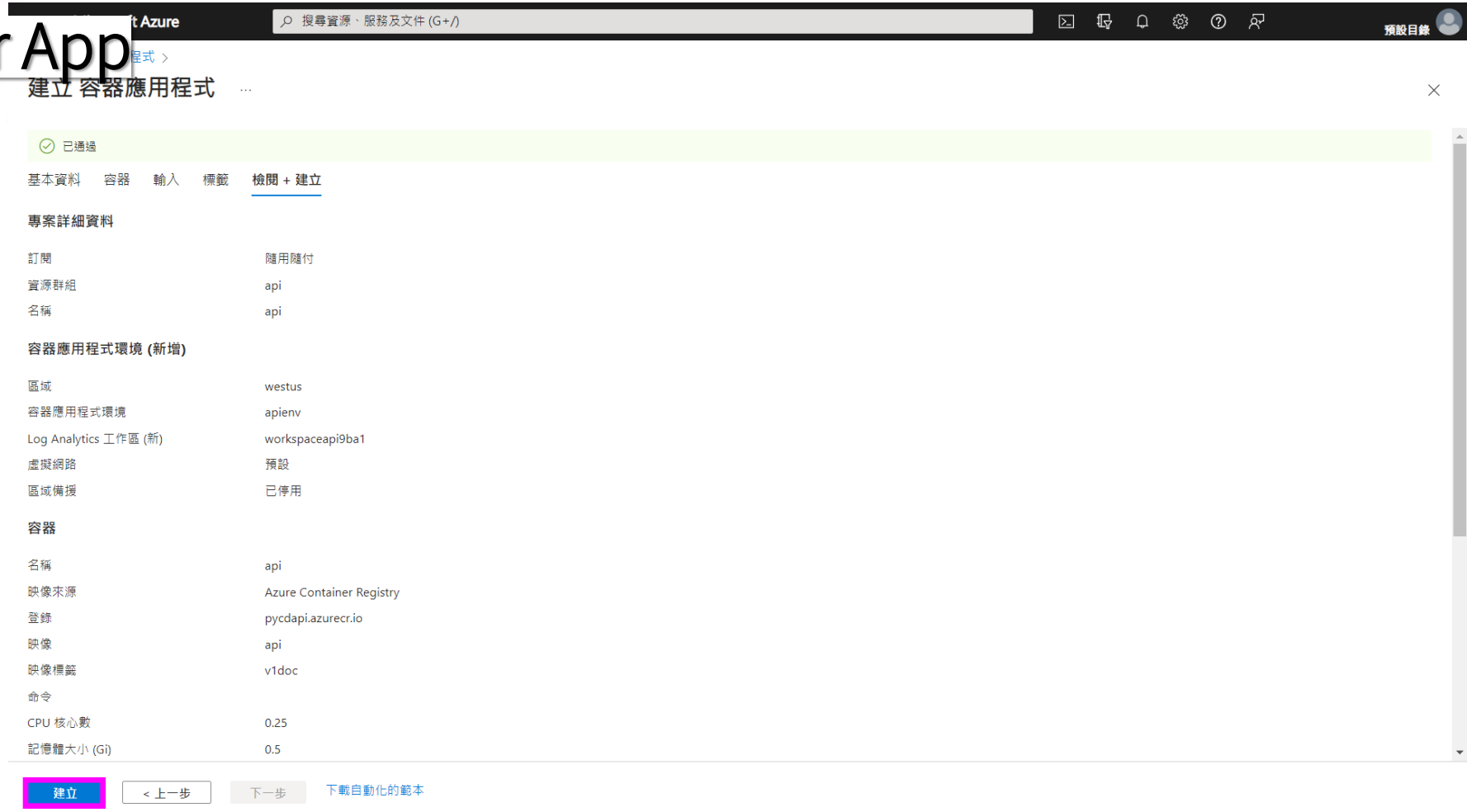
1. 建立 Container App

APP 部署 ft. Container App

1. 建立 Container App

APP 部署 ft. Container App

1. 建立 Container App



APP 部署 ft. Container App

1. 建立 Container App

APP 部署 ft. Container App

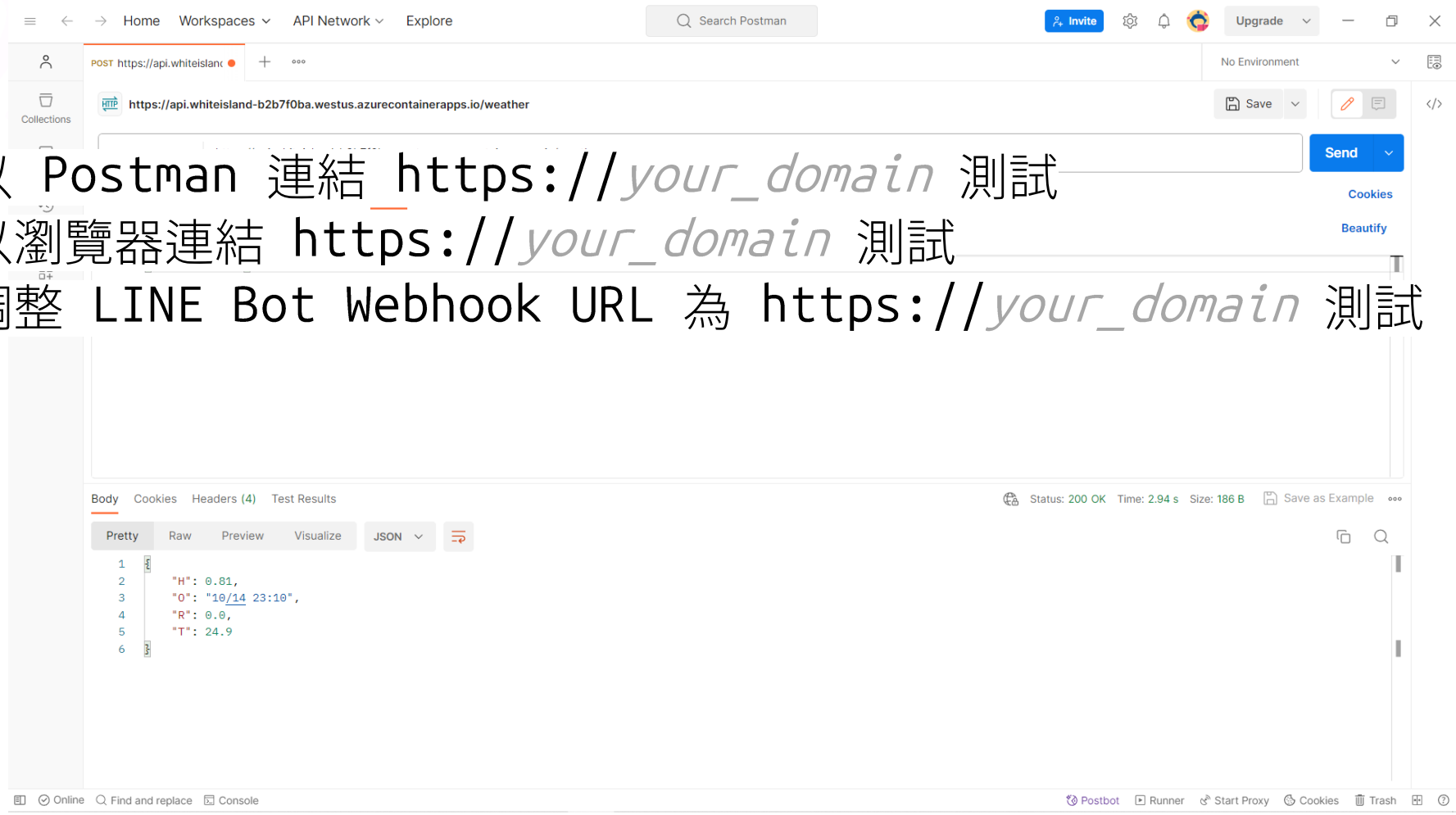
1. 建立 Container App

© 2026 Enos Chou

APP 部署 ft. Container App

2. 測試

```
# API       - 以 Postman 連結 https://your_domain 測試
# Web       - 以瀏覽器連結 https://your_domain 測試
# LINE Bot  - 調整 LINE Bot Webhook URL 為 https://your_domain 測試
```



APP 部署 ft. Container App

3. 連結內網 VM

於 容器應用程式環境 選擇 新建 之後需額外設定 網路功能

① 使用您自己的虛擬網路：是
② 虛擬網路：選擇欲連結 VM 之虛擬網路
③ 基礎結構子網路：建立新項目 >
子網路名稱：自行命名/ 子網路位址區塊：選擇虛擬網路位址區塊內的一段即可，
亦可直接使用預設內容值 >
確定
④ 虛擬 IP：外部
⑤ 建立

© 2026 Enos Chou

APP 部署 ft. Container App

4. 轉換網域

目的：提高 Container App 的網址識別度

方法：以自有域名取代 Azure Container App 提供之預設域名

© 2026 Enos Chou

APP 部署 ft. Container App

4. 轉換網域

進入建立完成之 Container App

① Networking：自訂網域
② 新增自訂網域
③ TLS/SSL 憑證：受控憑證
④ 網域：自行輸入欲轉換的網域，例：api.mydomain.ai
⑤ 主機名稱紀錄類型：CNAME
⑥ 網域驗證：依指示進入自己擁有的 DNS，增加下列兩組設定
CNAME record (CNAME, api, Azure 提供的 domain name)
TXT record (TXT, asuid.api, 指定的值)
⑦ 驗證
⑧ 新增

© 2026 Enos Chou

APP 部署 ft. Container App

5. 取得 Log

進入 Log Analytics 工作區取得 工作區識別碼

① `pip install azure-identity azure-monitor-query`

© 2026 Enos Chou

APP 部署 ft. Container App

5. 取得 Log

進入 Log Analytics 工作區取得 工作區識別碼

②
```
from azure.identity import DefaultAzureCredential
from azure.monitor.query import LogsQueryClient
from datetime import datetime, timezone, timedelta
```

© 2026 Enos Chou

APP 部署 ft. Container App

5. 取得 Log

進入 Log Analytics 工作區取得 工作區識別碼

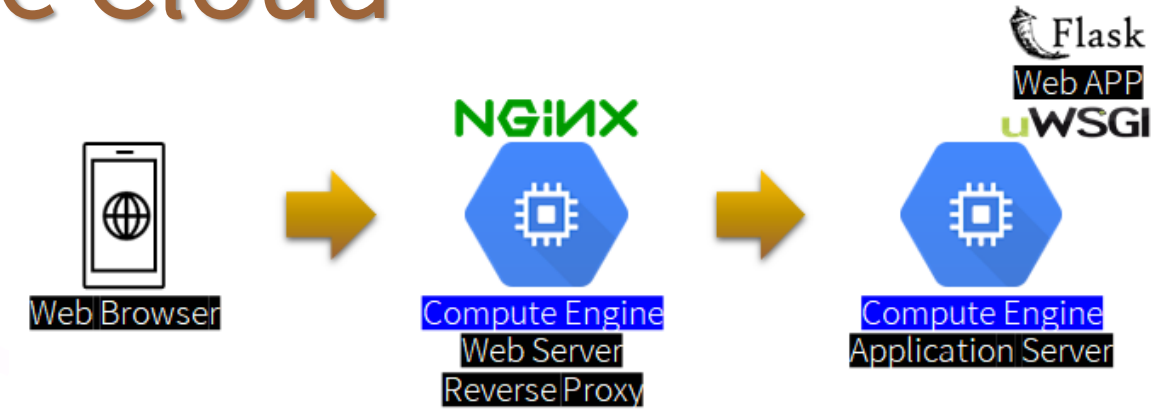
②
```
workspace_id = "工作區識別碼"
credential = DefaultAzureCredential()  # 本機認證
client = LogsQueryClient(credential)
start = datetime(2025, 2, 27, tzinfo=timezone(timedelta(hours=8)))
end = datetime(2025, 3, 6, tzinfo=timezone(timedelta(hours=8)))
query = '''ContainerAppConsoleLogs_CL
            | where ContainerAppName_s == "test" and Log_s contains "hello"
            | order by TimeGenerated'''
result = client.query_workspace(workspace_id, query, timespan=(start, end))
records = [row for table in result.tables for row in table.rows]
```

© 2026 Enos Chou

容器化部署 ft. Google Cloud

Google Compute Engine

Containerized VM



Google Cloud Run

Containerized/ Serverless

© 2026 Enos Chou

Docker Image into Google Artifact Registry

1. 建立容器登錄 Google Artifact Registry
2. 建立並推送 Docker Image 至 Artifact Registry

© Enos Chou

Docker Image into Google Artifact Registry

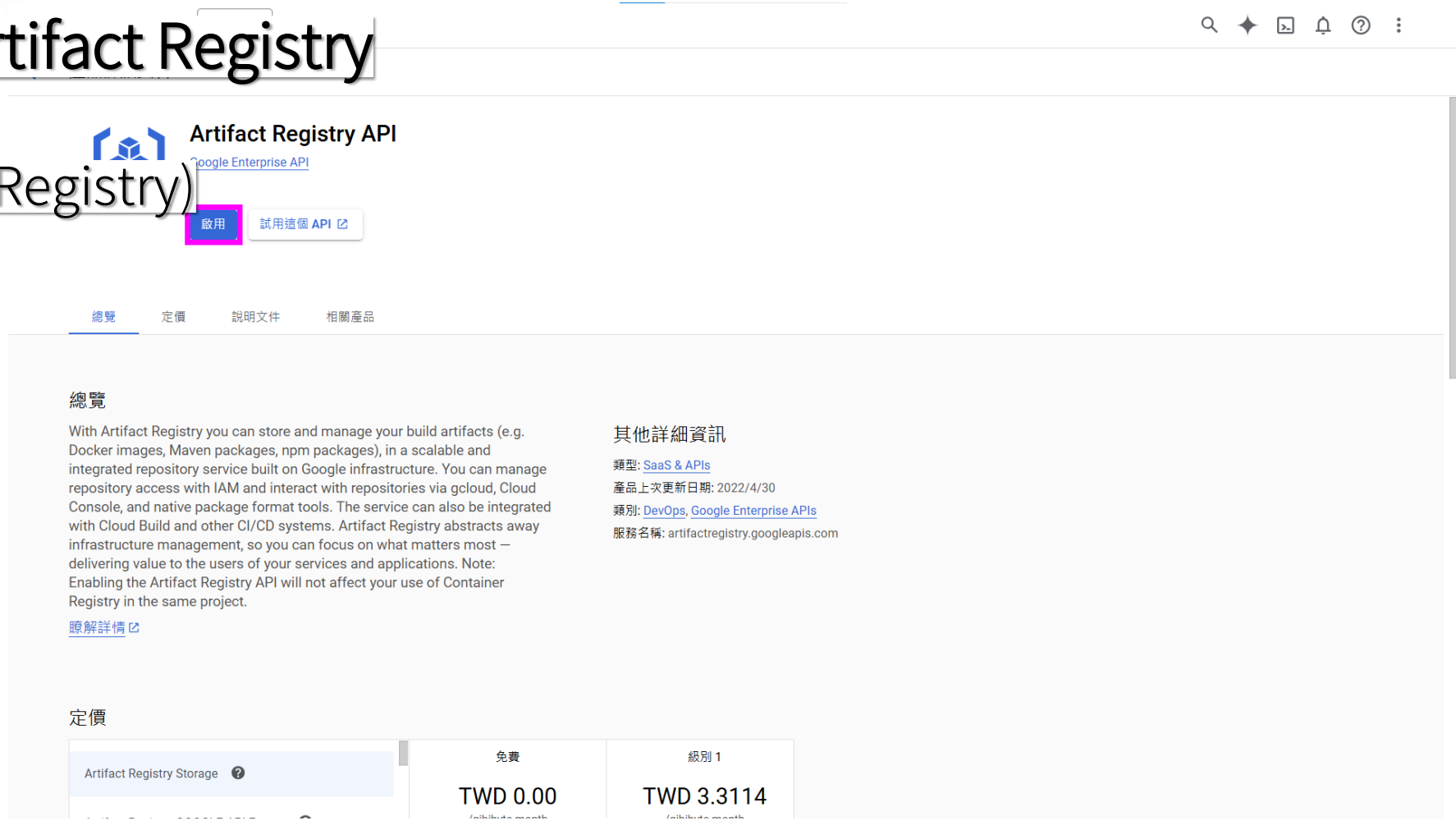
1. 建立 Google Artifact Registry
   a. (啟用 Artifact Registry)

© 2026 Enos Chou

Docker Image into Google Artifact Registry

1. 建立 Google Artifact Registry

   a. (啟用 Artifact Registry)



Docker Image into Google Artifact Registry

1. 建立 Google Artifact Registry

   a. (啟用 Artifact Registry)

Docker Image into Google Artifact Registry

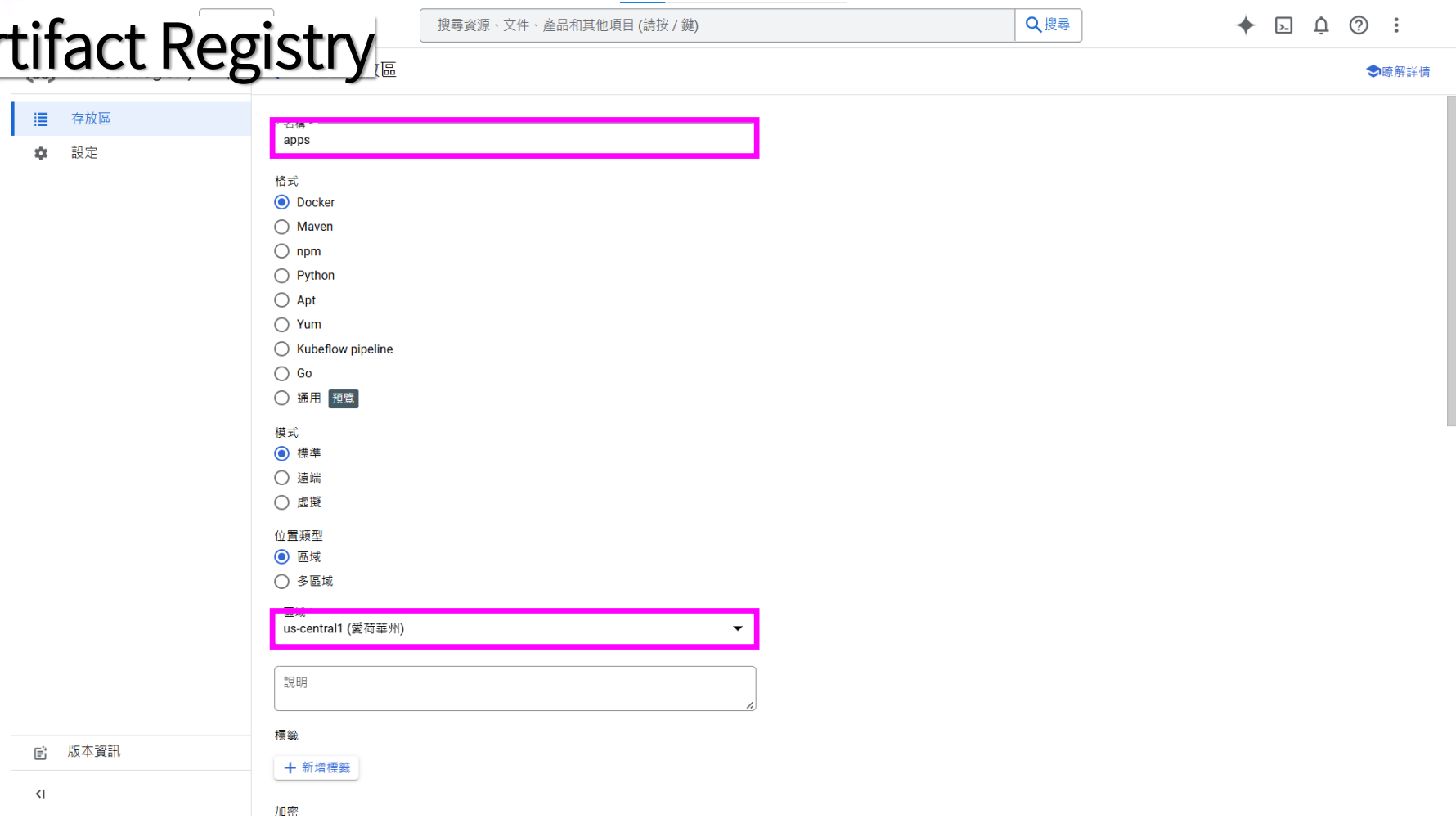
1. 建立 Google Artifact Registry

   b. (建立存放區)

Docker Image into Google Artifact Registry

1. 建立 Google Artifact Registry

   b. (建立存放區)



Docker Image into Google Artifact Registry

1. 建立 Google Artifact Registry

   b. (建立存放區)

Docker Image into Google Artifact Registry

1. 建立 Google Artifact Registry

   b. (建立存放區)

Docker Image into Google Artifact Registry

1. 建立 Google Artifact Registry

   c. 進入存放區

Docker Image into Google Artifact Registry

1. 建立 Google Artifact Registry

   c. 進入存放區

Docker Image into Google Artifact Registry

2. 建立並推送 Docker Image

a. by Google Cloud Shell

① 開啟 Cloud Shell

Docker Image into Google Artifact Registry

2. 建立並推送 Docker Image

a. by Google Cloud Shell

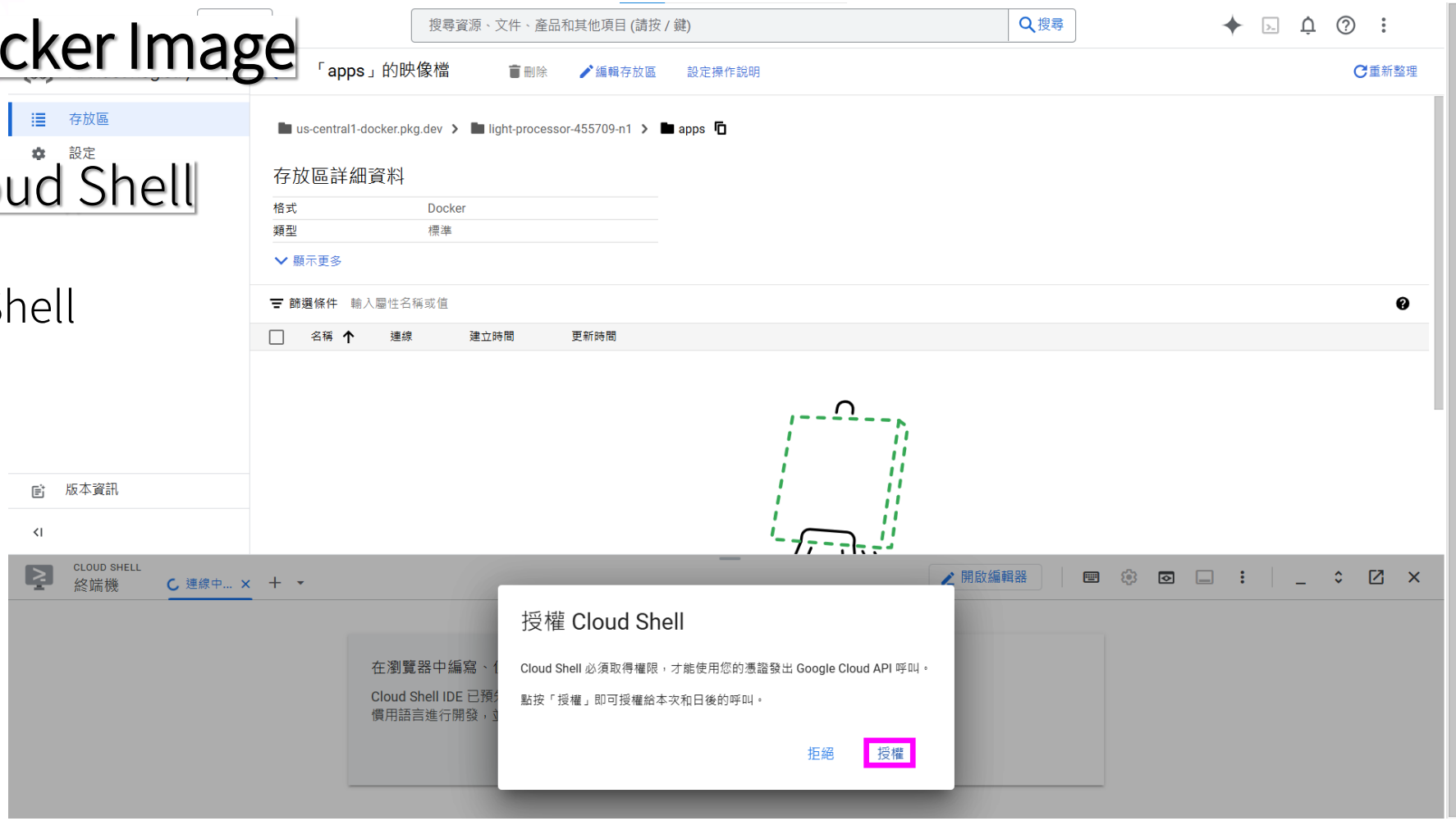
① 開啟 Cloud Shell

Docker Image into Google Artifact Registry

2. 建立並推送 Docker Image

a. by Google Cloud Shell

① 開啟 Cloud Shell



Docker Image into Google Artifact Registry

2. 建立並推送 Docker Image

a. by Google Cloud Shell

① 開啟 Cloud Shell

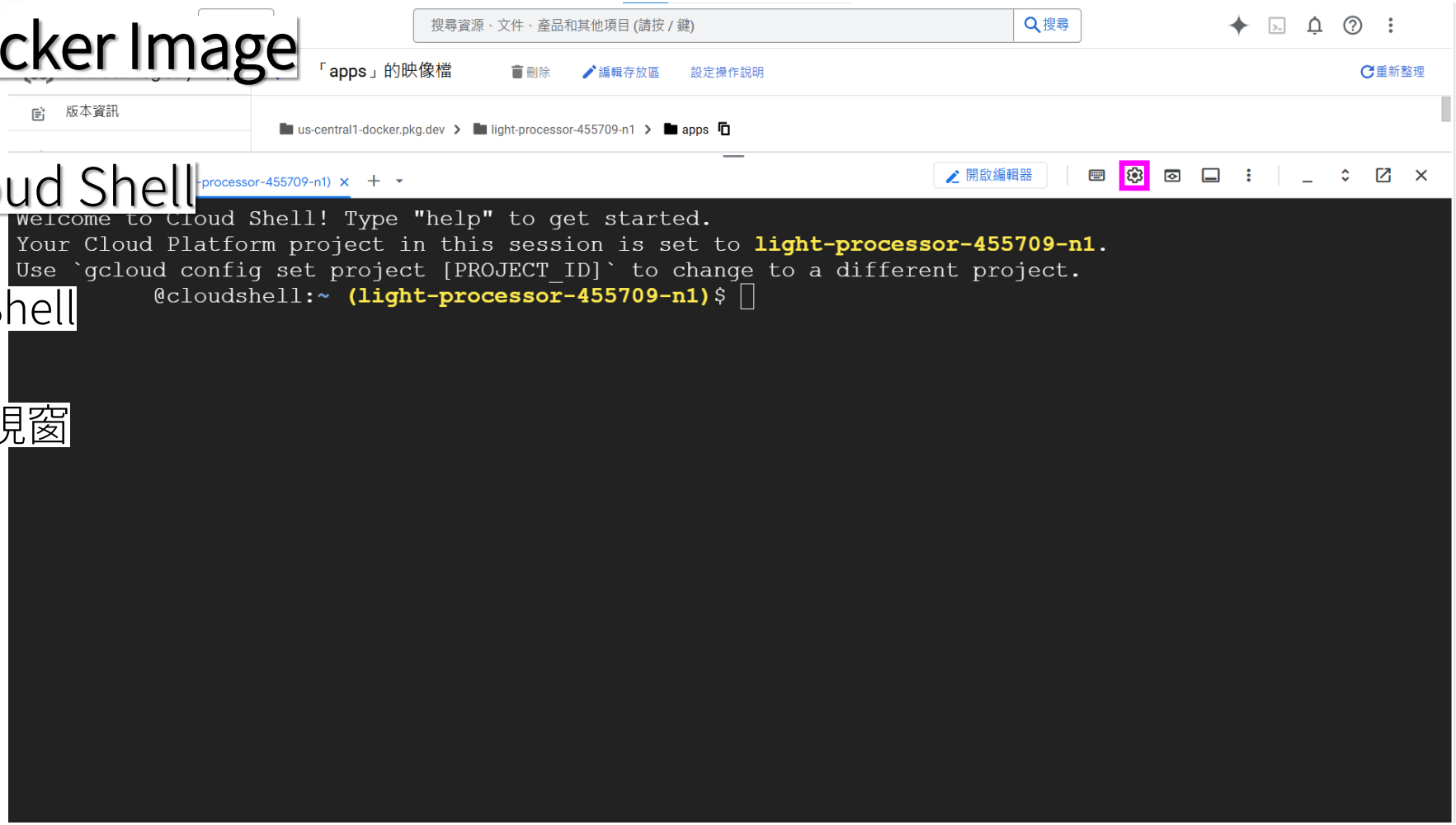
© 2026 Enos Chou

Docker Image into Google Artifact Registry

2. 建立並推送 Docker Image

a. by Google Cloud Shell

① 開啟 Cloud Shell



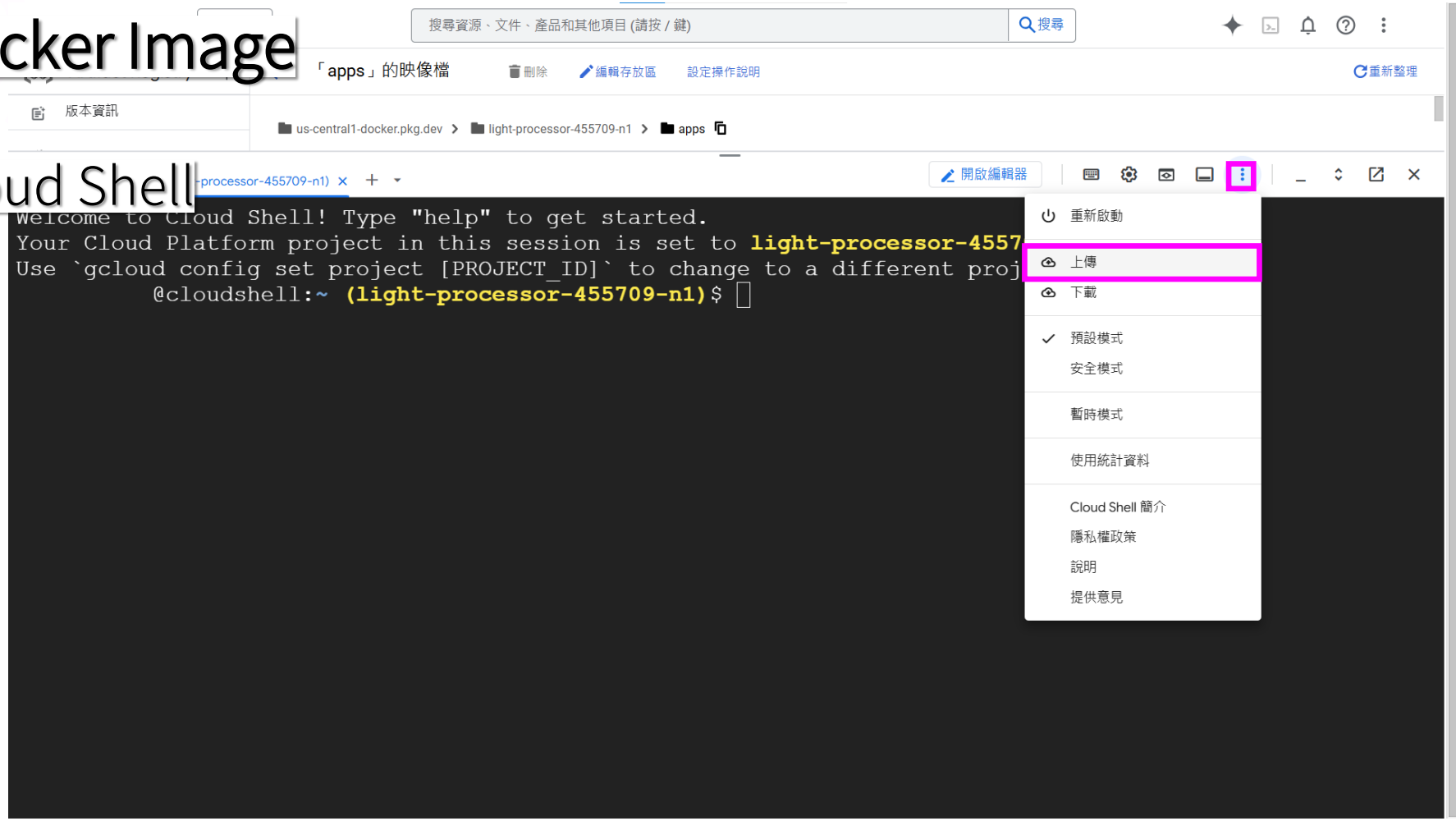
放大字型，拉大視窗

Docker Image into Google Artifact Registry

2. 建立並推送 Docker Image

a. by Google Cloud Shell

② 上傳 Source

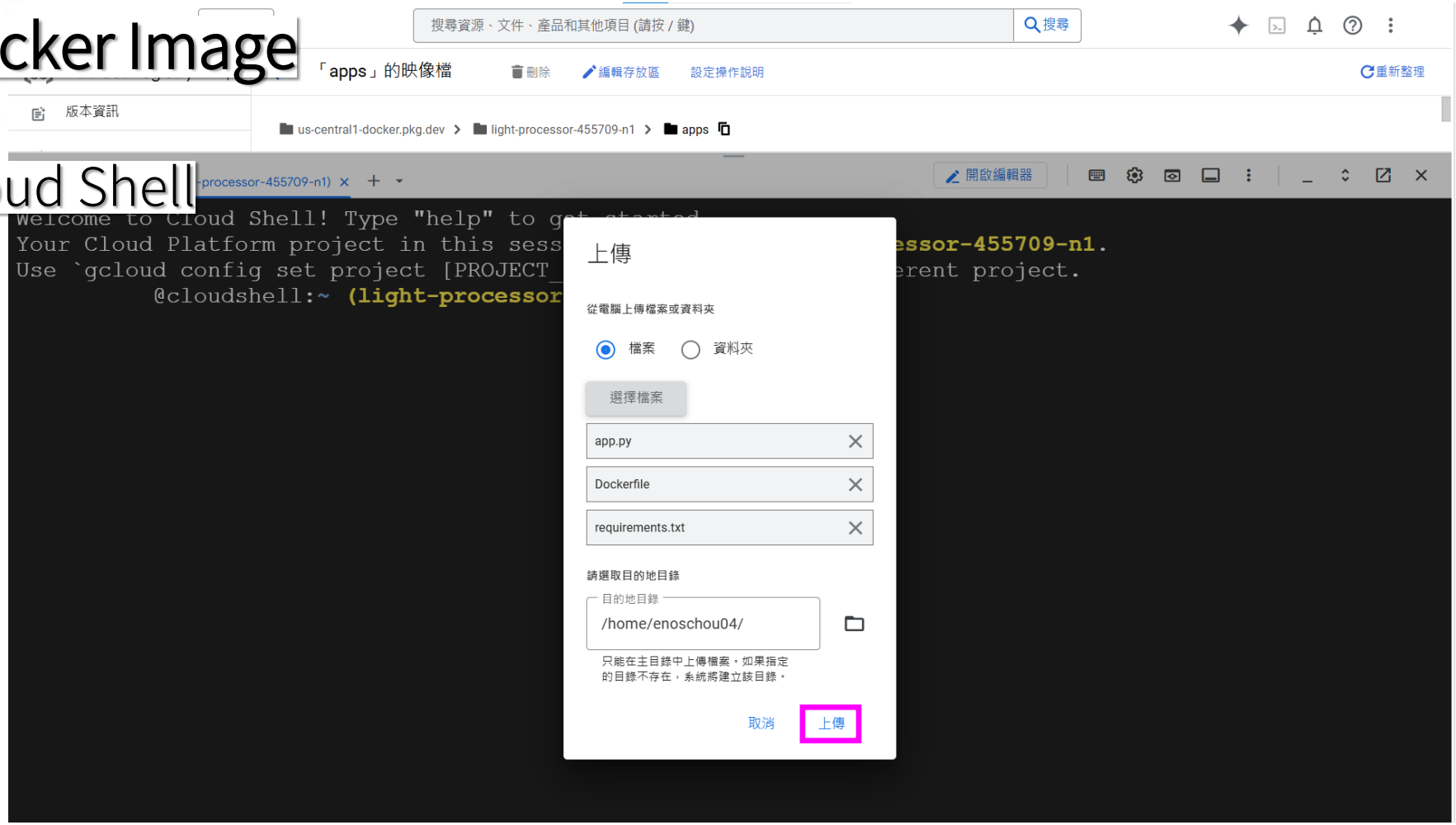


Docker Image into Google Artifact Registry

2. 建立並推送 Docker Image

a. by Google Cloud Shell

② 上傳 Source



Docker Image into Google Artifact Registry

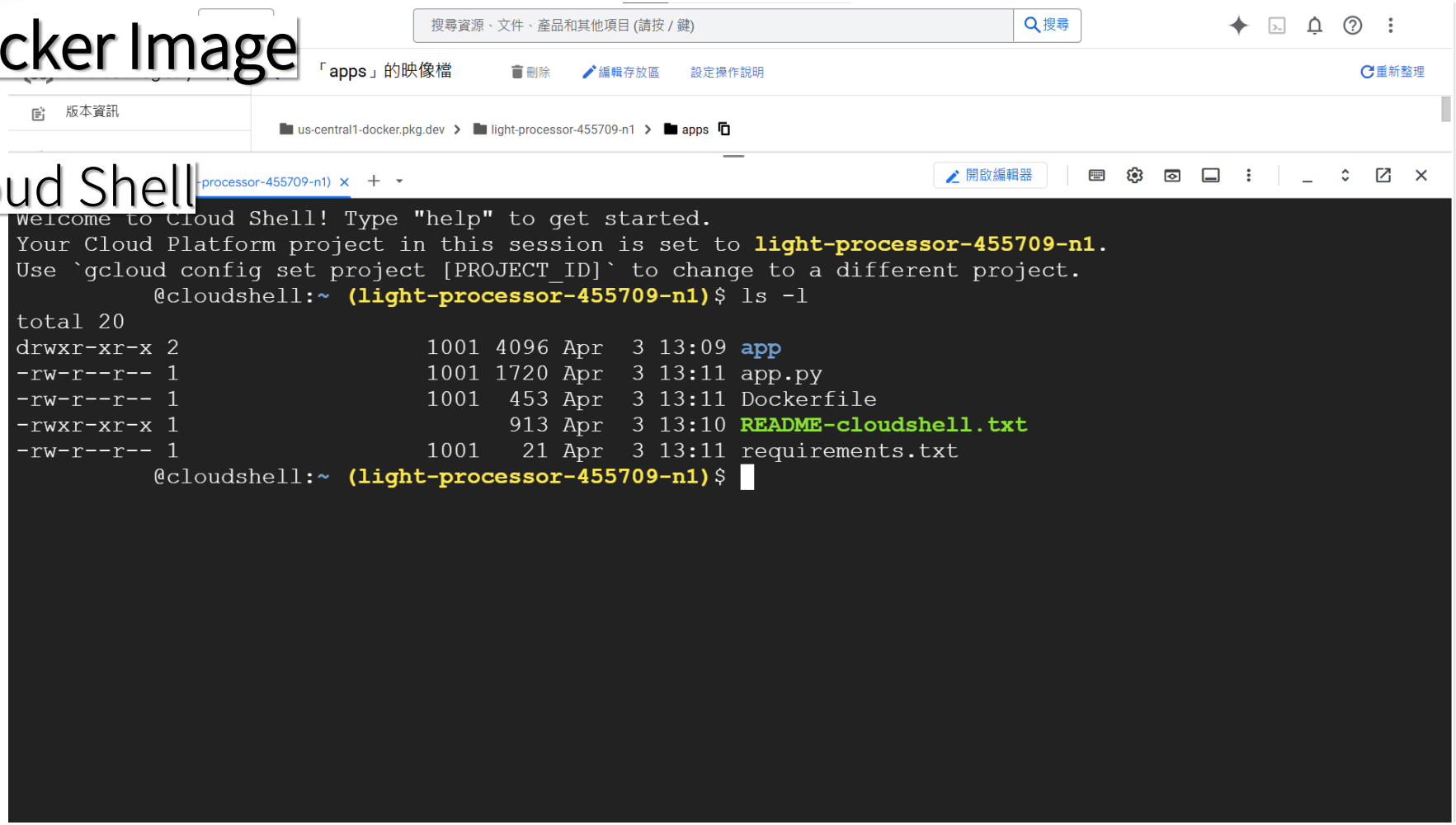
2. 建立並推送 Docker Image

a. by Google Cloud Shell

② 上傳 Source

確認 source

```
ls -l
```



Docker Image into Google Artifact Registry

2. 建立並推送 Docker Image

a. by Google Cloud Shell

② 上傳 Source

Reference of Dockerfile

```
FROM python:3.12.16--trixie
WORKDIR /app
COPY . ./
ENV TZ=Asia/Taipei \
    LANG=C.UTF-8 \
    PYTHONUNBUFFERED=1
RUN pip3 install --no-cache-dir -r requirements.txt gunicorn
# CMD ["bash", "-c", "gunicorn app:app -b :${PORT:-8080}"]
CMD exec gunicorn app:app -b :${PORT:-8080}
```

© 2026 Enos Chou

Docker Image into Google Artifact Registry

2. 建立並推送 Docker Image

a. by Google Cloud Shell

③ 啟用 Cloud Build

© 2026 Enos Chou

Docker Image into Google Artifact Registry

2. 建立並推送 Docker Image

a. by Google Cloud Shell

④ 新增預設服務帳戶與權限

新增 "Compute Engine default service account" 以及下列角色

Storage 物件使用者
Artifact Registry 推送建立寫入者

© 2026 Enos Chou

Docker Image into Google Artifact Registry

2. 建立並推送 Docker Image

a. by Google Cloud Shell

⑤ 建立並推送 Docker Image

```
gcloud builds submit --tag your_region-docker.pkg.dev/your_projectid/your_reg/your_repo:your_tag .
gcloud builds submit --tag us-central1-docker.pkg.dev/light-processor-455709-n1/apps/app:v1 .
```

© 2026 Enos Chou

Docker Image into Google Artifact Registry

2. 建立並推送 Docker Image

a. by Google Cloud Shell

⑥ 確認

Docker Image into Google Artifact Registry

2. 建立並推送 Docker Image

a. by Google Cloud Shell

⑥ 確認

Docker Image into Google Artifact Registry

2. 建立並推送 Docker Image

a. by Google Cloud Shell

⑥ 確認

Docker Image into Google Artifact Registry

2. 建立並推送 Docker Image

b. by Pure Docker

① 建立 Docker Image

```
docker build ...
```

© 2026 Enos Chou

Docker Image into Google Artifact Registry

2. 建立並推送 Docker Image

b. by Pure Docker

② 改名 Docker Image

```
docker tag ...
```

© 2026 Enos Chou

Docker Image into Google Artifact Registry

2. 建立並推送 Docker Image

b. by Pure Docker

③ 登入 Google Artifact Registry

```
type your_service.json | docker login -u _json_key --password-stdin https://your_region-docker.pkg.dev
```

或

```
cat your_service.json | docker login -u _json_key --password-stdin https://your_region-docker.pkg.dev
```

Note

your_service.json 為服務帳戶金鑰，須要 "Artifact Registry: Artifact Registry 推送建立寫入者" 角色

© 2026 Enos Chou

Docker Image into Google Artifact Registry

2. 建立並推送 Docker Image

b. by Pure Docker

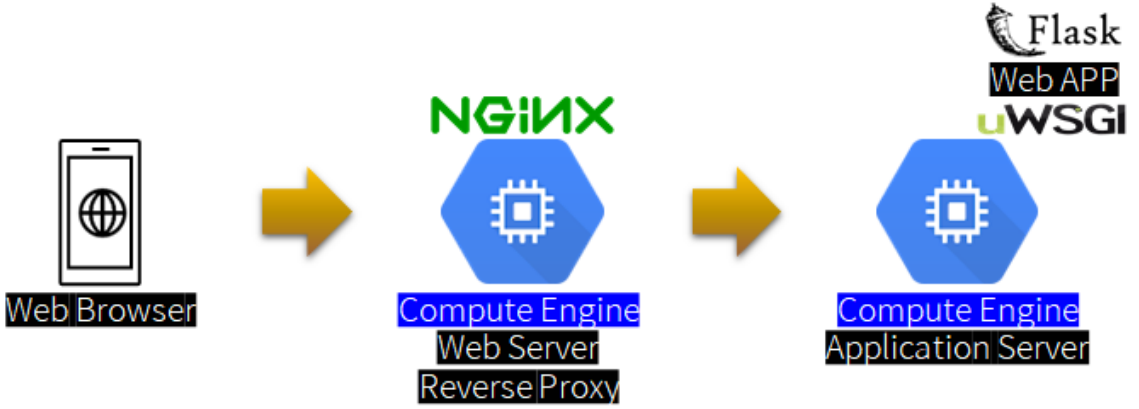
④ 上傳 Docker Image

```
docker push ...
```

© 2026 Enos Chou

Google Compute Engine

Containerized VM



Google Cloud Run

Containerized/ Serverless

APP 部署 ft. Containerized GCE

1. 建立容器化 GCE (VM)
2. 調整 NGINX 之反向代理設定
3. 測試與除錯

© 2026 Enos Chou

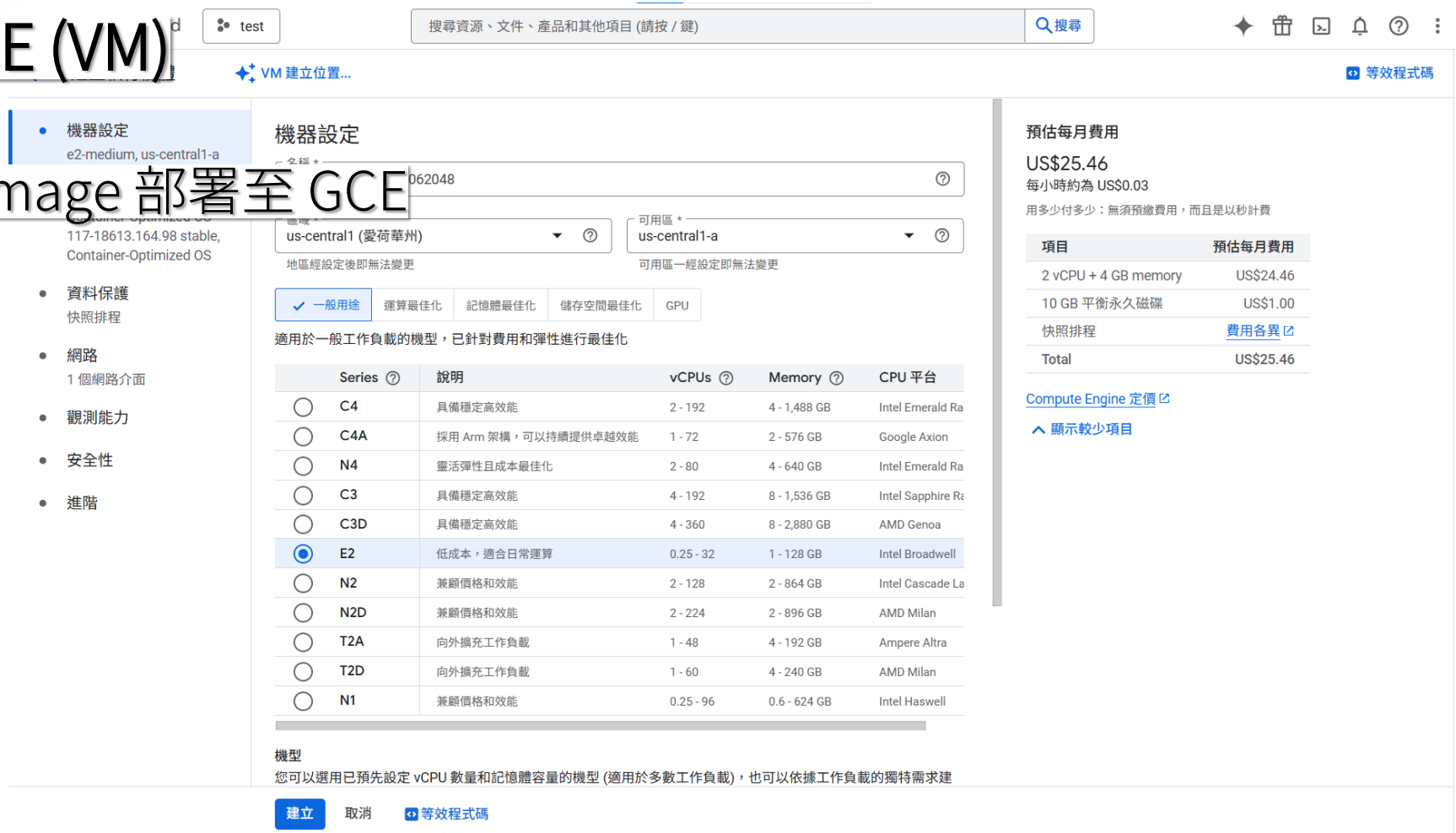
APP 部署 ft. Containerized GCE

1. 建立容器化 GCE (VM)
   a. 選取 Docker Image 部署至 GCE

© 2026 Enos Chou

APP 部署 ft. Containerized GCE

1. 建立容器化 GCE (VM)
   a. 選取 Docker Image 部署至 GCE



APP 部署 ft. Containerized GCE

1. 建立容器化 GCE (VM)

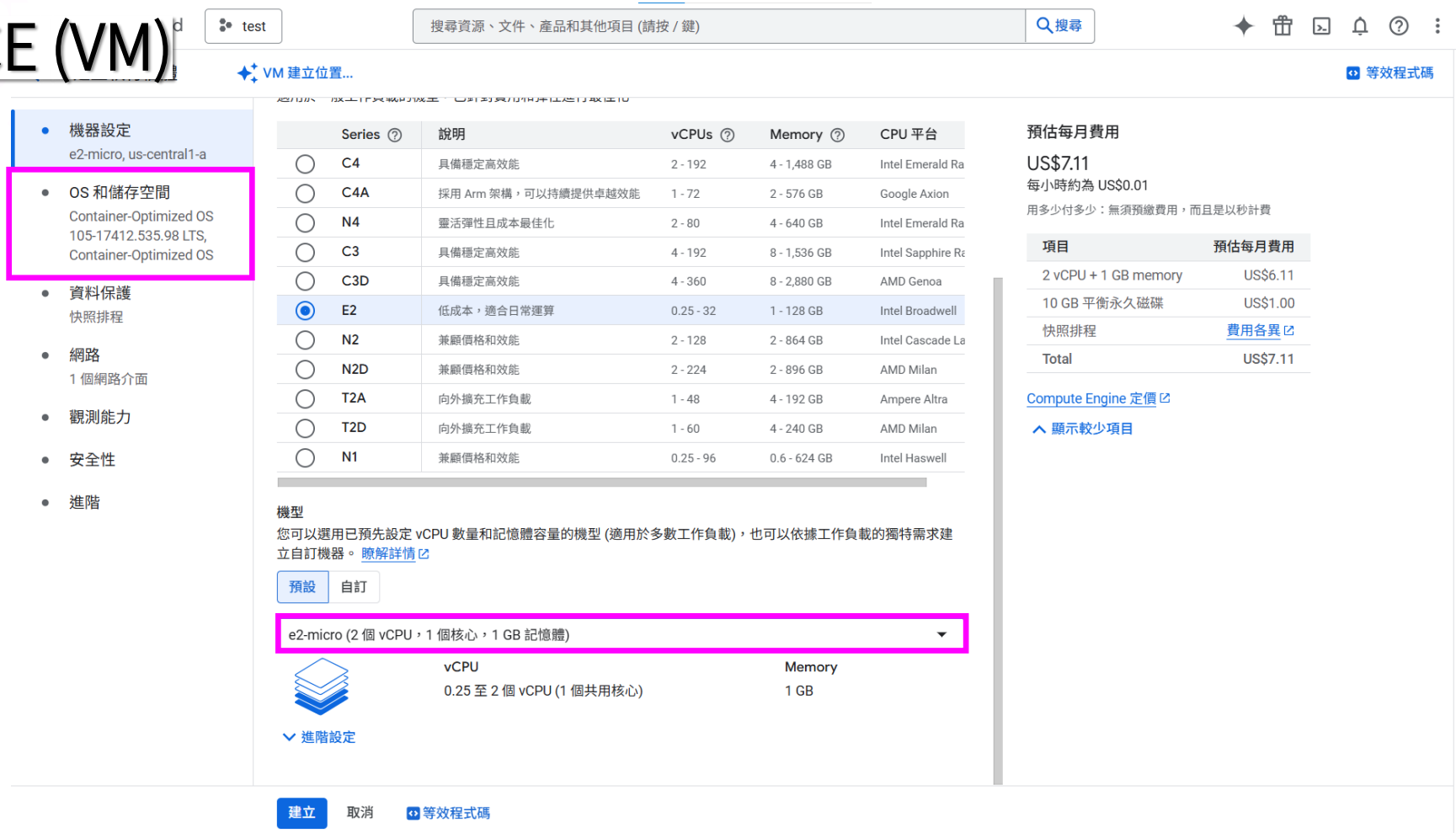
   b. 調整 VM 參數

© 2026 Enos Chou

APP 部署 ft. Containerized GCE

1. 建立容器化 GCE (VM)

   b. 調整 VM 參數



APP 部署 ft. Containerized GCE

1. 建立容器化 GCE (VM)

   b. 調整 VM 參數

APP 部署 ft. Containerized GCE

1. 建立容器化 GCE (VM)

   b. 調整 VM 參數

© 2026 Enos Chou

APP 部署 ft. Containerized GCE

1. 建立容器化 GCE (VM)

   b. 調整 VM 參數

      ① 視需要新增環境變數

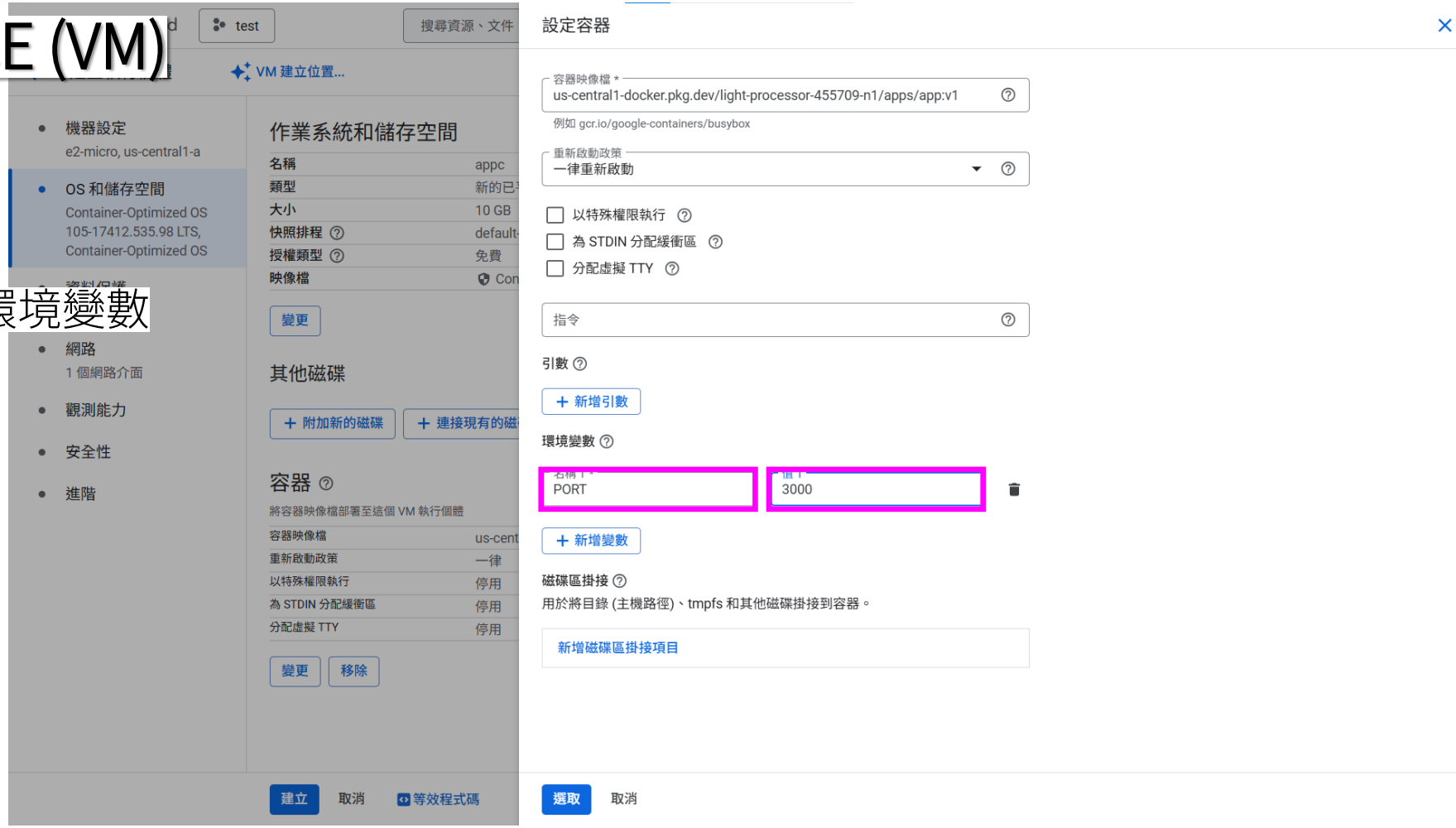
© 2026 Enos Chou

APP 部署 ft. Containerized GCE

1. 建立容器化 GCE (VM)

   b. 調整 VM 參數

      ① 視需要新增環境變數



© 2026 Enos Chou

APP 部署 ft. Containerized GCE

1. 建立容器化 GCE (VM)

   b. 調整 VM 參數

APP 部署 ft. Containerized GCE

1. 建立容器化 GCE (VM)

   b. 調整 VM 參數

APP 部署 ft. Containerized GCE

1. 建立容器化 GCE (VM)

b. 調整 VM 參數

② 視需求保留內部或外部固定 IP

© 2026 Enos Chou

APP 部署 ft. Containerized GCE

1. 建立容器化 GCE (VM)

b. 調整 VM 參數

② 視需求保留內部或外部固定 IP

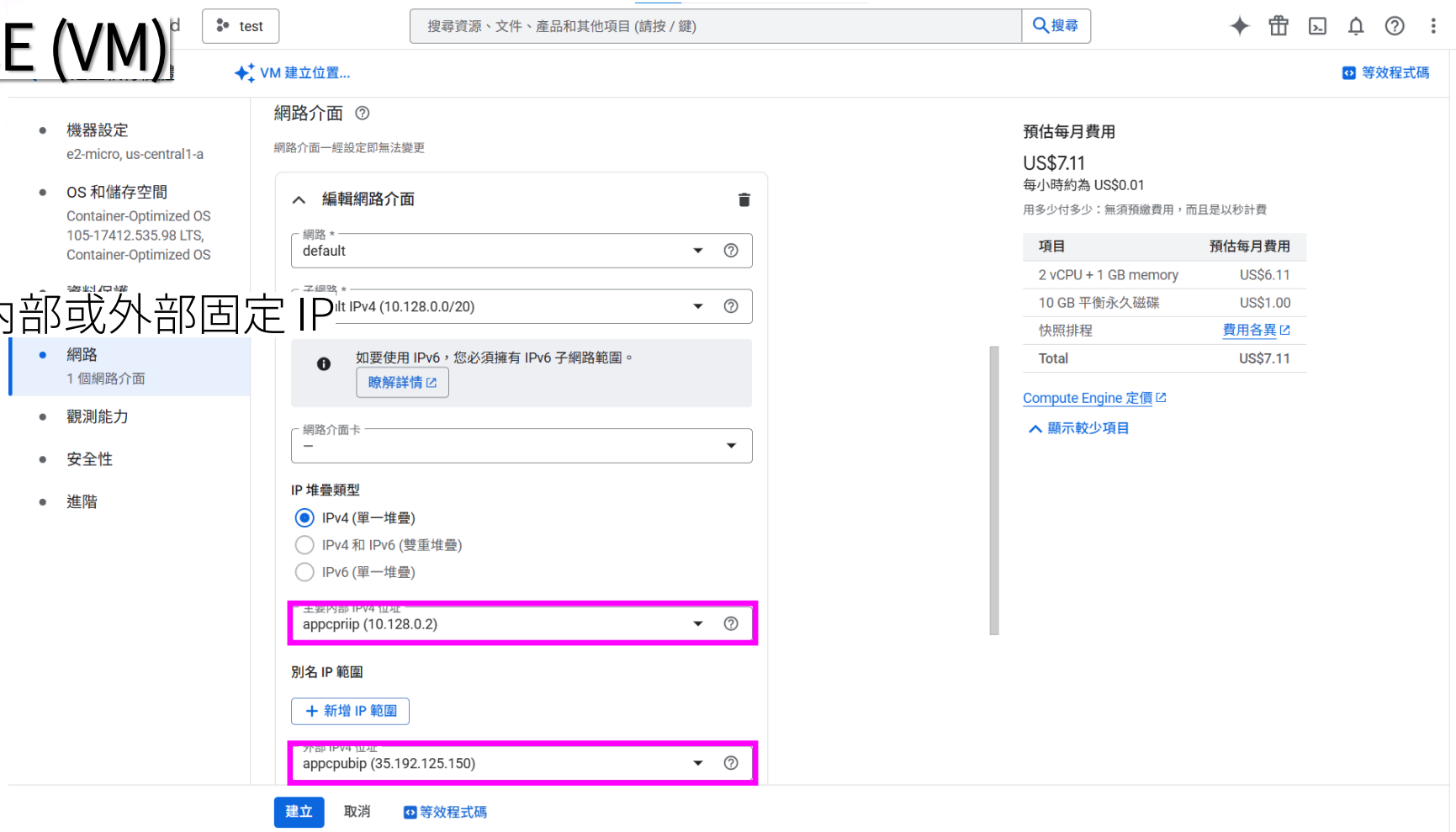
© 2026 Enos Chou

APP 部署 ft. Containerized GCE

1. 建立容器化 GCE (VM)

b. 調整 VM 參數

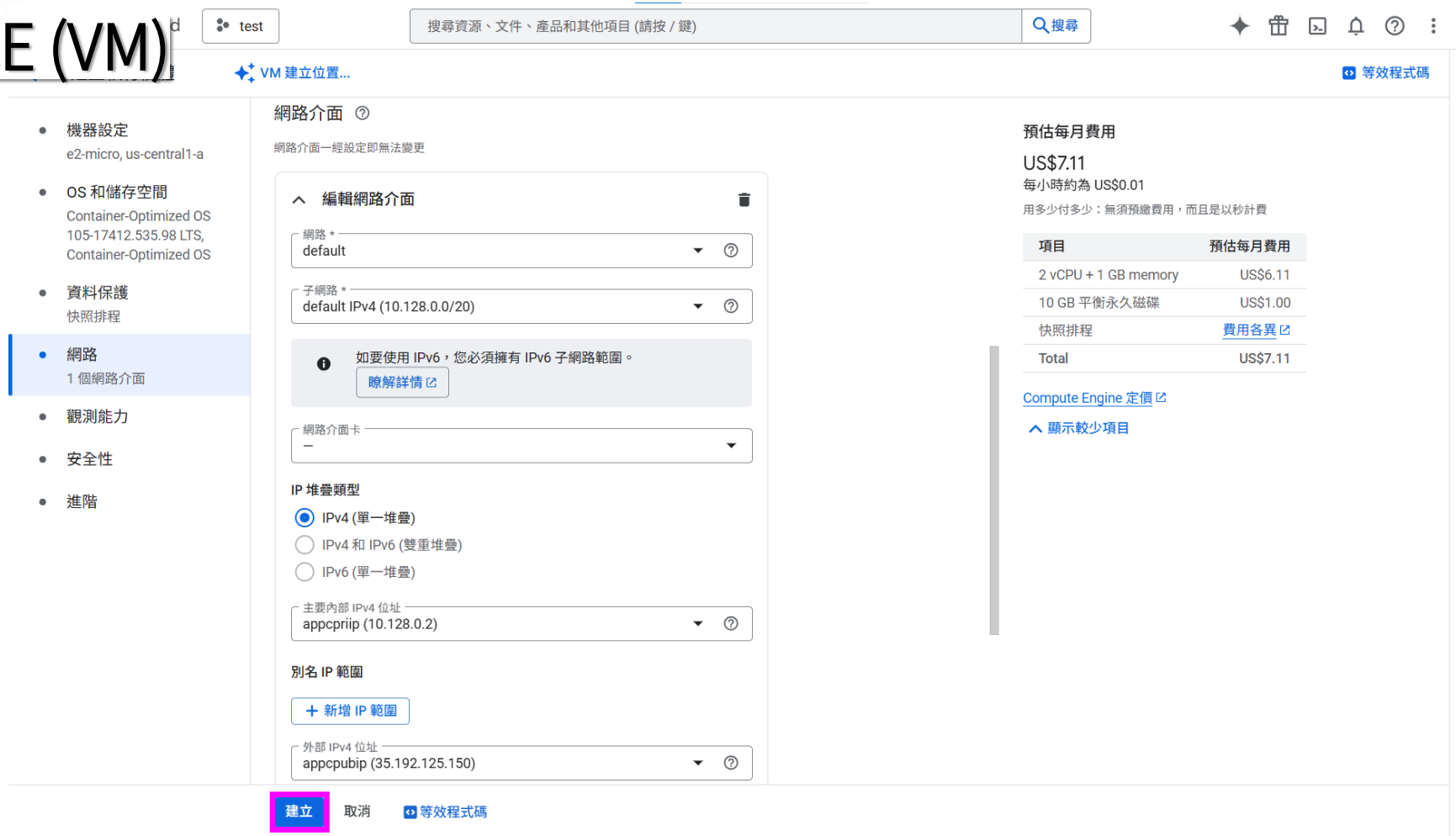
② 視需求保留內部或外部固定 IP



APP 部署 ft. Containerized GCE

1. 建立容器化 GCE (VM)

   c. 建立 GCE



APP 部署 ft. Containerized GCE

1. 建立容器化 GCE (VM)

   c. 建立 GCE

      取得 IP

© 2026 Enos Chou

APP 部署 ft. Containerized GCE

2. 調整 NGINX 之反向代理設定

a. 調整 NGINX 組態

① 以 SSH 登入 NGINX 所在 VM
② 以文字編輯器開啟 NGINX 組態檔 /etc/nginx/sites-enabled/*your_conf*.conf

```
server {
    server_name your_domain;

    location / {
        include uwsgi_params;
        #uwsgi_pass your_ip:your_port;
        proxy_pass http://your_ip:your_port;
    }
}
```

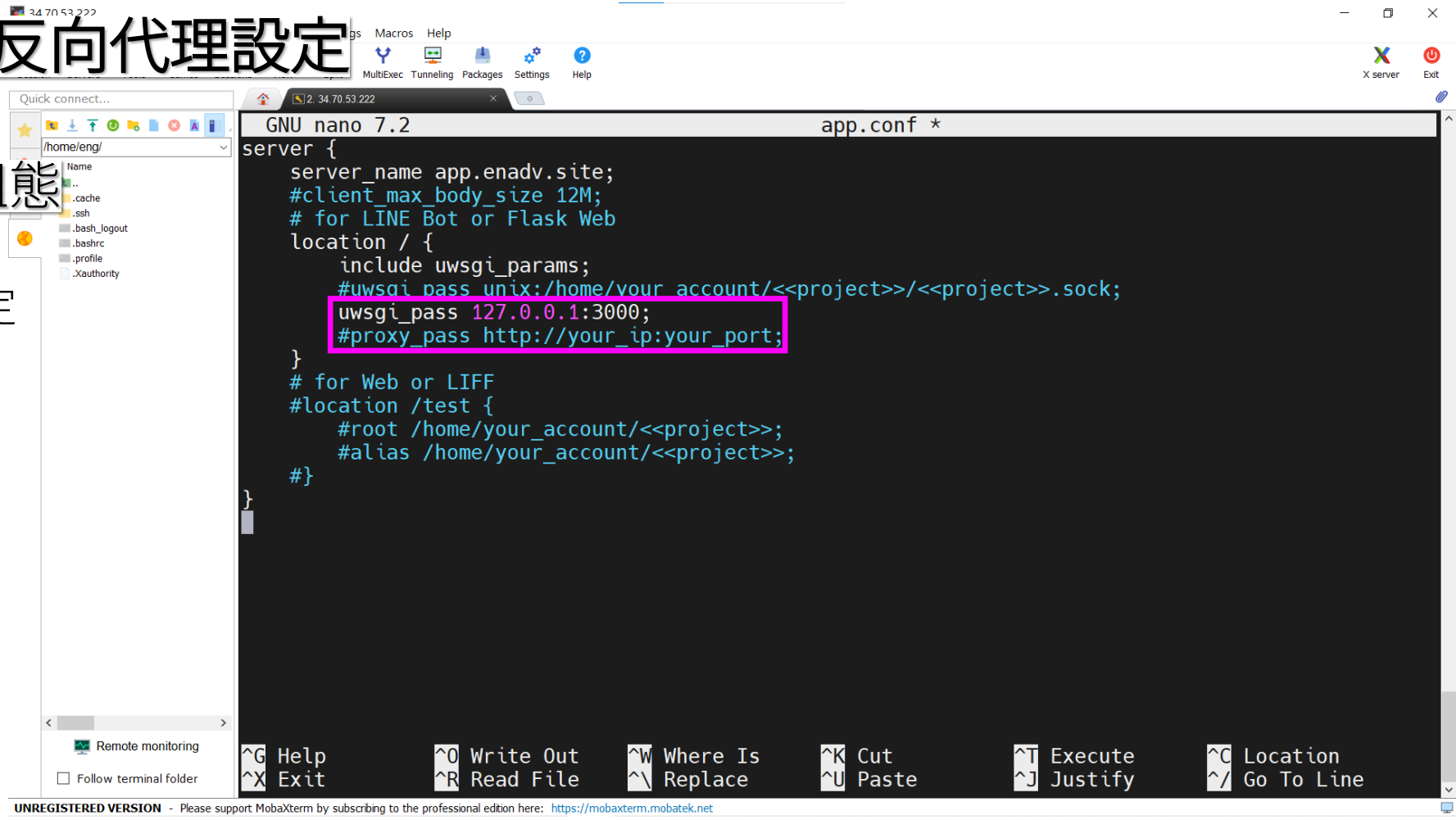
© 2026 Enos Chou

APP 部署 ft. Containerized GCE

2. 調整 NGINX 之反向代理設定

a. 調整 NGINX 組態

③ 調整轉導設定

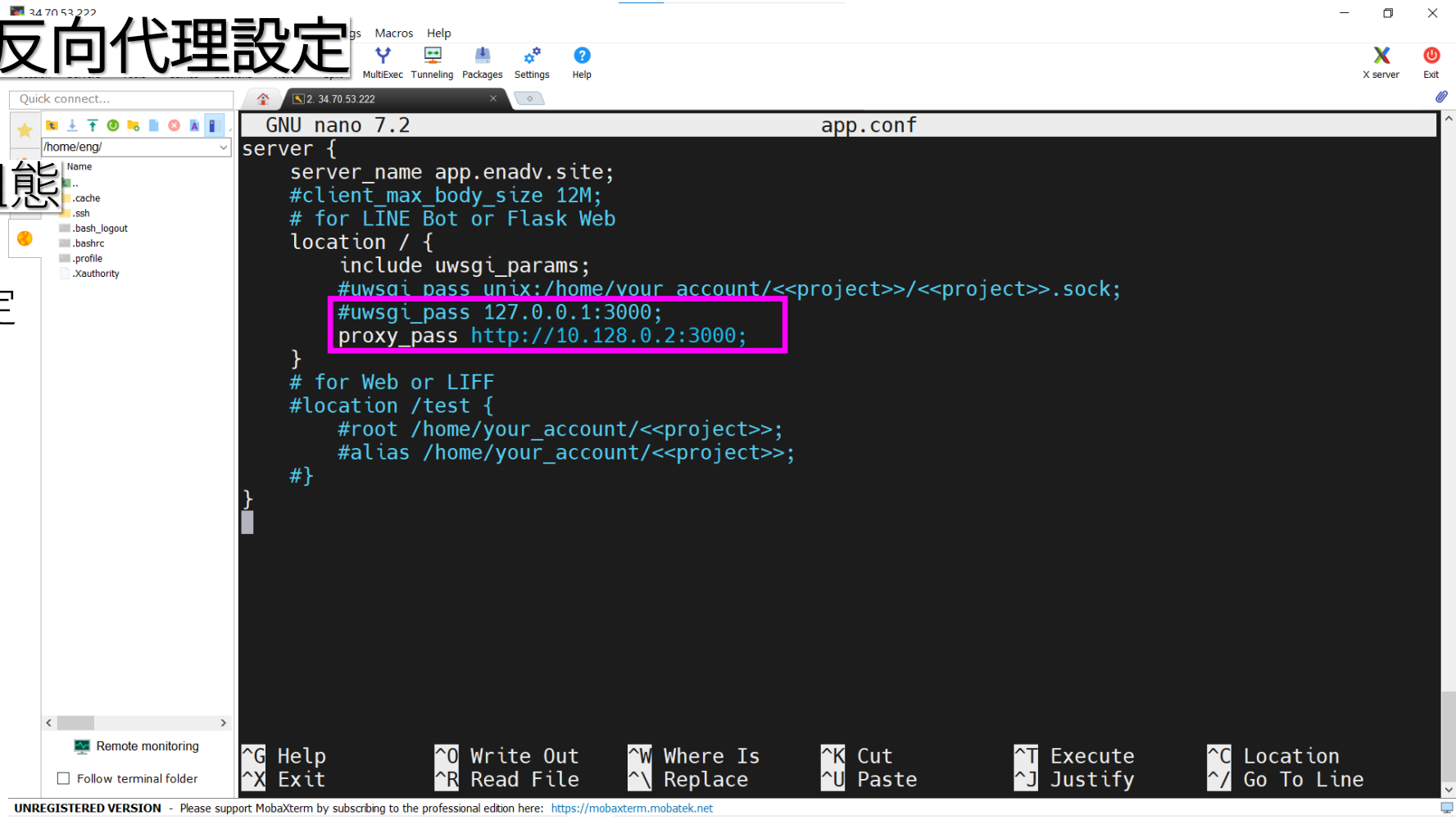


APP 部署 ft. Containerized GCE

2. 調整 NGINX 之反向代理設定

a. 調整 NGINX 組態

③ 調整轉導設定



© 2026 Enos Chou

APP 部署 ft. Containerized GCE

2. 調整 NGINX 之反向代理設定

a. 調整 NGINX 組態

④ 儲存並退出

```
# 儲存
CTRL+o

ENTER

# 退出
CTRL+x
```

© 2026 Enos Chou

APP 部署 ft. Containerized GCE

2. 調整 NGINX 之反向代理設定

b. 更新 NGINX 組態

① (以 SSH 登入 NGINX in VM)
② 重新載入 NGINX 組態檔

```
sudo nginx -s reload
```

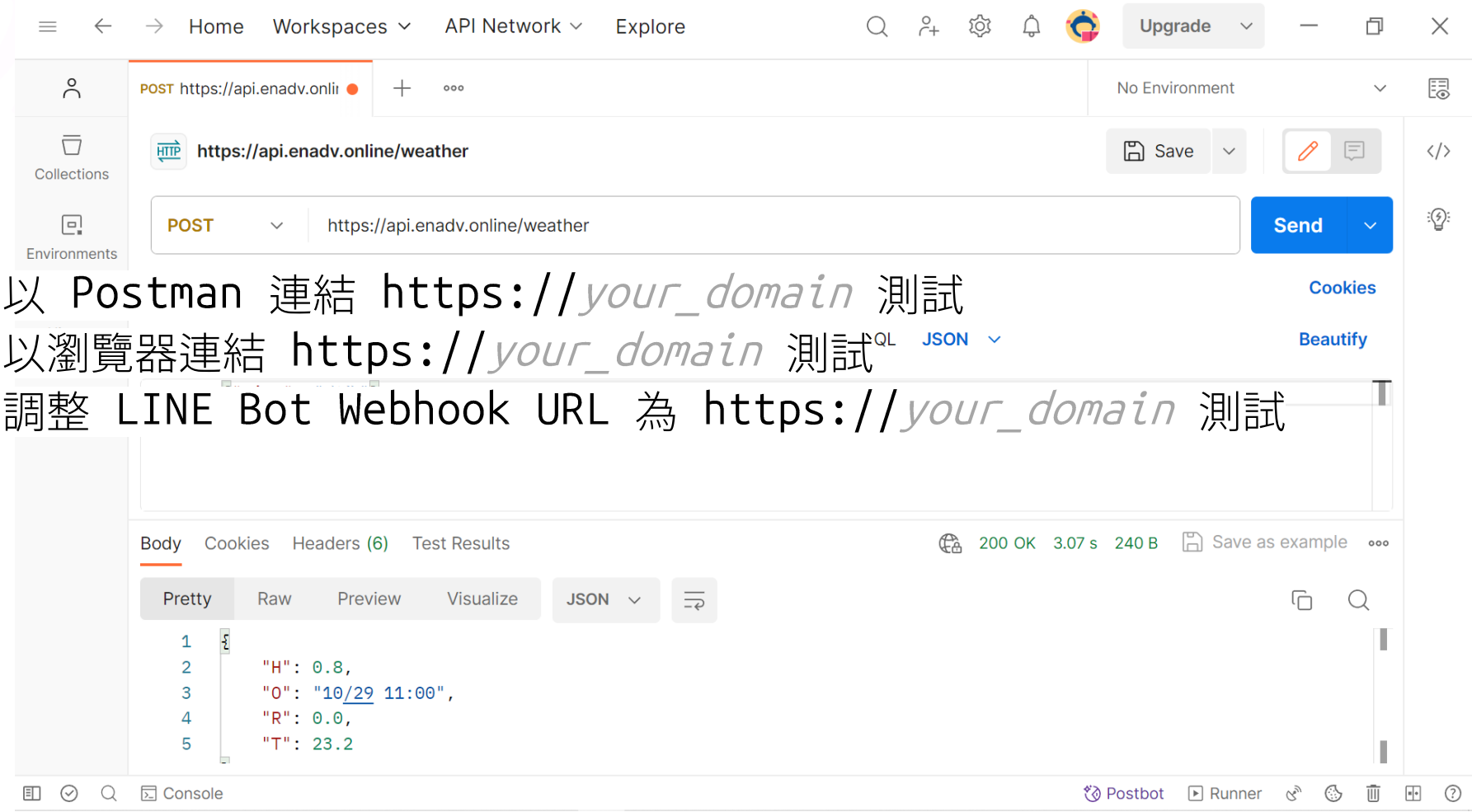
© 2026 Enos Chou

APP 部署 ft. Containerized GCE

3. 測試與除錯

a. 測試

 # API - 以 Postman 連結 https://*your_domain* 測試
 # Web - 以瀏覽器連結 https://*your_domain* 測試
 # LINE Bot - 調整 LINE Bot Webhook URL 為 https://*your_domain* 測試



APP 部署 ft. Containerized GCE

3. 測試與除錯

b. 除錯

登入 GCE 主機觀察容器輸出

① 以終端機登入 GCE 主機
② 取得 *Container_ID*

```
docker ps
```

③ 觀察既有執行環境的標準輸出

```
docker attach Container_ID
```

若 CTRL+c, CTRL+p, CTRL+q 無法退出可逕行關閉終端機

④ (建立新的執行環境觀察容器狀態)

```
docker exec -it Container_ID bash
```

© 2026 Enos Chou

Google Compute Engine
Containerized VM

Google Cloud Run

Containerized/ Serverless

APP 部署 ft. Google Cloud Run

1. 建立 Google Cloud Run 服務
2. 測試與除錯
3. 更新程式
4. 設定參數
5. 連結內網 VM/ DB
6. 伴隨容器 (Sidecar)
7. 共享磁碟
8. 轉換網域
9. 要求認證

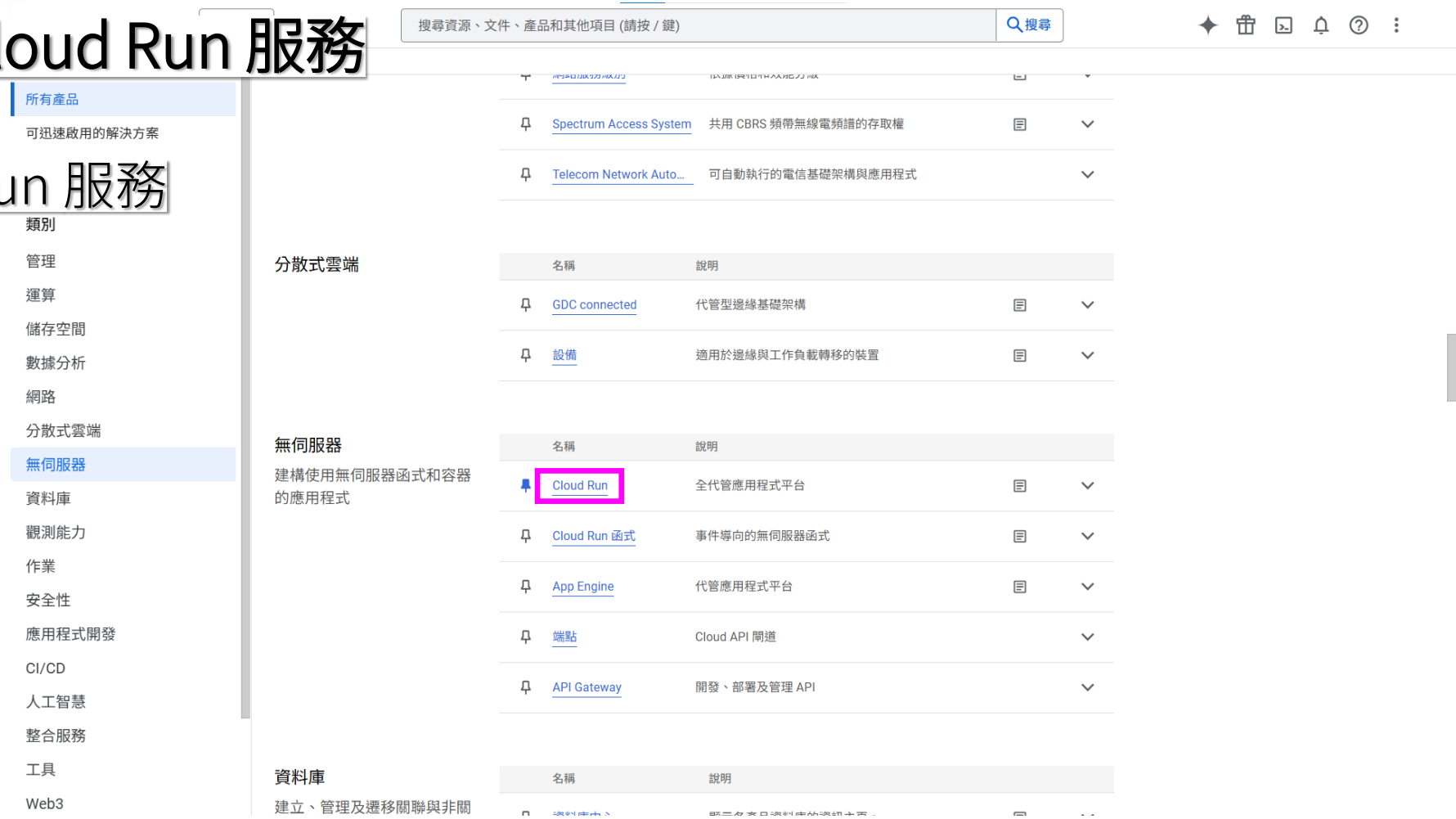
基礎

進階

© 2026 Enos Chou

APP 部署 ft. Google Cloud Run

1. 建立 Google Cloud Run 服務
   a. 建立 Cloud Run 服務



APP 部署 ft. Google Cloud Run

1. 建立 Google Cloud Run 服務

   a. 建立 Cloud Run 服務

APP 部署 ft. Google Cloud Run

1. 建立 Google Cloud Run 服務

   a. 建立 Cloud Run 服務

© 2026 Enos Chou

APP 部署 ft. Google Cloud Run

1. 建立 Google Cloud Run 服務

   a. 建立 Cloud Run 服務

© 2026 Enos Chou

APP 部署 ft. Google Cloud Run

1. 建立 Google Cloud Run 服務

   a. 建立 Cloud Run 服務

      取得 URL

APP 部署 ft. Google Cloud Run

2. 測試與除錯

a. 測試

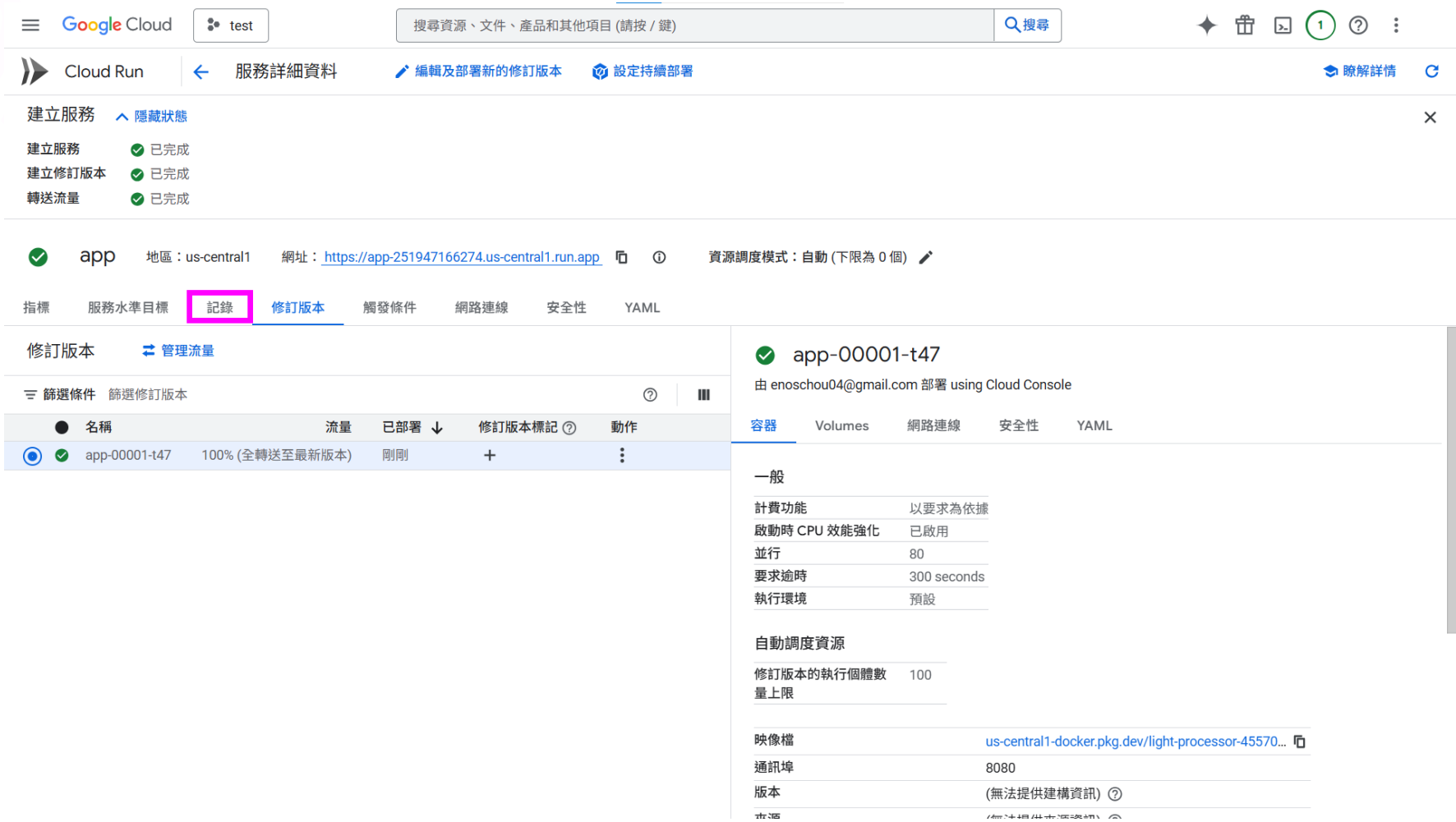
```
# API       - 以 Postman 連結 https://your_domain 測試
# Web       - 以瀏覽器連結 https://your_domain 測試
# LINE Bot  - 調整 LINE Bot Webhook URL 為 https://your_domain 測試
```

© 2026 Enos Chou

APP 部署 ft. Google Cloud Run

2. 測試與除錯

b. 除錯



© 2026 Enos Chou

APP 部署 ft. Google Cloud Run

2. 測試與除錯

b. 除錯

© 2026 Enos Chou

APP 部署 ft. Google Cloud Run

3. 更新程式

a. Push 新版 Docker Image 至 Google Artifact Registry

① 參閱 Docker Image into Google Artifact Registry

Note
須指定與既有 Docker Image 相同的 repository name ，不同的 tag
例：

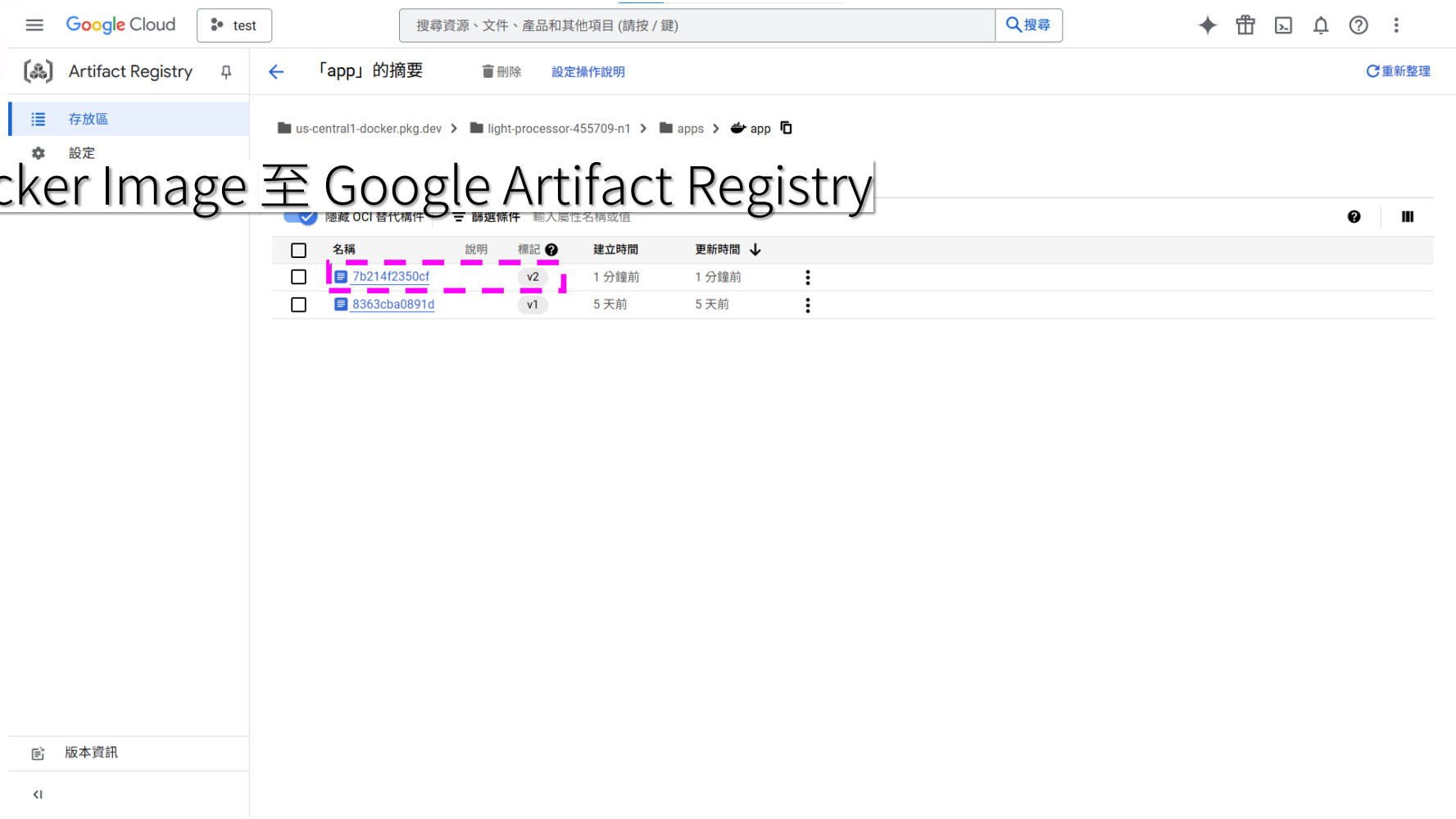
```
docker push us-central1-docker.pkg.dev/light-processor-455709-n1/apps/app:v2
```

APP 部署 ft. Google Cloud Run

3. 更新程式

a. Push 新版 Docker Image 至 Google Artifact Registry

② 確認

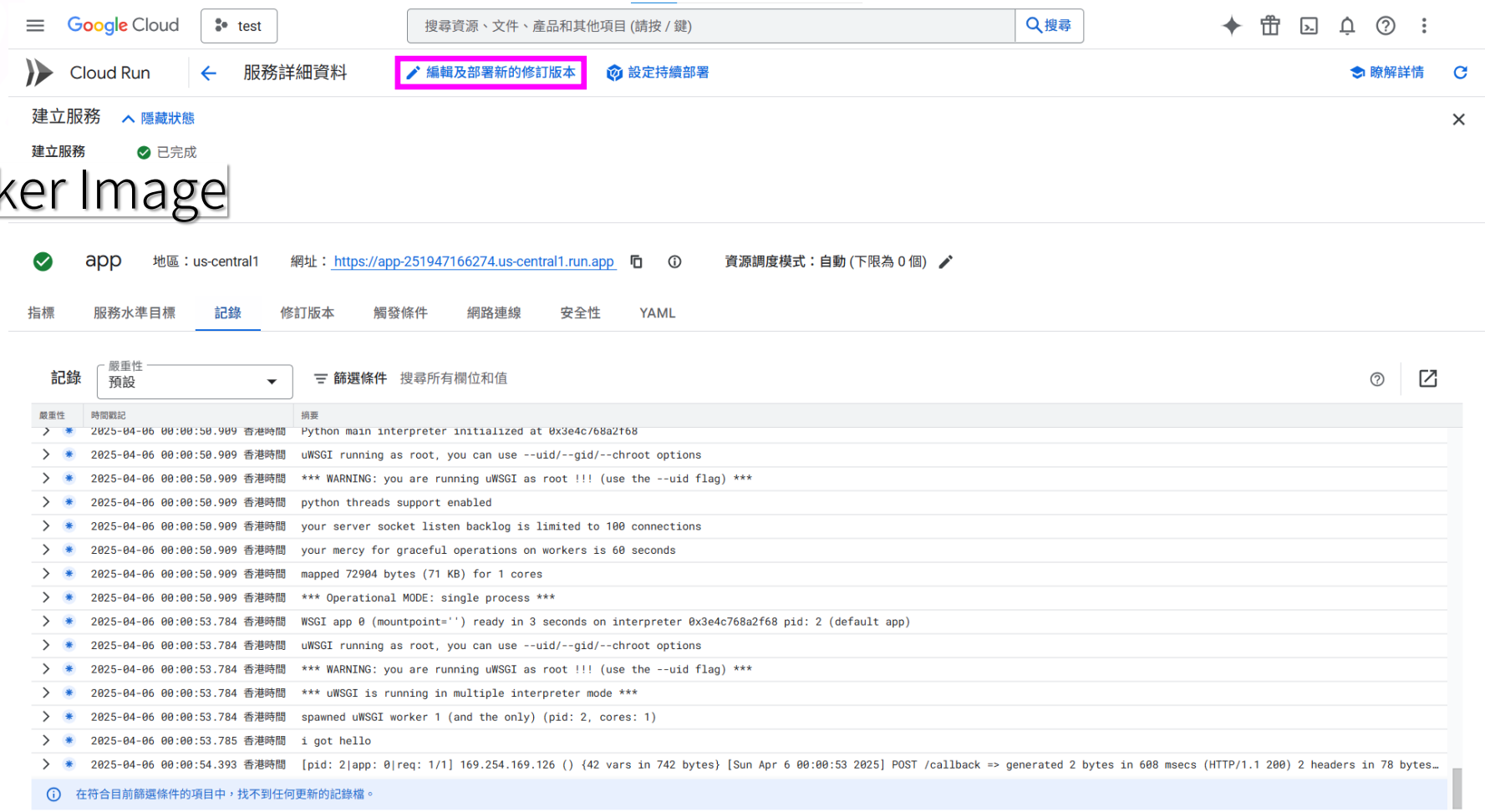


© 2026 Enos Chou

APP 部署 ft. Google Cloud Run

3. 更新程式

b. 部署新版 Docker Image



© 2026 Enos Chou

APP 部署 ft. Google Cloud Run

3. 更新程式

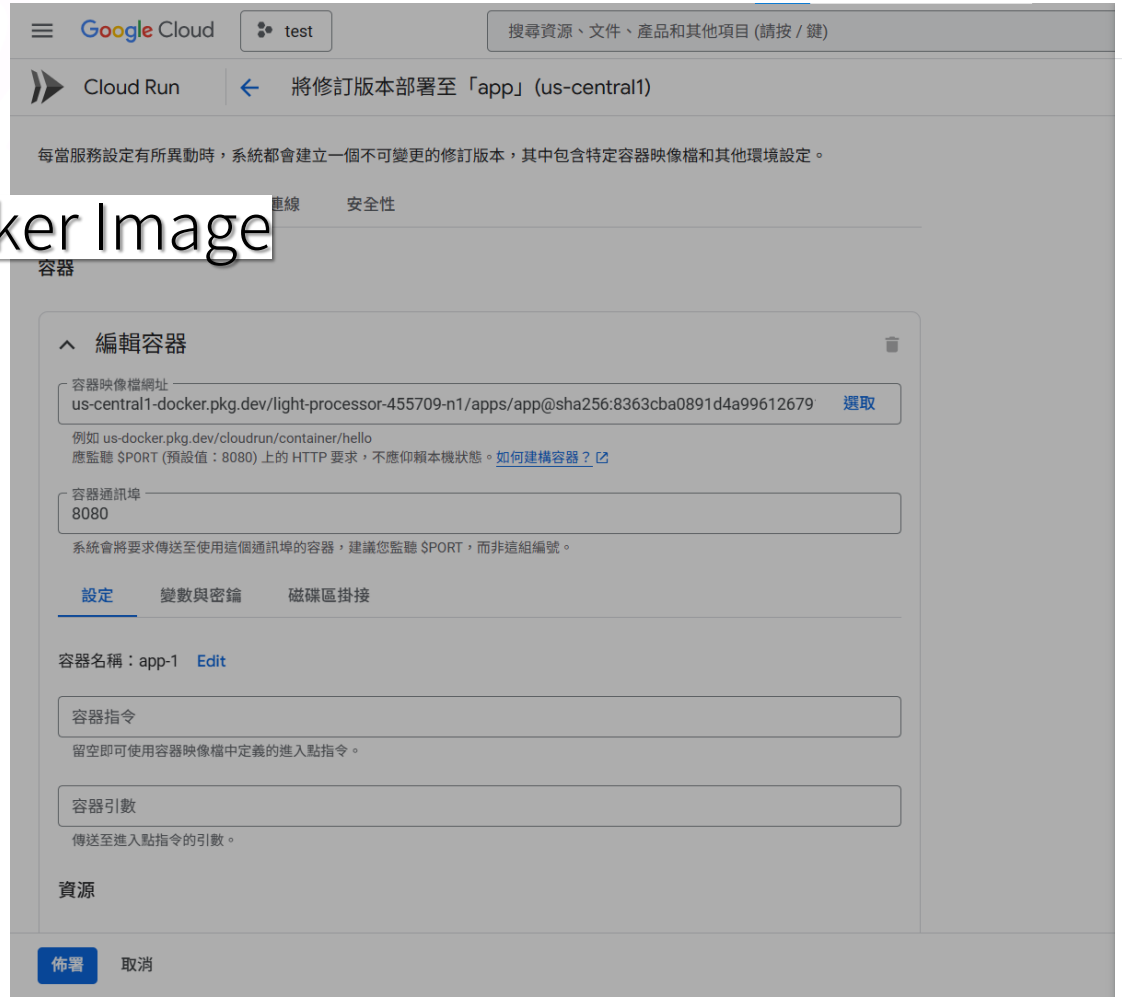
b. 部署新版 Docker Image

© 2026 Enos Chou

APP 部署 ft. Google Cloud Run

3. 更新程式

b. 部署新版 Docker Image



© 2026 Enos Chou

APP 部署 ft. Google Cloud Run

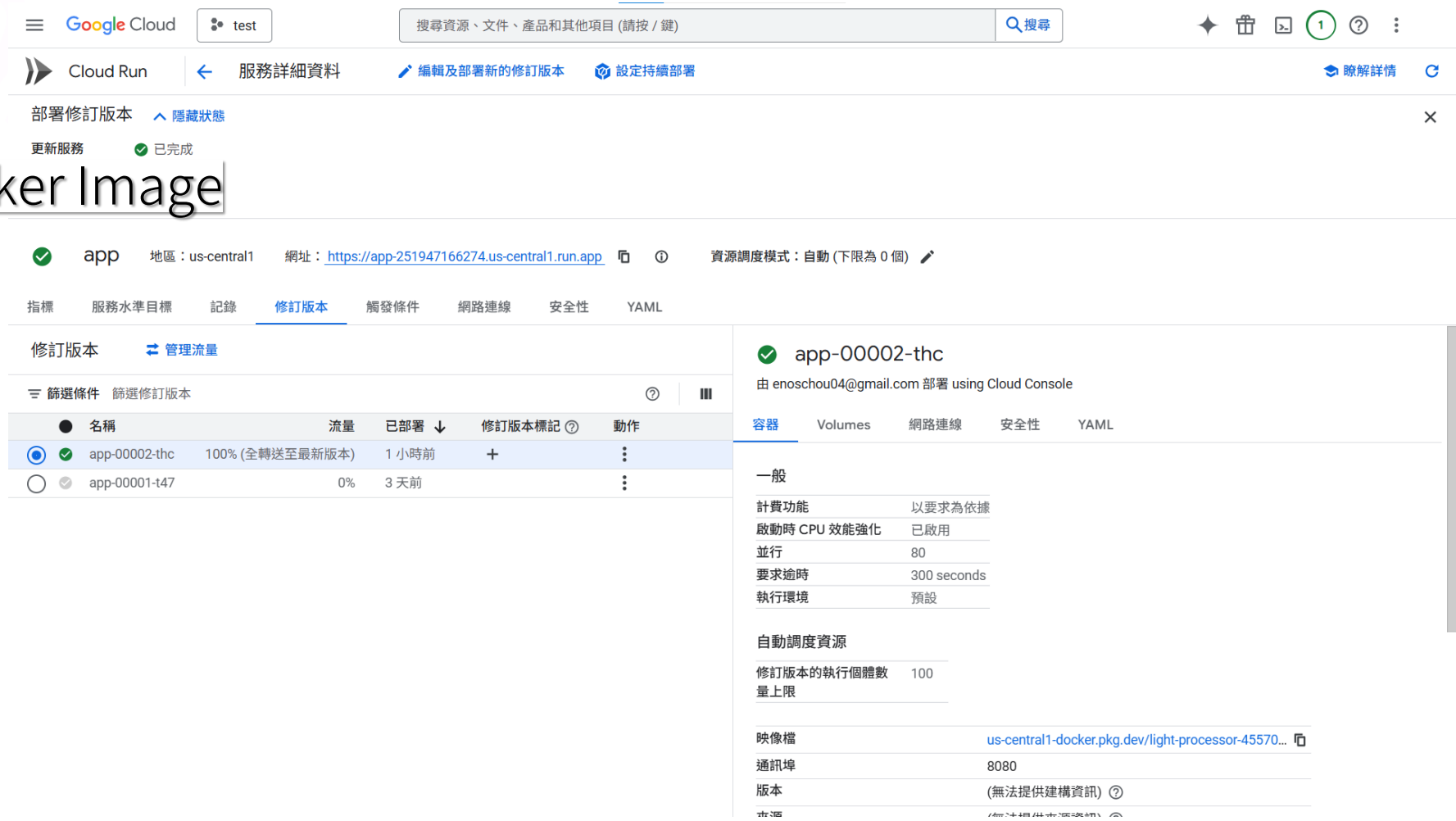
3. 更新程式

b. 部署新版 Docker Image

APP 部署 ft. Google Cloud Run

3. 更新程式

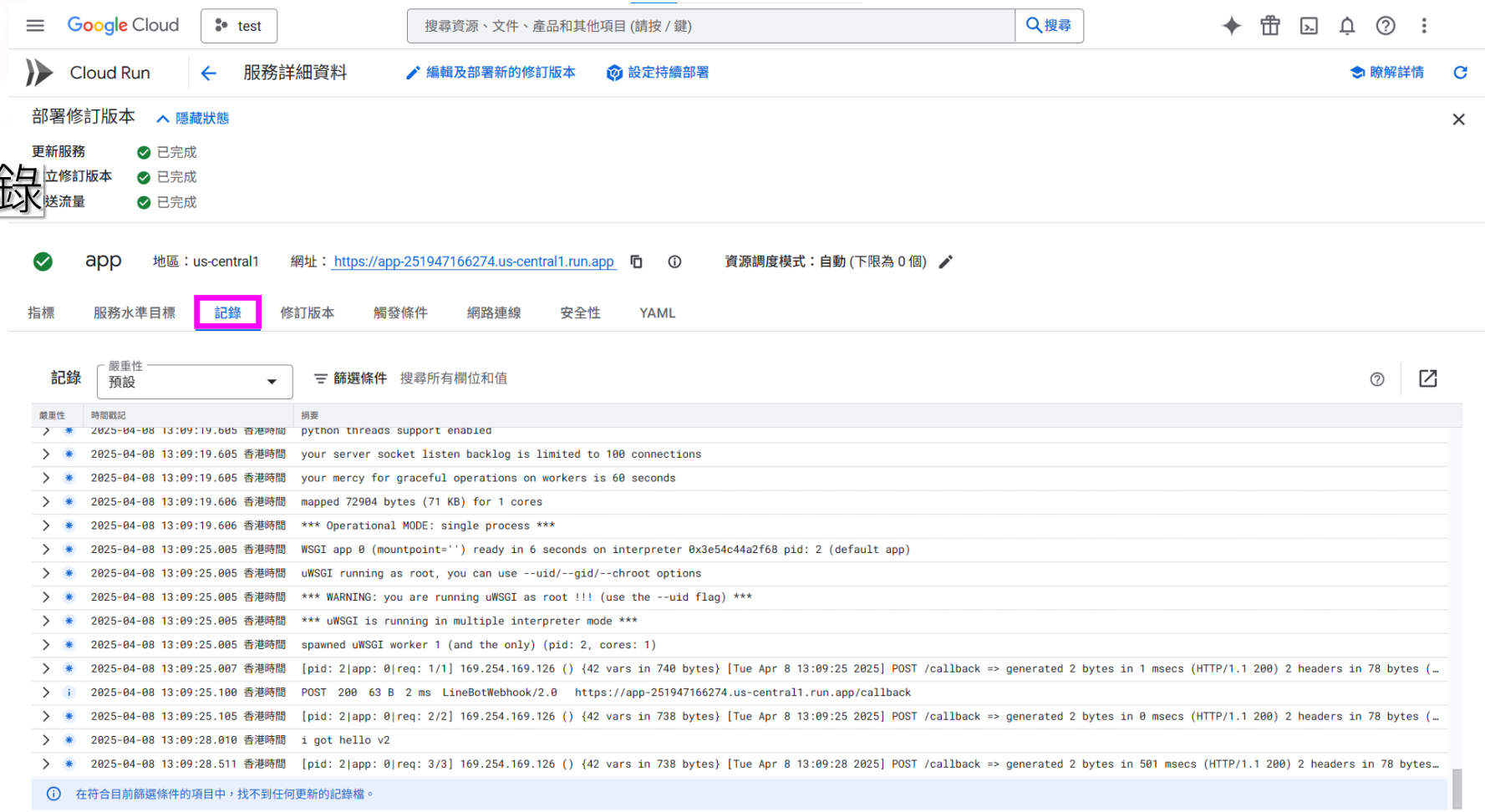
b. 部署新版 Docker Image



APP 部署 ft. Google Cloud Run

3. 更新程式

c. 測試並觀察記錄



© 2026 Enos Chou

APP 部署 ft. Google Cloud Run

4. 設定參數

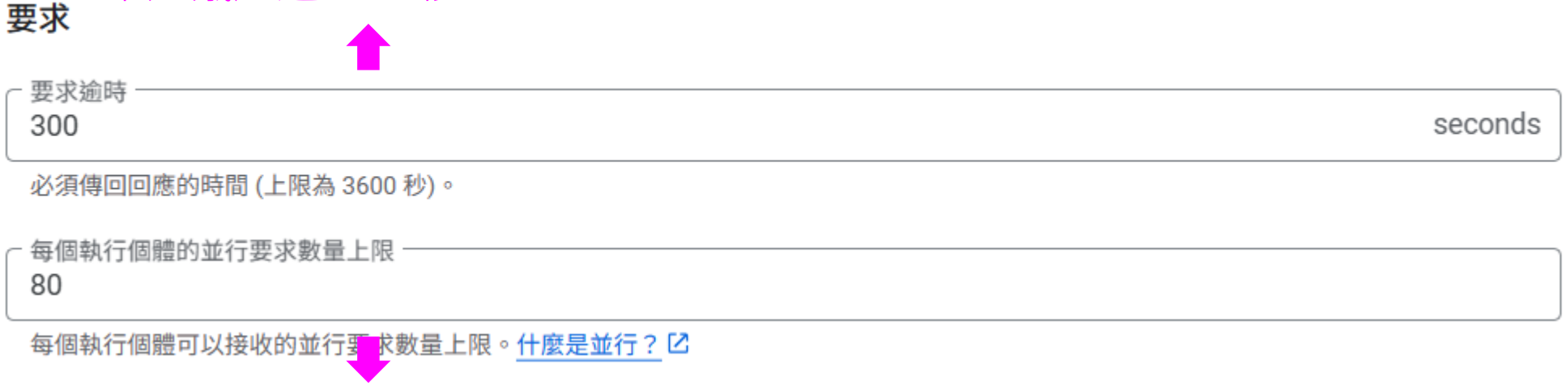
目的：Cloud Run 有 auto-scaling 特性，需要精確設定以確保
效率、成本、風險、用戶體驗 間的平衡

© 2026 Enos Chou

APP 部署 ft. Google Cloud Run

4. 設定參數

request 的 timeout 值，預設值太長，應經過測試取得合理值，
不建議超過 20 秒

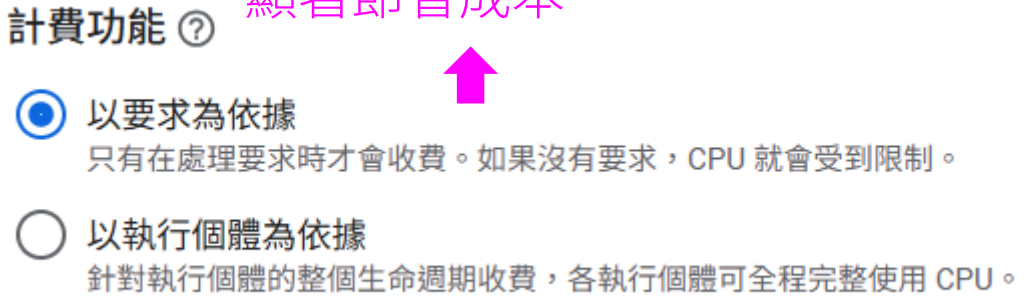


同時進入容器並行的 request 個數，須經過壓力測試決定此數值，
一般而言此預設值 80 須調整

APP 部署 ft. Google Cloud Run

4. 設定參數

預設 "以要求為依據" 計費，
在 request 量不多的情況下會
顯著節省成本



© 2026 Enos Chou

APP 部署 ft. Google Cloud Run

4. 設定參數

調整預設值為合理值以避免 DDoS 攻擊或異常流量導致成本暴增

設為 0 會有冷啟動問題，
省成本但用戶體驗較差。
商業應用建議設為 1 或以上

APP 部署 ft. Google Cloud Run

5. 連結內網 VM/ DB

目的：Cloud Run Service (位於外網) 因安全因素須連結位於內網的 VM 或 DB

方法：設定 Cloud Run Service 與 VPC 網路直連

© 2026 Enos Chou

APP 部署 ft. Google Cloud Run

5. 連結內網 VM/ DB

a. 重新部署

APP 部署 ft. Google Cloud Run

5. 連結內網 VM/ DB

b. 連結內網

APP 部署 ft. Google Cloud Run

5. 連結內網 VM/ DB

b. 連結內網

© 2026 Enos Chou

APP 部署 ft. Google Cloud Run

5. 連結內網 VM/ DB

b. 連結內網

© 2026 Enos Chou

APP 部署 ft. Google Cloud Run

5. 連結內網 VM/ DB

c. 完成部署

© 2026 Enos Chou

APP 部署 ft. Google Cloud Run

6. 伴隨容器 (Sidecar)

目的：完成 Cloud Run Service 須要搭配超過一個 Container

方法：啟動 Container 的同時也能啟動搭配的 Container

© 2026 Enos Chou

APP 部署 ft. Google Cloud Run

6. 伴隨容器 (Sidecar)

   a. 重新部署

APP 部署 ft. Google Cloud Run

6. 伴隨容器 (Sidecar)

b. 新增容器

Note
此容器 (Sidecar) 可經由 localhost 存取

APP 部署 ft. Google Cloud Run

6. 伴隨容器 (Sidecar)

b. 新增容器

Note
此容器 (Sidecar) 可經由 localhost 存取

指定 Sidecar Docker Image

決定 Sidecar 是否稍後啟動

© 2026 Enos Chou

APP 部署 ft. Google Cloud Run

6. 伴隨容器 (Sidecar)

b. 新增容器

Note
此容器 (Sidecar) 可經由 localhost 存取

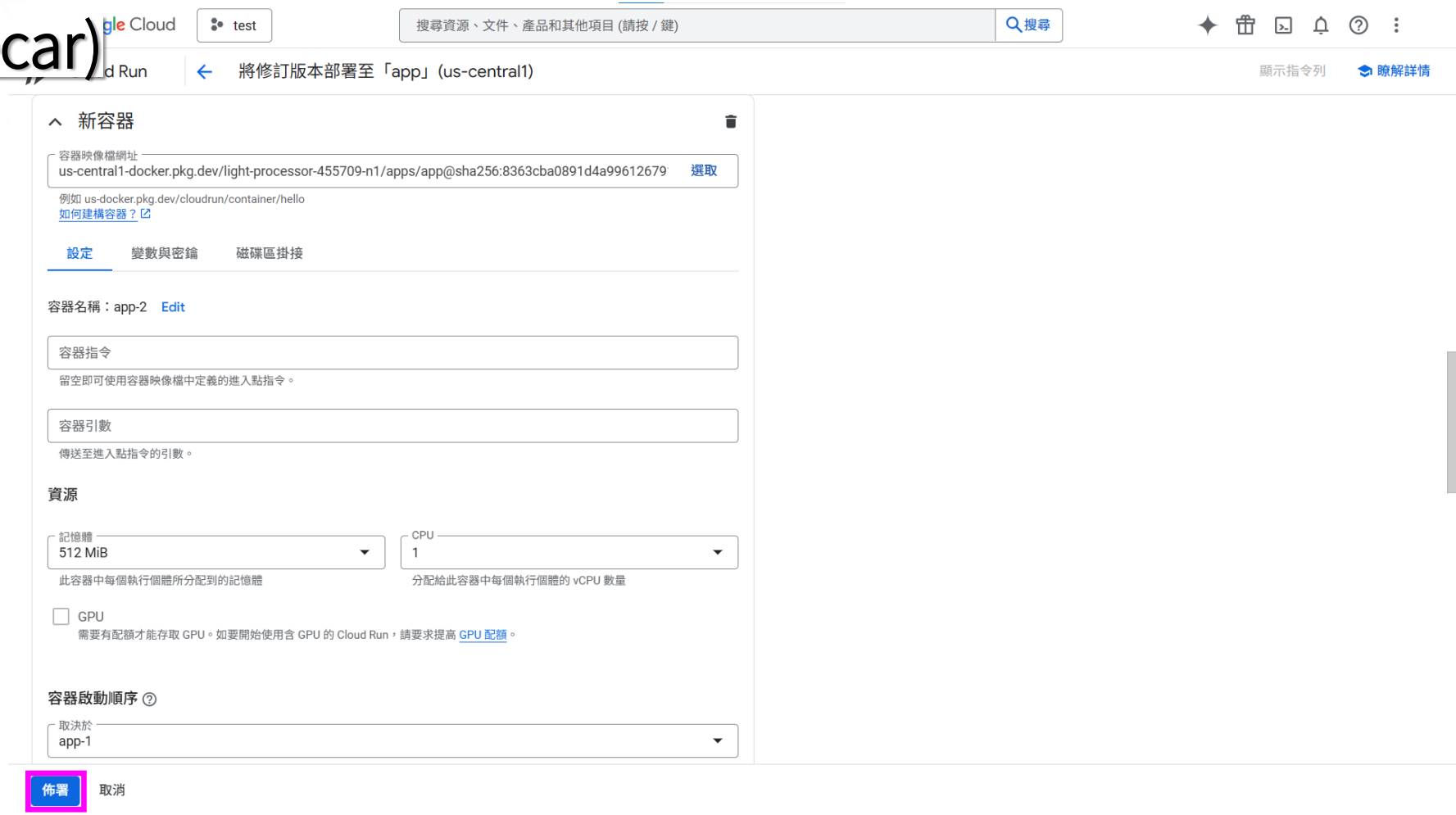
指定 Sidecar Docker Image

視需求決定 Sidecar 何時啟動

APP 部署 ft. Google Cloud Run

6. 伴隨容器 (Sidecar)

c. 完成部署



© 2026 Enos Chou

APP 部署 ft. Google Cloud Run

7. 共享磁碟

目的：同 Cloud Run Service 中的 instances 不易同步資料

方法：將 Google Storage Service 設定為 instances 之共用硬碟

© 2026 Enos Chou

APP 部署 ft. Google Cloud Run

7. 共享磁碟

a. 重新部署

© 2026 Enos Chou

APP 部署 ft. Google Cloud Run

7. 共享磁碟

b. 新增磁碟區

APP 部署 ft. Google Cloud Run

7. 共享磁碟

b. 新增磁碟區

© 2026 Enos Chou

APP 部署 ft. Google Cloud Run

7. 共享磁碟

b. 新增磁碟區

指定 Cloud Storage 值區

APP 部署 ft. Google Cloud Run

7. 共享磁碟

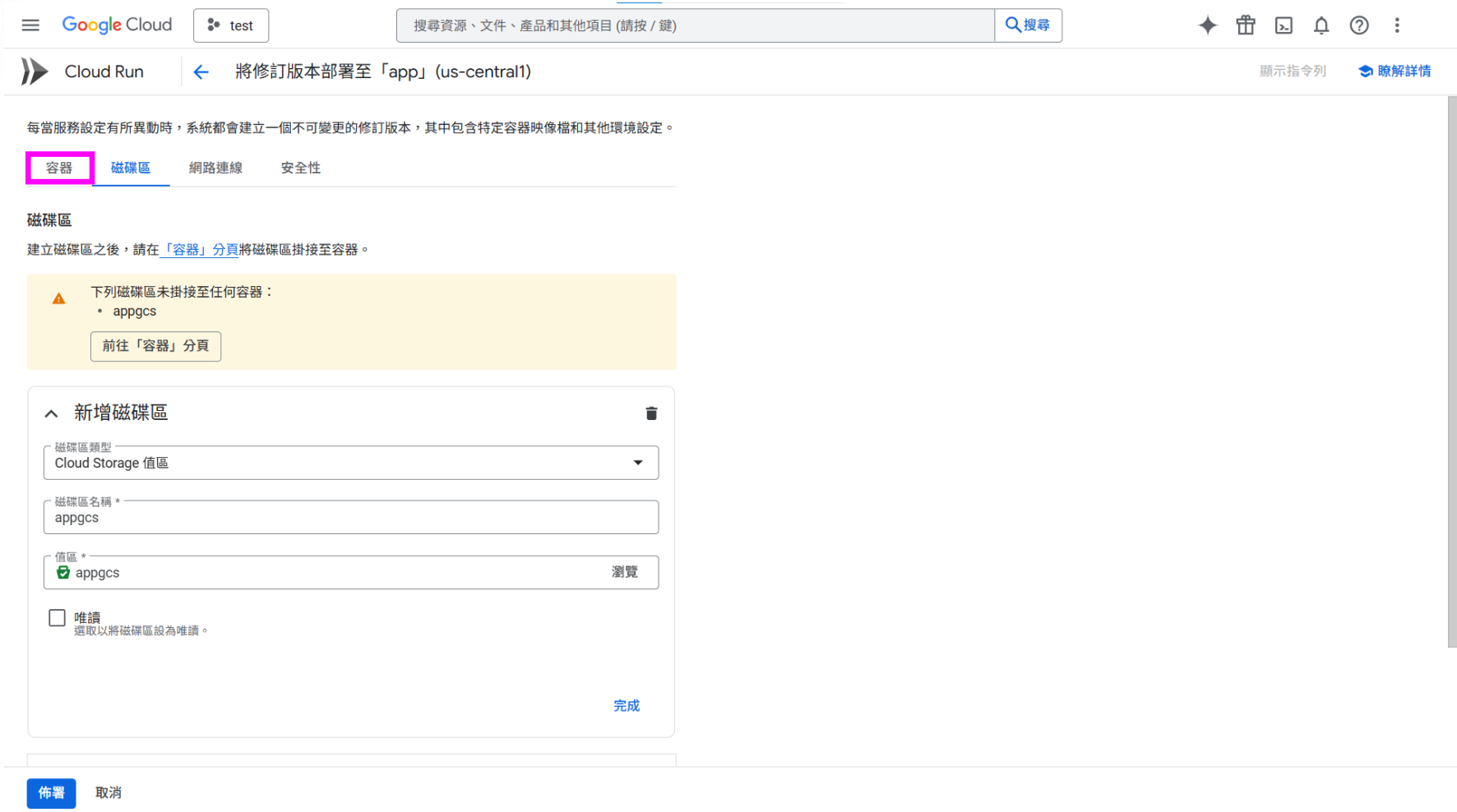
b. 新增磁碟區

© 2026 Enos Chou

APP 部署 ft. Google Cloud Run

7. 共享磁碟

c. 掛接磁碟區



© 2026 Enos Chou

APP 部署 ft. Google Cloud Run

7. 共享磁碟

c. 掛接磁碟區

APP 部署 ft. Google Cloud Run

7. 共享磁碟

c. 掛接磁碟區

APP 部署 ft. Google Cloud Run

7. 共享磁碟

c. 掛接磁碟區

APP 部署 ft. Google Cloud Run

7. 共享磁碟

c. 掛接磁碟區

© 2026 Enos Chou

APP 部署 ft. Google Cloud Run

7. 共享磁碟

d. 完成部署

APP 部署 ft. Google Cloud Run

8. 轉換網域

目的：提高 Cloud Run Service 的網址識別度

方法：以自有域名取代 Google Cloud Run 提供之預設域名

© 2026 Enos Chou

APP 部署 ft. Google Cloud Run

8. 轉換網域

a. (驗證網域)

① 新增對應關係

© 2026 Enos Chou

APP 部署 ft. Google Cloud Run

8. 轉換網域

a. (驗證網域)

① 新增對應關係

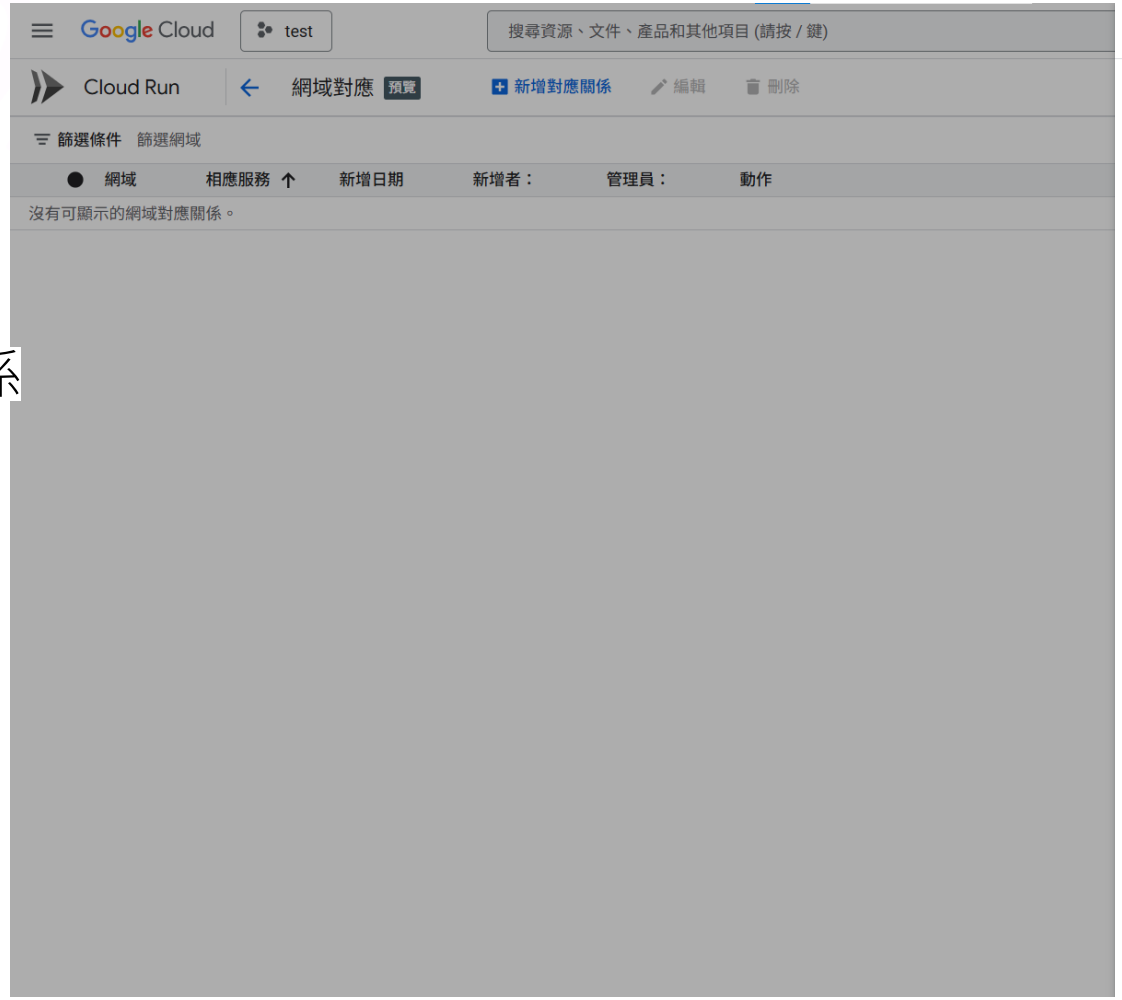
© 2026 Enos Chou

APP 部署 ft. Google Cloud Run

8. 轉換網域

a. (驗證網域)

① 新增對應關係

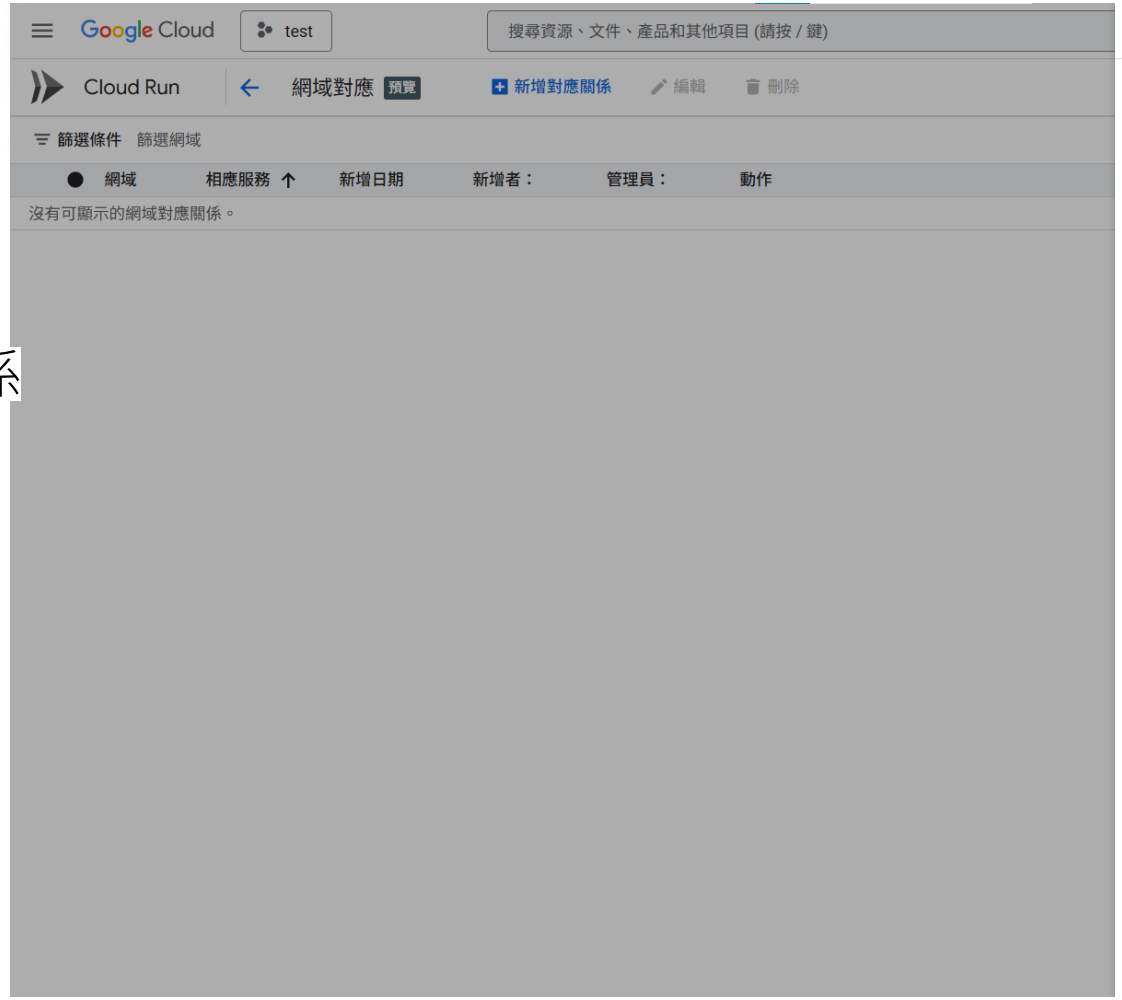


APP 部署 ft. Google Cloud Run

8. 轉換網域

a. (驗證網域)

① 新增對應關係



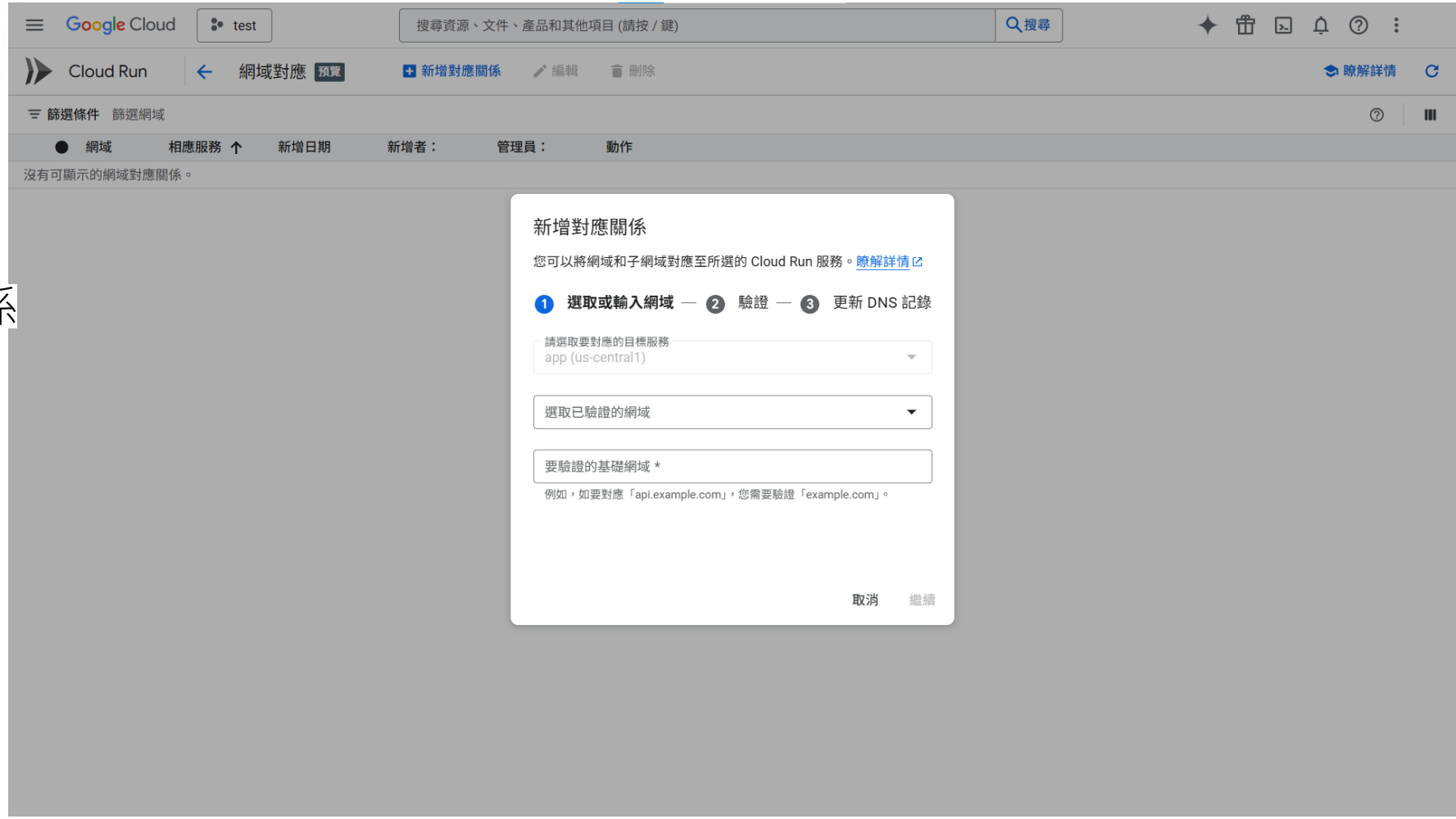
© 2026 Enos Chou

APP 部署 ft. Google Cloud Run

8. 轉換網域

a. (驗證網域)

① 新增對應關係

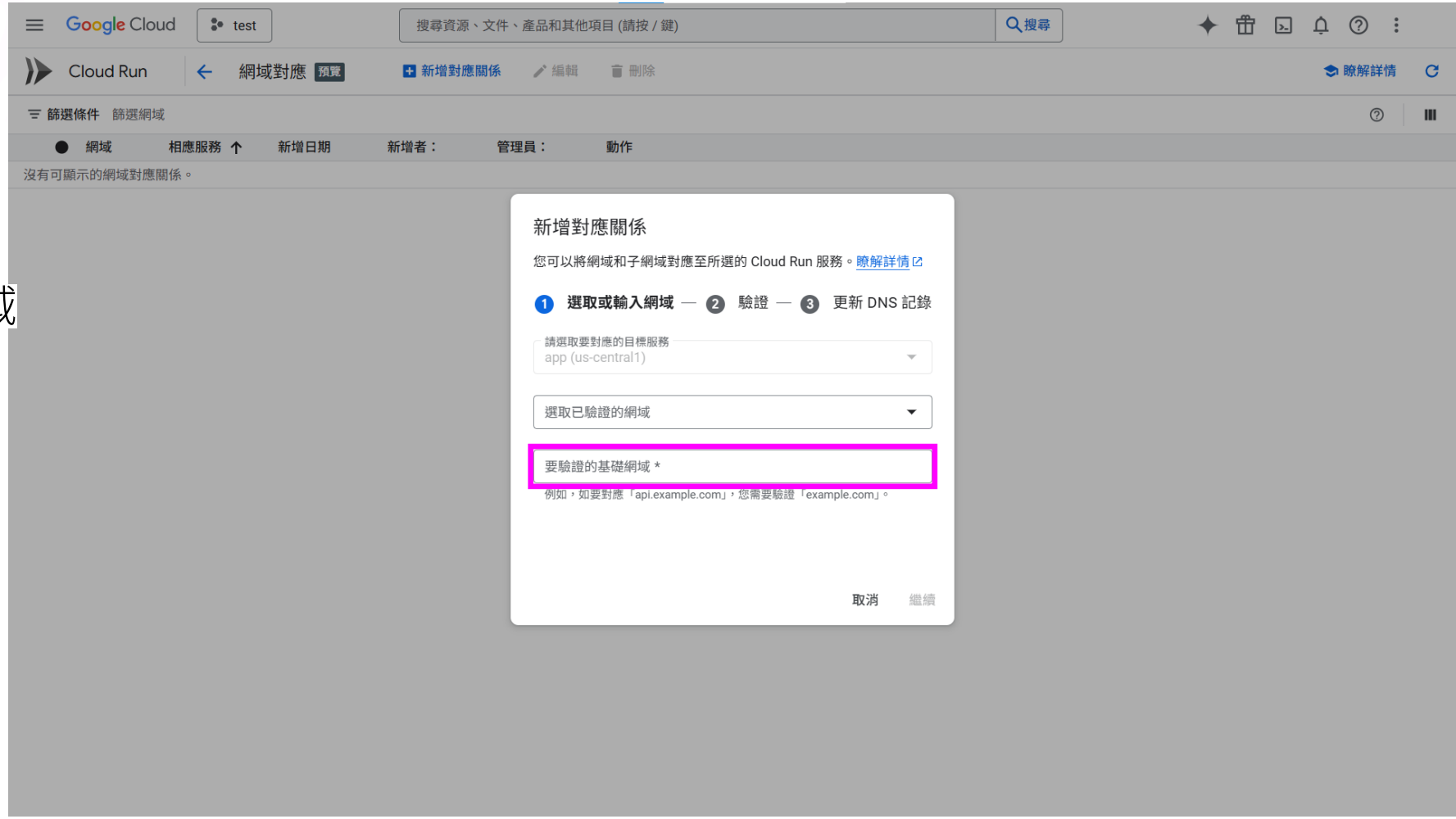


APP 部署 ft. Google Cloud Run

8. 轉換網域

a. (驗證網域)

② 驗證基礎網域



APP 部署 ft. Google Cloud Run

8. 轉換網域

a. (驗證網域)

② 驗證基礎網域

APP 部署 ft. Google Cloud Run

8. 轉換網域

a. (驗證網域)

② 驗證基礎網域

© 2026 Enos Chou

APP 部署 ft. Google Cloud Run

8. 轉換網域

a. (驗證網域)

② 驗證基礎網域

APP 部署 ft. Google Cloud Run

8. 轉換網域

a. (驗證網域)

② 驗證基礎網域

APP 部署 ft. Google Cloud Run

8. 轉換網域

a. (驗證網域)

② 驗證基礎網域

APP 部署 ft. Google Cloud Run

8. 轉換網域

a. (驗證網域)

③ 驗證網域所有權

APP 部署 ft. Google Cloud Run

8. 轉換網域

a. (驗證網域)

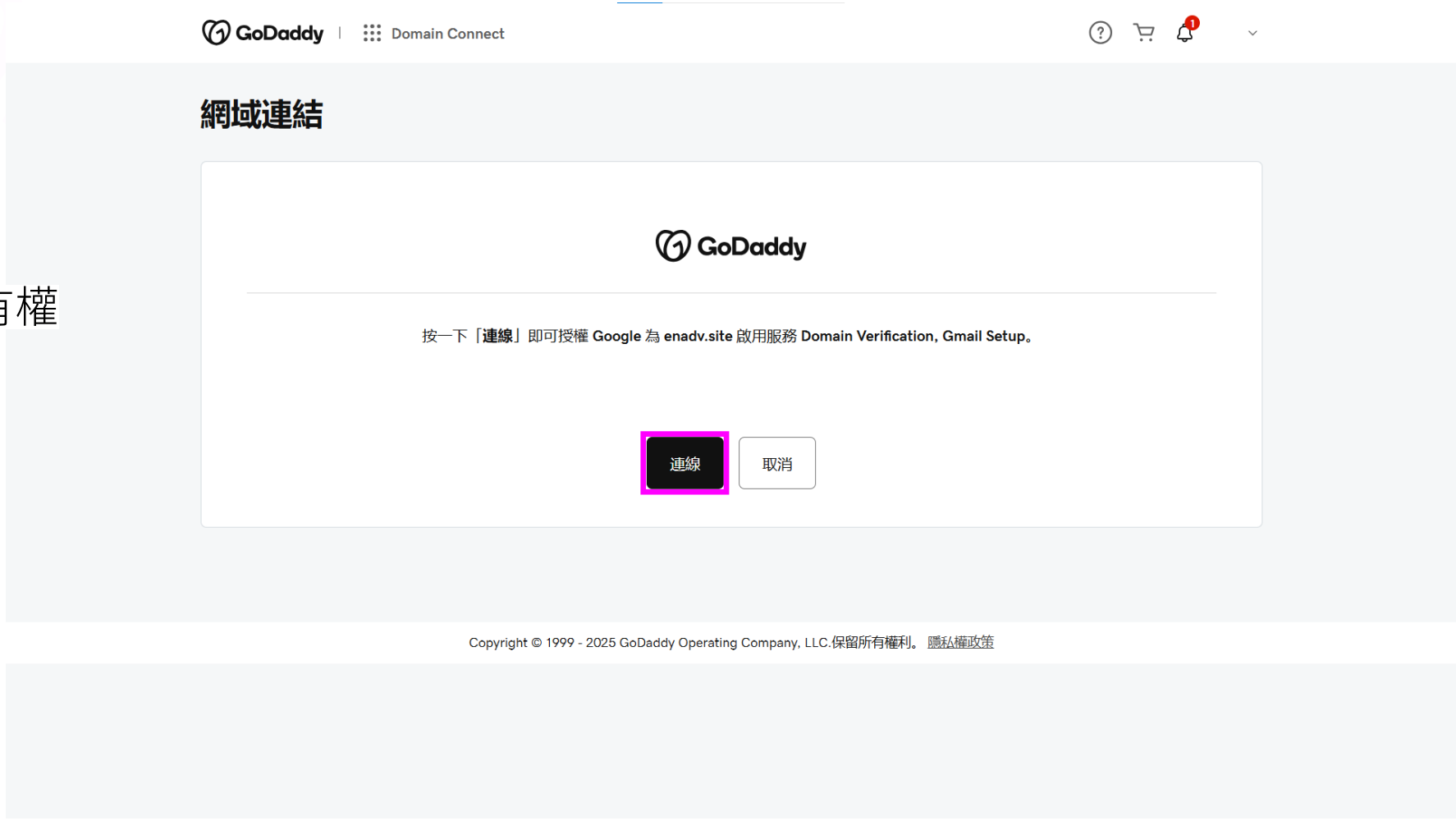
③ 驗證網域所有權

APP 部署 ft. Google Cloud Run

8. 轉換網域

a. (驗證網域)

③ 驗證網域所有權



APP 部署 ft. Google Cloud Run

8. 轉換網域

a. (驗證網域)

③ 驗證網域所有權

APP 部署 ft. Google Cloud Run

8. 轉換網域

a. (驗證網域)

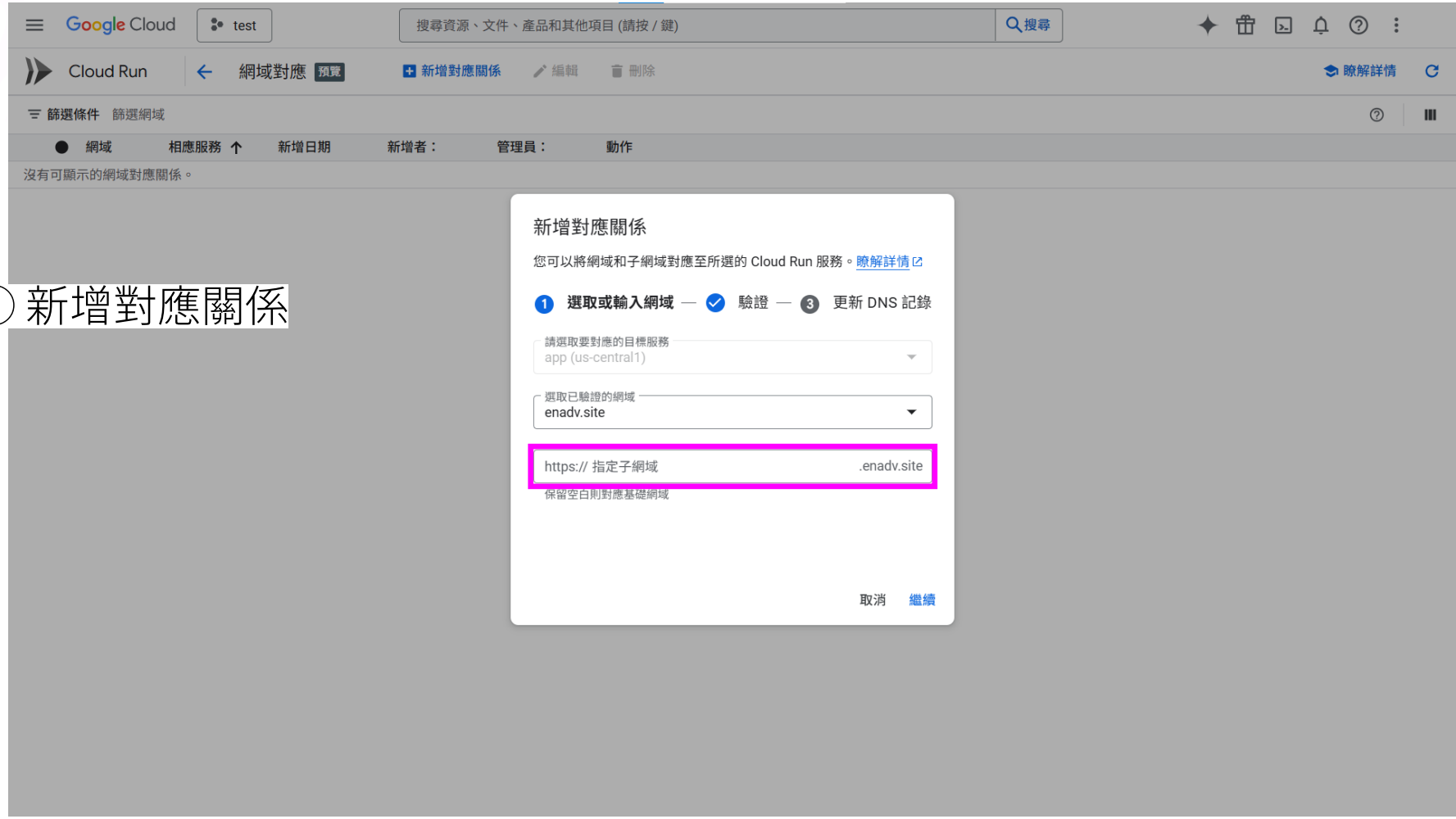
③ 驗證網域所有權

APP 部署 ft. Google Cloud Run

8. 轉換網域

b. 綁定子網域

① 重複步驟 a ① 新增對應關係

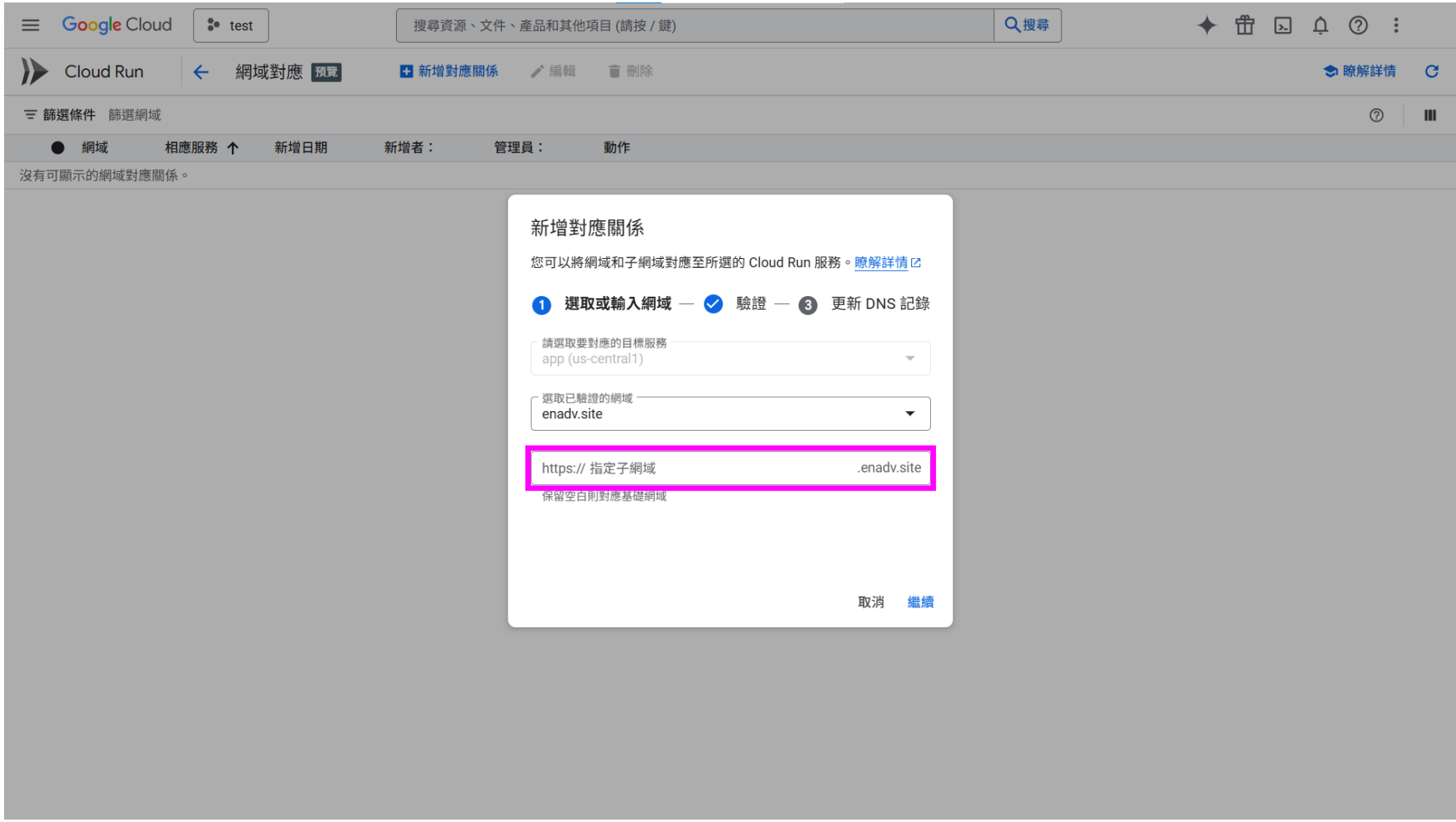


APP 部署 ft. Google Cloud Run

8. 轉換網域

b. 綁定子網域

② 指定子網域

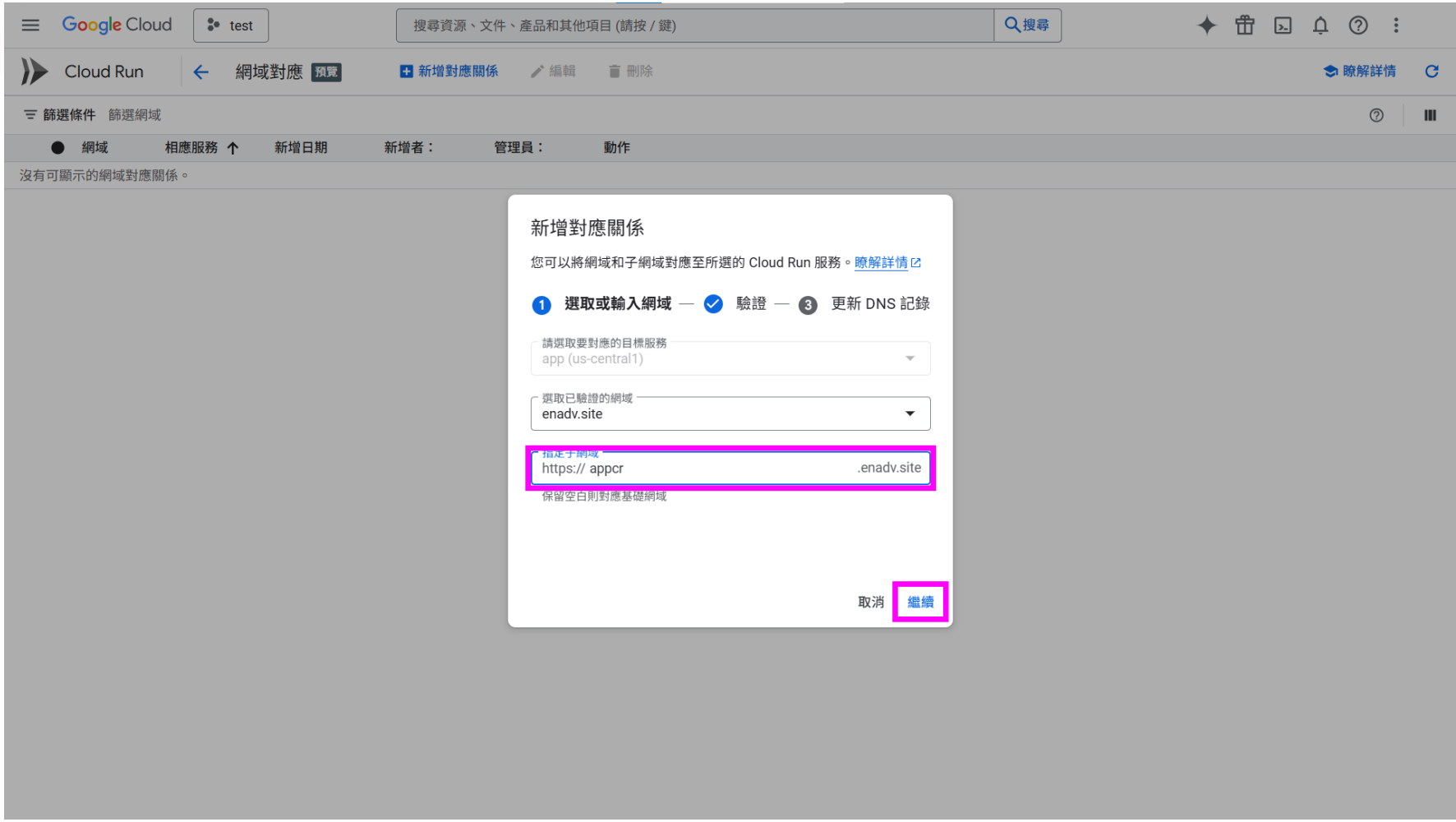


APP 部署 ft. Google Cloud Run

8. 轉換網域

b. 綁定子網域

② 指定子網域



APP 部署 ft. Google Cloud Run

8. 轉換網域

b. 綁定子網域

③ 設定 CNAME 資訊

APP 部署 ft. Google Cloud Run

8. 轉換網域

b. 綁定子網域

③ 設定 CNAME 資訊

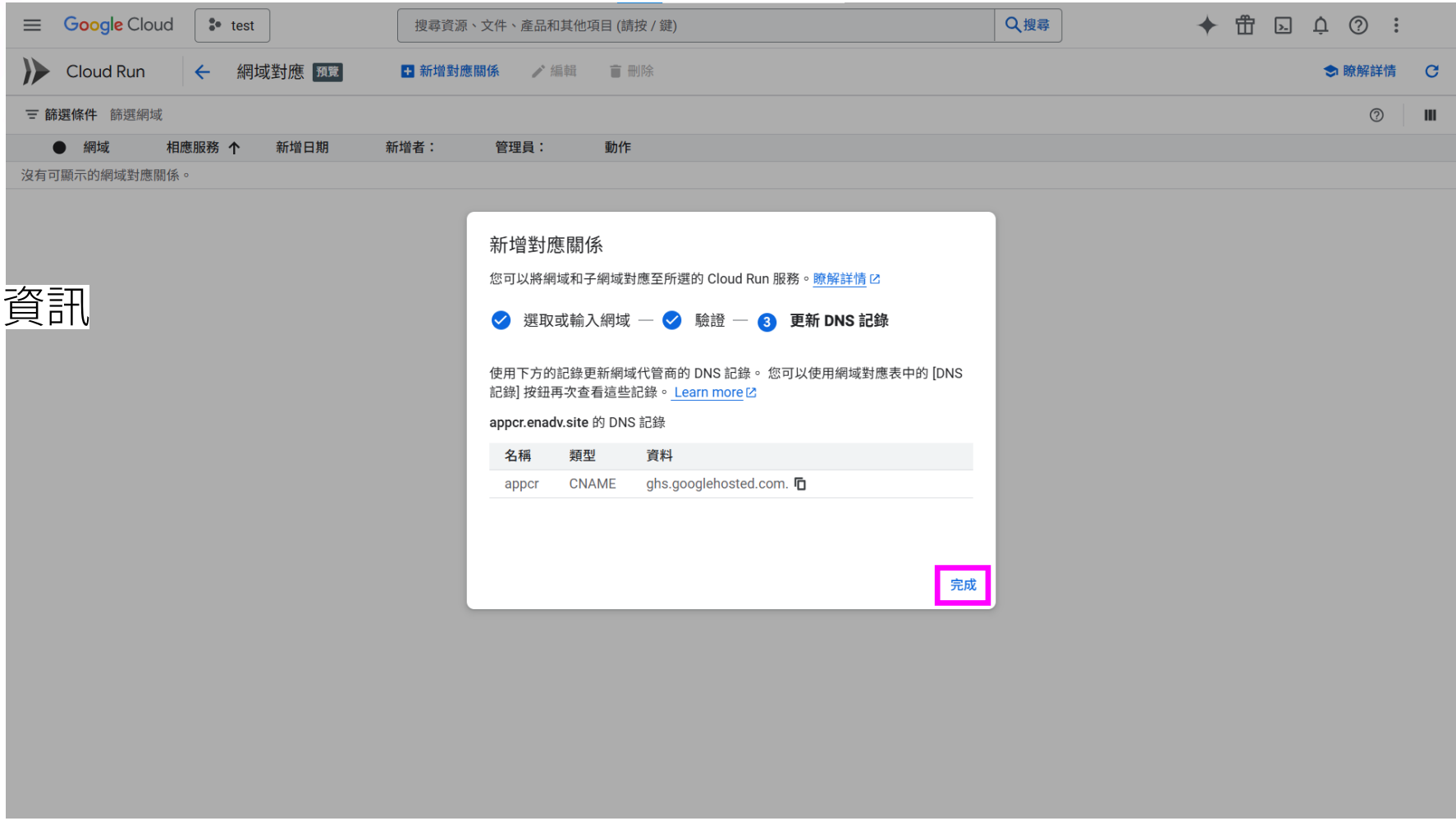
© 2026 Enos Chou

APP 部署 ft. Google Cloud Run

8. 轉換網域

b. 綁定子網域

③ 設定 CNAME 資訊

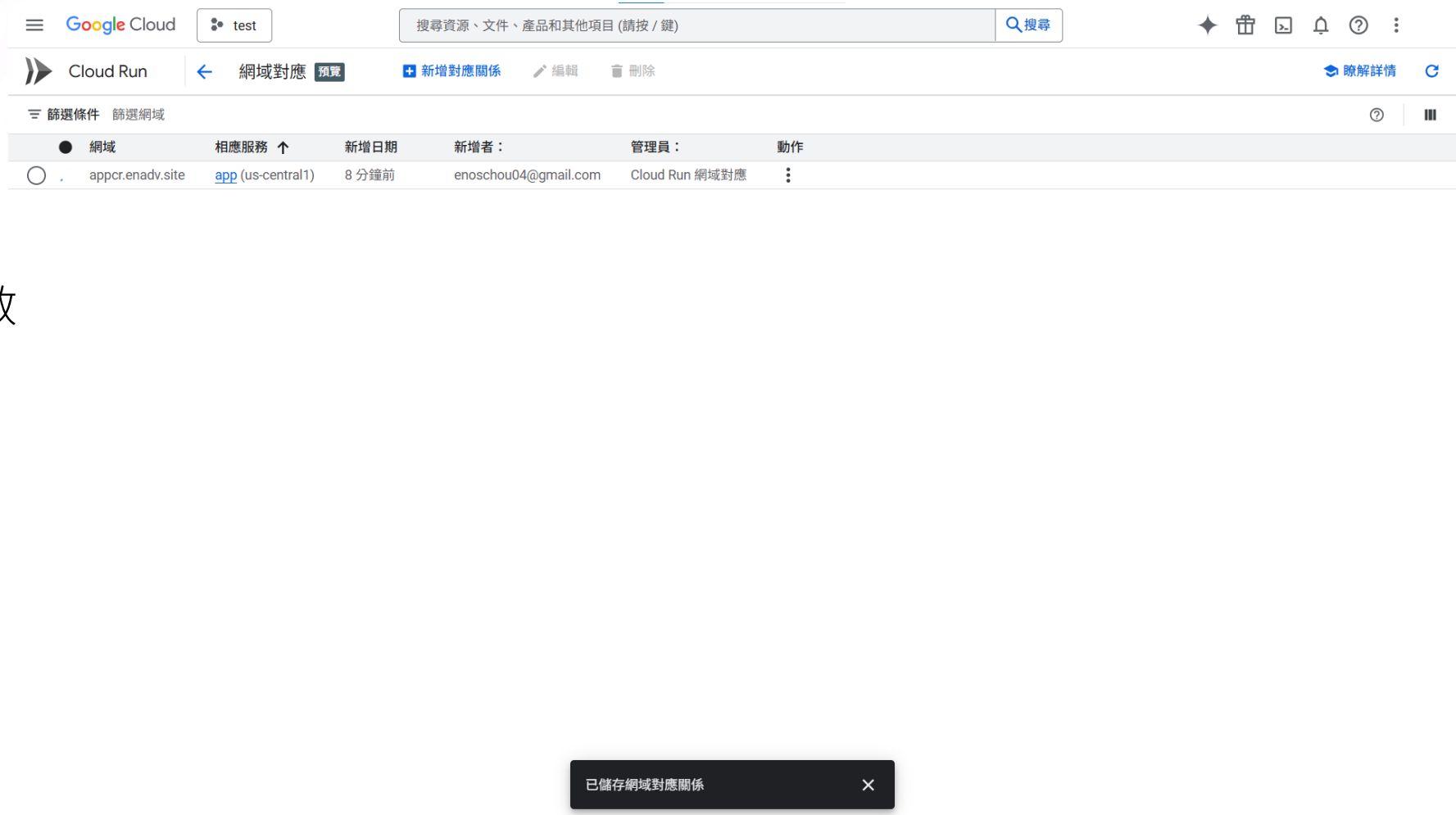


APP 部署 ft. Google Cloud Run

8. 轉換網域

b. 綁定子網域

④ 等待綁定生效



© 2026 Enos Chou

APP 部署 ft. Google Cloud Run

8. 轉換網域

b. 綁定子網域

⑤ 綁定完成

© 2026 Enos Chou

APP 部署 ft. Google Cloud Run

9. 要求認證

目的：提升叫用 Cloud Run Service 的安全性

方法：以效期 1 小時的 token 驗證 Cloud Run Service 叫用

© 2026 Enos Chou

APP 部署 ft. Google Cloud Run

9. 要求認證

a. 調整為要求認證

① 切換為 安全性

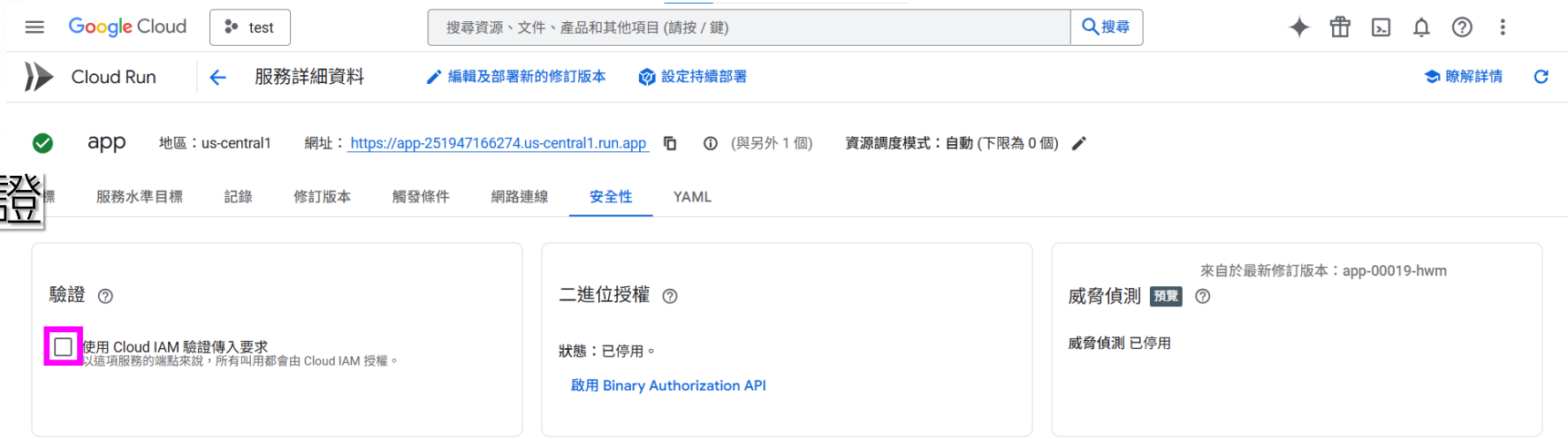
© 2026 Enos Chou

APP 部署 ft. Google Cloud Run

9. 要求認證

a. 調整為要求認證

② 設定驗證



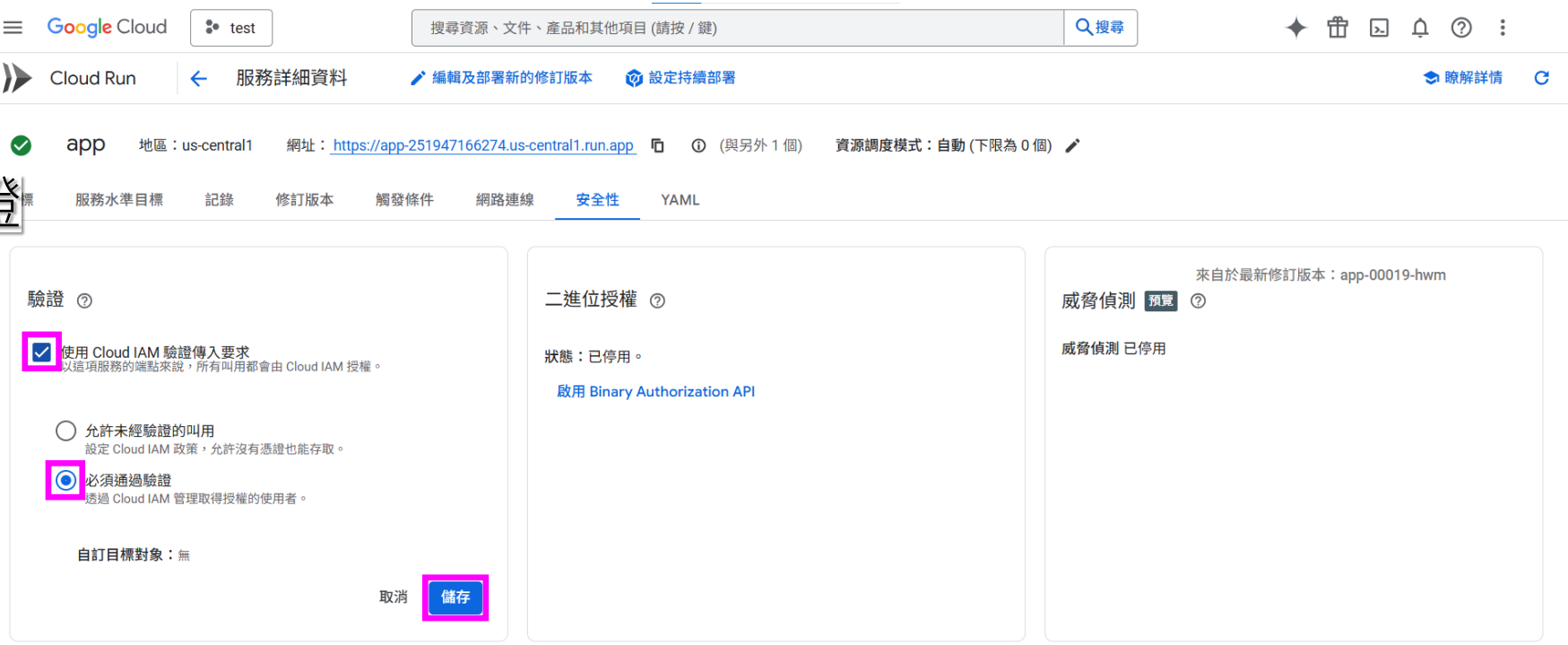
© 2026 Enos Chou

APP 部署 ft. Google Cloud Run

9. 要求認證

a. 調整為要求認證

② 設定驗證



APP 部署 ft. Google Cloud Run

9. 要求認證

b. 建立服務帳戶

建立無權限之服務帳戶並取得金鑰

© 2026 Enos Chou

APP 部署 ft. Google Cloud Run

9. 要求認證

c. 授權服務帳戶

於 Cloud Shell 執行指令授權服務帳戶存取 Cloud Run Service

```
gcloud run services add-iam-policy-binding your_cloud_run_name --
member='serviceAccount:your_service_account' --role='roles/run.invoker'
gcloud run services add-iam-policy-binding app --
member='serviceAccount:test-909@light-processor-455709-
n1.iam.gserviceaccount.com' --role='roles/run.invoker'
```

Note
過程中須指定 Cloud Run 區域

© 2026 Enos Chou

APP 部署 ft. Google Cloud Run

9. 要求認證

d. 程式叫用範例

```
# dependices: google-auth, requests

import os
import google.auth.transport.requests
import google.oauth2.id_token
import requests

os.environ['GOOGLE_APPLICATION_CREDENTIALS'] = 'YOUR_SERVICE.json'
auth_req = google.auth.transport.requests.Request()
url = 'YOUR_CLOUD_RUN_URL'
id_token = google.oauth2.id_token.fetch_id_token(auth_req, url)
headers = {'Authorization': f'Bearer {id_token}'}

r = requests.post(url, headers=headers)
```

© 2026 Enos Chou

APP 部署 ft. Google Cloud Run by gcloud CLI

1. 容器部署

Docker 映像檔部署範例

```
gcloud run deploy weapi --image=asia-east1-docker.pkg.dev/my-first-project/myreg/weapi:v3 --region=asia-east1 --min-instances=0 --max-instances=2 --concurrency=8 --timeout=14 --cpu=1 --memory=512Mi --set-env-vars=THREADS=8,WORKERS=1
```

© 2026 Enos Chou

APP 部署 ft. Google Cloud Run by gcloud CLI

2. 來源部署

程式碼部署範例

```
gcloud run deploy weapi --source=. --region=asia-east1 --min-
instances=0 --max-instances=2 --concurrency=8 --timeout=14 --
cpu=1 --memory=512Mi --set-env-vars=THREADS=8,WORKERS=1
```

NOTE
建議備妥 Dockerfile

© 2026 Enos Chou

AI 靜態網頁部署

VM + NGINX 靜態網頁部署

Container + Flask 靜態網頁部署

Firebase Hosting 靜態網頁部署

© 2026 Enos Chou

VM + NGINX 靜態網頁部署

© 2026 Enos Chou

VM + NGINX 靜態網頁部署

1. 建立 VM + NGINX
2. 上傳網頁檔案
3. 調整 NGINX

© 2026 Enos Chou

VM + NGINX 靜態網頁部署

1. 建立 VM + NGINX

   參閱 "安裝 Web Server"

© 2026 Enos Chou

VM + NGINX 靜態網頁部署

2. 上傳網頁檔案

a. 上傳網頁檔案至 $HOME

略

© 2026 Enos Chou

VM + NGINX 靜態網頁部署

2. 上傳網頁檔案

b. 移動網頁檔案至 /var/www

① 建立網頁專屬資料夾

```
sudo mkdir /var/www/subdomain_name
```

ex: sudo mkdir /var/www/rr.enadv.quest

© 2026 Enos Chou

VM + NGINX 靜態網頁部署

2. 上傳網頁檔案

b. 移動網頁檔案至 /var/www

② 複製網頁檔案

```
sudo cp file_names /var/www/subdomain_name
```

```
ex: sudo cp ~/index.html ~/style.css ~/models.txt /var/www/rr.enadv.quest
```

© 2026 Enos Chou

VM + NGINX 靜態網頁部署

2. 上傳網頁檔案

b. 移動網頁檔案至 /var/www

③ 調整網頁檔案與資料夾擁有者

```
sudo chown -R user_account:www-data /var/www/subdomain_name
```

```
ex: sudo chown -R eng:www-data /var/www/rr.enadv.quest
```

© 2026 Enos Chou

VM + NGINX 靜態網頁部署

3. 調整 NGINX

a. 調整 /etc/nginx/sites-enabled 中的 configuration file

① 於既有 server configuration 加入網頁設定 (範例)

```
server {
    server_name rr.enadv.quest;
    #### existing configurations ####

    # for Web or LIFF
    location / {
        root /var/www/rr.enadv.quest;
        index index.html;
    }
}
```

© 2026 Enos Chou

VM + NGINX 靜態網頁部署

3. 調整 NGINX

b. 重新載入 configuration file 內容

```
sudo nginx -s reload
```

© 2026 Enos Chou

VM + NGINX 靜態網頁部署

3. 調整 NGINX

c. 測試網頁

Container + Flask 靜態網頁部署

© 2026 Enos Chou

Container + Flask 靜態網頁部署

1. 製作 Flask App
2. 製作 Docker Image
3. 部署至 Google Cloud Run
4. 轉換網域

© 2026 Enos Chou

Container + Flask 靜態網頁部署

1. 製作 Flask App

a. 既有 Web 檔案

index.html
style.css
models.txt

© 2026 Enos Chou

Container + Flask 靜態網頁部署

1. 製作 Flask App

b. 製作 Flask App (rr.py)

```
import flask

app = flask.Flask(__name__,
                  template_folder='./',
                  static_folder='./',
                  static_url_path='')

@app.route('/')
def web():
    return flask.render_template('index.html')
```

© 2026 Enos Chou

Container + Flask 靜態網頁部署

2. 製作 Docker Image

a. 製作 Dockerfile

```
FROM python:3.12.16--trixie
WORKDIR /rr
COPY index.html style.css models.txt rr.py ./
ENV TZ=Asia/Taipei LANG=C.UTF-8 PYTHONUNBUFFERED=1
RUN pip3 install --no-cache-dir flask gunicorn
CMD exec gunicorn rr:app -b :${PORT:-8080}
```

© 2026 Enos Chou

Container + Flask 靜態網頁部署

2. 製作 Docker Image

b. 建立 Docker Image

```
docker build -t repository_name:tag .
```

```
ex: docker build -t rr:v1 .
```

© 2026 Enos Chou

Container + Flask 靜態網頁部署

3. 部署至 Google Cloud Run

a. 推送 Docker Image 至 Google Artifact Registry

參閱 "Docker Image into Google Artifact Registry"

© 2026 Enos Chou

Container + Flask 靜態網頁部署

3. 部署至 Google Cloud Run

b. 建立 Google Cloud Run 服務

參閱 "APP 部署 ft. Google Cloud Run"

© 2026 Enos Chou

Container + Flask 靜態網頁部署

3. 部署至 Google Cloud Run

   c. 測試網頁

Container + Flask 靜態網頁部署

4. 轉換網域

參閱 "APP 部署 ft. Google Cloud Run" > "轉換網域"

© 2026 Enos Chou

Firebase Hosting 靜態網頁部署

© 2026 Enos Chou

Firebase Hosting 靜態網頁部署

1. 建立 Firebase 專案
2. 登入 Firebase CLI
3. 準備並設定 AI 專案
4. 部署網頁
5. 轉換網域

© 2026 Enos Chou

Firebase Hosting 靜態網頁部署

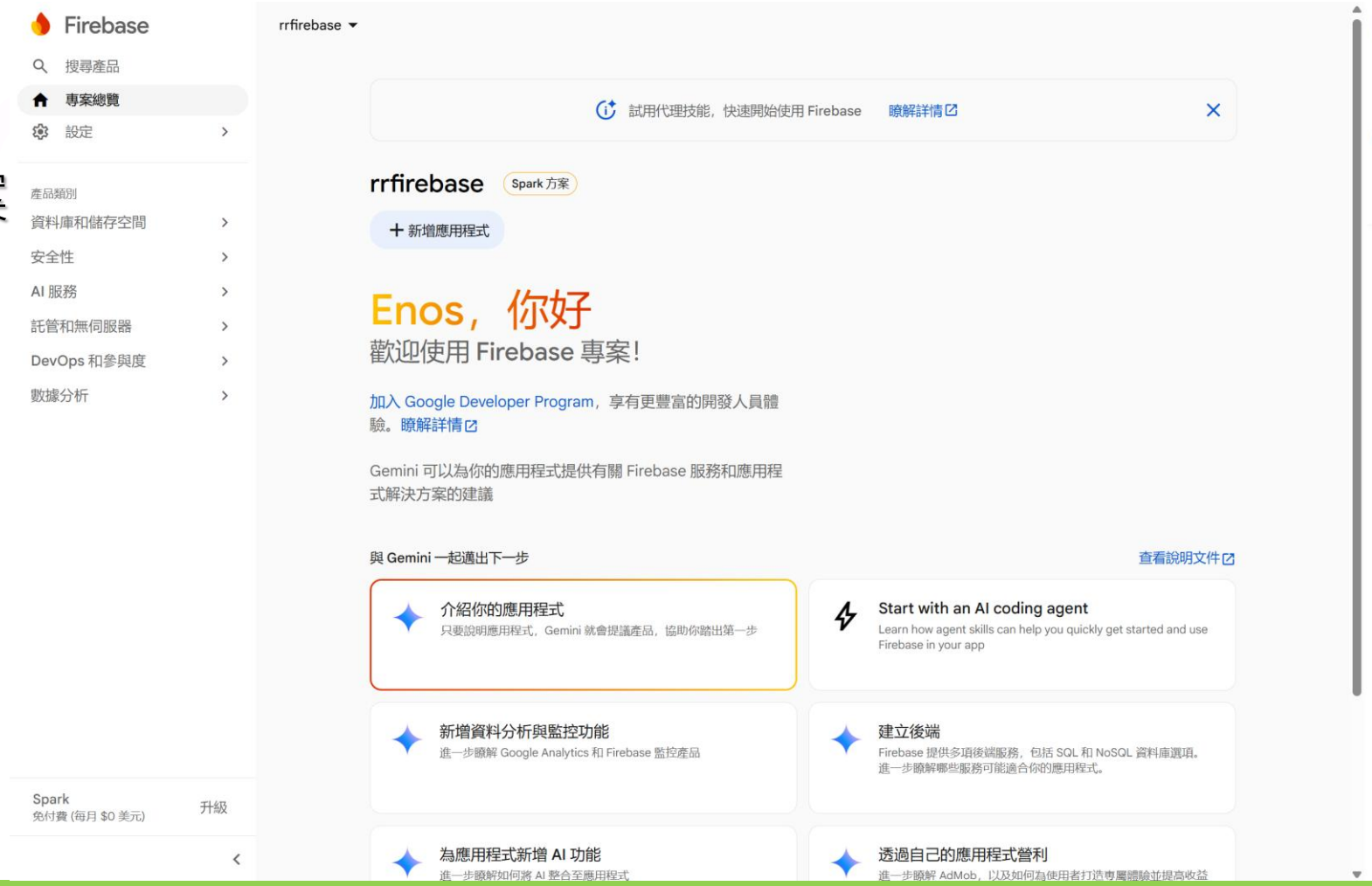
1. 建立 Firebase 專案

登入 https://console.firebase.google.com

Firebase Hosting 靜態網頁部署

1. 建立 Firebase 專案

建立新的 Firebase 專案

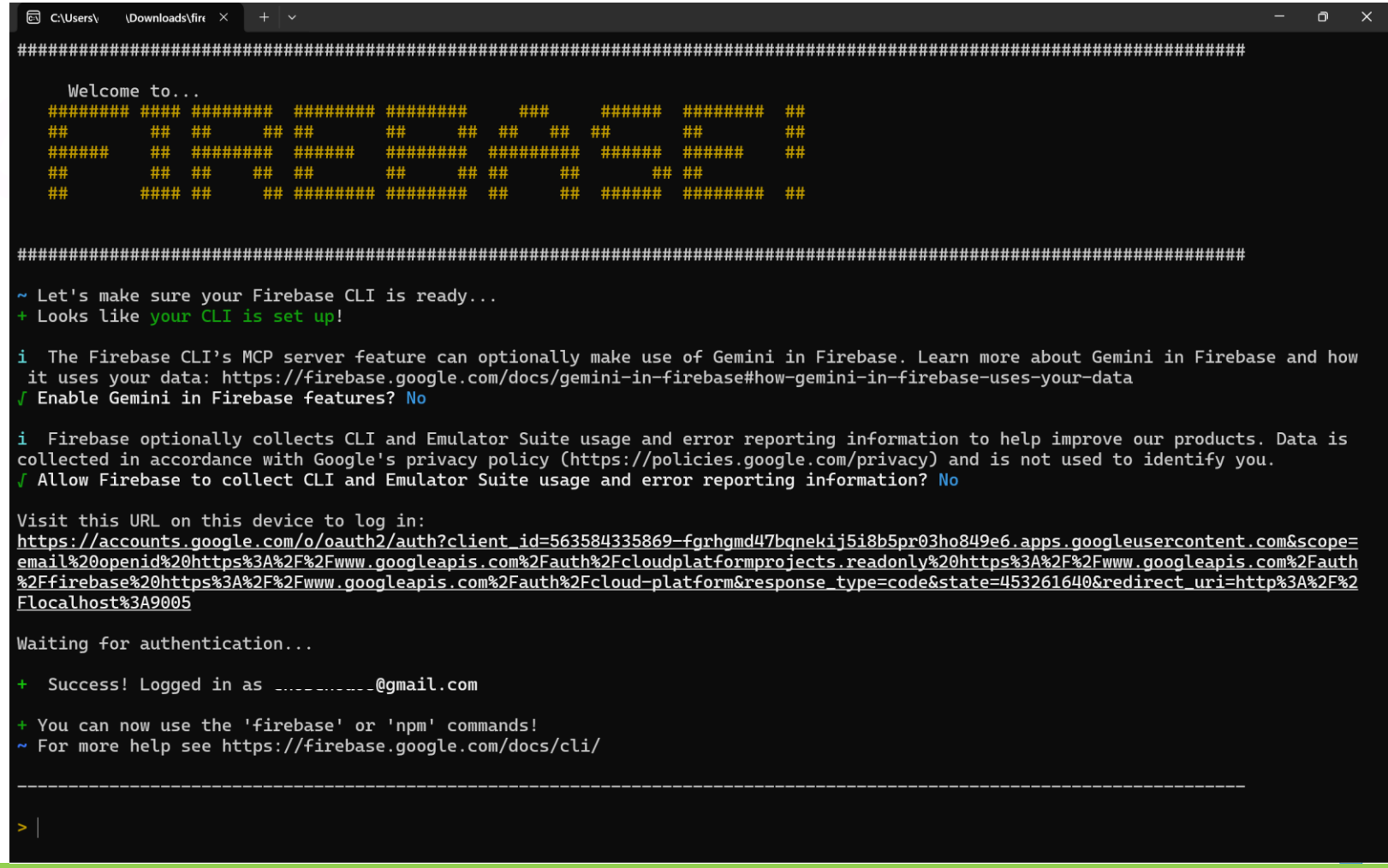


© 2026 Enos Chou

Firebase Hosting 靜態網頁部署

2. 登入 Firebase CLI

a. Windows



Firebase Hosting 靜態網頁部署

2. 登入 Firebase CLI

a. Windows

① 參考下列連結安裝 Firebase CLI，下載獨立二進位檔即可，不需要 Node.js
https://firebase.google.com/docs/cli?hl=zh-tw#install-cli-windows

② 執行 firebase-tools-instant-win.exe，啟動 Firebase CLI

③ `firebase login`

Firebase Hosting 靜態網頁部署

2. 登入 Firebase CLI

b. macOS/ Linux

① 參考下列連結安裝 Firebase CLI
https://firebase.google.com/docs/cli?hl=zh-tw#install-cli-mac-linux

② 於終端機安裝 Firebase CLI
```
curl -sL https://firebase.tools | bash
```

③ `firebase login`

© 2026 Enos Chou

Firebase Hosting 靜態網頁部署

2. 登入 Firebase CLI

c. Cloud Shell

① 於 Firebase Console 啟動 Cloud Shell (內建 Firebase CLI)

② `firebase login`

© 2026 Enos Chou

Firebase Hosting 靜態網頁部署

3. 準備並設定 AI 專案

於 Firebase CLI

① cd 至 AI 靜態網頁目錄（專案目錄）

© 2026 Enos Chou

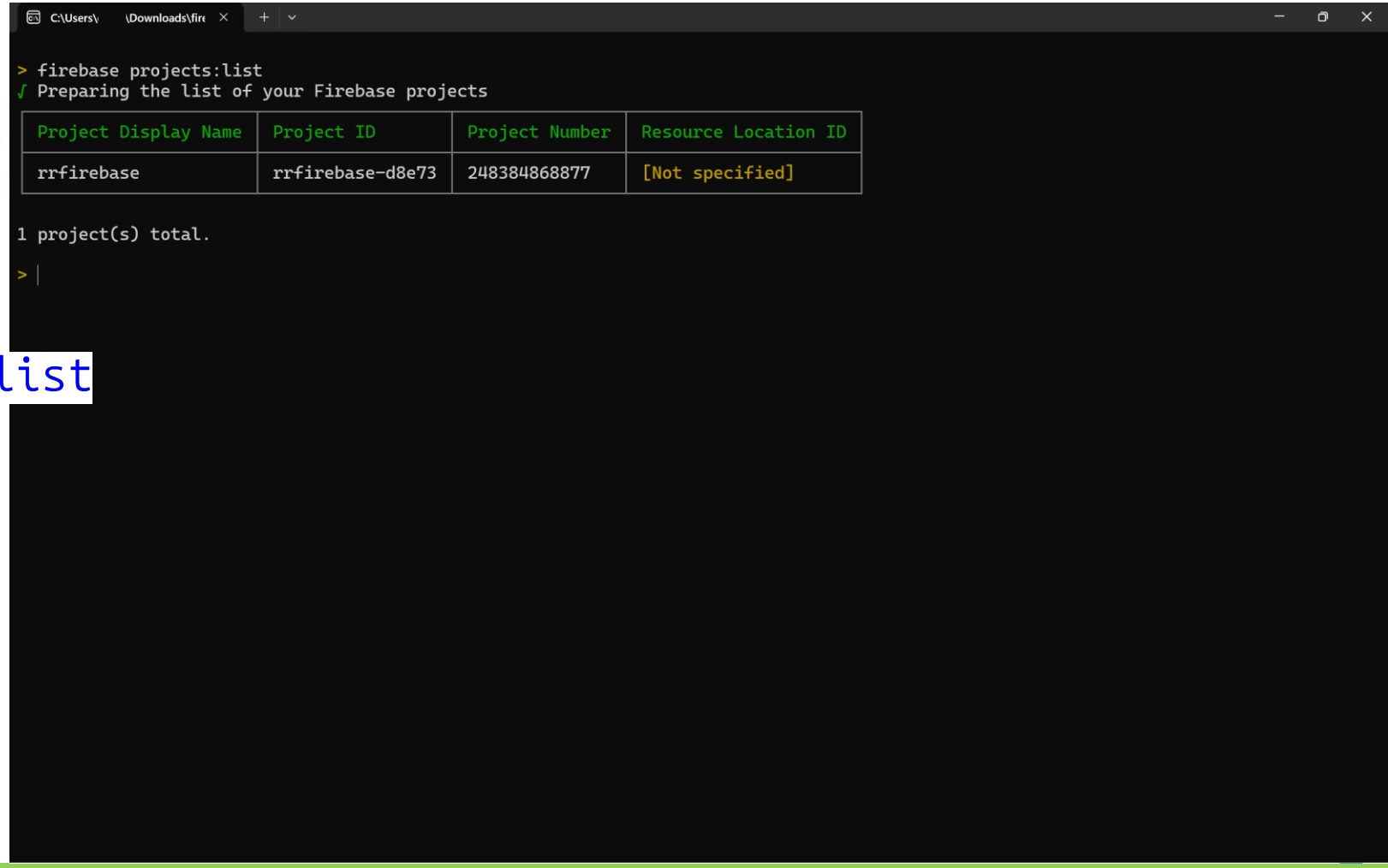
Firebase Hosting 靜態網頁部署

3. 準備並設定 AI 專案

於 Firebase CLI

② 查看所有專案

```
firebase projects:list
```



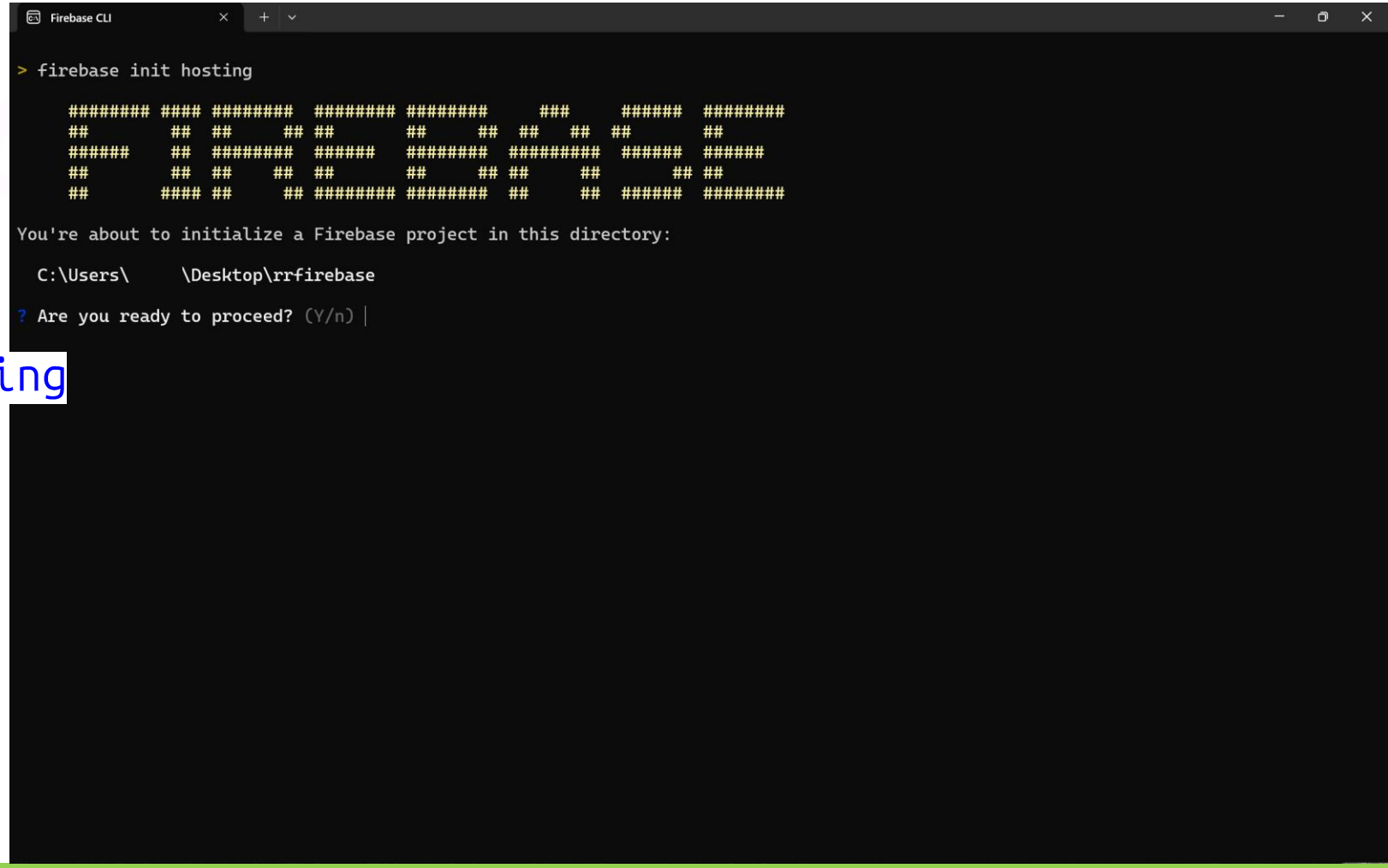
Firebase Hosting 靜態網頁部署

3. 準備並設定 AI 專案

於 Firebase CLI

③ 初始化專案

```
firebase init hosting
```



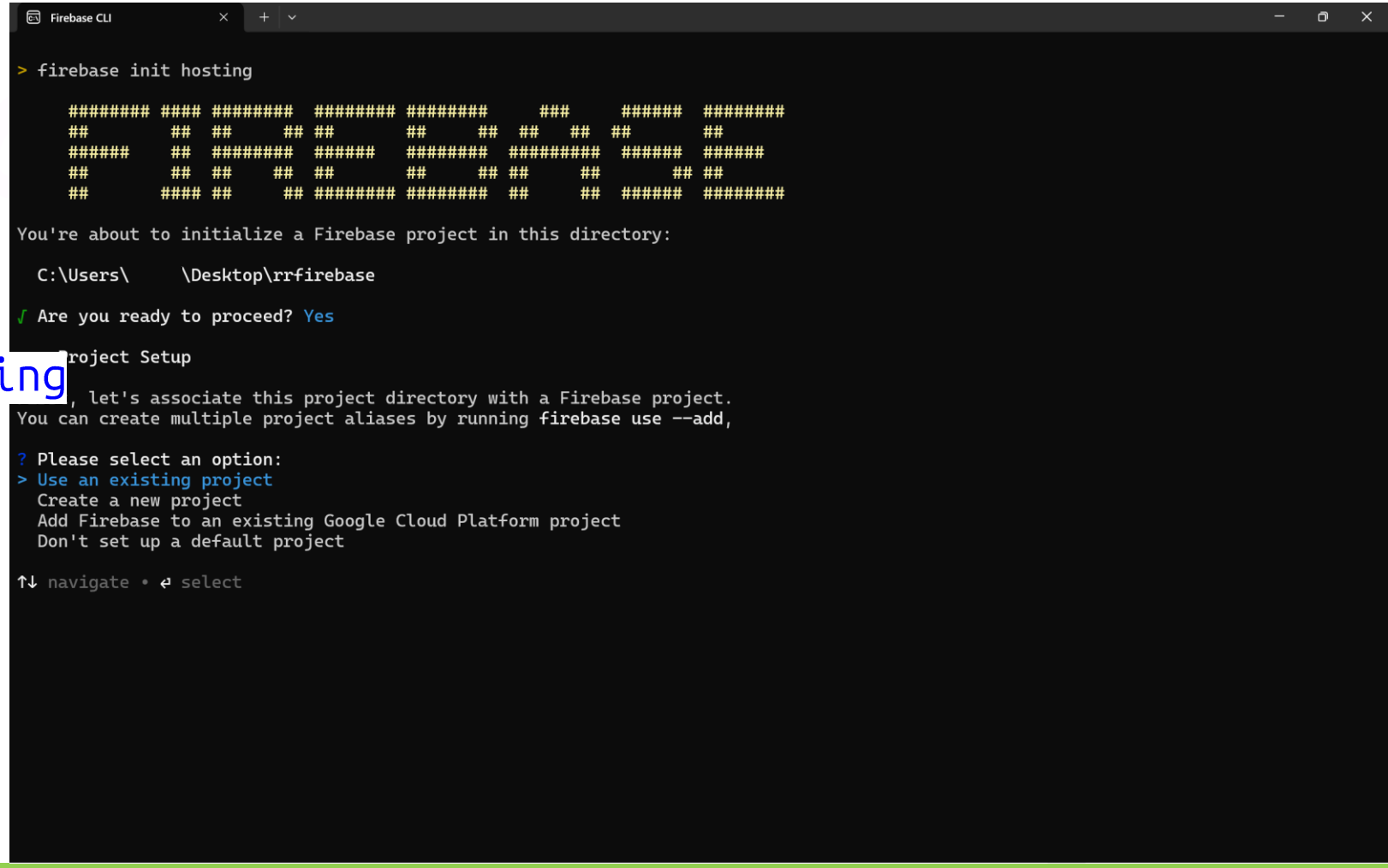
Firebase Hosting 靜態網頁部署

3. 準備並設定 AI 專案

於 Firebase CLI

③ 初始化專案

```
firebase init hosting
```



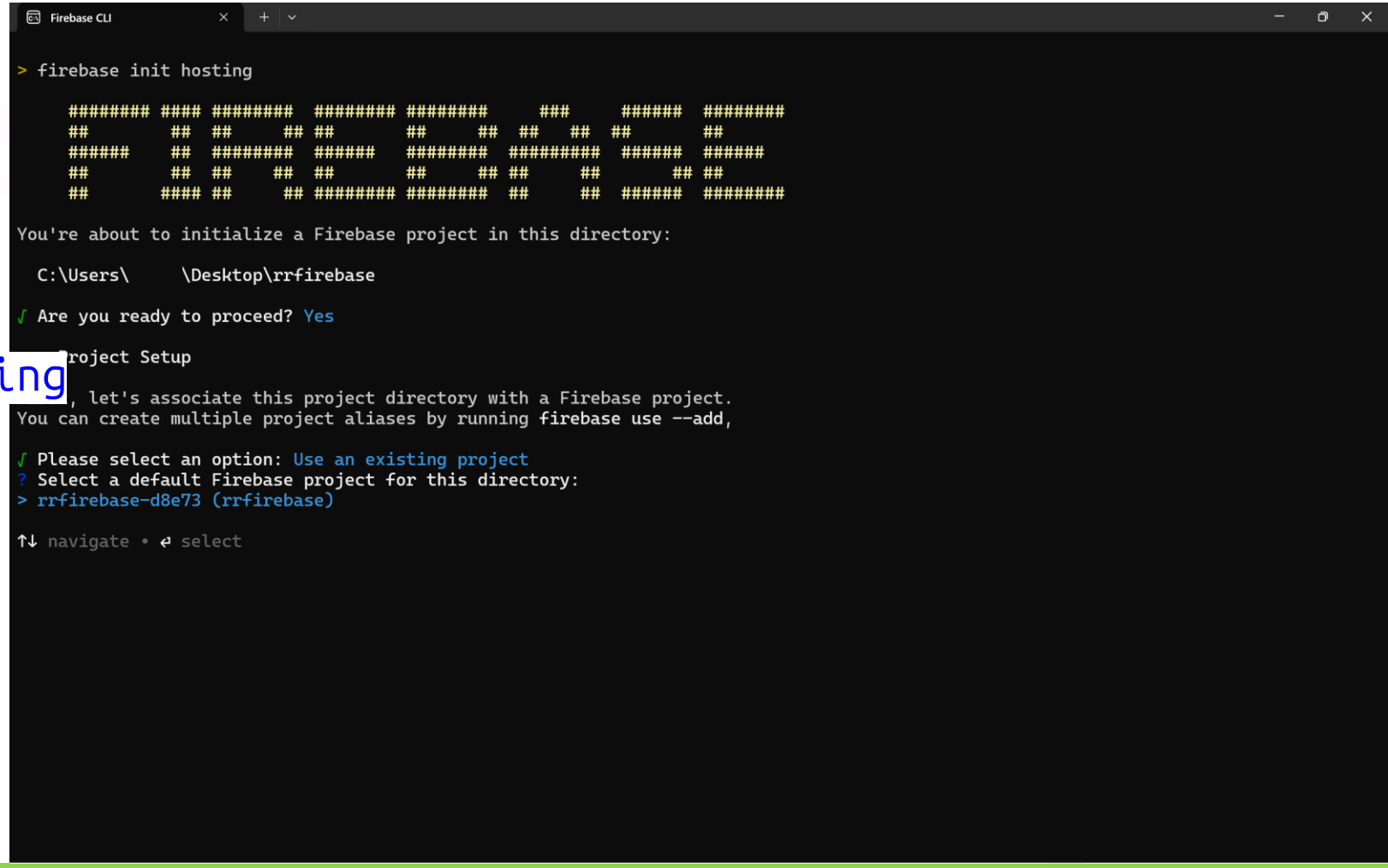
Firebase Hosting 靜態網頁部署

3. 準備並設定 AI 專案

於 Firebase CLI

③ 初始化專案

```
firebase init hosting
```



© 2026 Enos Chou

Firebase Hosting 靜態網頁部署

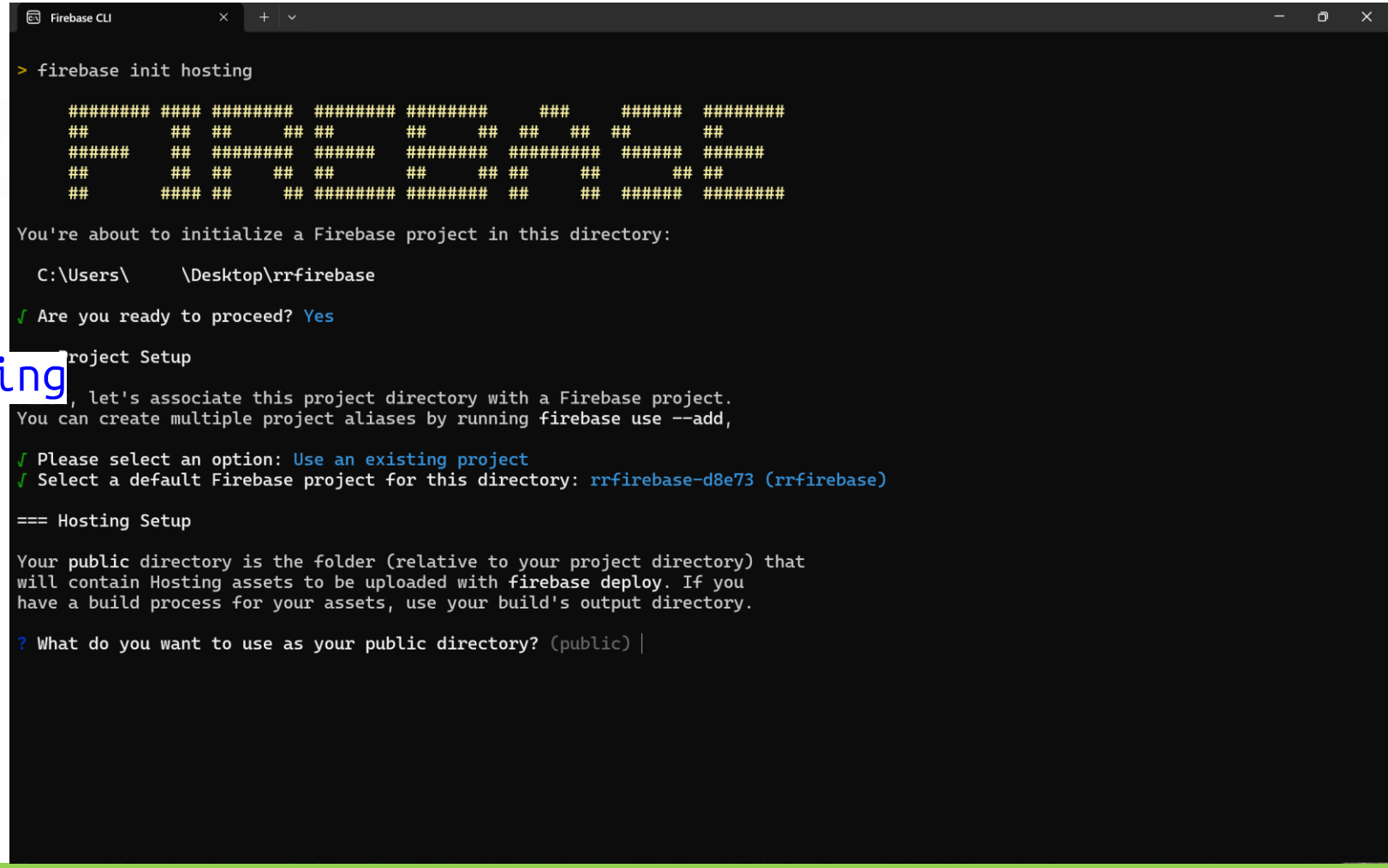
3. 準備並設定 AI 專案

於 Firebase CLI

③ 初始化專案

`firebase init hosting`

是否公開網站



© 2026 Enos Chou

Firebase Hosting 靜態網頁部署

3. 準備並設定 AI 專案

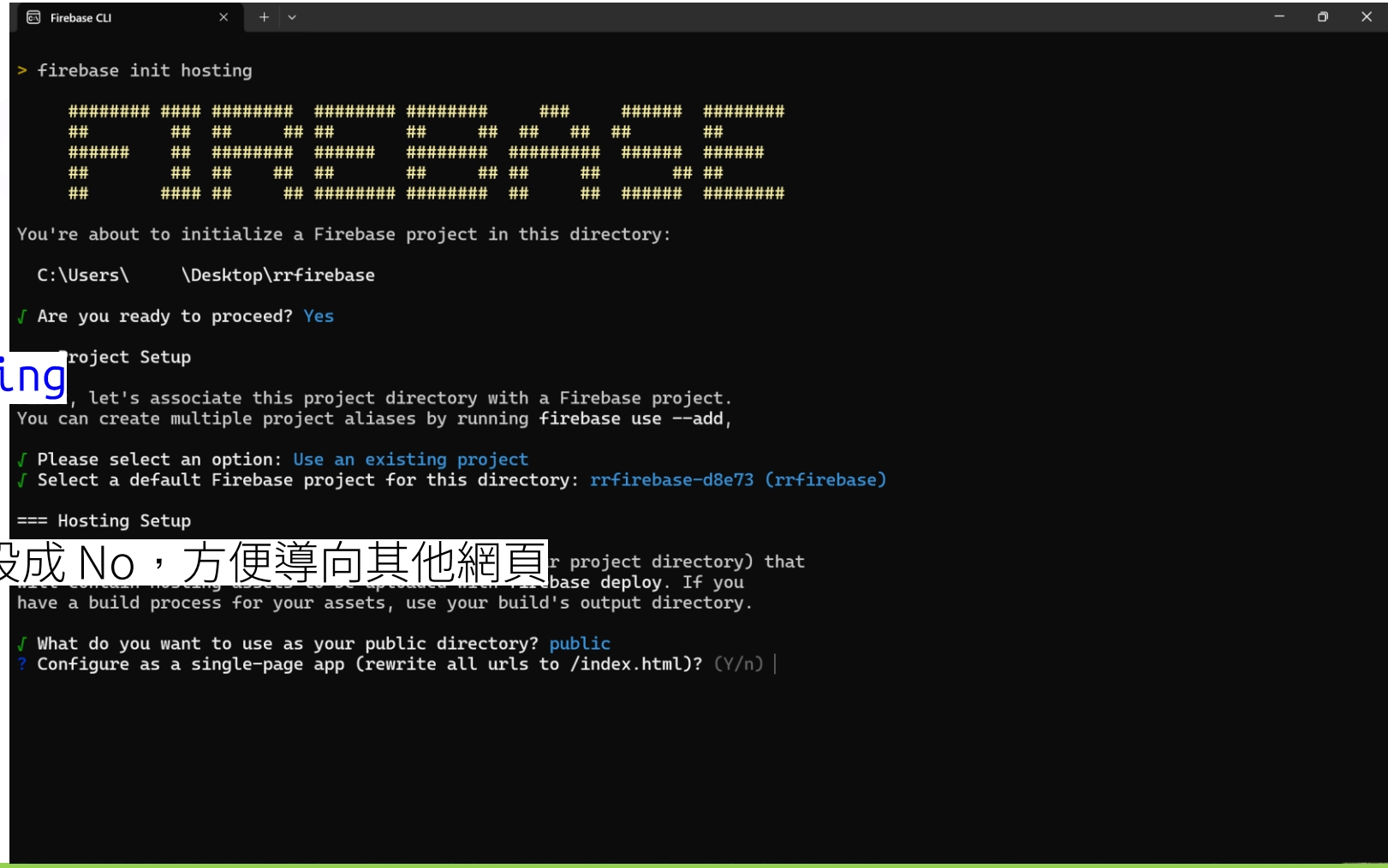
於 Firebase CLI

③ 初始化專案

`firebase init hosting`

是否都導向同一頁

若有多個 .html 要考慮設成 No，方便導向其他網頁



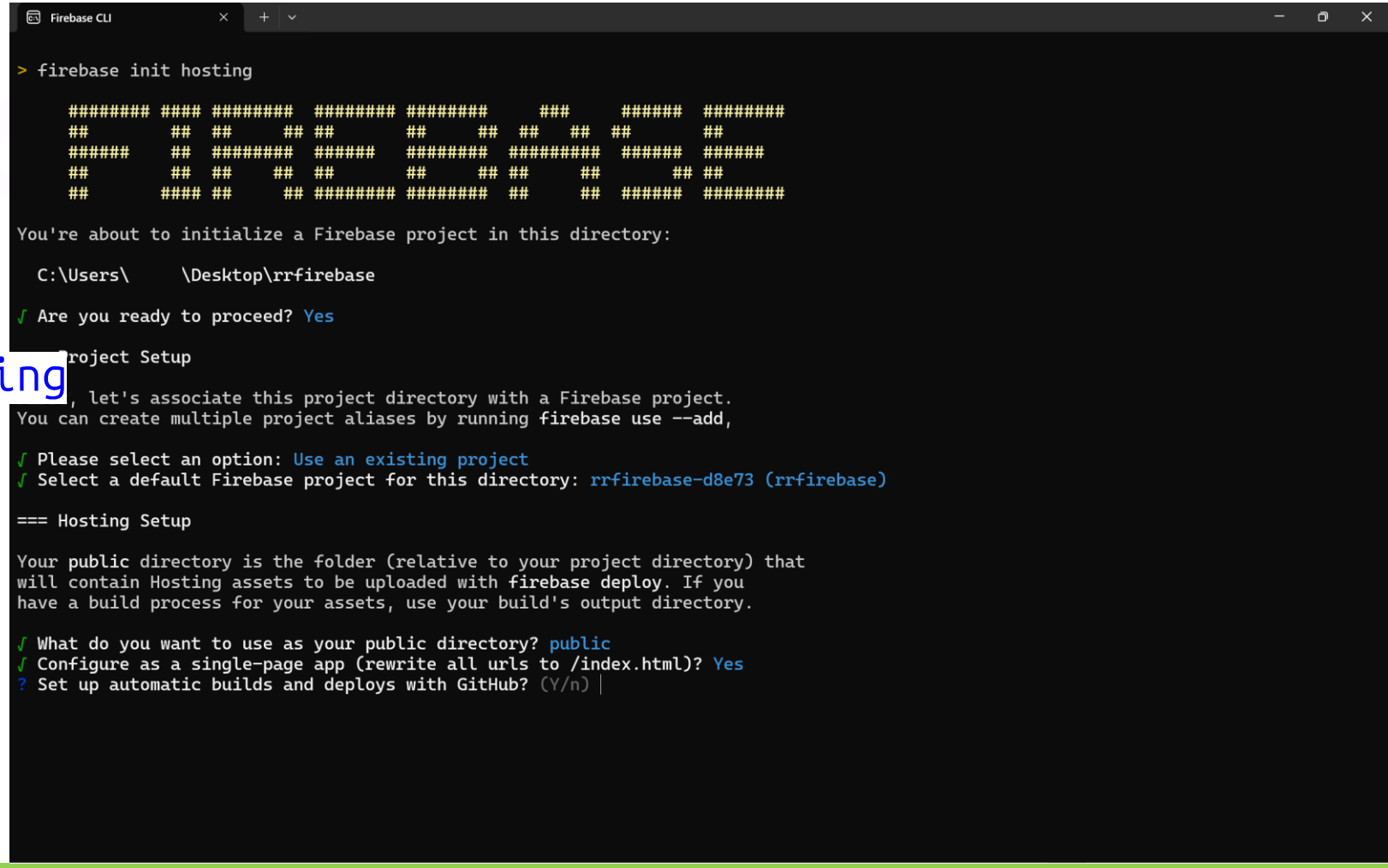
Firebase Hosting 靜態網頁部署

3. 準備並設定 AI 專案

於 Firebase CLI

③ 初始化專案

`firebase init hosting`



© 2026 Enos Chou

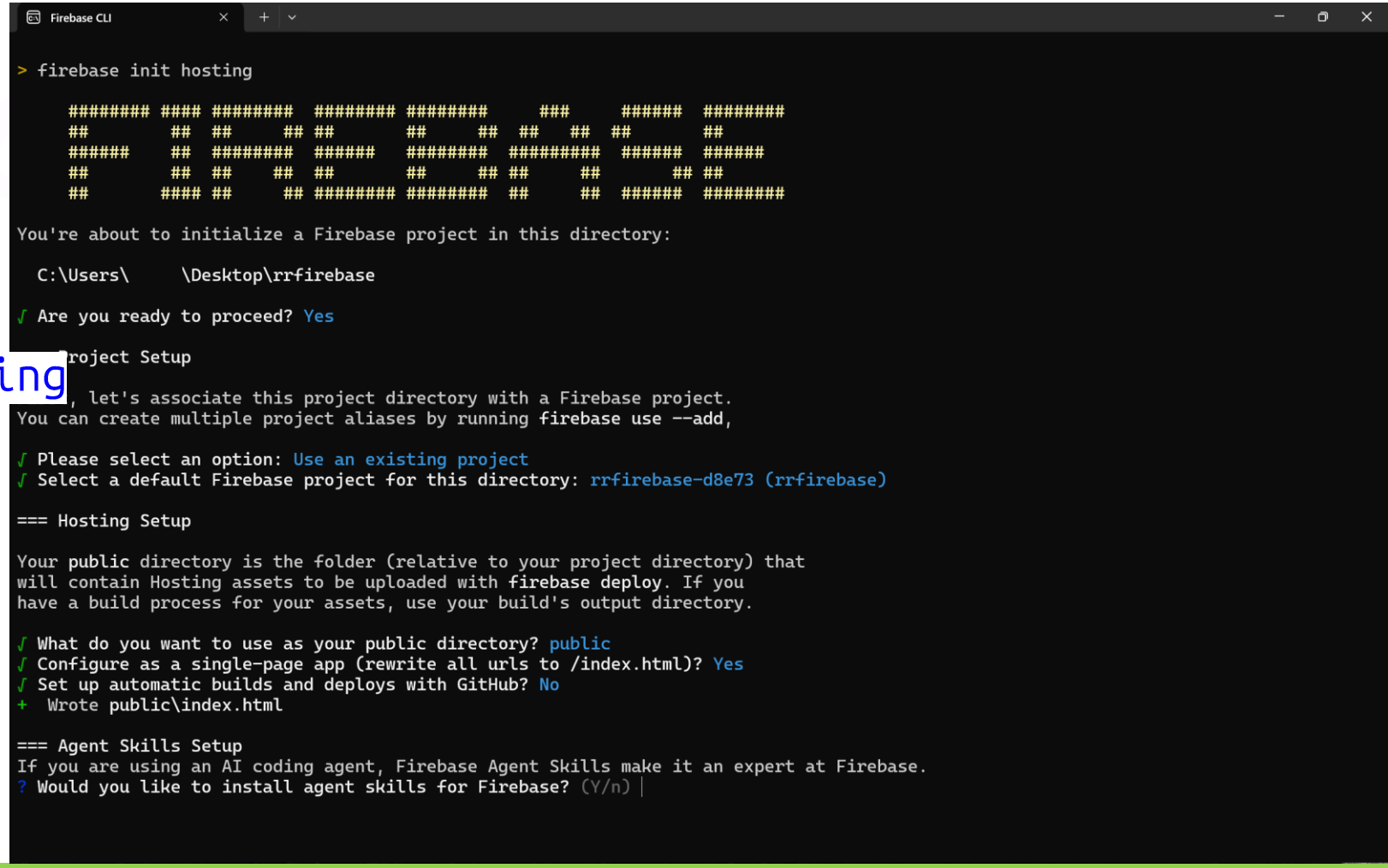
Firebase Hosting 靜態網頁部署

3. 準備並設定 AI 專案

於 Firebase CLI

③ 初始化專案

`firebase init hosting`



© 2026 Enos Chou

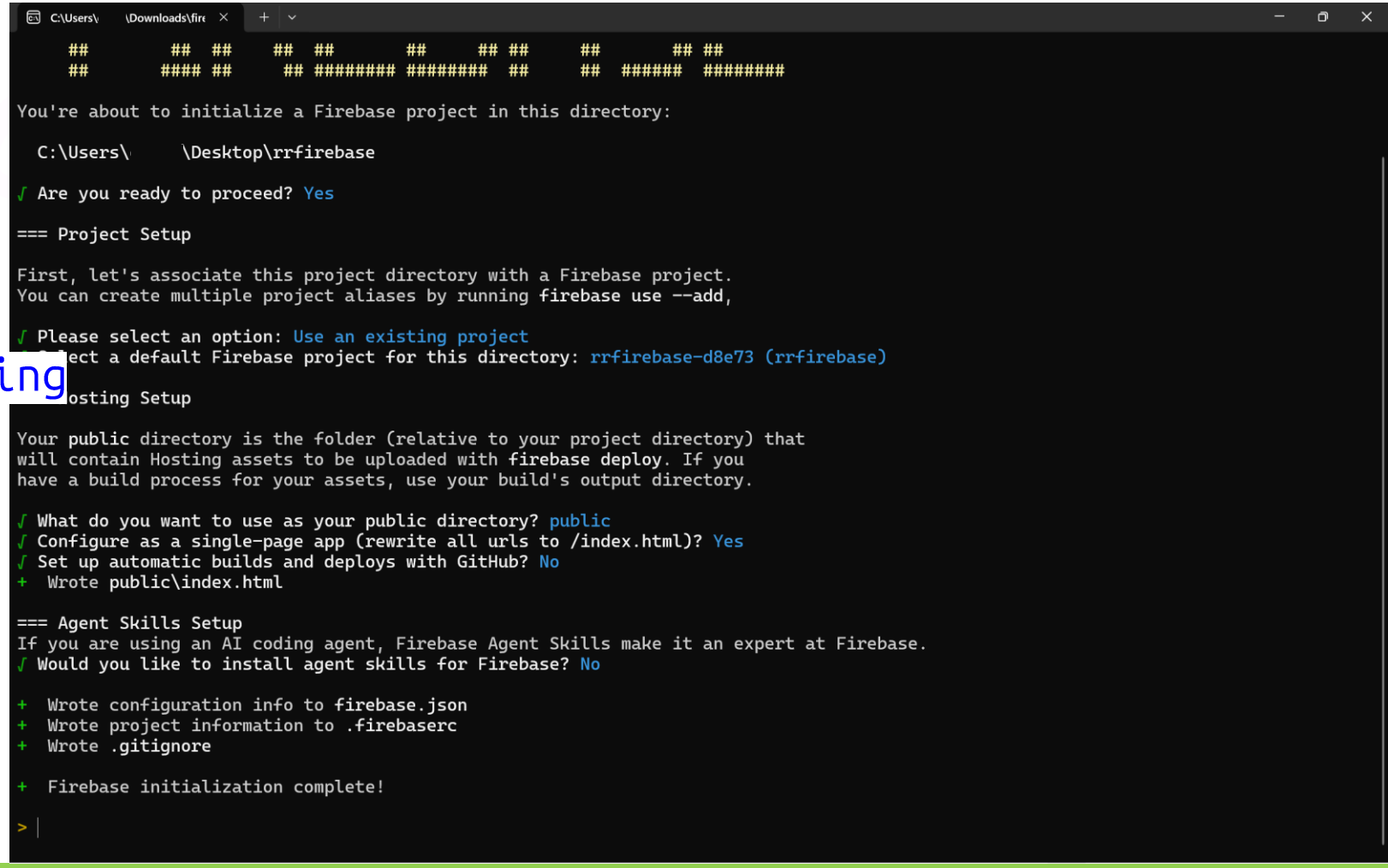
Firebase Hosting 靜態網頁部署

3. 準備並設定 AI 專案

於 Firebase CLI

③ 初始化專案

```
firebase init hosting
```



© 2026 Enos Chou

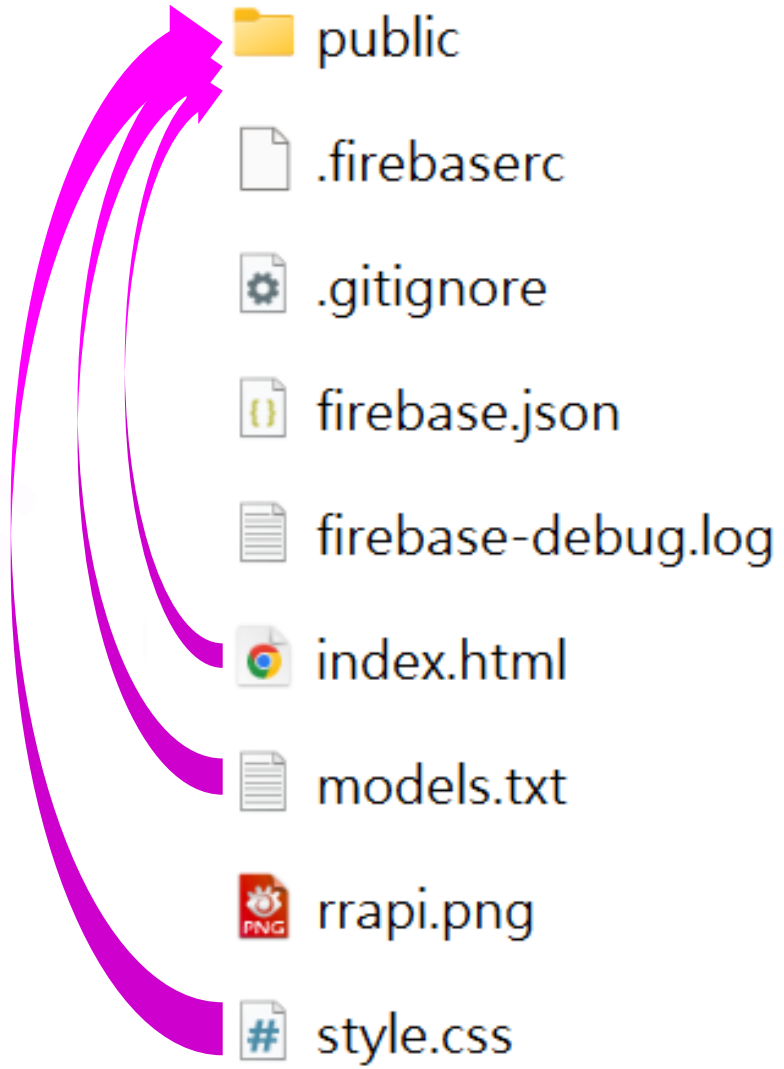
Firebase Hosting 靜態網頁部署

3. 準備並設定 AI 專案

於 Desktop File Explorer

④ 準備上線檔案

將上線需要的網站相關檔案移入 `Firebase` 建立之 `public` 子資料夾



© 2026 Enos Chou

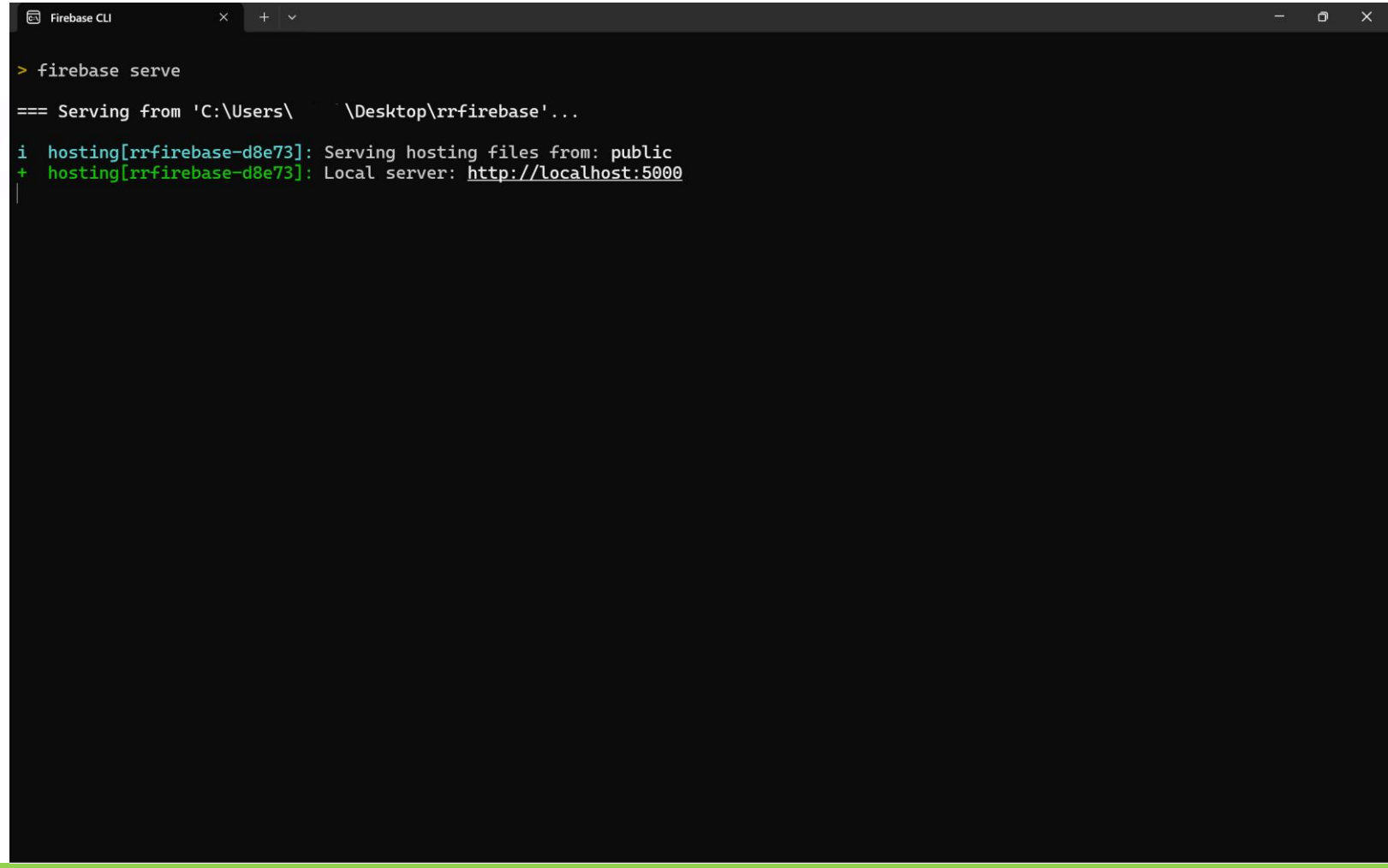
Firebase Hosting 靜態網頁部署

4. 部署網頁

於 Firebase CLI

① 部署前測試

```
firebase serve
```



© 2026 Enos Chou

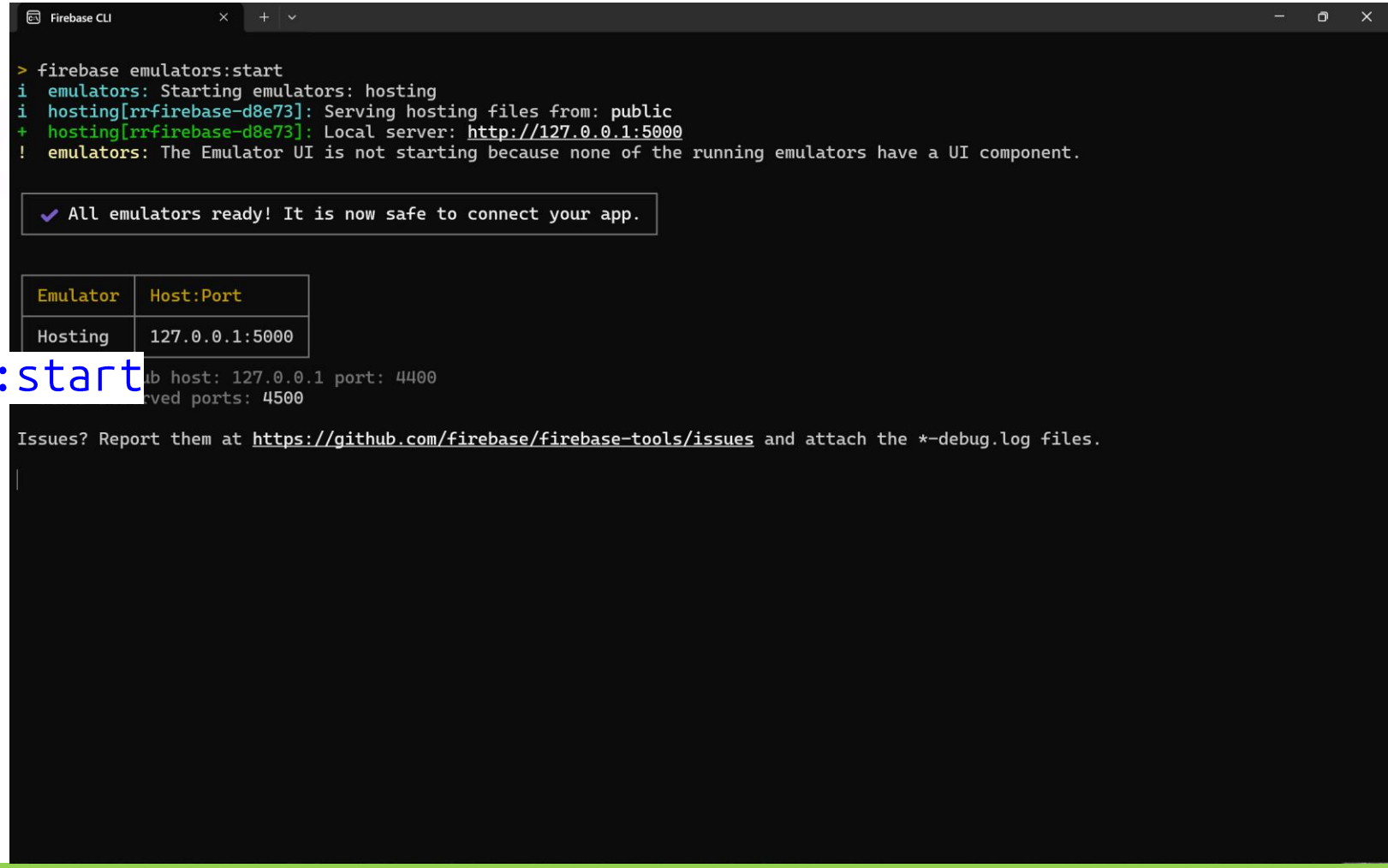
Firebase Hosting 靜態網頁部署

4. 部署網頁

於 Firebase CLI

① 部署前測試

```
firebase emulators:start
```

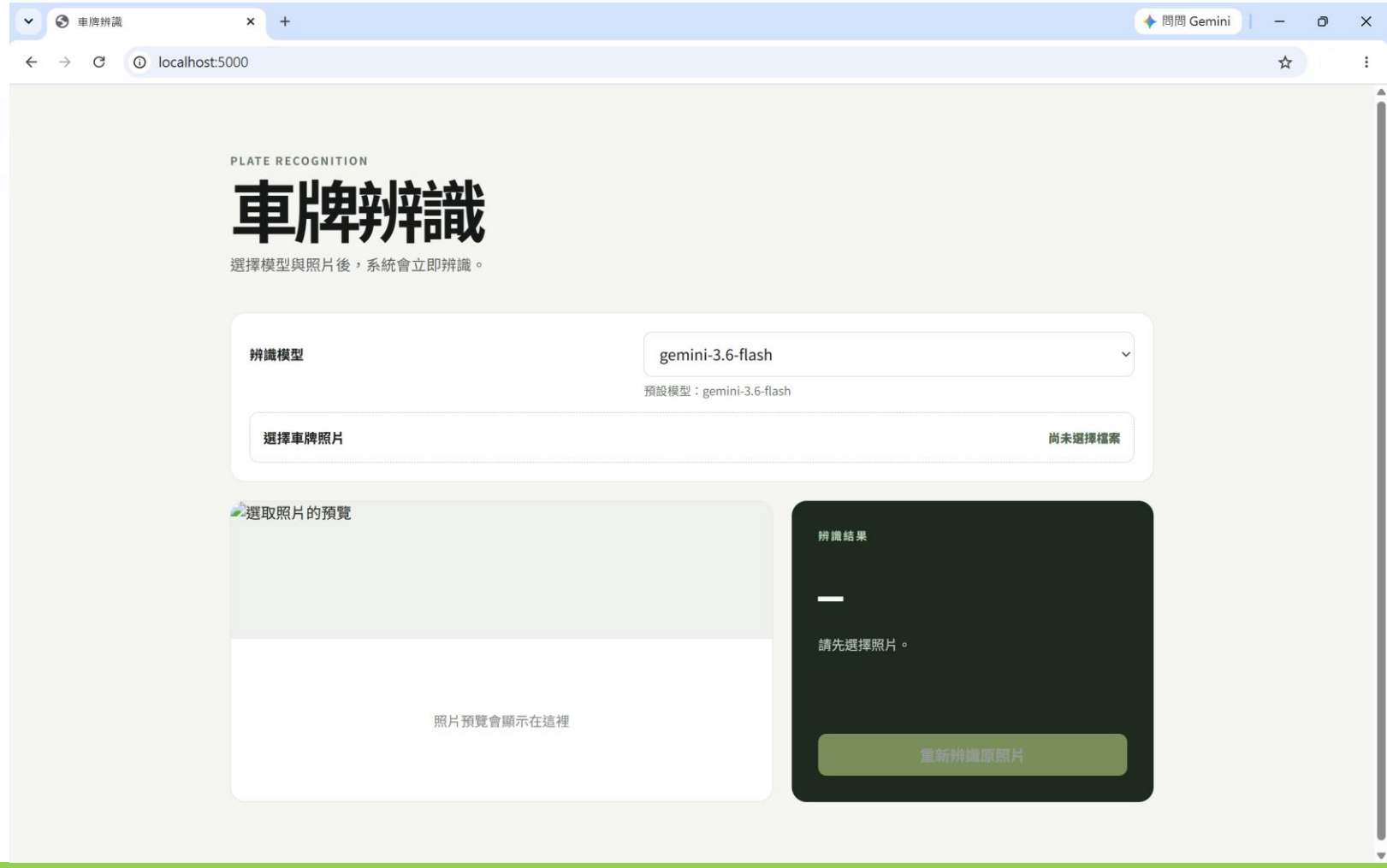


Firebase Hosting 靜態網頁部署

4. 部署網頁

於 Browser

① 部署前測試



Firebase Hosting 靜態網頁部署

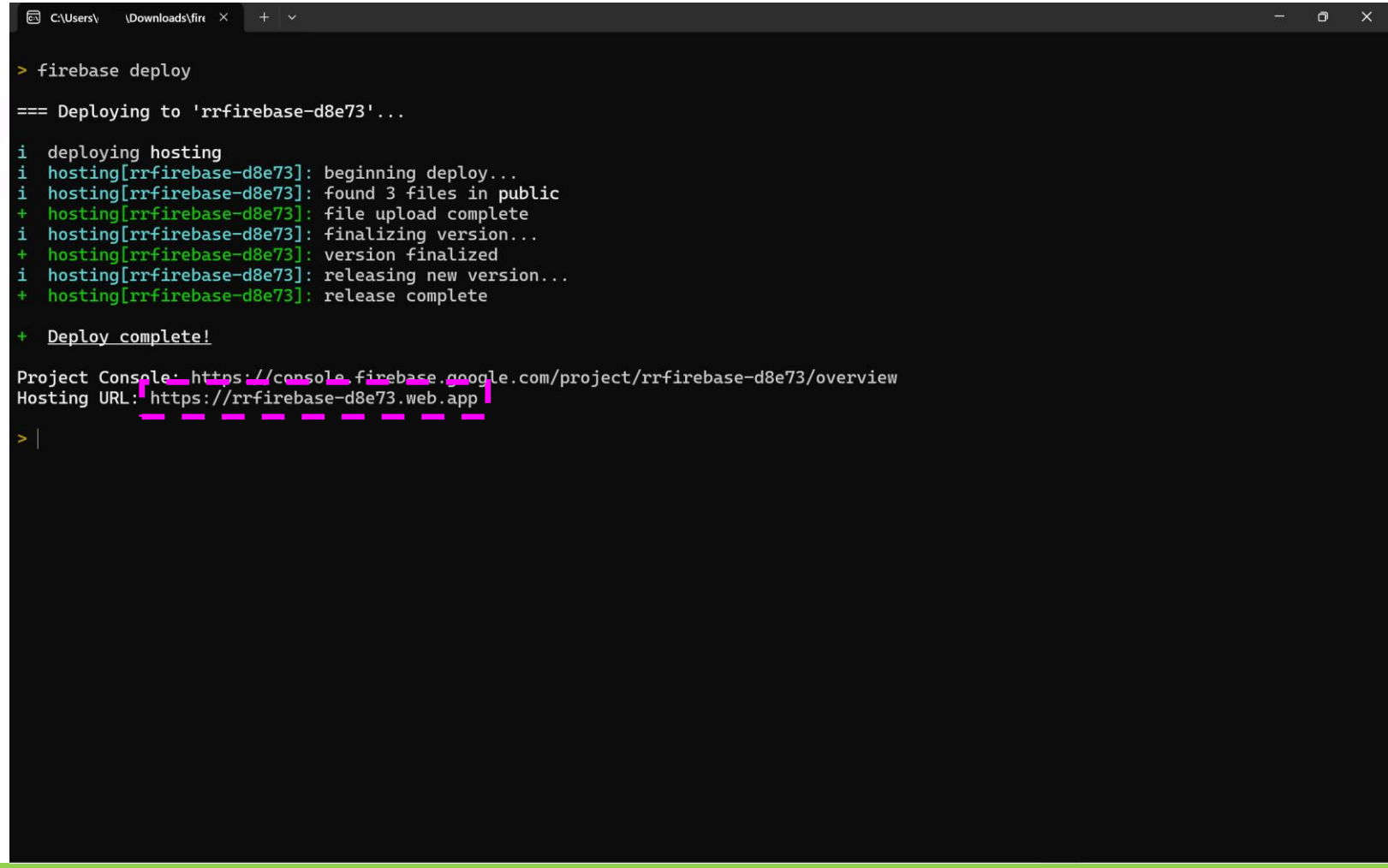
4. 部署網頁

於 Firebase CLI

② 部署

```
firebase deploy
```

記錄 Hosting URL



© 2026 Enos Chou

Firebase Hosting 靜態網頁部署

4. 部署網頁

於 Browser

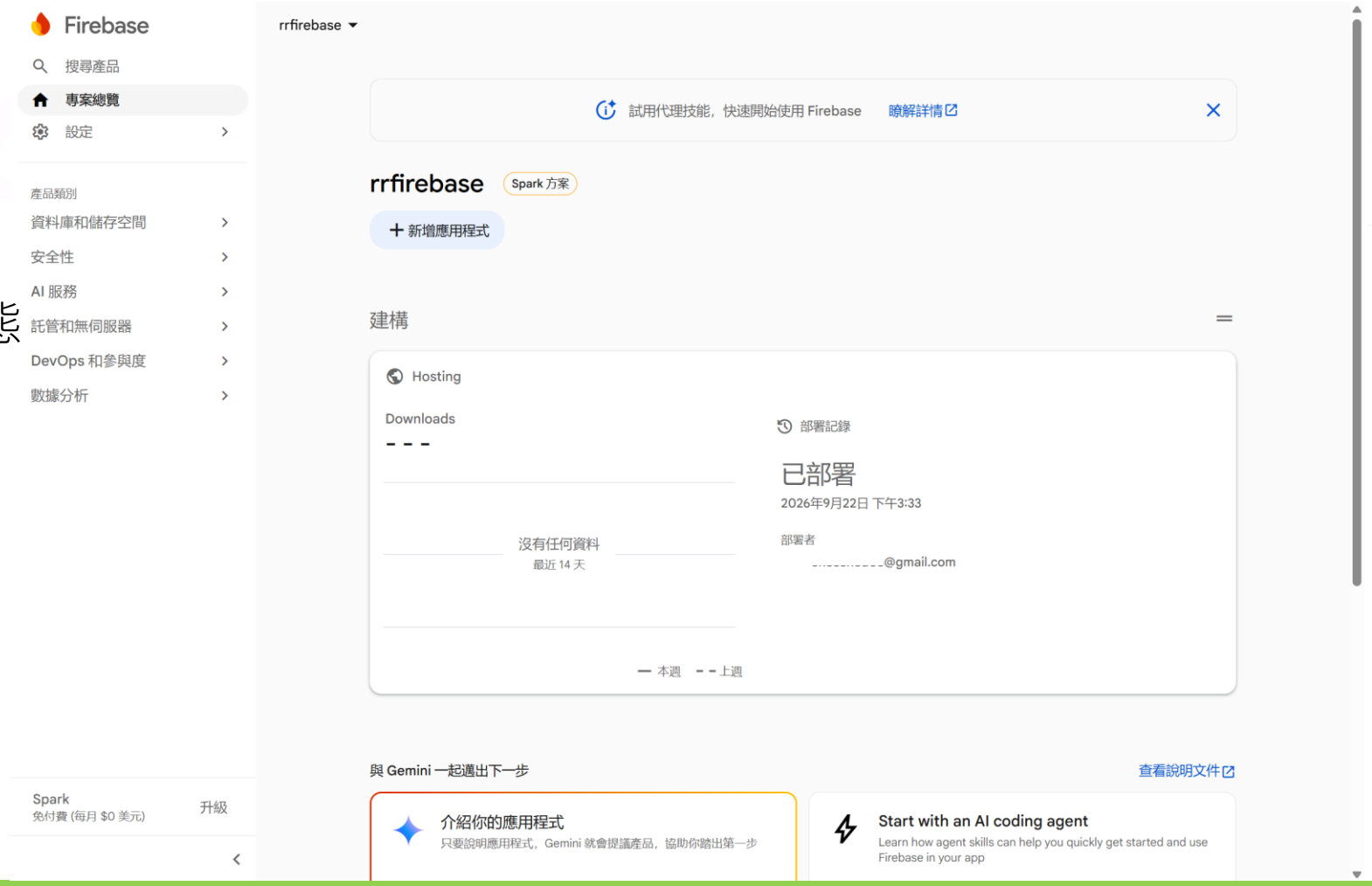
③ 部署後測試
以 Hosting URL 進行測試

Firebase Hosting 靜態網頁部署

4. 部署網頁

於 Firebase Console

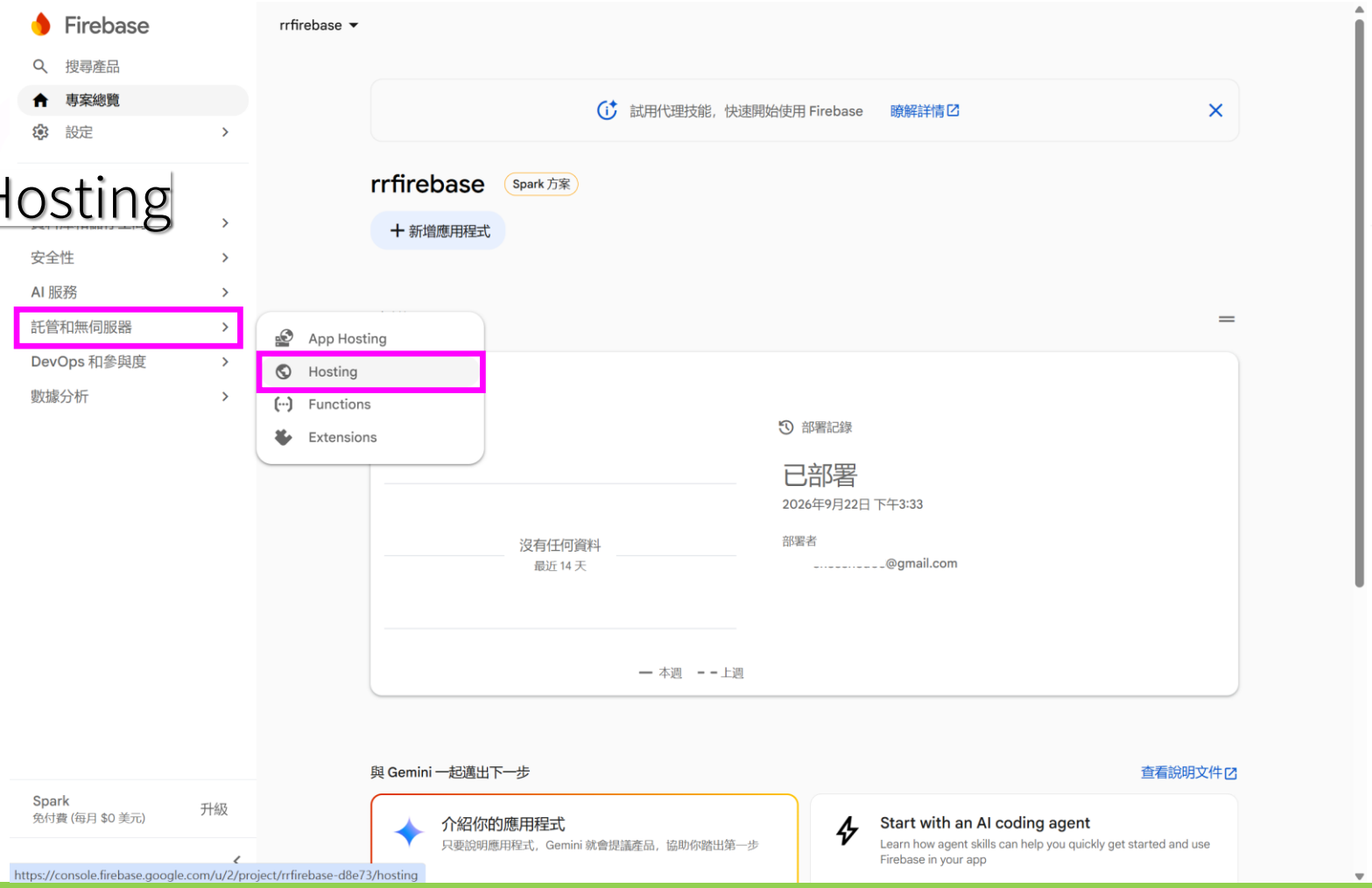
④ 確認部署後專案狀態



Firebase Hosting 靜態網頁部署

5. 轉換網域

a. 託管和無伺服器 > Hosting



Firebase Hosting 靜態網頁部署

5. 轉換網域

b. 新增自訂網域

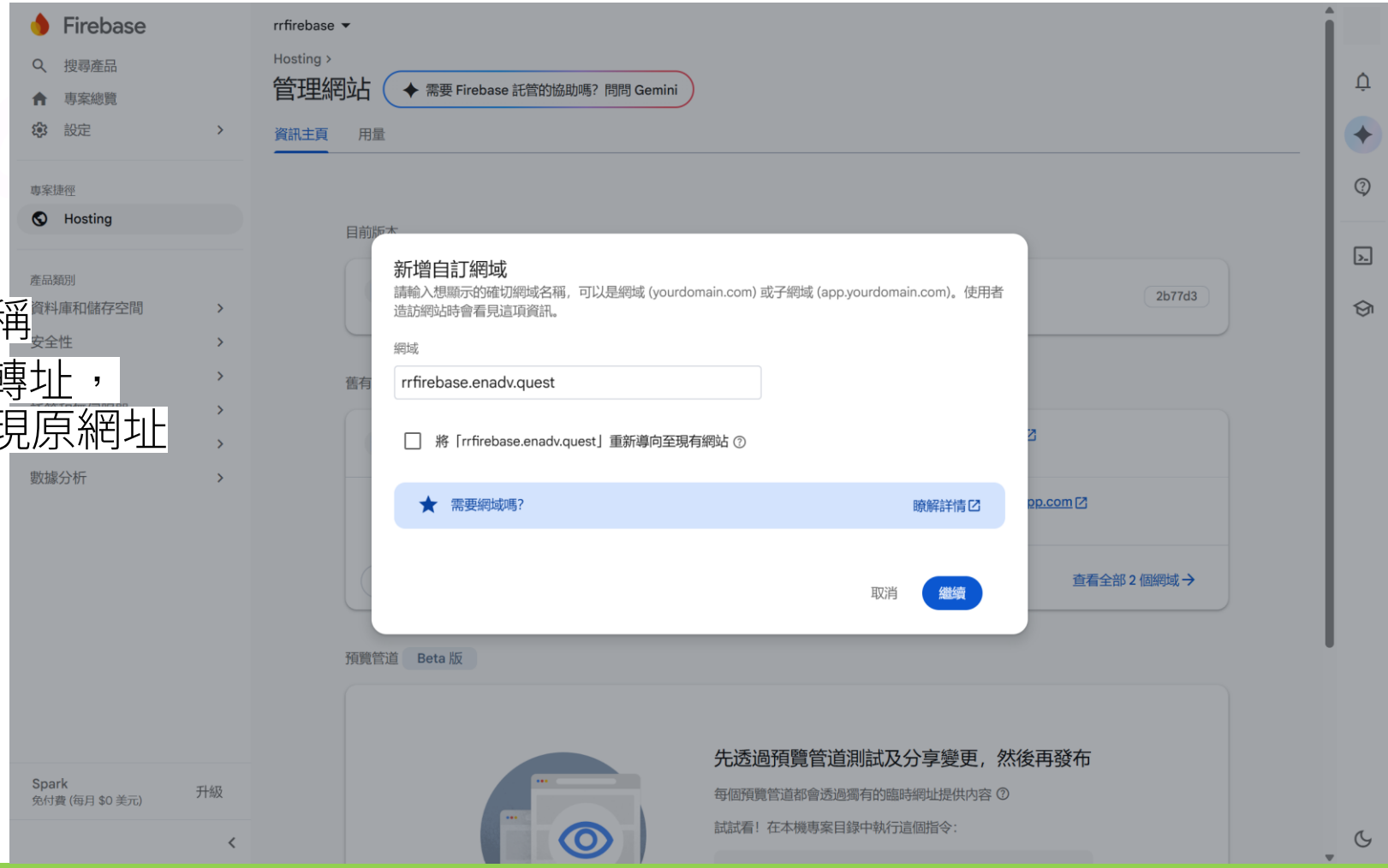
Firebase Hosting 靜態網頁部署

5. 轉換網域

b. 新增自訂網域

① 指定自訂之網域名稱

勾選 check box 會導致轉址，
連結新網址後最終會呈現原網址

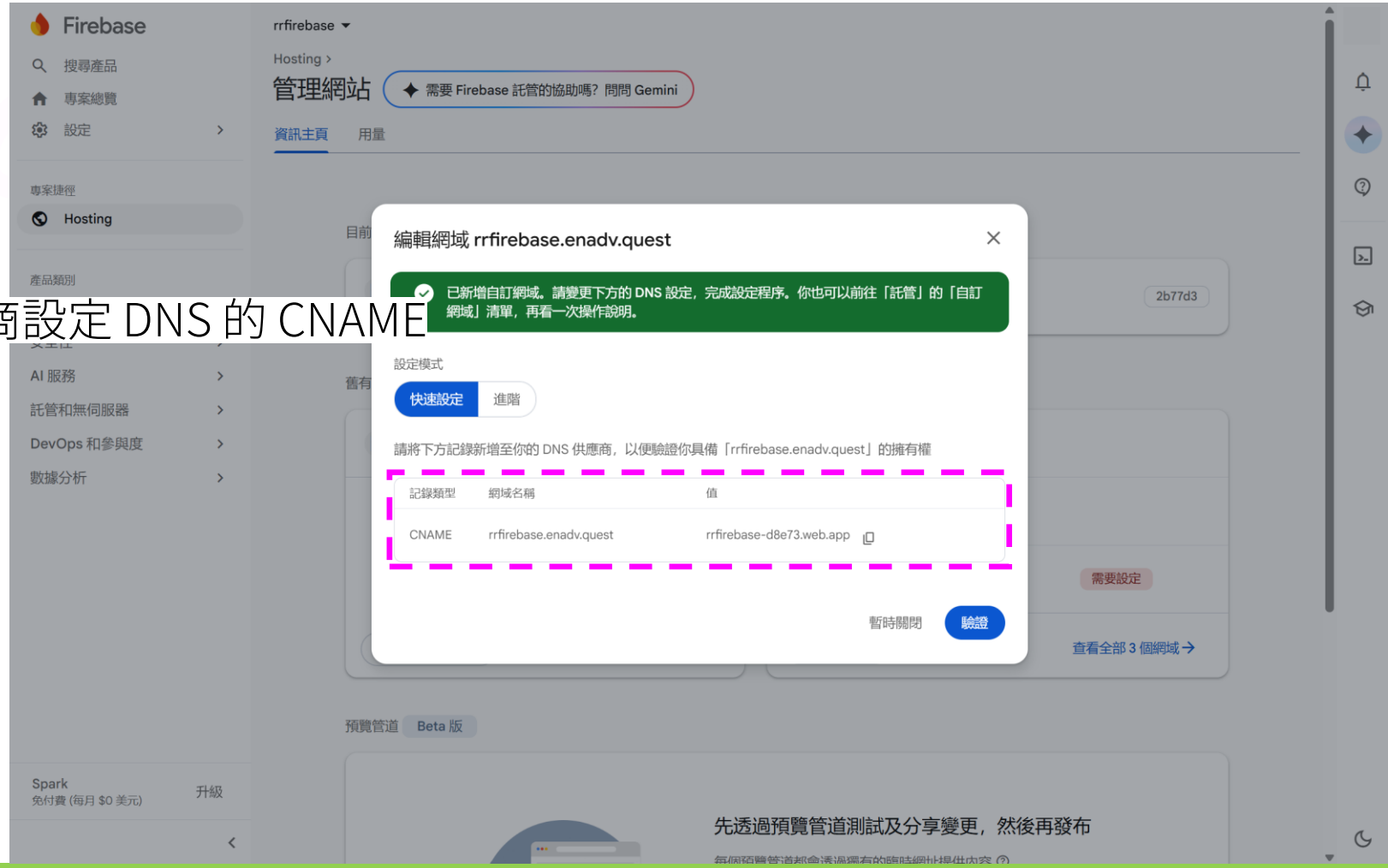


Firebase Hosting 靜態網頁部署

5. 轉換網域

b. 新增自訂網域

② 依提示於網域供應商設定 DNS 的 CNAME



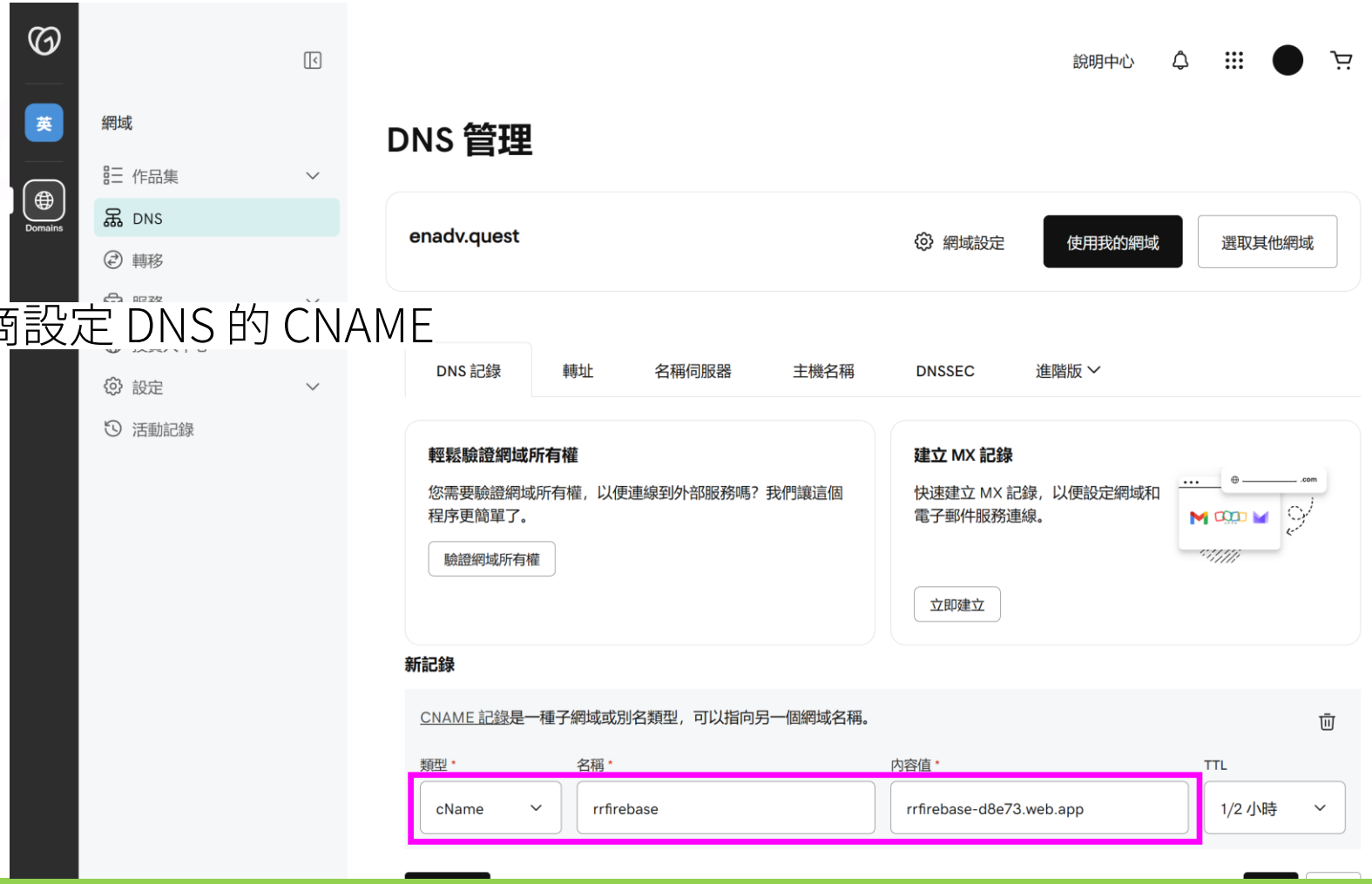
Firebase Hosting 靜態網頁部署

5. 轉換網域

b. 新增自訂網域

② 依提示於網域供應商設定 DNS 的 CNAME

於網域供應商設定 DNS



Firebase Hosting 靜態網頁部署

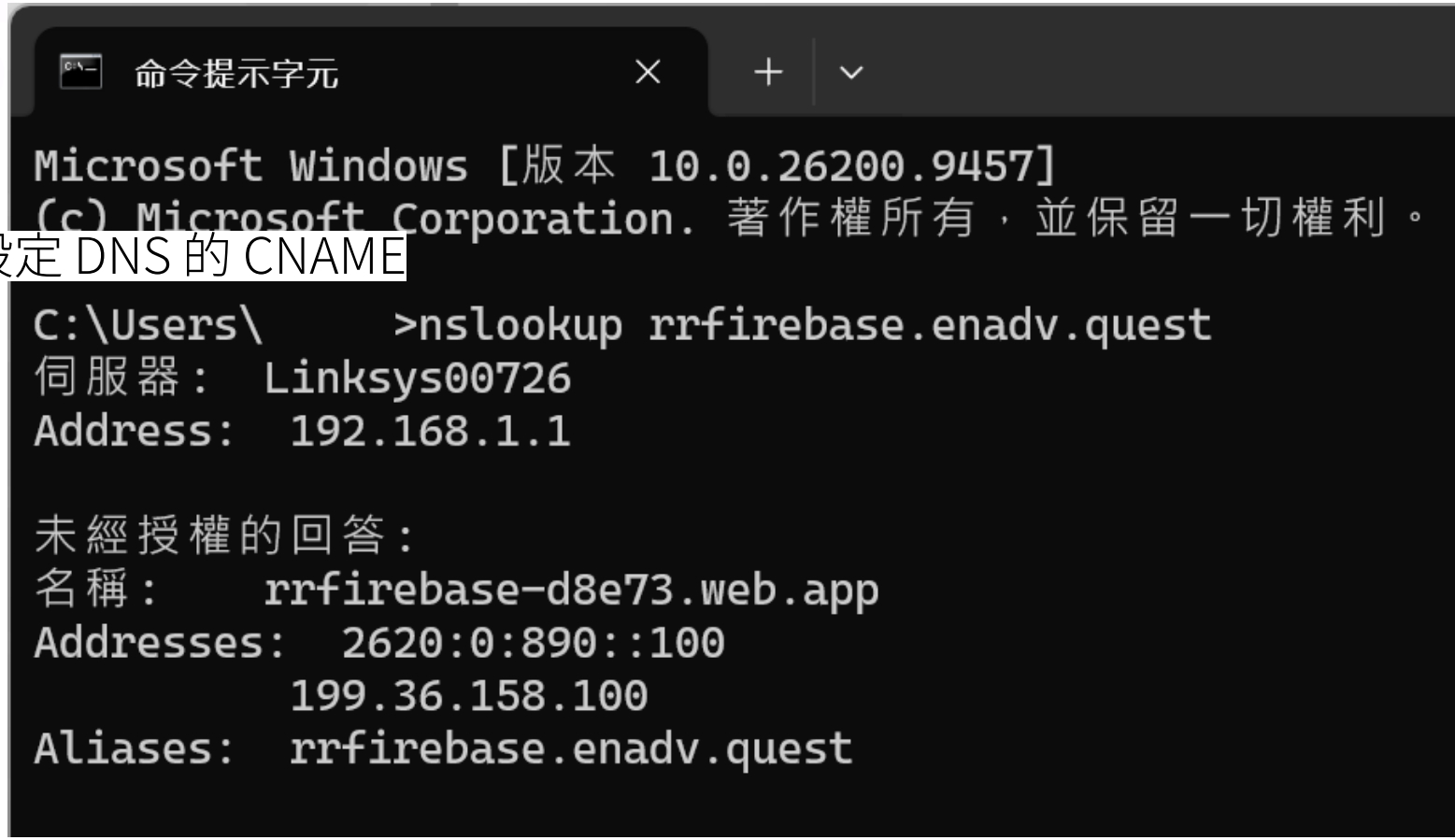
5. 轉換網域

b. 新增自訂網域

② 依提示於網域供應商設定 DNS 的 CNAME

確認設定成功

`nslookup cname`



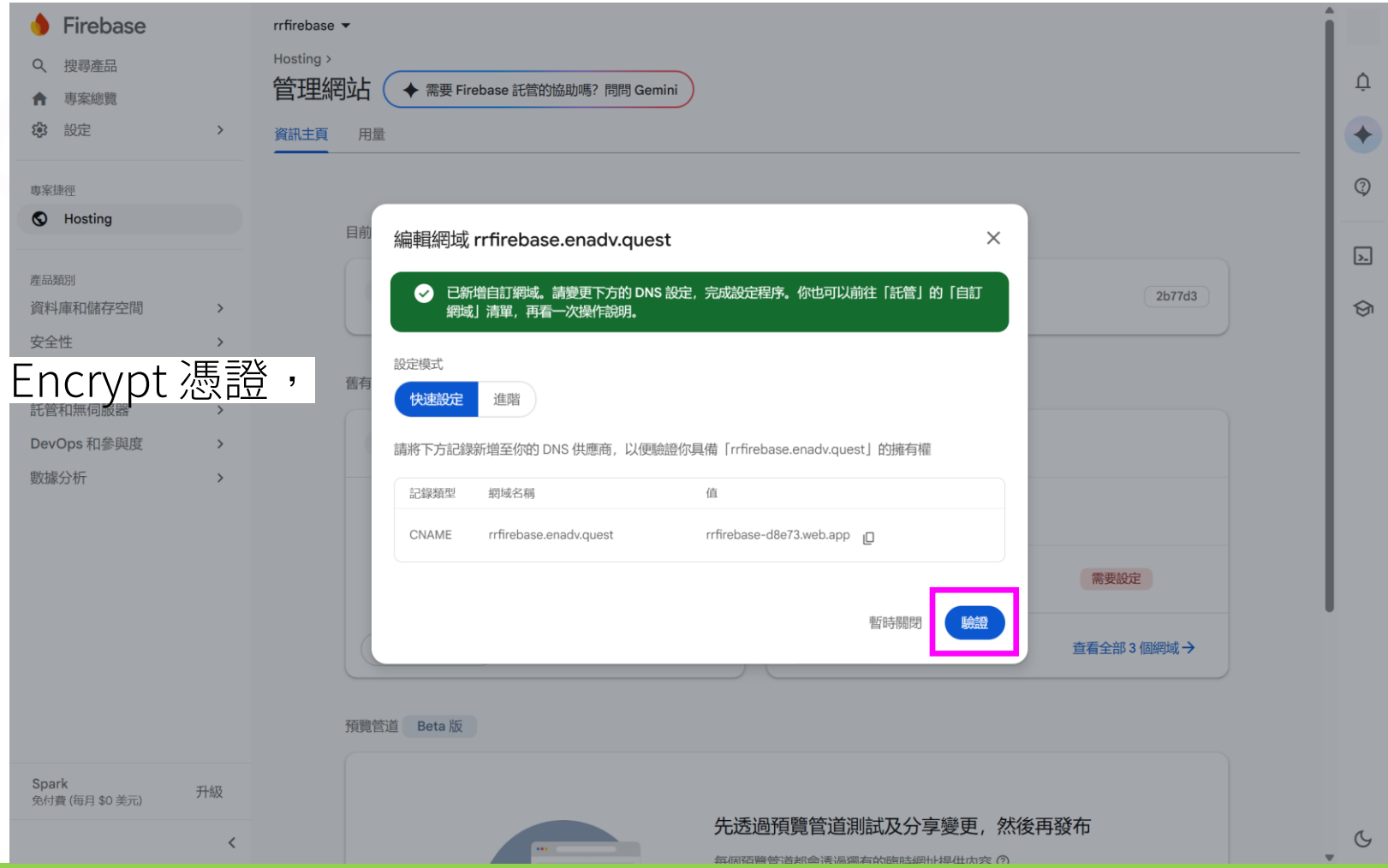
Firebase Hosting 靜態網頁部署

5. 轉換網域

b. 新增自訂網域

③ 驗證

接下來會自動申請 Let's Encrypt 憑證，要等一陣子



© 2026 Enos Chou

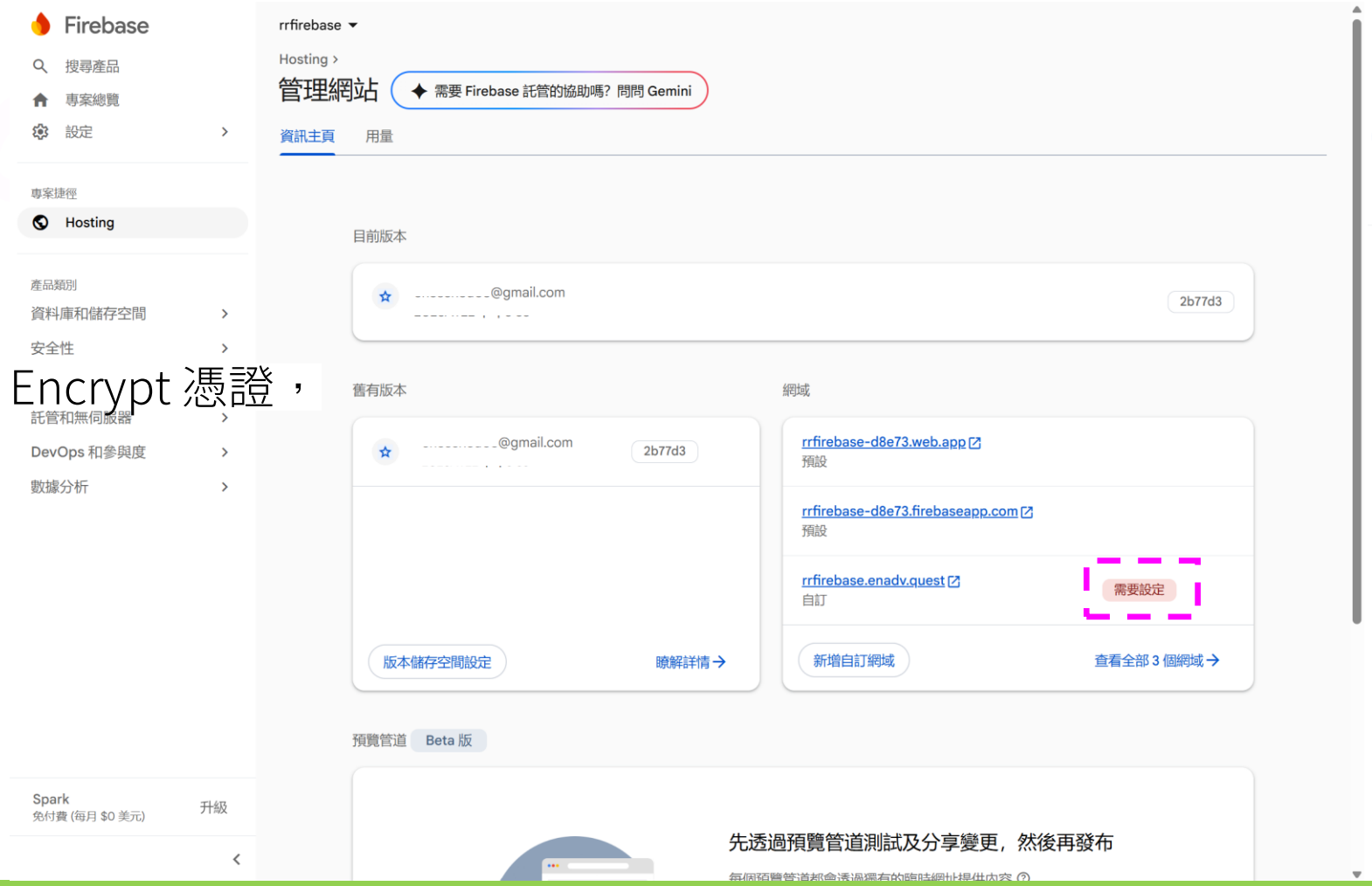
Firebase Hosting 靜態網頁部署

5. 轉換網域

b. 新增自訂網域

③ 驗證

接下來會自動申請 Let's Encrypt 憑證，要等一陣子



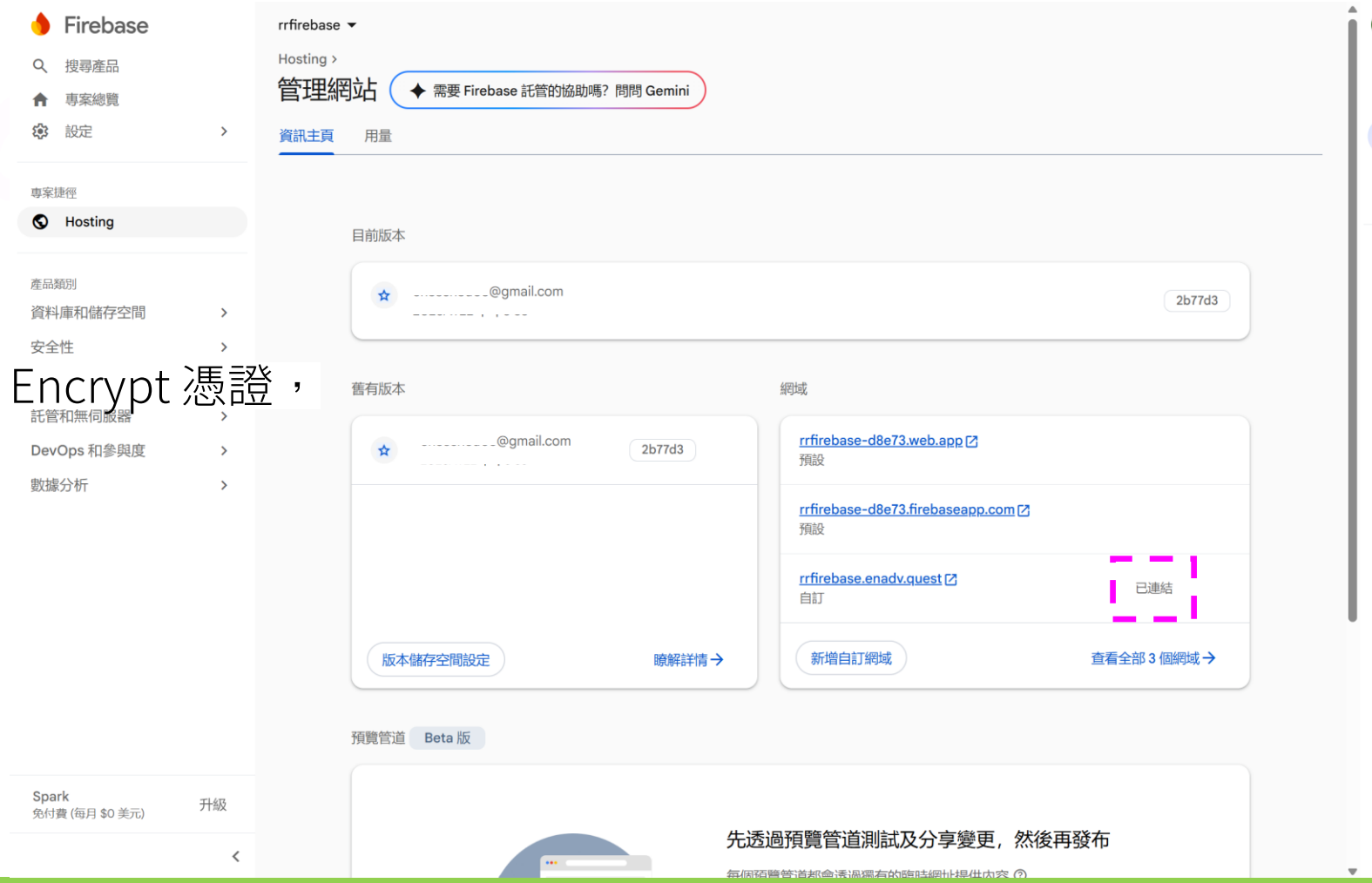
Firebase Hosting 靜態網頁部署

5. 轉換網域

b. 新增自訂網域

③ 驗證

接下來會自動申請 Let's Encrypt 憑證，要等一陣子



Firebase Hosting 靜態網頁部署

5. 轉換網域

b. 新增自訂網域

④ 轉換網域後測試

© 2026 Enos Chou

雲端 API Gateway

Azure API Management

Azure API Management

© 2026 Enos Chou

Azure API Management

目的

- 將所有 API 歸納至單一入口網域，統一管理，彈性交付

功能

- 來源控管
- 限速
- 限量
- 認證

© 2026 Enos Chou

Azure APIM

1. 建立 APIM

Azure APIM

1. 建立 APIM

Azure APIM

1. 建立 APIM

視需要加入識別用標籤

Azure APIM

1. 建立 APIM

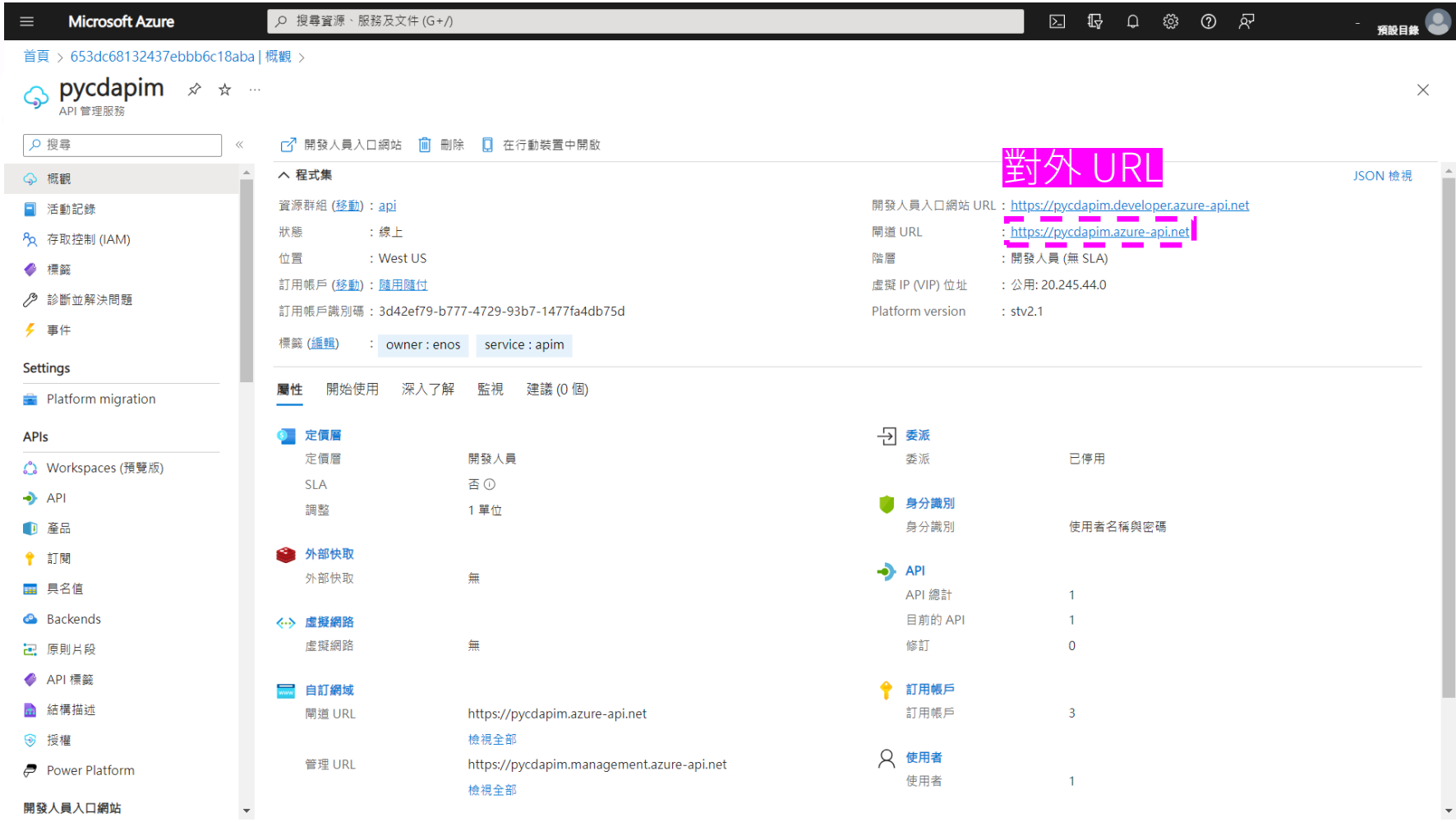
Azure APIM

1. 建立 APIM

© 2026 Enos Chou

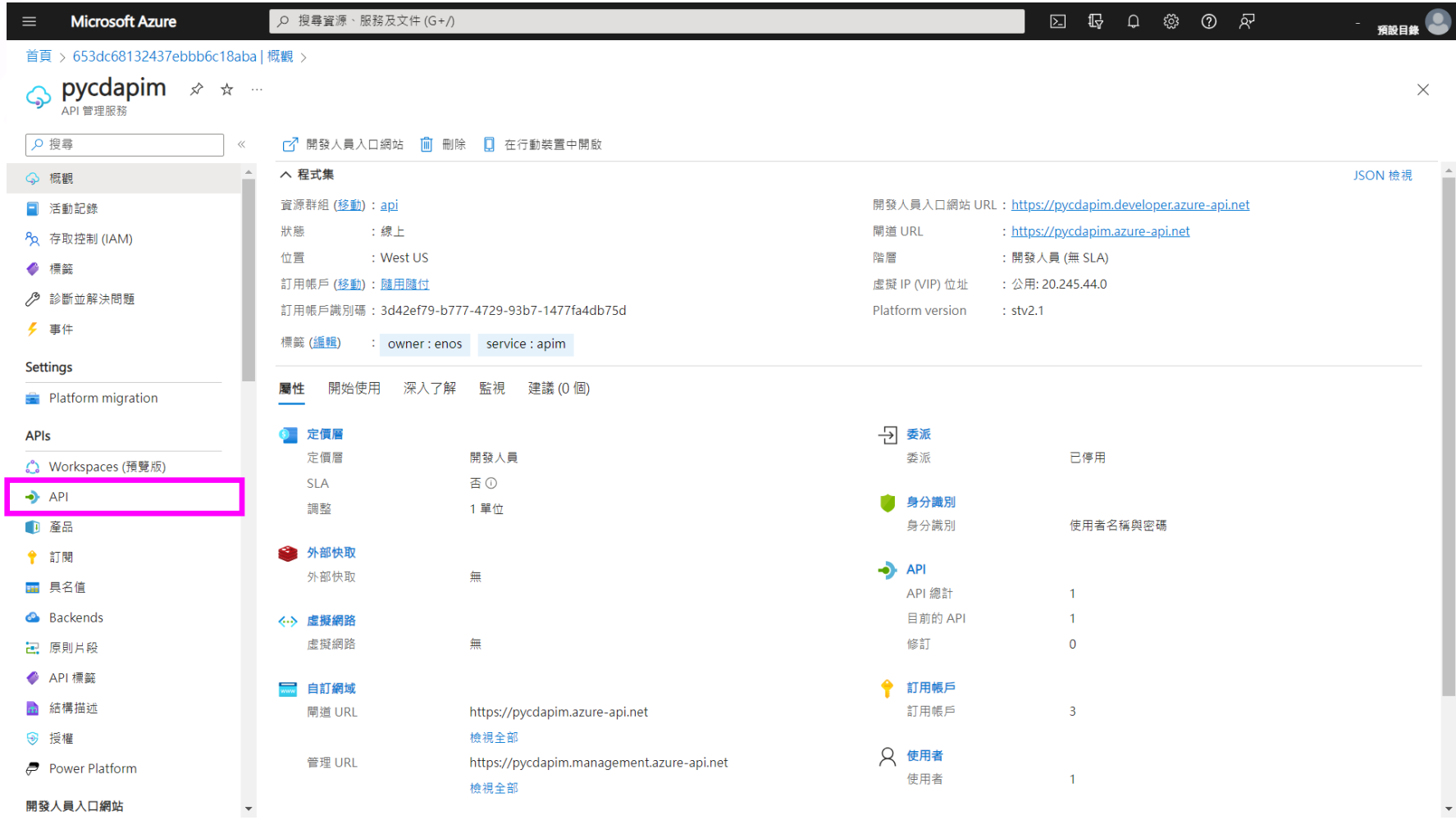
Azure APIM

1. 建立 APIM



Azure APIM

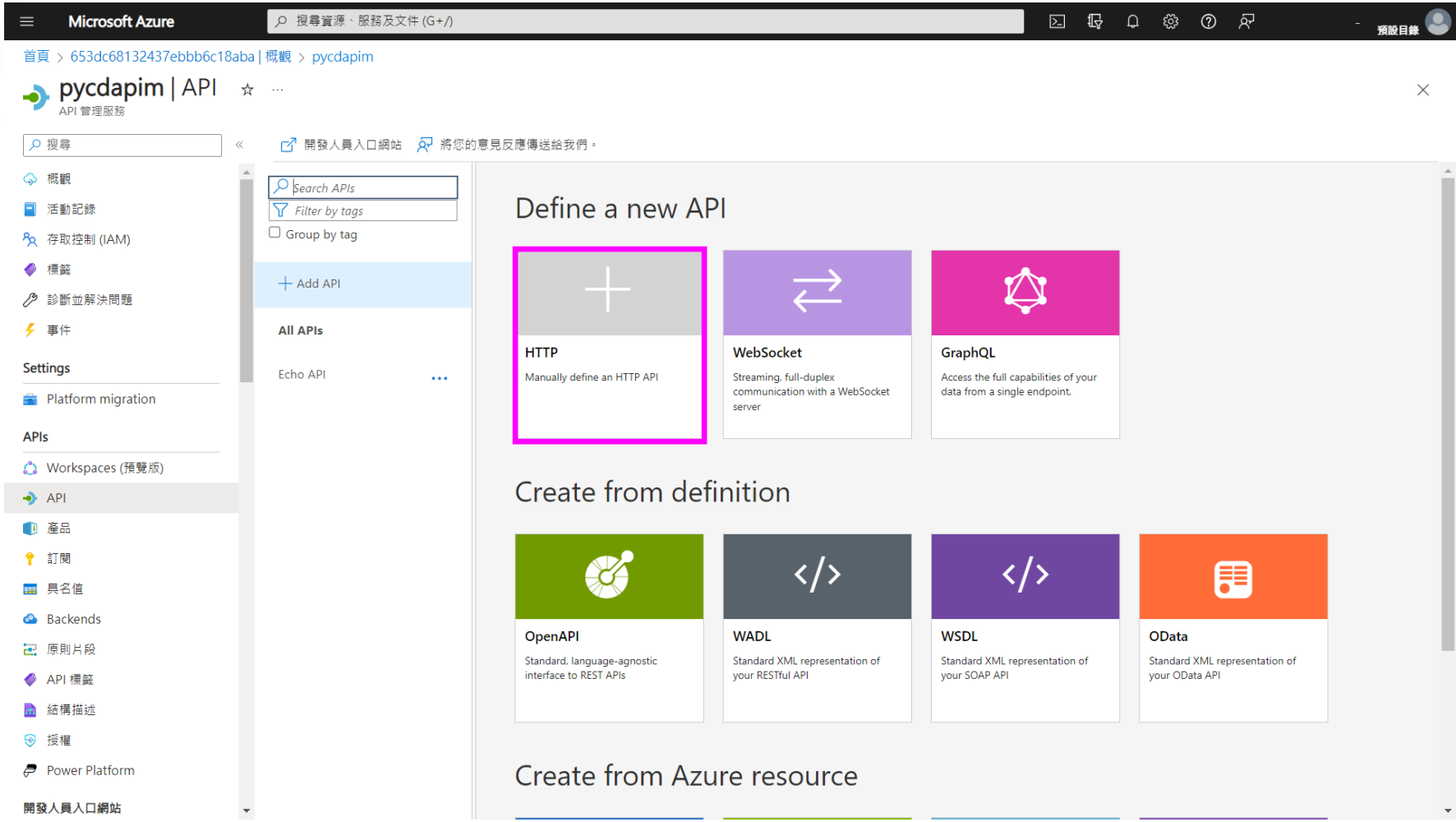
2. 綁定 API



© 2026 Enos Chou

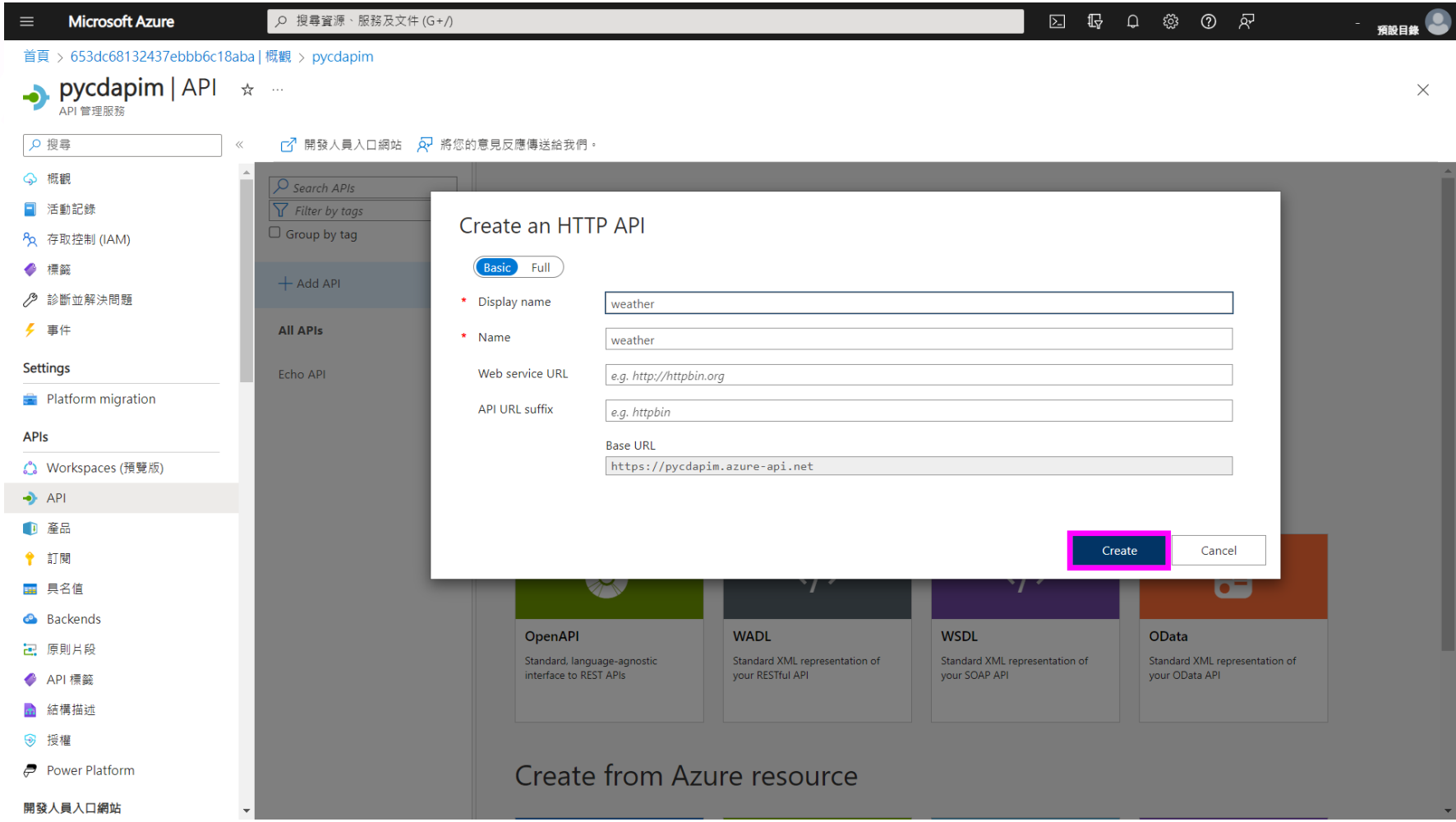
Azure APIM

2. 綁定 API



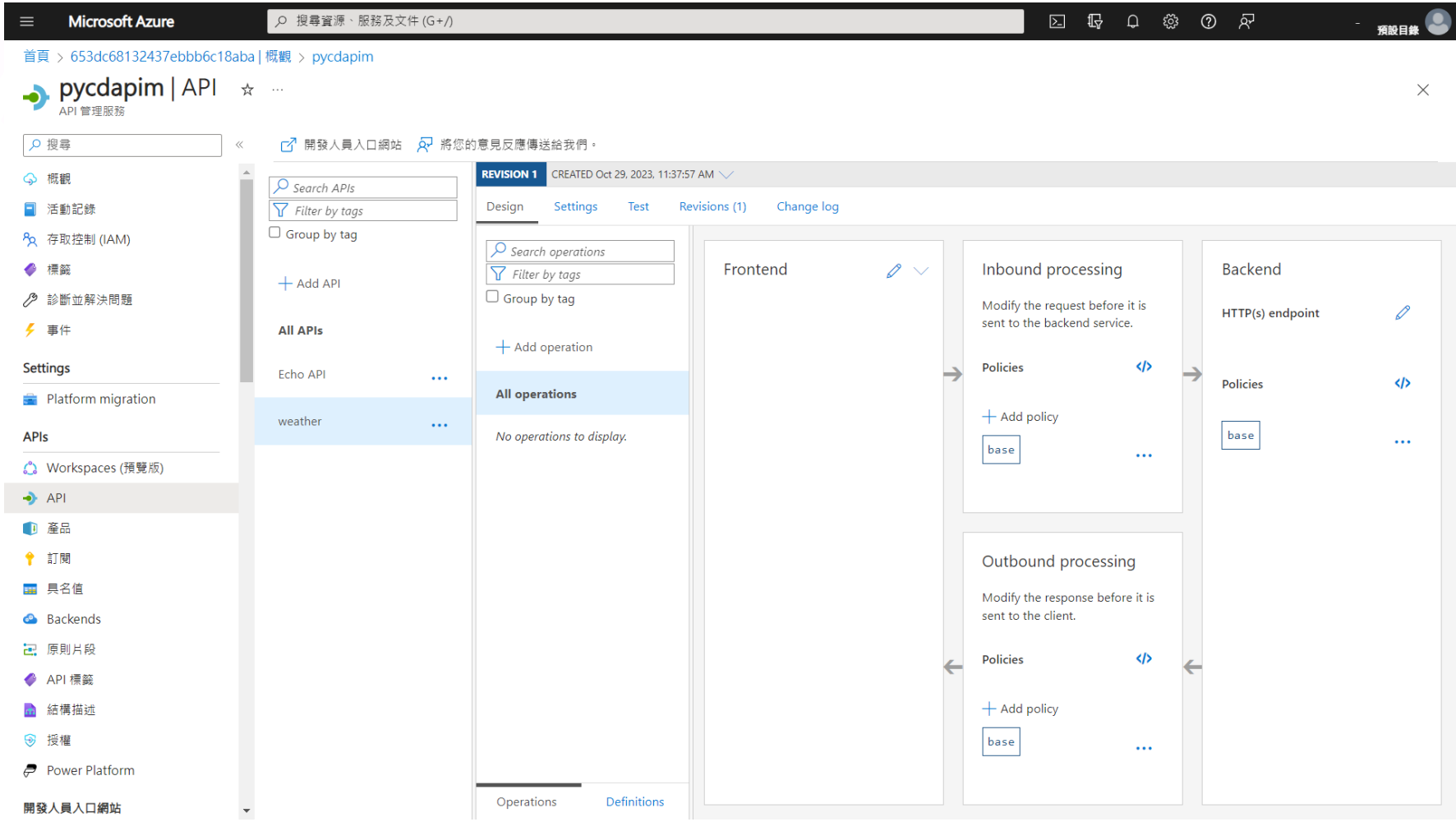
Azure APIM

2. 綁定 API



Azure APIM

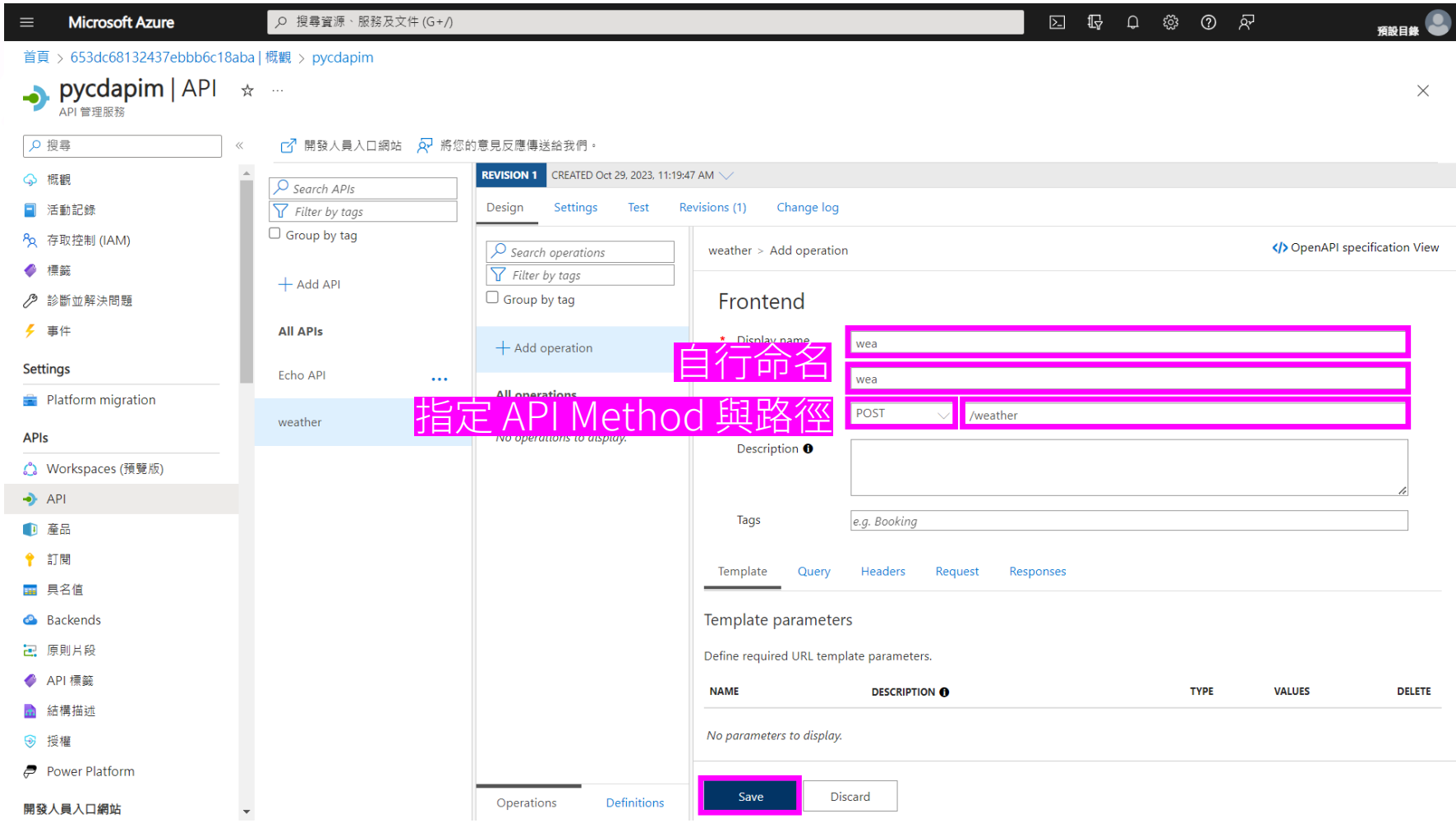
2. 綁定 API



© 2026 Enos Chou

Azure APIM

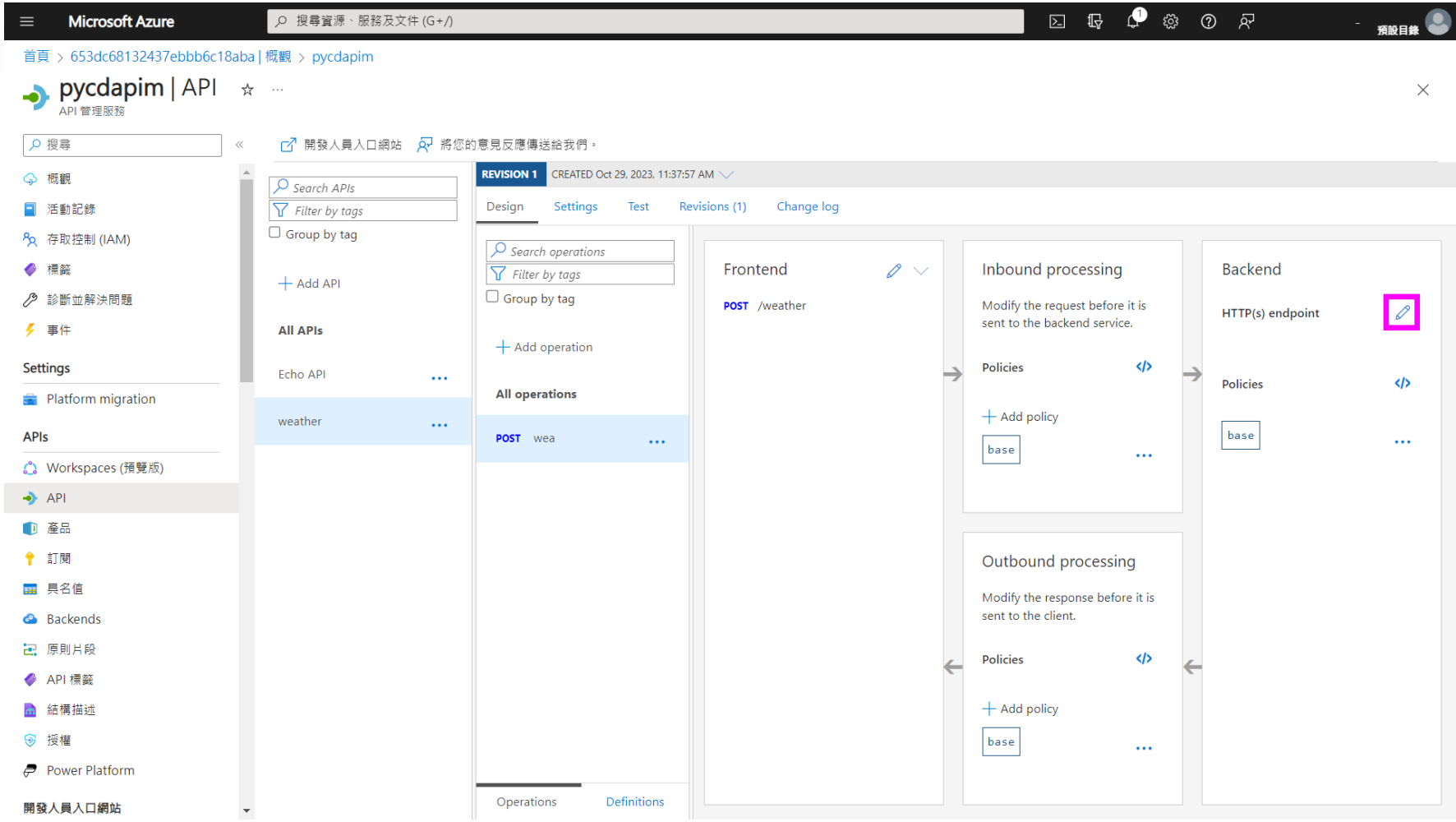
2. 綁定 API



© 2026 Enos Chou

Azure APIM

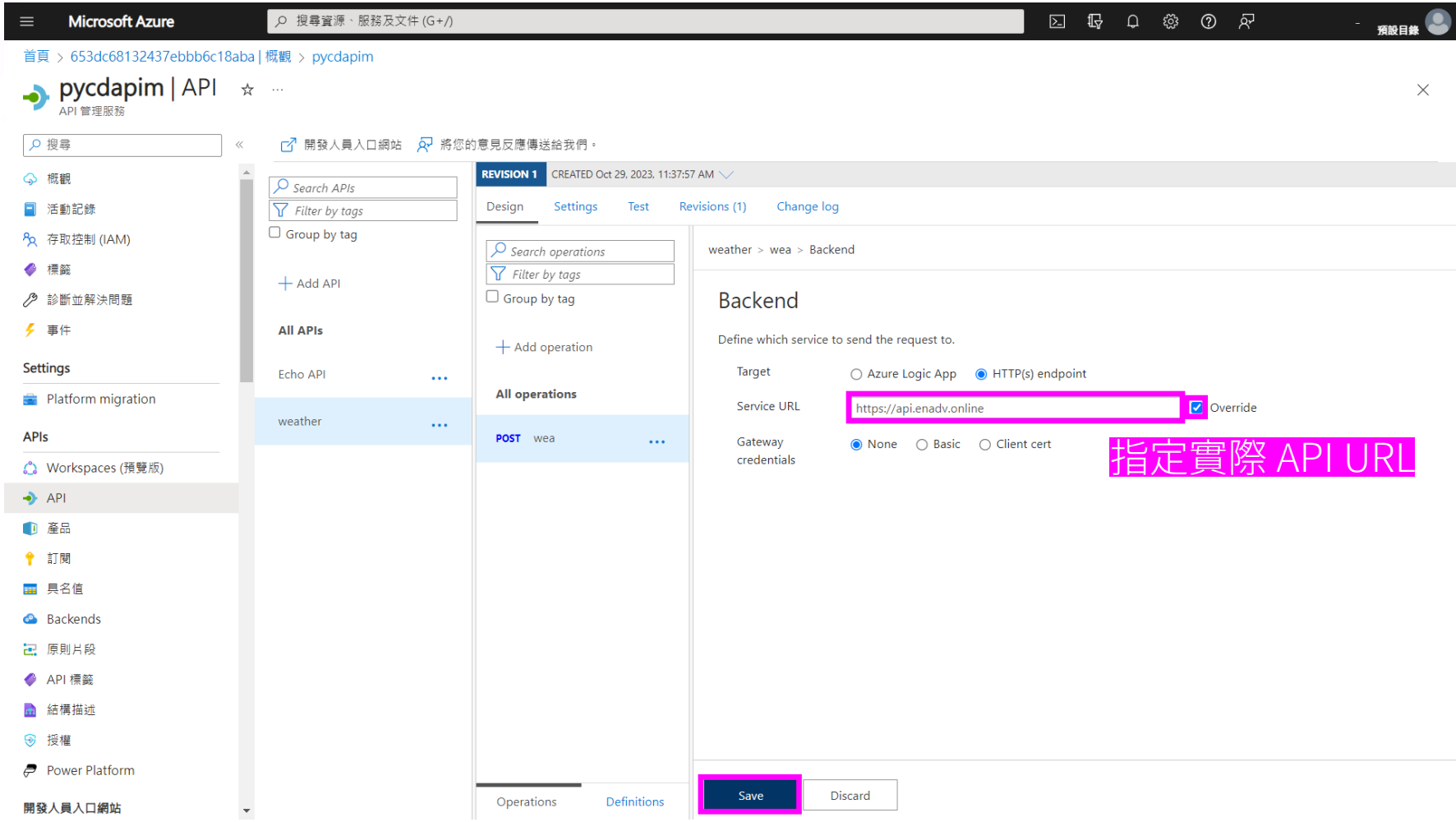
2. 綁定 API



© 2026 Enos Chou

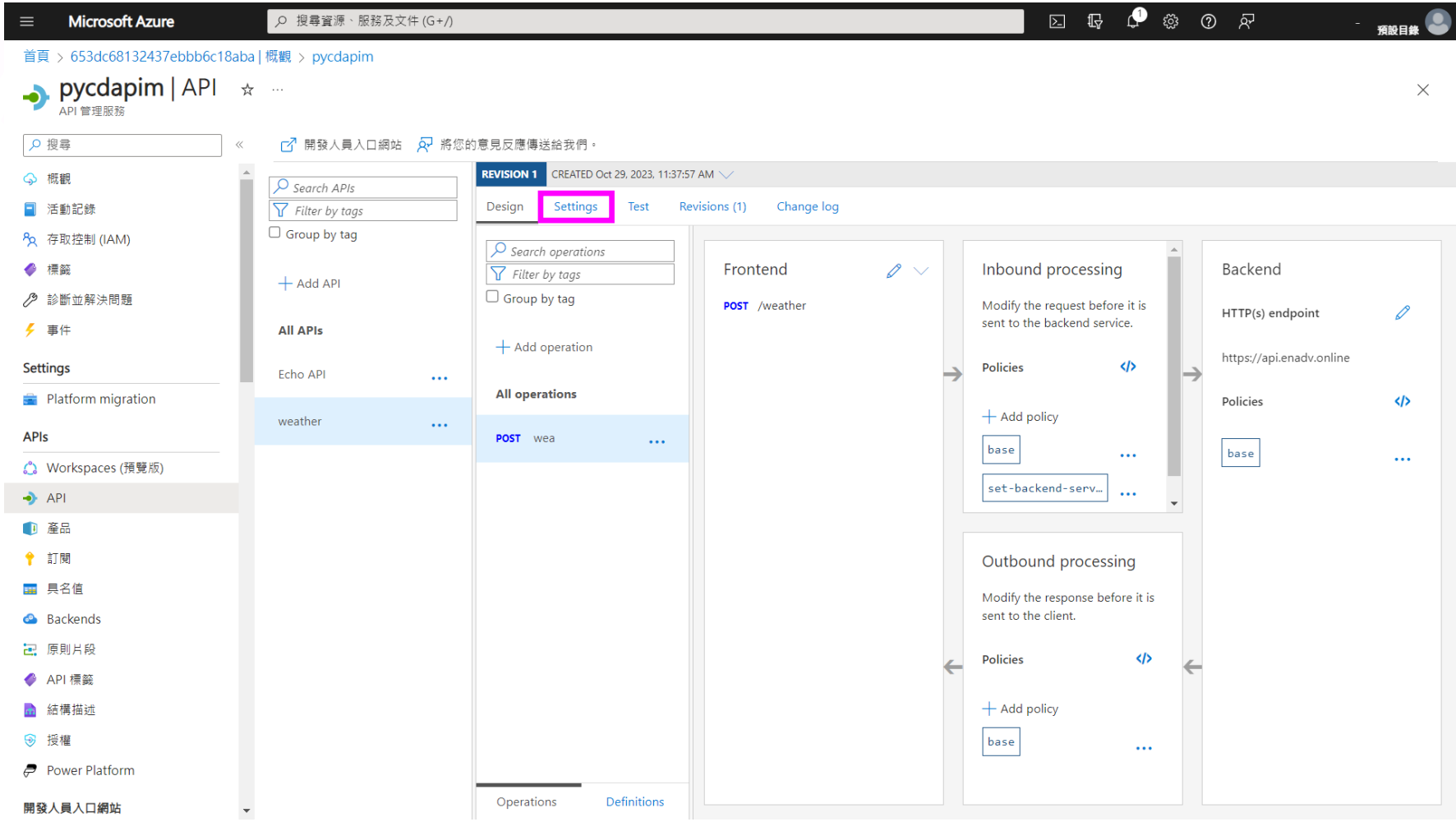
Azure APIM

2. 綁定 API



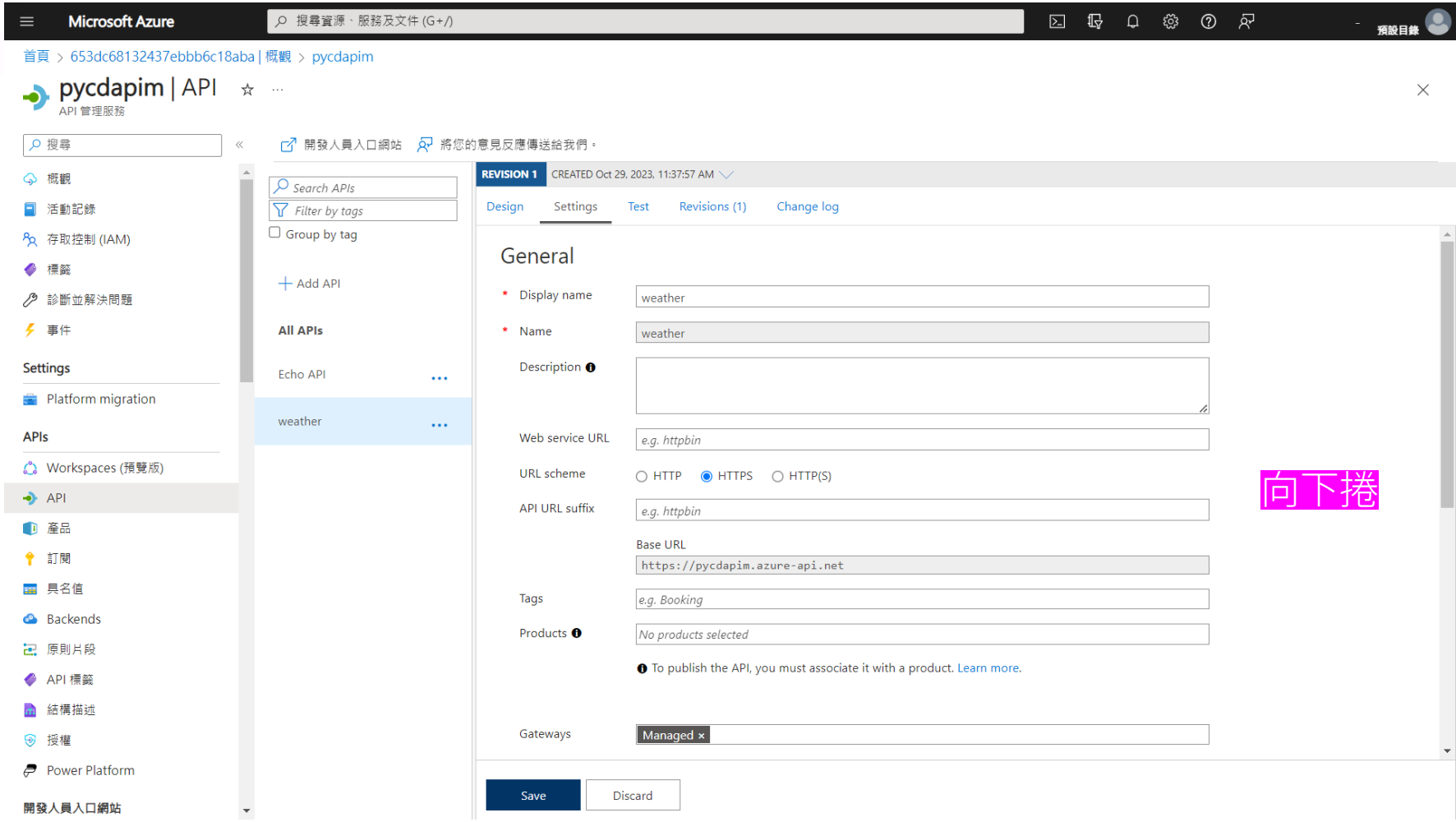
Azure APIM

2. 綁定 API



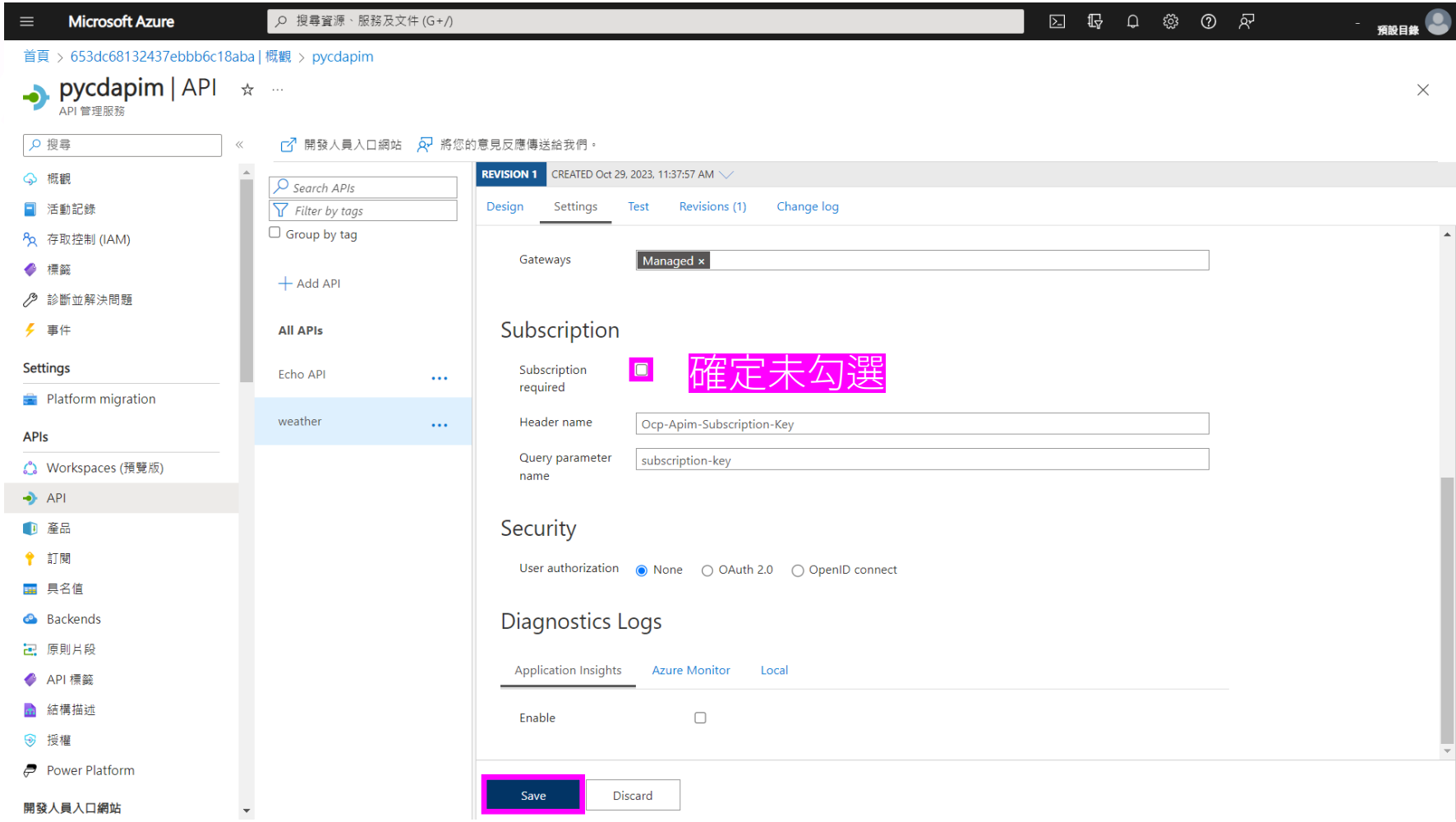
Azure APIM

2. 綁定 API



Azure APIM

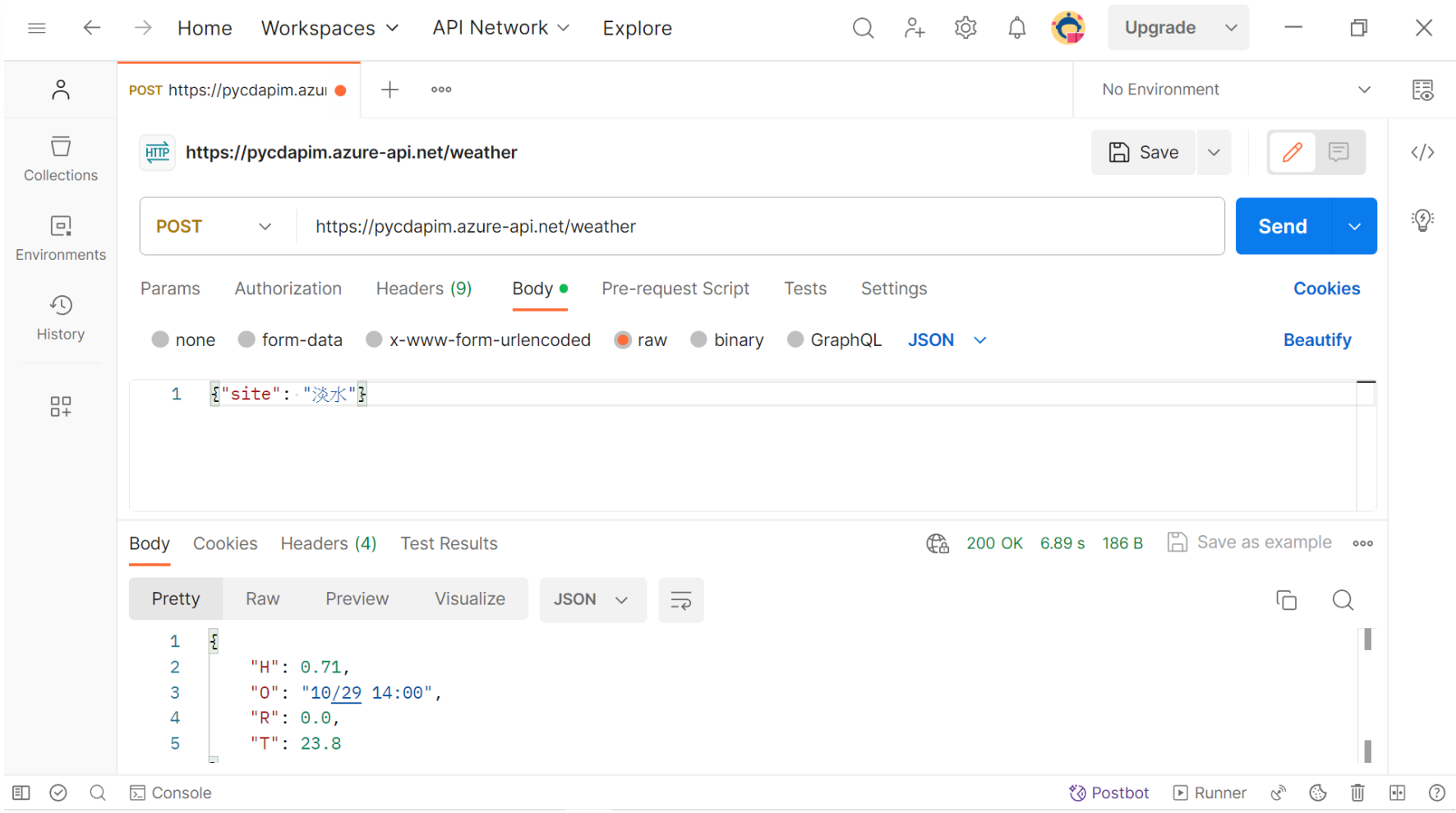
2. 綁定 API



Azure APIM

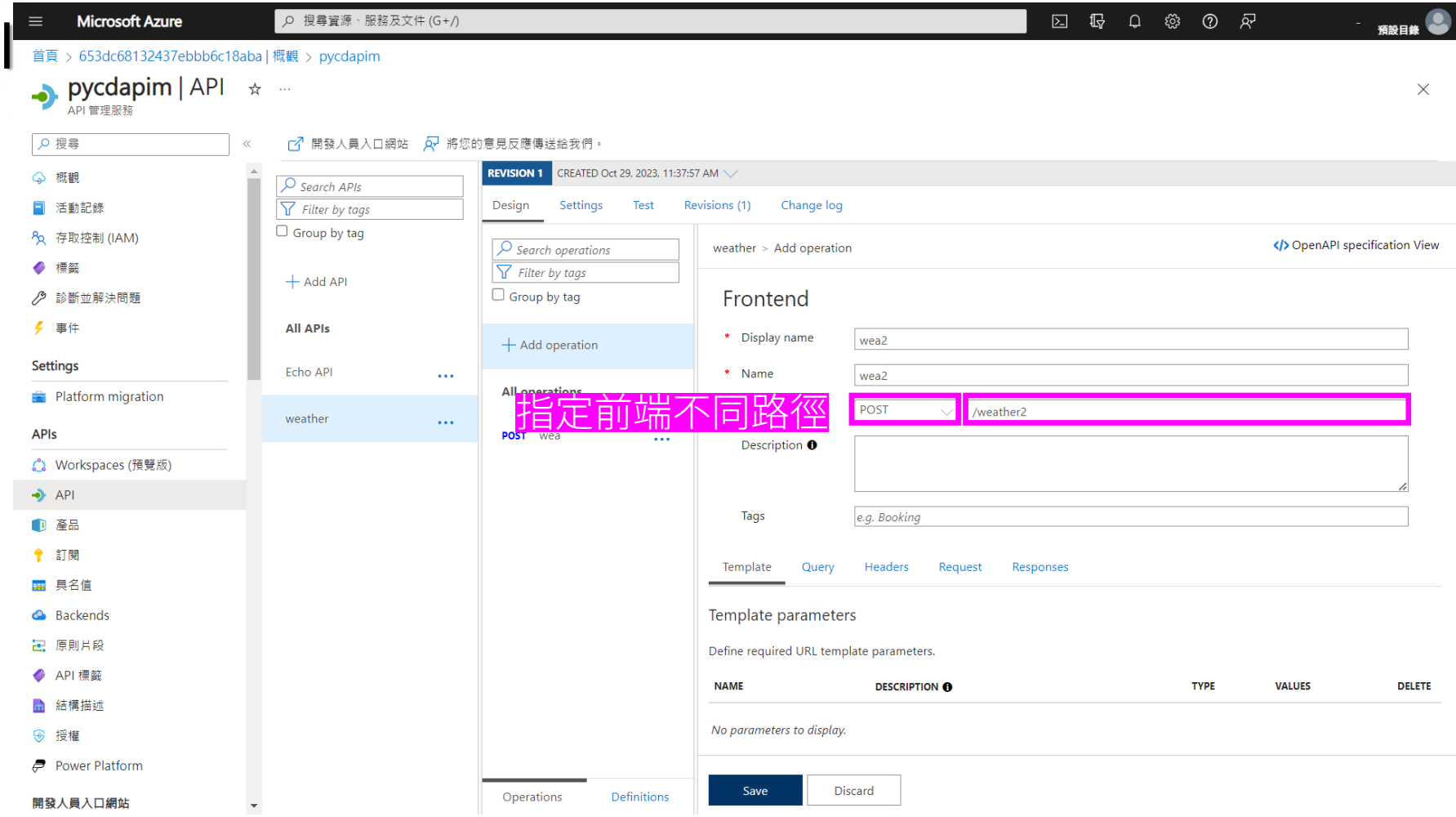
3. 測試 API

a. by Postman



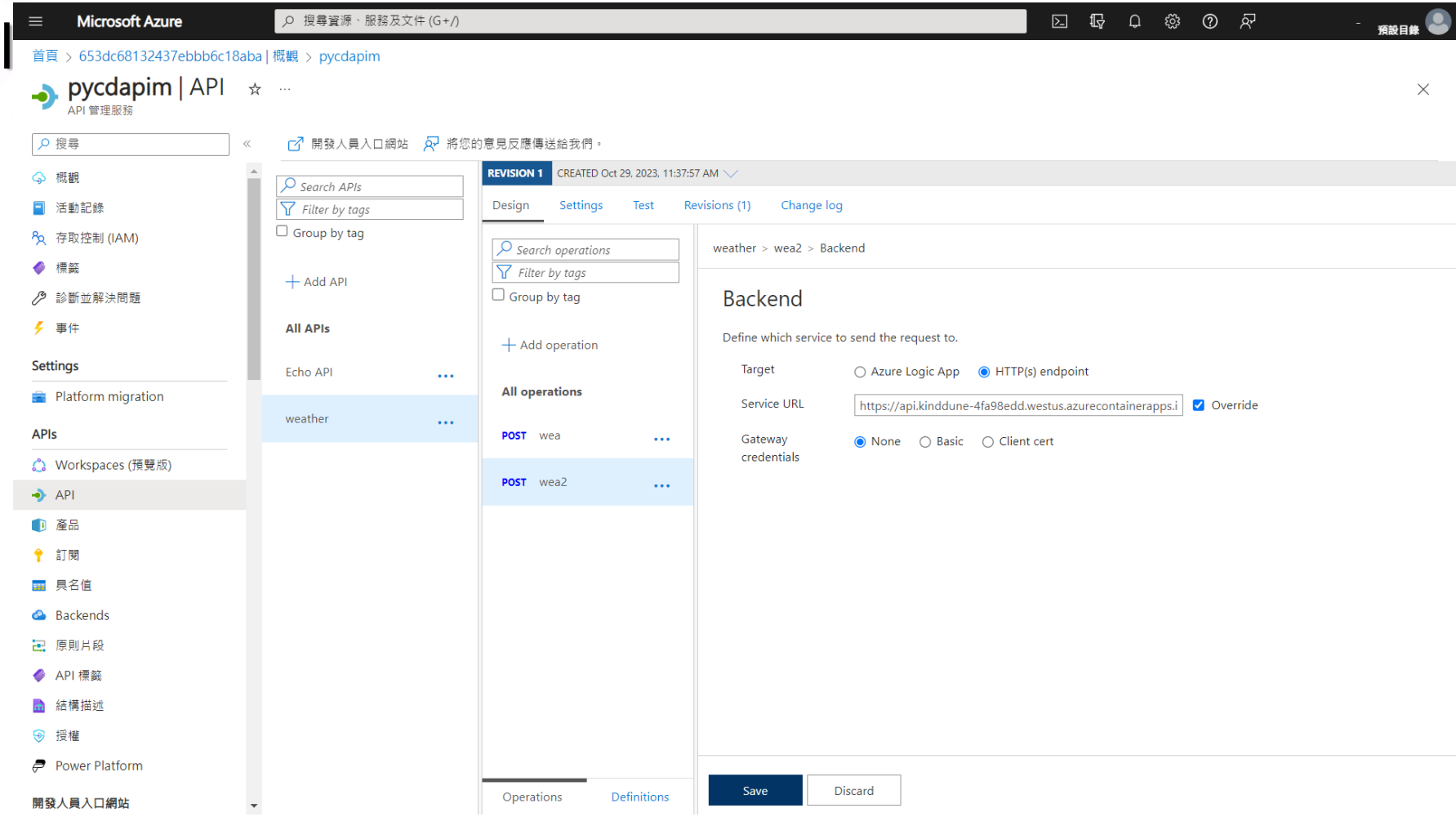
Azure APIM

4. 綁定同路徑 API



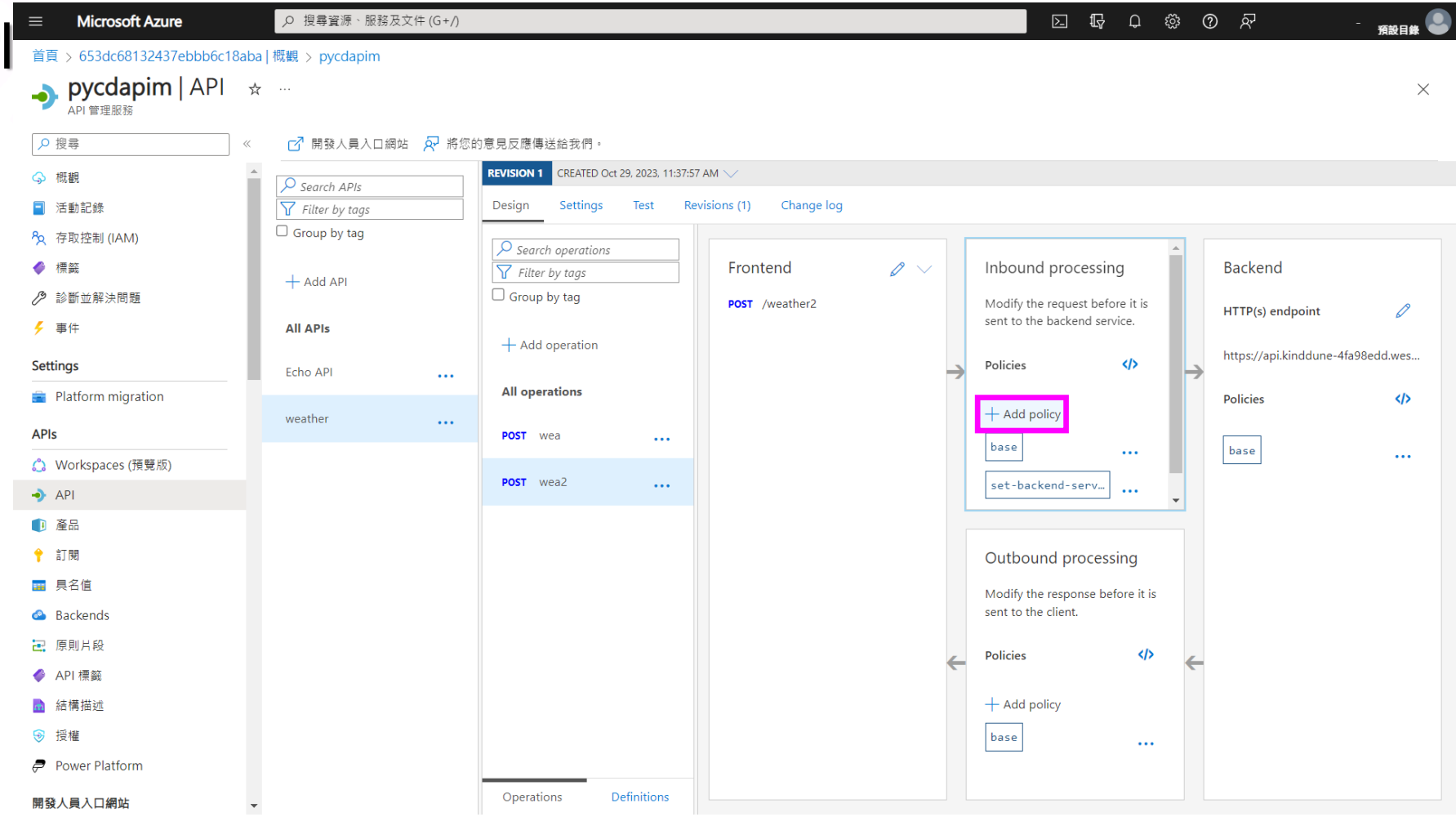
Azure APIM

4. 綁定同路徑 API



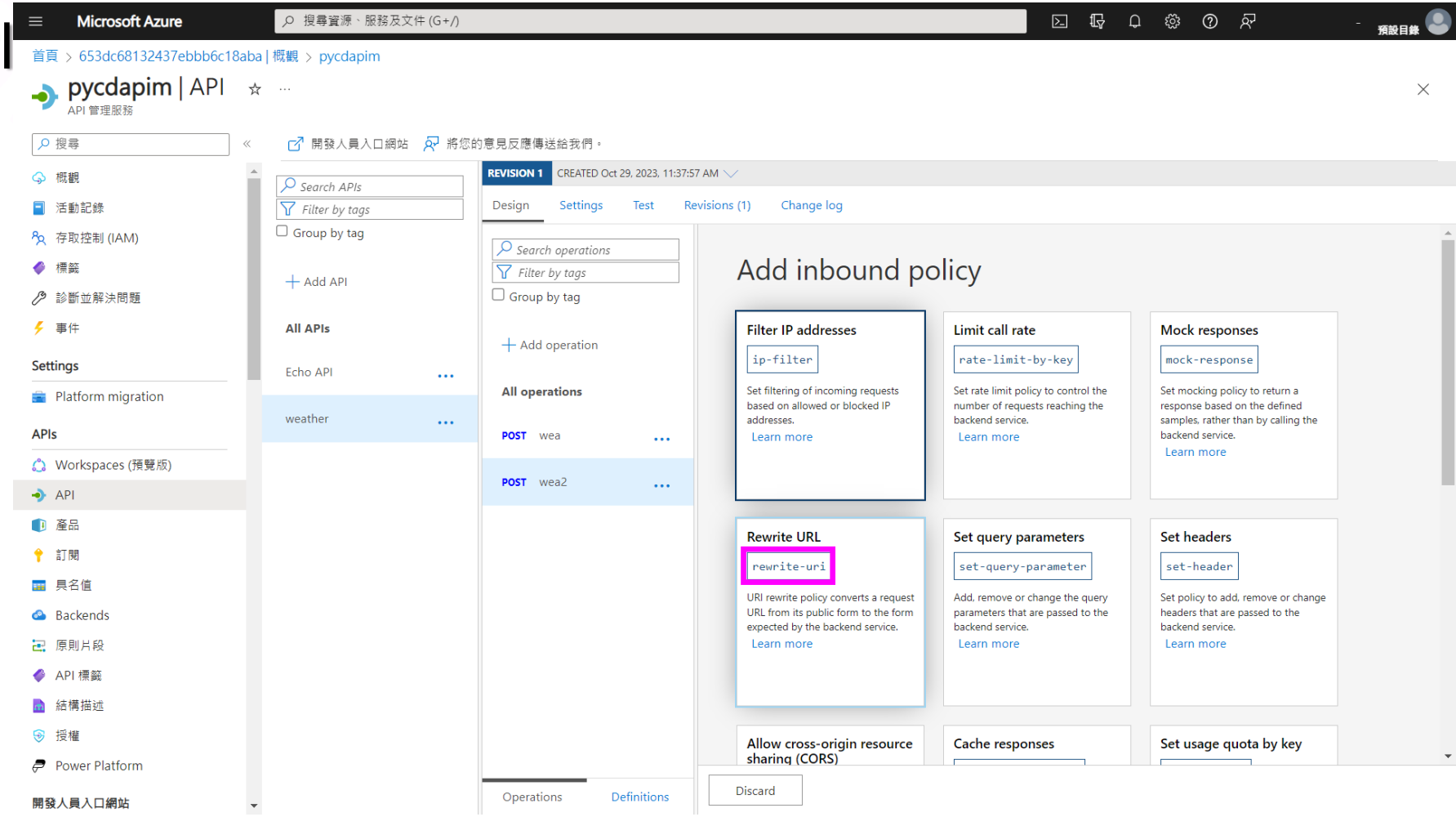
Azure APIM

4. 綁定同路徑 API



Azure APIM

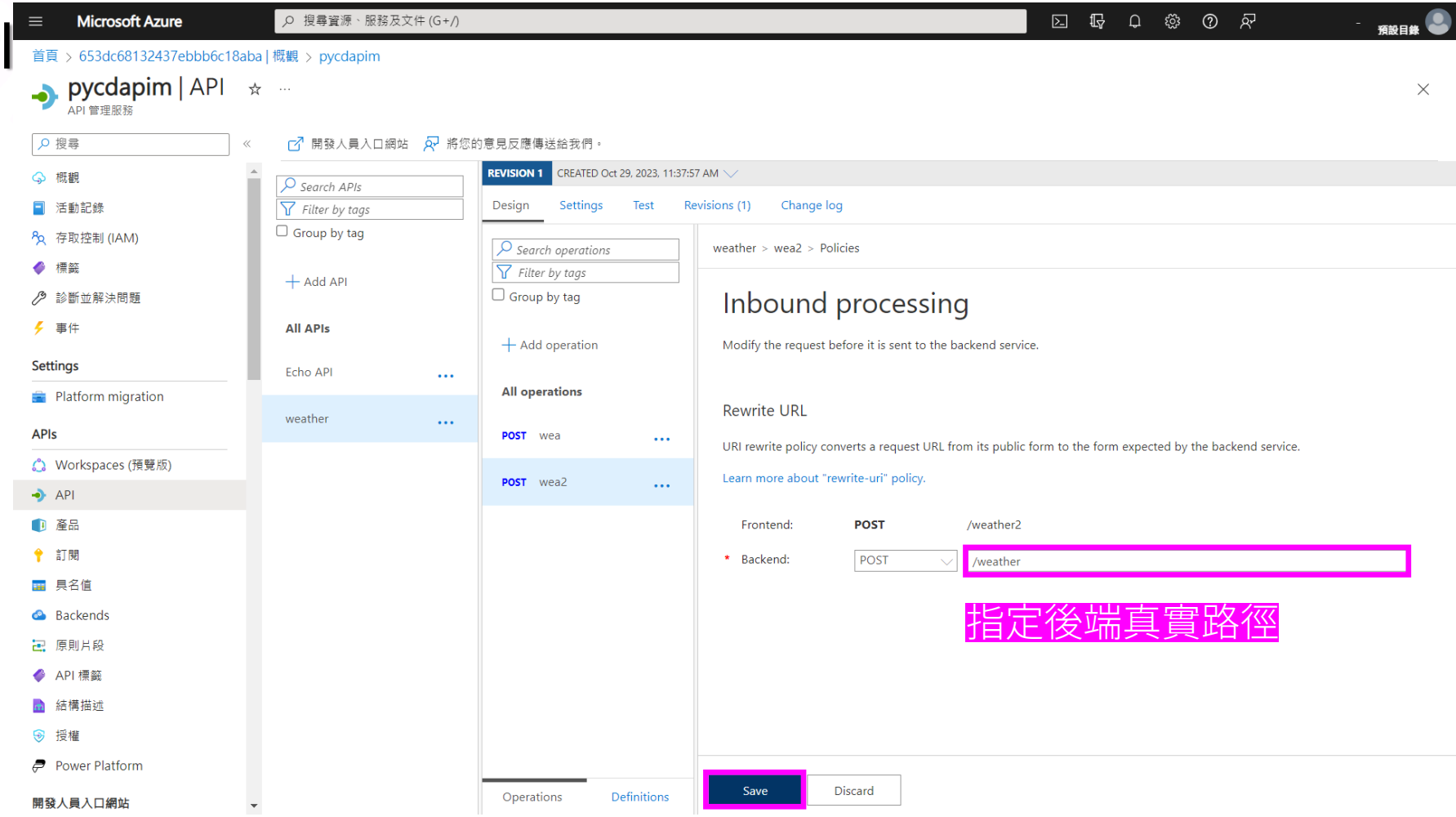
4. 綁定同路徑 API



© 2026 Enos Chou

Azure APIM

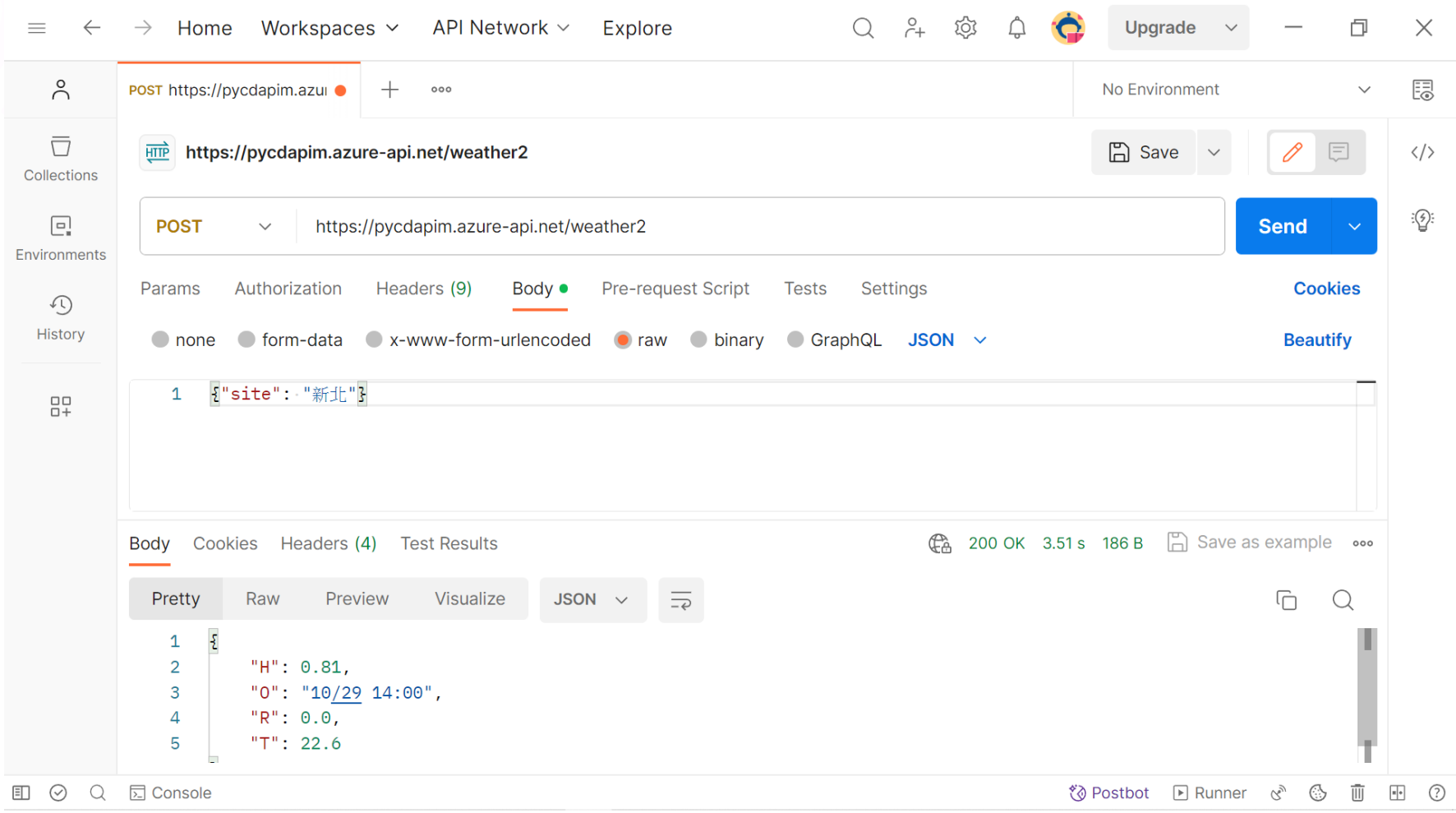
4. 綁定同路徑 API



Azure APIM

5. 測試 API

a. by Postman



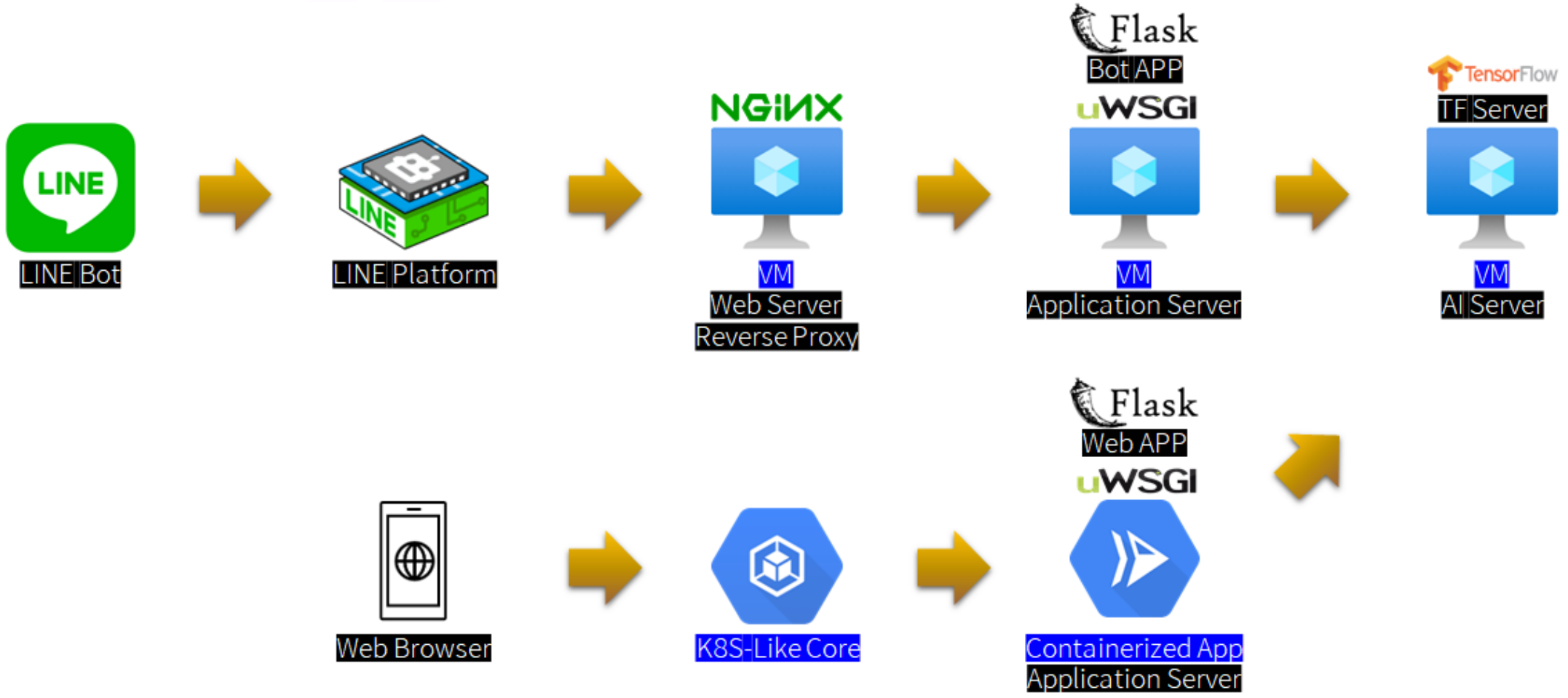
© 2026 Enos Chou

© 2026 Enos Chou

AI 模型部署

TensorFlow Models ft. TensorFlow Server

Why Standalone AI?



TensorFlow Server ft. VM

1. 建立模型

in TensorFlow SavedModel Format

```
./assets
./saved_model.pb
./variables/variables.data-00000-of-00001
./variables/variables.index
```

© 2026 Enos Chou

TensorFlow Server ft. VM

2. 建立 VM

with SPEC

```
OS:           Ubuntu 24.04 LTS
RAM:          1GB (or more)
Address:      Static IP Address
PORT:         TCP 8500, 8501
```

© 2026 Enos Chou

TensorFlow Server ft. VM

3. 上傳模型

into the Path

```
/home/your_account/your_model/1/assets
/home/your_account/your_model/1/saved_model.pb
/home/your_account/your_model/1/variables/variables.data-00000-of-00001
/home/your_account/your_model/1/variables/variables.index
```

© 2026 Enos Chou

TensorFlow Server ft. VM

4. 安裝 TensorFlow Server

via the Command

```
echo "deb [arch=amd64] http://storage.googleapis.com/tensorflow-serving-apt stable tensorflow-model-server tensorflow-model-server-universal" | sudo tee /etc/apt/sources.list.d/tensorflow-serving.list; curl https://storage.googleapis.com/tensorflow-serving-apt/tensorflow-serving.release.pub.gpg | sudo apt-key add -; sudo apt update; sudo apt install tensorflow-model-server
```

Note

系統不須安裝其他 TensorFlow 套件

© 2026 Enos Chou

TensorFlow Server ft. VM

5. 啟動 TensorFlow Server

a. via the Command in the Foreground for RESTful API & gRPC

```
tensorflow_model_server --model_base_path=your_model_path --
model_name=your_model --rest_api_port=your_rest_port --
port=your_grpc_port
tensorflow_model_server --model_base_path=/home/eng/trees --
model_name=trees --rest_api_port=8501 --port=8500
```

Note

① `--rest_api_port` for RESTful API

② `--port` for gRPC

© 2026 Enos Chou

TensorFlow Server ft. VM

5. 啟動 TensorFlow Server

b. via the Command in the Foreground for RESTful API only

```
tensorflow_model_server --model_base_path=your_model_path --
model_name=your_model --rest_api_port=your_rest_port --port=0 --
grpc_socket_path=$HOME/sk
tensorflow_model_server --model_base_path=/home/eng/trees --
model_name=trees --rest_api_port=8501 --port=0 --
grpc_socket_path=$HOME/sk
```

Note

① `--rest_api_port` for RESTful API

② `--port` for gRPC

© 2026 Enos Chou

TensorFlow Server ft. VM

5. 啟動 TensorFlow Server

c. via the Command in the Background for RESTful API & gRPC

```
nohup tensorflow_model_server --model_base_path=your_model_path --
model_name=your_model --rest_api_port=your_rest_port --
port=your_grpc_port &
nohup tensorflow_model_server --model_base_path=/home/eng/trees --
model_name=trees --rest_api_port=8501 --port=8500 &
```

Note

① `--rest_api_port` for RESTful API

② `--port` for gRPC

③ `nohup ... &` for 背景執行

© 2026 Enos Chou

TensorFlow Server ft. VM

6. (部署多模型)

a. via Configuration File

```
model_config_list {
  config {
    name: 'your_model'
    base_path: 'your_model_path'
    model_platform: 'tensorflow'
  }
}
```

```
# 同時啟動兩個模型的參數檔範例 models.conf

model_config_list {
  config {
    name: 'trees'
    base_path: '/home/eng/leaves4/'
    model_platform: 'tensorflow'
  }
  config {
    name: 'trees2'
    base_path: '/home/eng/trees17/'
    model_platform: 'tensorflow'
  }
}
```

© 2026 Enos Chou

TensorFlow Server ft. VM

6. (部署多模型)

b. Activate the Configuration

```
nohup tensorflow_model_server --model_config_file=your_conf --
rest_api_port=your_rest_port --port=your_grpc_port &
nohup tensorflow_model_server --model_config_file=models.conf --
rest_api_port=8501 --port=8500 &
```

Note

新增 --model_config_file 指定參數檔，取代 --model_base_path 與 --model_name

© 2026 Enos Chou

TensorFlow Server ft. Containerized App

1. 建立模型

in TensorFlow SavedModel Format

```
./1/assets
./1/saved_model.pb
./1/variables/variables.data-00000-of-00001
./1/variables/variables.index
```

© 2026 Enos Chou

TensorFlow Server ft. Containerized App

2. 製作 TensorFlow Server 容器映像檔

a. Dockerfile for RESTful API

```
FROM tensorflow/serving:2.19.0
COPY 1/ /models/your_model/1
ENTRYPOINT tensorflow_model_server --rest_api_port=${PORT:-8501} --
model_name=your_model --model_base_path=/models/your_model --port=0 --
grpc_socket_path=/sk
```

© 2026 Enos Chou

TensorFlow Server ft. Containerized App

2. 製作 TensorFlow Server 容器映像檔

b. Dockerfile for gRPC

```
FROM tensorflow/serving:2.19.0
COPY 1/ /models/your_model/1
ENTRYPOINT tensorflow_model_server --port=${PORT:-8500} --
model_name=your_model --model_base_path=/models/your_model
```

© 2026 Enos Chou

TensorFlow Server ft. Containerized App

2. 製作 TensorFlow Server 容器映像檔

c. Build Docker Image

```
docker build -t your_repo:your_tag -f your_Dockerfile .
docker build -t leaves4:v1 -f Dockerfile.rest .
```

© 2026 Enos Chou

TensorFlow Server ft. Containerized App

3. 推送 TensorFlow Server 容器映像檔

    a. Build Container Registry

    b. Push Docker Image into Container Registry

© 2026 Enos Chou

TensorFlow Server ft. Containerized App

4. 部署 Containerized App

with SPEC

```
RAM:          1GB (or more)
PORT:         8500 or 8501
```

© 2026 Enos Chou

TensorFlow Serving Experience

1. TensorFlow Serving via RESTful API

a. 建立 Python 環境

```
# dependencies
requests
pillow
numpy
```

© 2026 Enos Chou

TensorFlow Serving Experience

1. TensorFlow Serving via RESTful API

b. Python 測試

```
# load image
from PIL import Image, ImageOps
import numpy as np

image = Image.open(your_pic)
image = ImageOps.fit(image, (224, 224)) # customize if necessary
image = np.array(image).astype(np.float32) / 127.5 - 1 # customize if necessary
image = image.reshape((1, *image.shape))
```

© 2026 Enos Chou

TensorFlow Serving Experience

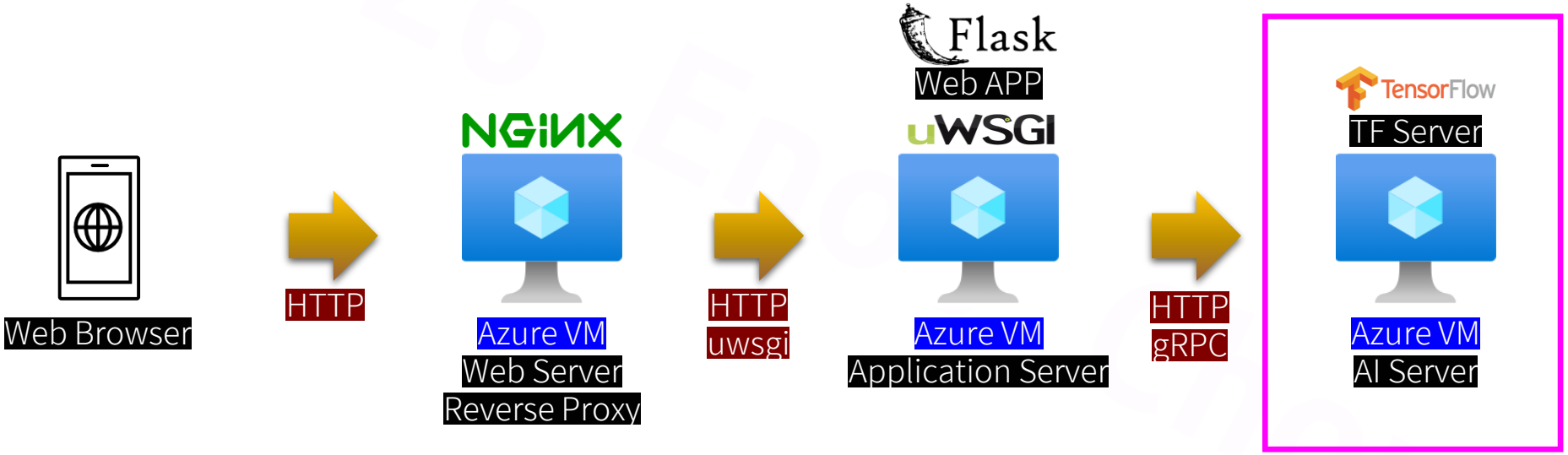
1. TensorFlow Serving via RESTful API

b. Python 測試

```
# inference
import requests
import numpy as np

r = requests.post('http://your_host:your_rest_port/v1/models/your_model:predict',
                  json={'instances': image.tolist()})
print(np.argmax(r.json()['predictions']))
```

© 2026 Enos Chou

TensorFlow Server in Azure ft. VM



© 2026 Enos Chou

TensorFlow Server in Azure ft. VM

1. 建立模型

in TensorFlow SavedModel Format

```
./assets
./saved_model.pb
./variables/variables.data-00000-of-00001
./variables/variables.index
```

© 2026 Enos Chou

TensorFlow Server in Azure ft. VM

1. 建立 Azure VM

a. 建立 VM

```
OS:    Ubuntu 24.04 LTS
RAM:   1GB
```

© 2026 Enos Chou

TensorFlow Server in Azure ft. VM

2. 建立 VM

with SPEC

```
OS:          Ubuntu 24.04 LTS
RAM:         1GB (or more)
Address:     Static IP Address
PORT:        TCP 8500, 8501
```

© 2026 Enos Chou

TensorFlow Server in Azure ft. VM

2. 建立 VM

開通防火牆

```
# PORT:                 TCP 8500, 8501
```

虛擬機器 > 進入指定 VM > 網路 > 網路設定 > + 建立連接埠規則 > 輸入連接埠規則 >

目的地：　　指定 VM 的 "私人 IP 位址"
目的地連接埠範圍：　　"8500, 8501" "TCP"
名稱：　　"自行命名" >

新增

© 2026 Enos Chou

TensorFlow Server in Azure ft. VM

3. 上傳模型

into the Path

```
/home/your_account/your_model/1/assets
/home/your_account/your_model/1/saved_model.pb
/home/your_account/your_model/1/variables/variables.data-00000-of-00001
/home/your_account/your_model/1/variables/variables.index
```

© 2026 Enos Chou

TensorFlow Server in Azure ft. VM

4. 安裝 TensorFlow Server

via the Command

```
echo "deb [arch=amd64] http://storage.googleapis.com/tensorflow-serving-apt stable tensorflow-model-server tensorflow-model-server-universal" | sudo tee /etc/apt/sources.list.d/tensorflow-serving.list; curl https://storage.googleapis.com/tensorflow-serving-apt/tensorflow-serving.release.pub.gpg | sudo apt-key add -; sudo apt update; sudo apt install tensorflow-model-server
```

Note

系統不須安裝其他 TensorFlow 套件

© 2026 Enos Chou

TensorFlow Server in Azure ft. VM

5. 啟動 TensorFlow Server

a. via the Command in the Foreground for RESTful API & gRPC

```
tensorflow_model_server --model_base_path=your_model_path --
model_name=your_model --rest_api_port=your_rest_port --
port=your_grpc_port
tensorflow_model_server --model_base_path=/home/eng/trees --
model_name=trees --rest_api_port=8501 --port=8500
```

Note

① `--rest_api_port` for RESTful API

② `--port` for gRPC

© 2026 Enos Chou

TensorFlow Server in Azure ft. VM

5. 啟動 TensorFlow Server

b. via the Command in the Foreground for RESTful API only

```
tensorflow_model_server --model_base_path=your_model_path --
model_name=your_model --rest_api_port=your_rest_port --port=0 --
grpc_socket_path=$HOME/sk
tensorflow_model_server --model_base_path=/home/eng/trees --
model_name=trees --rest_api_port=8501 --port=0 --
grpc_socket_path=$HOME/sk
```

Note

① `--rest_api_port` for RESTful API

② `--port` for gRPC

© 2026 Enos Chou

TensorFlow Server in Azure ft. VM

5. 啟動 TensorFlow Server

c. via the Command in the Background for RESTful API & gRPC

```
nohup tensorflow_model_server --model_base_path=your_model_path --
model_name=your_model --rest_api_port=your_rest_port --
port=your_grpc_port &
nohup tensorflow_model_server --model_base_path=/home/eng/trees --
model_name=trees --rest_api_port=8501 --port=8500 &
```

Note

① `--rest_api_port` for RESTful API

② `--port` for gRPC

③ `nohup ... &` for 背景執行

© 2026 Enos Chou

TensorFlow Server in Azure ft. VM

6. (部署多模型)

a. via Configuration File

```
model_config_list {
  config {
    name: 'your_model'
    base_path: 'your_model_path'
    model_platform: 'tensorflow'
  }
}
```

```
# 同時啟動兩個模型的參數檔範例 models.conf

model_config_list {
  config {
    name: 'trees'
    base_path: '/home/eng/leaves4/'
    model_platform: 'tensorflow'
  }
  config {
    name: 'trees2'
    base_path: '/home/eng/trees17/'
    model_platform: 'tensorflow'
  }
}
```

© 2026 Enos Chou

TensorFlow Server in Azure ft. VM

6. (部署多模型)

b. Enable the Configuration

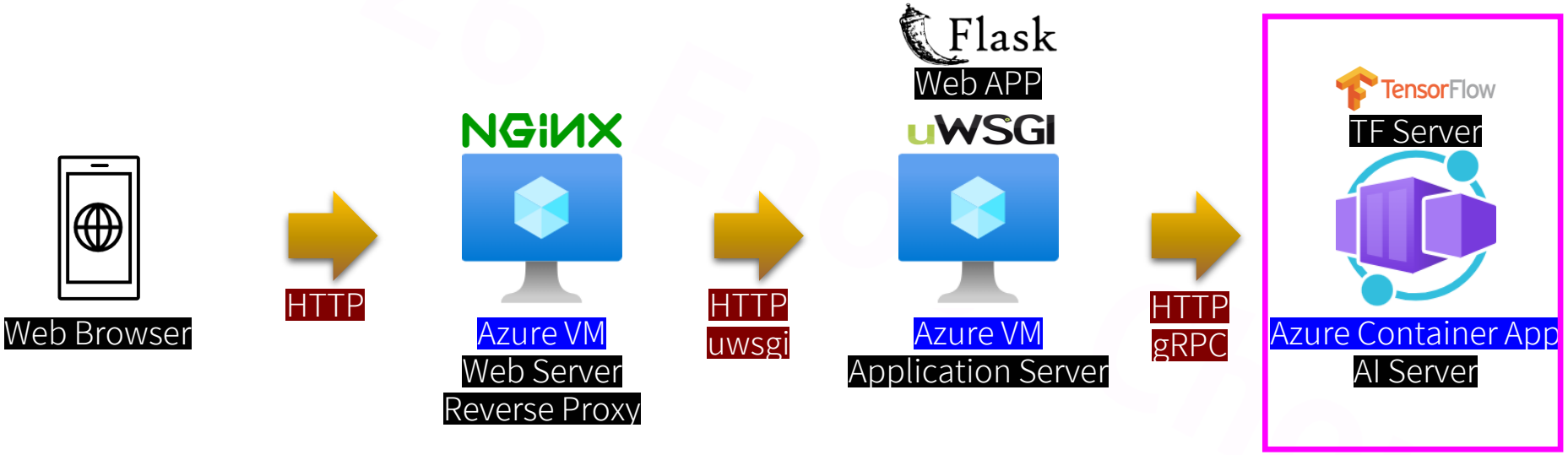
```
nohup tensorflow_model_server --model_config_file=your_conf --
rest_api_port=your_rest_port --port=your_grpc_port &
nohup tensorflow_model_server --model_config_file=models.conf --
rest_api_port=8501 --port=8500 &
```

Note

新增 --model_config_file 指定參數檔，取代 --model_base_path 與 --model_name

© 2026 Enos Chou

TensorFlow Server in Azure ft. Container App



© 2026 Enos Chou

TensorFlow Server in Azure ft. Container App

1. 建立模型

in TensorFlow SavedModel Format

```
./1/assets
./1/saved_model.pb
./1/variables/variables.data-00000-of-00001
./1/variables/variables.index
```

© 2026 Enos Chou

TensorFlow Server in Azure ft. Container App

2. 製作 TensorFlow Server 容器映像檔

a. Dockerfile for RESTful API

```
FROM tensorflow/serving:2.19.0
COPY 1/ /models/your_model/1
ENTRYPOINT tensorflow_model_server --rest_api_port=${PORT:-8501} --
model_name=your_model --model_base_path=/models/your_model --port=0 --
grpc_socket_path=/sk
```

© 2026 Enos Chou

TensorFlow Server in Azure ft. Container App

2. 製作 TensorFlow Server 容器映像檔

b. Dockerfile for gRPC

```
FROM tensorflow/serving:2.19.0
COPY 1/ /models/your_model/1
ENTRYPOINT tensorflow_model_server --port=${PORT:-8500} --
model_name=your_model --model_base_path=/models/your_model
```

© 2026 Enos Chou

TensorFlow Server in Azure ft. Container App

2. 製作 TensorFlow Server 容器映像檔

c. Build Docker Image

```
docker build -t your_repo:your_tag -f your_Dockerfile .
docker build -t leaves4:v1 -f Dockerfile.rest .
```

© 2026 Enos Chou

TensorFlow Server in Azure ft. Container App

3. 推送 TensorFlow Server 容器映像檔
    a. Build Container Registry
    b. Push Docker Image into Container Registry

© 2026 Enos Chou

TensorFlow Server in Azure ft. Container App

4. 部署 Containerized App

with SPEC

```
RAM:          1GB (or more)
PORT:         8500 or 8501
```

© 2026 Enos Chou

TensorFlow Serving Experience

1. TensorFlow Serving via RESTful API

a. 建立 Python 環境

```
# dependencies
requests
pillow
numpy
```

© 2026 Enos Chou

TensorFlow Serving Experience

1. TensorFlow Serving via RESTful API

b. Python 測試

```
# load image
from PIL import Image, ImageOps
import numpy as np

image = Image.open(your_pic)
image = ImageOps.fit(image, (224, 224))  # customize if necessary
image = np.array(image).astype(np.float32) / 127.5 - 1  # customize if necessary
image = image.reshape((1, *image.shape))
```

© 2026 Enos Chou

TensorFlow Serving Experience

1. TensorFlow Serving via RESTful API

b. Python 測試

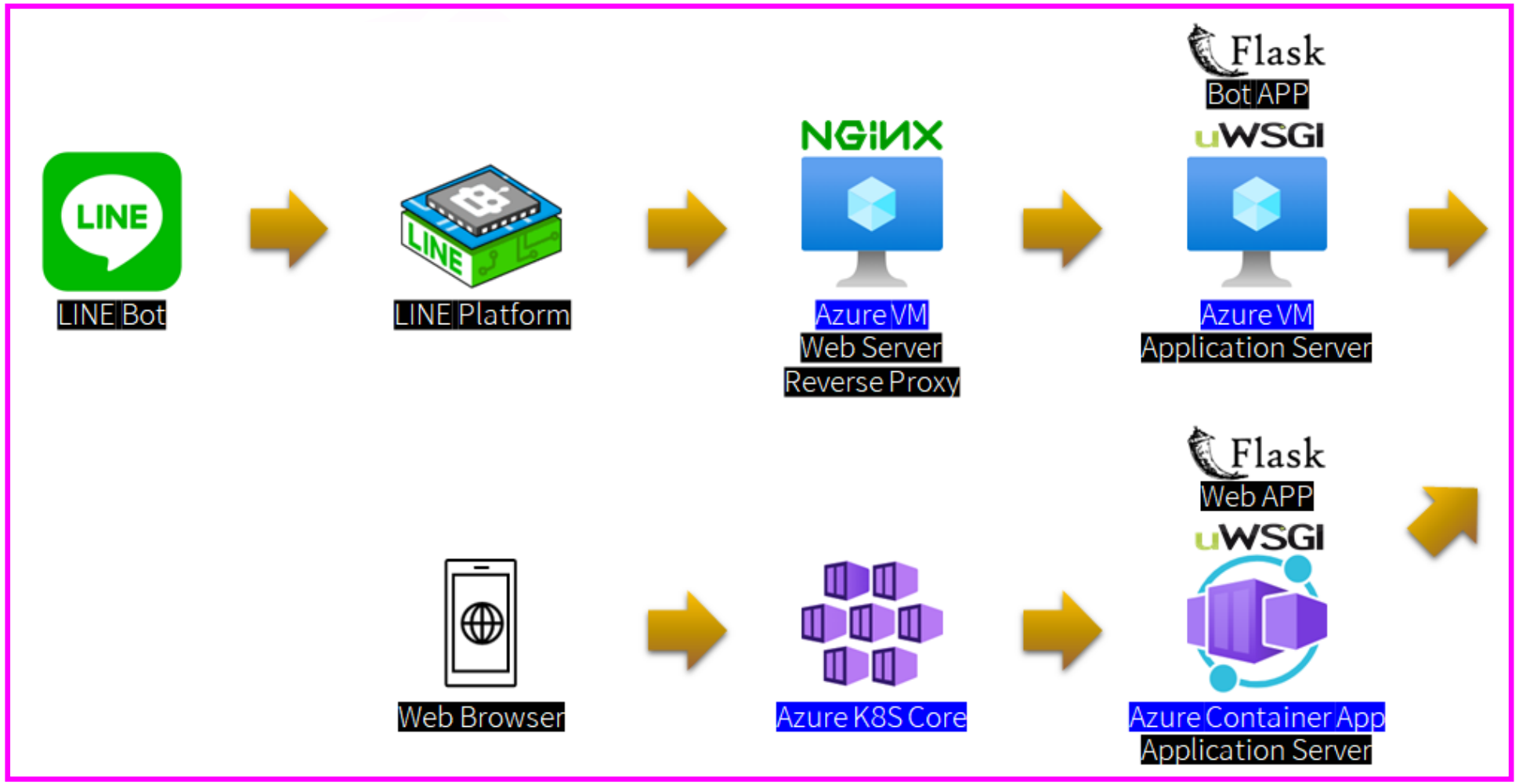
```
# inference
import requests
import numpy as np

r = requests.post('http://your_host:your_rest_port/v1/models/your_model:predict',
                  json={'instances': image.tolist()})
print(np.argmax(r.json()['predictions']))
```

© 2026 Enos Chou

AI Apps ft. TensorFlow Serving

© 2026 Enos Chou

AI Apps ft. TensorFlow Serving



© 2026 Enos Chou

AI Apps ft. TensorFlow Serving

1. Web

    a. source

```
treesweb.py
treeset_labels.txt
env.json (MODEL_NAME, REST_SSL, REST_HOST, REST_PORT)
*.jpg
```

© 2026 Enos Chou

AI Apps ft. TensorFlow Serving

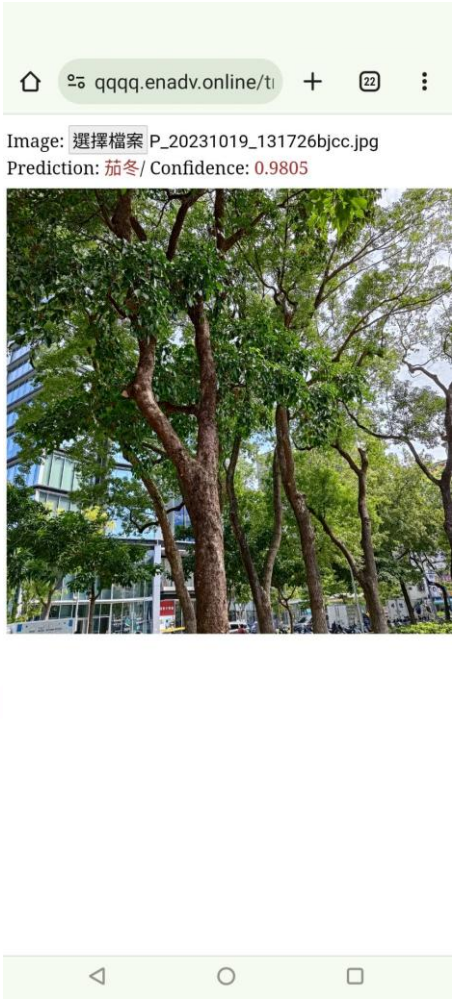
1. Web

   b. requirements.txt

```
numpy
Flask
pillow
requests
```

   c. URL

```
https://your_domain/treesr
```



© 2026 Enos Chou

Your Turn - TFS Bot

以 LINE Bot 叫用 TensorFlow Serving 的方式建立影像辨識 LINE Bot，辨識輸入照片回應名稱 (30 mins)

© 2026 Enos Chou

AI Apps ft. TensorFlow Serving

2. LINE Bot

a. source

```
trees17bot.py
treeset_labels.txt
env.json (MODE: REST, CHANNEL_ACCESS_TOKEN,
          CHANNEL_SECRET, MODEL_NAME, REST_SSL,
          REST_HOST, REST_PORT)
*.jpg
```

© 2026 Enos Chou

AI Apps ft. TensorFlow Serving

2. LINE Bot

b. requirements.txt

```
line-bot-sdk
numpy
Flask
pillow
requests
```

c. Webhook URL

```
https://your_domain/callback
```

© 2026 Enos Chou

© 2026 Enos Chou

LLM 部署

Gemma 4

Gemma 4

© 2026 Enos Chou

Gemma 4

官方版 Gemma 4 系列記憶體佔用

	BFP16 (16 bits)	SFP8 (8 bits)	Q4_0 (4 bits)
Gemma 4 E2B	11.4 GB	5.7 GB	2.9 GB
Gemma 4 E4B	17.9 GB	8.9 GB	4.5 GB
Gemma 4 12B	26.7 GB	13.4 GB	6.7 GB
Gemma 4 26B A4B	57.7 GB	28.8 GB	14.4 GB
Gemma 4 31B	69.9 GB	34.9 GB	17.5 GB

© 2026 Enos Chou

Gemma 4

Gemma 4 系列調教與優化版本

https://huggingface.co/collections/google/gemma-4
https://huggingface.co/collections/unsloth/gemma-4

Gemma 4 E2B 官方調教	google/gemma-4-E2B-it	10.23 GB
Gemma 4 E2B 社群量化	unsloth/gemma-4-E2B-it-Q4_K_M.gguf	3.11 GB
Gemma 4 E4B 官方調教	google/gemma-4-E4B-it	16.03 GB
Gemma 4 E4B 社群量化	unsloth/gemma-4-E4B-it-Q4_K_M.gguf	4.98 GB

© 2026 Enos Chou

Gemma 4

本機啟動 Gemma 4

LM Studio

圖形化介面測試環境，也可以啟動 llama-server
https://lmstudio.ai/

Ollama

https://ollama.com/

llama-server

終端機介面伺服器
https://github.com/ggml-org/llama.cpp/releases > Assets

© 2026 Enos Chou

Gemma 4 ft. llama-server

CPU 啟動

Windows ft. PowerShell

```
.\llama-server.exe -m gemma-4-E4B-it-Q4_K_M.gguf --port 8080 -c 8192
```

NOTE
-c 上下文長度
--reasoning off (關閉思考)

© 2026 Enos Chou

Gemma 4 ft. llama-server

CPU 啟動多模態

Windows ft. PowerShell

```
.\llama-server.exe -m gemma-4-E4B-it-Q4_K_M.gguf --port 8080 -c 4096 --mmproj mmproj-F16.gguf
```

NOTE
`--mmproj` 多模態模型

© 2026 Enos Chou

Gemma 4 ft. llama-server

CPU 啟動多模態

Ubuntu 24.04

① 安裝系統套件

```
sudo apt update
sudo apt install libgomp1
```

© 2026 Enos Chou

Gemma 4 ft. llama-server

CPU 啟動多模態

Ubuntu 24.04

② 下載軟體與模型

```
wget https://github.com/ggml-org/llama.cpp/releases/download/b9333/llama-b9333-bin-ubuntu-x64.tar.gz
tar xf llama-b9333-bin-ubuntu-x64.tar.gz
wget https://huggingface.co/unsloth/gemma-4-E4B-it-GGUF/resolve/main/gemma-4-E4B-it-Q4_K_M.gguf
wget https://huggingface.co/unsloth/gemma-4-E4B-it-GGUF/blob/main/mmproj-F16.gguf
```

© 2026 Enos Chou

Gemma 4 ft. llama-server

CPU 啟動多模態

Ubuntu 24.04

③ 測試啟動 `llama-server`

```
cd llama-b9333
./llama-server -m ../gemma-4-E4B-it-Q4_K_M.gguf -c 4096 --mmproj ../mmproj-F16.gguf
```

© 2026 Enos Chou

Gemma 4 ft. llama-server

CPU 啟動多模態

Ubuntu 24.04

④ 正式啟動 `llama-server`

```
cd llama-b9333
nohup ./llama-server -m ../gemma-4-E4B-it-Q4_K_M.gguf -c 4096 --mmproj ../mmproj-F16.gguf --host 0.0.0.0 &
```

NOTE

```
--host 監聽主機位址
--port 監聽主機埠
nohup ... & (背景啟動)
```

© 2026 Enos Chou

Gemma 4 ft. llama-server

叫用模型 - image

by OpenAI Completions API

```
from openai import OpenAI

url = 'http://localhost:8080/v1'

openai_client = OpenAI(base_url=url, api_key='...')
```

© 2026 Enos Chou

Gemma 4 ft. llama-server

叫用模型 - image

by OpenAI Completions API

```
import base64

image_file = 'pic.jpg'
with open(image_file, 'rb') as f:
    image_base64 = base64.b64encode(f.read()).decode('utf-8')
```

© 2026 Enos Chou

Gemma 4 ft. llama-server

叫用模型 - image

by OpenAI Completions API

```
messages = [
    {
        'role': 'user',
        'content': [
            {
                'type': 'image_url',
                'image_url': {'url': f'{image_base64}'}
            }
        ]
    }
]
```

© 2026 Enos Chou

Gemma 4 ft. llama-server

叫用模型 - image

by OpenAI Completions API

```
response_schema = {
    "type": "object",
    "properties": {
        "plates": {
            "type": "array",
            "items": {"type": "string"},
            "description": "車牌號碼"
        }
    },
    "required": ["plates"]
}
```

© 2026 Enos Chou

Gemma 4 ft. llama-server

叫用模型 - image

by OpenAI Completions API

```
r = openai_client.chat.completions.create(
    model='...',
    messages=messages,
    response_format={
        'type': 'json_object',
        'schema': response_schema
    },
    temperature=0
)
```

© 2026 Enos Chou

Gemma 4 ft. llama-server

叫用模型 - image

by OpenAI Completions API

```
import json

print(json.loads(r.choices[0].message.content))
```

© 2026 Enos Chou

Gemma 4 ft. llama-server

叫用模型 - audio

by OpenAI Completions API

```
from openai import OpenAI

url = 'http://localhost:8080/v1'

openai_client = OpenAI(base_url=url, api_key='...')
```

© 2026 Enos Chou

Gemma 4 ft. llama-server

叫用模型 - audio

by OpenAI Completions API

```
import base64

audio_file = 'audio.mp3'
with open(audio_file, 'rb') as f:
    audio_base64 = base64.b64encode(f.read()).decode('utf-8')
```

© 2026 Enos Chou

Gemma 4 ft. llama-server

叫用模型 - audio

by OpenAI Completions API

```
messages = [
    {
        'role': 'user',
        'content': [
            {
                'type': 'input_audio',
                'input_audio': {'data': audio_base64, 'format': 'mp3'}
            }
        ]
    }
]
```

© 2026 Enos Chou

Gemma 4 ft. llama-server

叫用模型 - audio

by OpenAI Completions API

```
r = openai_client.chat.completions.create(
    model='...',
    messages=messages,
    temperature=0.7
)
```

© 2026 Enos Chou

Gemma 4 ft. llama-server

叫用模型 - audio

by OpenAI Completions API

```
print(r.choices[0].message.content)
```

© 2026 Enos Chou

相關技術

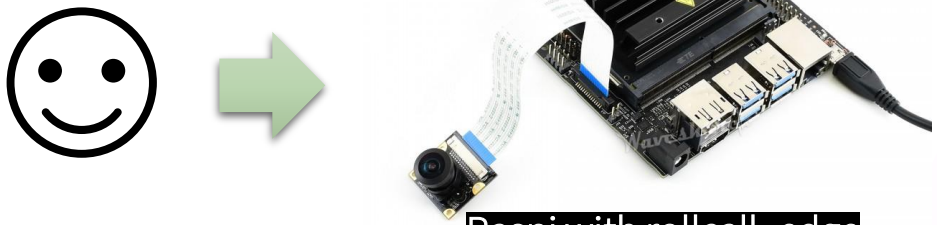
© 2026 Enos Chou

相關技術列表

目的	產品與主題
MQTT	AIoT with MQTT
RDB	MySQL ft. 虛擬機
Google Cloud Service Account	Google Cloud 服務帳戶建立與權限管理
SSH Login	SSH 登入虛擬機
Ports in Use	Port 佔用偵測
Securing Secrets	Google Cloud Secret Manager

© 2026 Enos Chou

AIoT with MQTT



Raspi with rollcall_edge
MQTT Publisher

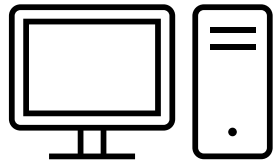
MQTT publish

GCE

mosquitto

MQTT Broker

MQTT subscribe

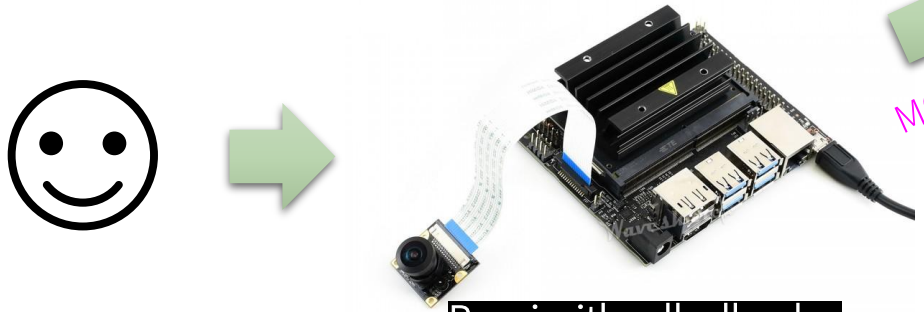


Application
MQTT Subscriber

MQTT subscribe

Node-RED

Management Panel
MQTT Subscriber



Raspi with rollcall_edge
MQTT Publisher

MQTT publish

© 2026 Enos Chou

AIoT with MQTT

MQTT Topic Example

- `rollcall/signin`
- `rollcall/register`
- `rollcall/#`
- `#`

MQTT Message Example

- `Someone on 2024/01/02`

© 2026 Enos Chou

AIoT with MQTT

1. Server as MQTT Broker
2. Server as MQTT Subscriber
3. Client as MQTT Publisher

© 2026 Enos Chou

AIoT with MQTT

1. Server as MQTT Broker

a. Ubuntu 24.04 on GCE

① 登入 https://console.cloud.google.com
② 建立並切換至新專案
③ 啟用 Compute Engine
④ Compute Engine > VM 執行個體 > 建立執行個體
- 名稱 mqtt
- 區域 us-central1/ 區域 us-central-b
- 機器類型 e2-micro
- 開機磁碟 > 作業系統 Ubuntu/ 版本 Ubuntu 24.04 LTS x86/64
- 進階選項 > 網路 > 網路介面 > 外部 IPv4 位址 > 保留靜態外部 IP 位址 > mqttip

© 2026 Enos Chou

AIoT with MQTT

1. Server as MQTT Broker

a. Ubuntu 24.04 on GCE

⑤ 虛擬私有雲網路 > 防火牆 > 建立防火牆規則
- 名稱 mqtt
- 目標 網路中的所有執行個體
- 來源 IPv4 範圍 0.0.0.0/0
- 通訊協定和埠 > 指定的通訊協定和埠 > TCP 1883

⑥ (optional) DNS 指向外部 IP 位址 (mqttip)

⑦ SSH 連線 GCE VM 執行個體 mqtt

```
sudo apt update; sudo apt install -y mosquitto; sudo timedatectl set-timezone Asia/Taipei
```

© 2026 Enos Chou

AIoT with MQTT

1. Server as MQTT Broker

a. Ubuntu 24.04 on GCE

⑧ (若使用 mosquitto 2.x)

於 /etc/mosquitto/mosquitto.conf 加入以下設定

```
listener 1883
allow_anonymous true
```

完成後重起 `sudo systemctl restart mosquitto`

© 2026 Enos Chou

AIoT with MQTT

2. Server as MQTT Subscriber

Python 3 Environment

套件 paho-mqtt

```
pip install paho-mqtt
```

© 2026 Enos Chou

AIoT with MQTT

2. Server as MQTT Subscriber

Python 3 Environment

官方範例

搜尋 `eclipse paho mqtt python` >
進入 paho python 官方 github

© 2026 Enos Chou

AIoT with MQTT

2. Server as MQTT Subscriber

Python 3 Environment

```
import paho.mqtt.client as mqtt

def on_connect(client, userdata, flags, rc):
    print("Connected with result code "+str(rc))
    client.subscribe("rollcall/#")

def on_message(client, userdata, msg):
    print(msg.topic+" "+str(msg.payload.decode('utf-8')))
```

© 2026 Enos Chou

AIoT with MQTT

2. Server as MQTT Subscriber

Python 3 Environment

```
YOUR_BROKER = broker_ip_address_str

client = mqtt.Client()
client.on_connect = on_connect
client.on_message = on_message

client.connect(YOUR_BROKER, 1883, 60)
client.loop_forever()
```

© 2026 Enos Chou

AIoT with MQTT

3. Client as MQTT Publisher

Python 3 Environment

```
from time import strftime

from paho.mqtt import publish

YOUR_BROKER = broker_ip_address_str
topic = 'rollcall'
msg = f'Mary sign in on {strftime("%Y/%m/%d-%H:%M:%S")}'
publish.single(topic, msg, hostname=YOUR_BROKER)
```

© 2026 Enos Chou

AIoT with MQTT

3. Client as MQTT Publisher

Python 3 Environment

```
msgs = [
    ['rollcall/register', f'Jerry on {strftime("%Y/%m/%d-%H:%M:%S")}'],
    ['rollcall/signin', f'Eric on {strftime("%Y/%m/%d-%H:%M:%S")}']
]
publish.multiple(msgs, hostname=YOUR_BROKER)
```

© 2026 Enos Chou

AIoT with MQTT

4. Send/ Receive Image via MQTT

a. Publisher

① read image with 'rb' into bytes
② send bytes as payload

b. Subscriber

① write msg.payload with 'wb' into file, or
② show image

```
from io import BytesIO
from PIL import Image
Image.open(BytesIO(msg.payload)).show()
```

© 2026 Enos Chou

MySQL ft. 虛擬機

© 2026 Enos Chou

MySQL ft. VM

© 2026 Enos Chou

架設 VM

HW

vCPU	1
RAM	1 GB
HDD	10 GB
OS	Ubuntu 24.04 LTS

FW

TCP 3306

© 2026 Enos Chou

安裝 MySQL 8.0

1. 安裝

- 終端機登入 VM

```
sudo apt update
sudo apt install -y mysql-server
```

© 2026 Enos Chou

安裝 MySQL 8.0

2. 設定

a. 取得管理員密碼

```
sudo cat /etc/mysql/debian.cnf
```

b. 登入確認安裝成功

```
mysql -u debian-sys-maint -p
```

c. 新增用戶帳號密碼

```
CREATE USER 'YOUR_USER'@'YOUR_IP' IDENTIFIED BY 'YOUR_PASSWORD';
CREATE USER 'kitty'@'%' IDENTIFIED BY 'hello';
```

d. 授予用戶權限

```
GRANT ALL PRIVILEGES ON YOUR_DB.YOUR_TABLE TO 'YOUR_USER'@'YOUR_IP' WITH GRANT OPTION;
GRANT ALL PRIVILEGES ON *.* TO 'kitty'@'%' WITH GRANT OPTION;
```

© 2026 Enos Chou

安裝 MySQL 8.0

2. 設定

e. 建立 DB

```
CREATE DATABASE YOUR_DB;
```

f. (選擇 DB)

```
USE YOUR_DB;
```

g. (建立 table)

```
CREATE TABLE users (userid varchar(32), nick varchar(32));
```

h. (新增資料)

```
INSERT INTO users VALUES ('enos', 'teacher'), ('kitty', 'hello');
```

i. 離開

```
QUIT
```

© 2026 Enos Chou

安裝 MySQL 8.0

3. 允許遠方登入

a. 調整組態檔允許遠端登入 (此設定須謹慎不隨意對外開放)

```
sudo nano /etc/mysql/mysql.conf.d/mysqld.cnf
```

修改 `bind-address = 127.0.0.1` 為 `bind-address = YOUR_IP or 0.0.0.0`

```
CTRL+o   # 儲存
CTRL+x   # 退出
```

b. 重啟 MySQL 生效新組態

```
sudo systemctl restart mysql
```

c. 設定 VM 防火牆允許外部 IP 連入

允許 `TCP 3306` 連入

© 2026 Enos Chou

安裝 MySQL 8.0

4. 驗證

a. by MySQL Client

```
sudo apt install -y mysql-client
mysql -h YOUR_IP -u YOUR_USER -p
```

© 2026 Enos Chou

安裝 MySQL 8.0

4. 驗證

   b. by Python

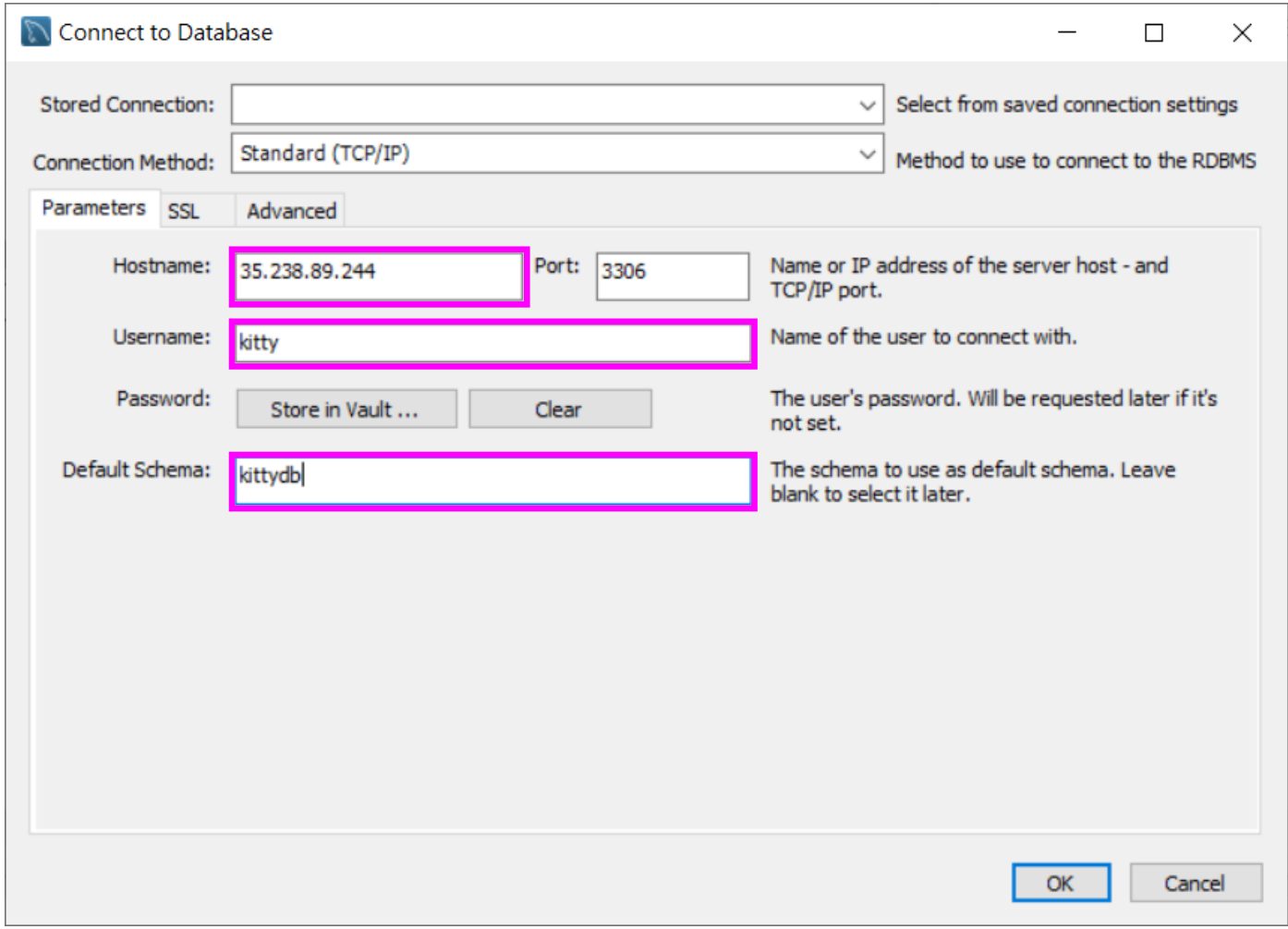
```
# pip install mysql-connector-python

import mysql.connector

conn = mysql.connector.connect(host='YOUR_IP',
                               user='YOUR_USER',
                               password='YOUR_PASSWORD',
                               database='YOUR_DB')

with conn.cursor() as cursor:
    cursor.execute('SELECT * FROM users')
    print(cursor.fetchall())

conn.close()
```

© 2026 Enos Chou

安裝 MySQL 8.0

4. 驗證

c. by MySQL Workbench

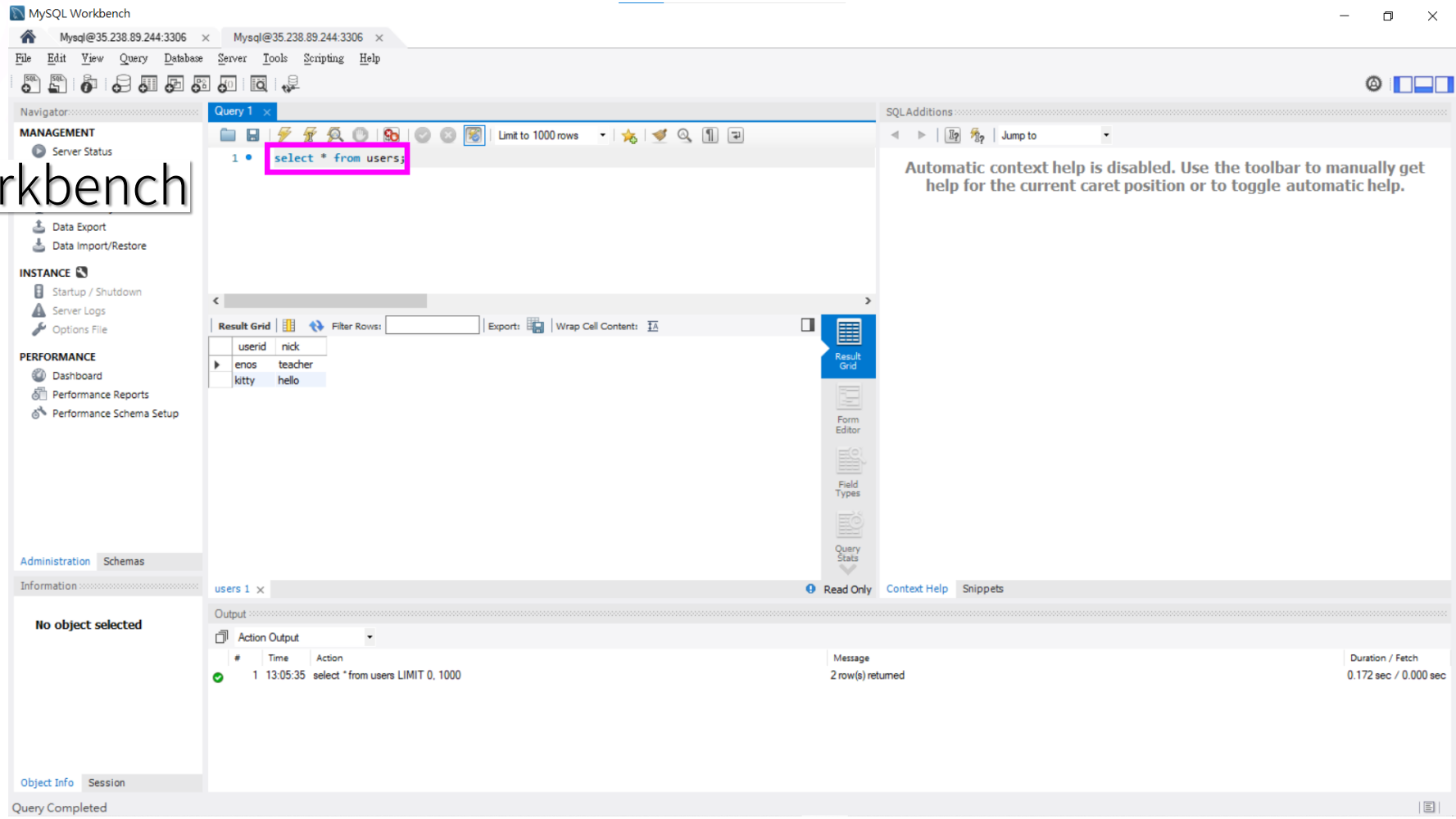


© 2026 Enos Chou

安裝 MySQL 8.0

4. 驗證

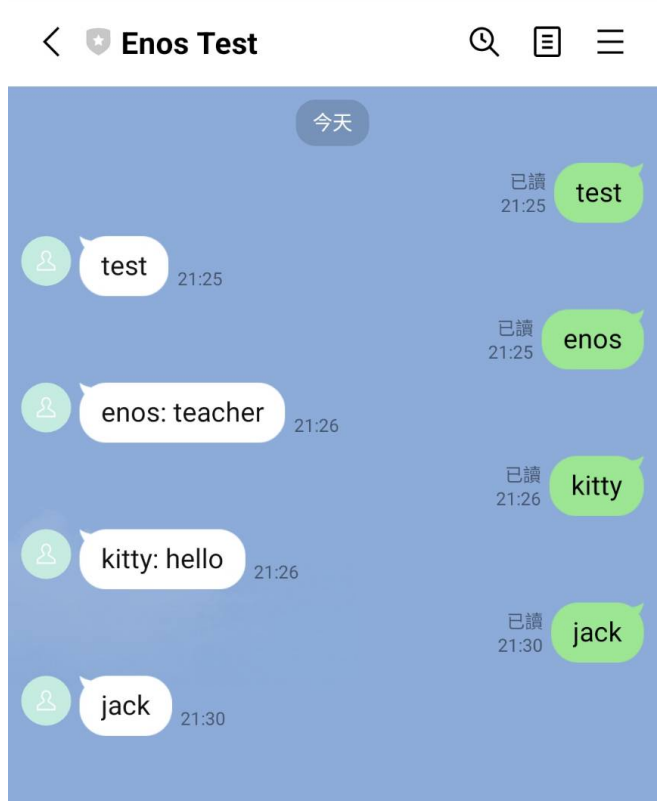
c. by MySQL Workbench

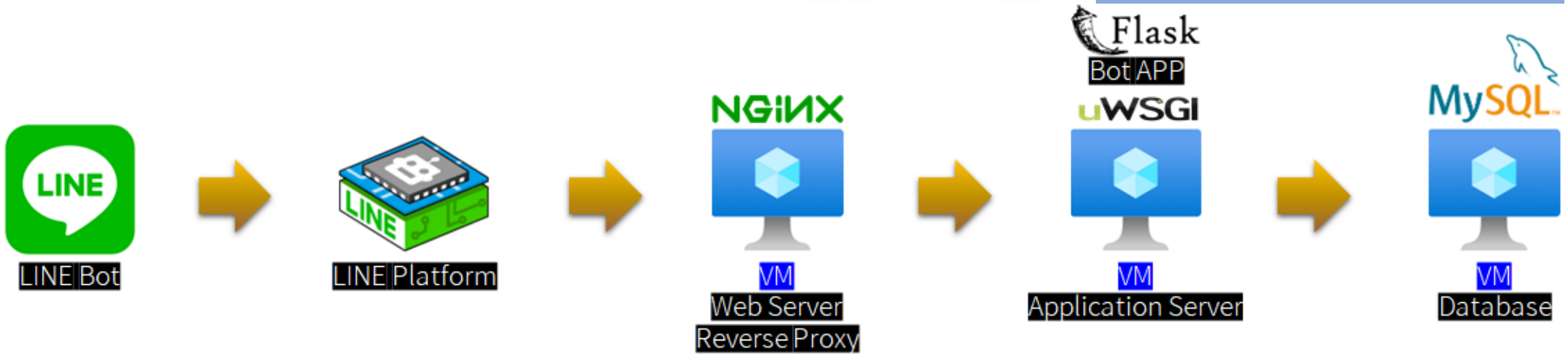


Your Turn

LINE Bot 透過內網連接 MySQL，與之連動

1. 存在於 DB 中的名字回應對應的 nick
2. 不存在於 DB 中的名字直接 echo





Google Cloud 服務帳戶建立與權限管理

IAM 與管理

服務帳戶

© 2026 Enos Chou

服務帳戶概念

服務帳戶 (Service Account) 概念

a. 服務帳戶目的
   ① 提供外部應用程式驗證 Google Cloud 服務 (例如：Google Cloud Storage) 之帳號
   ② 提供外部應用程式綁定必要 Google Cloud 服務的權限

b. 服務帳戶建立與使用程序
   ① 建立服務帳戶搭配外部應用需求之適當權限
   ② 產生對應之金鑰 (JSON) 後下載 (此金鑰可視 Google Cloud 驗證外部應用程式之密碼)
   ③ 外部應用程式存取 Google Cloud 服務時搭配金鑰使用

© 2026 Enos Chou

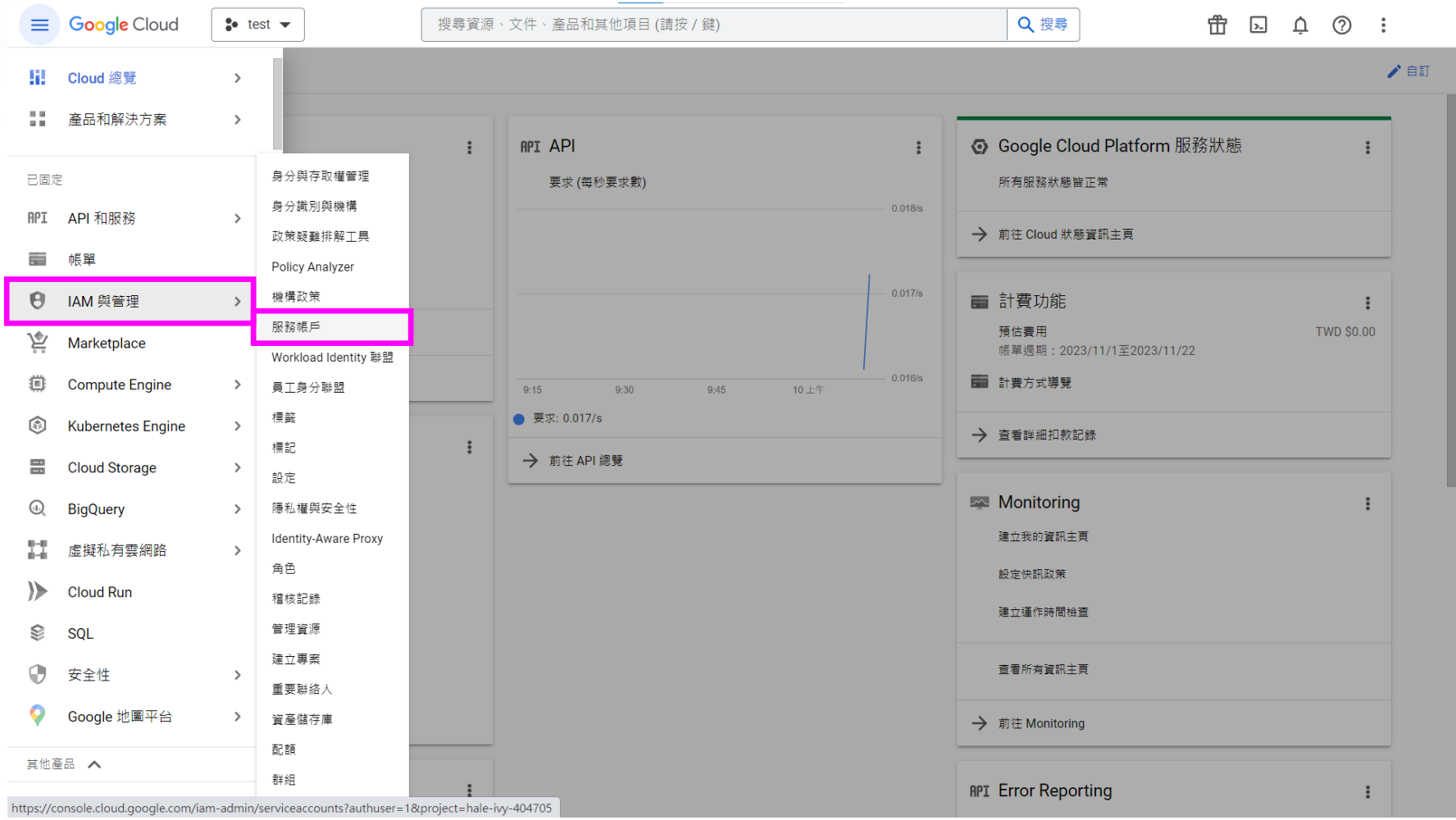
服務帳戶建立與調整

1. 建立服務帳戶
2. 下載金鑰
3. 調整權限
4. 調整權限案例

© 2026 Enos Chou

服務帳戶建立與調整

1. 建立服務帳戶



© 2026 Enos Chou

服務帳戶建立與調整

1. 建立服務帳戶

服務帳戶建立與調整

1. 建立服務帳戶

© 2026 Enos Chou

服務帳戶建立與調整

1. 建立服務帳戶

服務帳戶建立與調整

2. 下載金鑰

a. (開啟下載金鑰權限)

SYMPTOM
目前 Google Cloud 預設的組織政策不允許建立金鑰，直接下載會報錯。

ERROR
貴組織已強制執行組織政策，禁止建立金鑰

SOLUTION
欲下載金鑰須遵循以下程序調整組織政策

© 2026 Enos Chou

服務帳戶建立與調整

2. 下載金鑰

a. (開啟下載金鑰權限)

① 組織層級將登入帳號加入 "組織政策管理員" 權限

"IAM 與管理" > "管理資源" >

© 2026 Enos Chou

服務帳戶建立與調整

2. 下載金鑰

a. (開啟下載金鑰權限)

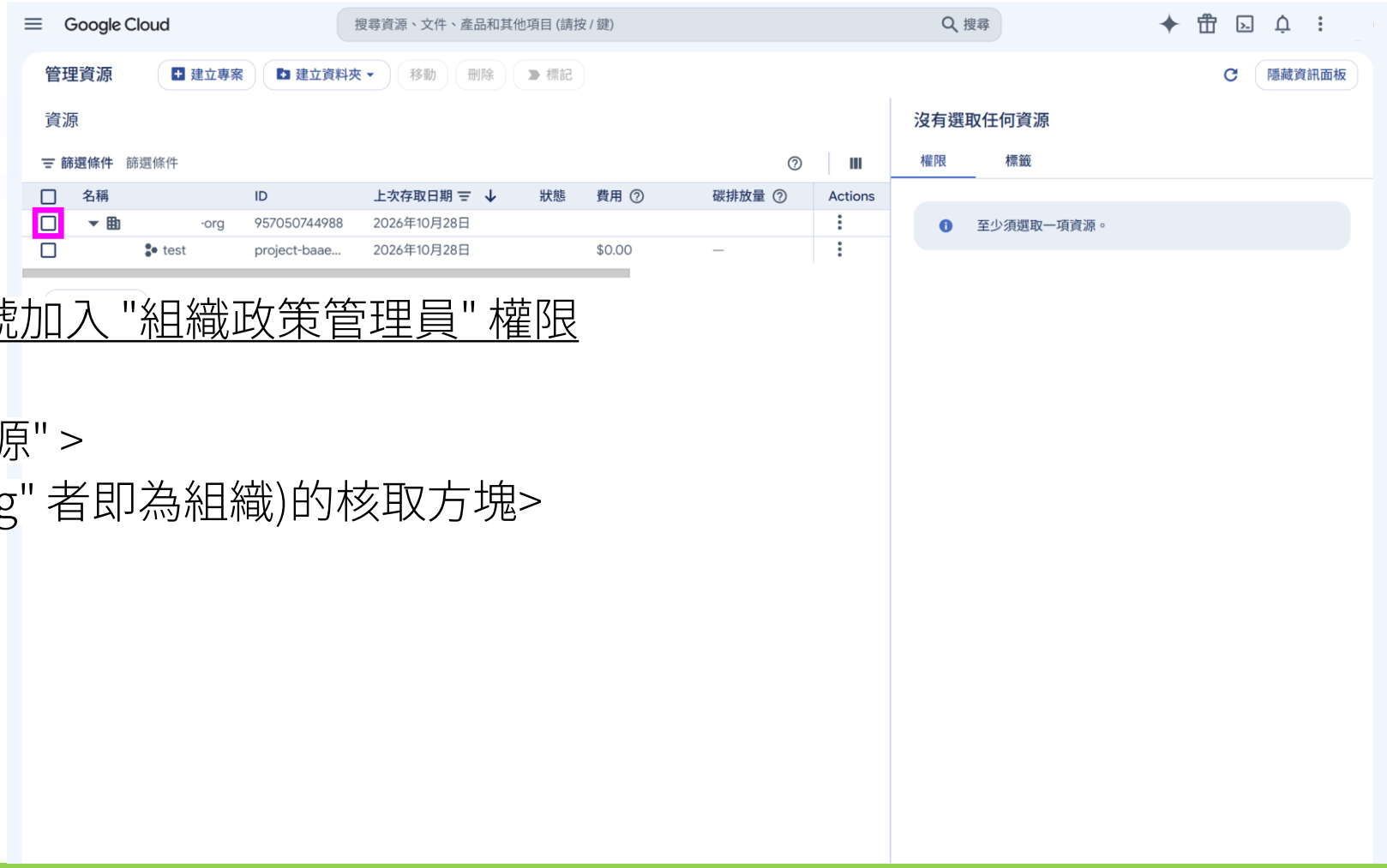
① 組織層級將登入帳號加入 "組織政策管理員" 權限

"IAM 與管理" > "管理資源" >

服務帳戶建立與調整

2. 下載金鑰

a. (開啟下載金鑰權限)



① 組織層級將登入帳號加入 "組織政策管理員" 權限

"IAM 與管理" > "管理資源" >
勾選組織(名稱後綴 "-org" 者即為組織)的核取方塊>

© 2026 Enos Chou

服務帳戶建立與調整

2. 下載金鑰

a. (開啟下載金鑰權限)

① 組織層級將登入帳號加入 "組織政策管理員" 權限

"IAM 與管理" > "管理資源" >
勾選組織(名稱後綴 "-org" 者即為組織)的核取方塊>

© 2026 Enos Chou

服務帳戶建立與調整

2. 下載金鑰

a. (開啟下載金鑰權限)

① 組織層級將登入帳號加入 "組織政策管理員" 權限

"IAM 與管理" > "管理資源" >
勾選組織(名稱後綴 "-org" 者即為組織)的核取方塊>
新增主體 >

© 2026 Enos Chou

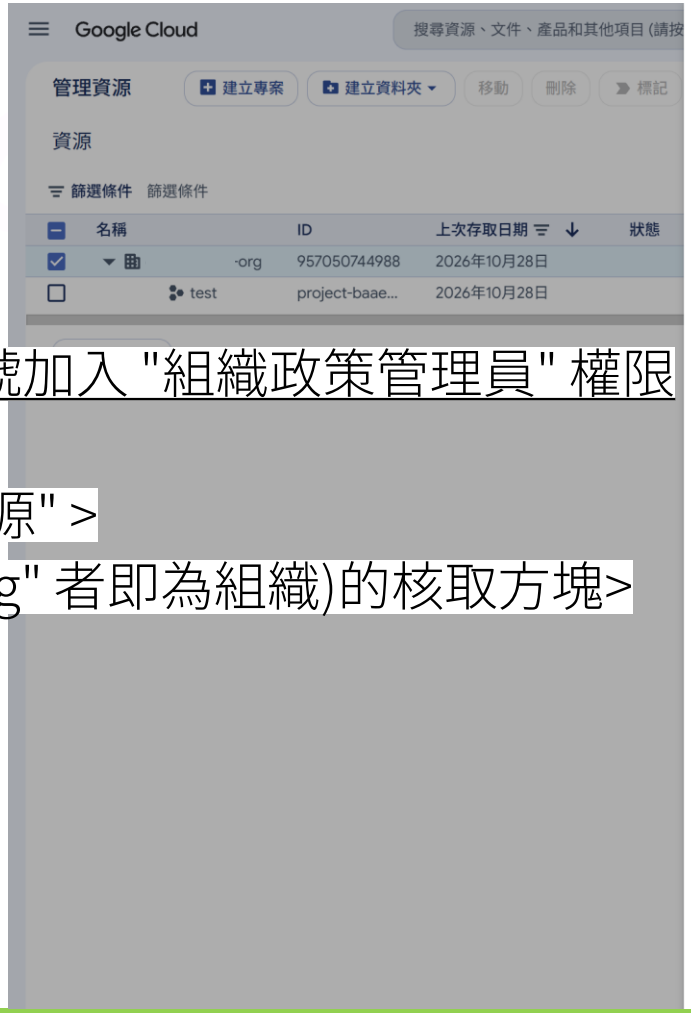
服務帳戶建立與調整

2. 下載金鑰

a. (開啟下載金鑰權限)

① 組織層級將登入帳號加入 "組織政策管理員" 權限

"IAM 與管理" > "管理資源" >
勾選組織(名稱後綴 "-org" 者即為組織)的核取方塊>
新增主體 >



© 2026 Enos Chou

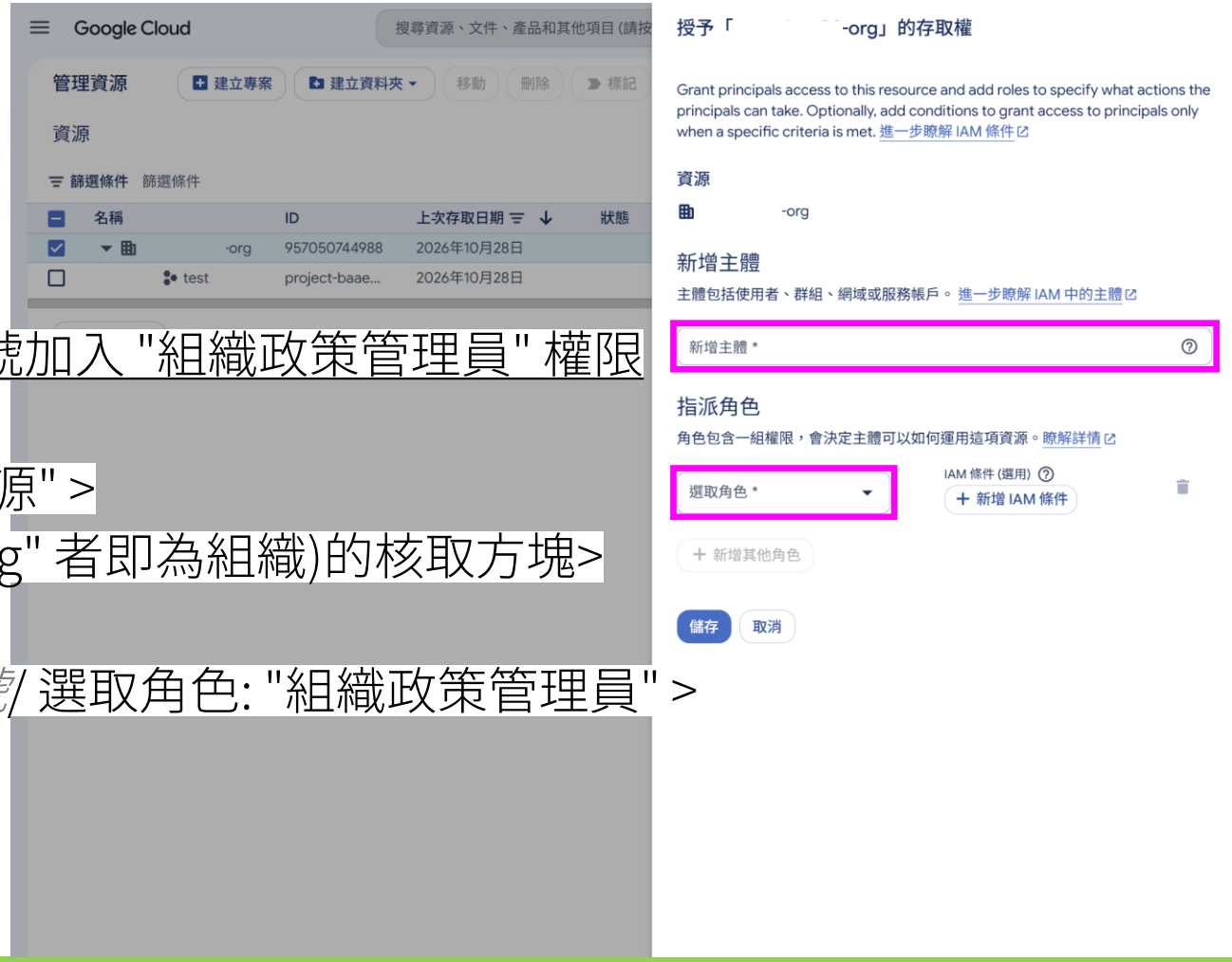
服務帳戶建立與調整

2. 下載金鑰

a. (開啟下載金鑰權限)

① 組織層級將登入帳號加入 "組織政策管理員" 權限

"IAM 與管理" > "管理資源" >
勾選組織(名稱後綴 "-org" 者即為組織)的核取方塊>
新增主體 >
新增主體: *輸入登入帳號*/ 選取角色: "組織政策管理員" >



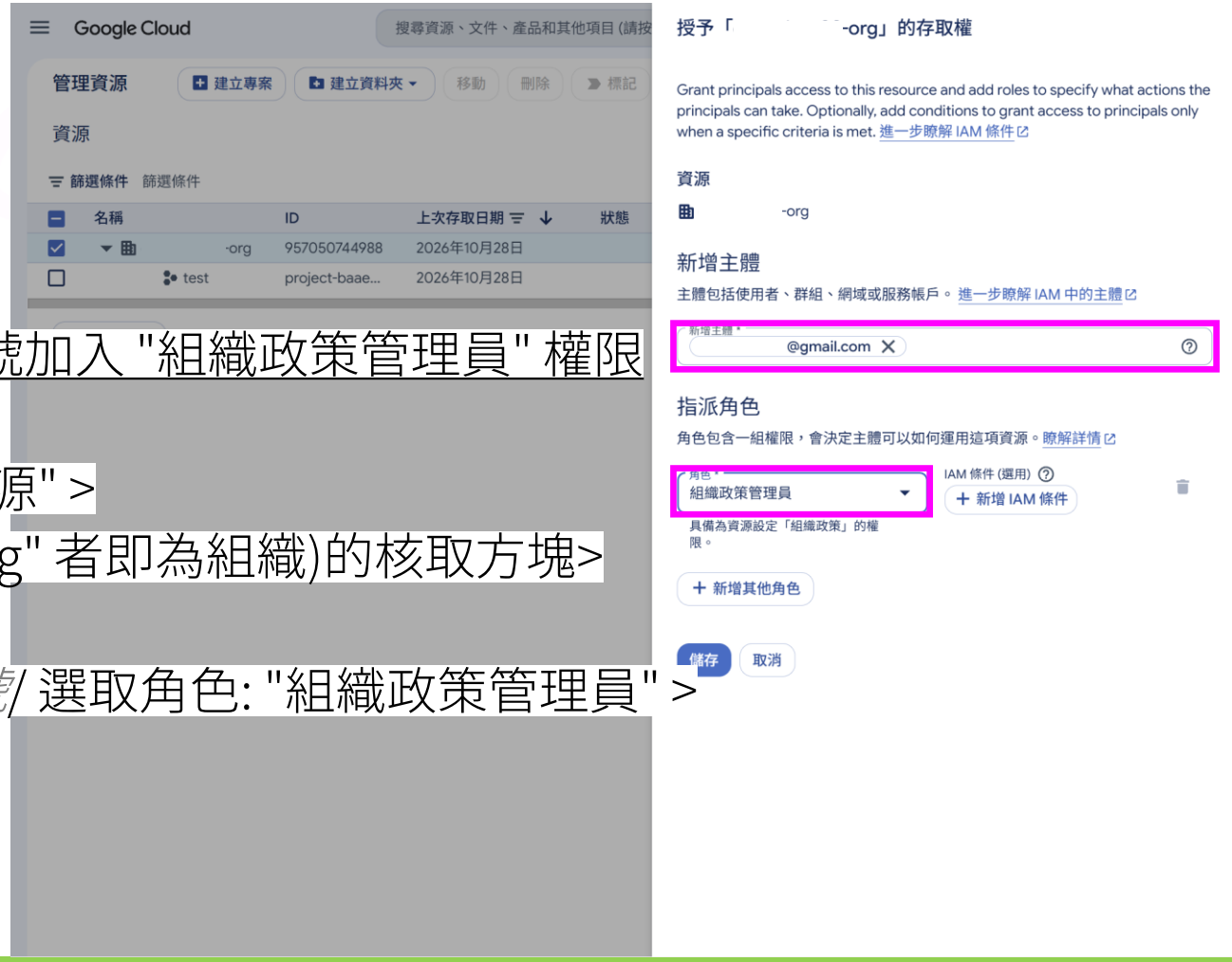
服務帳戶建立與調整

2. 下載金鑰

a. (開啟下載金鑰權限)

① 組織層級將登入帳號加入 "組織政策管理員" 權限

"IAM 與管理" > "管理資源" >
勾選組織(名稱後綴 "-org" 者即為組織)的核取方塊>
新增主體 >
新增主體: *輸入登入帳號*/ 選取角色: "組織政策管理員" >



服務帳戶建立與調整

2. 下載金鑰

a. (開啟下載金鑰權限)

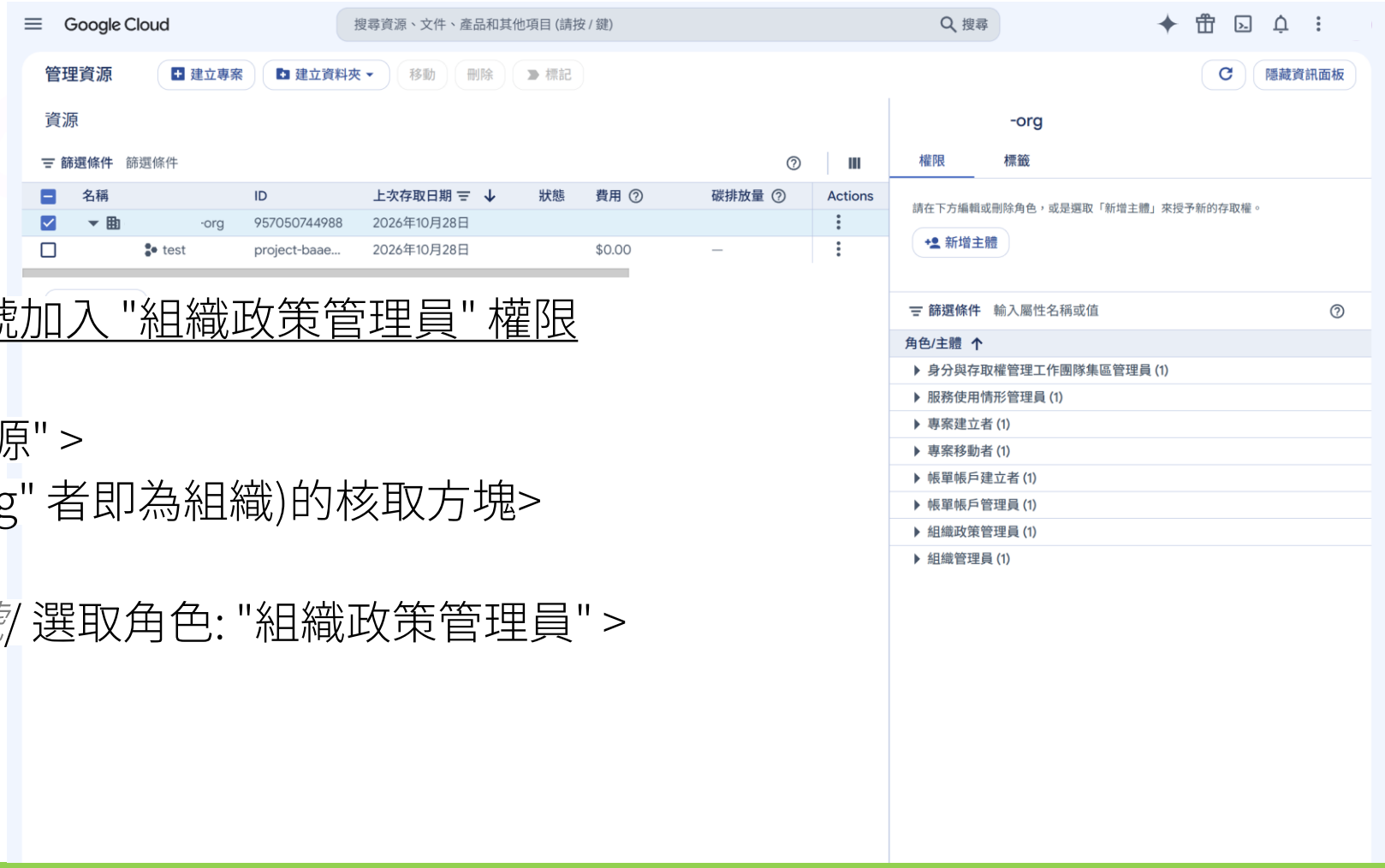
① 組織層級將登入帳號加入 "組織政策管理員" 權限

"IAM 與管理" > "管理資源" >
勾選組織(名稱後綴 "-org" 者即為組織)的核取方塊>
新增主體 >
新增主體: *輸入登入帳號*/ 選取角色: "組織政策管理員" >
儲存 >

服務帳戶建立與調整

2. 下載金鑰

a. (開啟下載金鑰權限)



① 組織層級將登入帳號加入 "組織政策管理員" 權限

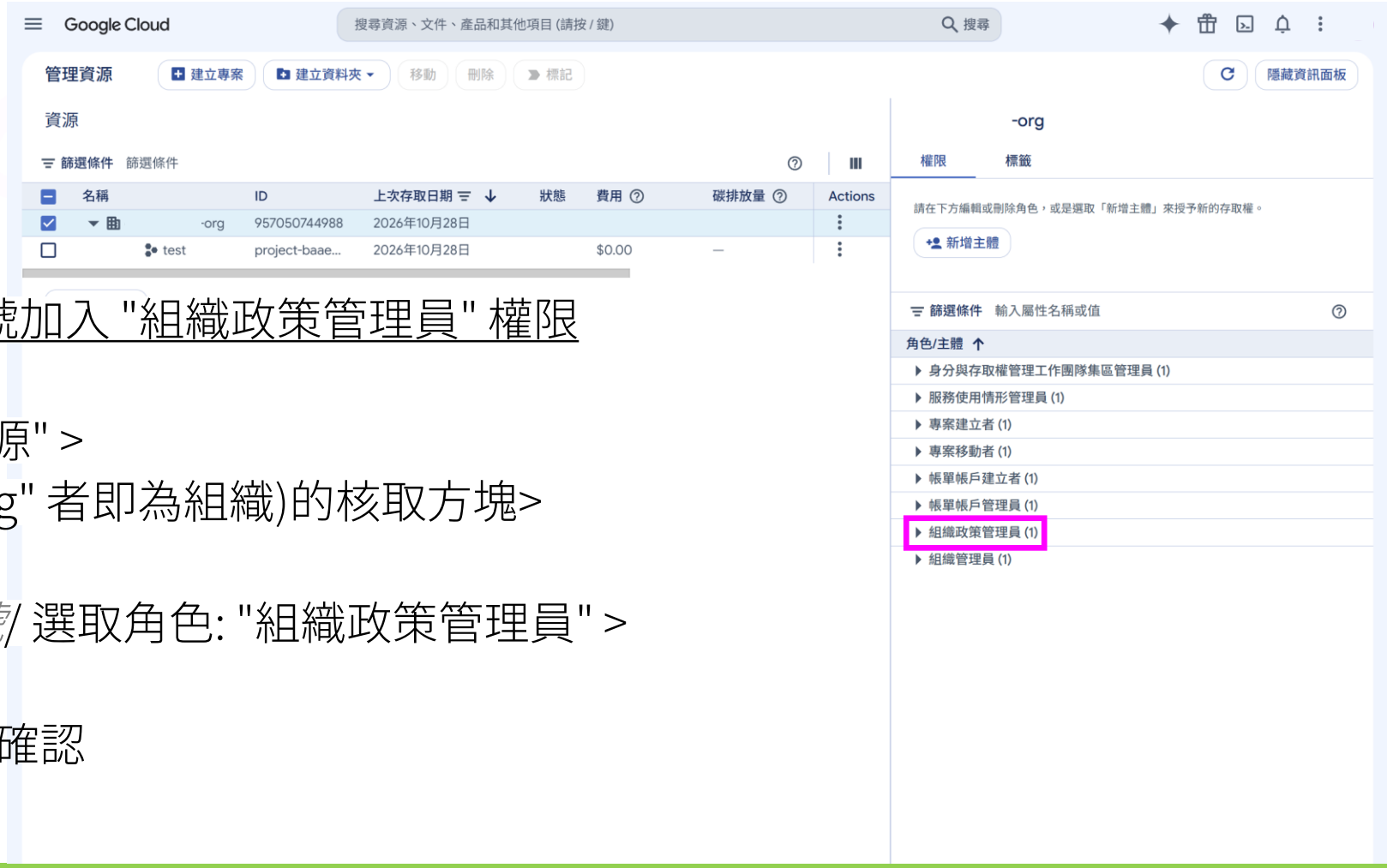
"IAM 與管理" > "管理資源" >
勾選組織(名稱後綴 "-org" 者即為組織)的核取方塊>
新增主體 >
新增主體: *輸入登入帳號*/ 選取角色: "組織政策管理員" >
儲存 >

© 2026 Enos Chou

服務帳戶建立與調整

2. 下載金鑰

a. (開啟下載金鑰權限)



① 組織層級將登入帳號加入 "組織政策管理員" 權限

"IAM 與管理" > "管理資源" >
勾選組織(名稱後綴 "-org" 者即為組織)的核取方塊>
新增主體 >
新增主體: *輸入登入帳號*/ 選取角色: "組織政策管理員" >
儲存 >
展開 "組織政策管理員" 確認

© 2026 Enos Chou

服務帳戶建立與調整

2. 下載金鑰

a. (開啟下載金鑰權限)

① 組織層級將登入帳號加入 "組織政策管理員" 權限

"IAM 與管理" > "管理資源" >
勾選組織(名稱後綴 "-org" 者即為組織)的核取方塊>
新增主體 >
新增主體: *輸入登入帳號*/ 選取角色: "組織政策管理員" >
儲存 >
展開 "組織政策管理員" 確認

© 2026 Enos Chou

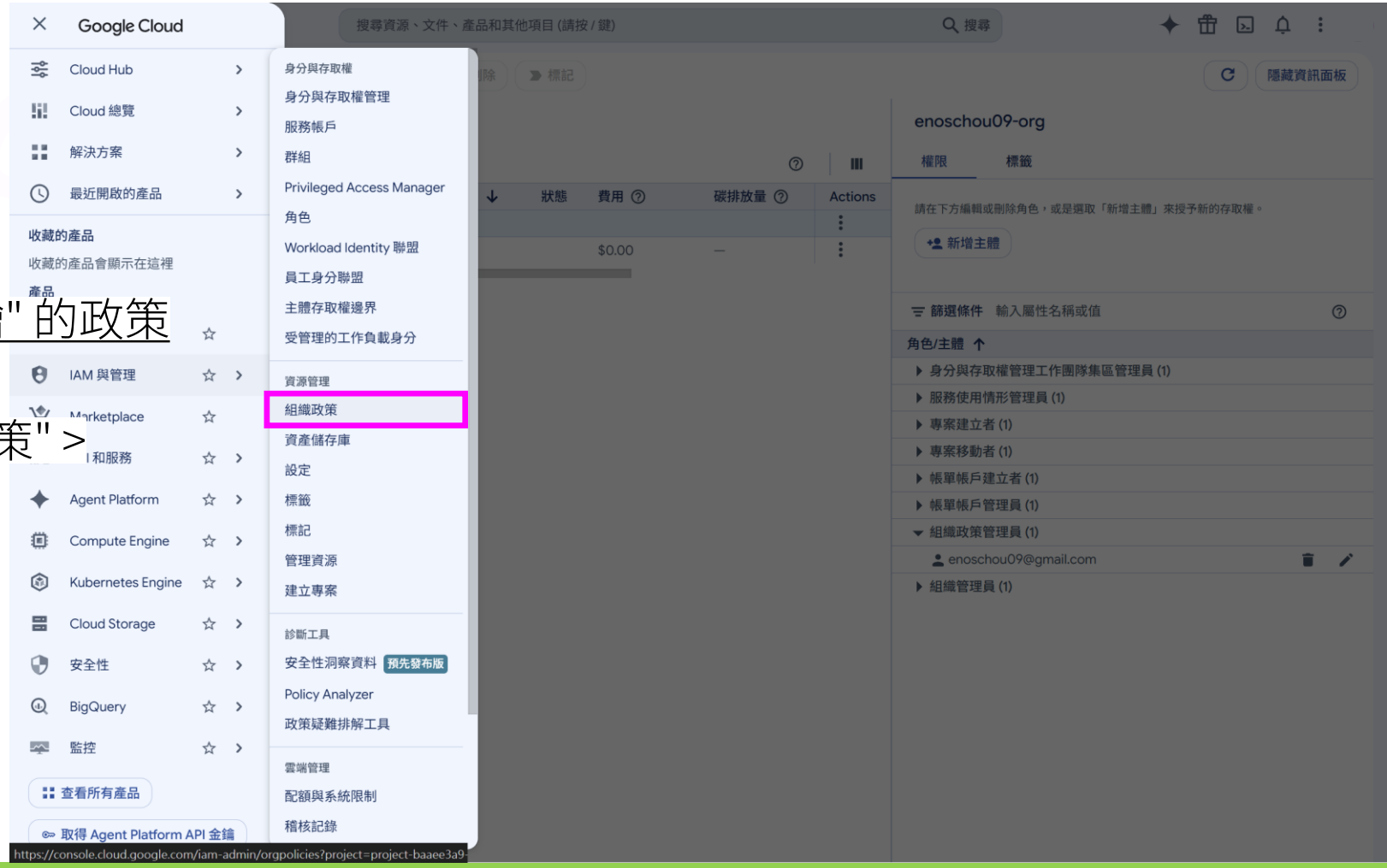
服務帳戶建立與調整

2. 下載金鑰

a. (開啟下載金鑰權限)

② 停用 "不可建立金鑰" 的政策

"IAM 與管理" > "組織政策" >



服務帳戶建立與調整

2. 下載金鑰

a. (開啟下載金鑰權限)

② 停用 "不可建立金鑰" 的政策

"IAM 與管理" > "組織政策" >

服務帳戶建立與調整

2. 下載金鑰

a. (開啟下載金鑰權限)

② 停用 "不可建立金鑰" 的政策

"IAM 與管理" > "組織政策" >
於 "篩選條件" 篩選 iam.disableServiceAccountKeyCreation >

© 2026 Enos Chou

服務帳戶建立與調整

2. 下載金鑰

a. (開啟下載金鑰權限)

② 停用 "不可建立金鑰" 的政策

"IAM 與管理" > "組織政策" >
於 "篩選條件" 篩選 iam.disableServiceAccountKeyCreation >

© 2026 Enos Chou

服務帳戶建立與調整

2. 下載金鑰

a. (開啟下載金鑰權限)

② 停用 "不可建立金鑰" 的政策

"IAM 與管理" > "組織政策" >
於 "篩選條件" 篩選 iam.disableServiceAccountKeyCreation >
點擊對應的 "Disable service account key creation" >

© 2026 Enos Chou

服務帳戶建立與調整

2. 下載金鑰

a. (開啟下載金鑰權限)

② 停用 "不可建立金鑰" 的政策

"IAM 與管理" > "組織政策" >
於 "篩選條件" 篩選 iam.disableServiceAccountKeyCreation >
點擊對應的 "Disable service account key creation" >

服務帳戶建立與調整

2. 下載金鑰

a. (開啟下載金鑰權限)

② 停用 "不可建立金鑰" 的政策

"IAM 與管理" > "組織政策" >
於 "篩選條件" 篩選 iam.disableServiceAccountKeyCreation >
點擊對應的 "Disable service account key creation" >
管理政策 >

服務帳戶建立與調整

2. 下載金鑰

a. (開啟下載金鑰權限)

② 停用 "不可建立金鑰" 的政策

"IAM 與管理" > "組織政策" >
於 "篩選條件" 篩選 `iam.disableServiceAccountKeyCreation` >
點擊對應的 "Disable service account key creation" >
管理政策 >

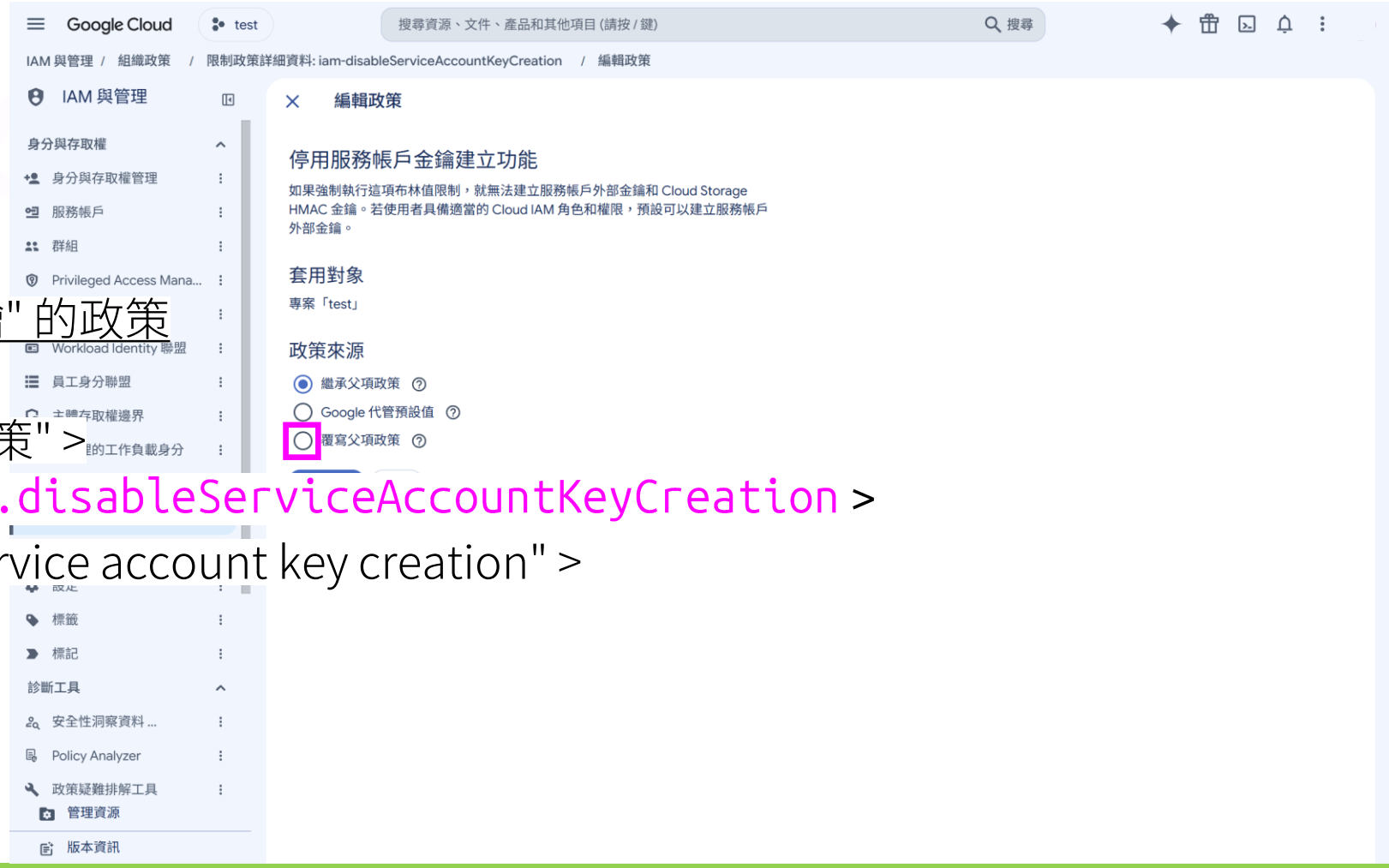
服務帳戶建立與調整

2. 下載金鑰

a. (開啟下載金鑰權限)

② 停用 "不可建立金鑰" 的政策

"IAM 與管理" > "組織政策" >
於 "篩選條件" 篩選 `iam.disableServiceAccountKeyCreation` >
點擊對應的 "Disable service account key creation" >
管理政策 >
勾選 "覆寫父項政策" >



服務帳戶建立與調整

2. 下載金鑰

a. (開啟下載金鑰權限)

② 停用 "不可建立金鑰" 的政策

"IAM 與管理" > "組織政策" >
於 "篩選條件" 篩選 iam.disableServiceAccountKeyCreation >
點擊對應的 "Disable service account key creation" >
管理政策 >
勾選 "覆寫父項政策" >

服務帳戶建立與調整

2. 下載金鑰

a. (開啟下載金鑰權限)

② 停用 "不可建立金鑰" 的政策

"IAM 與管理" > "組織政策" >
於 "篩選條件" 篩選 iam.disableServiceAccountKeyCreation >
點擊對應的 "Disable service account key creation" >
管理政策 >
勾選 "覆寫父項政策" >
新增規則 >

服務帳戶建立與調整

2. 下載金鑰

a. (開啟下載金鑰權限)

② 停用 "不可建立金鑰" 的政策

"IAM 與管理" > "組織政策" >
於 "篩選條件" 篩選 iam.disableServiceAccountKeyCreation >
點擊對應的 "Disable service account key creation" >
管理政策 >
勾選 "覆寫父項政策" >
新增規則 >

服務帳戶建立與調整

2. 下載金鑰

a. (開啟下載金鑰權限)

② 停用 "不可建立金鑰" 的政策

"IAM 與管理" > "組織政策" >
於 "篩選條件" 篩選 iam.disableServiceAccountKeyCreation >
點擊對應的 "Disable service account key creation" >
管理政策 >
勾選 "覆寫父項政策" >
新增規則 >
完成 >

服務帳戶建立與調整

2. 下載金鑰

a. (開啟下載金鑰權限)

② 停用 "不可建立金鑰" 的政策

"IAM 與管理" > "組織政策" >
於 "篩選條件" 篩選 iam.disableServiceAccountKeyCreation >
點擊對應的 "Disable service account key creation" >
管理政策 >
勾選 "覆寫父項政策" >
新增規則 >
完成 >

服務帳戶建立與調整

2. 下載金鑰

a. (開啟下載金鑰權限)

② 停用 "不可建立金鑰" 的政策

"IAM 與管理" > "組織政策" >
於 "篩選條件" 篩選 iam.disableServiceAccountKeyCreation >
點擊對應的 "Disable service account key creation" >
管理政策 >
勾選 "覆寫父項政策" >
新增規則 >
完成 >
設定政策

服務帳戶建立與調整

2. 下載金鑰

a. (開啟下載金鑰權限)

② 停用 "不可建立金鑰" 的政策

"IAM 與管理" > "組織政策" >
於 "篩選條件" 篩選 iam.disableServiceAccountKeyCreation >
點擊對應的 "Disable service account key creation" >
管理政策 >
勾選 "覆寫父項政策" >
新增規則 >
完成 >
設定政策

© 2026 Enos Chou

服務帳戶建立與調整

2. 下載金鑰

b. 建立並下載金鑰

管理金鑰

© 2026 Enos Chou

服務帳戶建立與調整

2. 下載金鑰

b. 建立並下載金鑰

新增金鑰

© 2026 Enos Chou

服務帳戶建立與調整

2. 下載金鑰

b. 建立並下載金鑰

建立新的金鑰

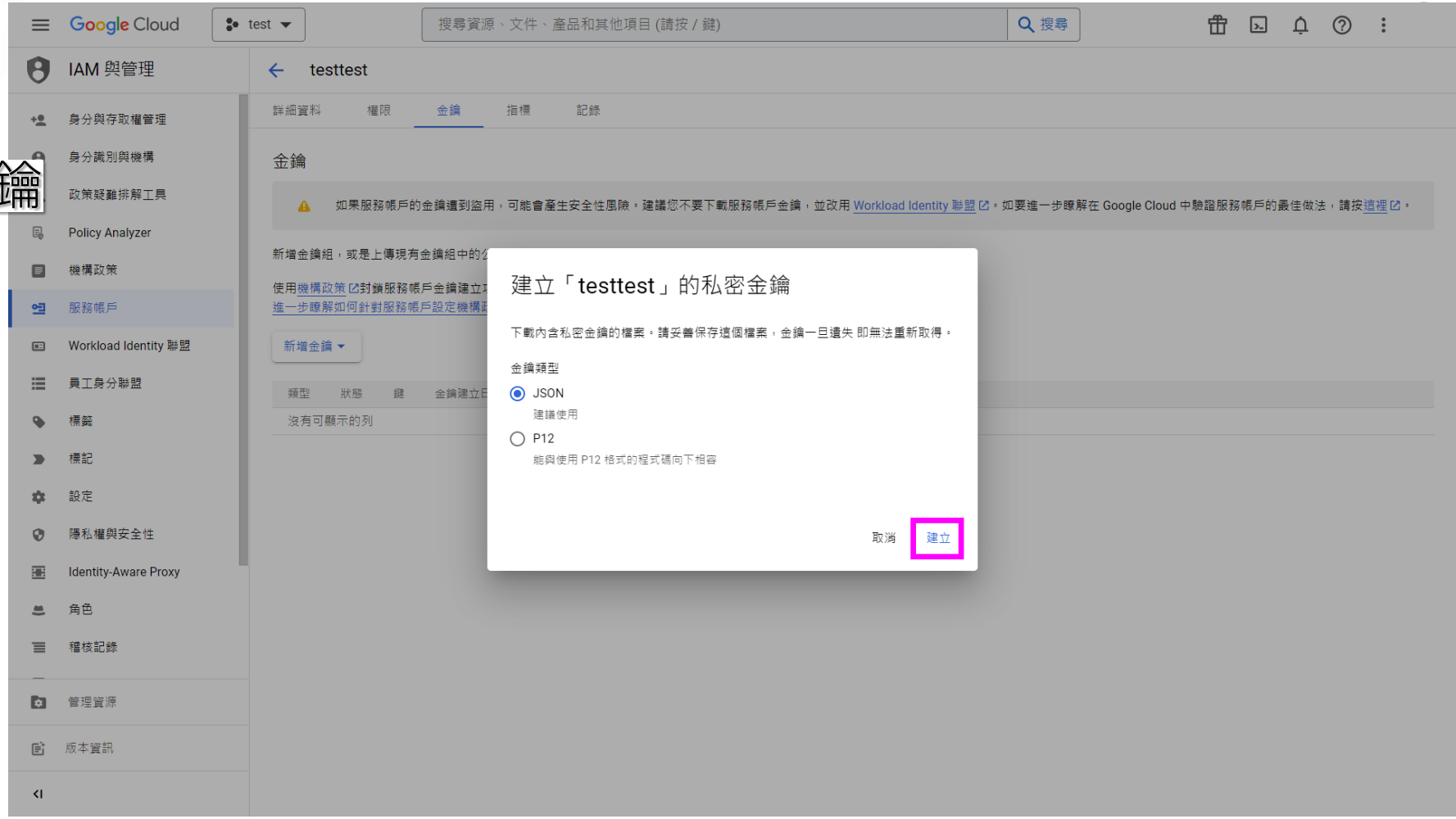
© 2026 Enos Chou

服務帳戶建立與調整

2. 下載金鑰

b. 建立並下載金鑰

建立 JSON 金鑰



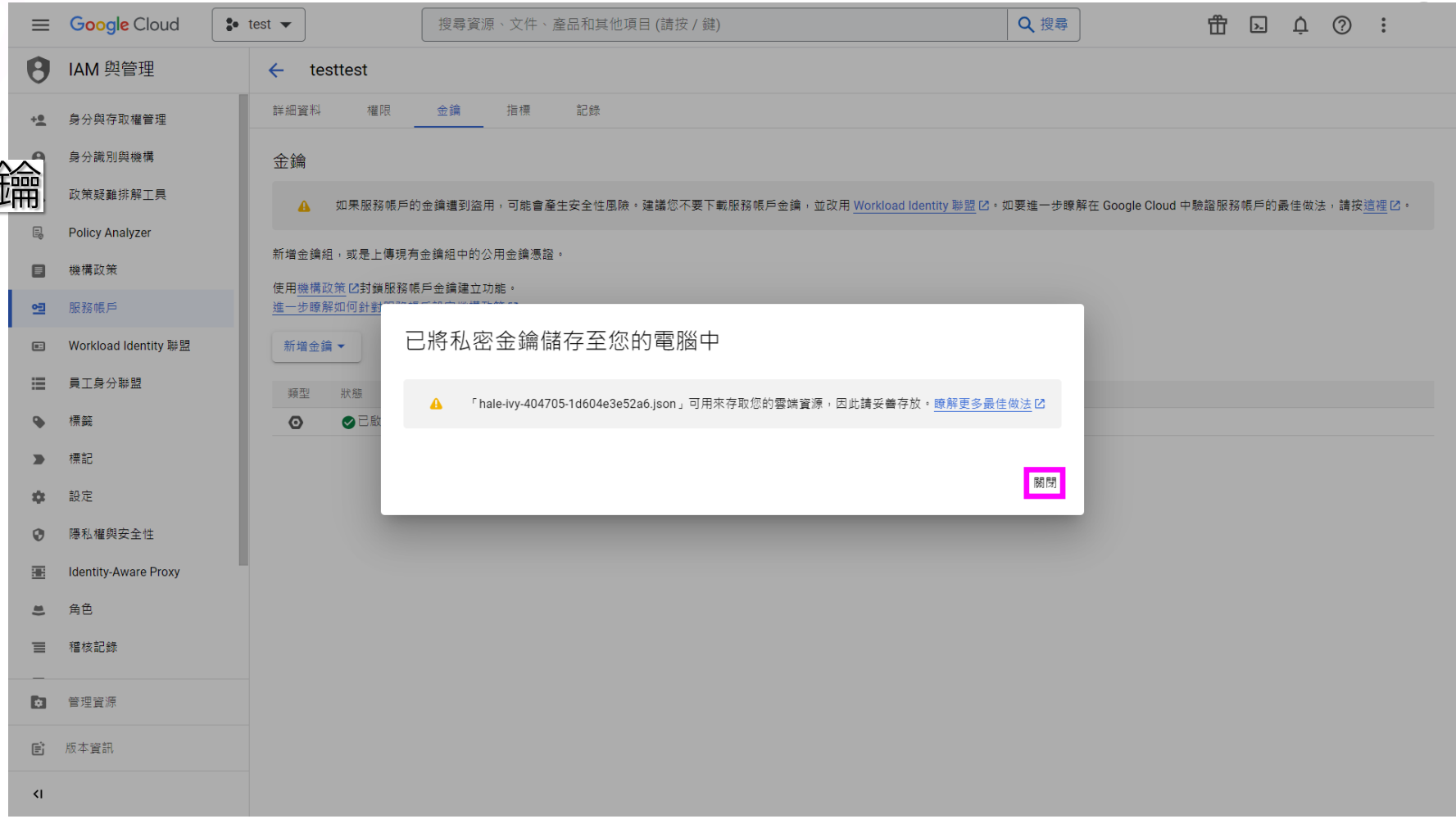
© 2026 Enos Chou

服務帳戶建立與調整

2. 下載金鑰

b. 建立並下載金鑰

妥善保管金鑰



服務帳戶建立與調整

3. 調整權限

a. 調整服務帳戶權限

ERROR

testtest-xxx@yyy.iam.gserviceaccount.com does not have storage.objects.create access to the Google Cloud Storage object. Permission 'storage.objects.create' denied on resource (or it may not exist).

SOLUTION

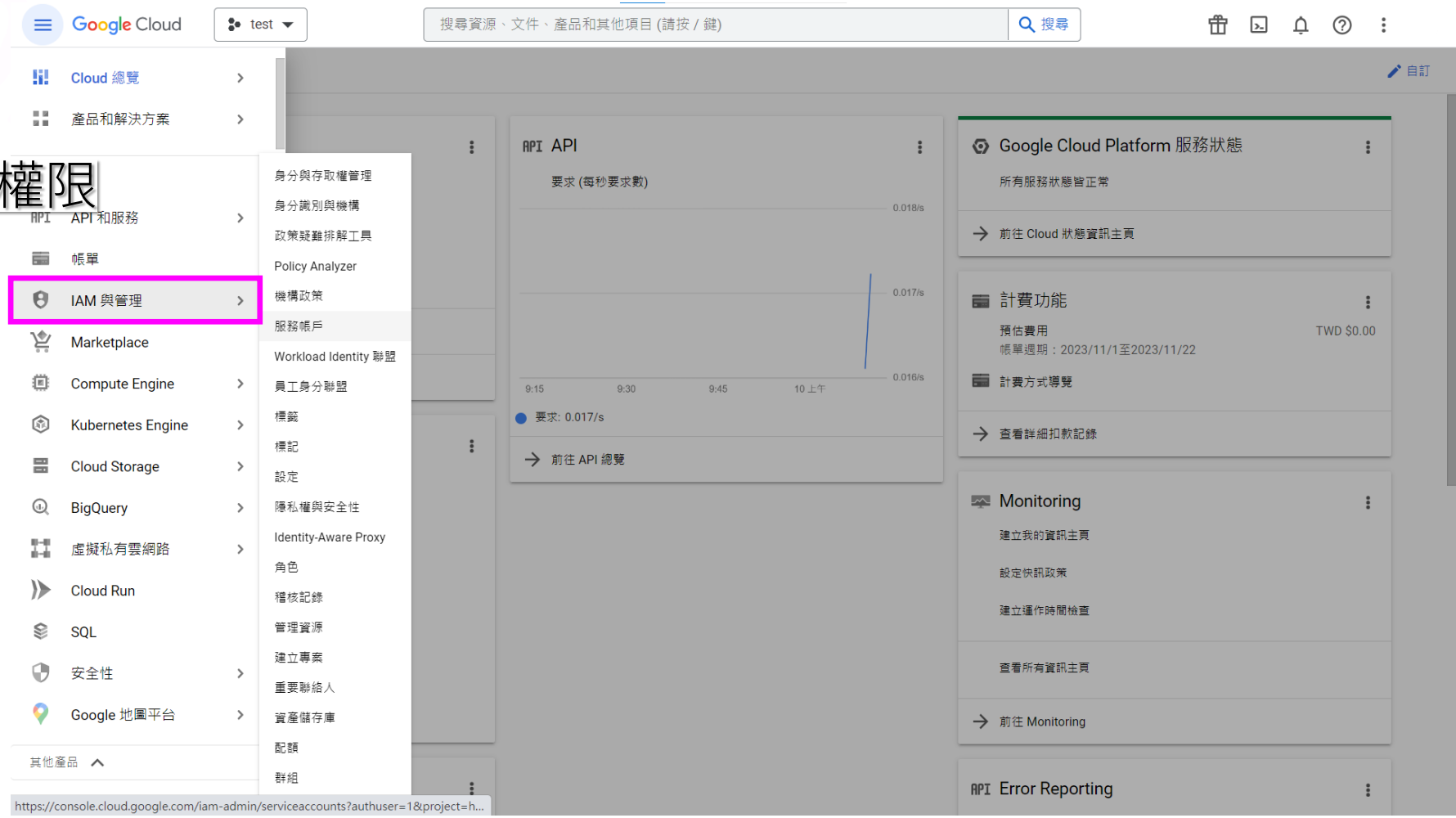
透過 "IAM 與管理" 針對服務帳戶增加 "Storage 物件使用者" 或 "Storage 管理員" 權限

© 2026 Enos Chou

服務帳戶建立與調整

3. 調整權限

a. 調整服務帳戶權限



© 2026 Enos Chou

服務帳戶建立與調整

3. 調整權限

a. 調整服務帳戶權限

授予存取權

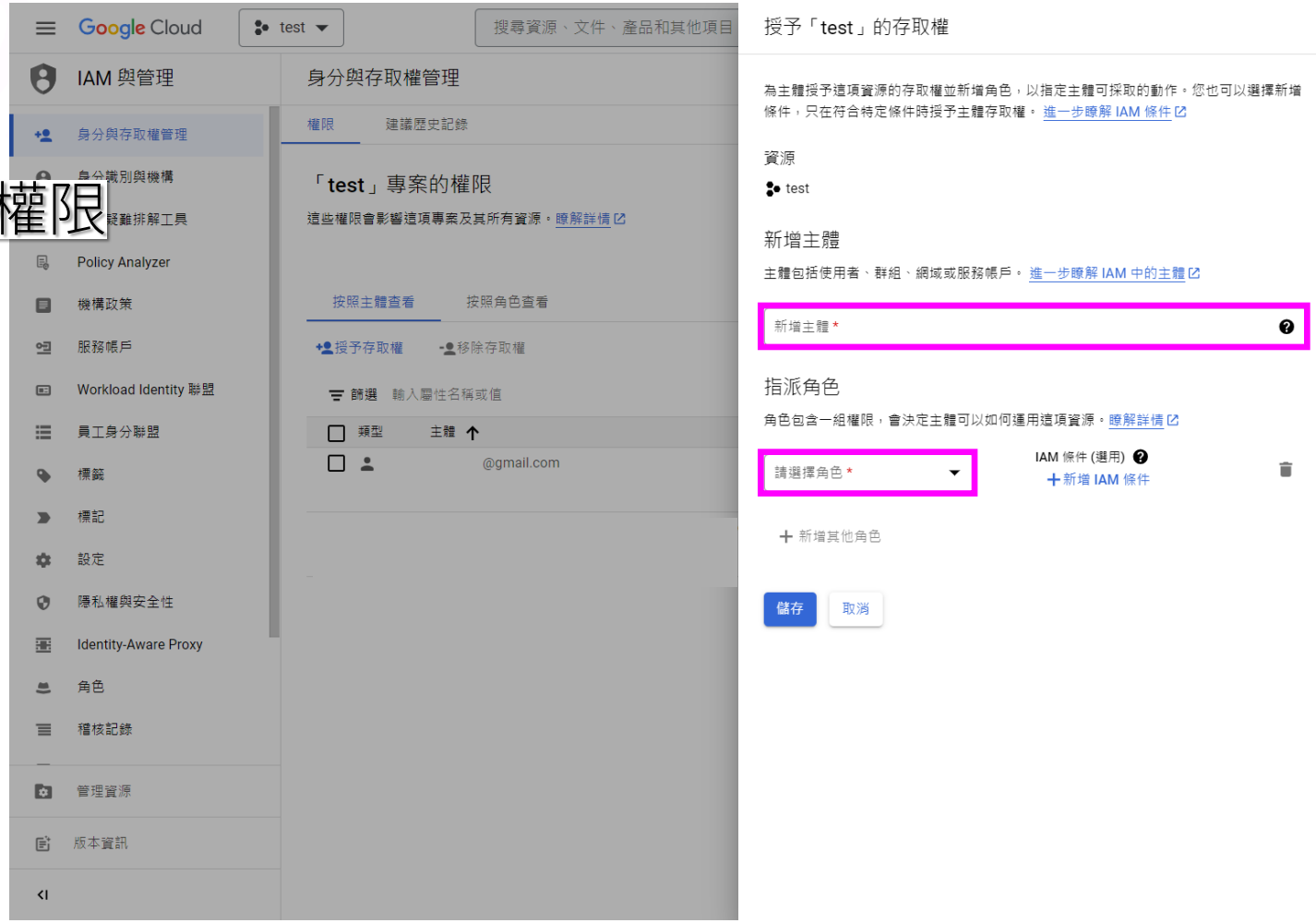
© 2026 Enos Chou

服務帳戶建立與調整

3. 調整權限

a. 調整服務帳戶權限

"新增主體" >
"請選擇角色"

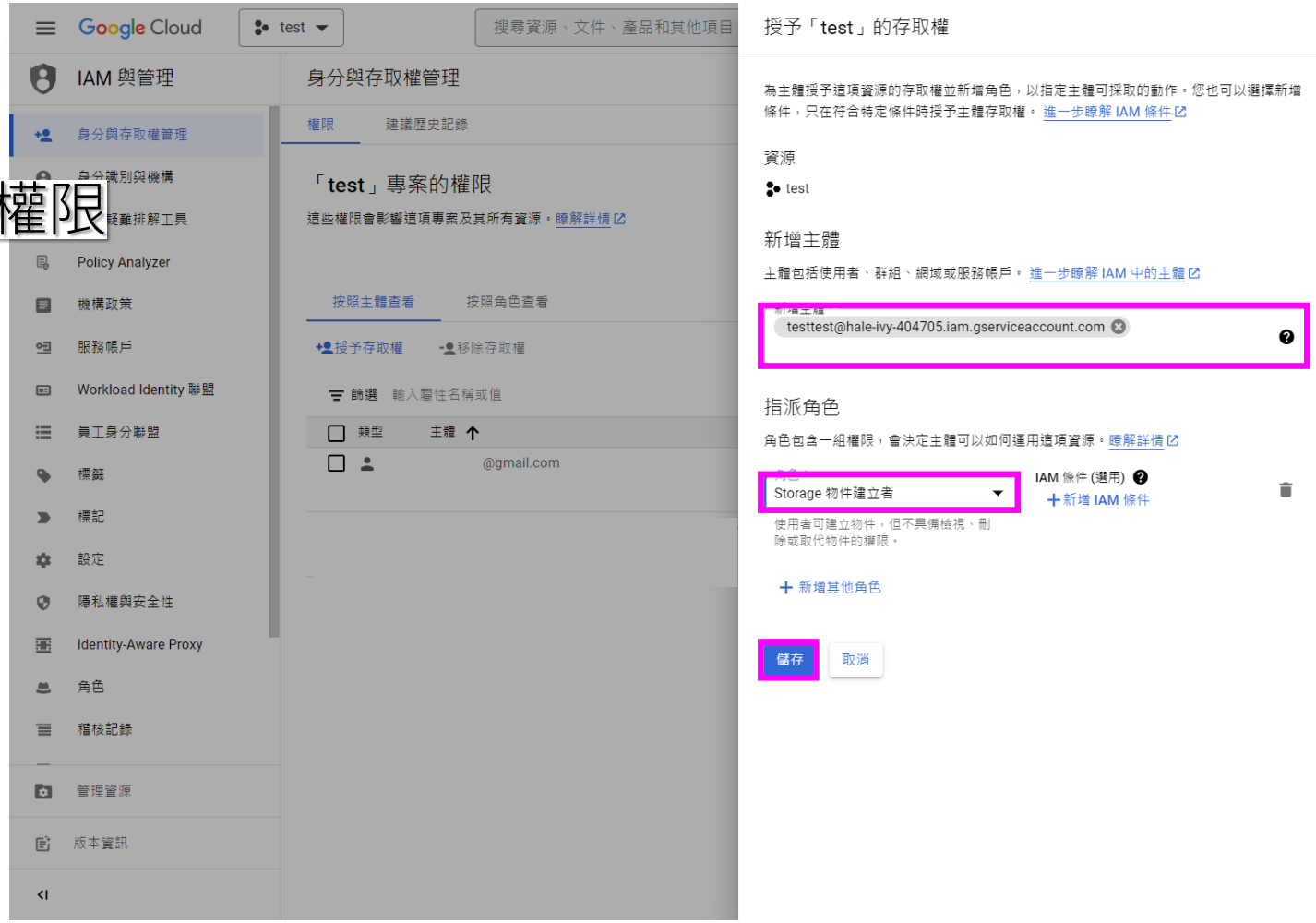


服務帳戶建立與調整

3. 調整權限

a. 調整服務帳戶權限

"新增主體" >
"請選擇角色" >
儲存



© 2026 Enos Chou

服務帳戶建立與調整

3. 調整權限

a. 調整服務帳戶權限

約 1 分鐘後生效

服務帳戶建立與調整

4. 調整權限案例

未啟用 Cloud Speech-toText API

ERROR

PermissionDenied: 403 Cloud Speech-to-Text API has not been used in project xxx before or it is disabled. Enable it by visiting https://console.developers.google.com/apis/api/speech.googleapis.com/overview?project=xxx then retry. If you enabled this API recently, wait a few minutes for the action to propagate to our systems and retry.

SOLUTION

啟用 Cloud Speech-to-Text API

© 2026 Enos Chou

服務帳戶建立與調整

4. 調整權限案例

未啟用 Cloud Vision API

ERROR

PermissionDenied: 403 Cloud Vision API has not been used in project xxx before or it is disabled. Enable it by visiting https://console.developers.google.com/apis/api/vision.googleapis.com/overview?project=xxx then retry. If you enabled this API recently, wait a few minutes for the action to propagate to our systems and retry.

SOLUTION

啟用 Cloud Vision API

© 2026 Enos Chou

服務帳戶建立與調整

4. 調整權限案例

無法使用 Vertex AI 產品，如 Gemini

ERROR

403 Permission 'aiplatform.endpoints.predict' denied on resource '//aiplatform.googleapis.com/projects/xxx/locations/global/publishers/google/models/gemini-3.5-flash' (or it may not exist).

SOLUTION

透過 "IAM 與管理" 針對服務帳戶增加 "Vertex AI: Agent Platform 使用者" 權限

© 2026 Enos Chou

服務帳戶建立與調整

4. 調整權限案例

其他常見服務帳戶權限需求

產品	角色類別	角色
Artifact Registry	Artifact Registry	Artifact Registry 推送建立寫入者
Cloud Firestore	Datastore	Cloud Datastore 使用者
Cloud Run Job	Cloud Run	Cloud Run 叫用者
Cloud Storage	Cloud Storage	Storage 物件使用者 Storage 檢視者
Gemini	Vertex AI	Agent Platform 使用者
Secret Manager	密鑰管理員	Secret Manager 密鑰存取者

© 2026 Enos Chou

服務帳戶使用

四種常見用戶端初始化服務帳戶範例 (以 Cloud Storage 為例)

case 1: 直接指定服務帳戶金鑰

```
from google.cloud import storage

storage_client = storage.Client.from_service_account_json(
    service_account_filename_str
)
```

© 2026 Enos Chou

服務帳戶使用

四種常見用戶端初始化服務帳戶範例 (以 Cloud Storage 為例)

case 2: 以服務帳戶金鑰建立憑證

```
from google.cloud import storage
from google.oauth2 import service_account

credentials = service_account.Credentials.from_service_account_file(
    service_account_filename_str,
    scopes=['https://www.googleapis.com/auth/cloud-platform'])
storage_client = storage.Client(credentials=credentials)
```

© 2026 Enos Chou

服務帳戶使用

四種常見用戶端初始化服務帳戶範例 (以 Cloud Storage 為例)

case 3: 以環境變數指定服務帳戶金鑰

```
import os

from google.cloud import storage

YOUR_SERVICE = service_account_filename_str

os.environ['GOOGLE_APPLICATION_CREDENTIALS'] = YOUR_SERVICE
storage_client = storage.Client()
```

© 2026 Enos Chou

服務帳戶使用

四種常見用戶端初始化服務帳戶範例 (以 Cloud Storage 為例)

case 4: Google Cloud 指定服務帳戶金鑰

```
from google.cloud import storage

storage_client = storage.Client()
```

© 2026 Enos Chou

服務帳戶使用

快速驗證服務帳戶範例 (以 Cloud Storage 為例)

安裝套件

```
pip install google-cloud-storage
```

© 2026 Enos Chou

服務帳戶使用

快速驗證服務帳戶範例 (以 Cloud Storage 為例)

範例程式

```
from google.cloud import storage

storage_client = storage.Client.from_service_account_json(
    service_account_filename_str
)
print(list(storage_client.list_buckets()))
```

© 2026 Enos Chou

SSH 登入虛擬機

© 2026 Enos Chou

SSH 登入虛擬機

1. 建立帳號金鑰

Windows

① (於指定路徑) 啟動終端機
② `ssh-keygen -f your_keyfile -C your_account`
`ssh-keygen -f eng -C eng  # 可選擇追加密碼或留空`
③ 產出 *your_keyfile* (私鑰) 以及 *your_keyfile*.pub (公鑰)
④ `type your_keyfile.pub`
`type eng.pub`
⑤ 複製螢幕輸出之公鑰內容

© 2026 Enos Chou

SSH 登入虛擬機

1. 建立帳號金鑰

macOS

① (於指定路徑) 啟動終端機
② `ssh-keygen -f your_keyfile -C your_account`
```
ssh-keygen -f eng -C eng  # 可選擇追加密碼或留空
```
③ 產出 *your_keyfile* (私鑰) 以及 *your_keyfile*.pub (公鑰)
④ `cat your_keyfile.pub`
```
cat eng.pub
```
⑤ 複製螢幕輸出之公鑰內容

© 2026 Enos Chou

SSH 登入虛擬機

2. 於虛擬機設定帳號公鑰

Azure Virtual Machine

Azure Portal 建立虛擬機器過程中 >
基本 >
Administrator 帳戶: 驗證類型 >
SSH 公開金鑰 >
使用者名稱: 與帳號金鑰指定的帳號相同 >
SSH 公開金鑰來源: 使用現有的公開金鑰 >
SSH 公開金鑰: 貼上先前複製的帳號公鑰 >
...

© 2026 Enos Chou

SSH 登入虛擬機

2. 於虛擬機設定帳號公鑰

Google Computer Engine

Google Cloud Console >
Compute Engine >
中繼資料 >
安全殼層金鑰 >
編輯 >
新增項目 >
安全殼層金鑰: 貼上先前複製的帳號公鑰 >
儲存

© 2026 Enos Chou

SSH 登入虛擬機

3. 遠端 SSH 登入

Windows

MobaXterm，以帳號私鑰登入

Windows/ macOS

Tabby，以帳號私鑰登入

© 2026 Enos Chou

SSH 登入虛擬機

3. 遠端 SSH 登入

Tabby 登入

① 初始化

指定 語言/ 亮度 > 關閉並不再顯示

② 設定

設定 > 設定檔與連線 > + 新增 > + 新設定檔 > SSH 連線 ENTER >
輸入 名稱/ 主機/ 使用者名稱/ 新增私鑰 > 儲存

③ 連線

點擊 ▶ 開始連線 > 接受並記住金鑰

© 2026 Enos Chou

Ports 佔用偵測

© 2026 Enos Chou

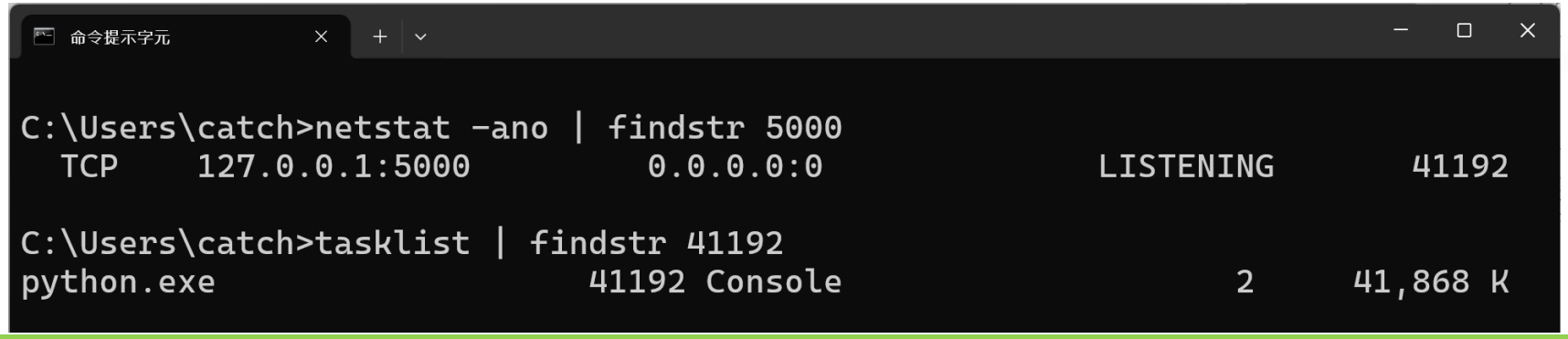
Ports in Use on Windows

1. 找出特定 port 被佔用的 PID

```
netstat -ano | findstr 5000
```

2. 找出特定 PID 代表的程式

```
tasklist | findstr 41192
```



© 2026 Enos Chou

Ports in Use on macOS or Linux

1. 找出特定 port 被佔用的 PID

```
lsof -i :5000
lsof | grep :5000

ps -ef | grep :5000
ps aux | grep :5000
```

© 2026 Enos Chou

Google Cloud Secret Manager

© 2026 Enos Chou

Google Cloud Secret Manager

1. 建立密鑰

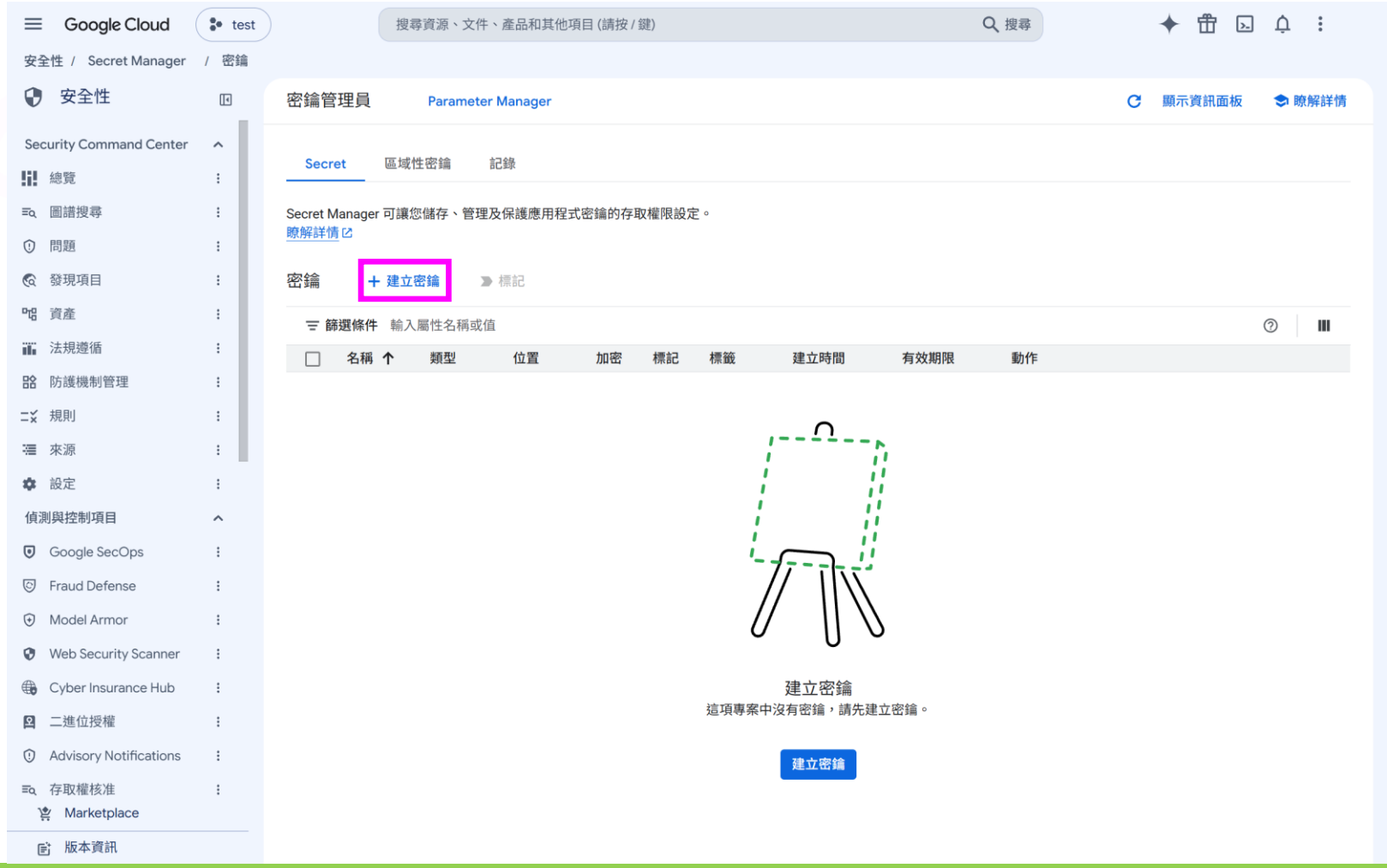
   a. 進入 Secret Manager

Google Cloud Secret Manager

1. 建立密鑰

   b. 建立密鑰

   建立密鑰



Google Cloud Secret Manager

1. 建立密鑰

b. 建立密鑰

建立密鑰

Google Cloud Secret Manager

1. 建立密鑰

b. 建立密鑰

名稱: 輸入密鑰名稱
密鑰值: 輸入密鑰內容

Google Cloud Secret Manager

1. 建立密鑰

b. 建立密鑰

名稱: 輸入密鑰名稱
密鑰值: 輸入密鑰內容

Google Cloud Secret Manager

1. 建立密鑰

b. 建立密鑰

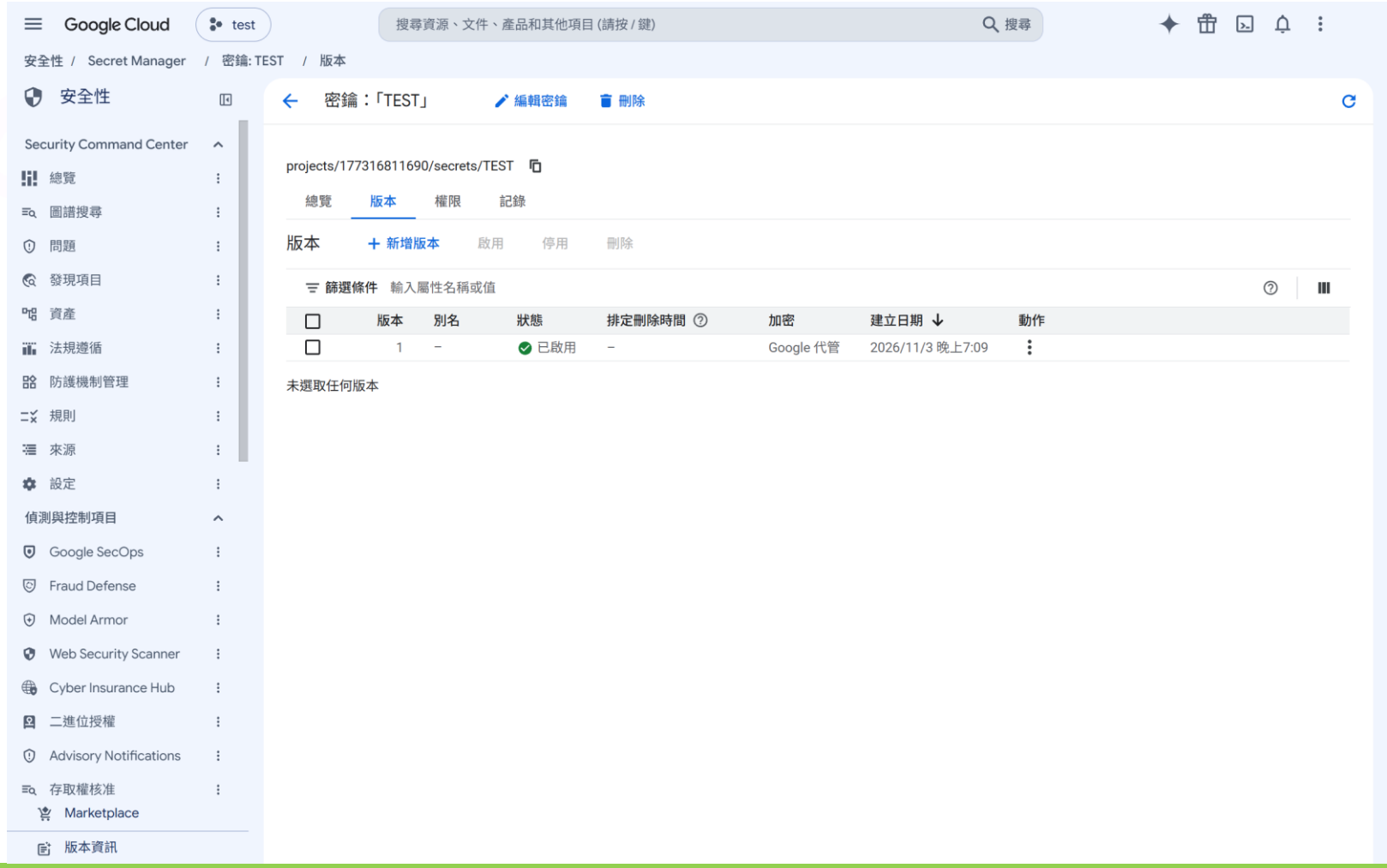
建立密鑰

Google Cloud Secret Manager

1. 建立密鑰

b. 建立密鑰

建立密鑰



Google Cloud Secret Manager

2. 讀取密鑰

a. 增加服務帳戶權限

密鑰管理員 > Secret Manager 密鑰存取者

© 2026 Enos Chou

Google Cloud Secret Manager

2. 讀取密鑰

a. 安裝套件

```
pip install google-cloud-secret-manager
```

© 2026 Enos Chou

Google Cloud Secret Manager

2. 讀取密鑰

b. 初始化 ft. Service Account JSON

```
from google.oauth2 import service_account

credentials = service_account.Credentials.from_service_account_file(
    service_account_filename_str,
    scopes=['https://www.googleapis.com/auth/cloud-platform']
)
```

© 2026 Enos Chou

Google Cloud Secret Manager

2. 讀取密鑰

b. 初始化 ft. Service Account JSON

```
from google.cloud import secretmanager

secretmanager_client = secretmanager.SecretManagerServiceClient(
    credentials=credentials
)
```

© 2026 Enos Chou

Google Cloud Secret Manager

2. 讀取密鑰

c. 取得密鑰

```
full_secret_name = f'projects/{credentials.project_id}/' \
                   f'secrets/{secret_name}/versions/latest'

response = secretmanager_client.access_secret_version(
    request={'name': full_secret_name}
)

secret_value = response.payload.data.decode('utf-8')  # 此值恆為 str
```

© 2026 Enos Chou

The End

© 2026 Enos Chou